Spring Handbook — Vol 2

security, testing, reactive, messaging, observability, Spring Cloud, Spring AI and what's new, interview questions, cheat sheet, question bank (Ch 10–20)

11 parts · one place to study: DesignGurus courses + classic books + the web, combined, beginner to intermediate

what it is · problem · how it works · trade-offs · real world · interview · self-check · TL;DR

Personal study notes. Course diagrams belong to DesignGurus. Not for redistribution.

Contents

1. **Chapter 10 — Spring Security 1: Fundamentals**
   - 10.1 Vocabulary and what the starter does by default
   - 10.2 Architecture: the filter chain
   - 10.3 Configuring a chain: the lambda DSL in Spring Security 7
   - 10.4 Authentication: users, providers, form login and Basic
   - 10.5 Password storage: hashing with `DelegatingPasswordEncoder`
   - 10.6 Authorizing requests: roles, authorities, 401 vs 403
   - 10.7 Method security
   - 10.8 CSRF protection
   - 10.9 CORS
   - 10.10 Sessions, the security context and logout
   - 10.11 Security headers
   - 10.12 Testing security with `spring-security-test`
   - 10.13 Common mistakes (whole chapter)
   - 10.14 In the interview
   - Chapter 10 TL;DR
2. **Chapter 11 — Spring Security 2: Tokens and OAuth2**
   - 11.1 From sessions to tokens: bearer tokens and JWT
   - 11.2 OAuth 2.0 and OpenID Connect in one picture
   - 11.3 A JWT resource server in Spring
   - 11.4 Minting tokens with `NimbusJwtEncoder`
   - 11.5 Mapping claims to authorities
   - 11.6 Testing token security
   - 11.7 Opaque tokens and introspection
   - 11.8 Multi-tenancy: several issuers in one API
   - 11.9 OAuth2 login: "Sign in with ..."
   - 11.10 Calling other services: client credentials and token relay
   - 11.11 Refresh tokens, lifetimes, revocation and logout
   - 11.12 Where should a browser keep tokens? The BFF pattern
   - 11.13 Storing keys and secrets
   - 11.14 Spring Authorization Server
   - 11.15 Rate limiting and brute-force protection
   - 11.16 Common mistakes (whole chapter)
   - 11.17 In the interview
   - 11.18 Security fundamentals beyond Spring
   - Chapter 11 TL;DR
3. **Chapter 12 — Testing Spring Applications**
   - 12.1 The application under test
   - 12.2 The test pyramid in a Spring app
   - 12.3 Test dependencies in Boot 4
   - 12.4 Unit tests: no Spring at all
   - 12.5 How the Spring TestContext Framework works
   - 12.6 `@SpringBootTest`: the whole application
   - 12.7 Test slices
   - 12.8 `MockMvcTester`: AssertJ-style MockMvc
   - 12.9 `RestTestClient` (Framework 7.0)
   - 12.10 `@MockitoBean` and `@MockitoSpyBean`
   - 12.11 Test context caching
   - 12.12 `@DynamicPropertySource`: properties known only at runtime
   - 12.13 Full-stack tests on a real port
   - 12.14 Testcontainers and `@ServiceConnection`
   - 12.15 Profiles and properties in tests
   - 12.16 Common mistakes — checklist
   - 12.17 In the interview
   - 12.18 Types of API testing — and the Spring tool for each
   - 12.19 Chapter TL;DR
4. **Chapter 13 — Reactive Programming and Spring WebFlux**
   - 13.1 Why reactive? The cost of waiting
   - 13.2 Reactive Streams: the contract
   - 13.3 Project Reactor: `Mono` and `Flux`
   - 13.4 Operators
   - 13.5 Testing with `StepVerifier` and virtual time
   - 13.6 Error handling
   - 13.7 Backpressure in practice
   - 13.8 Schedulers and threads
   - 13.9 Hot vs cold, sinks and context
   - 13.10 Spring WebFlux: annotated controllers
   - 13.11 Functional endpoints
   - 13.12 Full-stack WebFlux tests and Server-Sent Events
   - 13.13 `WebClient`: calling other services without blocking
   - 13.14 R2DBC: reactive SQL
   - 13.15 Virtual threads vs WebFlux: a measured comparison
   - 13.16 Common mistakes — checklist
   - 13.17 In the interview
   - 13.18 Chapter TL;DR
5. **Chapter 14 — Messaging, Async and Scheduling**
   - 14.1 The map: four ways to do work "somewhere else"
   - 14.2 Application events: synchronous vs `@Async` listeners
   - 14.3 `@TransactionalEventListener`: events tied to a commit
   - 14.4 `@Async` and executors
   - 14.5 `@Async` on virtual threads
   - 14.6 Exceptions in async methods
   - 14.7 Scheduling with `@Scheduled`
   - 14.8 Messaging concepts you need first
   - 14.9 Kafka in five minutes
   - 14.10 Setting up Spring Kafka (with JSON)
   - 14.11 Producing: `KafkaTemplate`, keys and partitions
   - 14.12 Consuming: `@KafkaListener`, concurrency and consumer groups
   - 14.13 Testing with `@EmbeddedKafka`
   - 14.14 Error handling: `DefaultErrorHandler` and the dead-letter topic
   - 14.15 Non-blocking retries with `@RetryableTopic`
   - 14.16 Producer guarantees: idempotence and transactions
   - 14.17 The transactional outbox
   - 14.18 RabbitMQ with Spring AMQP (configuration, not run)
   - 14.19 Common mistakes (whole chapter)
   - 14.20 In the interview
   - Chapter TL;DR
6. **Chapter 15 — Observability and Production**
   - 15.1 Observability in one page
   - 15.2 Spring Boot Actuator
   - 15.3 Health, health groups and Kubernetes probes
   - 15.4 The info endpoint
   - 15.5 Metrics with Micrometer
   - 15.6 The Observation API and `@Observed`
   - 15.7 Distributed tracing
   - 15.8 Structured logging
   - 15.9 Graceful shutdown
   - 15.10 Virtual threads
   - 15.11 Faster startup: extracted jars, CDS, AOT cache, native images
   - 15.12 Production configuration checklist
   - 15.13 Common mistakes (whole chapter)
   - 15.14 In the interview
   - Chapter TL;DR
7. **Chapter 16 — Spring Cloud and Microservices**
   - 16.1 The map: microservice problems and Spring answers
   - 16.2 Release trains and versions
   - 16.3 Centralized configuration: Spring Cloud Config

- 16.4 Service discovery: Eureka, Consul, Kubernetes
- 16.5 Client-side load balancing with Spring Cloud LoadBalancer
- 16.6 Declarative clients: OpenFeign vs HTTP interface clients
- 16.7 API gateway: Spring Cloud Gateway
- 16.8 Resilience: circuit breaker, retry, rate limiter
- 16.9 Distributed tracing across services
- 16.10 Contract testing
- 16.11 Messaging between services
- 16.12 Common mistakes (whole chapter)
- 16.13 In the interview
- Chapter TL;DR

8. **Chapter 17 — Spring AI and What's New in Boot 4 / Framework 7**
   - 17.1 What Spring AI is
   - 17.2 Setting up a project
   - 17.3 `ChatModel` and `ChatClient`
   - 17.4 Prompts and prompt templates
   - 17.5 Advisors
   - 17.6 Chat memory
   - 17.7 Structured output
   - 17.8 Tool calling
   - 17.9 RAG: retrieval-augmented generation
   - 17.10 MCP: the Model Context Protocol
   - 17.11 Production concerns
   - 17.12 What's new: how to read the rest of this chapter
   - 17.13 Spring Boot 3.x → 4.x migration table
   - 17.14 Spring Framework 6 → 7
   - 17.15 Spring Security 6 → 7
   - 17.16 What Boot 4.1 adds (June 2026)
   - 17.17 In the interview — quick round
   - Chapter TL;DR
9. **Chapter 18 — Spring Interview Questions**
   - 18.1 How to use this chapter
   - 18.2 Core container and dependency injection
   - 18.3 Configuration and auto-configuration
   - 18.4 AOP and proxies
   - 18.5 Spring MVC, REST and HTTP clients
   - 18.6 Data access, JPA and transactions
   - 18.7 Caching
   - 18.8 Security
   - 18.9 Testing
   - 18.10 Reactive and WebFlux
   - 18.11 Messaging, async and scheduling
   - 18.12 Production and observability
   - 18.13 Microservices and Spring Cloud
   - 18.14 Spring AI and the Boot 4 / Framework 7 changes
   - 18.15 Proofs: five tricky answers, run
   - 18.16 "Explain your Spring project" — a walkthrough template
   - 18.17 Debugging scenarios
   - Chapter TL;DR
10. **Chapter 19 — Cheat Sheet and Glossary**
    - 19.1 How to use this chapter
    - 19.2 Versions at a glance
    - 19.3 Annotations cheat sheet
    - 19.4 Starters in Boot 4.1
    - 19.5 Key properties cheat sheet
    - 19.6 Transactions: `@Transactional` at a glance
    - 19.7 Bean scopes and the lifecycle
    - 19.8 Spring Security 7 DSL snippets
    - 19.9 Testing annotations and slices
    - 19.10 Reactor operators quick table
    - 19.11 Actuator endpoints
    - 19.12 Which tool for which job
    - 19.13 Boot 3 → 4 quick migration checklist
    - 19.14 Interview index
    - 19.15 Proofs: the facts on these sheets, run
    - 19.16 Glossary A–Z
    - Chapter TL;DR
11. **Chapter 20 — Interview Question Bank**
    - How to use this bank
    - Spring and Spring Boot fundamentals
    - IoC container, dependency injection, beans and scopes
    - Configuration, properties, profiles, auto-configuration and starters
    - AOP and proxies
    - Spring MVC, REST and HTTP clients
    - Validation and exception handling
    - Spring Data JPA and Hibernate
    - Transactions
    - Caching
    - Spring Security: filter chain, authentication, JWT and OAuth2
    - Testing
    - WebFlux, async, scheduling and messaging
    - Actuator, observability, production and performance
    - Microservices and Spring Cloud
    - Scenario, troubleshooting and coding questions
    - Sources of the question lists

Chapter 10 — Spring Security 1: Fundamentals

Every request raises two questions: **who is calling?** and **may they do this?** Spring Security answers both with a chain of servlet filters (code the web server runs on every request before Spring MVC) that runs *before* your controllers, and switches on protections you would otherwise forget — CSRF tokens (§10.8), security headers (§10.11), session-fixation defence (§10.10) — the moment you add the starter. It is not magic: once you can picture the filter chain, name the object holding the current user, and tell a 401 ("I don't know who you are") from a 403 ("I know you, but no"), the rest is configuration.

All code lives in `code/` — three Spring Boot **4.1.1** projects (Spring Security **7.1.1**, Framework 7, Java 25). Output blocks are real `mvn -q -B -ntp verify` output (Spring log lines trimmed to `...`; hashes and timings change per run).

Project	Shows
`security-defaults`	the starter with zero configuration
`security-basics`	filter chain, authentication, passwords, authorization, method security
`security-web-defenses`	two chains (API + browser app), CSRF on/off, CORS preflight, headers, sessions

> **Prerequisites.** Proxies (Chapter 4) explain method security; controllers come from Spring MVC (Chapter 5). The *browser* side of CORS, CSRF and XSS (cross-site scripting: an attacker's JavaScript running on your pages) is in the JavaScript handbook, Chapter 14. Tokens, JWT (JSON Web Token: a signed token carrying user details) and OAuth2 (the standard protocol for issuing access tokens) continue in Chapter 11.

10.1 Vocabulary and what the starter does by default

What it is

Spring Security is the Spring project for authentication, authorization and protection against common web attacks; Boot 4 adds it with `spring-boot-starter-security`.

Term	Plain meaning
Principal	the "who" — a user, a service, a device
Credentials	the proof — password, token, certificate
Authentication	checking the proof and deciding who the caller is
Authorization	deciding whether that caller may do *this* action
Authority (`GrantedAuthority`)	one permission string the caller holds, e.g. `report:read`
Role	an authority whose name starts with `ROLE_`, e.g. `ROLE_ADMIN`; a coarse group of permissions
Security context	the per-request holder of the current `Authentication`

Authentication produces an `Authentication` object (principal + authorities + "is authenticated" flag). **Authorization reads it.**

What happens with zero configuration

Add the starter, write one controller, and run the test in `security-defaults`:

```
@Test
void everythingIsLockedByDefault(CapturedOutput startupLog) throws Exception {
    // 1. Boot generated a user called "user" and logged its random password at startup
    String line = startupLog.getAll().lines()
            .filter(l -> l.contains("Using generated security password")).findFirst().orElseThrow();
    String password = line.substring(line.indexOf(':') + 1).trim();
    // 2. browser-style request -> redirect to the generated login page
    MvcTestResult browser = mvc.get().uri("/hello").header(HttpHeaders.ACCEPT, "text/html").exchange();
    assertThat(browser).hasStatus(302).hasRedirectedUrl("/login");
    // 3. API-style request -> 401 plus a Basic challenge
    MvcTestResult api = mvc.get().uri("/hello").header(HttpHeaders.ACCEPT, "application/json").exchange();
    assertThat(api).hasStatus(401);
    // 4. with the generated credentials it works
    MvcTestResult ok = mvc.get().uri("/hello").with(httpBasic("user", password)).exchange();
    assertThat(ok).hasStatusOk().hasBodyTextEqualTo("hello");
}
```

Output:

```
Using generated security password: 4030de5f-fb5d-4c46-bd81-7a0af09efd28

This generated password is for development use only. Your security configuration must be updated before running your application in
production.
...
startup log   : Using generated security password: <36-char UUID>
browser, anon : 302 -> /login
api, anon     : 401 WWW-Authenticate=Basic realm="Realm", charset="UTF-8"
basic, user   : 200 body=hello
```

With no code at all, Boot and Spring Security:

1. Protect **every** URL.
2. Create one in-memory user `user` with a random UUID (128-bit identifier) password logged at startup (nothing stored). `spring.security.user.name/password/roles` fix it — for experiments only.
3. Enable **form login** (a generated `/login` page) and **HTTP Basic** (username and password sent in a header on every request).
4. Pick the response by client type: a browser asking for HTML (`Accept: text/html`, e.g. Chrome) gets a `302` redirect to `/login`; a JSON client (e.g. Postman, curl) gets `401` with `WWW-Authenticate: Basic`. The `charset="UTF-8"` parameter is new in Spring Security 7.1.
5. Turn on CSRF protection, security headers, session-fixation protection and a logout endpoint.

Declaring your own `SecurityFilterChain` bean makes Boot's default chain back off; declaring a `UserDetailsService`, `AuthenticationManager` or `AuthenticationProvider` (or an OAuth2 setup — Chapter 11) removes the generated user.

Secure by default means forgetting something fails *closed*: a new endpoint is protected, not exposed. The price is the "why does my new app return 401?" moment — because you have not told Spring who may call it yet.

TL;DR

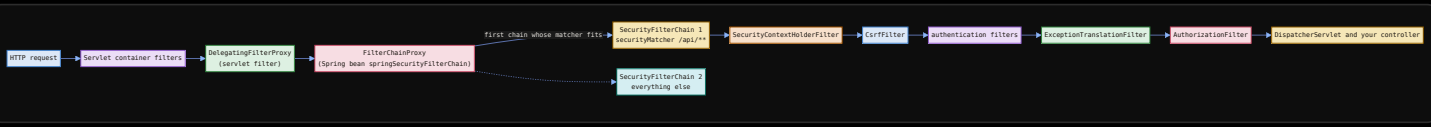
- Authentication = *who are you*, authorization = *may you*; the `Authentication` object links them.
- The starter alone locks every URL, generates a dev-only user and enables form login, Basic, CSRF and headers.

10.2 Architecture: the filter chain

What it is

Spring Security is a **servlet filter** — code that sees every HTTP request before the `DispatcherServlet` does — made of an ordered **list of small filters**, each with one job. If any filter says no, the request never reaches your controller.

How it works



1. **`DelegatingFilterProxy`** is a plain servlet filter Boot registers with the container. The container creates filters itself and cannot see Spring beans, so this filter only looks up a Spring bean and delegates to it.
2. That bean is **`FilterChainProxy`** (`springSecurityFilterChain`). It holds one or more **`SecurityFilterChain`s** and, per request, runs only the **first** chain whose request matcher fits (chains are never combined; order by `@Order`).
3. Each chain is an ordered list of **security filters**. The important ones:

Filter	Job
`SecurityContextHolderFilter`	loads the `SecurityContext` (e.g. from the HTTP session) into `SecurityContextHolder` for this request, and clears it afterwards
`HeaderWriterFilter`	adds security headers (§10.11)
`CorsFilter`	answers CORS preflights (§10.9), only if `cors()` is configured
`CsrfFilter`	rejects state-changing requests without a valid CSRF token (§10.8)
`LogoutFilter`	handles `POST /logout`
`UsernamePasswordAuthenticationFilter`	processes the login form `POST /login`
`BasicAuthenticationFilter`	reads `Authorization: Basic ...`
`BearerTokenAuthenticationFilter`	reads `Authorization: Bearer ...` (Chapter 11)
`AnonymousAuthenticationFilter`	if nobody authenticated, puts an *anonymous* `Authentication` in the context
`ExceptionTranslationFilter`	turns security exceptions from later filters into HTTP responses: 401/redirect or 403
`AuthorizationFilter`	the last gate: asks the `AuthorizationManager` whether this request is allowed

4. **`SecurityContextHolder`** stores the current `SecurityContext` in a `ThreadLocal` (a separate value per thread; each request runs on its own thread) by default. Any code on the request thread can read it with `SecurityContextHolder.getContext().getAuthentication()`.

The test in `security-basics` prints the real filter list of our chain:

```
@Autowired FilterChainProxy springSecurityFilterChain;

@Test
void printTheFilters() {
    List<SecurityFilterChain> chains = springSecurityFilterChain.getFilterChains();
    List<String> names = chains.getFirst().getFilters().stream().map(f -> f.getClass().getSimpleName()).toList();
    // ... print with numbers
    assertThat(names.getLast()).isEqualTo("AuthorizationFilter");
}
```

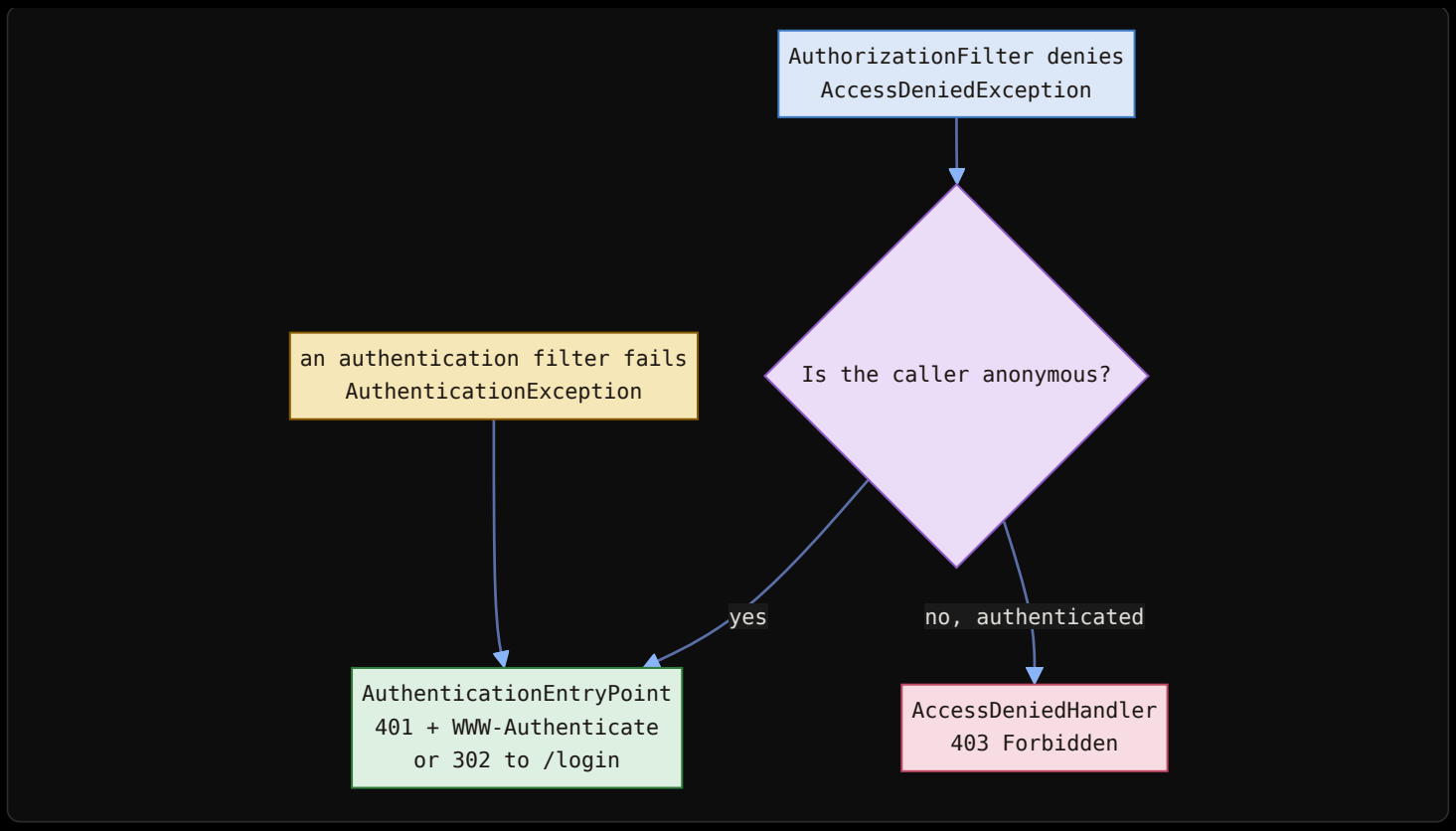
Output:

```
security filter chains: 1
 1 DisableEncodeUrlFilter
 2 WebAsyncManagerIntegrationFilter
 3 SecurityContextHolderFilter
 4 HeaderWriterFilter
 5 CsrfFilter
 6 LogoutFilter
 7 UsernamePasswordAuthenticationFilter
 8 DefaultResourcesFilter
 9 DefaultLoginPageGeneratingFilter
10 DefaultLogoutPageGeneratingFilter
11 BasicAuthenticationFilter
12 RequestCacheAwareFilter
13 SecurityContextHolderAwareRequestFilter
14 AnonymousAuthenticationFilter
15 ExceptionTranslationFilter
16 AuthorizationFilter
```

The list changes with configuration: removing `formLogin()` removes `UsernamePasswordAuthenticationFilter` and the login page; `oauth2ResourceServer()` adds `BearerTokenAuthenticationFilter`. Printing it, or `logging.level.org.springframework.security=TRACE`, is the fastest way to see what a chain really does.

How a denied request becomes 401 or 403

`ExceptionTranslationFilter` sits just before `AuthorizationFilter` and catches what it throws:



- An `AuthenticationEntryPoint` "starts authentication". It is `401` with a challenge header for APIs, or a redirect to a login page for browsers. Before redirecting, the original request is saved in the **request cache**, so after login the user lands where they wanted to go.
- An `AccessDeniedHandler` answers `403` for callers who *are* known but lack permission.

Why it matters

- **Debugging.** "403 before my controller runs" is a filter problem, not an MVC problem; the filter list tells you which one.
- **Ordering.** Place custom filters with `http.addFilterBefore(myFilter, BasicAuthenticationFilter.class)` and friends, never a guessed order number.
- **Threads.** The `ThreadLocal` context does *not* follow work handed to another thread (`@Async`, executors, `CompletableFuture`, scheduled jobs); use `DelegatingSecurityContextExecutor` (or the `DelegatingSecurityContext*` wrappers).

Common mistakes

- Registering your security filter as a plain `@Component` `Filter`: Boot *also* adds it to the servlet container, so it runs twice, once outside the chain. Add it via `http.addFilter...` and disable the servlet registration with a `FilterRegistrationBean` whose `setEnabled(false)`.

TL;DR

Container → `DelegatingFilterProxy` → `FilterChainProxy` → first matching `SecurityFilterChain` → ordered filters → `AuthorizationFilter` last; `ExceptionTranslationFilter` turns denials into 401/redirect (anonymous) or 403 (known). The current user lives in `SecurityContextHolder` (a `ThreadLocal`).

10.3 Configuring a chain: the lambda DSL in Spring Security 7

What it is

Declare a `SecurityFilterChain` bean and build it with the `HttpSecurity` **DSL** (domain-specific language: a fluent builder where each method configures one feature via a `Customizer` lambda). The whole configuration of `security-basics`:

```java
@Configuration
@EnableMethodSecurity // turns on @PreAuthorize / @PostAuthorize / @PreFilter / @PostFilter
class SecurityConfig {

    @Bean
    SecurityFilterChain appSecurity(HttpSecurity http) throws Exception {
        http
            .authorizeHttpRequests(auth -> auth                     // first match wins, top to bottom
                .requestMatchers("/public/**").permitAll()
                .requestMatchers("/admin/**").hasRole("ADMIN")        // checks authority "ROLE_ADMIN"
                .requestMatchers("/user/**").hasRole("USER")          // ADMIN also passes (hierarchy below)
                .requestMatchers(HttpMethod.GET, "/reports/**").hasAuthority("report:read")
                .requestMatchers("/me", "/orders/**").authenticated()
                .anyRequest().denyAll())                              // anything we forgot is closed
            .httpBasic(Customizer.withDefaults())                     // Authorization: Basic ... for APIs
            .formLogin(Customizer.withDefaults());                    // /login page for browsers
        return http.build();
    }

    @Bean
    PasswordEncoder passwordEncoder() {
        return PasswordEncoderFactories.createDelegatingPasswordEncoder();
    }

    @Bean
    static RoleHierarchy roleHierarchy() {
        return RoleHierarchyImpl.fromHierarchy("ROLE_ADMIN > ROLE_USER");
    }
}
```

- `Customizer.withDefaults()` is a no-op lambda: "turn this feature on with default settings". `feature(f -> f.disable())` turns one off.
- With Boot, `@EnableWebSecurity` is applied for you (harmless to add).

What changed in Spring Security 7

Spring Security 7 finished clean-ups that 6.x had deprecated; old tutorials use the removed style:

Removed in 7	Use instead
`.and()` chaining (`http.csrf().disable().and().httpBasic()`)	one lambda per feature (above)
`authorizeRequests(...)`	`authorizeHttpRequests(...)` (built on `AuthorizationManager`)
`AntPathRequestMatcher`, `MvcRequestMatcher`	`PathPatternRequestMatcher` — the same `PathPattern` syntax Spring MVC uses. `requestMatchers("/x/**")` creates them for you
`WebSecurityConfigurerAdapter` (already gone in 6.0)	`SecurityFilterChain` beans
`AccessDecisionManager` / voters in the main jar	moved to the legacy `spring-security-access` module; prefer `AuthorizationManager`
OAuth2 *password* grant support	authorization code or client credentials (Chapter 11)

Other 7.0 additions: built-in **multi-factor authentication** (MFA: two or more kinds of proof; §10.4), `csrf(c -> c.spa())` for single-page apps (SPAs; §10.8), Password4j-based encoders (§10.5), and Spring Authorization Server (issues OAuth2 tokens) now shipping *inside* Spring Security (Chapter 11).

Several chains in one application

Apps serving a JSON API *and* browser pages need different rules: the API stateless (the server keeps no login state; every request carries credentials) without CSRF, the pages on a session cookie with CSRF on. Declare two chains, each with a `securityMatcher` and an `@Order`:

```java
@Bean @Order(1)
SecurityFilterChain apiChain(HttpSecurity http) throws Exception {
    http.securityMatcher("/api/**")                       // this chain only handles /api/**
        .authorizeHttpRequests(auth -> auth.anyRequest().authenticated())
        .httpBasic(Customizer.withDefaults())
        .cors(Customizer.withDefaults())
        .csrf(csrf -> csrf.disable())                     // safe ONLY because nothing relies on cookies
        .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS));
    return http.build();
}

@Bean @Order(2)
SecurityFilterChain webChain(HttpSecurity http) throws Exception {   // no securityMatcher = everything else
    http.authorizeHttpRequests(auth -> auth.requestMatchers("/login").permitAll().anyRequest().authenticated())
        .formLogin(Customizer.withDefaults())
        .logout(Customizer.withDefaults());
    return http.build();
}
```

`securityMatcher` selects which *requests* the whole chain handles; `requestMatchers` inside `authorizeHttpRequests` defines *rules* within the chain. Mixing them up is a classic bug.

Common mistakes

- A chain **without** `securityMatcher` placed *before* a specific one matches everything, so the later chain never runs. Catch-all chains go last (highest `@Order`).
- Copying `.and()` or `authorizeRequests()` code from old posts — it no longer compiles in Spring Security 7.
- Declaring the `SecurityFilterChain` bean outside a scanned `@Configuration`: Boot's default chain stays active and nothing you wrote applies.

TL;DR

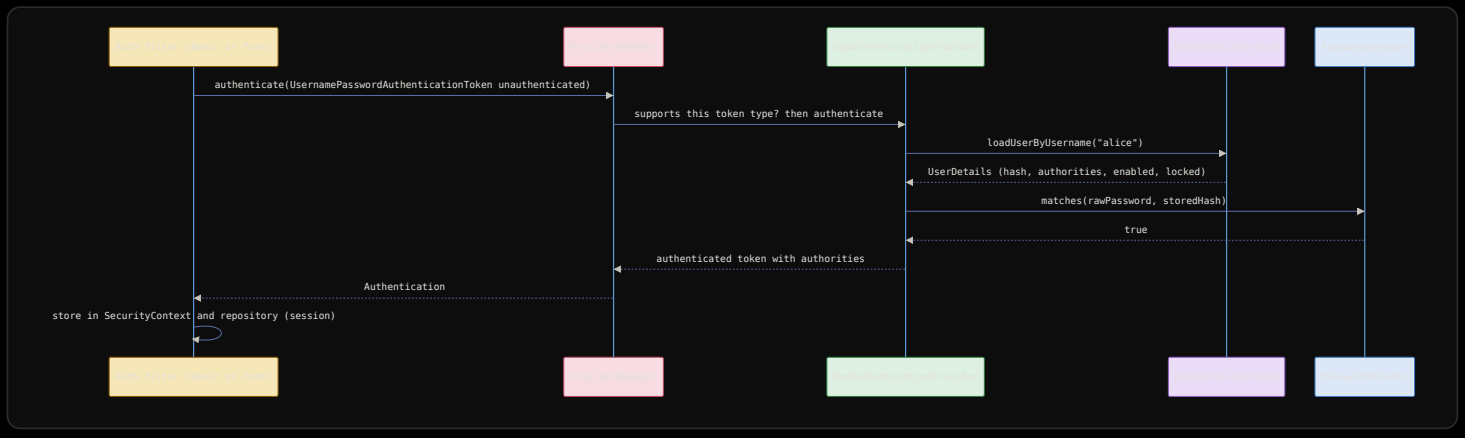
One `SecurityFilterChain` bean per chain, one lambda per feature. Spring Security 7 removed `and()`, `authorizeRequests` and the Ant/Mvc matchers. Several chains = `securityMatcher` + `@Order`, specific first.

10.4 Authentication: users, providers, form login and Basic

What it is

Authentication turns *credentials* into an `Authentication` saying who the caller is and what authorities they hold. Spring splits the work into small, replaceable interfaces.

How it works (username + password)



1. An **authentication filter** extracts credentials (`BasicAuthenticationFilter`: the `Authorization` header; `UsernamePasswordAuthenticationFilter`: the `POST /login` form) and builds an *unauthenticated* token.
2. It calls the **`AuthenticationManager`**; the standard **`ProviderManager`** asks each **`AuthenticationProvider`** whether it `supports` that token type.
3. **`DaoAuthenticationProvider`** loads the user through your **`UserDetailsService`**, checks account flags (enabled, not locked, not expired), and verifies the password with the **`PasswordEncoder`**.
4. On success the filter stores the token in the `SecurityContext`; form login also saves it in the HTTP session (recognised next time via the session cookie), while Basic resends credentials on *every* request.
5. On failure an `AuthenticationException` is thrown. Wrong password and unknown user both surface as `BadCredentialsException` (no username enumeration — discovering which accounts exist), and for an unknown user the provider still checks a dummy hash so timing does not leak it either.

`UserDetailsService`: your user store behind one method

`UserDetailsService` has one method, `loadUserByUsername(String)`, backed by a table, LDAP (a corporate directory protocol) or anything else. `security-basics` uses a map (a real app would use JPA — Chapter 8):

```java
@Service
public class AccountUserDetailsService implements UserDetailsService, UserDetailsPasswordService {

    public record Account(String username, String passwordHash, List<String> authorities, boolean enabled) {}
    private final Map<String, Account> accounts = new ConcurrentHashMap<>();

    AccountUserDetailsService(PasswordEncoder encoder) {
        save(new Account("alice", encoder.encode("alice-pass"), List.of("ROLE_USER", "report:read"), true));
        save(new Account("bob", encoder.encode("bob-pass"), List.of("ROLE_ADMIN"), true));
        save(new Account("carol", encoder.encode("carol-pass"), List.of("ROLE_USER"), true));
        save(new Account("dave", "{noop}dave-pass", List.of("ROLE_USER"), true));    // legacy plain-text row
        save(new Account("erin", encoder.encode("erin-pass"), List.of("ROLE_USER"), false)); // disabled
    }

    @Override
    public UserDetails loadUserByUsername(String username) {
        Account a = accounts.get(username);
        if (a == null) throw new UsernameNotFoundException(username);
        return User.withUsername(a.username()).password(a.passwordHash())
                .authorities(a.authorities().toArray(String[]::new))
                .disabled(!a.enabled()).build();
    }

    @Override   // see 10.5: called when a stored hash should be upgraded
    public UserDetails updatePassword(UserDetails user, String newEncodedPassword) { /* save new hash */ }
}
```

With this bean plus a `PasswordEncoder` bean, Spring builds the `DaoAuthenticationProvider` for you. Ready-made implementations: `InMemoryUserDetailsManager` (tests, demos), `JdbcUserDetailsManager` (Spring's `users`/`authorities` schema), LDAP.

Form login vs HTTP Basic

	Form login	HTTP Basic
Credentials sent	once, in `POST /login`	on **every** request, Base64 (a reversible text encoding) in `Authorization` (encoded, **not encrypted**, so HTTPS — HTTP over an encrypted TLS connection — only)
Remembered by	HTTP session cookie (`JSESSIONID`)	nothing, the client resends them
Unauthenticated response	`302` redirect to `/login`	`401` + `WWW-Authenticate: Basic realm=...`
Logout	`POST /logout` invalidates the session	impossible server-side (browser caches credentials)
Good for	server-rendered browser apps	scripts, service-to-service in trusted networks, quick demos

With both enabled, Spring picks the entry point per request: HTML-accepting browsers get the redirect, others (e.g. `Accept: application/json`) the `401` challenge:

```java
MvcTestResult noCreds  = mvc.get().uri("/me").header("Accept", "application/json").exchange();
MvcTestResult wrong    = mvc.get().uri("/me").with(httpBasic("alice", "nope")).exchange();
MvcTestResult disabled = mvc.get().uri("/me").with(httpBasic("erin", "erin-pass")).exchange();
MvcTestResult ok       = mvc.get().uri("/me").with(httpBasic("alice", "alice-pass")).exchange();
// formLogin() builds POST /login with username, password AND a valid CSRF token
mockMvc.perform(formLogin().user("alice").password("alice-pass")).andExpect(authenticated().withUsername("alice"));
mockMvc.perform(formLogin().user("alice").password("wrong")).andExpect(unauthenticated());
```

Output:

```
no credentials (JSON)        401 WWW-Authenticate=Basic realm="Realm", charset="UTF-8"
alice, wrong password        401 WWW-Authenticate=Basic realm="Realm", charset="UTF-8"
erin, disabled account       401 WWW-Authenticate=Basic realm="Realm", charset="UTF-8"
alice, right password        200 {"name":"alice","principalType":"User","authorities":["FACTOR_PASSWORD","ROLE_USER","report:read"]}
no credentials (HTML)        302 -> /login
form login, good password   302 -> /
form login, bad password    302 -> /login?error
```

Reading the current user

```java
@GetMapping("/me")
Me me(Authentication auth, @AuthenticationPrincipal UserDetails principal) {
    return new Me(auth.getName(), principal.getClass().getSimpleName(),
            auth.getAuthorities().stream().map(GrantedAuthority::getAuthority).sorted().toList());
}
```

Spring MVC fills an `Authentication` (or `Principal`) parameter; `@AuthenticationPrincipal` gives the principal object itself (here our `UserDetails`). Elsewhere use `SecurityContextHolder.getContext().getAuthentication()`, but prefer passing the user in — `ThreadLocal`

access is harder to test.

The `FACTOR_PASSWORD` authority: multi-factor authentication in Spring Security 7

Alice's authorities include `FACTOR_PASSWORD`. Spring Security 7 adds a **factor authority** (`FactorGrantedAuthority`) for every mechanism a user completes: `FACTOR_PASSWORD`, `FACTOR_OTT` (one-time token, e.g. an e-mailed code), `FACTOR_WEBAUTHN` (passkeys: password-less login with a key on the user's device). Rules can then demand several factors ("`/admin/**` needs `ROLE_ADMIN` **and** password **and** one-time token") via `@EnableMultiFactorAuthentication(authorities = ...)` or selectively `AuthorizationManagerFactories.multiFactor()`; a missing factor sends the user to that factor's login step. Don't be surprised by `FACTOR_*` entries, and filter them out when copying authorities into a token (Chapter 11 does).

Common mistakes

- Returning `null` from `loadUserByUsername` — the contract is to throw `UsernameNotFoundException`.
- Telling users "unknown username" vs "wrong password" — enables account enumeration.
- Logging or returning the `UserDetails` (with its hash). Map it to a DTO.
- HTTP Basic over plain HTTP — Base64 is readable by anyone on the path.

In the interview

> **30-second answer:** "An authentication filter extracts credentials and hands an unauthenticated token to the `AuthenticationManager`. `ProviderManager` delegates to a provider that supports the token — for passwords, `DaoAuthenticationProvider`, which loads the user via `UserDetailsService` and verifies the hash with the `PasswordEncoder`. The result goes into the `SecurityContext`, and form login persists it in the session."

- *Why is `UsernameNotFoundException` turned into `BadCredentialsException`?* → To prevent username enumeration; a dummy password check also equalises timing.
- *How would you add login by API key?* → A filter (or `AuthenticationConverter`) building an `ApiKeyAuthenticationToken`, an `AuthenticationProvider` validating it, registered with `addFilterBefore(..., BasicAuthenticationFilter.class)`.
- *Why can't you "log out" of HTTP Basic?* → No server-side session; the browser caches and resends the credentials.

TL;DR

Filter → `AuthenticationManager` (`ProviderManager`) → `AuthenticationProvider` (`DaoAuthenticationProvider`) → `UserDetailsService` + `PasswordEncoder` → `Authentication` in the `SecurityContext`. Browsers: form login + session. Scripts: Basic (HTTPS only). Tokens: Chapter 11.

10.5 Password storage: hashing with `DelegatingPasswordEncoder`

What it is

Never store passwords, not even encrypted. Store a **password hash**: the output of a one-way function, cheap to check once but expensive to compute billions of times, so a leaked database must be cracked one guess at a time. Three properties make it suitable:

Idea	Meaning	Why
One-way	you can't compute the password from the hash	a leak does not reveal passwords directly
Salt	a random value stored with each hash and mixed in	equal passwords get different hashes, and precomputed "rainbow tables" are useless
Work factor (cost)	a tunable amount of CPU and/or memory per check	makes each guess slow. Raise it as hardware gets faster

Fast general-purpose hashes (SHA-256, MD5 — built for checksums) are exactly wrong for passwords. Use an **adaptive password hash**:

Algorithm	Spring encoder	Notes
bcrypt	`BCryptPasswordEncoder` (default strength 10)	CPU-hard, battle-tested. Uses only the first **72 bytes** of the password
Argon2id	`Argon2PasswordEncoder` (needs Bouncy Castle `bcprov-jdk18on`)	memory-hard (resists GPU/ASIC cracking hardware). Password Hashing Competition winner; first choice of OWASP (Open Worldwide Application Security Project)
scrypt	`SCryptPasswordEncoder` (Bouncy Castle)	memory-hard
PBKDF2	`Pbkdf2PasswordEncoder`	when FIPS-140 (US approved-cryptography standard) compliance is required

Spring Security 7.0 also added **Password4j-based** encoders (`Argon2Password4jPasswordEncoder`, `BcryptPassword4jPasswordEncoder`, `ScryptPassword4jPasswordEncoder`, `Pbkdf2Password4jPasswordEncoder`, `BalloonHashingPassword4jPasswordEncoder`).

How `DelegatingPasswordEncoder` works

Algorithms age (MD5 → bcrypt → Argon2). `DelegatingPasswordEncoder` lets one database hold several generations; each stored value carries its algorithm id as a prefix:

```
{bcrypt}$2a$10$oExJt1cCgjfvb4DzBrmNo.VLE45g8zG8GLlEik43AL6a3Xbk8q20a
{argon2@SpringSecurity_v5_8}$argon2id$v=19$m=16384,t=2,p=1$...
{noop}dave-pass                        <- legacy plain text (never do this for real)
```

1. **`encode(raw)`** always uses the *current default* id. `PasswordEncoderFactories.createDelegatingPasswordEncoder()` uses `bcrypt`.
2. **`matches(raw, stored)`** reads the `{id}` prefix and delegates to that algorithm's encoder. Old hashes keep working.
3. **`upgradeEncoding(stored)`** is `true` when the stored hash is not in the current default format. `DaoAuthenticationProvider` calls it after each successful login; if true *and* your `UserDetailsService` implements **`UserDetailsPasswordService`**, it re-hashes the just-received password and calls `updatePassword(...)`. Users migrate silently, one login at a time.

`PasswordEncodingTest` (a plain unit test, no Spring context) shows all of this:

```
PasswordEncoder enc = PasswordEncoderFactories.createDelegatingPasswordEncoder();
String h1 = enc.encode("s3cret");
String h2 = enc.encode("s3cret");
// ...
String argon = "{argon2@SpringSecurity_v5_8}" + Argon2PasswordEncoder.defaultsForSpringSecurity_v5_8().encode("s3cret");
enc.matches("s3cret", argon);          // true: same encoder reads other algorithms
enc.upgradeEncoding("{noop}s3cret");   // true
```

Output:

```
encode #1 : {bcrypt}$2a$10$oExJt1cCgjfvb4DzBrmNo.VLE45g8zG8GLlEik43AL6a3Xbk8q20a
encode #2 : {bcrypt}$2a$10$oYZZ7zXC3LyJEl6nbQ9KbuQxmg1.oblxK/JPZ0/EufIiGHGFWX2ai
same input, different hash (random salt): true
matches(s3cret) = true, matches(S3cret) = false
argon2 row: {argon2@SpringSecurity_v5_8}$argon2id$v=19$m=16384,t=2,p=1$0...
matches(argon2 row) = true
upgradeEncoding({noop})  = true
upgradeEncoding(argon2)  = true
upgradeEncoding(bcrypt)  = false
```

A bcrypt value: `$2a$` variant, `10` cost (2^10 rounds), 22 chars of salt, 31 of hash — the salt is inside the string, no extra column. `upgradeEncoding(argon2)` is `true` only because this encoder's *default* is bcrypt: "upgrade" means "not the current default".

Choosing Argon2 as the default

To hash new passwords with Argon2 while accepting everything stored, build your own `DelegatingPasswordEncoder`:

```
Map<String, PasswordEncoder> known = Map.of(
        "argon2@SpringSecurity_v5_8", Argon2PasswordEncoder.defaultsForSpringSecurity_v5_8(),
        "bcrypt", new BCryptPasswordEncoder(),
        "noop", NoOpPasswordEncoder.getInstance());          // only to READ legacy rows
var enc = new DelegatingPasswordEncoder("argon2@SpringSecurity_v5_8", known);
```

Output:

```
argon2 default: {argon2@SpringSecurity_v5_8}$argon2id$v=19$m=16384,t=2,p=1$Y...
old bcrypt row still matches: true, needs upgrade: true
```

`m=16384,t=2,p=1` = 16 MiB memory, 2 passes, 1 lane (Spring's `v5_8` defaults). OWASP's Password Storage Cheat Sheet currently suggests somewhat more memory for Argon2id (e.g. 19 MiB, 2 passes) — use the `Argon2PasswordEncoder(saltLength, hashLength, parallelism, memory, iterations)` constructor. Changing parameters under the same id makes `upgradeEncoding` flag weaker stored hashes too. Measure on production hardware.

The work factor in numbers

```
bcrypt strength 10 -> $2a$10$...  ~59 ms per check
bcrypt strength 12 -> $2a$12$...  ~239 ms per check
```

Each +1 doubles the work (~4x for +2). Aim for "as slow as your login latency budget allows", often several hundred ms — but the cost hits *your* servers on every login, so slow hashing without rate limiting makes login an easy denial-of-service target (Chapter 11 rate limiting).

bcrypt's 72-byte limit

```
73-byte password -> IllegalArgumentException: password cannot be more than 72 bytes
```

bcrypt only reads the first 72 bytes; current Spring Security refuses longer input instead of silently truncating. Allow long passphrases → validate length at sign-up or choose Argon2.

Upgrading on login — proved

`dave` is stored as `{noop}dave-pass`; after one successful login his row holds a bcrypt hash:

```
System.out.println("before login: " + accounts.find("dave").passwordHash());
assertThat(mvc.get().uri("/me").with(httpBasic("dave", "dave-pass"))).hasStatusOk();
String after = accounts.find("dave").passwordHash();
assertThat(after).startsWith("{bcrypt}$2a$10$");
```

Output:

```
before login: {noop}dave-pass
after login : {bcrypt}$2a$10$6G6hiKY...
```

Common mistakes

- **Encrypting** passwords (AES) "so support can see them" — anyone with the key decrypts them all.
- Hand-made `sha256(password + "pepper")` — fast, so it cracks quickly.
- `NoOpPasswordEncoder` as the bean "for now" (deprecated for a reason).
- Hashes stored without the `{id}` prefix, then switching encoders: every old row breaks. `DelegatingPasswordEncoder.setDefaultPasswordEncoderForMatches(...)` tells Spring how to read unprefixed values.
- Too-short hash column: bcrypt is 60 chars plus `{bcrypt}`, Argon2 longer. Use `VARCHAR(255)`+.

In the interview

> **30-second answer:** "Store only adaptive, salted hashes such as bcrypt or Argon2id, never encrypted or plain passwords. In Spring I use `DelegatingPasswordEncoder`: each hash carries an `{id}` prefix, new passwords use the current default, old formats still verify, and with `UserDetailsPasswordService` hashes upgrade transparently on the next successful login."

- *Why a salt?* → Equal passwords get different hashes; precomputed tables become useless. The salt is not secret.
- *bcrypt or Argon2?* → Both fine. Argon2id is memory-hard (GPU cracking costlier), OWASP's first choice; bcrypt is ubiquitous but has the 72-byte limit.
- *What is a pepper?* → An optional secret mixed into every hash and stored *outside* the database (in a vault). Helps only if the database leaks without the app secrets.

TL;DR

Hash, don't encrypt: Argon2id or bcrypt via `DelegatingPasswordEncoder` (`{id}hash`), `UserDetailsPasswordService` to upgrade on login, cost tuned to your hardware.

10.6 Authorizing requests: roles, authorities, 401 vs 403

What it is

`authorizeHttpRequests` is a list of **rules**, each pairing a request matcher with a requirement. For each request, `AuthorizationFilter` walks the list **top to bottom** and applies the **first rule that matches**. The rules you write most often:

Rule	Passes when
`permitAll()`	always (still goes through the other filters, e.g. CSRF)
`denyAll()`	never
`authenticated()`	caller is not anonymous
`hasAuthority("report:read")`	caller holds exactly that authority string
`hasAnyAuthority("a", "b")`	holds at least one
`hasRole("ADMIN")`	holds authority `ROLE_ADMIN` (the prefix is added for you)
`hasAnyRole("ADMIN", "OPS")`	holds `ROLE_ADMIN` or `ROLE_OPS`
`access(authorizationManager)`	your own `AuthorizationManager` decides, e.g. IP ranges or time of day

Matchers can take HTTP methods (`requestMatchers(HttpMethod.GET, "/reports/**")`) and use Spring MVC's `PathPattern` syntax (`*` one segment, `**` any number).

Roles vs authorities

Under the hood **there is only `GrantedAuthority`, a string**; a "role" is the convention that it starts with `ROLE_`.

	Role	Authority (permission)
Example	`ROLE_ADMIN`	`report:read`, `order:refund`
Granularity	coarse: *who the user is*	fine: *what the user may do*
Checked with	`hasRole("ADMIN")`	`hasAuthority("report:read")`
Typical source	user's group membership	role-to-permission mapping, OAuth2 scopes (permissions granted to a token, e.g. `read`; `SCOPE_...`, Chapter 11)

Many teams check **authorities** in code and assign them to users *through* roles, so a role change is not a code change. Spring rejects both prefix mistakes:

```
hasRole("ROLE_ADMIN") -> ROLE_ADMIN should not start with ROLE_ since ROLE_ is automatically prepended when using hasRole. Consider using hasAuthority instead.
User.roles("ROLE_ADMIN") -> ROLE_ADMIN cannot start with ROLE_ (it is automatically added)
```

`User.withUsername(...).roles("ADMIN")` stores `ROLE_ADMIN`; `.authorities("ROLE_ADMIN", "report:read")` stores the strings as given. The two setters **replace** each other (last call wins), so mixed lists use `authorities(...)`.

401 vs 403 — proved

Status	Name	Meaning	Client should
401	Unauthorized (really "unauthenticated")	"I don't know who you are"	authenticate, then retry
403	Forbidden	"I know who you are, and the answer is no"	not retry with the same identity

The matrix test calls five URLs as four callers:

```
assertThat(status("/admin/stats", null)).isEqualTo(401);      // who are you?
assertThat(status("/admin/stats", "carol")).isEqualTo(403);   // I know you, but no
assertThat(status("/reports/daily", "alice")).isEqualTo(200); // authority, not role
assertThat(status("/reports/daily", "bob")).isEqualTo(403);   // ADMIN role is not report:read
assertThat(status("/user/profile", "bob")).isEqualTo(200);    // role hierarchy ADMIN > USER
assertThat(status("/nowhere", "bob")).isEqualTo(403);         // denyAll catch-all
```

Output:

```
                          /public/hello  /user/profile  /admin/stats   /reports/daily /nowhere
anonymous                 200            401            401            401           401
alice (ROLE_USER, report:read) 200       200            403            200           403
carol (ROLE_USER)         200            200            403            403           403
bob (ROLE_ADMIN)          200            200            200            403           403
```

- **anonymous** gets 401 everywhere except the public URL, even `/nowhere`: a denied *anonymous* caller is always asked to authenticate first (§10.2).
- **alice** reaches `/reports/daily` via the `report:read` **authority**, though not an admin; **bob** (admin) gets **403** there — a role does not imply permissions never mapped to it.
- **bob** reaches `/user/profile` via the **role hierarchy** `ROLE_ADMIN > ROLE_USER` (applies to URL rules and method security).
- `/nowhere` has no controller yet is 403 for logged-in users: `anyRequest().denyAll()` closes everything unlisted, and security runs before MVC looks for a handler.

Why `denyAll()` last: with `anyRequest().authenticated()`, next month's new endpoint is open to *every logged-in user*; with `denyAll()` it stays closed until someone writes a rule. `authenticated()` is fine for internal apps; `denyAll()` is safer customer-facing.

Common mistakes

- **Rule order.** `requestMatchers("/**").authenticated()` first makes every later rule dead code. Most specific first.
- **Securing only the URL.** Can alice read `/orders/2`, which is carol's? That needs the *data* — method security or the query (IDOR, insecure direct object reference: reaching another user's record by changing an id; §10.7).
- Returning **404** for "exists but you may not see it" hides existence — decide deliberately; Spring gives 403.
- Relying on `permitAll()` to *skip* security — the request still passes CSRF and header filters. For static files use their own `permitAll()` chain; avoid `WebSecurityCustomizer.ignoring()`, which also drops security headers.

In the interview

> **30-second answer:** "401 means the server doesn't know who you are, so authenticate. 403 means it knows and refuses. In Spring, an `AccessDeniedException` for an anonymous user goes to the `AuthenticationEntryPoint` (401 or login redirect), for an authenticated user to the `AccessDeniedHandler` (403). Roles are authorities with a `ROLE_` prefix. I check fine-grained authorities and map them to roles."

- *`hasRole("ADMIN")` vs `hasAuthority("ADMIN")`?* → The first checks `ROLE_ADMIN`, the second the literal `ADMIN`.
- *Admins inheriting user permissions?* → A `RoleHierarchy` bean (`RoleHierarchyImpl.fromHierarchy("ROLE_ADMIN > ROLE_USER")`).

TL;DR

First matching rule wins: specific first, finish with `denyAll()` (or `authenticated()`). A role is a `ROLE_` authority; `hasRole` adds the prefix. 401 = unknown caller, 403 = known but not allowed.

10.7 Method security

What it is

URL rules protect *endpoints*. **Method security** (`@EnableMethodSecurity`) protects *business methods* whoever calls them — controller, listener, scheduled job — and can inspect **arguments and return value**, which URL rules cannot.

Annotation	When it runs	Typical use
`@PreAuthorize("expr")`	before the call	role/authority checks, argument checks (`#owner == authentication.name`)
`@PostAuthorize("expr")`	after the call, sees `returnObject`	"load it, then check it belongs to the caller"
`@PreFilter("expr")`	before, filters a collection argument	rarely needed
`@PostFilter("expr")`	after, removes elements of the returned collection (`filterObject`)	small lists only
`@Secured("ROLE_ADMIN")`	before (enable with `securedEnabled = true`)	legacy, no SpEL
`@RolesAllowed("ADMIN")`	before (enable with `jsr250Enabled = true`; JSR-250 is the Java standard that defined it)	Jakarta standard annotation

Expressions are **SpEL** (Spring Expression Language) with `authentication`, `principal`, `hasRole(...)`, `hasAuthority(...)`, parameters as `#name`, and `returnObject`.

```
@Service
public class OrderService {

    @PreAuthorize("#owner == authentication.name or hasRole('ADMIN')")
    public List<Order> ordersOf(String owner) { ... }

    @PostAuthorize("returnObject.owner() == authentication.name")
    public Order find(long id) { ... }

    @PostFilter("filterObject.owner() == authentication.name or hasRole('ADMIN')")
    public List<Order> all() { return new ArrayList<>(orders); }   // must be mutable

    @PreAuthorize("hasAuthority('order:refund')")
    public String refund(long id) { ... }
}
```

How it works

Method security is **proxy-based AOP** (Chapter 4): a proxy's interceptor evaluates the expression and throws `AccessDeniedException` if false. Same proxy rules as `@Transactional`: calls from inside the same class (`this.refund(1)`, **self-invocation** — proved in Chapter 4) skip the check, and `private` (and, with CGLIB, `final`) methods are never intercepted.

Proved

The tests call the service directly, with `@WithMockUser` supplying the security context:

```
@Test
@WithMockUser(username = "alice") // default role USER
void ownerCanReadOwnOrders() {
    orders.ordersOf("alice");                                   // allowed
    assertThatThrownBy(() -> orders.ordersOf("carol")).isInstanceOf(AccessDeniedException.class);
    assertThat(orders.all()).extracting(OrderService.Order::owner).containsOnly("alice");
    assertThatThrownBy(() -> orders.find(2)).isInstanceOf(AccessDeniedException.class);
    assertThatThrownBy(() -> orders.refund(1)).isInstanceOf(AccessDeniedException.class);
}
```

Output:

```
no SecurityContext, refund(1) -> AuthenticationCredentialsNotFoundException
alice ordersOf(alice)        -> [Order[id=1, owner=alice, cents=1999], Order[id=3, owner=alice, cents=250]]
alice ordersOf(carol)        -> AccessDeniedException
alice all() after @PostFilter -> [Order[id=1, owner=alice, cents=1999], Order[id=3, owner=alice, cents=250]]
alice find(2) (carol's)      -> AccessDeniedException (@PostAuthorize)
alice refund(1)              -> AccessDeniedException (needs order:refund)
bob(ADMIN) ordersOf(carol)   -> [Order[id=2, owner=carol, cents=500]]
support refund(1)            -> refunded 1
GET /orders/of/carol as alice -> HTTP 403
```

- A secured method called with **no** authentication (e.g. a background thread) fails with `AuthenticationCredentialsNotFoundException`.
- Over HTTP the service's `AccessDeniedException` travels back through the filters and becomes **403** via `ExceptionTranslationFilter`. A `@ControllerAdvice` `@ExceptionHandler(Exception.class)` must not swallow it into a 500.

Trade-offs

	URL rules	Method security
Protects	HTTP endpoints	any bean method, any caller
Can see	path, method, headers	arguments, return value, domain objects
Cost	cheap, before MVC	a proxy per bean. `@PostAuthorize` runs the method (and its SQL) before denying
Best for	coarse gates (`/admin/**`)	ownership and business rules

Use both: URLs as the outer fence, methods for "may *this* user touch *this* record". For large collections skip `@PostFilter` (loads everything, discards most) — filter in the query (`where owner = :currentUser`).

Common mistakes

- Forgetting `@EnableMethodSecurity` — annotations silently ignored.
- Self-invocation; annotating `private` methods.
- `@PostFilter` on an immutable `List.of(...)` — it removes elements, so needs a mutable collection.
- Complex SpEL strings — move logic into a bean: `@PreAuthorize("@orderGuard.canRead(#id, authentication)")`.

TL;DR

`@EnableMethodSecurity` + `@PreAuthorize` for business rules and ownership. Proxy-based: self-invocation and private methods are unprotected; `@PostAuthorize` runs the method (and its SQL) before denying. A denial becomes 403 over HTTP.

10.8 CSRF protection

What it is

Cross-site request forgery (CSRF) tricks a logged-in user's browser into sending a state-changing request to your site from another site; the browser attaches the session cookie automatically. The attack and browser-side defences (tokens, `SameSite` cookies) are in JS 14.11; here, what Spring does.

How Spring's CSRF protection works

1. `CsrfFilter` generates a random **CSRF token** per session, stored by default in the HTTP session (`HttpSessionCsrfTokenRepository`).
2. Server-rendered pages include it: Spring's login page, Thymeleaf forms and JSP tags add a hidden `_csrf` field automatically.
3. For **unsafe** methods (`POST` , `PUT` , `PATCH` , `DELETE`) the filter compares the `_csrf` parameter or `X-CSRF-TOKEN` / `X-XSRF-TOKEN` header with the stored token; missing or wrong → **403**.
4. **Safe** methods (`GET` , `HEAD` , `OPTIONS` , `TRACE`) are not checked — never change state in a `GET`.
5. Since Spring Security 6 the token is loaded **lazily** (no session touch when not needed) and the rendered value is **XOR-masked** with fresh randomness per request (defeats BREACH, which recovers secrets from compressed-response sizes): it looks different each time but validates the same.

Proved: on for the cookie app, off for the stateless API

```
// web chain (session cookie): CSRF on
mvc.post().uri("/web/notes").param("text", "hi").with(user("alice"))                          // no token
mvc.post().uri("/web/notes").param("text", "hi").with(user("alice")).with(csrf().useInvalidToken())
mvc.post().uri("/web/notes").param("text", "hi").with(user("alice")).with(csrf())                // valid token
// api chain (Basic auth, STATELESS): CSRF disabled
mvc.post().uri("/api/notes").contentType(MediaType.TEXT_PLAIN).content("from curl").with(httpBasic("alice", "alice-pass"))
```

Output:

```
login page hidden field: <input name="_csrf" type="hidden" value="<token>" />
POST /web/notes, logged in, no CSRF token  -> 403
POST /web/notes, logged in, wrong token    -> 403
POST /web/notes, logged in, valid token    -> 200
GET  /web/home,  no token needed (safe)     -> 200
POST /api/notes, Basic auth, no CSRF token -> 201
```

When may you disable it?

CSRF needs **ambient credentials** — ones the *browser* attaches on its own: cookies, browser-cached HTTP Basic, client certificates.

Your API authenticates with...	CSRF protection
session cookie (form login, OAuth2 login, BFF — Backend for Frontend, a server holding tokens for a browser app)	**keep it on**
`Authorization: Bearer <token>` added by JavaScript or a server, never cookies	can be disabled for that chain
Basic auth from scripts / other services only	can be disabled
both cookies *and* tokens on the same endpoints	keep it on

Single-page apps: `csrf.spa()` (Spring Security 7)

A cookie-session SPA still needs CSRF protection but can't read a hidden field. `http.csrf(c -> c.spa())` sets up the standard arrangement:

- token in a JavaScript-readable cookie `XSRF-TOKEN` (`HttpOnly=false`),
- the app echoes it in `X-XSRF-TOKEN` (Angular and axios do this automatically),
- a handler accepting both the raw cookie value and the masked form.

Before 7.0 this took `CookieCsrfTokenRepository.withHttpOnlyFalse()` plus a custom handler, as many tutorials still show.

Common mistakes

- `csrf.disable()` "because POST returned 403" in a session-cookie app — re-opens the attack. Send the token.
- Actions on `GET` (`GET /delete?id=5`) — unprotected, and link prefetchers can trigger them.
- Forgetting that `POST /logout` also needs the token (§10.10).

TL;DR

On by default for unsafe methods. Keep it wherever the browser sends credentials automatically; disable only for explicit-header chains; `csrf.spa()` for cookie-based SPAs.

10.9 CORS

What it is

CORS (Cross-Origin Resource Sharing) is how a *browser* lets JavaScript from origin A (scheme + host + port) read responses from origin B; the server opts in with `Access-Control-Allow-*` headers. For "non-simple" requests (e.g. `POST` with JSON or an `Authorization` header) the browser first sends a **preflight** `OPTIONS`. Browser rules: JS 14.2. CORS **protects nothing on the server** — it relaxes a browser restriction, and `curl` ignores it.

Why Spring Security must be involved

A preflight carries **no credentials** (browsers never send cookies or `Authorization` on it). If authentication ran first it would answer 401 and the browser would block the real request. `http.cors(...)` adds a `CorsFilter` *early* so preflights are answered before authentication.

```
@Bean
CorsConfigurationSource corsConfigurationSource() {
    CorsConfiguration api = new CorsConfiguration();
    api.setAllowedOrigins(List.of("https://app.example.com"));
    api.setAllowedMethods(List.of("GET", "POST"));
    api.setAllowedHeaders(List.of("Authorization", "Content-Type"));
    api.setMaxAge(3600L);                              // browser may cache the preflight 1 hour
    UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
    source.registerCorsConfiguration("/api/**", api);
    return source;
}
// in the API chain:  .cors(Customizer.withDefaults())   -> uses this bean
```

Proved: a preflight, allowed and refused

```
mvc.options().uri("/api/notes")
        .header("Origin", origin)
        .header("Access-Control-Request-Method", "POST")
        .header("Access-Control-Request-Headers", "authorization,content-type")
```

Output:

```
preflight from https://app.example.com -> 200
  Access-Control-Allow-Origin: https://app.example.com
  Access-Control-Allow-Methods: GET,POST
  Access-Control-Allow-Headers: authorization, content-type
  Access-Control-Max-Age: 3600
preflight from https://evil.example   -> 403 body=Invalid CORS request
GET with Origin + Basic -> 200, Access-Control-Allow-Origin: https://app.example.com, Vary: [Origin, Access-Control-Request-Method, Access-Control-Request-Headers]
```

The allowed preflight gets **200 without credentials**; an unknown origin **403 "Invalid CORS request"**; the real request also gets `Access-Control-Allow-Origin`, with `Vary: Origin` telling caches the answer depends on the origin.

Trade-offs and choices

Option	Where	Use when
`CorsConfigurationSource` bean + `http.cors(...)`	security chain	the endpoints are secured (almost always)
`@CrossOrigin` on controllers / `WebMvcConfigurer.addCorsMappings`	Spring MVC (Chapter 5)	no Spring Security, or as the source that `http.cors()` falls back to when no `CorsConfigurationSource` bean exists
gateway / reverse proxy (see System Design Fundamentals)	outside the app	many services behind one edge

The general CORS mistakes (`"*"` with credentials, CORS only in MVC) are listed in Chapter 5, section 5.10.

TL;DR

`CorsConfigurationSource` + `http.cors(...)` so preflights are answered before authentication. Exact origins.

10.10 Sessions, the security context and logout

What it is

After form login the `Authentication` must survive to the next request: by default the `SecurityContext` is saved in the **HTTP session** (`HttpSessionSecurityContextRepository`), whose id the browser keeps in the `JSESSIONID` cookie. `sessionManagement(...)` controls this:

`SessionCreationPolicy`	Meaning
`IF_REQUIRED` (default)	create a session when needed (e.g. at login)
`ALWAYS`	always create one
`NEVER`	never create one, but use an existing session
`STATELESS`	never create or use one for the security context, so every request must authenticate itself (Basic, Bearer token)

Session fixation, concurrency and logout — proved

Session fixation: an attacker plants a known session id in the victim's browser *before* login and reuses it after. Defence: **change the session id at login** — Spring's default (`changeSessionId()`, via Servlet `HttpServletRequest.changeSessionId()`).

```
MockHttpSession session = new MockHttpSession();       // the anonymous visitor's session
String before = session.getId();
var login = mvc.post().uri("/login").session(session)
        .param("username", "alice").param("password", "alice-pass").with(csrf()).exchange();
MockHttpSession after = (MockHttpSession) login.getRequest().getSession(false);
// ... use the new session, then POST /logout with it
```

Output:

```
login  -> 302 /, session id changed: true (session fixation protection)
GET /web/home with that session -> 200
logout -> 302 /login?logout, session invalidated: true
API call -> session created? false, Set-Cookie: null
```

- Login changed the session id; the old one is worthless.
- `POST /logout` (with CSRF token) invalidated the session, cleared the context and redirected to `/login?logout`. `GET /logout` only shows a confirmation page — logout is a state change needing CSRF protection.
- The `STATELESS` API chain never created a session or sent `Set-Cookie`.

Other session features:

- **`maximumSessions(1)`** limits concurrent logins (the second expires the first, or `maxSessionsPreventsLogin(true)` rejects it); register an `HttpSessionEventPublisher` bean so Spring hears about timeouts.
- **Remember-me** (`rememberMe(...)`): a long-lived cookie across browser restarts; prefer the persistent-token variant.

- **Timeout:** `server.servlet.session.timeout=30m`; harden the cookie with `server.servlet.session.cookie.secure=true`, `http-only=true` (default), `same-site=lax` or `strict`.
- Several instances need a shared store: **Spring Session** keeps sessions in Redis or JDBC.

Stateless vs session: which one?

	Server session + cookie	Stateless (token per request)
Logout / revoke	immediate (invalidate session)	hard until the token expires (Chapter 11)
Scaling	needs sticky sessions (load balancer pins a user to one server) or Spring Session	any node can serve any request
CSRF	needed	not needed if the token is not in a cookie
Best for	browser apps, BFF	APIs called by other services, mobile apps

TL;DR

Form login stores the context in the session; the id changes at login; logout is `POST /logout` with a CSRF token; `STATELESS` chains never create sessions.

10.11 Security headers

What it is

`HeaderWriterFilter` adds response headers that switch on browser defences. From the test:

```
response headers on GET /web/home:
  Cache-Control: no-cache, no-store, max-age=0, must-revalidate
  Content-Length: 4
  Content-Security-Policy: default-src 'self'; frame-ancestors 'none'
  Content-Type: text/plain;charset=UTF-8
  Expires: 0
  Pragma: no-cache
  Referrer-Policy: strict-origin-when-cross-origin
  X-Content-Type-Options: nosniff
  X-Frame-Options: DENY
  X-XSS-Protection: 0
over HTTPS also: Strict-Transport-Security: max-age=31536000 ; includeSubDomains
```

Header	Default?	What it tells the browser
`Cache-Control` / `Pragma` / `Expires`	yes	don't cache authenticated pages (shared computers, proxies). Your own `Cache-Control` wins
`X-Content-Type-Options: nosniff`	yes	trust `Content-Type`, don't guess (stops some script-injection tricks)
`X-Frame-Options: DENY`	yes	don't render inside a frame (**clickjacking**: hiding your page in an invisible frame to steal clicks)
`Strict-Transport-Security` (HSTS — HTTP Strict Transport Security)	yes, **HTTPS responses only**	use HTTPS for this host for one year
`X-XSS-Protection: 0`	yes	*turn off* the old browser XSS filter, which itself caused vulnerabilities
`Content-Security-Policy`	**no**, you write it	which sources may load scripts, styles and frames. It is the strongest defence against **XSS**
`Referrer-Policy`	no (configured here)	how much of the URL to leak in `Referer`

We added CSP and Referrer-Policy in the web chain:

```
.headers(h -> h
    .contentSecurityPolicy(csp -> csp.policyDirectives("default-src 'self'; frame-ancestors 'none'"))
    .referrerPolicy(r -> r.policy(ReferrerPolicy.STRICT_ORIGIN_WHEN_CROSS_ORIGIN)))
```

XSS and CSP authoring: JS 14.10, JS 14.12. Roll out a new CSP with `.reportOnly()` first.

Common mistakes

- Disabling all headers (`headers(h -> h.disable())`) to fix one (e.g. an iframe) — change just that one: `frameOptions(f -> f.sameOrigin())`.
- Expecting HSTS on `http://localhost` — HTTPS only. Behind a TLS-terminating proxy set `server.forward-headers-strategy=framework` so the app knows the request was HTTPS.

TL;DR

Five protective headers on by default (HSTS only over HTTPS); add a CSP yourself; change individual headers rather than disabling all.

10.12 Testing security with `spring-security-test`

What it is

`spring-boot-starter-security-test` brings **spring-security-test**: helpers that put an identity or token into a test request — mock identities for rules, real credentials where login itself is under test:

Helper	Kind	What it does
`@WithMockUser(username, roles, authorities)`	annotation	puts a `UsernamePasswordAuthenticationToken` in the context for the test method, with no `UserDetailsService` involved. Default user `user`, role `USER`
`@WithUserDetails("alice")`	annotation	loads the real user from *your* `UserDetailsService`
`@WithAnonymousUser`	annotation	explicit anonymous caller
`with(user("alice").roles("USER"))`	request post-processor	like `@WithMockUser`, per request
`with(httpBasic("alice", "alice-pass"))`	request post-processor	adds a real `Authorization: Basic` header, so the full login path runs
`with(csrf())` / `csrf().useInvalidToken()`	request post-processor	adds a valid (or broken) CSRF token
`formLogin().user(..).password(..)`	request builder	a real `POST /login` including CSRF
`logout()`	request builder	a real `POST /logout`
`jwt()`, `opaqueToken()`, `oauth2Login()`, `oidcLogin()`	request post-processors	token and OAuth2 identities (Chapter 11)
`authenticated().withUsername(..)`, `unauthenticated()`, `withAuthorities(..)`	result matchers	assert on the security context after the request. `withAuthorities(String...)` is new in 7.0

`MockMvcTester` accepts these with `.with(...)`. `@SpringBootTest` + `@AutoConfigureMockMvc` applies the security chain automatically; with the `@WebMvcTest` slice add `@Import(SecurityConfig.class)` so *your* rules run, not Boot's defaults (Chapter 12).

What to test

1. **Each rule's boundaries**: anonymous, wrong role, right role (the 401/403 matrix is one test).
2. **The catch-all**: an unlisted URL is denied.
3. **CSRF**: unsafe requests fail without a token, succeed with one.
4. **Method rules** directly on the service with `@WithMockUser`, including ownership.
5. **At least one real login** (`httpBasic`/`formLogin`) to exercise the encoder and user lookup.

Common mistakes

- Testing only the happy path with `@WithMockUser(roles = "ADMIN")` — proves admins get in, not that others are kept out.
- `@WithMockUser` when code needs your custom principal class — use `@WithUserDetails` or a custom `@WithSecurityContext`.
- Forgetting `with(csrf())` and concluding "security blocks my POST".

TL;DR

`@WithMockUser`/`user()` for rules, `httpBasic()`/`formLogin()` for real logins, `csrf()` for unsafe methods, result matchers for "who is logged in". Test the denials.

10.13 Common mistakes (whole chapter)

Mistake	Symptom	Fix
Own `SecurityFilterChain` not picked up	generated password still printed	bean inside a scanned `@Configuration`
Catch-all chain ordered first	specific chain ignored	`securityMatcher` + `@Order`, specific first
`hasRole("ROLE_X")`	startup `IllegalArgumentException`	`hasRole("X")` or `hasAuthority("ROLE_X")`
`roles()` then `authorities()` on `User`	roles vanish	one call with the full list
`csrf.disable()` with session login	CSRF hole	send the token, or `csrf.spa()`
CORS only in MVC	preflight 401	`http.cors(...)` + `CorsConfigurationSource`
Plain/SHA hashed passwords	leak = cracked passwords	`DelegatingPasswordEncoder` with bcrypt/Argon2
Method security not enabled / self-invocation	annotations ignored	`@EnableMethodSecurity`, call through another bean
`anyRequest().permitAll()` at the end	new endpoints public	`denyAll()` or `authenticated()`
Security filter declared as `@Component`	runs twice, once outside the chain	`addFilterBefore` + disabled `FilterRegistrationBean`
Context lost in `@Async`	`AuthenticationCredentialsNotFoundException`	`DelegatingSecurityContextExecutor`
Actuator exposed with the app rules	`/actuator/env` public	its own chain or rules (Chapter 15)

10.14 In the interview

"Walk me through what happens when a request hits a Spring Security app." "The container calls `DelegatingFilterProxy`, which hands off to the `FilterChainProxy` bean; it picks the first `SecurityFilterChain` whose matcher fits. Filters run in order: the context is loaded (e.g. from the session), headers added, CSRF checked for unsafe methods, authentication filters try form/Basic/bearer, an anonymous token is set if nobody authenticated, and `AuthorizationFilter` checks the rules. `ExceptionTranslationFilter` turns a denial into 401/redirect for anonymous callers or 403 for known ones. If everything passes, the request reaches the `DispatcherServlet`."

Follow-ups:

- *Where is the user stored during and between requests?* → During: `SecurityContextHolder` (`ThreadLocal`). Between: the HTTP session (form login), or nowhere for stateless chains (client resends credentials/token).
- *Passwords?* → §10.5. *When is disabling CSRF safe?* → §10.8 (no ambient credentials, e.g. pure bearer-token APIs).
- *"Users only see their own orders"?* → Method security ownership expression, or better, scope the query by the current user; URL rules can't see ownership.
- *What changed in Spring Security 7?* → Lambda DSL only, old matchers and `authorizeRequests` removed, built-in MFA, `csrf.spa()`, Authorization Server merged in (§10.3).

Chapter 10 TL;DR

- **Architecture**: `DelegatingFilterProxy` → `FilterChainProxy` → first matching `SecurityFilterChain` → ordered filters → `AuthorizationFilter`. The user lives in `SecurityContextHolder`.
- **Configuration**: `SecurityFilterChain` beans, one lambda per feature. Spring Security 7 removed `and()`, `authorizeRequests` and the Ant/Mvc matchers. Use several chains with `securityMatcher` + `@Order`.
- **Authentication**: filter → `ProviderManager` → `DaoAuthenticationProvider` → `UserDetailsService` + `PasswordEncoder`. Browsers get a 302 to `/login`, APIs get a 401 challenge. Spring Security 7 adds `FACTOR_*` authorities for MFA.
- **Passwords**: adaptive salted hashes (bcrypt, Argon2id) via `DelegatingPasswordEncoder` (`{id}hash`), upgraded on login.
- **Authorization**: first matching rule wins. Roles are `ROLE_`-prefixed authorities. **`401 = who are you, 403 = no`**. End with `denyAll()`. Use method security for ownership.
- **Web defences**: CSRF on for cookie sessions (off only for header-token chains, `csrf.spa()` for SPAs). `http.cors()` so preflights pass. Session id changes at login. Default headers plus your own CSP.
- **Testing**: spring-security-test's `@WithMockUser`, `user()`, `httpBasic()`, `csrf()`, `formLogin()`, and test the denials.

Next: Chapter 11 — Tokens and OAuth2 replaces sessions and passwords with signed tokens and identity providers.

Chapter 11 — Spring Security 2: Tokens and OAuth2

Chapter 10 authenticated people with a password and a session cookie — fine for one browser and one server. Modern systems add a mobile app, a **single-page app** (SPA: loads one page, then updates it with JavaScript), many **microservices** (small, separately deployed services forming one system), partner backends, and an **identity provider** (IdP: the system owning user accounts and checking logins — Keycloak, Okta, Entra ID, Google). They must prove identity **without sharing passwords or sessions**: with **tokens**, handed out via **OAuth 2.0** with **OpenID Connect** on top. Signing keys are generated locally, so **no external IdP is needed** to run anything.

All code lives in `code/` — three Spring Boot **4.1.1** projects plus one plain-JDK project (Spring Security **7.1.1**, Java 25). Output blocks are real `mvn -q -B -ntp verify` output (log lines trimmed as `...`; timestamps and random values change per run).

Project	Shows
`jwt-resource-server`	local RSA key, `/token` endpoint with `NimbusJwtEncoder`, a JWT-protected API, every validation failure, `jwt()` tests
`resource-server-variants`	opaque tokens with an introspector, multi-tenant JWTs (two issuers, two keys)
`oauth2-login-client`	OAuth2/OIDC login configuration. The redirect to the provider, `state`/`nonce`/PKCE, and `oidcLogin()` tests
`crypto-basics`	plain JDK: Base64 vs SHA-256 vs AES-GCM vs Ed25519 signatures, an X25519 key exchange, and TOTP checked against the RFC 6238 test vectors (§11.18)

> **Prerequisites.** Filter chain, 401 vs 403, authorities, CSRF: Chapter 10. Browser storage, cookies and XSS: JavaScript handbook Chapter 14. Gateways and BFFs as patterns: API Gateway, Backend for Frontend.

11.1 From sessions to tokens: bearer tokens and JWT

What it is

A **bearer token** grants access to whoever *bears* (holds) it, like a cinema ticket: the server checks only that the ticket is valid. It travels in an HTTP header:

```
GET /api/messages HTTP/1.1
Authorization: Bearer eyJraWQiOiJkZW1vLWtleS0xIiwidH...
```

A **JWT (JSON Web Token, RFC 7519)** is the most common format (an RFC is a numbered internet-standard document): three **Base64URL**-encoded parts (URL-safe, reversible by anyone) joined by dots, `header.payload.signature`. A token from our `/token` endpoint, decoded:

```
token (3 base64url parts) : eyJraWQiOiJkZW1vLWtleS0xIiwidH... length=670
header  : {"kid":"demo-key-1","typ":"JWT","alg":"RS256"}
payload : {"sub":"bob","aud":"notes-api","scope":"messages.read messages.write","roles":["ADMIN"],"iss":"https://auth.demo.test","exp":
<epoch>,"iat":<epoch>,"jti":"<uuid>"}
```

Part	Contains	Notes
Header	`alg` (signature algorithm), `kid` (key id), `typ`	tells the receiver *how* to verify
Payload	**claims**: statements about the subject	readable by anyone, since it is only encoded, not encrypted
Signature	signature over `header.payload`	proves who issued it and that nothing changed

The registered claims you will meet everywhere:

Claim	Meaning	Checked by Spring by default?
`iss` (issuer)	who created the token	yes, if you configure an issuer
`sub` (subject)	who the token is about (user or client id)	becomes `Authentication#getName()`
`aud` (audience)	which API(s) the token is meant for	no, you add it (property `audiences` or a validator)
`exp` / `nbf` / `iat`	expires at / not before / issued at (epoch seconds: since 1 Jan 1970 UTC)	`exp` and `nbf` yes, with 60 s clock skew (allowed gap between server clocks)
`jti`	unique token id	no, useful for revocation lists
`scope` / `scp`	granted OAuth scopes (space-separated string or list)	mapped to `SCOPE_x` authorities

Signing: symmetric vs asymmetric

	HMAC (`HS256` , a hash keyed with a shared secret)	RSA / EC (`RS256` , `ES256` , `PS256` ; public-key algorithms, EC = elliptic curve)
Keys	one shared secret signs *and* verifies	the **private** key signs, the **public** key verifies
Who can mint tokens	everybody who can verify	only the key owner
Fits	one service issuing tokens to itself	an identity provider and many APIs (the normal case)

Resource servers download the issuer's public keys as a **JWK Set** (JSON Web Key Set, JWKS) from a URL such as `https://idp.example.com/.well-known/jwks.json`; the header's `kid` names the key, which is how keys **rotate** (§11.13).

JWS vs JWE. A signed JWT is a **JWS** (JSON Web Signature); an *encrypted* JWT is a **JWE**, rare in APIs. Never put secrets or personal data you would not show the user into a normal JWT — any holder can decode the payload; the signature protects integrity, not confidentiality.

Why tokens

- **No server-side session.** Any instance verifies the signature locally, so APIs **scale horizontally** without a shared session store.
- **Many parties.** The IdP issues, APIs only verify and never see passwords.
- **Delegation.** A token carries *limited* rights (scopes), e.g. "read calendar" but not "delete account".

Trade-offs

- **Revocation is hard.** A self-contained JWT stays valid until `exp`, even after logout or a password change. Keep access tokens short-lived (minutes) with refresh tokens (§11.11), or use **opaque tokens** (§11.7).
- **Size.** Our small token is 670 bytes, sent on every request.
- **Theft = access.** Always HTTPS (System Design Fundamentals), short lifetimes, safe storage (§11.12).

TL;DR

A JWT is `header.payload.signature`: claims readable, signature proves origin and integrity. Use asymmetric keys (APIs hold only the public key, so a compromised API cannot mint tokens), short lifetimes, and verify `exp`, `iss` and `aud`.

11.2 OAuth 2.0 and OpenID Connect in one picture

What it is

OAuth 2.0 (RFC 6749) is a framework for **delegated authorization**: letting an application act on a user's behalf *without receiving the user's password*. Like a hotel key card: reception (authorization server) checks your passport once and issues a card (access token) opening *your* room (scopes) until checkout (expiry); doors (APIs) only check the card.

OpenID Connect (OIDC) is a thin layer on top of OAuth 2.0 that adds **authentication**: an **ID token** (always a JWT) telling the client *who logged in*, a standard `userinfo` endpoint, and discovery metadata at `/.well-known/openid-configuration`.

OAuth role	Who it is	Spring module
Resource owner	the user	—
Client	the app that wants access (web app, SPA backend, mobile app, service)	`spring-security-oauth2-client` (`oauth2Login` , `oauth2Client`)
Authorization server (identity provider, IdP)	issues tokens: Keycloak, Okta, Entra ID, Auth0, Google, Spring Authorization Server	`spring-security-oauth2-authorization-server` (§11.14)
Resource server	the API that accepts tokens	`spring-security-oauth2-resource-server` (`oauth2ResourceServer`)

The tokens

Token	For whom	Format	Lifetime
Access token	the resource server (API)	JWT or opaque	short: 5–60 minutes
Refresh token	the authorization server only, to get new access tokens	usually opaque	hours to weeks, and can be revoked
ID token (OIDC)	the client, to learn who logged in	always a JWT	short, used once at login

Never send an ID token to an API as an access token, nor use an access token to "log a user in" to the client — different audiences.

Grant types (how a client gets a token)

Grant	Used by	Status
Authorization code + PKCE	any app with a user: server web apps, SPAs (via BFF), mobile	the standard (§11.9)
Client credentials	service-to-service, no user involved	standard (§11.10)
Refresh token	a client renewing an access token	standard (§11.11)
Device code (RFC 8628)	TVs, CLIs without a browser	standard
Token exchange (RFC 8693)	a service swapping a token for one aimed at another API	growing
Implicit	old SPAs (token in the URL fragment)	**deprecated**, left out of OAuth 2.1
Resource owner password	app collects the user's password	**deprecated**. Spring Security 7 **removed** its client support

The **OAuth 2.1** draft consolidates these best practices: PKCE for every authorization-code client, no implicit and no password grant, exact redirect-URI matching.

Scopes vs roles

A **scope** (`messages.read`) limits what *the client* may do on the user's behalf (the user consented to it); a **role** is what *the user* may do in your system. A token can carry both; Spring maps scopes to `SCOPE_x` authorities by default (§11.5).

TL;DR

OAuth 2.0 = delegated authorization with access tokens. OIDC adds login with ID tokens. Use authorization code + PKCE for users and client credentials for services. Implicit and password grants are obsolete.

11.3 A JWT resource server in Spring

What it is

A **resource server** is an API that accepts access tokens. With `spring-boot-starter-security-oauth2-resource-server` (Boot 4's name; the old `spring-boot-starter-oauth2-resource-server` is deprecated), one DSL call adds a `BearerTokenAuthenticationFilter` to the filter chain:

```
http.oauth2ResourceServer(rs -> rs.jwt(Customizer.withDefaults()));
```

With a real identity provider, one property is usually all you need:

```
# Spring reads https://idp.example.com/.well-known/openid-configuration (or oauth-authorization-server)
# to find the JWK Set URL, and validates that iss equals this value
spring.security.oauth2.resourceserver.jwt.issuer-uri=https://idp.example.com/realms/demo
# optional extras
spring.security.oauth2.resourceserver.jwt.audiences=notes-api
spring.security.oauth2.resourceserver.jwt.jws-algorithms=RS256
# alternatives: jwk-set-uri=... (no discovery) or public-key-location=classpath:key.pub (static key)
```

Without an IdP, we declare the `JwtDecoder` bean ourselves with a locally generated RSA key pair. A `JwtDecoder` bean always takes precedence over the properties.

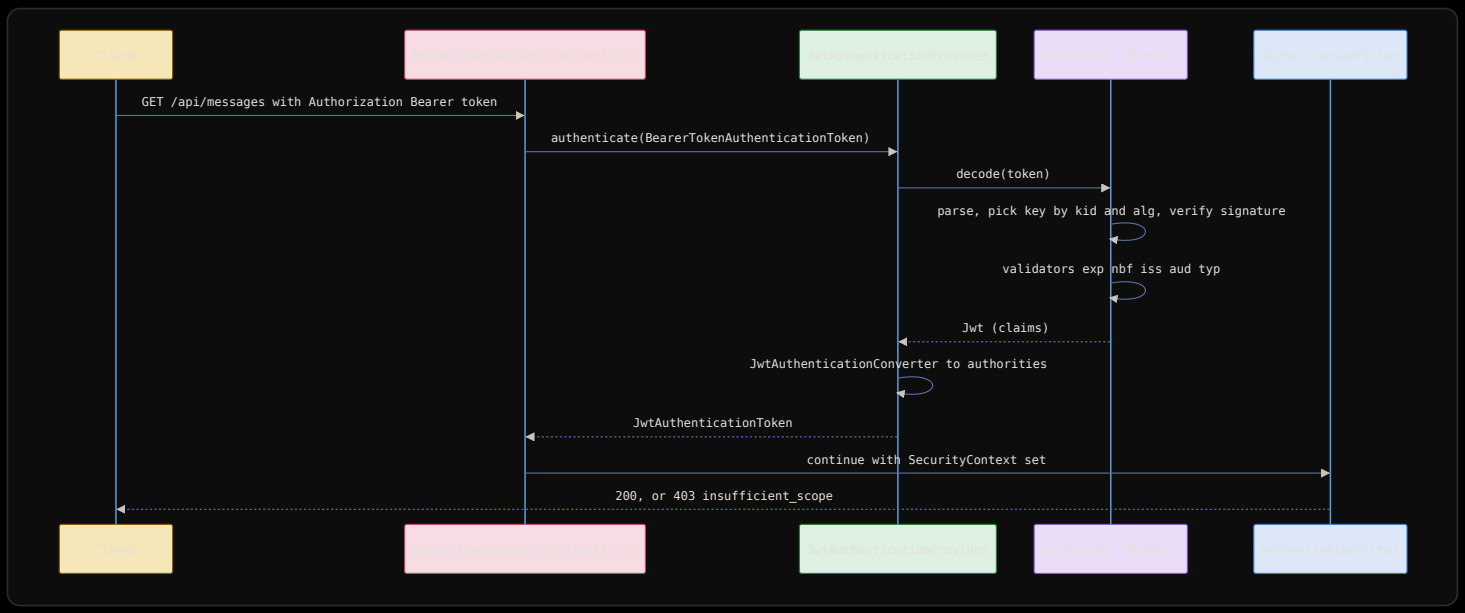
```
@Bean
KeyPair signingKeys() throws Exception {                    // demo only: new keys at each start
    KeyPairGenerator gen = KeyPairGenerator.getInstance("RSA");
    gen.initialize(2048);
    return gen.generateKeyPair();
}

@Bean
JwtDecoder jwtDecoder(KeyPair keys, @Value("${demo.jwt.issuer}") String issuer,
                      @Value("${demo.jwt.audience}") String audience) {
    NimbusJwtDecoder decoder = NimbusJwtDecoder.withPublicKey((RSAPublicKey) keys.getPublic()).build();
    OAuth2TokenValidator<Jwt> audienceCheck =
            new JwtClaimValidator<List<String>>("aud", aud -> aud != null && aud.contains(audience));
    decoder.setJwtValidator(new DelegatingOAuth2TokenValidator<>(
            JwtValidators.createDefaultWithIssuer(issuer), audienceCheck));
    return decoder;
}
```

The API chain:

```
@Bean @Order(2)
SecurityFilterChain api(HttpSecurity http, JwtAuthenticationConverter converter) throws Exception {
    http.authorizeHttpRequests(a -> a
            .requestMatchers(HttpMethod.GET, "/api/messages").hasAuthority("SCOPE_messages.read")
            .requestMatchers(HttpMethod.POST, "/api/messages").hasAuthority("SCOPE_messages.write")
            .requestMatchers("/api/admin/**").hasRole("ADMIN")
            .anyRequest().authenticated())
        .oauth2ResourceServer(rs -> rs.jwt(jwt -> jwt.jwtAuthenticationConverter(converter)))
        .csrf(c -> c.disable())                // no cookies -> no CSRF risk
        .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS));
    return http.build();
}
```

How it works



1. `BearerTokenAuthenticationFilter` takes the token from `Authorization: Bearer ...` (form parameters and query strings are off by default — tokens in URLs leak into logs).
2. `JwtAuthenticationProvider` calls the **`JwtDecoder`**: Nimbus (the JWT library underneath) parses it, rejects algorithms not configured (the classic `alg: none` trick fails here), and **verifies the signature**.
3. **Validators** run: `JwtTimestampValidator` (`exp` / `nbf`, 60 s skew), the issuer validator if configured, your own (audience), and in Spring Security 7 a `JwtTypeValidator` accepting `typ` = `JWT` or none (now in all `JwtValidators.createDefault...()`).
4. A **`JwtAuthenticationConverter`** maps claims to authorities (§11.5) and builds a `JwtAuthenticationToken`; controllers can take `@AuthenticationPrincipal Jwt jwt`.
5. Any failure → **401** with `WWW-Authenticate: Bearer error="invalid_token"` (RFC 6750). Valid token without the needed authority → **403** `error="insufficient_scope"`.

Proved: status codes with real tokens

Nothing is mocked: the test obtains real tokens from `/token` and calls the API:

```
no token                          401 WWW-Authenticate: Bearer resource_metadata="http://localhost/.well-known/oauth-protected-resource"
alice GET (messages.read)         200 ["welcome"]
alice POST (needs messages.write) 403 WWW-Authenticate: Bearer error="insufficient_scope", error_description="The request requires higher
privileges than provided by the access token.", error_uri="https://tools.ietf.org/html/rfc6750#section-3.1"
bob POST                          201 added by bob
bob GET admin (roles=[ADMIN])     200 audit log for bob
garbage token                     401 WWW-Authenticate: Bearer error="invalid_token", error_description="An error occurred while attempting
to decode the Jwt: Malformed token", error_uri="https://tools.ietf.org/html/rfc6750#section-3.1", resource_metadata="http://localhost/.well-
known/oauth-protected-resource"
```

`resource_metadata` points to the **OAuth 2.0 Protected Resource Metadata** document (RFC 9728) the resource server publishes, so a client knowing nothing about the API can discover which authorization server to ask:

```
GET /.well-known/oauth-protected-resource -> 200 {"resource":"http://localhost","bearer_methods_supported":
["header"],"tls_client_certificate_bound_access_tokens":true}
```

With a real issuer it also lists `authorization_servers`; customise via `oauth2ResourceServer(rs -> rs.protectedResourceMetadata(...))`. The Model Context Protocol (MCP: how AI assistants call external tools) authorization spec relies on this discovery.

Proved: every validation rule

The test mints tokens with deliberate defects:

```
String expiredLongAgo = mint(encoder, c -> c.issuedAt(now.minusSeconds(3600)).expiresAt(now.minusSeconds(600)));
String expiredJustNow = mint(encoder, c -> c.issuedAt(now.minusSeconds(600)).expiresAt(now.minusSeconds(30)));
String wrongIssuer    = mint(encoder, c -> c.issuer("https://evil.test"));
String wrongAudience  = mint(encoder, c -> c.audience(List.of("billing-api")));
String otherKey       = mint(attacker, c -> {});              // attacker's own RSA key pair
// valid token, payload swapped for one with more scopes, original signature kept
String tampered = parts[0] + "." + forgedPayload + "." + parts[2];
```

Output (the `error_uri` and `resource_metadata` parameters are trimmed as `...`):

```
valid                           200 ["welcome","hi"]
expired 10 min ago              401 WWW-Authenticate: Bearer error="invalid_token", error_description="An error occurred while attempting
to decode the Jwt: Jwt expired at 2026-09-28T23:19:21Z", ...
expired 30 s ago (clock skew 60 s) 200 ["welcome","hi"]
wrong issuer                    401 WWW-Authenticate: Bearer error="invalid_token", error_description="An error occurred while attempting
to decode the Jwt: The iss claim is not valid", ...
wrong audience                  401 WWW-Authenticate: Bearer error="invalid_token", error_description="An error occurred while attempting
to decode the Jwt: The aud claim is not valid", ...
signed with another key         401 WWW-Authenticate: Bearer error="invalid_token", error_description="An error occurred while attempting
to decode the Jwt: Signed JWT rejected: Invalid signature", ...
payload edited after signing    401 WWW-Authenticate: Bearer error="invalid_token", error_description="An error occurred while attempting
to decode the Jwt: Signed JWT rejected: Invalid signature", ...
```

- A token that expired **30 s ago still passes** — 60 s clock skew. Remember this when testing expiry.
- **Changing one payload character breaks the signature** — why readable claims can be trusted.
- **Audience matters.** A `billing-api` token from the *same* trusted issuer is refused; without an audience check, every API would accept any token from your IdP.

Common mistakes

- **Skipping the audience check** — set `spring.security.oauth2.resourceserver.jwt.audiences` or add a validator.
- **Replacing** the validators: `decoder.setJwtValidator(audienceCheck)` alone drops the expiry check! Combine with `JwtValidators.createDefault...()`.
- Session-based policy (`IF_REQUIRED`) on the resource server, so a session cookie becomes a second way in. Use `STATELESS` and disable CSRF only on the token chain.
- Logging full tokens — log `jti` or `sub`.

TL;DR

`oauth2ResourceServer(jwt)` + `issuer-uri` (or your own `JwtDecoder`). Spring verifies the signature, `exp`/`nbf` (60 s skew), `iss`, `typ`, and your `aud` rule. Bad or missing token → 401 `invalid_token`. Missing authority → 403 `insufficient_scope`.

11.4 Minting tokens with `NimbusJwtEncoder`

What it is

`JwtEncoder` signs claims into a JWT. Spring Security 7 added builder methods to `NimbusJwtEncoder`, so creating one from a key pair or secret is a one-liner (previously a hand-built Nimbus `JWKSource`):

```
@Bean
JwtEncoder jwtEncoder(KeyPair keys) {
    return NimbusJwtEncoder.withKeyPair((RSAPublicKey) keys.getPublic(), (RSAPrivateKey) keys.getPrivate())
            .jwkPostProcessor(jwk -> jwk.keyID("demo-key-1"))   // becomes "kid" in the header
            .build();
}
// also: withKeyPair(ECPublicKey, ECPrivateKey) for ES256, withSecretKey(SecretKey) for HS256
```

Our `/token` endpoint (HTTP Basic, its own chain `@Order(1)`) turns the user's authorities into claims:

```
@PostMapping("/token")
TokenResponse token(Authentication auth) {
    Instant now = Instant.now();
    Duration ttl = Duration.ofMinutes(15);
    List<String> authorities = auth.getAuthorities().stream().map(GrantedAuthority::getAuthority)
            .filter(a -> !a.startsWith("FACTOR_")).toList();          // drop Security 7 MFA factors
    String scope = String.join(" ", authorities.stream().filter(a -> !a.startsWith("ROLE_")).toList());
    List<String> roles = authorities.stream().filter(a -> a.startsWith("ROLE_")).map(a -> a.substring(5)).toList();
    JwtClaimsSet claims = JwtClaimsSet.builder()
            .issuer(issuer).audience(List.of(audience)).subject(auth.getName())
            .issuedAt(now).expiresAt(now.plus(ttl)).id(UUID.randomUUID().toString())
            .claim("scope", scope).claim("roles", roles)
            .build();
    JwsHeader header = JwsHeader.with(SignatureAlgorithm.RS256).type("JWT").build();
    String jwt = encoder.encode(JwtEncoderParameters.from(header, claims)).getTokenValue();
    return new TokenResponse(jwt, "Bearer", ttl.toSeconds());
}
```

The `FACTOR_` filter drops Spring Security 7's `FACTOR_PASSWORD` (10.4): it says *how* the user logged in and doesn't belong in a scope.

When to issue your own tokens — and when not

Situation	Self-issued JWT (`JwtEncoder`)	Real authorization server
tests, demos, this book	ideal: no infrastructure	overkill
one internal service issuing short tokens to its own clients	acceptable	better long-term
users from several apps, SSO (single sign-on), social login, MFA, consent, refresh-token rotation, revocation	**no**, you would be re-writing an IdP (password storage, lockout, key rotation, revocation — forever)	**yes**: Keycloak, a cloud IdP, or Spring Authorization Server (§11.14)

TL;DR

`NimbusJwtEncoder.withKeyPair(...)` / `withSecretKey(...)` + `JwtClaimsSet` + `JwsHeader` = a signed token in a few lines. Tests and demos yes; real users need an authorization server.

11.5 Mapping claims to authorities

What it is

Authorization rules need **authorities**. By default `JwtAuthenticationConverter` uses **`JwtGrantedAuthoritiesConverter`**: reads `scope` (or `scp`), splits on spaces, prefixes `SCOPE_` (`"messages.read messages.write"` → `SCOPE_messages.read`, `SCOPE_messages.write`). Nothing else becomes an authority unless you say so.

IdPs put roles in different places (Keycloak `realm_access.roles`, Entra ID `roles`, Okta `groups`). Our token has `roles`, so the project combines two converters:

```
public class ScopeAndRoleAuthorities implements Converter<Jwt, Collection<GrantedAuthority>> {
    private final JwtGrantedAuthoritiesConverter scopes = new JwtGrantedAuthoritiesConverter(); // scope/scp -> SCOPE_
    private final JwtGrantedAuthoritiesConverter roles = new JwtGrantedAuthoritiesConverter();

    public ScopeAndRoleAuthorities() {
        roles.setAuthoritiesClaimName("roles");
        roles.setAuthorityPrefix("ROLE_");
    }

    @Override
    public Collection<GrantedAuthority> convert(Jwt jwt) {
        Collection<GrantedAuthority> all = new ArrayList<>(scopes.convert(jwt));
        all.addAll(roles.convert(jwt));
        return all;
    }
}

@Bean
JwtAuthenticationConverter jwtAuthenticationConverter(ScopeAndRoleAuthorities authorities) {
    JwtAuthenticationConverter converter = new JwtAuthenticationConverter();
    converter.setJwtGrantedAuthoritiesConverter(authorities);
    converter.setPrincipalClaimName("sub");                // Authentication#getName() = sub claim
    return converter;
}
```

So `roles: ["ADMIN"]` became `ROLE_ADMIN` and `bob GET admin` returned 200. If one claim is enough, Boot properties configure the default converter without code:

```
spring.security.oauth2.resourceserver.jwt.authorities-claim-name=roles
spring.security.oauth2.resourceserver.jwt.authority-prefix=ROLE_
spring.security.oauth2.resourceserver.jwt.principal-claim-name=preferred_username
```

Common mistakes

- `hasRole("messages.read")` / `hasAuthority("messages.read")` for a scope — the authority is `SCOPE_messages.read`.
- Nested claims (`realm_access.roles`): `JwtGrantedAuthoritiesConverter` reads top-level claims only — write a converter walking the map.
- `sub` for display: often an opaque id. Show `preferred_username` or `email`, but key data by `sub` (+ `iss`).

TL;DR

Default: `scope` → `SCOPE_x`. Anything else (roles, groups) needs a custom converter or the Boot `authorities-claim-name`/`authority-prefix` properties.

11.6 Testing token security

Two ways to test

Approach	How	Proves
Real tokens	mint with `JwtEncoder` (or call `/token`), send `Authorization: Bearer ...`	decoder, validators, converter *and* rules. Everything in §11.3 was tested this way
`jwt()` post-processor	`mvc.get().uri(..).with(jwt()...)`	only the **rules**: decoding and validation are skipped, a ready-made `JwtAuthenticationToken` is placed in the context

Use real tokens for a handful of decoder tests (issuer, audience, expiry), `jwt()` for the many rule tests (faster, no keys).

The `jwt()` pitfall — proved

```
var plain = mvc.get().uri("/api/messages").with(jwt());                  // sub=user, scope=read
var explicit = mvc.post().uri("/api/messages").contentType(MediaType.TEXT_PLAIN).content("x")
        .with(jwt().jwt(j -> j.subject("carol")).authorities(new SimpleGrantedAuthority("SCOPE_messages.write")));
var scopeClaim = mvc.get().uri("/api/messages").with(jwt().jwt(j -> j.claim("scope", "messages.read")));
// PITFALL: jwt() does not use your JwtAuthenticationConverter unless you pass it in
var rolesIgnored = mvc.get().uri("/api/admin/audit").with(jwt().jwt(j -> j.claim("roles", List.of("ADMIN"))));
var rolesMapped  = mvc.get().uri("/api/admin/audit")
        .with(jwt().jwt(j -> j.claim("roles", List.of("ADMIN"))).authorities(appConverter));
```

Output:

```
jwt() default, GET messages             -> 403
jwt() SCOPE_messages.write, POST        -> 201 added by carol
jwt() scope claim messages.read, GET    -> 200
jwt() roles=[ADMIN], default converter  -> 403
jwt() roles=[ADMIN], app converter      -> 200
```

1. `jwt()` alone gives subject `user`, scope `read` → `SCOPE_read`, not `SCOPE_messages.read` → 403.
2. `.authorities(...)` sets authorities directly, whatever the claims say.
3. Claims are mapped with the **default** `JwtGrantedAuthoritiesConverter`, so `scope` works…
4. …but your custom `roles` mapping is **not applied**: 403 although a real IdP token would get 200. Pass `.authorities(converter)`, or test that path with a real token.

Siblings: `opaqueToken()` (§11.7), `oauth2Login()` / `oidcLogin()` (§11.9), `oauth2Client()`. `@WithMockUser` tests URL rules too, but the principal is then not a `Jwt`, so `@AuthenticationPrincipal Jwt` parameters receive `null`.

TL;DR

Real locally signed tokens for the decoder; `jwt()` for rules — with your authorities converter, or you test a different mapping from production.

11.7 Opaque tokens and introspection

What it is

An **opaque token** is a random string with no readable content (`tok-alice-7f3a`). The resource server asks the authorization server's **introspection endpoint** (RFC 7662) "is it active, and for whom?" on every request (cache answers if you add it):

```
POST /oauth2/introspect   token=tok-alice-7f3a   (client authenticates with its own id/secret)
-> {"active": true, "sub": "alice", "scope": "notes.read", "exp": 1790000000, ...}
```

```
spring.security.oauth2.resourceserver.opaquetoken.introspection-uri=https://idp.example.com/oauth2/introspect
spring.security.oauth2.resourceserver.opaquetoken.client-id=notes-api
spring.security.oauth2.resourceserver.opaquetoken.client-secret=${INTROSPECTION_SECRET}
```

These properties make Boot create an `OpaqueTokenIntrospector` (classic: `SpringOpaqueTokenIntrospector`; Spring Security 7.1 adds `RestClientOpaqueTokenIntrospector`). To stay offline, our project supplies one answering from a map in the real endpoint's shape:

```
class FakeIntrospector implements OpaqueTokenIntrospector {
    private final Map<String, Map<String, Object>> activeTokens = Map.of(
            "tok-alice-7f3a", Map.of("active", true, "sub", "alice", "scope", "notes.read", "exp", Instant.now().plusSeconds(600)),
            "tok-bob-91c2", Map.of("active", true, "sub", "bob", "scope", "notes.read notes.write", "exp", Instant.now().plusSeconds(600)));

    @Override
    public OAuth2AuthenticatedPrincipal introspect(String token) {
        Map<String, Object> claims = activeTokens.get(token);
        if (claims == null) throw new BadOpaqueTokenException("Provided token isn't active"); // {"active":false}
        List<GrantedAuthority> authorities = Arrays.stream(((String) claims.get("scope")).split(" "))
                .map(s -> (GrantedAuthority) new SimpleGrantedAuthority("SCOPE_" + s)).toList();
        return new OAuth2IntrospectionAuthenticatedPrincipal((String) claims.get("sub"), claims, authorities);
    }
}

// chain for /opaque/**
http.securityMatcher("/opaque/**")
    .authorizeHttpRequests(a -> a
        .requestMatchers("/opaque/notes/write").hasAuthority("SCOPE_notes.write")
        .anyRequest().hasAuthority("SCOPE_notes.read"))
    .oauth2ResourceServer(rs -> rs.opaqueToken(o -> o.introspector(introspector)));
```

Output:

```
alice GET   -> 200 notes for alice (scope=notes.read)
alice write -> 403
bob write   -> 200 written by bob
inactive    -> 401 Bearer error="invalid_token", error_description="Provided token isn't active", ...
opaqueToken() -> 200 notes for carol (scope=notes.read)
```

The principal is an `OAuth2AuthenticatedPrincipal` (not a `Jwt`); the last line uses the `opaqueToken()` test post-processor.

JWT vs opaque

	JWT (self-contained)	Opaque + introspection
Validation	local, just a signature check	network call to the IdP per request (or per cache period)
Revocation	only when it expires (or via a deny-list)	**immediate**: the IdP answers `active: false`
Latency / IdP load	minimal	higher, and the IdP must be highly available
Privacy	claims readable by any holder	nothing readable
Typical use	APIs inside your platform	high-risk operations, tokens handed to third parties

A common hybrid is **opaque outside, JWT inside** ("phantom token"): the **API gateway** (single entry point routing to all services) introspects the opaque token and forwards a short-lived JWT internally.

TL;DR

Opaque tokens are checked with the IdP (RFC 7662): instant revocation and privacy, paid in latency and IdP load. `oauth2ResourceServer(rs -> rs.opaqueToken(...))`.

11.8 Multi-tenancy: several issuers in one API

What it is

A **multi-tenant** API serves several customers ("tenants"), each with its *own* IdP or **realm** (Keycloak: a separate set of users, clients and keys), hence its own issuer and keys. The API must verify each token with **that tenant's keys** and reject untrusted issuers.

`JwtIssuerAuthenticationManagerResolver` reads `iss` *without trusting it yet*, looks up an `AuthenticationManager` for that issuer, and lets it verify the signature with that issuer's keys:

```
Map<String, AuthenticationManager> managers = new ConcurrentHashMap<>();
JwtIssuerAuthenticationManagerResolver resolver = new JwtIssuerAuthenticationManagerResolver(issuer -> {
    if (!keys.isTrusted(issuer)) return null;         // unknown issuer -> 401
    return managers.computeIfAbsent(issuer, iss -> {
        NimbusJwtDecoder decoder = NimbusJwtDecoder.withPublicKey(keys.publicKey(iss)).build();
        decoder.setJwtValidator(JwtValidators.createDefaultWithIssuer(iss));
        return new ProviderManager(new JwtAuthenticationProvider(decoder));
    });
});
http.securityMatcher("/tenant/**")
    .authorizeHttpRequests(a -> a.anyRequest().authenticated())
    .oauth2ResourceServer(rs -> rs.authenticationManagerResolver(resolver));
```

Output:

```
tenant A token            -> 200 ann from https://idp.tenant-a.test
tenant B token            -> 200 ben from https://idp.tenant-b.test
B-signed, iss = tenant A  -> 401
untrusted issuer          -> 401
```

The third line matters: a token **signed by tenant B's key claiming tenant A** is rejected, because it was verified with A's key. With real IdPs publishing discovery metadata, one line suffices: `JwtIssuerAuthenticationManagerResolver.fromTrustedIssuers("https://idp.tenant-a.test", "https://idp.tenant-b.test")` (discovers each JWK Set on first use).

Common mistakes

- Building a decoder for **any** `iss` the token names (no allow-list) — an attacker runs their own IdP and names it.
- Forgetting tenant isolation *after* authentication — queries must still filter by tenant.

TL;DR

Several issuers → `JwtIssuerAuthenticationManagerResolver` with an allow-list. The `iss` claim chooses the keys, and the signature must match that issuer's key.

11.9 OAuth2 login: "Sign in with ..."

What it is

OAuth2 login lets users sign in to *your web app* with an IdP account (Google, GitHub, Keycloak, Entra ID). Your app is the **client** and never sees the password: it receives an **authorization code** (short-lived, single-use), exchanges it for tokens *server-side*, and creates a normal browser session. This is the **authorization-code flow**; with OIDC it also yields an ID token.

How it works



The protective parameters:

Parameter	Protects against	How
`state`	CSRF on the callback: an attacker making your browser finish *their* login	random value stored in the session and compared on return
`nonce` (OIDC; "number used once")	**replay** (an attacker re-sending a captured valid message) of an old ID token	random value that must come back inside the ID token
PKCE (`code_challenge` + `code_verifier`, RFC 7636)	a stolen authorization code	the app sends `SHA-256(verifier)` first; only the verifier's holder can redeem the code
exact `redirect_uri`	codes sent to an attacker's site	the IdP only redirects to pre-registered URIs
code exchange on the **back channel**	tokens exposed to the browser	tokens travel server-to-server, and the browser only gets a session cookie

Configuration

With a real OIDC provider, Boot needs just the client credentials and the issuer:

```
spring.security.oauth2.client.registration.demo-idp.client-id=notes-web
spring.security.oauth2.client.registration.demo-idp.client-secret=${DEMO_IDP_SECRET}
spring.security.oauth2.client.registration.demo-idp.scope=openid,profile,email
spring.security.oauth2.client.provider.demo-idp.issuer-uri=https://idp.example.test
```

Boot has provider defaults for `google`, `github`, `facebook` and `okta` (registration id is enough). Our project spells out every endpoint instead of `issuer-uri`, so it starts offline:

```
spring.security.oauth2.client.registration.demo-idp.authorization-grant-type=authorization_code
spring.security.oauth2.client.registration.demo-idp.redirect-uri={baseUrl}/login/oauth2/code/{registrationId}
spring.security.oauth2.client.provider.demo-idp.authorization-uri=https://idp.example.test/oauth2/authorize
spring.security.oauth2.client.provider.demo-idp.token-uri=https://idp.example.test/oauth2/token
spring.security.oauth2.client.provider.demo-idp.jwk-set-uri=https://idp.example.test/oauth2/jwks
spring.security.oauth2.client.provider.demo-idp.user-info-uri=https://idp.example.test/userinfo
spring.security.oauth2.client.provider.demo-idp.user-name-attribute=sub
```

```
http.authorizeHttpRequests(a -> a.requestMatchers("/", "/public/**").permitAll().anyRequest().authenticated())
    .oauth2Login(Customizer.withDefaults())
    .logout(Customizer.withDefaults());

@GetMapping("/profile")
String profile(@AuthenticationPrincipal OidcUser user) {
    return "hello " + user.getFullName() + " <" + user.getEmail() + ">, sub=" + user.getSubject()
           + ", issuer=" + user.getIssuer();
}
```

What we can run without an IdP

Steps 1–4 (everything before the browser leaves our app) and the callback's `state` check run for real; the IdP sign-in and code exchange cannot run without an IdP, so `oidcLogin()` simulates a finished login:

```
1. GET /profile (anonymous)       -> 302 /oauth2/authorization/demo-idp
2. GET /oauth2/authorization/demo-idp -> 302 to https://idp.example.test/oauth2/authorize
     client_id=notes-web
     code_challenge=<random>
     code_challenge_method=S256
     nonce=<random>
     redirect_uri=http://localhost/login/oauth2/code/demo-idp
     response_type=code
     scope=openid%20profile%20email
     state=<random>
   session attribute saved: true
3. callback with forged state      -> 302 /login?error
oidcLogin() GET /profile -> 200 hello Jane Doe <jane@example.com>, sub=248289761001, issuer=https://idp.example.test
```

- The request has `state`, `nonce` **and PKCE** (`S256`) although this client has a secret: Spring Security 7 uses PKCE for **confidential clients** too (server-side clients that can keep a secret; browser/mobile apps are *public* clients). For an old IdP that can't handle it: `.clientSettings(ClientRegistration.ClientSettings.builder().requireProofKey(false).build())`.
- A callback with a `state` never issued to this session is refused (`/login?error`) — the login flow's CSRF protection.
- After a real login the principal is an `OidcUser` (ID-token claims + userinfo); tokens are stored server-side as an `OAuth2AuthorizedClient`, the browser holds only the session cookie. OAuth2 login is **session-based**: keep CSRF on.

OAuth2 vs OIDC login

With the `openid` scope Spring does **OIDC** login (validates the ID token, builds an `OidcUser`). Without it (e.g. GitHub, plain OAuth2) it calls the user-info endpoint and builds an `OAuth2User`. Mapping external users to *your* accounts (first login = sign-up, linking by verified e-mail, local roles) is your job: a custom `OidcUserService` / `OAuth2UserService`, or a listener for the login success event.

TL;DR

`oauth2Login()` + a client registration = "Sign in with X" via authorization code, `state`, `nonce` and PKCE. Tokens stay on the server, and the browser gets a session. Test with `oidcLogin()` / `oauth2Login()`.

11.10 Calling other services: client credentials and token relay

What it is

Calling *another* protected API has two cases:

Case	Whose token?	Grant
a nightly job calls the reports API	the **service's own** identity	**client credentials**: client id + secret (or private-key JWT / mTLS, mutual TLS) → access token
a user's request fans out to **downstream** APIs	the **user's** identity	**token relay**: forward the user's token, or better, exchange it (RFC 8693) for one aimed at the downstream API

How it looks in Spring (not run: needs a token endpoint)

Configure a registration and let `OAuth2AuthorizedClientManager` fetch, cache and renew tokens:

```
spring.security.oauth2.client.registration.reports.provider=demo-idp
spring.security.oauth2.client.registration.reports.client-id=nightly-job
spring.security.oauth2.client.registration.reports.client-secret=${REPORTS_SECRET}
spring.security.oauth2.client.registration.reports.authorization-grant-type=client_credentials
spring.security.oauth2.client.registration.reports.scope=reports.read
```

```
// RestClient (Chapter 6) with an interceptor that adds "Authorization: Bearer <token>"
@Bean
RestClient reportsClient(RestClient.Builder builder, OAuth2AuthorizedClientManager manager) {
    return builder.baseUrl("https://reports.internal")
            .requestInterceptor(new OAuth2ClientHttpRequestInterceptor(manager))
            .build();
}

String csv = reportsClient.get().uri("/daily")
        .attributes(clientRegistrationId("reports"))   // which registration's token to use
        .retrieve().body(String.class);
```

With **HTTP interfaces** (Chapter 6), Spring Security 7 allows `@ClientRegistrationId` on the whole interface; register an `OAuth2RestClientHttpServiceGroupConfigurer` bean so the clients get the interceptor:

```
@HttpExchange("/reports")
@ClientRegistrationId("reports")            // type-level support is new in Spring Security 7
interface ReportsApi {
    @GetExchange("/daily") String daily();
}

@Bean
OAuth2RestClientHttpServiceGroupConfigurer securityConfigurer(OAuth2AuthorizedClientManager manager) {
    return OAuth2RestClientHttpServiceGroupConfigurer.from(manager);
}
```

The **interceptor** asks the manager for a token on each outgoing request; the manager returns the cached one while valid and requests or refreshes when needed — you never parse expiry yourself.

Token relay. In a resource server the incoming token is `JwtAuthenticationToken#getToken().getTokenValue()`; an interceptor can copy it to outgoing calls. In **Spring Cloud Gateway** the `TokenRelay` filter does this for logged-in users (Chapter 16). Forwarding one token everywhere means every service accepts the same audience; **token exchange** (Spring Security OAuth2 client since 6.3) gets a narrower token per downstream API.

TL;DR

Services call APIs with **client credentials**, via `OAuth2AuthorizedClientManager` + `OAuth2ClientHttpRequestInterceptor` on `RestClient` (or `@ClientRegistrationId` on HTTP interfaces). User context flows by token relay or, better, token exchange.

11.11 Refresh tokens, lifetimes, revocation and logout

What it is

Users shouldn't log in every 10 minutes. A **refresh token** is a long-lived credential the client sends **only to the authorization server** to get a new access token without user interaction.

Practice	Why
access token 5–15 min, refresh token hours/days	a stolen access token is soon useless
refresh-token rotation: every use returns a *new* refresh token and invalidates the old one	a stolen refresh token used twice is detected, and the IdP revokes the whole family
bind refresh tokens to the client (secret, DPoP — proof of holding a private key — or mTLS)	a stolen token alone is not enough
never give refresh tokens to browser JavaScript	XSS could steal a long-lived credential (§11.12)

In Spring the resource server never sees refresh tokens; the client-side `OAuth2AuthorizedClientManager` (behind `oauth2Login` and the interceptor) refreshes expired access tokens automatically.

Revocation and logout

- **Revoking** (RFC 7009) tells the IdP a token is dead: opaque tokens stop at once; JWTs work until `exp` unless APIs check a `jti` deny-list (rarely worth it with short lifetimes).
- **Logging out** of your app (`POST /logout`) ends *your* session, not the IdP's — the next "Sign in with ..." logs straight back in. For single logout use **OIDC RP-initiated logout** (RP = relying party, the app relying on the IdP) to also redirect to the IdP's `end_session_endpoint`:

```
http.logout(l -> l.logoutSuccessHandler(new OidcClientInitiatedLogoutSuccessHandler(clientRegistrationRepository)));
```

OIDC back-channel logout (`http.oidcLogout(...)`) lets the IdP call your app to end sessions when the user logs out elsewhere.

TL;DR

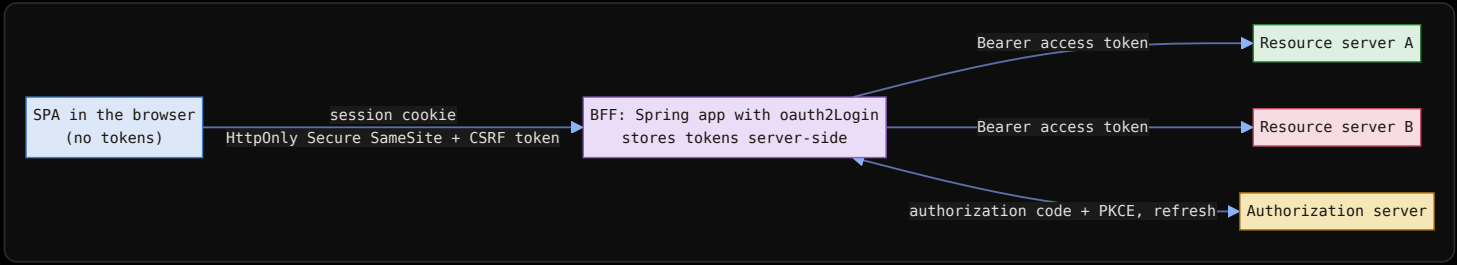
Short access tokens + rotating refresh tokens, handled by the client-side `OAuth2AuthorizedClientManager`. Revocation is instant only for opaque tokens. Use OIDC RP-initiated logout to end the IdP session too.

11.12 Where should a browser keep tokens? The BFF pattern

The problem it solves

SPA tokens in `localStorage` / `sessionStorage` are readable by **any script on the page**, so one XSS bug leaks them (JS 14.3, JS 14.10). In-memory storage is lost on reload and still reachable by injected script.

The recommended answer: Backend for Frontend (BFF)



1. The SPA talks only to its **BFF** on the same site, with an `HttpOnly` session cookie JavaScript cannot read.
2. The BFF is a confidential OAuth2 client (`oauth2Login`, §11.9) keeping and refreshing tokens server-side.
3. It forwards API calls with the bearer token (token relay, §11.10); Spring Cloud Gateway with `TokenRelay` is a common BFF.
4. The SPA now uses cookies, so **CSRF protection is required** — `csrf.spa()` (10.8).

The IETF draft *OAuth 2.0 for Browser-Based Applications* recommends this as the most secure option. See also the BFF pattern notes.

Where tokens live	XSS steals tokens?	CSRF risk	Complexity
`localStorage`	**yes**	no	low
JS memory only	harder, still possible while the page runs	no	low, but lost on reload
BFF (`HttpOnly` cookie session)	no, the tokens never reach the browser	yes, so use CSRF tokens	an extra backend

TL;DR

No long-lived tokens in browser JavaScript: a BFF keeps them server-side; the browser gets an `HttpOnly` session cookie plus CSRF protection.

11.13 Storing keys and secrets

Passwords: 10.5. Token systems add more secrets:

Secret	Where it lives	Rules
JWT signing private key	the authorization server only: vault, KMS (key management service) or HSM (hardware security module)	never in Git or a resource server; resource servers get *public* keys from the JWKS URL
Client secret	environment variable / secret manager, injected as `${...}`	rotate it; prefer `private_key_jwt` or mTLS for high-value clients
Refresh tokens stored by a BFF	server-side session or database, encrypted at rest	never sent to the browser
HMAC secret (if you use HS256)	secret manager	at least 256 bits of randomness. Everyone who can verify can also forge

Key rotation without downtime:

1. Publish the *new* public key in the JWKS next to the old one (a different `kid`).
2. Wait longer than the verifiers' JWKS cache time. `NimbusJwtDecoder` caches the JWK Set and re-fetches it when it meets an unknown `kid`.
3. Start signing with the new key.
4. Remove the old key after the longest token lifetime has passed.

Our demo generates a fresh key pair per start: tokens die on restart and several instances would disagree — fine for tests, wrong for production.

11.14 Spring Authorization Server

If you need your own IdP, **Spring Authorization Server** implements OAuth 2.1 and OpenID Connect 1.0: authorization code + PKCE, client credentials, refresh tokens, device authorization, token exchange, introspection, revocation, JWKS and discovery endpoints, dynamic client registration. Since **Spring Security 7.0 it ships inside Spring Security** (module `spring-security-oauth2-authorization-server`); Boot 4 provides `spring-boot-starter-security-oauth2-authorization-server`, configured under `spring.security.oauth2.authorizationserver.*`. PKCE is required by default for authorization-code clients.

Choose it to keep the IdP in your Spring codebase, customised in Java; choose Keycloak or a cloud IdP for an off-the-shelf product with admin UI, federation and user management. Either way *your APIs* look the same (resource servers with an `issuer-uri`). It needs a user store, persistent client/authorization storage (JDBC) and key management, so it is beyond this chapter's runnable examples.

11.15 Rate limiting and brute-force protection

Spring Security does **not** include **rate limiting** (capping requests per client per time window). Yet login and token endpoints are what attackers hammer: password guessing, **credential stuffing** (pairs leaked from other sites), and making you pay for slow bcrypt/Argon2 checks (10.5). Layers:

Layer	Tool	Limits
edge	API gateway / WAF (web application firewall) / CDN, Spring Cloud Gateway `RequestRateLimiter` (Redis, token bucket)	requests per IP / client / API key
application	Bucket4j (token bucket, local or distributed via Redis/JCache) in a servlet filter	per user or per endpoint rules
login logic	count failures per account *and* per IP (listen to `AuthenticationFailureBadCredentialsEvent`), add delays, CAPTCHA or MFA after N failures	brute force on one account
identity provider	Keycloak/cloud IdPs have brute-force detection built in	when login happens at the IdP

Avoid hard **account lockout** — it lets anyone lock out any user; prefer progressive delays and step-up checks (an extra factor only when something looks risky). Framework 7's `@ConcurrencyLimit` (Chapter 4) caps *concurrent* executions, not requests per second — not a rate limiter. Algorithms: System Design Patterns — Rate Limiting.

11.16 Common mistakes (whole chapter)

Mistake	Consequence	Fix
no audience check	tokens for *other* APIs accepted	`jwt.audiences` property or `JwtClaimValidator("aud", ...)`
custom validator *replacing* defaults	expired tokens accepted	combine with `JwtValidators.createDefaultWithIssuer(...)`
`hasAuthority("messages.read")`	always 403	scopes are `SCOPE_messages.read`
`jwt()` tests without your converter	tests disagree with production	`.authorities(yourConverter)` or real tokens
long-lived JWT access tokens	stolen token valid for days	minutes + refresh tokens, or opaque tokens
tokens in `localStorage`	XSS = account takeover	BFF with `HttpOnly` cookie
accepting any `iss` in multi-tenant setups	attacker-run IdP accepted	allow-list of issuers
CSRF disabled on the BFF / OAuth2-login app	CSRF on a cookie session	keep CSRF on (`csrf.spa()`)
ID token sent to APIs	wrong audience, identity confusion	send the **access** token
rolling your own token server for real users	endless security work	use an authorization server
private key in `application.properties` / Git	anyone can mint tokens	vault/KMS, JWKS for public keys

11.17 In the interview

"How does a Spring resource server validate a JWT?" "`BearerTokenAuthenticationFilter` extracts the token from the `Authorization` header. `JwtAuthenticationProvider` calls the `JwtDecoder`, which Nimbus backs with the issuer's JWK Set found via `issuer-uri` discovery. It checks the algorithm, picks the key by `kid`, and verifies the signature. Then validators check `exp` and `nbf` with 60 seconds of skew, `iss`, `typ`, and the audience I configure. `JwtAuthenticationConverter` maps `scope` to `SCOPE_` authorities, plus any role claims I map. A failure returns 401 `invalid_token`, and a missing authority returns 403 `insufficient_scope`."

Follow-ups:

- *JWT or opaque token?* → JWT for fast local validation inside the platform; opaque for instant revocation or hidden claims, at the cost of introspection.
- *Why PKCE if the client has a secret?* → It binds the code to the party that started the flow, so a stolen code is useless. OAuth 2.1 requires it for all clients; Spring Security 7 sends it by default.
- *Logout with stateless tokens?* → Short access tokens, revoke the refresh token, end the local session, OIDC RP-initiated logout for the IdP session.
- *Tokens in an SPA?* → Preferably none: a BFF with an `HttpOnly` cookie and CSRF protection.
- *Service-to-service auth?* → Client credentials via `OAuth2AuthorizedClientManager` + `OAuth2ClientHttpRequestInterceptor`, or mTLS in a mesh; relay or exchange the user's token when user context matters.
- *What is `state` for?* → CSRF protection of the login callback: ties the response to the session that started the flow.

11.18 Security fundamentals beyond Spring

What it is

Everything above rests on a few cryptographic building blocks — hashes, symmetric and asymmetric keys, signatures, key exchange — which Spring hides behind `PasswordEncoder`, `JwtDecoder` and `https://` URLs. Each is shown with the plain JDK in `code/crypto-basics`, followed by the protocols built from them: TLS, SSH keys, one-time codes, passkeys, single sign-on.

Analogy: a **hash** is a fingerprint (identifies, cannot be turned back); **symmetric encryption** a padlock both sides hold keys to; **asymmetric encryption** a mailbox anyone can post into but only the owner opens; a **signature** a wax seal only one ring makes but anyone can check.

Encoding vs hashing vs encryption vs tokenisation

	Purpose	Key?	Reversible?	Example	Wrong use
Encoding	change representation so bytes survive a channel	no	yes, by anyone	Base64, URL encoding, UTF-8	"hiding" a password in Base64
Hashing	fixed-size fingerprint; detect change; store passwords	no (HMAC adds one)	**no**	SHA-256, BLAKE3; for passwords bcrypt/Argon2 (10.5)	fast SHA-256 for passwords (GPU-crackable)
Encryption	confidentiality: only key holders can read	yes	yes, with the key	AES-GCM, ChaCha20-Poly1305, RSA-OAEP	encrypting passwords (then someone can decrypt them)
Tokenisation	replace sensitive data with a random stand-in; the real value lives in a secured vault	vault lookup	only via the vault	card number → `tok_8f3a…` in payment systems	—

An **HMAC** (hash-based message authentication code) is a hash mixed with a secret key: only key holders can compute or check it. HS256 JWTs and TOTP codes (below) are HMACs.

Symmetric vs asymmetric encryption

	Symmetric	Asymmetric (public-key)
Keys	one shared secret key	a key pair: **public** (share freely) + **private** (never leaves the owner)
Speed	very fast (AES has CPU instructions)	100–1000× slower; small data only
Problem	how do both sides get the key safely?	who does this public key really belong to?
Algorithms	AES-GCM, ChaCha20-Poly1305	RSA, elliptic curves (ECDH/X25519 for key exchange, ECDSA/Ed25519 for signatures)

Real systems are **hybrid**: asymmetric crypto agrees or protects a random symmetric key; bulk data goes through AES (Advanced Encryption Standard). **Authenticated encryption** modes (GCM, Poly1305) add a tag, so tampering makes decryption fail rather than return garbage (the demo proves it).

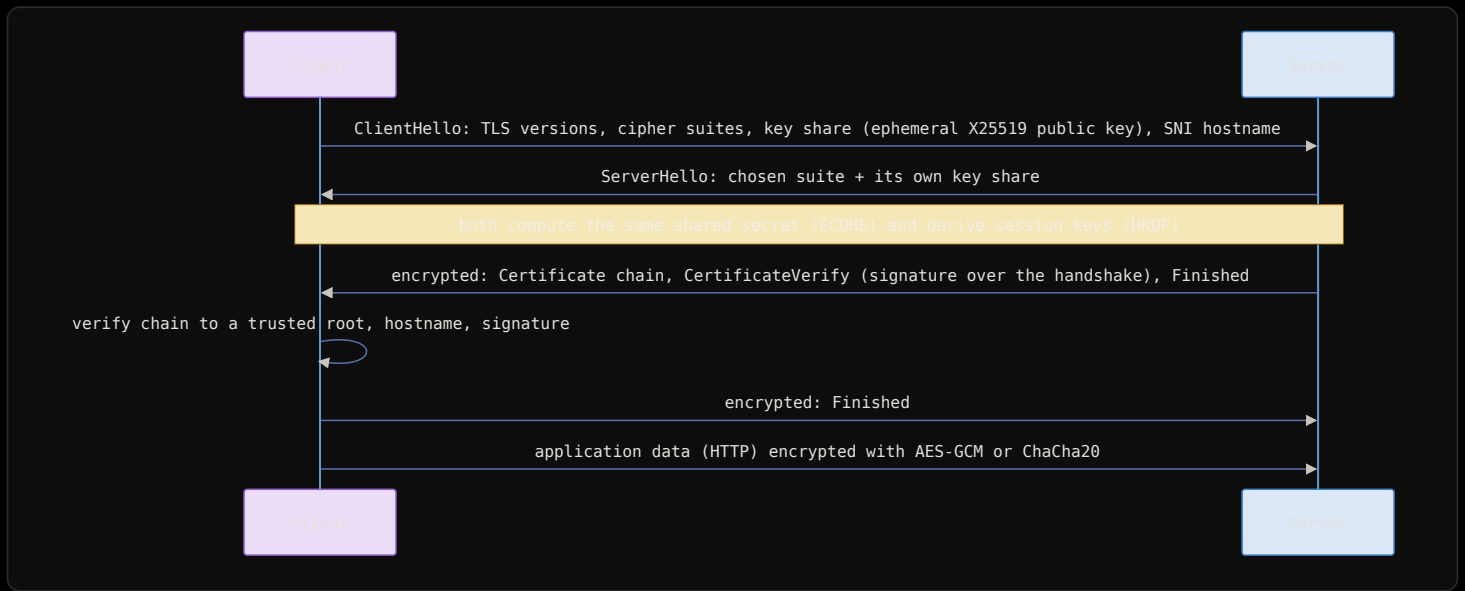
Digital signatures and certificates

A **digital signature** is computed with the private key over a hash of the message; anyone with the public key verifies that (1) that key's holder signed it and (2) not one bit changed — how an RS256/ES256 JWT is trusted (§11.1).

A **certificate** (X.509) is a signed statement "this public key belongs to `api.example.com`", signed by a **certificate authority (CA)** that browsers and OSes trust. Browsers and the JDK ship trusted root CAs; the server sends its certificate plus intermediates, and the client checks the **chain** up to a trusted root, the name, and the validity dates.

The TLS 1.3 handshake

HTML book §1.4 says what TLS gives (confidentiality, integrity, server authentication). Here is how, in one round trip (RFC 8446):



Key ideas: the **key exchange** (ECDHE) gives both sides a secret nobody on the wire can compute; the `CertificateVerify` **signature** proves the server owns the certificate's private key; keys are **ephemeral**, so recording traffic now and stealing the server key later reveals nothing (**forward secrecy** — the long-term key only signs). **mTLS** adds a client certificate so the server

authenticates the client too — common between services (§11.10). Spring Boot configures TLS with SSL bundles (`server.ssl.bundle`), though it is usually terminated at a load balancer or Kubernetes ingress (System Design Fundamentals).

SSH keys

SSH logs you into servers and Git hosts. `ssh-keygen -t ed25519` creates a key pair; the **public** key goes in the server's `~/.ssh/authorized_keys` (or GitHub settings). On login the server sends a challenge, the client **signs** it with the private key, the server verifies — no password crosses the wire. On first connect SSH shows the **server's** host-key fingerprint; accepting stores it in `known_hosts`, and a later mismatch warns of a possible **man-in-the-middle** (an attacker between you and the server). Protect the private key with a passphrase and `ssh-agent`; never copy it between machines.

TOTP: the six digits in an authenticator app

TOTP (time-based one-time password, RFC 6238) builds on **HOTP** (RFC 4226):

1. At enrolment the server generates a random secret and shows it as a QR code (`otpauth://totp/...?secret=BASE32...`); the app stores it. Both sides now share a secret.
2. Every 30 seconds both compute `counter = floor(unixTime / 30)`, then `HMAC-SHA1(secret, counter)`.
3. **Dynamic truncation**: the last 4 bits of the HMAC pick an offset; 31 bits from there, `mod 10^6`, give six digits.
4. The server accepts the current step and usually ±1 step for clock drift, and should remember used codes against replay.

`Totp.java` implements it in 30 lines; `TotpTest` checks every RFC 6238 Appendix B test vector (SHA-1, SHA-256 and SHA-512) and the RFC 4226 HOTP vectors.

```
byte[] h = mac.doFinal(ByteBuffer.allocate(8).putLong(counter).array()); // HMAC(secret, 8-byte counter)
int offset = h[h.length - 1] & 0x0f;                                     // dynamic truncation
int binary = ((h[offset] & 0x7f) << 24) | ((h[offset + 1] & 0xff) << 16)
           | ((h[offset + 2] & 0xff) << 8) | (h[offset + 3] & 0xff);
return String.format("%0" + digits + "d", binary % (int) Math.pow(10, digits));
```

TOTP is phishable (a fake look-alike site can ask for the code and replay it within 30 s) and relies on a server-stored shared secret — what passkeys fix.

Passkeys (WebAuthn / FIDO2)

A **passkey** is a key pair created by your device (phone, laptop, security key) **for one website**. Registration: the browser's WebAuthn API asks the device's **authenticator** to create a key pair scoped to the site's domain (the **relying party ID**); the site stores only the public key (plus a credential ID). Login: the site sends a random challenge; the authenticator checks you (fingerprint, face, PIN), **signs** the challenge together with the **origin** (scheme + domain + port), and the site verifies with the stored public key.

Stronger than passwords + TOTP: nothing secret on the server (a leak yields only public keys), and the signature covers the origin, so a look-alike phishing domain gets a useless signature. Synced passkeys (iCloud Keychain, Google Password Manager) survive losing a phone. Spring Security supports them since 6.4: `http.webAuthn(w -> w.rpName("Shop").rpId("shop.example.com").allowedOrigins("https://shop.example.com"))` (module `spring-security-webauthn` in 7.x), recording a `FACTOR_WEBAUTHN` authority for multi-factor rules (10.4).

Single sign-on: SAML vs OpenID Connect

SSO means logging in once at an identity provider (IdP) and being let into many applications (service providers / relying parties) without new passwords.

	SAML 2.0 (2005)	**OpenID Connect (2014, on OAuth 2.0)**
Token	signed **XML assertion**	signed **JWT** ID token (§11.2)
Transport	browser POSTs/redirects the assertion to the app	authorization-code flow + PKCE, back-channel token call
Fits	enterprise web apps, older corporate IdPs (AD FS)	web, SPA/BFF, mobile, APIs; also gives access tokens
Spring	`saml2Login()` (`spring-security-saml2-service-provider`)	`oauth2Login()` (§11.9)

Both rely on the IdP **signing** a statement about the user, verified with its published key or certificate. Pick OIDC for new systems; add SAML when an enterprise customer's IdP only speaks SAML (Keycloak, Okta and Entra ID speak both and can bridge).

Seeing it: `code/crypto-basics`

Plain JDK 25, no Spring. `mvn -q -B -ntp verify` runs 13 JUnit tests (RFC vectors, AES-GCM tamper detection, signatures, X25519 agreement); `java -cp target/classes handbook.security.CryptoBasicsDemo` prints:

```
encode  Base64      : cGF5IGJvYiAxMDA=  -> decodes back to: pay bob 100
hash    SHA-256     : aa843d8373c66e26...  ('pay bob 900' -> 70ef8a2487a6d483...)
encrypt AES-256-GCM : 27 bytes (11 plaintext + 16 auth tag) -> decrypts to: pay bob 100
encrypt tampered    : AEADBadTagException (integrity check failed)
sign    Ed25519     : 64-byte signature; verify original = true, verify 'pay bob 900' = false
ECDHE   X25519      : both sides derived the same 32-byte secret = true (e4582f7aff4d...)
ECDHE   record      : server decrypts client's record: GET /
TOTP    t=59          : counter=1         code=94287082
TOTP    t=1111111109  : counter=37037036  code=07081804
TOTP    t=1234567890  : counter=41152263  code=89005924
TOTP    t=2000000000  : counter=66666666  code=69279037
TOTP    6-digit app : 921300 valid 25 s later = true, valid 5 min later = false
```

The four `TOTP t=` codes are exactly the SHA-1 column of RFC 6238 Appendix B. Keys and the X25519 secret are random on every run.

Common mistakes

- Calling Base64 "encryption" — it is encoding.
- Hashing passwords with plain SHA-256 or MD5 — use bcrypt, Argon2, PBKDF2.
- Reusing an AES-GCM IV (initialisation vector: a per-message nonce) with the same key — breaks confidentiality and integrity; use a random 12-byte IV per message.
- Rolling your own crypto or protocol — use the JDK, TLS and vetted libraries; your job is parameters and key management.
- Disabling certificate checks ("trust all") to pass a test — removes TLS's *authentication* half in production too.
- Comparing codes or MACs with `equals` — use constant-time `MessageDigest.isEqual` to avoid timing leaks.

In the interview

30-second answer — how HTTPS works: "TLS 1.3 does an ephemeral Diffie-Hellman key exchange (two parties agreeing a shared secret over an open network) in the first round trip, so both derive the same session keys without sending them. The server proves its identity by signing the handshake with its certificate's private key; the client checks the chain up to a trusted CA and the hostname. Then data is encrypted and integrity-protected with a fast symmetric cipher such as AES-GCM. Ephemeral keys give forward secrecy."

- *Hashing vs encryption?* → Hashing is one-way and keyless (fingerprints, passwords); encryption is reversible with a key (confidentiality).
- *Why both symmetric and asymmetric?* → Asymmetric solves key distribution and identity but is slow; symmetric is fast. TLS uses the first to set up the second.
- *Authenticator app offline?* → Shared secret + current time → HMAC → truncation (TOTP); ±1 step tolerates clock drift.
- *Why are passkeys phishing-resistant?* → The signature is bound to the site's origin and the server holds only a public key.
- *SAML vs OIDC?* → Both SSO via an IdP-signed statement; SAML: XML assertions via browser POST; OIDC: JWT ID tokens over OAuth 2.0, suits mobile and APIs.
- *What protects a JWT's claims from change, and what would hide them?* → A signature (JWS) / encryption (JWE).

TL;DR

Encoding is keyless and reversible, hashing one-way, encryption keyed, tokenisation a vault reference. Symmetric = one fast shared key; asymmetric = public/private pair for key exchange and signatures; TLS 1.3 combines them (ECDHE + certificate signature + AES-GCM). SSH and passkeys sign a challenge; TOTP = HMAC(secret, time/30) truncated to six digits. SSO: SAML (XML) or OIDC (JWT) — prefer OIDC.

Sources: RFC 6238 (TOTP), RFC 4226 (HOTP), RFC 8446 (TLS 1.3), RFC 4251/4252 (SSH), RFC 7748 (X25519), RFC 8032 (Ed25519), NIST SP 800-38D (GCM), W3C Web Authentication Level 2, OASIS SAML 2.0 Core, OpenID Connect Core 1.0, Spring Security reference (Passkeys, SAML 2.0 Login), JDK 25 `java.security` / `javax.crypto` API docs.

Chapter 11 TL;DR

- **Tokens**: JWT = `header.payload.signature`; claims readable, signature makes them trustworthy. Prefer asymmetric keys (RS256/ES256) published as a JWKS.
- **OAuth 2.0/OIDC**: authorization code + PKCE for users, client credentials for services, refresh tokens for renewal. Implicit and password grants are gone (Spring Security 7 removed password-grant support).
- **Resource server**: `oauth2ResourceServer(jwt)` + `issuer-uri`; validates signature, `exp` / `nbf` (60 s skew), `iss`, `typ`, your `aud`. 401 `invalid_token` vs 403 `insufficient_scope`; publishes RFC 9728 metadata.
- **Authorities**: `scope` → `SCOPE_x`; map role claims with a converter or Boot properties.
- **Testing**: real tokens from a local `NimbusJwtEncoder` for decoder rules; `jwt()` / `opaqueToken()` / `oidcLogin()` for access rules — pass your converter to `jwt()`.
- **Variants**: opaque tokens via introspection (instant revocation); multi-tenant via `JwtIssuerAuthenticationManagerResolver` with an issuer allow-list.

- **Clients**: `oauth2Login` (state, nonce, PKCE, tokens server-side); `OAuth2AuthorizedClientManager` + `RestClient` interceptor or `@ClientRegistrationId` HTTP interfaces; token relay or exchange.
- **Browsers**: BFF + `HttpOnly` cookie + CSRF; no tokens in `localStorage`.
- **Operations**: keys in a vault, rotated via `kid`; Spring Authorization Server (inside Spring Security 7) if you must run an IdP; rate limiting at the edge.
- **Fundamentals** (§11.18): encoding ≠ hashing ≠ encryption; TLS 1.3 = asymmetric key exchange + signatures + AES-GCM; SSH keys and passkeys sign a challenge; TOTP = HMAC(secret, time/30) truncated; SSO via SAML or OIDC.

Next: Chapter 12 — Testing Spring apps covers the test slices used here in depth.

Chapter 12 — Testing Spring Applications

A Spring application is plain Java objects plus a **container** (the *application context*: it creates your **beans** — the objects Spring manages — and wires them together), which adds **proxies** (wrappers adding behaviour such as transactions) and talks to HTTP, databases and other services. Tests should match that shape: most logic is tested **without Spring at all**; things that exist only inside the container — URL mappings, JSON conversion, validation, Spring Data SQL, **transactions** (changes that succeed or fail together), security filters — get **just enough of the container**; a few tests start **everything** and send real HTTP. This chapter covers each with Spring Boot 4's test support, keeping the **suite** (all tests of a project) fast, and proves every behaviour with real output.

All code is in `code/`: Spring Boot **4.1.1** (Framework 7.0, JUnit 6.0 — the Java test runner, Mockito 5 — fake objects, AssertJ 3 — readable `assertThat(...)` checks), Java 25:

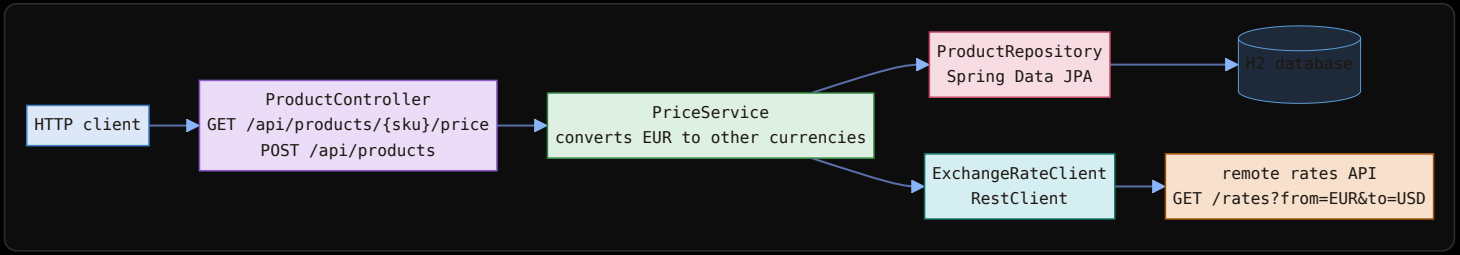
Project	Shows	Run here?
`testing-demo`	a small product-pricing API tested at every level: unit, slices, full context, real port, plus a seeded fuzz test	yes
`context-caching`	seven test classes that hit or miss the context cache, with timings	yes
`testcontainers-demo`	PostgreSQL in a container via `@ServiceConnection`	**needs Docker — not run** (compiled only)

Output blocks are real `mvn -q -B -ntp verify` output, Spring log lines trimmed to `...`. **Timings are machine-dependent** (14-core Apple M4 Pro laptop); compare them only with each other.

> **Prerequisites.** JUnit 5/6, Mockito and AssertJ basics: Java handbook Chapter 18; this chapter covers what Spring adds. Code under test uses Spring MVC (Chapter 5), `RestClient` (Chapter 6) and Spring Data JPA (Chapter 8). Security testing (`@WithMockUser`, `jwt()`): Chapter 10, Chapter 11. Reactive testing (`StepVerifier`, `WebTestClient`): Chapter 13.

12.1 The application under test

`testing-demo` is a tiny catalogue service with the layers a real service has:



- `ProductController` maps HTTP to Java, validates the body (`@Valid`) and turns `ProductNotFoundException` into a `404` Problem Detail (standard JSON error shape).
- `PriceService` holds the rule: prices are stored in EUR; for another currency fetch the rate and round to 2 decimals (`HALF_EVEN`, "banker's rounding": a 5 rounds to the even neighbour). For EUR it must **not** call the remote API.
- `ProductRepository` is a Spring Data JPA interface with a **derived query** (`findBySku`: SQL generated from the method name) and a **JPQL** query (`cheaperThan`: JPA's SQL-like language over entity classes).
- `ExchangeRateClient` calls the remote API with `RestClient`; base URL from `rates.base-url`.

The service, for reference:

```
@Service
public class PriceService {
    private final ProductRepository products;
    private final ExchangeRateClient rates;
    private final String home;

    public PriceService(ProductRepository products, ExchangeRateClient rates,
                        @Value("${catalog.home-currency}") String home) { ... }

    public PriceView priceOf(String sku, String currency) {
        Product p = products.findBySku(sku).orElseThrow(() -> new ProductNotFoundException(sku));
        BigDecimal amount = currency.equals(home)
                ? p.getPrice()                                  // no remote call needed
                : convert(p.getPrice(), rates.rate(home, currency));
        return new PriceView(p.getSku(), p.getName(), amount, currency);
    }
    ...
}
```

12.2 The test pyramid in a Spring app

What it is

The **test pyramid**: many small, fast tests at the bottom, fewer slower, broader tests near the top. In Spring the layers map to how much of the container a test starts:

Layer	What starts	Spring annotation	Proves	Speed (this project)
Unit	nothing — `new` your class, mock its collaborators	none (`@ExtendWith(MockitoExtension.class)`)	business logic, edge cases	~1 ms or less per test
Slice	one layer of the container (web, JPA, JSON, HTTP client)	`@WebMvcTest`, `@DataJpaTest`, `@JsonTest`, `@RestClientTest`, ...	mappings, validation, JSON shape, SQL, client parsing	context 0.03–0.14 s once warm (1.6 s for the first, cold JVM still loading classes)
Integration	the whole application context (mock servlet, no port)	`@SpringBootTest` + `@AutoConfigureMockMvc`	the wiring: all real beans together	context ~0.7 s
Full stack / end-to-end	whole context + real server on a random port + real HTTP	`@SpringBootTest(webEnvironment = RANDOM_PORT)`	the app as a client sees it	context ~0.3–0.7 s, plus real network calls

Timings are Boot's `Started <TestClass> in N seconds` lines for each new context in this run:

```
... Started ProductRepositoryDataJpaTest in 1.574 seconds (process running for 2.205)   <- first context: cold JVM
... Started MockMvcFullContextTest in 0.726 seconds
... Started PriceViewJsonTest in 0.053 seconds
... Started ExchangeRateClientTest in 0.137 seconds
... Started ProductControllerWebMvcTest in 0.099 seconds
... Started PropertyPrecedenceTest in 0.034 seconds
... Started FullStackTest in 0.285 seconds
... Started RestTestClientMockMvcTest in 0.052 seconds
unit test: PriceView[sku=MUG-001, name=Mug, amount=10.85, currency=USD] in 0 ms
```

On a toy app the gap looks small; on a real service (400 beans, Flyway migrations, Kafka, a security chain) a full context takes seconds, and 300 test classes each building one take many minutes. The pyramid plus context caching (§12.11) keeps a Spring build fast.

Why it matters

- **Fast feedback**: millisecond unit tests run on every change.
- **Precise failures**: a failing unit/slice test points at one class; a failing end-to-end test only says "something is broken".
- **Confidence where it counts**: only an integration test catches a missing bean, a wrong property name, or two beans of the same type.

How to decide which layer

1. Pure logic (calculations, rules, mapping)? → **unit test**, no Spring.
2. Depends on something only Spring does (URL mapping, `@Valid`, JSON, a JPQL query, a `@Transactional` boundary)? → that layer's **slice** (e.g. `POST /api/products` returning `400` for a bad SKU → `@WebMvcTest`; `cheaperThan` sorting → `@DataJpaTest` on a real embedded database, one running inside the test JVM such as H2).
3. Proves beans work *together*? → `@SpringBootTest`, usually on the mock servlet environment.
4. Depends on a real server (filters in front of the servlet, compression, the port, SSL/TLS)? → `RANDOM_PORT`.

Trade-offs

	Unit	Slice	`@SpringBootTest`	`RANDOM_PORT`
Speed	fastest	fast	slower	slowest
Realism	lowest (mocks everywhere)	real for one layer	real wiring	real HTTP
Failure points at	one class	one layer	"somewhere in the app"	"somewhere in the system"
Typical share of a suite	most	many	some	few

Common mistakes

- **Only `@SpringBootTest`**: every test starts the whole app, and each config difference forces a new context — the most common cause of 20-minute builds.
- **Mocking everything in integration tests**: then they prove nothing the unit test didn't.
- **Testing the framework**: no need to prove `JpaRepository.save` saves. Test *your* queries, mappings and rules.

TL;DR

Most tests should be plain unit tests. Use a slice when the behaviour lives in Spring. Use a few `@SpringBootTest`s to prove the wiring. Use even fewer `RANDOM_PORT` tests.

12.3 Test dependencies in Boot 4

What changed

Boot 4 split the monolithic **auto-configuration** (beans Boot sets up based on the classpath) into **modules**, one per technology (Chapter 3), and test support likewise: each technology has `spring-boot-starter-<technology>-test`, pulling in the main starter, `spring-boot-starter-test`, and that technology's test module (slice annotation, test clients).

`testing-demo/pom.xml`:

```
<!-- Boot 4: one test starter per technology -->
<!-- @WebMvcTest, MockMvcTester, RestTestClient, JUnit 6, AssertJ, Mockito -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-webmvc-test</artifactId>
  <scope>test</scope>
</dependency>
<!-- @DataJpaTest, TestEntityManager -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa-test</artifactId>
  <scope>test</scope>
</dependency>
<!-- @RestClientTest, MockRestServiceServer -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-restclient-test</artifactId>
  <scope>test</scope>
</dependency>
```

Test starter	Adds (besides `spring-boot-starter-test`)
`spring-boot-starter-webmvc-test`	`@WebMvcTest`, `@AutoConfigureMockMvc`, `MockMvcTester`, `RestTestClient` / `TestRestTemplate` support
`spring-boot-starter-webflux-test`	`@WebFluxTest`, `WebTestClient` support, `reactor-test`
`spring-boot-starter-data-jpa-test`	`@DataJpaTest`, `TestEntityManager`, `@AutoConfigureTestDatabase`
`spring-boot-starter-jdbc-test`	`@JdbcTest`
`spring-boot-starter-data-r2dbc-test`	`@DataR2dbcTest`, `reactor-test`
`spring-boot-starter-restclient-test`	`@RestClientTest`, `MockRestServiceServer` wiring
`spring-boot-starter-jackson-test`	Jackson 3 + `spring-boot-starter-test` (`@JsonTest` itself lives in `spring-boot-test-autoconfigure`); pulled in by the webmvc/webflux/restclient test starters
`spring-boot-starter-security-test`	`spring-security-test` (`@WithMockUser`, `jwt()`, ...)
`spring-boot-starter-kafka-test`	`spring-kafka-test` (`@EmbeddedKafka`)

`spring-boot-starter-test` itself contains, in Boot 4.1.1: **JUnit Jupiter 6.0** (needs Java 17+), **AssertJ**, **Mockito** (+ `mockito-junit-jupiter`), **Hamcrest** (older matcher library), **JSONassert** (compares JSON documents), **JsonPath** (extracts values with paths like `$.amount`), **Awaitility** (waits for asynchronous results), XMLUnit, `spring-test` and `spring-boot-test`.

The slice annotations **moved packages** with their modules — old imports break on upgrade:

Annotation	Boot 3.x package	Boot 4 package
`@WebMvcTest`, `@AutoConfigureMockMvc`	`org.springframework.boot.test.autoconfigure.web.servlet`	`org.springframework.boot.webmvc.test.autoconfigure`
`@DataJpaTest`	`...boot.test.autoconfigure.orm.jpa`	`org.springframework.boot.data.jpa.test.autoconfigure`
`TestEntityManager`	`...boot.test.autoconfigure.orm.jpa`	`org.springframework.boot.jpa.test.autoconfigure`
`@RestClientTest`	`...boot.test.autoconfigure.web.client`	`org.springframework.boot.restclient.test.autoconfigure`
`@WebFluxTest`	`...boot.test.autoconfigure.web.reactive`	`org.springframework.boot.webflux.test.autoconfigure`
`@JsonTest`	`org.springframework.boot.test.autoconfigure.json`	unchanged
`@SpringBootTest`, `@TestConfiguration`	`org.springframework.boot.test.context`	unchanged

Common mistakes

- Adding only `spring-boot-starter-test` and wondering why `@WebMvcTest` doesn't resolve — in Boot 4 it lives in the **`webmvc`** test module.
- Keeping Boot 3 imports after an upgrade — fix them per the table; the annotations work the same.

12.4 Unit tests: no Spring at all

What it is

A **unit test** creates the class with `new`, hands it **test doubles** (stand-ins for real collaborators, see Java ch18), calls a method and checks the result — no context, no scanning, no database. **Constructor injection** (Chapter 2) makes this possible: the test passes the dependencies itself.

```
/** Layer 1 of the pyramid: a plain unit test. No Spring, no context, milliseconds. */
@ExtendWith(MockitoExtension.class)
class PriceServiceTest {

    @Mock ProductRepository products;
    @Mock ExchangeRateClient rates;
    PriceService service;

    @BeforeEach
    void setUp() {                       // constructor injection = no framework needed
        service = new PriceService(products, rates, "EUR");
    }

    @Test
    void convertsWithTheRemoteRate() {
        long t0 = System.nanoTime();
        when(products.findBySku("MUG-001"))
                .thenReturn(Optional.of(new Product("MUG-001", "Mug", new BigDecimal("10.00"))));
        when(rates.rate("EUR", "USD")).thenReturn(new BigDecimal("1.0850"));

        PriceView v = service.priceOf("MUG-001", "USD");

        System.out.println("unit test: " + v + " in " + (System.nanoTime() - t0) / 1_000_000 + " ms");
        assertThat(v.amount()).isEqualByComparingTo("10.85");   // 10.00 * 1.085 = 10.85
    }

    @Test
    void homeCurrencySkipsTheRemoteCall() {
        when(products.findBySku("MUG-001"))
                .thenReturn(Optional.of(new Product("MUG-001", "Mug", new BigDecimal("10.00"))));
        assertThat(service.priceOf("MUG-001", "EUR").amount()).isEqualByComparingTo("10.00");
        verify(rates, never()).rate(anyString(), anyString());
    }

    @Test
    void unknownSkuThrows() { ... assertThatThrownBy(...).hasMessage("No product with SKU NOPE"); }
}
```

Output:

```
unit test: PriceView[sku=MUG-001, name=Mug, amount=10.85, currency=USD] in 0 ms
```

- `@ExtendWith(MockitoExtension.class)` — Mockito's own JUnit extension creates the `@Mock` fields (not Spring).
- `isEqualByComparingTo("10.85")` — compares `BigDecimal`s by value; `isEqualTo` fails on `10.85` vs `10.850` because `BigDecimal.equals` also compares the scale (digits after the point).
- `verify(rates, never())` — proves "no remote call for the home currency", cheaply.

When not to use it

When the behaviour is **produced by Spring** — a `@Transactional` rollback, a `@PreAuthorize` check, JSON field names, a derived query. A unit test bypasses the proxy or framework and proves nothing; use a slice.

Common mistakes

- **Field injection** (`@Autowired private Repo repo;`) needs reflection or a context just to set the field. Prefer constructors.
- **Starting Spring "because it's easier"** — `@SpringBootTest` for a calculation costs seconds and hides design problems.

TL;DR

Constructor injection + Mockito + AssertJ = Spring-free unit tests in milliseconds. Most tests should look like this.

12.5 How the Spring TestContext Framework works

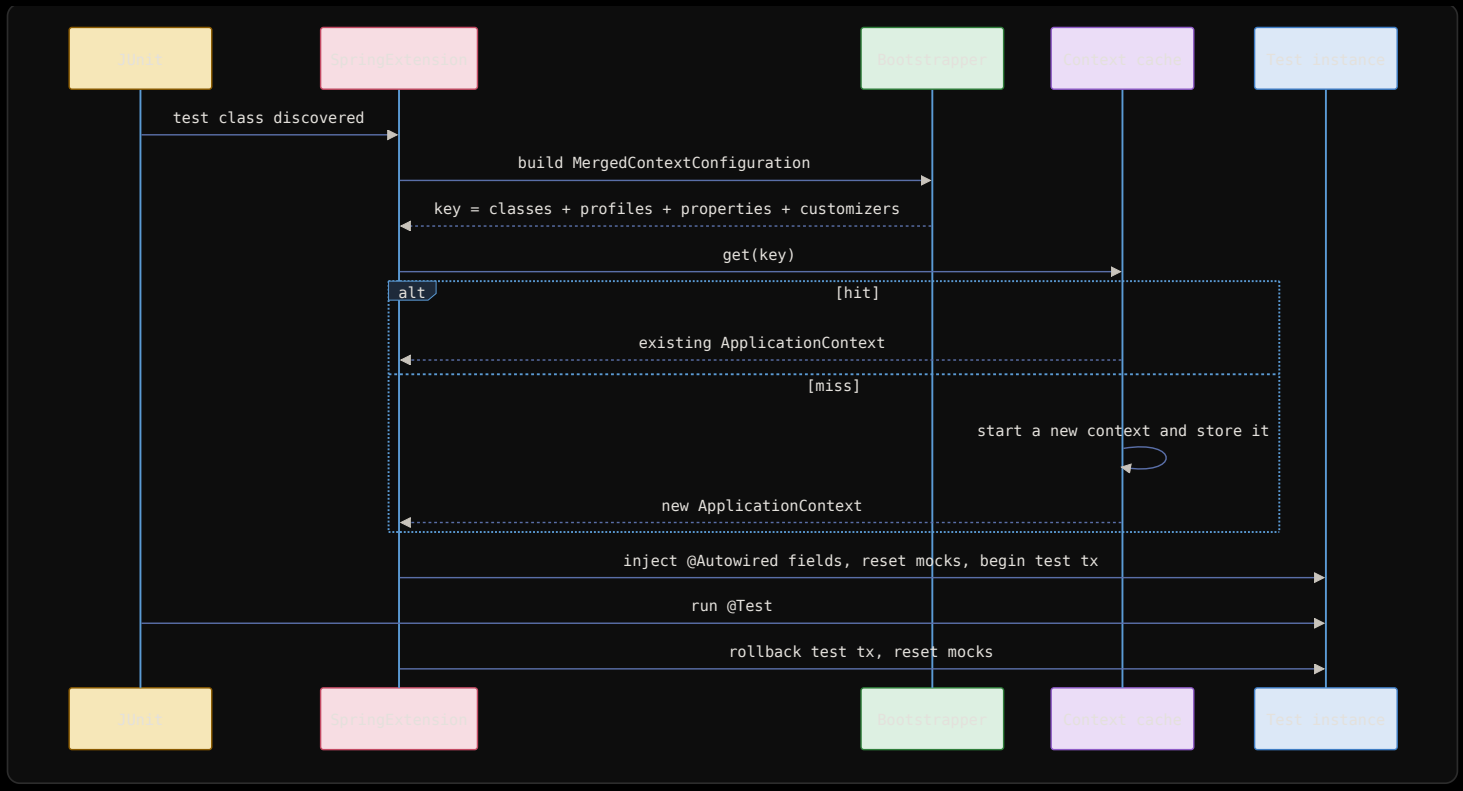
What it is

The **Spring TestContext Framework** (in `spring-test`) is the engine behind every Spring test annotation; `@SpringBootTest` and the slices sit on top. Its four parts explain most "why is my test slow / why is this bean missing" questions:

Part	What it is
`SpringExtension`	the JUnit Jupiter extension connecting JUnit's lifecycle to Spring. Every Boot test annotation is meta-annotated with `@ExtendWith(SpringExtension.class)`, so you never add it.
`TestContext`	per test class: holds the test class, current instance and method, and gives access to the `ApplicationContext`.
`TestExecutionListener`s	plug-ins called before/after each class and method: dependency injection into test fields, `@Transactional` test transactions, `@DirtiesContext`, `@MockitoBean` resets, `@Sql` scripts...
Context cache	a static, JVM-wide map from a *context configuration key* to a started `ApplicationContext`.

How it works

1. JUnit discovers the test class. `SpringExtension` asks a **bootstrapper** (Boot's `SpringBootTestContextBootstrapper`, or a slice's own) to build a **`MergedContextConfiguration`**: configuration classes, active **profiles** (named configuration sets such as `test` or `prod`), test properties, context customizers (`@MockitoBean` definitions, `@DynamicPropertySource` methods), etc.
2. For `@SpringBootTest` without classes, the bootstrapper searches **up the package tree** for `@SpringBootConfiguration` (your `@SpringBootApplication`) — so tests must live in the main class's package or below.
3. The merged configuration is the **cache key** (two classes share a context iff their keys are equal): found → reuse (**hit**); else create, start and store one (**miss**).
4. Listeners inject `@Autowired` fields (`DependencyInjectionTestExecutionListener` — the test instance is not a bean), start a transaction if the test is `@Transactional`, etc.
5. Afterwards they roll back the test transaction, reset `@MockitoBean` mocks, and close the context if marked dirty.
6. The context stays cached for later classes with the same key until the JVM exits (or eviction).



12.6 `@SpringBootTest`: the whole application

What it is

`@SpringBootTest` starts your **entire application context** as `SpringApplication.run` would: **component scanning** (turning `@Component`, `@Service`, ... classes into beans), all auto-configuration, `application.properties`. The tool for proving beans work together.

The four web environments

webEnvironment	What you get	Use for
`MOCK` (default)	a web context with a **mock servlet environment** — no server, no port. Add `@AutoConfigureMockMvc` to get `MockMvcTester`.	most integration tests of a web app
`RANDOM_PORT`	a **real embedded server** (Tomcat/Netty) on a free port; inject it with `@LocalServerPort`	end-to-end HTTP tests
`DEFINED_PORT`	a real server on `server.port` (default 8080)	rare: breaks parallel builds when the port is taken
`NONE`	no web environment	batch jobs, messaging consumers, clients

Boot 4 change: `@SpringBootTest` no longer auto-configures test clients by itself. Add `@AutoConfigureMockMvc` for MockMvc, `@AutoConfigureRestTestClient` for `RestTestClient`, `@AutoConfigureTestRestTemplate` for `TestRestTemplate`, `@AutoConfigureWebTestClient` for `WebTestClient` (per the Boot 4.0 migration guide).

Example: real beans, one fake

```
/** Whole context, mock servlet environment (no port): every real bean except the remote client. */
@SpringBootTest(properties = "catalog.home-currency=EUR")
@AutoConfigureMockMvc
class MockMvcFullContextTest {

    @TestConfiguration            // extra beans for this test only; not picked up by component scanning
    static class SeedData {
        @Bean ApplicationRunner seed(ProductRepository repo) {
            return args -> repo.save(new Product("LMP-001", "Lamp", new BigDecimal("40.00")));
        }
    }

    @Autowired MockMvcTester mvc;
    @Autowired ApplicationContext ctx;
    @MockitoBean ExchangeRateClient rates;    // the only fake: no network in this test

    @Test
    void realServiceAndDatabaseMockedRemote() throws Exception {
        given(rates.rate("EUR", "GBP")).willReturn(new BigDecimal("0.85"));

        MvcTestResult r = mvc.get().uri("/api/products/LMP-001/price?currency=GBP").exchange();

        System.out.println("@SpringBootTest beans: " + ctx.getBeanDefinitionCount());
        System.out.println("GET GBP -> " + r.getResponse().getContentAsString());
        assertThat(r).hasStatusOk().bodyJson().extractingPath("$.amount").isEqualTo(34.0);
    }
}
```

Output:

```
@SpringBootTest beans: 253
GET GBP -> {"sku":"LMP-001","name":"Lamp","amount":34.00,"currency":"GBP"}
```

- `properties = "..."` — test-only properties with high precedence (§12.15).
- **`@TestConfiguration`** — a test-only `@Configuration`. As a static nested class it is **added to** the app's configuration (a nested plain `@Configuration` would *replace* the auto-detected `@SpringBootApplication` — rarely wanted). Top-level `@TestConfiguration` classes are skipped by component scanning; pull them in with `@Import`.
- Controller, service, repository, JPA and H2 are real; only the internet-calling class is replaced.

When not to use it

- To test one controller's validation or one query — a slice is 5–10× faster and fails more precisely.
- With a different `properties=` / mock set per class — each variation is a new context (§12.11).

Common mistakes

- **Expecting a `@Transactional` test to roll back what the server did in a `RANDOM_PORT` test.** The request runs on a server thread in its own transaction. Clean up explicitly (`FullStackTest` calls `repo.deleteAll()` in `@BeforeEach`).
- **`@SpringBootTest` in a package outside the main class's** → "Unable to find a @SpringBootConfiguration".

12.7 Test slices

What it is

A **test slice** starts only the part of the context one layer needs. The slice annotation (e.g. `@WebMvcTest`):

1. **Disables full auto-configuration**, importing only that slice's auto-configurations (`@WebMvcTest`: MVC, Jackson — the JSON library — validation, security filters, ...).
2. Adds a **type-exclude filter** to component scanning so only matching beans of *your* code register (`@WebMvcTest`: `@Controller`, `@ControllerAdvice`, `@JacksonComponent`, `Converter`, `Filter`, `WebMvcConfigurer`, `HandlerInterceptor`, ...). `@Service`, `@Repository` and `@Component` classes are **not** registered.
3. Adds test helpers (`MockMvcTester`, `TestEntityManager`, `MockRestServiceServer`, ...).

Like a rehearsal of one scene: only that scene's actors are on stage; everyone else is a stand-in (`@MockitoBean`).

The slices you will meet

Annotation	Starts	Your beans registered	Extras	Transactional?
`@WebMvcTest`	Spring MVC, Jackson, validation, Security	controllers, advice, filters, converters, `WebMvcConfigurer`	`MockMvcTester`, `MockMvc`	no
`@WebFluxTest`	WebFlux, codecs, validation	controllers, `WebFluxConfigurer`, ...	`WebTestClient` (ch13)	no
`@DataJpaTest`	JPA, Hibernate, Spring Data JPA, embedded DB, Flyway/Liquibase	entities, repositories	`TestEntityManager`, SQL logging on	**yes, rolled back**
`@JdbcTest`	`DataSource`, `JdbcTemplate`, `JdbcClient`, embedded DB	none	—	yes, rolled back
`@DataR2dbcTest`	R2DBC (non-blocking database access), Spring Data R2DBC	repositories	—	**no** (ch13)
`@JsonTest`	Jackson (or Gson / JSON-B)	`@JacksonComponent`, Jackson modules	`JacksonTester`, `BasicJsonTester`	no
`@RestClientTest`	`RestClient.Builder`, `RestTemplateBuilder`, Jackson	the classes you name	`MockRestServiceServer`	no
`@DataMongoTest`, `@DataRedisTest`, `@DataJdbcTest`, ...	that store	repositories	—	depends

`@WebMvcTest`

```
/** Web slice: only the MVC layer is real; the service is a Mockito mock. */
@WebMvcTest(ProductController.class)
class ProductControllerWebMvcTest {

    @Autowired MockMvcTester mvc;
    @Autowired ApplicationContext ctx;
    @MockitoBean PriceService service;       // replaces (or adds) the bean in the test context

    @Test
    void sliceContainsOnlyWebBeans() {
        System.out.println("@WebMvcTest beans: " + ctx.getBeanDefinitionCount()
                + ", ProductController present: " + (ctx.getBeanNamesForType(ProductController.class).length == 1)
                + ", ProductRepository present: " + (ctx.getBeanNamesForType(ProductRepository.class).length == 1));
        assertThat(ctx.getBeanNamesForType(ProductRepository.class)).isEmpty();
    }
    ...
}
```

Output:

```
@WebMvcTest beans: 127, ProductController present: true, ProductRepository present: false
```

127 beans instead of 253: no JPA, database or `RestClient`. `@WebMvcTest(ProductController.class)` narrows it to **one** controller; without the argument every controller registers and all their dependencies must be mocked. Web tests themselves: §12.8.

`@DataJpaTest`

```
/** JPA slice: entities, repositories, an embedded DB; every test runs in a rolled-back transaction. */
@DataJpaTest
@TestMethodOrder(MethodOrderer.OrderAnnotation.class)
class ProductRepositoryDataJpaTest {

    @Autowired ProductRepository repo;
    @Autowired TestEntityManager em;
    @Autowired DataSource dataSource;

    @Test @Order(1)
    void derivedAndJpqlQueries() throws Exception {
        System.out.println("DB = " + dataSource.getConnection().getMetaData().getURL()
                + ", test transaction active = " + TestTransaction.isActive());
        em.persist(new Product("MUG-001", "Mug", new BigDecimal("10.00")));
        em.persist(new Product("PEN-001", "Pen", new BigDecimal("2.50")));
        em.persist(new Product("LMP-001", "Lamp", new BigDecimal("40.00")));
        em.flush();   // send the INSERTs now
        em.clear();   // forget cached entities so the queries really hit the database

        assertThat(repo.findBySku("PEN-001")).map(Product::getName).contains("Pen");
        assertThat(repo.cheaperThan(new BigDecimal("15"))).extracting(Product::getSku)
                .containsExactly("PEN-001", "MUG-001");
        System.out.println("test 1 saw " + repo.count() + " rows");
    }

    @Test @Order(2)
    void previousTestWasRolledBack() {
        System.out.println("test 2 starts with " + repo.count() + " rows");
        assertThat(repo.count()).isZero();
    }
}
```

Output (`@DataJpaTest` turns on `spring.jpa.show-sql`, so Hibernate prints its SQL):

```
DB = jdbc:h2:mem:fa1ebe8d-78e8-4d82-a212-1a1b655ce2ab, test transaction active = true
Hibernate: select next value for product_seq
Hibernate: select next value for product_seq
Hibernate: insert into product (name,price,sku,id) values (?,?,?,?)
Hibernate: insert into product (name,price,sku,id) values (?,?,?,?)
Hibernate: insert into product (name,price,sku,id) values (?,?,?,?)
Hibernate: select p1_0.id,p1_0.name,p1_0.price,p1_0.sku from product p1_0 where p1_0.sku=?
Hibernate: select p1_0.id,p1_0.name,p1_0.price,p1_0.sku from product p1_0 where p1_0.price<=? order by p1_0.price
Hibernate: select count(*) from product p1_0
test 1 saw 3 rows
Hibernate: select count(*) from product p1_0
test 2 starts with 0 rows
```

- **Embedded database**: `@DataJpaTest` includes `@AutoConfigureTestDatabase`, whose default `Replace.NON_TEST` (since Boot 3.4) swaps your `DataSource` for a fresh embedded one (random `jdbc:h2:mem:` name) — unless it is already a *test* database such as a Testcontainers `@ServiceConnection` (§12.14). `replace = NONE` always keeps the configured one.
- **Rolled-back transaction**: each test method's transaction rolls back, so test 2 sees 0 rows — independence without cleanup code.
- **`TestEntityManager`**: a helper around JPA's `EntityManager` (`persist`, `persistAndFlush`, `find`, `clear`) to set up data without the repository under test.
- **`flush()` + `clear()`**: otherwise the INSERTs sit in the **persistence context** (the **first-level cache** of loaded/changed entities, ch08) and `findBySku` might be answered from it. Clearing forces real SQL.

> The rolled-back test transaction can **hide bugs**: **lazy loading** (fetching related rows on first access) always works inside it, and nothing is ever **committed**, so commit-deferred constraint checks, `@TransactionalEventListener(AFTER_COMMIT)` and code needing its own transaction never run. Chapter 7 and Chapter 8 use non-transactional `@SpringBootTest`s for this reason. Opt out per class with `@Transactional(propagation = Propagation.NOT_SUPPORTED)`.

`@JsonTest`

Checks the **wire format** (how an object looks once **serialized** to JSON): field names, number and date formats, and that nothing internal leaks.

```java
@JsonTest
class PriceViewJsonTest {

    @Autowired JacksonTester<PriceView> json;

    @Test
    void serialize() throws Exception {
        JsonContent<PriceView> out = json.write(new PriceView("MUG-001", "Mug", new BigDecimal("10.85"), "USD"));
        System.out.println("serialized: " + out.getJson());
        assertThat(out).extractingJsonPathStringValue("$.currency").isEqualTo("USD");
        assertThat(out).extractingJsonPathNumberValue("$.amount").isEqualTo(10.85);
        assertThat(out).doesNotHaveJsonPath("$.id");
    }

    @Test
    void deserialize() throws Exception {
        PriceView v = json.parseObject("""
                {"sku":"PEN-001","name":"Pen","amount":2.5,"currency":"EUR"}""");
        System.out.println("parsed: " + v);
        assertThat(v.amount()).isEqualByComparingTo("2.50");
    }
}
```

Output:

```
serialized: {"sku":"MUG-001","name":"Mug","amount":10.85,"currency":"USD"}
parsed: PriceView[sku=PEN-001, name=Pen, amount=2.5, currency=EUR]
```

The slice builds the same Jackson 3 `JsonMapper` your app uses, including `spring.jackson.*` properties and `@JacksonComponent` classes — a plain `new JsonMapper()` would miss them.

`@RestClientTest`

Tests an HTTP **client** class without a network: the auto-configured `RestClient.Builder` is connected to a **`MockRestServiceServer`**, which checks what the client sent and plays back a canned answer.

```java
@RestClientTest(ExchangeRateClient.class)
@TestPropertySource(properties = "rates.base-url=https://rates.test")
class ExchangeRateClientTest {

    @Autowired ExchangeRateClient client;
    @Autowired MockRestServiceServer server;

    @Test
    void parsesTheRate() {
        server.expect(requestTo("https://rates.test/rates?from=EUR&to=USD"))
              .andExpect(method(HttpMethod.GET))
              .andRespond(withSuccess("""
                      {"from":"EUR","to":"USD","rate":1.0850}""", MediaType.APPLICATION_JSON));

        var rate = client.rate("EUR", "USD");
        System.out.println("rate EUR->USD = " + rate);
        assertThat(rate).isEqualByComparingTo("1.085");
        server.verify();                  // every expected request really happened
    }

    @Test
    void serverErrorBecomesAnException() {
        server.expect(requestTo("https://rates.test/rates?from=EUR&to=JPY"))
              .andRespond(withServerError());

        assertThatThrownBy(() -> client.rate("EUR", "JPY"))
                .isInstanceOf(HttpServerErrorException.InternalServerError.class);
        System.out.println("500 from rates API -> HttpServerErrorException.InternalServerError");
    }
}
```

Output:

```
rate EUR->USD = 1.0850
500 from rates API -> HttpServerErrorException.InternalServerError
```

`MockRestServiceServer` works only for clients built from the auto-configured builder. Other options (a real local server, WireMock — a configurable fake HTTP server): Chapter 6.

Common mistakes

- **"No qualifying bean of type PriceService" in a `@WebMvcTest`** — services aren't in the slice. Add `@MockitoBean PriceService`, or `@Import` the real class if cheap.
- **Extra `@ComponentScan` or `@EnableJpaRepositories` on the main class** bypasses the slices' type-exclude filters, so `@WebMvcTest` suddenly loads repositories. Put such configuration in a separate `@Configuration` class.

- **Heavy `@Configuration` classes in the main package** are scanned by every slice (e.g. a Kafka client). Guard with a profile or condition, or move them out of the scanned package.
- **Assuming `@DataJpaTest` = production database behaviour** — H2 accepts SQL PostgreSQL rejects and vice versa. Test database-specific SQL on the real engine (§12.14).

When not to use it

Not a slice per class: a controller that only forwards to a service needs one web-slice test per endpoint (happy path + error mapping).

In the interview

Q: What is a test slice and why use one? 30-second answer: "A slice like `@WebMvcTest` or `@DataJpaTest` starts only one layer of the application context: its auto-configurations plus the matching beans from my code. It's much faster than `@SpringBootTest` and tests exactly what Spring adds to that layer: mappings, validation and JSON for the web, real SQL for JPA. Everything outside the slice I mock with `@MockitoBean`."

- *Does `@WebMvcTest` load `@Service` beans?* → No; only web components. Mock or `@Import` services.
- *Are `@DataJpaTest` tests transactional?* → Yes, each rolls back; the `DataSource` is replaced by an embedded one unless already a test database or `replace = NONE`.
- *How would you test a `RestClient`-based client?* → `@RestClientTest` + `MockRestServiceServer` (`server.verify()` checks every expected request happened), or a real local stub server for timeouts and connection-level behaviour.

TL;DR

Slices = partial contexts: `@WebMvcTest` (web), `@DataJpaTest` (JPA, rolled back, embedded DB), `@JsonTest` (Jackson), `@RestClientTest` (clients + `MockRestServiceServer`). Mock what's outside the slice.

12.8 `MockMvcTester`: AssertJ-style MockMvc

What it is

MockMvc sends requests straight into the `DispatcherServlet` (Spring MVC's front controller) without a server or network, through filters, mapping, argument resolution, validation, the controller, exception handlers and message converters — exactly as in production.

`MockMvcTester` (Framework 6.2, auto-configured since Boot 3.4) wraps MockMvc in **AssertJ** assertions: build a request, `exchange()`, `assertThat(...)`. It replaces the older `perform(...).andExpect(...)` style.

	Classic `MockMvc`	`MockMvcTester`
Request	`mvc.perform(get("/x"))`	`mvc.get().uri("/x").exchange()`
Assertions	`andExpect(status().isOk())`, `jsonPath(...).value(...)` (Hamcrest)	`assertThat(result).hasStatusOk()`, `.bodyJson().extractingPath("$.x")` (AssertJ)
Checked exceptions	`perform` throws `Exception`	none
Unhandled exceptions	thrown out of `perform`	captured in the result: `assertThat(result).hasFailed()`
Both available?	yes — Boot auto-configures both with `@WebMvcTest` / `@AutoConfigureMockMvc`	

Example

```
@Test
void returnsPriceAsJson() throws Exception {
    given(service.priceOf("MUG-001", "USD"))
            .willReturn(new PriceView("MUG-001", "Mug", new BigDecimal("10.85"), "USD"));

    MvcTestResult r = mvc.get().uri("/api/products/{sku}/price?currency=USD", "MUG-001").exchange();

    System.out.println("GET price -> " + r.getResponse().getStatus() + " " + r.getResponse().getContentAsString());
    assertThat(r).hasStatusOk()
            .hasContentType(MediaType.APPLICATION_JSON)
            .bodyJson().extractingPath("$.amount").isEqualTo(10.85);
}

@Test
void unknownSkuIsProblemDetail404() throws Exception {
    given(service.priceOf("NOPE-000", "EUR")).willThrow(new ProductNotFoundException("NOPE-000"));

    MvcTestResult r = mvc.get().uri("/api/products/NOPE-000/price").exchange();

    assertThat(r).hasStatus(404)
            .hasContentType(MediaType.APPLICATION_PROBLEM_JSON)
            .bodyJson().extractingPath("$.detail").isEqualTo("No product with SKU NOPE-000");
}

@Test
void invalidBodyIs400AndServiceIsNeverCalled() {
    MvcTestResult r = mvc.post().uri("/api/products")
            .contentType(MediaType.APPLICATION_JSON)
            .content("""
                    {"sku":"mug1","name":"","price":0}""")
            .exchange();

    assertThat(r).hasStatus(400);
    verifyNoInteractions(service);
}

@Test
void validBodyIs201WithLocation() {
    given(service.create(any())).willReturn(new Product("MUG-002", "Big mug", new BigDecimal("12.50")));
    MvcTestResult r = mvc.post().uri("/api/products")
            .contentType(MediaType.APPLICATION_JSON)
            .content("""
                    {"sku":"MUG-002","name":"Big mug","price":12.50}""")
            .exchange();
    assertThat(r).hasStatus(201).hasHeader("Location", "/api/products/MUG-002/price");
}
```

Output:

```
GET unknown -> 404 {"detail":"No product with SKU NOPE-000","instance":"/api/products/NOPE-000/price","status":404,"title":"Not Found"}
GET price -> 200 {"sku":"MUG-001","name":"Mug","amount":10.85,"currency":"USD"}
POST valid -> 201 Location=/api/products/MUG-002/price
... WARN ... DefaultHandlerExceptionResolver : Resolved [org.springframework.web.bind.MethodArgumentNotValidException: Validation failed for
argument [0] ... with 3 errors: ...]
POST invalid -> 400
```

The tests prove: the URL template, the `currency` default and JSON field names (`amount`, not `price`); `ProductNotFoundException` → **`application/problem+json`** with status, title, detail, instance (RFC 9457, ch05); invalid input rejected **before** the controller (`verifyNoInteractions(service)`; the WARN line logs three violations: bad pattern, blank name, price below 0.01); `201 Created` with a `Location` header.

Common mistakes

- **Asserting `getContentAsString()` with `equals`** — key order and spacing make it brittle. Use `bodyJson()` paths or `bodyJson().isLeniently EqualTo("{...}")`.
- **Expecting servlet-container behaviour** — no real Tomcat: multipart size limits, HTTP/2, compression and container error pages aren't exercised. Use `RANDOM_PORT`.
- **Forgetting security** — with Spring Security on the classpath, `@WebMvcTest` applies Boot's default security, not your `SecurityFilterChain`, unless you `@Import` your config (ch10).
- Unhandled exceptions: assert with `assertThat(result).hasFailed().failure().isInstanceOf(...)`.

12.9 `RestTestClient` (Framework 7.0)

What it is

`RestTestClient` (new in **Spring Framework 7.0**) has the fluent `WebTestClient`-style API (`get().uri(...).exchange().expectStatus()...`) for **servlet** (Spring MVC) apps, without WebFlux. One API, several back ends — the same test code can run against MockMvc or a real server:

Binding	Request goes to	Typical use
`RestTestClient.bindTo(mockMvc)`	an existing `MockMvc` (no server)	slice / mock-environment tests
`bindToController(controller)`	a standalone MockMvc for one controller	pure controller tests without Spring Boot
`bindToRouterFunction(...)`	functional MVC routes	`RouterFunction` tests
`bindToApplicationContext(ctx)`	MockMvc built from a `WebApplicationContext`	custom setups
`bindToServer()`	a **real HTTP server**	`RANDOM_PORT` end-to-end tests

Boot 4 status (verified): Boot 4.0 auto-configures it when the test class has `@AutoConfigureRestTestClient` (module `spring-boot-resttestclient`, in `spring-boot-starter-webmvc-test`): bound to `MockMvc` in a mock environment, pointed at the running server (base URL set) in `RANDOM_PORT`/`DEFINED_PORT` tests. Without the annotation there is no bean:

```
No qualifying bean of type 'org.springframework.test.web.servlet.client.RestTestClient' available
```

Example against MockMvc

```
/** RestTestClient (Framework 7.0) bound to MockMvc: a fluent HTTP-client API, no real server. */
@WebMvcTest(ProductController.class)
@AutoConfigureRestTestClient
class RestTestClientMockMvcTest {

    @Autowired RestTestClient client;
    @MockitoBean PriceService service;

    @Test
    void fluentExpectations() {
        given(service.priceOf("MUG-001", "EUR"))
                .willReturn(new PriceView("MUG-001", "Mug", new BigDecimal("10.00"), "EUR"));

        PriceView body = client.get().uri("/api/products/MUG-001/price")
                .exchange()
                .expectStatus().isOk()
                .expectHeader().contentType("application/json")
                .expectBody(PriceView.class)              // JSON -> record, with the app's JsonMapper
                .returnResult().getResponseBody();

        System.out.println("RestTestClient (MockMvc) body = " + body);
        assertThat(body.amount()).isEqualByComparingTo("10.00");
    }

    @Test
    void assertjStyle() {
        given(service.priceOf("NOPE-000", "EUR")).willThrow(new ProductNotFoundException("NOPE-000"));
        RestTestClient.ResponseSpec spec = client.get().uri("/api/products/NOPE-000/price").exchange();

        assertThat(RestTestClientResponse.from(spec))       // AssertJ adapter (Framework 7.0)
                .hasStatus(404)
                .bodyJson().extractingPath("$.title").isEqualTo("Not Found");
        System.out.println("RestTestClient (AssertJ) 404 title = Not Found");
    }
}
```

Output:

```
RestTestClient (AssertJ) 404 title = Not Found
RestTestClient (MockMvc) body = PriceView[sku=MUG-001, name=Mug, amount=10.00, currency=EUR]
```

The real-server version is in §12.13.

Which test client?

Client	Stack	Server needed	Status in Boot 4
`MockMvcTester`	MVC	no	recommended for MVC slice/mock tests
`RestTestClient`	MVC (any HTTP server with `bindToServer`)	optional	**new**, recommended for fluent HTTP-style tests
`WebTestClient`	WebFlux (can also bind to MockMvc)	optional	recommended for WebFlux (ch13)
`TestRestTemplate`	any	yes	still available (`@AutoConfigureTestRestTemplate`, moved to `org.springframework.boot.resttestclient`); `RestTemplate` itself is heading for deprecation (ch06)

`RestTestClient` adds over `MockMvcTester` a client-style API that targets MockMvc *or* a real server with the same code, plus body decoding to types (`expectBody(PriceView.class)`).

12.10 `@MockitoBean` and `@MockitoSpyBean`

What it is

- `@MockitoBean` (Framework 6.2, `org.springframework.test.context.bean.override.mockito`) puts a **Mockito mock into the application context**, replacing the bean of that type (or name) or adding one; every dependent bean (e.g. the controller) receives the mock.
- `@MockitoSpyBean` wraps the **real existing bean** in a Mockito **spy**: real methods run, but you can `verify` calls or **stub** single methods.

Mockito's own `@Mock` is held only by your test; `@MockitoBean` is **injected by the container** into other beans.

> **Boot 4:** Boot's `@MockBean` and `@SpyBean` (deprecated in 3.4) are **removed** — use `@MockitoBean` / `@MockitoSpyBean`. Difference: the new annotations work on test-class fields (including superclasses and `@Nested` classes), **not inside `@Configuration` classes**. Share mocks via a common base class, or class-level `@MockitoBean(types = {A.class, B.class})`.

How it works

1. While building the `MergedContextConfiguration`, Spring adds bean-override fields to the **cache key** (each distinct mock set = a different context, §12.11).
2. At startup the bean definition is replaced by one returning the mock (a spy wraps the real bean after creation), which is injected into the test field.
3. **After each test method the mock is reset** (default `MockReset.AFTER`), so stubbing doesn't leak.

Example: a spy on the real client

In `FullStackTest` the real `ExchangeRateClient` makes real HTTP calls; the spy counts them (the `@MockitoBean PriceService` mocks above are the more common case):

```
@MockitoSpyBean ExchangeRateClient rates; // the REAL bean, wrapped so we can verify calls
...
verify(rates, times(1)).rate("EUR", "USD");   // EUR request did not call the API
```

Related: `@TestBean`

`@TestBean` (Framework 6.2) overrides a bean with the instance returned by a **static factory method** in the test class — when a hand-written fake is clearer than a mock:

```
@TestBean ExchangeRateClient rates;                    // looks for static method rates()
static ExchangeRateClient rates() { return new FixedRateClient(new BigDecimal("1.10")); }
```

Common mistakes

- **Changing the context per class**: class A mocks `PriceService`, B `ExchangeRateClient`, C both → three contexts. Standardise the mock set in a base class.
- **Mocking what you test**: a `@SpringBootTest` with repository, client *and* service mocked is a slow unit test.
- `@MockitoSpyBean` **on a non-existent bean** → context fails; a spy needs a real bean.
- **Final classes/methods**: Mockito 5's default inline mock maker can mock them; on current JDKs it self-attaches as a runtime **agent** (prints *"Mockito is currently self-attaching..."*). Mockito recommends a `-javaagent` in the Surefire (Maven test runner) config before the JDK disallows dynamic agent loading by default.

In the interview

Q: `@Mock` vs `@MockitoBean`? → `@Mock` is plain Mockito, held only by the test. `@MockitoBean` registers the mock *in the context*, so Spring injects it into other beans, and it joins the cache key.

- *What replaced `@MockBean` in Boot 4?* → `@MockitoBean` from Spring Framework (6.2+).
- *When a spy bean?* → To keep real behaviour but verify interactions or stub one method.

TL;DR

`@MockitoBean` = mock inside the context; `@MockitoSpyBean` = spy around the real bean. Both reset after each test and change the cache key.

12.11 Test context caching

What it is

Starting a context is the most expensive part of a Spring test, so started contexts live in a **static cache** for the whole test JVM; the next class with an *identical* configuration gets it in about 0 ms (like keeping an oven hot between orders with the same setting).

What is in the cache key

A new context is created when **any** of these differ:

- configuration classes / locations (`@SpringBootTest(classes=...)`, `@ContextConfiguration`, `@Import` on the test, nested `@TestConfiguration`)
- active profiles (`@ActiveProfiles`)
- test properties (`@TestPropertySource`, `@SpringBootTest(properties=...)`) and property source files
- **context customizers**: `@MockitoBean` / `@MockitoSpyBean` / `@TestBean` definitions, `@DynamicPropertySource` methods, `@AutoConfigureMockMvc` and other `@AutoConfigure...` settings, `webEnvironment`
- the slice type (`@WebMvcTest(A.class)` and `@WebMvcTest(B.class)` are different)
- context initializers, the context loader, the parent context (`@ContextHierarchy`)

`@DirtiesContext` doesn't change the key. It **closes and evicts** the context after the class or method, so the next user must build it again.

The measured demo

`context-caching` has one base class and seven test classes run in name order (`junit.jupiter.testclass.order.default=...ClassOrderer$ClassName` in `junit-platform.properties`). A bean sleeping **400 ms** in its constructor stands in for real startup work (warming a cache, loading a model):

```
@Component
public static class SlowStartup {
    public static final AtomicInteger CREATED = new AtomicInteger();
    public SlowStartup() throws InterruptedException {
        Thread.sleep(400);
        CREATED.incrementAndGet();          // = number of contexts built so far
    }
}
```

```
/** One shared configuration: every subclass that adds nothing reuses the same cached context. */
@SpringBootTest
@AutoConfigureMockMvc
abstract class BaseIntegrationTest { }

class C1_GreetingApiTest   extends BaseIntegrationTest { ... }  // first user -> MISS
class C2_SameConfigTest    extends BaseIntegrationTest { ... }  // identical config -> HIT
class C3_WithMockTest      extends BaseIntegrationTest {        // a @MockitoBean changes the key -> MISS
    @MockitoBean CachingApplication.GreetingService service; ... }
@TestPropertySource(properties = "feature.discounts=true")    // different properties -> MISS
class C4_OtherPropertyTest extends BaseIntegrationTest { ... }
@DirtiesContext                                               // HIT, then closed and evicted
class C5_DirtiesContextTest extends BaseIntegrationTest { ... }
class C6_AfterDirtiesTest  extends BaseIntegrationTest { ... }  // base config again, evicted -> MISS
class C7_BackToBaseTest    extends BaseIntegrationTest { ... }  // HIT on the rebuilt context
```

A `TestExecutionListener` (registered in `src/test/resources/META-INF/spring.factories`) measures per class the time from `beforeTestClass` until the context is available, and reads the cache statistics:

```java
public class CacheReportListener implements TestExecutionListener, Ordered {

    /** The JVM-wide context cache is reachable through this protected accessor. */
    static final class Stats extends DefaultCacheAwareContextLoaderDelegate {
        static ContextCache cache() { return new Stats().getContextCache(); }
    }

    @Override public int getOrder() { return Ordered.LOWEST_PRECEDENCE; }
    @Override public void beforeTestClass(TestContext tc) { start = System.nanoTime(); }

    @Override
    public void prepareTestInstance(TestContext tc) {
        ...
        tc.getApplicationContext();                        // loads it, or takes it from the cache
        long ms = (System.nanoTime() - start) / 1_000_000;
        ContextCache c = Stats.cache();
        System.out.printf("%-26s context in %4d ms | cache size=%d hits=%d misses=%d | contexts built=%d%n", ...);
    }
}
```

Output:

```
C1_GreetingApiTest         context in  892 ms | cache size=1 hits=3 misses=1 | contexts built=1
C2_SameConfigTest          context in    0 ms | cache size=1 hits=18 misses=1 | contexts built=1
C3_WithMockTest            context in  575 ms | cache size=2 hits=32 misses=2 | contexts built=2
C4_OtherPropertyTest       context in  503 ms | cache size=3 hits=45 misses=3 | contexts built=3
C5_DirtiesContextTest      context in    0 ms | cache size=3 hits=60 misses=3 | contexts built=3
C6_AfterDirtiesTest        context in  467 ms | cache size=3 hits=73 misses=4 | contexts built=4
C7_BackToBaseTest          context in    0 ms | cache size=3 hits=88 misses=4 | contexts built=4
```

- **misses** = contexts built (4), matching `SlowStartup`'s counter.
- **hits** count every cache *lookup* (several listeners look up the context per test, ~15 per class) — watch the change, not the value.
- Each miss costs ~0.4 s simulated + ~0.1 s real Boot work (the first also JVM/class loading: 892 ms); each hit **0 ms**.
- A mock (C3) and one property (C4) each forced a new context; `@DirtiesContext` (C5) discarded a good one, so C6 rebuilt it.
- Seven classes, four builds; one shared configuration without `@DirtiesContext` would be **one**.

Spring's own statistics: set logger **`org.springframework.test.context.cache`** to `DEBUG` → `Spring test ApplicationContext cache statistics: [DefaultContextCache@... size = 1, maxSize = 32, ... hitCount = ..., missCount = ...]`.

Limits and Framework 7 behaviour

Setting	Default	How to change
Maximum cached contexts	**32**, least-recently-used evicted (and closed)	JVM system property or `spring.properties`: `spring.test.context.cache.maxSize=16`
Context pausing (new in Framework 7.0)	`ON_CONTEXT_SWITCH`: when the next test needs a *different* context, the inactive one is **paused** — its `Lifecycle` / `SmartLifecycle` beans (schedulers, message listener containers) stopped, restarted on reuse	`spring.test.context.cache.pause=always` or `never`

Pausing fixes an old problem: a cached context's `@Scheduled` job or Kafka/JMS (Java Message Service) listener kept consuming messages while *another* class ran, stealing its messages or skewing results.

Tips for a fast suite

1. Put common annotations (`@SpringBootTest`, `@AutoConfigureMockMvc`, `@ActiveProfiles("test")`, shared `@MockitoBean`s, container config) on **one abstract base class**.
2. Prefer **slices** for layer tests — small, so extra contexts are cheap.
3. Avoid `@DirtiesContext`; reset state instead (clean tables, `reset()` fakes).
4. Watch for **one-off** `properties=` values and mocks — each is a new context kept in memory until eviction.
5. Surefire runs all classes in one JVM by default (`forkCount=1`, `reuseForks=true`), which makes the cache useful; forking a JVM per class throws it away.

Common mistakes

- **Too many contexts in memory**: up to 32, each with thread pools, connection pools and embedded servers, can exhaust memory, connections or ports (`Too many connections` or out-of-memory late in the build).
- **Static state across contexts**: contexts in one JVM share static fields, and the same H2 database if the URL is fixed.

In the interview

Q: Why is our Spring test suite slow and how do you fix it? "Almost always too many application contexts. The TestContext Framework caches contexts by their configuration. Every different set of `@MockitoBean`s, properties, profiles or `@Import`s, and every `@DirtiesContext`, builds a new one. I'd enable DEBUG logging for `org.springframework.test.context.cache` to count misses, move

shared configuration to a base class, replace one-off `@SpringBootTest`s with slices or unit tests, and remove `@DirtiesContext` where cleanup is enough."

- *What's the default cache size?* → 32, LRU eviction; `spring.test.context.cache.maxSize`.
- *What is context pausing?* → Framework 7.0: inactive cached contexts get their lifecycle beans stopped when the tests switch to another context, and restarted on reuse.

TL;DR

Same configuration → same cached context. Mocks, properties, profiles, imports and `@DirtiesContext` (closes and evicts it; it does not change the key) cost a new one. Aim for a handful of configurations per suite.

12.12 `@DynamicPropertySource`: properties known only at runtime

The problem it solves

Some configuration exists only **after** the test starts — a stub server's random port, a container's mapped port, a temp directory — so it can't go in `application-test.properties`.

What it is

A **static** method annotated `@DynamicPropertySource` (Framework 5.2.5; static because it runs before any test instance exists) receives a `DynamicPropertyRegistry`. Each property is a `Supplier` evaluated lazily when first read, so it can return a value known only once something has started. These properties have the **highest precedence**, above `application.properties`, `@TestPropertySource` and `@SpringBootTest(properties)`.

Framework 6.2 added `DynamicPropertyRegistrar`: a *bean* that registers properties — for values coming from another bean (e.g. a stub server bean) rather than a static field.

Example: a fake exchange-rate API on a random port

```
@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT)
@AutoConfigureRestTestClient
@ActiveProfiles("test")
class FullStackTest {

    // A tiny fake "rates" API started before the Spring context (JDK built-in HTTP server).
    static final HttpServer RATES = startRatesServer();

    // startRatesServer(): HttpServer.create(new InetSocketAddress("127.0.0.1", 0), 0) -- port 0 = any free port --
    // with a /rates context answering {"from":"EUR","to":"USD","rate":1.10} (full code in FullStackTest.java)

    @DynamicPropertySource
    static void ratesUrl(DynamicPropertyRegistry registry) {
        // evaluated lazily, after the fake server has picked its port
        registry.add("rates.base-url", () -> "http://127.0.0.1:" + RATES.getAddress().getPort());
    }

    @AfterAll
    static void stop() { RATES.stop(0); }
    ...
}
```

The `ExchangeRateClient` is created with `rates.base-url` pointing at the fake server, so the test exercises the real `RestClient`, JSON parsing and **sockets**, using the JDK's built-in `com.sun.net.httpserver.HttpServer` (no extra dependency).

Common mistakes

- **Non-static method** → ignored or rejected.
- **Passing a value, not a supplier** (`registry.add("x", port)` computed too early) — pass a lambda (`() -> port`).
- **Each class with its own `@DynamicPropertySource`** → different cache keys. Put shared containers/stubs and their properties in a base class.

12.13 Full-stack tests on a real port

`FullStackTest` (continued) runs the **whole app on a real Tomcat port** with real H2 and a real HTTP call to the fake rates API, via `RestTestClient` bound to the server:

```
    @LocalServerPort int port;
    @Autowired RestTestClient client;        // bound to the running server in RANDOM_PORT tests
    @Autowired ProductRepository repo;
    @MockitoSpyBean ExchangeRateClient rates; // the REAL bean, wrapped so we can verify calls

    @BeforeEach
    void data() {
        repo.deleteAll();
        repo.save(new Product("MUG-001", "Mug", new BigDecimal("10.00")));
    }

    @Test
    void endToEnd() {
        System.out.println("app on port " + port + ", rates API on port " + RATES.getAddress().getPort());

        PriceView usd = client.get().uri("/api/products/MUG-001/price?currency=USD")
                .exchange().expectStatus().isOk()
                .expectBody(PriceView.class).returnResult().getResponseBody();
        PriceView eur = client.get().uri("/api/products/MUG-001/price")
                .exchange().expectStatus().isOk()
                .expectBody(PriceView.class).returnResult().getResponseBody();

        System.out.println("USD -> " + usd + "\nEUR -> " + eur);
        assertThat(usd.amount()).isEqualByComparingTo("11.00");
        verify(rates, times(1)).rate("EUR", "USD");   // EUR request did not call the API
    }

    @Test
    void createThenRead() {
        client.post().uri("/api/products")
                .contentType(MediaType.APPLICATION_JSON)
                .body("""
                        {"sku":"PEN-001","name":"Pen","price":2.50}""")
                .exchange()
                .expectStatus().isCreated()
                .expectHeader().location("/api/products/PEN-001/price");

        assertThat(repo.findBySku("PEN-001")).isPresent();   // committed for real, no test rollback
        System.out.println("rows after POST = " + repo.count());
    }
```

Output:

```
app on port 51807, rates API on port 51806
USD -> PriceView[sku=MUG-001, name=Mug, amount=11.00, currency=USD]
EUR -> PriceView[sku=MUG-001, name=Mug, amount=10.00, currency=EUR]
rows after POST = 2
```

- `11.00` = 10.00 × 1.10 from the fake server: controller → service → repository → H2, and service → `RestClient` → HTTP → JSON all worked.
- The spy confirms the rule end-to-end: exactly **one** remote call, for USD only.
- `rows after POST = 2` (MUG-001 + PEN-001): data is **committed**, hence `repo.deleteAll()` before each test.
- Random ports let parallel builds on one CI (continuous integration) machine run without clashes.

12.14 Testcontainers and `@ServiceConnection`

> **(needs Docker — not run.)** `testcontainers-demo` has a `.tools_run_skip` marker; its code was **compiled** (`mvn test-compile`) against Boot 4.1.1 and Testcontainers 2.0.5 but not executed.

What it is

Testcontainers is a Java library that starts real services (PostgreSQL, Kafka, Redis, ...) in throw-away **Docker containers** (isolated lightweight boxes started from an **image**) from a test and removes them afterwards. `@ServiceConnection` (Boot 3.1) makes Boot read connection details (host, mapped port, user, password) straight from the container — no copying into properties.

Why it matters

H2 in PostgreSQL mode is still H2. `ILIKE`, `jsonb`, `ON CONFLICT`, partial indexes, `SKIP LOCKED`, exact numeric and collation (text sorting/comparison) behaviour, and PostgreSQL Flyway migrations can only be proven on PostgreSQL. A container gives every developer and CI run the same real version, no shared test server.

How it works

1. `@Testcontainers` (JUnit extension) starts `@Container` fields: **static** once per class, instance fields once per test.
2. The container maps a random free host port to PostgreSQL's 5432.

3. `@ServiceConnection` registers a `JdbcConnectionDetails` bean from the running container; Boot's `DataSource` auto-configuration uses it *instead of* `spring.datasource.*`.
4. After the tests the container stops; Testcontainers' "Ryuk" helper container removes leftovers if the JVM crashes.

Code (not run)

`pom.xml` — note the **Testcontainers 2.x artifact names** (`testcontainers-<module>`; 1.x used plain `postgresql`, `junit-jupiter`):

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-testcontainers</artifactId>
  <scope>test</scope>
</dependency>
<dependency>
  <groupId>org.testcontainers</groupId>
  <artifactId>testcontainers-junit-jupiter</artifactId>
  <scope>test</scope>
</dependency>
<dependency>
  <groupId>org.testcontainers</groupId>
  <artifactId>testcontainers-postgresql</artifactId>
  <scope>test</scope>
</dependency>
```

A repository method that uses PostgreSQL-only SQL:

```
public interface CustomerRepository extends JpaRepository<Customer, Long> {
    // PostgreSQL-only SQL (ILIKE): H2 would not prove this works in production
    @Query(value = "select * from customer where email ilike concat('%', :part, '%')", nativeQuery = true)
    List<Customer> search(String part);
}
```

The test:

```
/** (needs Docker - not run in this book) A real PostgreSQL in a throw-away container. */
@DataJpaTest     // default Replace.NON_TEST: a @ServiceConnection database is a "test" one, so it is kept
@Testcontainers
class CustomerRepositoryPostgresTest {

    @Container                // started before the tests, stopped after (static = once per class)
    @ServiceConnection        // Boot reads host/port/user/password from the container
    static PostgreSQLContainer postgres = new PostgreSQLContainer("postgres:17-alpine");

    @Autowired CustomerRepository repo;

    @Test
    void ilikeWorksOnRealPostgres() {
        repo.save(new Customer("Ada@Example.com"));
        assertThat(repo.search("example")).extracting(Customer::getEmail).containsExactly("Ada@Example.com");
    }
}
```

In Testcontainers 2 the class is `org.testcontainers.postgresql.PostgreSQLContainer`, no longer generic; the 1.x `org.testcontainers.containers.PostgreSQLContainer<?>` still ships, deprecated.

Containers as beans, reused by tests and by local development

```
@TestConfiguration(proxyBeanMethods = false)
public class ContainersConfig {
    @Bean
    @ServiceConnection
    PostgreSQLContainer postgres() {
        return new PostgreSQLContainer("postgres:17-alpine");
    }
}
```

- Tests: `@SpringBootTest @Import(ContainersConfig.class)` — the container is a bean, started with the context and **shared through the context cache**.
- Local development: a `main` in `src/test/java` starts the real app plus containers:

```
public class TestTcApplication {
    public static void main(String[] args) {
        SpringApplication.from(TcApplication::main).with(ContainersConfig.class).run(args);
    }
}
```

Run it from the IDE or `mvn spring-boot:test-run` (Boot 3.1+): new developers get a database without installing one.

`@ServiceConnection` **vs** `@DynamicPropertySource`

	`@ServiceConnection`	`@DynamicPropertySource`
Code	one annotation	`registry.add("spring.datasource.url", postgres::getJdbcUrl)` + user + password
Supported services	those Boot knows: JDBC and R2DBC databases, MongoDB, Redis, Kafka, RabbitMQ, Elasticsearch, Cassandra, Neo4j, ... (Boot reference, "Testcontainers")	anything
Use when	the container is a supported type	custom containers (WireMock, your own image), custom properties

Trade-offs

- **Pro:** production-like behaviour, same version everywhere, no shared test database.
- **Con:** Docker on every machine and CI agent; first run pulls images; each start adds seconds. Mitigate with static/shared containers, the context cache, CI image caching.

Common mistakes

- **One container per test class** without sharing → dozens of PostgreSQL starts. Share via a base class or `@Import(ContainersConfig.class)`.
- `image:latest` → a new major version breaks the build one morning. Pin the production version.
- `@ServiceConnection` **plus** `spring.datasource.url` **in** `application-test.properties` → connection details win; the property silently does nothing. Remove it.

In the interview

Q: How do you test against a real database in Spring Boot? "Testcontainers with `@ServiceConnection`: a static `PostgreSQLContainer` field or a `@TestConfiguration` bean, and Boot takes the JDBC details from the container. I keep it in one shared configuration so the context cache reuses it, and pin the image to the production version. For services Boot doesn't support, `@DynamicPropertySource`."

12.15 Profiles and properties in tests

The tools

Mechanism	Scope	Example
`src/test/resources/application-test.properties` + `@ActiveProfiles("test")`	classes that activate the profile	`FullStackTest` in this chapter
`@SpringBootTest(properties = "k=v")`	one test class	`MockMvcFullContextTest`
`@TestPropertySource(properties = "k=v")` or `(locations = "classpath:x.properties")`	one test class (inherited by subclasses)	`ExchangeRateClientTest`, `C4_OtherPropertyTest`
`@DynamicPropertySource`	one test class, runtime values	`FullStackTest`

Who wins? Measured

`PropertyPrecedenceTest` sets the same keys in several places, using the tiny `@JsonTest` slice (slice annotations also have `properties`):

```
@JsonTest(properties = {"demo.a=from-properties-attribute", "demo.b=from-properties-attribute", "demo.d=from-properties-attribute"})
@TestPropertySource(properties = {"demo.a=from-@TestPropertySource", "demo.c=from-@TestPropertySource", "demo.d=from-@TestPropertySource"})
@ActiveProfiles("test")
class PropertyPrecedenceTest {

    @DynamicPropertySource
    static void dynamic(DynamicPropertyRegistry r) {
        r.add("demo.a", () -> "from-@DynamicPropertySource");
        r.add("demo.b", () -> "from-@DynamicPropertySource");
        r.add("demo.c", () -> "from-@DynamicPropertySource");
    }

    @Value("${demo.a}") String a;
    ...
    @Value("${catalog.home-currency}") String fromProfileFile;   // from application-test.properties
}
```

Output:

```
all three set demo.a   -> from-@DynamicPropertySource
attribute vs dynamic   -> from-@DynamicPropertySource
@TestPropertySource vs dynamic -> from-@DynamicPropertySource
attribute vs @TestPropertySource -> from-@TestPropertySource
application-test.properties    -> catalog.home-currency=EUR
```

Highest first: **`@DynamicPropertySource` → `@TestPropertySource` → the `properties` attribute of `@SpringBootTest`/slices → command-line arguments → system properties → environment variables → `application-{profile}.properties` → `application.properties`**. This matches the Spring Framework reference ("dynamic properties have higher precedence than `@TestPropertySource`"); the Boot reference's list shows `@TestPropertySource` above `@DynamicPropertySource`, but Boot 4.1.1 doesn't behave that way. Full non-test list: Chapter 3.

Common mistakes

- **A `src/test/resources/application.properties`** *shadows* the main file completely (test classpath first), so all main properties vanish in tests. Use `application-test.properties` + `@ActiveProfiles("test")`, which is layered *on top*.
- **A different profile per class** → a new context each time.
- **Production secrets in test profiles** — use fakes, containers or local stubs.

12.16 Common mistakes — checklist

Symptom	Likely cause	Fix
"Unable to find a @SpringBootConfiguration"	test package is not under the main class's package	move the test, or `@SpringBootTest(classes = App.class)`
`No qualifying bean of type ...Service` in `@WebMvcTest`	services aren't in the slice	`@MockitoBean` or `@Import`
`No qualifying bean of type ...MockMvc/RestTestClient/TestRestTemplate` in `@SpringBootTest`	Boot 4 needs explicit opt-in	`@AutoConfigureMockMvc` / `@AutoConfigureRestTestClient` / `@AutoConfigureTestRestTemplate`
`@MockBean` doesn't compile after upgrading to Boot 4	removed	`@MockitoBean` / `@MockitoSpyBean`
Slice loads repositories/services	custom `@ComponentScan` on the main class	move scanning config to a separate `@Configuration`
Test passes, production fails with `LazyInitializationException`	test-managed transaction kept the session open	non-transactional integration test (ch08)
Data from one test visible in the next	real-port / non-transactional test didn't clean up	`@BeforeEach` cleanup, `@Sql` scripts
Suite gets slower every sprint	many unique context configurations, `@DirtiesContext`	base class, slices, cache DEBUG log
Random failures only on CI	fixed ports, shared databases, time or order dependence	`RANDOM_PORT`, containers or embedded DBs, fixed `Clock` bean, no static state

---

12.17 In the interview

Q: How do you test a Spring Boot REST API? "Three levels. Unit tests for the service logic with Mockito, no Spring. A `@WebMvcTest` per controller with `MockMvcTester` and `@MockitoBean` services to check mappings, validation, status codes and Problem Details. A few `@SpringBootTest`s: mock environment with `@AutoConfigureMockMvc` for wiring, and one or two `RANDOM_PORT` tests with `RestTestClient` for the real HTTP path. Repositories get `@DataJpaTest`, against PostgreSQL via Testcontainers when the SQL is database-specific."

- *`@SpringBootTest` vs `@WebMvcTest`?* → Whole context vs web slice only; the slice is faster and needs mocks for non-web beans.
- *What is `MockMvcTester`?* → Framework 6.2's AssertJ-based MockMvc API: `mvc.get().uri(..).exchange()`, `assertThat(result).hasStatusOk().bodyJson()...`.
- *What's new for testing in Spring Boot 4 / Framework 7?* → Per-technology test starters and moved packages; `@MockBean` removed in favour of `@MockitoBean`; explicit `@AutoConfigure...` for test clients; `RestTestClient`; context pausing; JUnit 6.

Q: How does Spring cache test contexts and what breaks the cache? → §12.11: key = merged configuration; mocks, properties, profiles, imports, dynamic properties and `@DirtiesContext` cause new contexts; max 32 (LRU).

Q: How do you test code that calls another service? → Unit: mock the client. Client class: `@RestClientTest` + `MockRestServiceServer`, or a local stub server via `@DynamicPropertySource`. End-to-end: stub server or WireMock container; consumer-driven contract tests (Pact; Spring Cloud Contract has been archived) keep stubs honest (§16.10).

Q: Why might a `@Transactional` test give false confidence? → It rolls back everything, so commit-time behaviour (deferred constraints, `AFTER_COMMIT` listeners, lazy loading outside the session) never runs; in `RANDOM_PORT` tests it doesn't even cover

the server's transaction.

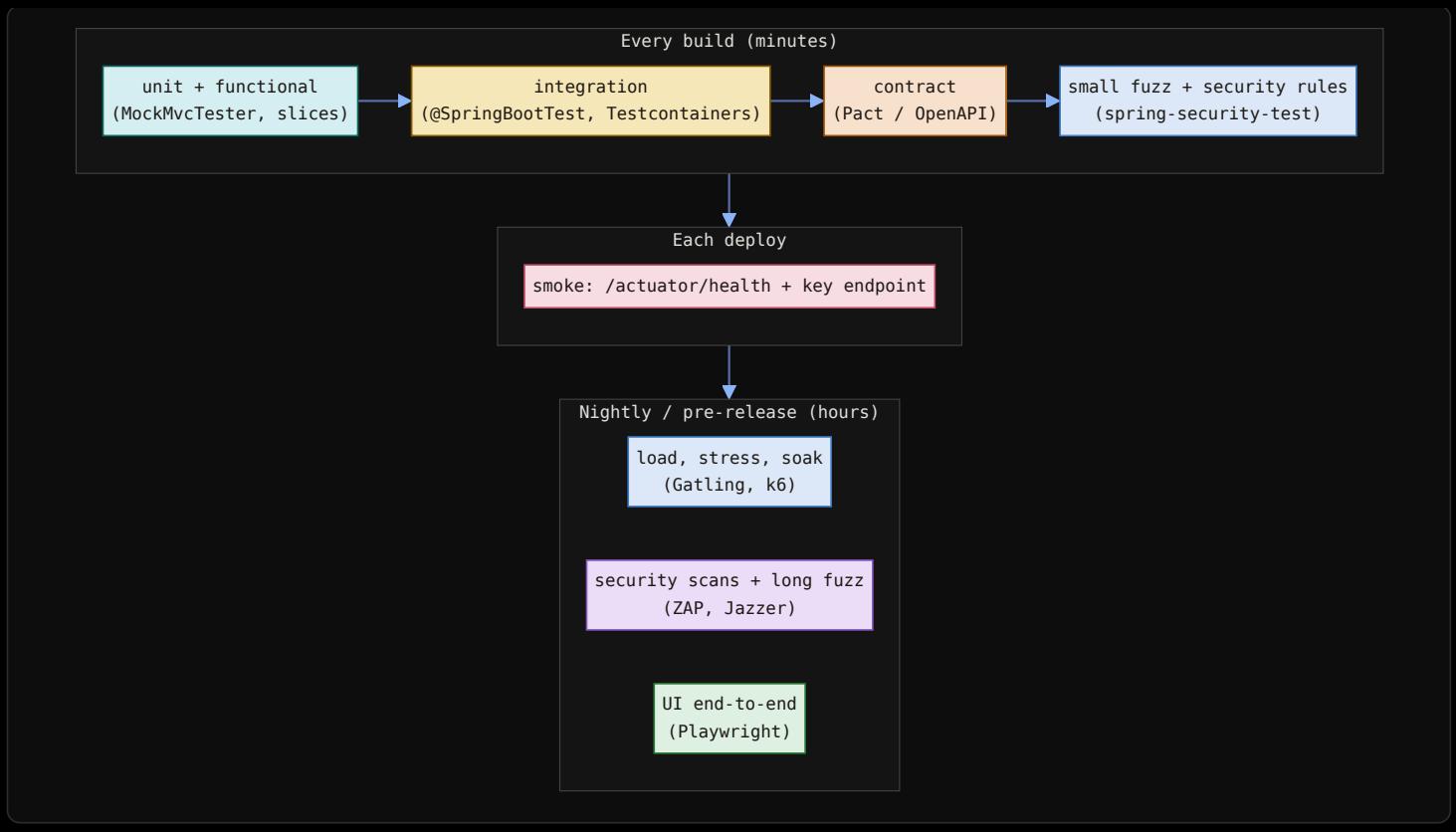
12.18 Types of API testing — and the Spring tool for each

What it is

§12.2 sorted tests by **how much of the container they start**; QA (quality assurance) engineers and interviewers also sort them by **the question they answer** — "does it start?", "does it follow the spec?", "does it survive 5,000 users?", "can an attacker break it?". 400 green `@WebMvcTest`s can still ship an API that falls over at 300 requests/s, breaks the mobile app over a renamed field, or leaks another user's order when an ID in the URL changes. The type names are a checklist of *untested risks*.

How it works — the matrix

Type	Question it answers	When it runs	Spring / Java tool	In this book
Smoke	Does the app start and answer at all?	every build; right after every deploy	one `@SpringBootTest` that loads the context; after deploy, a script hitting `/actuator/health` and one key endpoint	§12.6, ch15 health
Functional	Does each endpoint behave as the spec says (status, body, errors, validation)?	every build	`@WebMvcTest` + `MockMvcTester`, `@MockitoBean` for services	§12.7, §12.8
Integration	Do the real parts work together: DB, SQL, transactions, HTTP clients, messaging?	every build	`@SpringBootTest`, `@DataJpaTest`, `@RestClientTest`, Testcontainers + `@ServiceConnection`	§12.6, §12.7, §12.14
Contract	Do provider and consumers still agree on the API's shape?	provider and consumer builds	consumer-driven contracts (Pact), OpenAPI (machine-readable HTTP API description) schema validation	§16.10
Regression	Did this change break something that used to work?	every build	not a separate tool: the whole automated suite, plus one new test for every bug fixed	whole chapter
Load	Does it meet its latency/throughput goal (SLO, service level objective) at expected peak?	before releases, nightly	Gatling (Java/Scala DSL), k6, JMeter against a production-like environment; watch Micrometer metrics	ch15
Stress / spike / soak	Where does it break and how; does it recover from a sudden burst; does it leak over 8 hours?	before big launches	same tools, different load shape	—
Security	Are unauthenticated users rejected, authorisation rules enforced, inputs sanitised?	every build (+ periodic scans)	`spring-security-test` (`@WithMockUser`, `jwt()`), OWASP ZAP scans, dependency scanning (OWASP Dependency-Check, Dependabot)	10.12, 11.6
Fuzz	Does random, malformed or hostile input ever crash it (5xx, hang, leak)?	every build (small), nightly (large)	a seeded random loop over `MockMvcTester` (below), property-based testing (jqwik: checks a rule holds for many generated inputs), coverage-guided fuzzing (Jazzer: mutates inputs to reach new code paths), OpenAPI-driven fuzzers (Schemathesis)	below
UI / end-to-end	Does a real user journey work through the browser?	before release	Playwright or Selenium against a `RANDOM_PORT` app or staging	§12.13



Load testing checks behaviour *at* the expected peak (e.g. 2,000 requests/s with **p99** — the latency 99% of requests beat — under 200 ms; System Design Fundamentals); **stress** pushes *beyond* it to find the breaking point and check the failure is graceful (429 "Too Many Requests" and **back-pressure** — pushing back on callers — not an out-of-memory crash); **soak** holds normal load for hours to reveal leaks and slow degradation. A load test on a laptop measures the laptop — use a production-sized environment.

Seeing it: a fuzz test — `ProductApiFuzzTest`

The test sends 500 requests to the web slice: `POST /api/products` bodies built from valid and hostile values (empty strings, `null`, wrong types, `1e400`, SQL-injection and `<script>` strings, a 5,000-character SKU, truncated JSON, 200 levels of nesting) and `GET`s with random path segments, asserting one property: **never 5xx**. The `Random` **seed** is fixed so any failure replays exactly (and becomes a regression test). Same configuration as `ProductControllerWebMvcTest`, so no extra context start.

```
Random rnd = new Random(12);                                  // simplified excerpt: POST branch only
for (int i = 0; i < 500; i++) {
    String body = rnd.nextInt(6) == 0 ? pick(rnd, JUNK)
            : "{\"sku\":%s,\"name\":%s,\"price\":%s}".formatted(pick(rnd, SKUS), pick(rnd, NAMES), pick(rnd, PRICES));
    int status = mvc.post().uri("/api/products").contentType(MediaType.APPLICATION_JSON).content(body)
            .exchange().getResponse().getStatus();
    assertThat(status).as("request %d", i).isLessThan(500);   // 4xx is fine; 5xx is a bug
}
```

```
fuzz: 500 requests, status histogram {201=5, 400=395, 404=100}
```

Bean Validation (`@Valid` + constraints like `@NotBlank`), Jackson parse errors and the Problem Details handler turned every bad input into 400 or 404. A 500 would mean a missing validation rule or `@ExceptionHandler`, plus a functional test for that input (a new **regression** test).

When not to use it

- No separate "smoke suite" duplicating functional tests — a context-load test plus a post-deploy health check suffice.
- No load/stress tests on every commit — they need a stable production-like environment and minutes to hours.
- No UI tests for rules a functional test can prove — keep to a handful of critical journeys.

Common mistakes

- **Only happy-path functional tests** → add 400, 401, 403, 404, 409, and "user A reads user B's order" (broken object-level authorisation, OWASP API Security Top 10 #1).
- **Load testing without a goal** → state the target (requests/s, p95/p99 latency, error rate) first.
- **Unseeded random tests** → unreproducible failures; log or fix the seed.
- **Mocking the other service everywhere, never checking the real contract** → green builds, broken integration; add contract tests.

In the interview

> **30-second answer:** "I map test types to risks. Functional tests with `@WebMvcTest` and `MockMvcTester` check each endpoint's contract, including error cases. Integration tests with `@SpringBootTest` and Testcontainers check the real database and wiring. Contract tests with Pact keep services compatible. Security tests use `spring-security-test` for 401/403 rules plus a ZAP scan. A smoke check runs after every deploy. Load, stress and soak tests with Gatling or k6 run before releases against a production-like environment, and I fuzz inputs to make sure nothing returns a 5xx."

- *Regression test?* → Any automated test guarding behaviour that once worked; one new test per fixed bug.
- *What does a fuzz test assert without known expected output?* → Invariants: no 5xx, no hang, no stack trace in the body, response matches the schema.
- *Load vs stress?* → Load checks the SLO at expected peak; stress goes past it to find the breaking point and failure mode.
  Renamed JSON field before the mobile app breaks? → A contract test.

TL;DR

Smoke = does it start; functional = each endpoint per spec (`@WebMvcTest`); integration = real parts together (`@SpringBootTest`, Testcontainers); contract = services agree (Pact); regression = the suite guarding old behaviour; load/stress/soak = at, beyond, over time (Gatling, k6); security and fuzz = hostile users and inputs (`spring-security-test`, ZAP, seeded random inputs); UI = a few browser journeys (Playwright).

Sources: Spring Framework reference (MockMvc, TestContext), Spring Boot reference (testing, Actuator health), OWASP API Security Top 10 (2023), OWASP ZAP docs, Gatling and k6 documentation, Pact documentation, jqwik and Jazzer user guides.

12.19 Chapter TL;DR

- **Pyramid**: many Spring-free unit tests (ms) → slices for layer behaviour (~0.1 s context) → a few `@SpringBootTest`s (~0.7 s context) → very few real-port tests.
- **Boot 4**: test starters per technology (`spring-boot-starter-webmvc-test`, `-data-jpa-test`, `-restclient-test`, ...); slice annotations moved packages; JUnit 6, AssertJ, Mockito inside `spring-boot-starter-test`.
- **TestContext Framework**: `SpringExtension` + listeners + a JVM-wide **context cache** keyed by the merged configuration.
- `@SpringBootTest`: `MOCK` (default), `RANDOM_PORT`, `DEFINED_PORT`, `NONE`; add `@AutoConfigureMockMvc` / `@AutoConfigureRestTestClient` explicitly in Boot 4; `@TestConfiguration` adds test beans.
- **Slices**: `@WebMvcTest` (127 vs 253 beans here), `@DataJpaTest` (embedded DB, rolled back, SQL shown), `@JsonTest`, `@RestClientTest` + `MockRestServiceServer`.
- `MockMvcTester`: AssertJ assertions on MockMvc results; `RestTestClient` (Framework 7.0): one fluent client for MockMvc or a real server.
- `@MockitoBean` / `@MockitoSpyBean` replace the removed `@MockBean` / `@SpyBean`; reset after each test; part of the cache key.
- **Caching demo**: 7 classes, 4 context builds; hits cost 0 ms, misses ~0.5 s. Mocks, properties and `@DirtiesContext` cause misses. Max 32 contexts; Framework 7 **pauses** inactive ones.
- `@DynamicPropertySource`: lazy suppliers for runtime values (random ports), highest precedence.
- **Testcontainers +** `@ServiceConnection`: real PostgreSQL with zero URL config (needs Docker — not run here); TC 2 artifacts `testcontainers-postgresql`; containers as beans also power `mvn spring-boot:test-run`.
- **Test types by goal** (§12.18): smoke, functional, integration, contract, regression, load/stress/soak, security, fuzz, UI — map each risk to a type; fuzz with a fixed seed and assert "never 5xx".
- Next: Chapter 13 — Reactive programming and WebFlux.

Chapter 13 — Reactive Programming and Spring WebFlux

Most Spring applications spend their time **waiting** — for the database, another HTTP service, a **message broker** (Kafka, RabbitMQ: a server storing messages between producers and consumers). In the classic model (Spring MVC on Tomcat) each request holds one **thread** (an independent execution path with its own call stack) from start to finish, waiting included. **Reactive programming** does the opposite: no thread ever waits; work is a pipeline of steps, and a few threads run each step when its data arrives. **Spring WebFlux** is Spring's web framework on that model, using **Project Reactor**. Since Java 21 a second route to the same **scalability** exists: **virtual threads** (lightweight JVM-managed threads) make waiting cheap instead of avoiding it. This chapter teaches the reactive model, then **measures** blocking MVC on virtual threads against WebFlux.

All code is in `code/`. Spring Boot **4.1.1** (Framework 7.0, Reactor 3.8 from the Reactor 2025.0 release train — library versions released together, Reactor Netty — a non-blocking server on Netty, R2DBC H2 1.1), Java 25:

Project	Shows
`reactor-basics`	Reactor only (no Spring context): creation, operators, errors, virtual time, backpressure, schedulers, sinks, context — all as `StepVerifier` tests
`webflux-app`	a book API: annotated controller, functional routes, SSE, `WebClient`, R2DBC repository + `DatabaseClient` + reactive transaction
`load-mvc`, `load-webflux`	the same I/O-bound endpoint (spends its time waiting on I/O, not computing) under load: MVC on platform threads (ordinary OS threads), MVC on virtual threads, WebFlux

Output blocks are real `mvn -q -B -ntp verify` output (Spring log lines trimmed to `...`). **All timings are machine-dependent** (14-core Apple M4 Pro, load generator in the same JVM as the server).

> **Prerequisites and links.** The mental model is Node's: JavaScript handbook Chapter 12 — Asynchronous JavaScript ("The event loop", "Promises"). A `Mono` is roughly a lazy, cancellable Promise; a `Flux` a lazy async stream. Virtual threads and `CompletableFuture` (Java's Promise): Java handbook Chapter 15. Spring MVC: Chapter 5; blocking `RestClient`: Chapter 6; Spring testing: Chapter 12.

13.1 Why reactive? The cost of waiting

The problem it solves

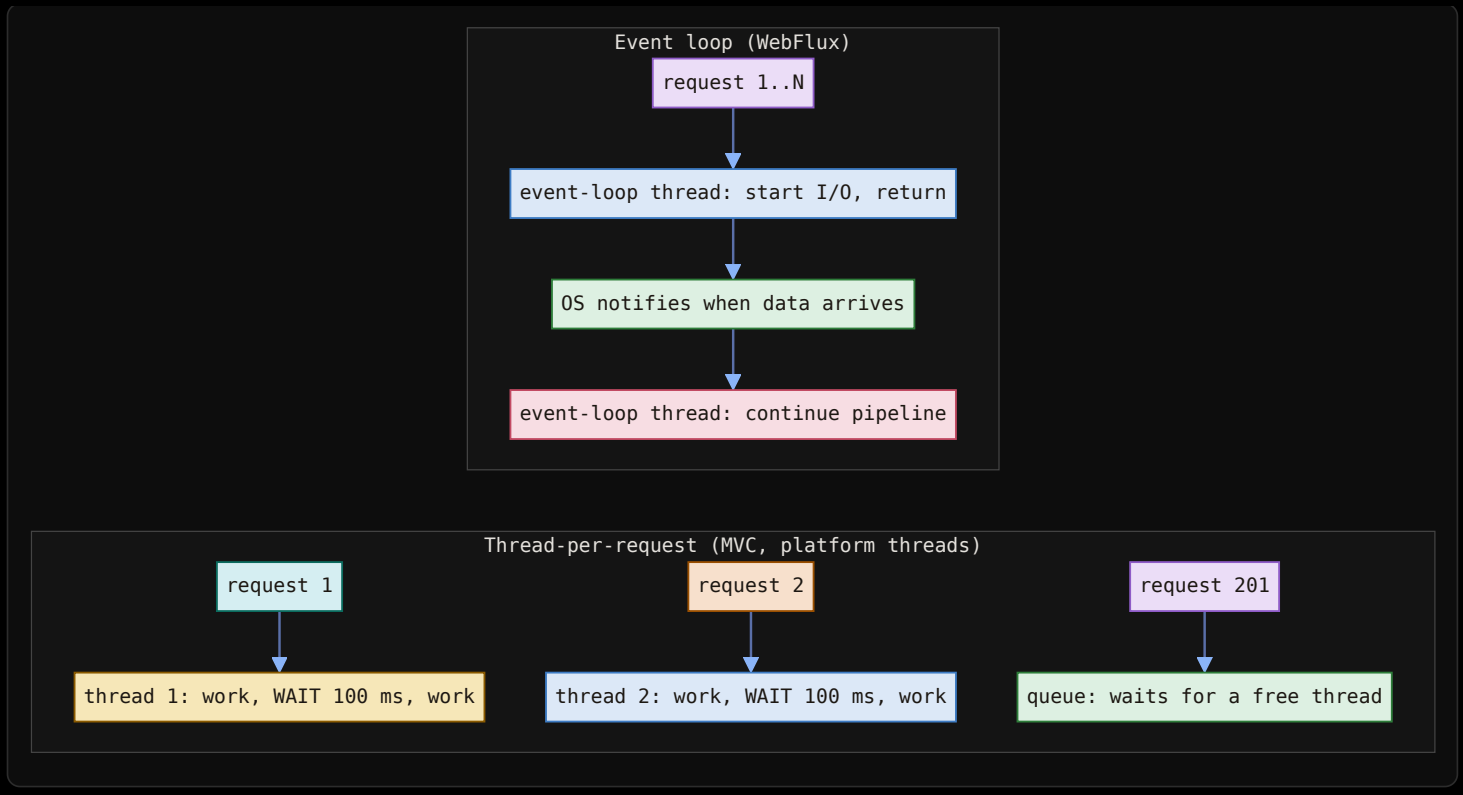
In Spring MVC a request is served by **one thread from start to finish** (*thread-per-request*). If the handler waits 100 ms for the database, the thread is unavailable to anyone else for those 100 ms. Tomcat's worker pool has **200 threads** by default, so at most 200 requests are *in progress* (in flight) at once, and with 100 ms of waiting each the server tops out near **200 / 0.1 s = 2,000 requests per second**, however idle the CPU. More platform threads cost memory (each **stack** reserves up to ~1 MB) and OS scheduling overhead, so "just add threads" stops working at a few thousand.

What reactive does instead

I/O is reading or writing outside the program (network, disk, database). **Blocking** I/O stops the calling thread until data arrives; non-blocking I/O never parks a thread on a **socket** (an open network connection):

1. The handler starts the call and registers **what to do when the answer arrives**.
2. The thread immediately returns to the **event loop** for other work.
3. When the OS reports data ready (epoll/kqueue on Linux/macOS), an event-loop thread runs the next pipeline step.

A handful of event-loop threads (one per core by default in Reactor Netty) keep thousands of requests in progress — like a waiter who hands the order to the kitchen and serves other tables until the bell rings. This is exactly Node.js: the JavaScript event-loop rules (never block the loop; **callbacks** run when I/O completes; one slow **synchronous** function freezes everyone) apply to WebFlux too.



The price

- Code becomes a **pipeline of operators**. Stack traces, debugging and `ThreadLocal`-based libraries (per-thread variables: MDC logging — the Mapped Diagnostic Context adding fields like a request id to log lines — security context, transactions) need reactive-aware replacements.
- **Everything** on the request path must be non-blocking: server, HTTP client, database driver. One blocking call on an event loop stalls every request that loop serves (§13.15 measures this).
- Since Java 21, **virtual threads** remove the thread-count limit for ordinary blocking code — removing the main *performance* reason to go reactive for many apps.

13.2 Reactive Streams: the contract

What it is

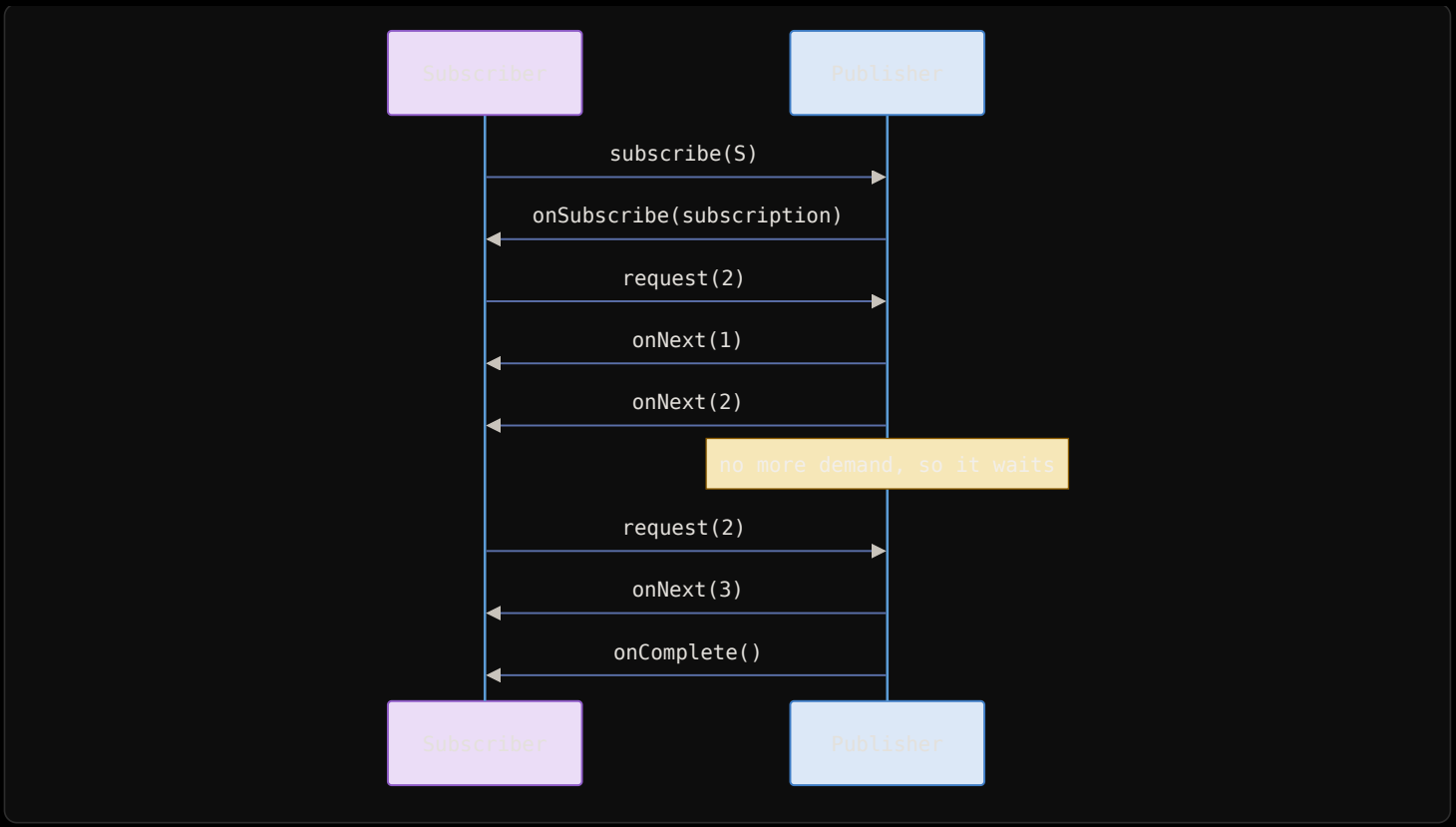
Reactive Streams (1.0, 2015) is a small specification for **asynchronous stream processing with non-blocking backpressure** (*asynchronous*: the caller doesn't wait; the result comes later). Four interfaces, copied into Java 9 as `java.util.concurrent.Flow`:

Interface	Role	Methods
`Publisher<T>`	produces items	`subscribe(Subscriber)`
`Subscriber<T>`	consumes items	`onSubscribe(Subscription)`, `onNext(T)`, `onError(Throwable)`, `onComplete()`
`Subscription`	the link between the two	`request(long n)`, `cancel()`
`Processor<T,R>`	both a subscriber and a publisher	—

Backpressure: the **consumer controls the flow rate** — the publisher sends at most as many items as requested, so a slow consumer is never flooded.

How it works

1. `publisher.subscribe(subscriber)` — nothing flows yet.
2. The publisher calls `subscriber.onSubscribe(subscription)`.
3. The subscriber calls `subscription.request(n)`, meaning "I can take n more".
4. The publisher sends up to n `onNext` signals, then waits for more demand.
5. The stream ends with at most one terminal signal — `onComplete` **or** `onError` (none if cancelled or infinite) — or the subscriber calls `cancel()`.



You rarely implement these yourself: **Project Reactor** (Spring's choice), RxJava, Mutiny, Akka Streams and drivers such as R2DBC do, and interoperate through the shared contract.

13.3 Project Reactor: `Mono` and `Flux`

What it is

Project Reactor implements Reactive Streams with two publisher types and hundreds of operators:

Type	Emits	Like	Examples
`Mono<T>`	0 or 1 item, then complete (or an error)	a lazy `CompletableFuture<Optional<T>>` / a lazy Promise	find by id, save, an HTTP response
`Flux<T>`	0..N items (possibly infinite), then complete (or an error)	a lazy async `Stream<T>` / async iterator	query results, SSE events, Kafka records

An **operator** (`map`, `filter`, `flatMap`, ...) returns a *new* publisher wrapping the previous one; chaining **assembles a pipeline**. Nothing runs until something **subscribes**.

Creating publishers — `CreationAndLazinessTest`

```
StepVerifier.create(Mono.just("one")).expectNext("one").verifyComplete();
StepVerifier.create(Mono.empty()).verifyComplete();                    // 0 items, then complete
StepVerifier.create(Flux.range(1, 3)).expectNext(1, 2, 3).verifyComplete();
StepVerifier.create(Flux.fromIterable(List.of("a", "b"))).expectNext("a", "b").verifyComplete();
StepVerifier.create(Flux.error(new IllegalStateException("boom")))
        .expectErrorMessage("boom").verify();
```

Other factories: `Mono.fromCallable(() -> ...)` (wrap a computation), `Mono.fromFuture(cf)`, `Flux.interval(Duration)` (ticks on the parallel scheduler — a **scheduler** decides which threads run work, §13.8), `Flux.create(sink -> ...)` (bridge callback APIs), `Mono.delay(Duration)`.

Nothing happens until you subscribe

```
@Test
void nothingHappensUntilYouSubscribe() {
    AtomicInteger calls = new AtomicInteger();
    Flux<Integer> pipeline = Flux.range(1, 3)
            .map(i -> { calls.incrementAndGet(); return i * 10; });

    System.out.println("after assembling the pipeline, map ran " + calls.get() + " times");

    List<Integer> out = pipeline.collectList().block();   // block() subscribes (fine in a test)
    System.out.println("after subscribing: " + out + ", map ran " + calls.get() + " times");

    pipeline.blockLast();                                 // cold: a 2nd subscriber re-runs everything
    System.out.println("after a second subscriber, map ran " + calls.get() + " times");
}

@Test
void justIsEagerDeferIsLazy() {
    AtomicInteger dbHits = new AtomicInteger();
    Mono<Integer> eager = Mono.just(dbHits.incrementAndGet());                // runs NOW, once
    Mono<Integer> lazy  = Mono.defer(() -> Mono.just(dbHits.incrementAndGet())); // runs per subscribe
    System.out.println("hits after building both monos: " + dbHits.get());
    eager.block(); eager.block(); lazy.block(); lazy.block();
    System.out.println("hits after 2x eager + 2x lazy subscribe: " + dbHits.get());
}
```

Output:

```
just/empty/range/fromIterable/error verified
after assembling the pipeline, map ran 0 times
after subscribing: [10, 20, 30], map ran 3 times
after a second subscriber, map ran 6 times
hits after building both monos: 1
hits after 2x eager + 2x lazy subscribe: 3
```

- **Assembly vs subscription.** Building the chain does nothing; subscribing runs it. In WebFlux **the framework subscribes** to the `Mono`/`Flux` your controller returns. A pipeline neither returned nor subscribed never runs — the classic bug: a controller calls `repo.save(book);` and returns `Mono.just(book)`, saving nothing. Return `repo.save(book)`.
- **Cold publishers** replay the whole pipeline per subscriber: two subscribers = two database queries.
- **`Mono.just(expr)` evaluates `expr` immediately** at assembly, even if nobody subscribes. Use `Mono.defer(() -> ...)` or `Mono.fromCallable(...)` for per-subscription work.

`block()`

`block()` subscribes and **waits**. Fine in tests, `main` methods and at the edge of a non-reactive app; inside a pipeline or on an event-loop thread it is a bug: Reactor throws `IllegalStateException("block()/blockFirst()/blockLast() are blocking, which is not supported in thread reactor-http-nio-...")`.

13.4 Operators

The ones you need daily

Goal	Operator	Note
transform each item (sync)	`map`	must not block or return a publisher
keep some items	`filter`, `take(n)`, `skip(n)`, `distinct`	
async call per item	**`flatMap`**	runs inner publishers **concurrently**, output **interleaved**
async call per item, in order, one at a time	**`concatMap`**	sequential; slower
async per item, concurrent, output in source order	**`flatMapSequential`**	buffers early results
combine by position	`zip`, `zipWith`	waits for one from each
one after another	`concat`, `concatWith`	
as they come	`merge`, `mergeWith`	
fold to one value	`reduce`, `count`, `collectList`, `collectMap`	returns `Mono`
fallback if empty	`switchIfEmpty`, `defaultIfEmpty`	common for 404s
group	`buffer(n)`, `window(n)`, `groupBy`	
side effects (logging, metrics)	`doOnNext`, `doOnError`, `doOnSubscribe`, `doFinally`	don't put business logic here
ignore values, wait for completion	`then()`, `then(mono)`	"do A, then B"

`flatMap` vs `concatMap` vs `flatMapSequential` — `OperatorsTest`

Three users; fetching the orders of user N takes (4 − N) seconds per order, so user 3 answers first:

```
/** Simulated remote call: user N answers after (4 - N) seconds, so later users answer first. */
static Flux<String> ordersOf(int user) {
    return Flux.just("u" + user + "-a", "u" + user + "-b")
            .delayElements(Duration.ofSeconds(4 - user));
}

@Test
void flatMapInterleavesConcatMapKeepsOrder() {
    // withVirtualTime: the delays run on a fake clock, so this 12-"second" test takes milliseconds
    StepVerifier.withVirtualTime(() -> Flux.range(1, 3).flatMap(OperatorsTest::ordersOf).collectList())
            .thenAwait(Duration.ofHours(1))
            .consumeNextWith(l -> System.out.println("flatMap           : " + l))
            .verifyComplete();
    // ... same with concatMap and flatMapSequential
}

@Test
void howLongDoTheyTake() {
    // concatMap: user 1 (3 s + 3 s), then user 2 (2 s + 2 s), then user 3 (1 s + 1 s) = 12 s
    var concat = StepVerifier.withVirtualTime(() -> Flux.range(1, 3).concatMap(OperatorsTest::ordersOf))
            .expectSubscription()
            .thenAwait(Duration.ofSeconds(11)).expectNextCount(5)
            .expectNoEvent(Duration.ofMillis(999))
            .thenAwait(Duration.ofMillis(1)).expectNext("u3-b").verifyComplete();
    // flatMap: all three run at the same time, the slowest (user 1) needs 6 s
    var flat = StepVerifier.withVirtualTime(() -> Flux.range(1, 3).flatMap(OperatorsTest::ordersOf))
            .expectSubscription()
            .thenAwait(Duration.ofSeconds(5)).expectNextCount(5)
            .thenAwait(Duration.ofSeconds(1)).expectNext("u1-b").verifyComplete();
    System.out.println("concatMap needs 12 virtual s, flatMap 6 virtual s; real time: "
            + concat.toMillis() + " ms and " + flat.toMillis() + " ms");
}
```

Output:

```
flatMap           : [u3-a, u2-a, u3-b, u1-a, u2-b, u1-b]
concatMap         : [u1-a, u1-b, u2-a, u2-b, u3-a, u3-b]
flatMapSequential : [u1-a, u1-b, u2-a, u2-b, u3-a, u3-b]
concatMap needs 12 virtual s, flatMap 6 virtual s; real time: 0 ms and 0 ms
```

- `flatMap` subscribes to all inner publishers at once (up to `concurrency`, default 256) and emits **as results arrive**: fastest (time of the slowest, 6 s), order lost.

- `concatMap` finishes each inner publisher before starting the next: order kept, time = sum (12 s).
- `flatMapSequential` runs concurrently (6 s) but re-orders output to the source order.
- Limit fan-out against a fragile service with `flatMap(fn, 8)`: at most 8 calls in flight (e.g. 1,000 ids, 10 at a time: `Flux.fromIterable(ids).flatMap(this::call, 10)`).

Combining

```
Flux<String> names = Flux.just("Ada", "Linus", "Grace");
Flux<Integer> years = Flux.just(1815, 1969, 1906);
StepVerifier.create(Flux.zip(names, years, (n, y) -> n + "@" + y))
        .expectNext("Ada@1815", "Linus@1969", "Grace@1906").verifyComplete();

StepVerifier.create(Flux.concat(Flux.just(1, 2), Flux.just(3))).expectNext(1, 2, 3).verifyComplete();

Mono<String> fromCache = Mono.empty();
StepVerifier.create(fromCache.switchIfEmpty(Mono.just("from DB")))      // fallback when empty
        .expectNext("from DB").verifyComplete();

StepVerifier.create(Flux.range(1, 7).buffer(3))
        .expectNext(List.of(1, 2, 3), List.of(4, 5, 6), List.of(7)).verifyComplete();
```

`Mono.zip(a, b, c)` is the reactive "call three services in parallel and combine" — `Promise.all` in JavaScript, `CompletableFuture.allOf` in Java.

Common mistakes

- **`map` with an async call**: `flux.map(id -> repo.findById(id))` yields a `Flux<Mono<Book>>` whose inner Monos never run. Use `flatMap`.
- **`flatMap` when order matters** (applying events in sequence) — use `concatMap`.
- **Unbounded `flatMap` against a rate-limited API** — pass a concurrency limit.
- **Business logic in `doOnNext`** — it's for side effects; an exception there becomes an error signal, and it's easy to forget it isn't part of the data flow.

13.5 Testing with `StepVerifier` and virtual time

What it is

`StepVerifier` (in `reactor-test`) subscribes to a publisher and checks its signals step by step: `expectNext(...)`, `expectNextCount(n)`, `assertNext(v -> ...)`, `expectError(Type)`, `expectComplete()`, `thenRequest(n)`, `thenCancel()`. `verify()` (or `verifyComplete()` / `verifyError()`) **runs** the subscription, **blocks** until the script is done, and returns the time taken.

Virtual time replaces Reactor's time-based schedulers with a `VirtualTimeScheduler` whose clock moves only when told (`thenAwait(Duration)`) or jumps when idle. A one-hour interval or a retry with **exponential backoff** (1 s, 2 s, 4 s, ...) is tested in milliseconds, **deterministically**.

How to use virtual time

1. Build the publisher **inside the supplier**: `StepVerifier.withVirtualTime(() -> Flux.interval(...))`, so its time-based operators pick up the virtual scheduler; a publisher created outside keeps real time.
2. Use `expectSubscription()` first, then `expectNoEvent(d)` to assert that nothing happened for a period (it also advances the clock).
3. Use `thenAwait(d)` to move the clock forward, then `expectNext(...)`.

The trap: a test that waits forever

If the script expects a signal that only comes after virtual time passes but never calls `thenAwait`, `verify()` waits **forever** — this chapter's first `howLongDoTheyTake` hung the build until an external alarm killed it. Two defences:

```
# src/test/resources/junit-platform.properties
# safety net: a reactive test that waits for a signal that never comes fails instead of hanging
junit.jupiter.execution.timeout.default=30 s
```

and `verify(Duration.ofSeconds(5))` on real-time scripts (used throughout `webflux-app`).

`StepVerifier.create(flux, 0)` subscribes with initial demand 0, so you drive demand with `thenRequest(n)` (§13.7).

13.6 Error handling

How errors flow

An exception inside an operator becomes an `onError` **signal** that travels down, skipping `map`/`filter`, until an error operator handles it or it reaches the subscriber. **An error terminates the sequence** — later items are never processed (like a rejected Promise, not a `try/catch` inside a loop).

```java
@Test
void anErrorEndsTheStream() {
    StepVerifier.create(Flux.just(1, 2, 0, 4).map(i -> 12 / i))
            .expectNext(12, 6)
            .expectError(ArithmeticException.class)   // 4 is never processed
            .verify();
}

@Test
void fallbacks() {
    Flux<Integer> risky = Flux.just(1, 2, 0, 4).map(i -> 12 / i);
    StepVerifier.create(risky.onErrorReturn(-1)).expectNext(12, 6, -1).verifyComplete();
    StepVerifier.create(risky.onErrorResume(e -> Flux.just(100, 200))).expectNext(12, 6, 100, 200).verifyComplete();
    StepVerifier.create(risky.onErrorMap(e -> new IllegalArgumentException("bad input", e)))
            .expectNext(12, 6).expectErrorMessage("bad input").verify();
}
```

Operator	Effect	Imperative equivalent
`onErrorReturn(v)`	emit a fallback value, then complete	`catch → return v`
`onErrorResume(e -> publisher)`	switch to another publisher (cache, default, another service)	`catch → return other()`
`onErrorMap(e -> new X(e))`	translate the exception	`catch → throw new X(e)`
`doOnError(e -> log)`	side effect, error continues	`catch → log; throw`
`retry(n)`	**re-subscribe** to the source up to n times	loop around the call
`retryWhen(Retry.backoff(...))`	retry with delays, filters, limits	retry library
`timeout(d[, fallback])`	error (or fallback) if no signal within d	`future.get(timeout)`
`onErrorContinue`	drop the failing item and continue (only with supporting operators)	avoid — surprising semantics

Retry with exponential backoff, verified on virtual time

```java
@Test
void retryWithExponentialBackoffOnVirtualTime() {
    List<Long> attemptTimes = new ArrayList<>();
    StepVerifier.withVirtualTime(() -> {
                VirtualTimeScheduler clock = VirtualTimeScheduler.get();
                return Mono.defer(() -> {
                            attemptTimes.add(clock.now(TimeUnit.MILLISECONDS));
                            return Mono.<String>error(new IllegalStateException("down"));
                        })
                        .retryWhen(Retry.backoff(3, Duration.ofSeconds(1)).jitter(0)); // 1 s, 2 s, 4 s
            })
            .thenAwait(Duration.ofMinutes(1))
            .expectErrorMatches(e -> e.getMessage().startsWith("Retries exhausted: 3/3"))
            .verify();
    System.out.println("attempts at virtual ms: " + attemptTimes);
}

@Test
void timeoutWithFallback() {
    StepVerifier.withVirtualTime(() -> Mono.delay(Duration.ofSeconds(5)).map(x -> "slow answer")
                    .timeout(Duration.ofSeconds(2), Mono.just("fallback")))
            .expectSubscription()
            .thenAwait(Duration.ofSeconds(2))
            .expectNext("fallback")
            .verifyComplete();
    ...
}
```

Output:

```
attempts at virtual ms: [0, 1000, 3000, 7000]
retry(5): succeeded after 3 attempts
timeout(2s) on a 5 s call -> fallback / TimeoutException
onErrorReturn / onErrorResume / onErrorMap verified
```

Attempt 1 runs at 0; retries wait 1 s, 2 s, 4 s (1,000, 3,000, 7,000 ms); then `Retries exhausted: 3/3`. Real systems keep the default **jitter** (random ±50 % here) so clients don't retry in lockstep; `jitter(0)` is for the deterministic test. Retry and circuit-breaker (stop calling a failing service for a while) design: microservices notes — retry.

Retry re-subscribes, which repeats work only for a lazy source (`Mono.defer(...)`, `fromCallable`, `WebClient` calls); `Mono.just(valueComputedEarlier)` just replays the value.

Common mistakes

- **Retrying non-idempotent calls** (*idempotent*: same effect once or many times; a card-charging `POST` is not) without an idempotency key (a unique id the server uses to ignore duplicates; System Design Fundamentals).
- **Retrying on 4xx** (client errors fail again). Filter to transient errors (e.g. 503): `Retry.backoff(...).filter(e -> e instanceof ServiceUnavailable)`.
- `try/catch` **around a reactive chain** catches only assembly-time errors, not those raised when the pipeline runs.

13.7 Backpressure in practice

What it is

Backpressure = the consumer tells the producer how much it can handle via `request(n)`. Most Reactor operators pass demand **upstream** (towards the source; *downstream* is towards the subscriber) automatically; `block()` or WebFlux's HTTP writer request in batches.

Demand you can see — `BackpressureTest`

```
@Test
void subscriberControlsTheRate() {
    List<Long> requests = new CopyOnWriteArrayList<>();
    List<Integer> received = new ArrayList<>();

    Flux.range(1, 10)
        .doOnRequest(requests::add)                 // what the subscriber asked for
        .subscribe(new BaseSubscriber<Integer>() {
            @Override protected void hookOnSubscribe(Subscription s) { request(2); }  // start with 2
            @Override protected void hookOnNext(Integer v) {
                received.add(v);
                if (received.size() % 2 == 0) request(2);                          // then 2 at a time
            }
        });
    System.out.println("requests = " + requests + "\nreceived = " + received);
}

@Test
void limitRateReplenishesInBatches() {
    List<Long> upstreamRequests = new CopyOnWriteArrayList<>();
    Flux.range(1, 100)
        .doOnRequest(upstreamRequests::add)
        .limitRate(10)                              // never ask upstream for more than 10
        .blockLast();                               // downstream asks for "unbounded"
    System.out.println("limitRate(10) upstream requests = " + upstreamRequests);
}

@Test
void stepVerifierCanRequestManually() {
    StepVerifier.create(Flux.range(1, 5), 0)          // initial demand 0
            .expectSubscription()
            .expectNoEvent(Duration.ofMillis(50))     // nothing arrives without demand
            .thenRequest(2).expectNext(1, 2)
            .thenRequest(3).expectNext(3, 4, 5)
            .verifyComplete();
}
```

Output:

```
limitRate(10) upstream requests = [10, 8, 8, 8, 8, 8, 8, 8, 8, 8, 8, 8, 8]
demand 0 -> nothing; request(2) -> 1,2; request(3) -> 3,4,5
requests = [2, 2, 2, 2, 2, 2]
received = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

- The source sent exactly as many items as requested, 2 at a time; with demand 0 a well-behaved publisher sends nothing.
- `limitRate(10)` turns a downstream "everything" (`Long.MAX_VALUE`) into bounded requests: 10 first, then **8 more when 75 % are consumed** (default "low tide"), so the next batch is in flight before the buffer runs dry. Use it to protect paged sources (a database cursor walking results in batches, a paged API).

When the source can't slow down: overflow strategies

Some sources **ignore demand** (UI events, sensors, a price ticker, a hot message stream); you choose what happens to the excess. The test's `ticker()` pushes 1..20 at once through `Flux.create(..., OverflowStrategy.IGNORE)` while the consumer asks for 5:

```java
static Flux<Integer> ticker() {
    return Flux.create(sink -> {
        for (int i = 1; i <= 20; i++) sink.next(i);
        sink.complete();
    }, FluxSink.OverflowStrategy.IGNORE);   // IGNORE = no built-in protection, the operator below decides
}

StepVerifier.create(ticker().onBackpressureDrop(dropped::add), 5).expectNext(1, 2, 3, 4, 5).verifyComplete();

SlowConsumer latest = new SlowConsumer();              // requests 5 on subscribe
ticker().onBackpressureLatest().subscribe(latest);     // 20 items pushed during this call
latest.request(5);                                     // ask for more afterwards

SlowConsumer buffered = new SlowConsumer();
ticker().onBackpressureBuffer(10).subscribe(buffered); // 6..15 fit in the buffer, 16 overflows
buffered.request(50);
```

Output:

```
DROP   : kept 1..5, dropped 15 items (6..20)
LATEST : [1, 2, 3, 4, 5, 20] complete
BUFFER(10): [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15] error: The receiver is overrun by more signals than expected (bounded queue...)
```

Strategy	What happens to excess	Use for
`onBackpressureLatest`	keep only the newest waiting item	"current price" (a feed faster than the UI renders), UI state
`onBackpressureBuffer(n)`	queue up to n, then **error** (or a chosen overflow action)	short bursts that must not be lost
`onBackpressureBuffer()` (unbounded)	queue everything	risky: a consumer slower than the producer grows it until `OutOfMemoryError`
`onBackpressureError`	fail immediately	when overflow is a bug
`onBackpressureError`	fail immediately	when overflow is a bug

`Flux.create` accepts the same choices directly (`OverflowStrategy.DROP/LATEST/BUFFER/ERROR`).

Backpressure across the network

Inside one JVM demand is exact; across HTTP it becomes **TCP flow control** (TCP stops the sender when the receiver's buffer is full; System Design Fundamentals). When WebFlux writes a `Flux` to a slow client, the socket send buffer fills, Netty stops requesting, and demand stops at the source (e.g. the R2DBC cursor). Between services HTTP/1.1 has no message-level backpressure; RSocket or gRPC (binary protocols with flow control), or a broker in between, provide it.

13.8 Schedulers and threads

What it is

Reactor is **concurrency-agnostic**: each signal runs on whatever thread emitted it (subscriber, Netty event loop, timer). **Schedulers** move work onto specific pools:

Scheduler	Threads	For
`Schedulers.parallel()`	one per CPU core	CPU-bound work, timers (`interval`, `delayElements`)
`Schedulers.boundedElastic()`	grows on demand up to 10 × cores (default), idle threads reclaimed	**wrapping blocking calls** (JDBC, legacy SDKs, file I/O)
`Schedulers.single()`	one reusable thread	low-latency one-off tasks
`Schedulers.immediate()`	the current thread	tests, defaults
Reactor Netty event loops (`reactor-http-nio-N`)	one per core (min 4)	network I/O — **never block these**

- `publishOn(s)` switches the thread for everything **after** it in the chain.
- `subscribeOn(s)` chooses the thread on which the **subscription happens**, moving the **source** (and everything up to the first `publishOn`); its position in the chain doesn't matter.

```java
@Test
void publishOnSwitchesDownstreamSubscribeOnMovesTheSource() {
    List<String> log = new CopyOnWriteArrayList<>();
    Flux.just(1)
        .doOnNext(i -> log.add("source    on " + thread()))
        .publishOn(Schedulers.parallel())               // everything BELOW runs on parallel
        .doOnNext(i -> log.add("after pub on " + thread()))
        .subscribeOn(Schedulers.boundedElastic())       // the subscription (and source) runs here
        .blockLast();
    log.forEach(System.out::println);
}

@Test
void wrapABlockingCall() {
    // A blocking API (JDBC, legacy SDK) must not run on the few event-loop/parallel threads.
    Mono<String> legacy = Mono.fromCallable(() -> {
                Thread.sleep(50);                       // pretend: blocking I/O
                return "legacy result on " + thread();
            })
            .subscribeOn(Schedulers.boundedElastic());   // dedicated, growable pool for blocking work
    System.out.println(legacy.block());
}
```

Output (thread numbers replaced by `N`):

```
legacy result on boundedElastic-N
source    on boundedElastic-N
after pub on parallel-N
```

`Mono.fromCallable(blocking).subscribeOn(Schedulers.boundedElastic())` is how to call unavoidable blocking code from a reactive app: event loops stay free, at the cost of a thread per concurrent call — exactly what reactive meant to avoid. If most calls block, a reactive stack buys nothing. **`BlockHound`** (a Java agent from the Reactor team) detects blocking calls on non-blocking threads in tests.

13.9 Hot vs cold, sinks and context

Cold vs hot

- **Cold** (default): each subscriber gets its own run from the start (a query, a `WebClient` call, `Flux.range`).
- **Hot**: data is produced whether or not anyone listens; late subscribers **miss** earlier items (stock ticks, mouse events, a chat room).

Sinks (`Sinks.many()`, `Sinks.one()`) are the safe way to push values into a `Flux` from **imperative** code, e.g. broadcasting events to all SSE clients:

```java
Sinks.Many<String> prices = Sinks.many().multicast().directBestEffort();
prices.asFlux().subscribe(early::add);
prices.tryEmitNext("p1");
prices.asFlux().subscribe(late::add);   // joins after p1 was emitted
prices.tryEmitNext("p2");
prices.tryEmitComplete();
```

Output:

```
early subscriber: [p1, p2], late subscriber: [p2]
```

Other sinks: `Sinks.many().replay().limit(n)` (late subscribers get the last n), `unicast()` (one subscriber, buffers). Make a cold publisher hot with `share()`, `publish().autoConnect()`, or `cache()` (replays to later subscribers).

Context: the reactive replacement for `ThreadLocal`

A pipeline may hop threads, so `ThreadLocal` values (MDC request id, security context, current transaction) get lost. Reactor's **`Context`** is an immutable key/value map attached to the **subscription**: written at the bottom with `contextWrite`, read anywhere above with `deferContextual`:

```java
Mono<String> greeting = Mono.deferContextual(ctx -> Mono.just("request " + ctx.get("requestId")));
StepVerifier.create(greeting.contextWrite(c -> c.put("requestId", "r-42")))
        .expectNext("request r-42").verifyComplete();
```

Output:

```
context value read inside the pipeline: r-42
```

Spring uses it for you: reactive Spring Security stores the `SecurityContext` there, reactive transactions keep the connection there. For MDC and **tracing** (following a request across threads and services, Chapter 15), Boot bridges `ThreadLocal`s and the Reactor `Context` via Micrometer's context-propagation library (`spring.reactor.context-propagation=auto`).

13.10 Spring WebFlux: annotated controllers

What it is

Spring WebFlux (Framework 5.0+) is the reactive counterpart of Spring MVC, on **Reactor Netty** by default (Tomcat and Jetty work via the Servlet non-blocking I/O API). The flow mirrors MVC: `HttpHandler` → `WebFilter`s → **`DispatcherHandler`** → `HandlerMapping` → `HandlerAdapter` → your handler → `HttpMessageWriter` (Jackson 3 **codecs**: byte ↔ object encoders/decoders), all non-blocking.

	Spring MVC	Spring WebFlux
I/O model	blocking (one thread per request) or virtual threads	non-blocking event loop
Server	Tomcat / Jetty (Servlet)	Reactor Netty (default), Tomcat, Jetty
Return types	`T`, `ResponseEntity<T>` (also accepts `Mono`/`Flux`)	`Mono<T>`, `Flux<T>`, `T`
Data access	JDBC, JPA	R2DBC, reactive MongoDB/Redis/Cassandra
HTTP client	`RestClient`	`WebClient`
Test client	`MockMvcTester`, `RestTestClient` (ch12)	`WebTestClient`
Starter (Boot 4)	`spring-boot-starter-webmvc`	`spring-boot-starter-webflux`

If **both** starters are on the classpath, Boot starts **MVC**. Force WebFlux with `spring.main.web-application-type=reactive`.

`webflux-app/pom.xml` (main parts):

```
<!-- Spring WebFlux + Reactor Netty server + Jackson 3 -->
<dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-webflux</artifactId></dependency>
<!-- Boot 4: the auto-configured WebClient.Builder lives in its own module/starter -->
<dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-webclient</artifactId></dependency>
<!-- Spring Data R2DBC + the reactive H2 driver -->
<dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-data-r2dbc</artifactId></dependency>
<dependency><groupId>io.r2dbc</groupId><artifactId>r2dbc-h2</artifactId><scope>runtime</scope></dependency>
<!-- tests: @WebFluxTest, WebTestClient, reactor-test / @DataR2dbcTest -->
<dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-webflux-test</artifactId><scope>test</scope>
</dependency>
<dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-data-r2dbc-test</artifactId><scope>test</scope>
</dependency>
```

> **Boot 4 gotcha (found while building this chapter):** `spring-boot-starter-webflux` no longer brings the `WebClient.Builder` auto-configuration. Without `spring-boot-starter-webclient` the app fails with *"No qualifying bean of type 'org.springframework.web.reactive.function.client.WebClient$Builder' available"*.

The controller

Same annotations as MVC (Chapter 5); only the return types change:

```
@RestController
@RequestMapping("/api/books")
public class BookController {
    private final BookRepository repo;                // reactive repository (§13.14)

    public BookController(BookRepository repo) { this.repo = repo; }

    @GetMapping
    public Flux<Book> all(@RequestParam(required = false) String q) {
        return q == null ? repo.findAll() : repo.search(q);
    }

    @GetMapping("/{id}")
    public Mono<Book> one(@PathVariable long id) {
        return repo.findById(id)
                .switchIfEmpty(Mono.error(new ResponseStatusException(HttpStatus.NOT_FOUND, "no book " + id)));
    }

    @PostMapping
    public Mono<ResponseEntity<Book>> create(@RequestBody Mono<Book> body) {
        return body.map(b -> b.withId(null))                    // never trust a client-sent id
                .flatMap(repo::save)
                .map(saved -> ResponseEntity.created(URI.create("/api/books/" + saved.id())).body(saved));
    }
    ...
}
```

- `Flux<Book>` is written as a JSON array (or streamed as NDJSON — one JSON object per line — / SSE when the client asks for `application/x-ndjson` / `text/event-stream`).

- An empty `Mono` would give `200` with an empty body; `switchIfEmpty(Mono.error(...))` makes "not found" a 404.
- `@RequestBody Mono<Book>` makes even body parsing non-blocking (plain `@RequestBody Book` also works).
- **You return the pipeline, the framework subscribes** — no `subscribe()` or `block()` anywhere.

Testing it — `@WebFluxTest` + `WebTestClient`

```
/** WebFlux slice: controller + codecs + error handling; the repository is a mock returning Mono/Flux. */
@WebFluxTest(BookController.class)
class BookControllerWebFluxTest {

    @Autowired WebTestClient client;
    @MockitoBean BookRepository repo;

    static final Book DDD = new Book(1L, "Domain-Driven Design", "Eric Evans", "9780321125217");

    @Test
    void listReturnsJsonArray() {
        given(repo.findAll()).willReturn(Flux.just(DDD));
        client.get().uri("/api/books").exchange()
              .expectStatus().isOk()
              .expectHeader().contentType(MediaType.APPLICATION_JSON)
              .expectBodyList(Book.class).hasSize(1).contains(DDD);
    }

    @Test
    void missingBookIs404() {
        given(repo.findById(99L)).willReturn(Mono.empty());
        client.get().uri("/api/books/99").exchange()
              .expectStatus().isNotFound()
              .expectBody()
              .consumeWith(r -> System.out.println("GET /api/books/99 -> 404 " + new String(r.getResponseBody())))
              .jsonPath("$.status").isEqualTo(404);
    }

    @Test
    void createReturns201() {
        given(repo.save(new Book(null, "Refactoring", "Martin Fowler", "9780134757599")))
                .willReturn(Mono.just(new Book(7L, "Refactoring", "Martin Fowler", "9780134757599")));
        client.post().uri("/api/books").contentType(MediaType.APPLICATION_JSON)
              .bodyValue("""
                      {"id":123,"title":"Refactoring","author":"Martin Fowler","isbn":"9780134757599"}""")
              .exchange()
              .expectStatus().isCreated()
              .expectHeader().location("/api/books/7")
              .expectBody().jsonPath("$.id").isEqualTo(7);
    }
}
```

Output:

```
POST /api/books (client id 123 ignored) -> 201 Location: /api/books/7
GET /api/books/99 -> 404 {"timestamp":"2026-09-29T00:24:18.499Z","path":"/api/books/99","status":404,"error":"Not
Found","requestId":"25c24062"}
GET /api/books -> 200 [Domain-Driven Design]
```

- `@WebFluxTest` is the WebFlux slice (ch12 §12.7); unlike `@SpringBootTest` it provides a `WebTestClient` without an extra annotation.
- Mocked reactive repositories must return `Mono` / `Flux`, never `null` (→ `NullPointerException` inside the framework).
- The 404 body is Boot's default reactive error format; for RFC 9457 Problem Details set `spring.webflux.problemdetails.enabled=true` or add a `@RestControllerAdvice` returning `ProblemDetail`, as in MVC.

13.11 Functional endpoints

What it is

A second model (also in MVC via `RouterFunctions` in `spring-webmvc`): routes are **code** — a `RouterFunction<ServerResponse>` bean — and handlers are plain functions `ServerRequest → Mono<ServerResponse>`. No annotations or reflection: explicit, easily unit-tested routing.

```java
@Configuration(proxyBeanMethods = false)
public class BookRoutes {

    @Bean
    RouterFunction<ServerResponse> bookRouter(BookHandler h) {
        return route()
                .path("/fn/books", b -> b
                        .GET("/{isbn}", h::byIsbn)
                        .GET("", h::byAuthor))                      // /fn/books?author=...
                .filter((req, next) -> {                            // a filter for this router only
                    req.exchange().getResponse().getHeaders().add("X-Router", "functional");
                    return next.handle(req);
                })
                .build();
    }

    @Component
    static class BookHandler {
        private final BookRepository repo;
        BookHandler(BookRepository repo) { this.repo = repo; }

        Mono<ServerResponse> byIsbn(ServerRequest req) {
            return repo.findByIsbn(req.pathVariable("isbn"))
                    .flatMap(b -> ServerResponse.ok().contentType(MediaType.APPLICATION_JSON).bodyValue(b))
                    .switchIfEmpty(ServerResponse.notFound().build());
        }

        Mono<ServerResponse> byAuthor(ServerRequest req) {
            return req.queryParam("author")
                    .map(a -> ServerResponse.ok().body(repo.findByAuthor(a), Book.class))
                    .orElseGet(() -> ServerResponse.badRequest().bodyValue("author is required"));
        }
    }
}
```

Found while testing: the first filter changed the returned `ServerResponse`'s headers (`res.headers().add(...)`) and failed with `UnsupportedOperationException` (500) — a built `ServerResponse` is immutable. Add headers through the exchange (as above) or build a new response.

Testing without Spring

Routes are plain objects, so `WebTestClient` binds straight to them — no context, no server:

```java
class FunctionalRoutesTest {
    BookRepository repo = mock(BookRepository.class);
    WebTestClient client = WebTestClient
            .bindToRouterFunction(new BookRoutes().bookRouter(new BookRoutes.BookHandler(repo)))
            .build();

    @Test
    void byIsbnFoundAndNotFound() {
        given(repo.findByIsbn("111")).willReturn(Mono.just(new Book(1L, "T", "A", "111")));
        given(repo.findByIsbn("222")).willReturn(Mono.empty());

        client.get().uri("/fn/books/111").exchange()
              .expectStatus().isOk()
              .expectHeader().valueEquals("X-Router", "functional")
              .expectBody().jsonPath("$.title").isEqualTo("T");
        client.get().uri("/fn/books/222").exchange().expectStatus().isNotFound();
    }
    ...
}
```

Output:

```
GET /fn/books/111 -> 200 (X-Router: functional), /fn/books/222 -> 404
GET /fn/books?author=Evans -> 200 [1 book], /fn/books -> 400 author is required
```

Annotated vs functional

	Annotated controllers	Functional endpoints
Routing	annotations, discovered by scanning	explicit code, composable
Argument binding / validation	automatic (`@PathVariable` , `@Valid`)	manual (`req.pathVariable` , `req.bodyToMono` , call a `Validator`)
Familiar to most Spring devs	yes	less
Good for	most CRUD APIs	small services, dynamic routes, tight control, GraalVM-friendly code (ahead-of-time native executables)

Both can live in one app (as here).

13.12 Full-stack WebFlux tests and Server-Sent Events

Server-Sent Events

Server-Sent Events (SSE) is a simple standard for a server to **push** text events over one long-lived HTTP response (`Content-Type: text/event-stream`); browsers consume it with `EventSource` , reconnecting automatically. One-way (server → client); for two-way use WebSockets (an HTTP connection upgraded to a long-lived two-way channel). In WebFlux an SSE endpoint is just a `Flux` with that media type:

```
/** Server-Sent Events: one "tick" per interval, pushed over a single open HTTP response. */
@GetMapping(path = "/ticks", produces = MediaType.TEXT_EVENT_STREAM_VALUE)
public Flux<String> ticks(@RequestParam(defaultValue = "1000") long everyMs) {
    return Flux.interval(Duration.ofMillis(everyMs)).map(i -> "tick " + i);
}
```

Each element is written as `data:tick 0` plus a blank line; for event names, ids or retry hints return `Flux<ServerSentEvent<T>>` . When the client disconnects the subscription is **cancelled** and `Flux.interval` stops — no leaked timers. (Spring MVC offers `SseEmitter` or returning a `Flux` , but each open stream then holds a servlet async request.)

The full-stack test

```
/** Real Netty server on a random port, real R2DBC/H2 database seeded by data.sql. */
@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT)
@AutoConfigureWebTestClient
class FullStackWebFluxTest {

    @Autowired WebTestClient client;

    @Test
    void searchHitsTheDatabase() {
        client.get().uri("/api/books?q=reactive").exchange()
              .expectStatus().isOk()
              .expectBody()
              .consumeWith(r -> System.out.println("GET /api/books?q=reactive -> " + new String(r.getResponseBody())))
              .jsonPath("$.length()").isEqualTo(2);
    }

    @Test
    void serverSentEvents() {
        Flux<String> events = client.get().uri("/api/books/ticks?everyMs=100")
                .accept(MediaType.TEXT_EVENT_STREAM)
                .exchange()
                .expectStatus().isOk()
                .expectHeader().contentTypeCompatibleWith(MediaType.TEXT_EVENT_STREAM)
                .returnResult(String.class)
                .getResponseBody();                      // a live Flux of the streamed events

        StepVerifier.create(events)
                .expectNext("tick 0", "tick 1", "tick 2")
                .thenCancel()                            // closes the connection; the server stops its interval
                .verify(Duration.ofSeconds(5));
    }
}
```

Output:

```
GET /api/books?q=reactive -> [{"id":2,"title":"Hands-On Reactive Programming in Spring 5","author":"Oleh Dokuka","isbn":"9781787284951"},
{"id":1,"title":"Reactive Spring","author":"Josh Long","isbn":"9781732910409"}]
SSE received: tick 0, tick 1, tick 2 (then cancelled)
```

- In Boot 4, `WebTestClient` in a `@SpringBootTest` needs **`@AutoConfigureWebTestClient`** (from `spring-boot-starter-webflux-test`); it is then bound to the random-port server.
- `returnResult(String.class).getResponseBody()` gives the **live stream**; `WebTestClient` + `StepVerifier` (take a few items, cancel) is the standard way to test infinite/streaming endpoints.

13.13 `WebClient`: calling other services without blocking

What it is

`WebClient` is the non-blocking reactive HTTP client (Framework 5.0+). Boot auto-configures a `WebClient.Builder` with codecs and observability (per-call metrics and tracing, Chapter 15) — from `spring-boot-starter-webclient` in Boot 4. Its API mirrors `RestClient` (Chapter 6) but returns `Mono`/`Flux`.

```
@Component
public class InventoryClient {
    public record Stock(String isbn, int copies) { }

    private final WebClient http;

    public InventoryClient(WebClient.Builder builder, @Value("${inventory.base-url}") String baseUrl) {
        this.http = builder.baseUrl(baseUrl).build();
    }

    public Mono<Stock> stock(String isbn) {
        return http.get().uri("/stock/{isbn}", isbn)
                .retrieve()
                .onStatus(HttpStatusCode::is5xxServerError,
                        r -> Mono.error(new InventoryUnavailable("inventory said " + r.statusCode().value())))
                .bodyToMono(Stock.class)
                .timeout(Duration.ofMillis(500))                                  // per attempt
                .retryWhen(Retry.backoff(2, Duration.ofMillis(100))
                        .filter(e -> e instanceof InventoryUnavailable));          // only retry 5xx
    }

    /** Several calls at once: total time is roughly the slowest call, not the sum. */
    public Mono<Integer> totalCopies(List<String> isbns) {
        return Flux.fromIterable(isbns)
                .flatMap(this::stock)
                .map(Stock::copies)
                .reduce(0, Integer::sum);
    }
}
```

- `retrieve()` + `bodyToMono` is the simple path; 4xx/5xx become `WebClientResponseException` unless mapped with `onStatus`.
- `timeout` **before** `retryWhen` limits **each attempt**; after it, the whole operation.
- Only `InventoryUnavailable` (5xx) is retried, not a 404.

Tested against a real local server

The test starts a real **Reactor Netty HTTP server** on a random port, scripted to be fast, slow, flaky or down; `@DynamicPropertySource` points the client at it:

```java
@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.NONE)
class InventoryClientTest {

    static final AtomicInteger flakyCalls = new AtomicInteger();
    static final AtomicInteger downCalls = new AtomicInteger();

    static final DisposableServer INVENTORY = HttpServer.create()
            .host("127.0.0.1").port(0)
            .route(r -> r.get("/stock/{isbn}", (req, res) -> {
                String isbn = req.param("isbn");
                String json = "{\"isbn\":\"" + isbn + "\",\"copies\":3}";
                return switch (isbn) {
                    case "slow" -> res.header("Content-Type", "application/json")
                            .sendString(Mono.just(json).delayElement(Duration.ofSeconds(2)));
                    case "flaky" -> flakyCalls.incrementAndGet() <= 2
                            ? res.status(503).send()
                            : res.header("Content-Type", "application/json").sendString(Mono.just(json));
                    case "down" -> { downCalls.incrementAndGet(); yield res.status(500).send(); }
                    default -> res.header("Content-Type", "application/json")      // 300 ms "network + work"
                            .sendString(Mono.just(json).delayElement(Duration.ofMillis(300)));
                };
            }))
            .bindNow();

    @DynamicPropertySource
    static void props(DynamicPropertyRegistry r) {
        r.add("inventory.base-url", () -> "http://127.0.0.1:" + INVENTORY.port());
    }

    @Autowired InventoryClient client;

    @Test
    void callsRunConcurrently() {
        long t0 = System.nanoTime();
        Integer total = client.totalCopies(List.of("a", "b", "c", "d", "e")).block();
        long ms = (System.nanoTime() - t0) / 1_000_000;
        System.out.println("5 calls of 300 ms each -> total copies " + total + " in " + ms + " ms (machine-dependent)");
    }

    @Test
    void timeoutFires() {
        StepVerifier.create(client.stock("slow")).expectError(TimeoutException.class).verify(Duration.ofSeconds(5));
    }

    @Test
    void retriesTransient5xx() {
        StepVerifier.create(client.stock("flaky")).expectNextCount(1).verifyComplete();
        StepVerifier.create(client.stock("down"))
                .expectErrorMatches(e -> e.getMessage().startsWith("Retries exhausted: 2/2"))
                .verify(Duration.ofSeconds(5));
        assertThat(downCalls).hasValue(3);   // 1 try + 2 retries
    }
}
```

Output:

```
5 calls of 300 ms each -> total copies 15 in 322 ms (machine-dependent)
slow (2 s) -> TimeoutException after the 500 ms timeout
flaky: 503, 503, 200 -> success after 3 calls; down: gave up after 3 calls
```

Five 300 ms calls finished in **322 ms**, not 1,500 ms: `flatMap` ran them concurrently and no thread waited. (`RestClient` on virtual threads gets the same wall time — compared in §13.15.)

Common mistakes

- **`.block()` on a `WebClient` call inside a WebFlux handler** → `IllegalStateException`. Return the `Mono`.
- **`WebClient.create()` per call** → no shared **connection pool** (reused open connections), no Boot customisation. Build once from the injected builder.
- **Ignoring an error response's body** → the connection may not be released. Let `retrieve()` handle it, or consume/release the body in `exchangeToMono`.
- **No timeout** → a hung remote service keeps the request open forever. Add `timeout(...)` and/or connect/response timeouts on the underlying `HttpClient`.

13.14 R2DBC: reactive SQL

What it is

JDBC is **blocking by design**: `executeQuery()` holds the thread until rows arrive. **R2DBC** (Reactive Relational Database Connectivity, spec 1.0 in 2022) is a non-blocking SPI (service provider interface: a standard API each vendor's driver

implements) for SQL databases, with drivers for PostgreSQL, MySQL/MariaDB, SQL Server, Oracle and H2. **Spring Data R2DBC** adds repositories and `R2dbcEntityTemplate`; `DatabaseClient` offers fluent SQL.

It is **not an ORM** (like Hibernate): no lazy loading, relationships, cascades, dirty checking (auto-detecting changed fields) or first-level cache. One entity = one table; joins are hand-written. Deliberately — lazy loading would mean hidden blocking I/O. Compare Chapter 8.

Configuration

```
spring.r2dbc.url=r2dbc:h2:mem:///books;DB_CLOSE_DELAY=-1
# run schema.sql / data.sql through R2DBC at startup
spring.sql.init.mode=always
```

For PostgreSQL: `spring.r2dbc.url=r2dbc:postgresql://host:5432/db` with `org.postgresql:r2dbc-postgresql`. Boot adds a pool (`r2dbc-pool`) automatically. Flyway and Liquibase migrations need JDBC, so apps usually keep a JDBC driver *only* for migrations.

Entity and repository

```
/**
 * A row of table BOOK. id == null means "not saved yet" (INSERT), otherwise UPDATE.
 * No explicit name: Spring Data R2DBC quotes identifiers, so @Table("book") would look for a
 * lower-case "book" table, while H2 stored the unquoted name as BOOK.
 */
@Table
public record Book(@Id Long id, String title, String author, String isbn) {
    public Book withId(Long newId) { return new Book(newId, title, author, isbn); }
}

public interface BookRepository extends ReactiveCrudRepository<Book, Long> {

    Flux<Book> findByAuthor(String author);            // derived query, like Spring Data JPA

    Mono<Book> findByIsbn(String isbn);

    @Query("SELECT * FROM book WHERE LOWER(title) LIKE LOWER(CONCAT('%', :word, '%')) ORDER BY title")
    Flux<Book> search(String word);
}
```

The Javadoc records a real failure: with `@Table("book")` every statement failed with *`Table "book" not found (candidates are: "BOOK")`*. Spring Data R2DBC **quotes** identifiers (case-sensitive), and the unquoted `CREATE TABLE book` produced `BOOK` in H2 (PostgreSQL folds to lower case instead). Omit the name, so Spring derives it in the dialect's default case, or match the case exactly.

Tests — `@DataR2dbcTest`, `DatabaseClient`, a reactive transaction

```java
/** R2DBC slice: repositories + DatabaseClient on H2. Reactive tests are NOT rolled back automatically. */
@DataR2dbcTest
class BookRepositoryR2dbcTest {

    @Autowired BookRepository repo;
    @Autowired DatabaseClient db;
    @Autowired ReactiveTransactionManager txManager;

    @Test
    void derivedAndCustomQueries() {
        StepVerifier.create(repo.findByAuthor("Brian Goetz").map(Book::title))
                .expectNext("Java Concurrency in Practice").verifyComplete();
        StepVerifier.create(repo.search("SPRING").map(Book::title).collectList())
                .assertNext(l -> System.out.println("search('SPRING') -> " + l))
                .verifyComplete();
    }

    @Test
    void saveAssignsId() {
        StepVerifier.create(repo.save(new Book(null, "Release It!", "Michael Nygard", "9781680502398"))
                        .flatMap(saved -> repo.findById(saved.id())))
                .assertNext(b -> System.out.println("saved and re-read: " + b))
                .verifyComplete();
        repo.deleteAll(repo.findByIsbn("9781680502398")).block();   // clean up ourselves
    }

    @Test
    void databaseClientAndReactiveTransaction() {
        TransactionalOperator tx = TransactionalOperator.create(txManager);

        Mono<Long> insertThenFail = db.sql("INSERT INTO book(title, author, isbn) VALUES ('Tmp', 'X', 'tmp-1')")
                .fetch().rowsUpdated()
                .then(Mono.<Long>error(new IllegalStateException("fail after insert")))
                .as(tx::transactional);                     // the whole pipeline is one transaction

        StepVerifier.create(insertThenFail).expectErrorMessage("fail after insert").verify();

        StepVerifier.create(db.sql("SELECT COUNT(*) AS n FROM book WHERE isbn = 'tmp-1'")
                        .map(row -> row.get("n", Long.class)).one())
                .assertNext(n -> System.out.println("rows with isbn tmp-1 after rollback: " + n))
                .verifyComplete();
    }
}
```

Output:

```
saved and re-read: Book[id=7, title=Release It!, author=Michael Nygard, isbn=9781680502398]
rows with isbn tmp-1 after rollback: 0
search('SPRING') -> [Hands-On Reactive Programming in Spring 5, Reactive Spring]
```

- **Derived queries and `@Query`** work as in Spring Data JPA, returning `Flux`/`Mono`. `save` with `id == null` INSERTs and returns the row with its generated id (7, because the in-memory DB is shared with other test contexts and `data.sql` ran several times).
- **Reactive transactions**: `TransactionalOperator` (or `@Transactional` on a method returning `Mono`/`Flux`, backed by `R2dbcTransactionManager`) binds the connection to the **subscription's Context**, not a thread (Chapter 7 has the thread-bound version). The error after the INSERT rolled it back: 0 rows.
- **`@DataR2dbcTest` is not transactional**: unlike `@DataJpaTest` it isn't meta-annotated `@Transactional` (checked in Boot 4.1.1), because test-managed transactions are thread-bound. Clean up in the test (`saveAssignsId`).

When not to use R2DBC

When you need JPA features (relationships, lazy loading, caching), the team knows JPA well, or you're on virtual threads anyway. The ecosystem is smaller and some drivers lag their JDBC versions.

13.15 Virtual threads vs WebFlux: a measured comparison

The question

Java 21 made **virtual threads** final (Java ch15): JVM-managed threads costing a few hundred bytes to a few KB instead of a platform thread's MB-sized stack. A virtual thread blocking on I/O is **unmounted** from its carrier (platform) thread, which runs another one — event-loop-like scalability *without code changes*. Spring Boot (3.2+) switches Tomcat request handling, `@Async`, `@Scheduled` and more to virtual threads with **one property**:

```
spring.threads.virtual.enabled=true
```

For an I/O-bound endpoint, how do **MVC on platform threads**, **MVC on virtual threads** and **WebFlux** compare?

The setup

Two tiny projects share one endpoint: wait 100 ms (simulated I/O), then answer.

```
// load-mvc: blocking. The thread WAITS 100 ms, like a JDBC query or a blocking HTTP call.
@GetMapping("/io")
String io(@RequestParam(defaultValue = "100") long ms) throws InterruptedException {
    Thread.sleep(ms);
    return "ok on " + (Thread.currentThread().isVirtual() ? "virtual" : "platform") + " thread";
}

// load-webflux: non-blocking. A timer completes the response after 100 ms; no thread waits.
@GetMapping("/io")
Mono<String> io(@RequestParam(defaultValue = "100") long ms) {
    return Mono.delay(Duration.ofMillis(ms)).map(x -> "ok (non-blocking)");
}

// load-webflux: ANTI-PATTERN. Blocking inside a reactive handler parks an event-loop thread.
@GetMapping("/io-blocking")
Mono<String> ioBlocking(@RequestParam(defaultValue = "100") long ms) throws InterruptedException {
    Thread.sleep(ms);
    return Mono.just("ok (blocked " + ... + ")");
}
```

- `load-mvc` runs twice: `spring.threads.virtual.enabled=false` (Tomcat's 200 platform workers) and `=true`, each a separate `@SpringBootTest(webEnvironment = RANDOM_PORT)` class marked `@DirtiesContext` so its server stops before the next measurement.
- The load generator (`LoadGenerator.java`) is a **closed loop**: `concurrency` clients, each a virtual thread with the JDK `HttpClient` (HTTP/1.1), send requests back-to-back. It reports wall time, throughput (req/s), p50 (median) and p99 (99% faster) latency, and the JVM's **peak platform thread count** (`ThreadMXBean`, excluding virtual threads).
- An unreported warm-up runs first (JIT compilation, connection setup).
- **Caveats:** client and server share one JVM and laptop; `Thread.sleep`/`Mono.delay` stand in for real I/O; no database pool or downstream to saturate. The numbers show the *shape* of the difference, not production behaviour.

Results (machine-dependent)

Real output (14-core Apple M4 Pro, JDK 26 running Java 25 bytecode):

```
MVC, spring.threads.virtual.enabled=false -> "ok on platform thread"
| mode                        | conc. |   reqs | wall ms |  req/s | p50 ms | p99 ms | peak platform threads | errors |
| MVC Platform threads        |   100 |   2000 |    2157 |    927 |    105 |    119 |                   231 |      0 |
| MVC Platform threads        |  1000 |  10000 |    5238 |   1909 |    517 |    535 |                   231 |      0 |
MVC, spring.threads.virtual.enabled=true -> "ok on virtual thread"
| mode                        | conc. |   reqs | wall ms |  req/s | p50 ms | p99 ms | peak platform threads | errors |
| MVC Virtual threads         |   100 |   2000 |    2131 |    938 |    106 |    111 |                    33 |      0 |
| MVC Virtual threads         |  1000 |  10000 |    1259 |   7942 |    108 |    346 |                    33 |      0 |
CPU cores = 14 (Reactor Netty event-loop threads default to this number)
| mode                        | conc. |   reqs | wall ms |  req/s | p50 ms | p99 ms | peak platform threads | errors |
| WebFlux non-blocking        |   100 |   2000 |    2185 |    915 |    107 |    122 |                    56 |      0 |
| WebFlux non-blocking        |  1000 |  10000 |    1241 |   8058 |    110 |    263 |                    56 |      0 |
| WebFlux + Thread.sleep (bad) |   100 |    500 |    5107 |     97 |    724 |   2479 |                    56 |      0 |
```

Reading the numbers

1. **At 100 clients all three are equal** (~930 req/s, p50 ≈ 105 ms): the ceiling is the clients (100 × 10 req/s). Below the thread limit platform threads are fine.
2. **At 1,000 clients platform threads hit the wall**: 200 workers × 10 req/s = 2,000 theoretical, 1,909 measured. Extra requests **queue**: median jumps from ~105 ms to **517 ms** (≈ 5 rounds of 100 ms).
3. **Virtual threads remove the wall without code changes**: 7,942 req/s at 108 ms median, ~4× platform throughput, with **33** platform threads in the JVM instead of 231 — each request's carrier served others while it slept.
4. **WebFlux matches virtual threads** (8,058 req/s, p50 110 ms; the difference is noise). Both are now CPU/load-generator bound. (p99 263–346 ms at 1,000 clients comes from client/server CPU contention in one JVM plus GC pauses — treat tail latencies here with care.)
5. **Blocking on an event loop is a disaster**: the same `Thread.sleep(100)` in a WebFlux handler gave **97 req/s**, p99 **2.5 s**. Each of 14 loops serves one sleeping request at a time (≤ 14 × 10 = 140 req/s; uneven connection spread lowers it), and every *other* request on a blocked loop waits too.

Which one should you choose?

Situation	Choose	Why
Typical CRUD/REST service on JDBC/JPA, blocking clients	**MVC + virtual threads**	same scalability for I/O-bound work, plain imperative code, all blocking libraries work, easy debugging
Streaming: SSE, WebSockets, long-lived connections, NDJSON, backpressure end-to-end	**WebFlux**	a stream is a `Flux`; backpressure and cancellation are built in
Gateways/proxies that mostly forward bytes (Spring Cloud Gateway's reactive variant)	**WebFlux**	minimal per-connection cost, non-blocking all the way
Heavy fan-out/composition: call 10 services, combine, time out, retry, race	either; **Reactor** operators are very expressive; structured concurrency (subtasks treated as one unit; still preview) is the imperative alternative	
CPU-bound work (image processing, crypto)	neither helps	you need more cores; keep it off event loops
Team unfamiliar with reactive, blocking dependencies you can't replace	**MVC + virtual threads**	a reactive stack with blocking calls inside is the worst of both

Virtual-thread caveats

- **Pinning**: before Java 24 a virtual thread blocking inside `synchronized` **pinned** its carrier (couldn't unmount). JEP 491 (JDK Enhancement Proposal; Java 24) fixed this for `synchronized`; native calls and some class-initialisation paths still pin. See Java ch15 §15.5.
- **Resources are still finite**: 10,000 virtual threads still share, say, 10 JDBC connections — the pool becomes the queue (Chapter 7). Limit downstream calls with semaphores (N holders at once) or bulkheads (capped resource pools per dependency).
- `ThreadLocal` **caches** (big per-thread buffers) multiply with millions of threads; prefer scoped values (immutable per-call alternative) or explicit passing.

> **In the interview**

Q: WebFlux or virtual threads? "For I/O-bound request/response services I'd default to Spring MVC with `spring.threads.virtual.enabled=true`. In my measurements it matches WebFlux for a 100 ms I/O endpoint — about 8,000 req/s at 1,000 concurrent clients vs about 1,900 on 200 platform threads — and the code stays simple and debuggable. I'd pick WebFlux for streaming (SSE, WebSockets), gateways, or end-to-end backpressure and rich async composition, and only if the whole call path is non-blocking: a single blocking call on an event loop cut WebFlux to ~100 req/s."

- *Do virtual threads make code faster?* → No; they make *waiting* cheap, so more requests can wait at once. CPU-bound work gets no faster.
- *What is pinning?* → A virtual thread that can't unmount from its carrier while blocked (before Java 24 inside `synchronized`; still for native frames), so it holds a platform thread.

13.16 Common mistakes — checklist

Mistake	Symptom	Fix
Blocking call (JDBC, `RestClient`, `Thread.sleep`, `block()`) on an event loop	throughput collapses, timeouts under load; `block()` throws `IllegalStateException`	non-blocking alternative, or `fromCallable(..).subscribeOn(boundedElastic())`; BlockHound in tests
Pipeline built but not returned/subscribed	nothing happens (no save, no call)	return the `Mono` / `Flux`; chain with `then` / `flatMap`
`map` with an async function	`Flux<Mono<T>>`, inner calls never run	`flatMap` / `concatMap`
`Mono.just(expensiveCall())`	runs eagerly, even when not subscribed; retry doesn't repeat it	`Mono.fromCallable` / `Mono.defer`
Relying on `ThreadLocal` (MDC, security, tx)	values missing after a thread hop	Reactor `Context`, context-propagation
Unbounded `flatMap` / `onBackpressureBuffer()`	overwhelmed downstream, memory growth	concurrency argument, bounded buffers, `limitRate`
`@Table("book")` with Spring Data R2DBC on H2	`Table "book" not found (candidates are: "BOOK")`	omit the name or match the case
Missing `spring-boot-starter-webclient` in Boot 4	no `WebClient.Builder` bean	add the starter
Virtual-time test without `thenAwait`	test hangs	advance the clock; JUnit default timeout
Mixing MVC and WebFlux starters	Boot silently starts MVC	pick one, or set `spring.main.web-application-type`

13.17 In the interview

Q: What is reactive programming and what problem does it solve? "Programming with asynchronous data streams and a subscriber-controlled flow rate (backpressure). In a server no thread waits for I/O: a few event-loop threads handle thousands of concurrent requests because work resumes only when data arrives. Spring uses Project Reactor — `Mono` for 0..1, `Flux` for 0..N. Pipelines are lazy and cold until subscribed; in WebFlux the framework subscribes."

- *`Mono` vs `Flux`?* → 0..1 vs 0..N items, both ending with complete or error. *`flatMap` vs `concatMap`?* → Concurrent/interleaved vs sequential/ordered.
- *`publishOn` vs `subscribeOn`?* → `publishOn` moves the operators after it; `subscribeOn` moves the subscription (source) regardless of position.
- *Blocking library from WebFlux?* → `Mono.fromCallable(...).subscribeOn(Schedulers.boundedElastic())` — and question whether WebFlux fits.
- *Backpressure example?* → `request(n)` limits the publisher; `limitRate`, `onBackpressureDrop/Latest/Buffer` handle sources that can't slow down.
- *Testing?* → `StepVerifier` (virtual time for delays/retries), `WebTestClient`, `@WebFluxTest` / `@DataR2dbcTest` slices.
- *JPA with WebFlux?* → Not without blocking (wrap every call on `boundedElastic`). Use R2DBC, or MVC with virtual threads.
- *Reactive transactions?* → `ReactiveTransactionManager` (`R2dbcTransactionManager`); `@Transactional` on `Mono`/`Flux` methods or `TransactionalOperator`; the transaction lives in the Reactor `Context`, not a `ThreadLocal`.

13.18 Chapter TL;DR

- **Why**: thread-per-request wastes threads on waiting; a 200-thread Tomcat caps near 2,000 req/s for 100 ms I/O (measured 1,909). Event loops never wait. Same idea as Node's event loop.
- **Reactive Streams**: `Publisher`/`Subscriber`/`Subscription`/`Processor`; demand via `request(n)`; one terminal signal. JDK copy: `java.util.concurrent.Flow`.
- **Reactor**: `Mono` (0..1), `Flux` (0..N); lazy and cold until subscribed; `just` is eager, `defer`/`fromCallable` are lazy; don't `block()` in reactive code.
- **Operators**: `map`, `filter`, `flatMap` (concurrent, 6 s) vs `concatMap` (ordered, 12 s) vs `flatMapSequential`; `zip`, `concat`, `merge`, `switchIfEmpty`, `buffer`, `reduce`.
- **Errors** end the stream; `onErrorReturn/Resume/Map`, `retryWhen(Retry.backoff(3, 1s))` → attempts at 0/1/3/7 s, `timeout`.
- **Testing**: `StepVerifier`, virtual time (`withVirtualTime(() -> ...)` + `thenAwait`), JUnit default timeout against hangs.
- **Backpressure**: `request(n)`, `limitRate(10)` → requests 10 then 8s; overflow: DROP / LATEST / BUFFER(n) → error.
- **Schedulers**: `publishOn` (downstream) vs `subscribeOn` (source); `boundedElastic` for unavoidable blocking calls; never block event loops.

- **WebFlux**: annotated controllers returning `Mono` / `Flux`, functional `RouterFunction`s, SSE with `text/event-stream`; tests with `@WebFluxTest` + `WebTestClient` and `@SpringBootTest(RANDOM_PORT)` + `@AutoConfigureWebTestClient`. Boot 4: add `spring-boot-starter-webclient` for `WebClient`.
- **WebClient**: `retrieve`, `onStatus`, per-attempt `timeout`, filtered `retryWhen`; 5 × 300 ms calls in 322 ms with `flatMap`.
- **R2DBC**: non-blocking SQL, not an ORM; `ReactiveCrudRepository`, `DatabaseClient`, `TransactionalOperator`; `@DataR2dbcTest` doesn't roll back; mind identifier quoting.
- **Measured**: at 1,000 concurrent clients, MVC platform threads 1,909 req/s (p50 517 ms), MVC **virtual threads 7,942 req/s**, **WebFlux 8,058 req/s**, WebFlux with a blocking call **97 req/s**. Default to MVC + virtual threads for I/O-bound CRUD; choose WebFlux for streaming, gateways and end-to-end non-blocking pipelines.
- Next: Chapter 14 — Messaging, async and scheduling.

Chapter 14 — Messaging, Async and Scheduling

So far all work happened **inside the request**. Real systems also need work that happens **later**, **elsewhere** or **on a timer**:

- "After sign-up, send a welcome e-mail without making the user wait for the SMTP (e-mail sending) server."
- "Every night at 02:00, expire unpaid orders."
- "When an order is placed, tell shipping, billing and analytics — other **processes** (running programs with their own memory, e.g. one JVM) that may be down right now."

Spring has one tool per layer: **application events** (in-process notifications), `@Async` (run a method on another **thread**), `@Scheduled` (run on a timer), and **messaging** with Kafka or RabbitMQ (**message brokers**: servers storing messages between processes, **durably** — on disk, surviving a crash; see System Design Fundamentals). The chapter ends with the **transactional outbox**, which joins a database **transaction** (changes that commit together or not at all) and a broker without losing messages.

The code (Spring Boot 4.1.1, Spring Framework 7.0, Spring for Apache Kafka 4.1.1, Kafka clients 4.2.1, Java 25):

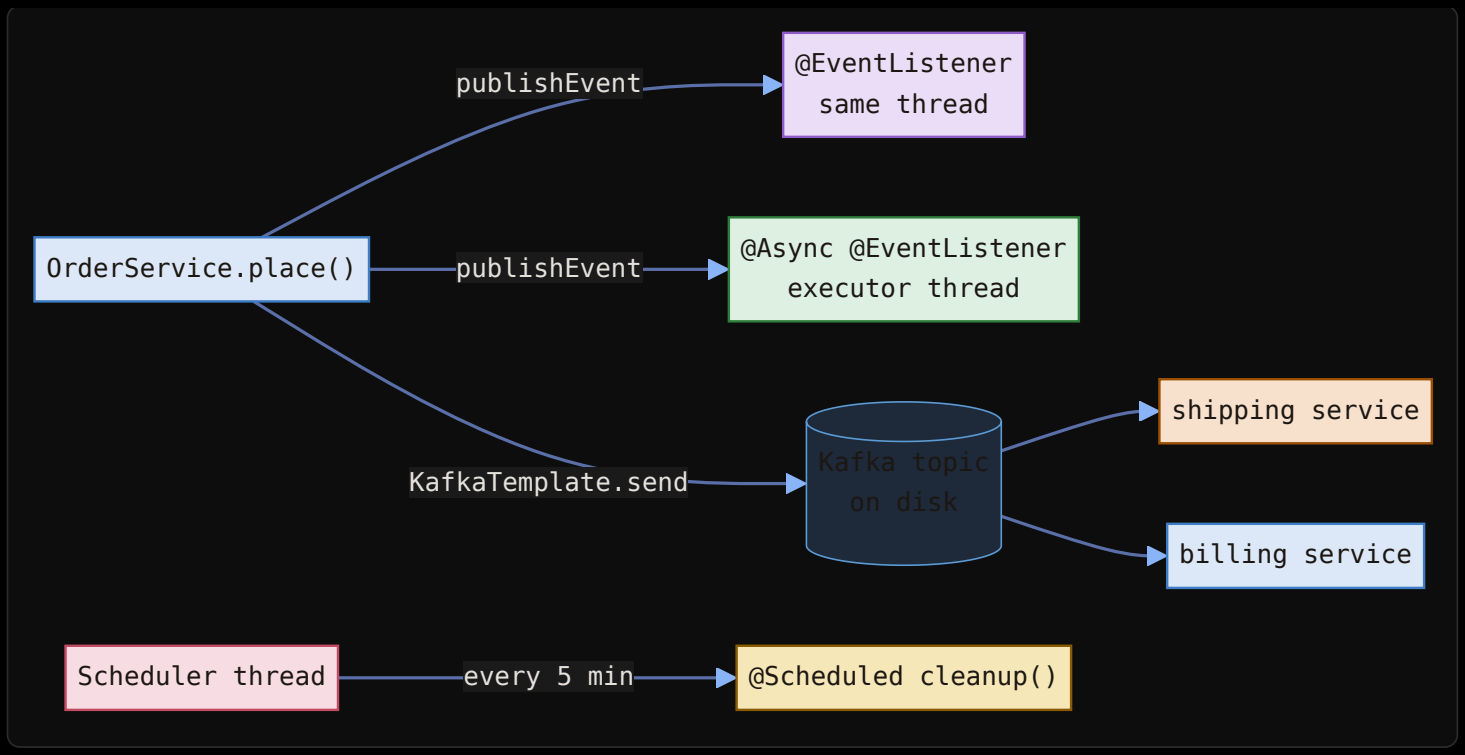
Project	Needs	What it proves
`code/events-async`	H2 in memory (a Java database living inside the JVM)	sync vs async listeners, transaction phases, `@Async` executors and thread names, MDC propagation, virtual threads, async exceptions
`code/scheduling`	nothing	`fixedRate` vs `fixedDelay` vs `cron` timing, turning jobs off in tests, overlap on platform vs virtual threads
`code/kafka-orders`	nothing: an embedded Kafka broker runs inside the test JVM	keys → partitions, consumer groups, JSON on the wire, retries + DLT, `@RetryableTopic`, idempotence, transactions
`code/outbox`	H2 in memory	atomic order + outbox row, polling publisher, broker outage, duplicate delivery, idempotent consumer
`code/rabbit-orders`	a RabbitMQ broker (Docker)	exchange/queue/binding/DLX configuration; **compiles, not run**

Every output below was printed by a test and checked by an assertion. Run all with `bash tools_run_spring.sh spring '14-*'` from the repository root. No Docker needed: Kafka runs as `@EmbeddedKafka`, a real broker inside the test JVM.

14.1 The map: four ways to do work "somewhere else"

What it is

Tool	Where the work runs	Survives a crash?	Typical use
Application event (`publishEvent` + `@EventListener`)	same JVM, same thread by default	no, it lives in memory	decoupling modules inside one app
`@Async` method	same JVM, another thread	no	"don't make the caller wait": e-mails, reports, fan-out calls
`@Scheduled` method	same JVM, a scheduler thread	the schedule restarts with the app, a run in progress is lost	housekeeping, polling for new work, periodic reports
Message broker (Kafka, RabbitMQ)	another process, maybe another service	yes: the broker stores the message on disk	integration between services, buffering load, event streaming



How to choose

1. **Only this app cares, and losing the work in a crash is acceptable?** Application event; add `@Async` if the listener is slow.
2. **Must happen at a time or interval?** `@Scheduled`; on several instances add a lock (14.7).
3. **Another service must react, or the work must not be lost?** A broker; if the message must match a database change exactly, the **transactional outbox** (14.17).

The patterns (message queue, publish-subscribe, event-driven architecture) are explained without code in System-design patterns, Module 2 — Moving data and the event-driven architecture notes; this chapter is the Spring side.

14.2 Application events: synchronous vs `@Async` listeners

What it is

Chapter 2 introduced the container's **publish/subscribe**: a bean publishes an object with `ApplicationEventPublisher.publishEvent(...)`, and every `@EventListener` whose parameter type matches receives it (2.12) — **synchronously, on the publisher's thread, before `publishEvent` returns.**

`@Async` on a listener changes that: the publisher hands the event to an **executor** (owner of worker threads and a task queue, §14.4) and continues at once — dropping a letter in a colleague's in-tray instead of waiting while they read it.

How it works

1. `@EnableAsync` installs a post-processor that wraps every bean with `@Async` methods in a **proxy** (ch04).
2. `publishEvent` finds the matching listeners and calls them in order: a plain listener runs right away; an `@Async` listener's call goes through the proxy, which submits it to an executor and returns immediately.

```java
@SpringBootApplication
@EnableAsync   // turns on the proxy that makes @Async methods run on an executor
public class AsyncApplication { ... }

public record UserSignedUp(String email) {}

@Service
public class SignupService {
    // constructor injection of ApplicationEventPublisher events, Trace trace ...
    public void signUp(String email) {
        trace.add("publisher: publishing on " + Trace.thread());
        events.publishEvent(new UserSignedUp(email));
        trace.add("publisher: publishEvent returned on " + Trace.thread());
    }
}

@Component
public class SignupListeners {
    @EventListener                      // synchronous: runs inside publishEvent, same thread
    void createProfile(UserSignedUp e) {
        trace.add("  sync listener: profile for " + e.email() + " on " + Trace.thread());
    }

    @Async                              // asynchronous: handed to the executor, publisher does not wait
    @EventListener
    void sendWelcomeMail(UserSignedUp e) throws InterruptedException {
        Thread.sleep(200);              // pretend the SMTP call is slow
        trace.add("  async listener: welcome mail to " + e.email() + " on " + Trace.thread());
    }
}
```

`Trace` is a thread-safe list (`CopyOnWriteArrayList`) the tests print. The test calls `signUp`, prints the trace, then waits with **Awaitility** (polls a condition until true or timeout):

```java
signups.signUp("ana@example.com");
trace.lines().forEach(System.out::println);
assertThat(trace.lines()).hasSize(3);                    // the async listener has not finished yet
await().atMost(Duration.ofSeconds(2)).until(() -> trace.lines().size() == 4);
```

Output:

```
--- right after signUp() returned ---
publisher: publishing on main
  sync listener: profile for ana@example.com on main
publisher: publishEvent returned on main
--- 200 ms later ---
  async listener: welcome mail to ana@example.com on task-1
```

The sync listener ran on `main` (the test thread) between the publisher lines; the async one ran 200 ms later on `task-1`, Boot's auto-configured executor (14.4).

Trade-offs

	Sync listener	`@Async` listener
Publisher waits?	yes	no
Exception reaches the publisher?	yes, and it can roll back the publisher's transaction	no: it goes to the `AsyncUncaughtExceptionHandler` (14.6)
Runs in the publisher's transaction?	yes	no: a new thread has no transaction
Return value published as a new event?	yes	no
`@Order` between listeners	respected	only the hand-off is ordered; executions race

Common mistakes

- **Forgetting `@EnableAsync`.** Boot auto-configures an executor but does **not** enable `@Async`; without it `@Async` is silently ignored and the listener runs synchronously.
- **Expecting `ThreadLocal` state on the async thread** — the current transaction, Security context, logging MDC (§14.4). Nothing crosses threads unless copied (a `TaskDecorator`, 14.4).
- **Treating async events as reliable** — they live in memory; if the JVM stops first, the work is gone.

TL;DR

`@EventListener` = same thread, publisher waits. `@Async @EventListener` = executor thread, publisher continues; exceptions and transactions don't cross over. Remember `@EnableAsync`.

14.3 `@TransactionalEventListener`: events tied to a commit

What it adds to 7.15

7.15 explains the four phases (`BEFORE_COMMIT`, `AFTER_COMMIT` (default), `AFTER_ROLLBACK`, `AFTER_COMPLETION`) and how the listener registers with the transaction as a **synchronization** (a callback around commit/rollback). Here: what each phase *sees* in the database, and the common production combination `@Async` + `AFTER_COMMIT`.

```
@Transactional
public void place(String item, boolean paymentFails) {
    jdbc.sql("INSERT INTO orders(item) VALUES (?)").param(item).update();
    events.publishEvent(new OrderPlaced(item));
    trace.add("service: event published inside the transaction");
    if (paymentFails) throw new IllegalStateException("payment declined");
}

@Component
public class OrderListeners {
    @TransactionalEventListener(phase = TransactionPhase.BEFORE_COMMIT)
    void beforeCommit(OrderPlaced e) { /* counts the order row: visible, same transaction */ }

    @TransactionalEventListener   // phase = AFTER_COMMIT is the default
    void afterCommit(OrderPlaced e) { ... }

    @TransactionalEventListener(phase = TransactionPhase.AFTER_ROLLBACK)
    void afterRollback(OrderPlaced e) { /* counts the order row again: gone */ }

    @TransactionalEventListener(phase = TransactionPhase.AFTER_COMPLETION)
    void afterCompletion(OrderPlaced e) { ... }

    /** The common production combo: only after commit, off the request thread, in its own transaction. */
    @Async
    @TransactionalEventListener
    @Transactional(propagation = Propagation.REQUIRES_NEW)
    void emailAfterCommit(OrderPlaced e) {
        jdbc.sql("INSERT INTO email_log(message) VALUES (?)").param("confirmation for " + e.item()).update();
        trace.add("async AFTER_COMMIT email for " + e.item() + " on " + Trace.thread());
    }
}
```

Output (`EventsTest`):

```
--- commit ---
service: event published inside the transaction
BEFORE_COMMIT    book (row visible in tx: 1)
AFTER_COMMIT     book on main
AFTER_COMPLETION book
async AFTER_COMMIT email for book on task-2
--- rollback ---
service: event published inside the transaction
AFTER_ROLLBACK   lamp (rows now: 0)
AFTER_COMPLETION lamp
```

- On **commit**, `BEFORE_COMMIT` still saw the uncommitted row (it runs inside the same, still open transaction); `AFTER_COMMIT` and `AFTER_COMPLETION` ran on `main`; the async e-mail ran on `task-2` and wrote `email_log` in its own transaction.
- On **rollback**, only `AFTER_ROLLBACK` (row gone) and `AFTER_COMPLETION` ran, and **no e-mail was written** (the test waited 300 ms and asserted zero `email_log` rows for `lamp`).

Why `REQUIRES_NEW` on the async listener

Since Framework 6.1 a `@TransactionalEventListener` that is also `@Transactional` must use `REQUIRES_NEW` (a fresh, separate transaction) or `NOT_SUPPORTED` (none), or the context fails to start (7.15). With `@Async` it runs on another thread anyway; `REQUIRES_NEW` states the intent.

When not to use it

When the side effect **must** happen: `AFTER_COMMIT` + `@Async` is an in-memory hand-off, so a JVM death between commit and e-mail loses it with no retry. For "must not be lost", write an outbox row in the same transaction (14.17).

In the interview

> **30-second answer:** see 7.15, plus: "With `@Async` the side effect also leaves the request thread and gets its own transaction (`REQUIRES_NEW`). It is still an in-memory hand-off, so critical messages go through an outbox."

- *No transaction active?* → Transactional listeners are skipped, unless `fallbackExecution = true`.
- *Can `BEFORE_COMMIT` veto the commit?* → Yes: an exception there propagates and the transaction rolls back.

14.4 `@Async` and executors

What it is

`@Async` on a public (at least non-private, non-final) bean method means "run on another thread, return to the caller at once". An **executor** owns worker threads and a queue of waiting tasks (the `Executor` of Java ch14 §14.11). Spring picks **which** executor, wraps the call, and adapts the return value.

How it works

1. The caller calls `reports.build("daily")`. `reports` is really the proxy.
2. The proxy's `AsyncExecutionInterceptor` picks an executor:
   - the one named in `@Async("name")`, if given;
   - else the one returned by an `AsyncConfigurer`;
   - else a unique `TaskExecutor` bean, or the bean named `taskExecutor`. With Boot this is the auto-configured `applicationTaskExecutor`.
3. It submits the real method call as a task and returns right away:
   - `void` → returns nothing; an exception goes to the `AsyncUncaughtExceptionHandler`.
   - `CompletableFuture<T>` (or `Future<T>`) → returns a **future** (placeholder for a later result) completing when the task finishes; inside the method return `CompletableFuture.completedFuture(value)` and Spring completes the caller's future.

Boot's auto-configured executor

If you define **no** `Executor` bean, Boot creates one called `applicationTaskExecutor`:

Setting	Default	Property
type	`ThreadPoolTaskExecutor`; `SimpleAsyncTaskExecutor` if virtual threads are on	`spring.threads.virtual.enabled`
core threads	8	`spring.task.execution.pool.core-size`
queue capacity	unbounded	`spring.task.execution.pool.queue-capacity`
max threads	unlimited, but used only when the queue is full — with an unbounded queue never more than core threads	`spring.task.execution.pool.max-size`
thread names	`task-1`, `task-2`, ...	`spring.task.execution.thread-name-prefix`
wait for tasks on shutdown	no	`spring.task.execution.shutdown.await-termination(-period)`

(This chapter's example sets `core-size=4`.)

> **Watch out:** declaring *any* `Executor` bean (e.g. a reports pool) makes Boot's `applicationTaskExecutor` back off, and `@Async`, MVC async requests and other integrations fall back to other defaults. Fixes: declare the extra pool with `@Bean(defaultCandidate = false)` (the Boot docs' recipe), or `spring.task.execution.mode=force` (Boot 3.5+) to create the auto-configured executor anyway.

A dedicated pool, a TaskDecorator, and the self-call trap

```
@Configuration(proxyBeanMethods = false)
public class AsyncConfig {

    /** A second, dedicated pool for slow report jobs. defaultCandidate = false keeps it out of
     *  by-type injection, so Boot still auto-configures the normal applicationTaskExecutor. */
    @Bean(defaultCandidate = false)
    @Qualifier("reportExecutor")
    ThreadPoolTaskExecutor reportExecutor() {
        ThreadPoolTaskExecutor executor = new ThreadPoolTaskExecutor();
        executor.setThreadNamePrefix("report-");
        executor.setCorePoolSize(2);
        executor.setMaxPoolSize(2);
        executor.setQueueCapacity(10);          // bounded: an 11th waiting job is rejected
        return executor;
    }

    /** Boot applies a TaskDecorator bean to the executor it auto-configures: copy the MDC across threads. */
    @Bean
    TaskDecorator mdcCopyingDecorator() {
        return runnable -> {
            Map<String, String> callerMdc = MDC.getCopyOfContextMap();   // runs on the caller thread
            return () -> {                                               // runs on the worker thread
                if (callerMdc != null) MDC.setContextMap(callerMdc);
                try { runnable.run(); } finally { MDC.clear(); }
            };
        };
    }
}

@Service
public class ReportService {
    @Async   // default executor: Boot's applicationTaskExecutor
    public CompletableFuture<String> build(String name) {
        return CompletableFuture.completedFuture(name + " built on " + Trace.thread()
                + " requestId=" + MDC.get("requestId"));
    }

    @Async("reportExecutor")   // pick a specific executor bean by name / qualifier
    public CompletableFuture<String> buildBig(String name) { /* same body */ }

    /** Self-invocation: the call below does not go through the proxy, so it is NOT asynchronous. */
    public String buildViaThis(String name) {
        return this.build(name).join();
    }
}
```

The **MDC** (Mapped Diagnostic Context) is SLF4J's (the standard Java logging facade) per-thread map of values such as a request id that log lines include. Being a `ThreadLocal`, a new thread starts empty. A **`TaskDecorator`** wraps each submitted task: it reads the caller's values on the caller thread and restores them on the worker.

The test puts `requestId=r-42` into the MDC and calls the three methods:

Output (`AsyncMethodsTest`):

```
auto-configured executor: ThreadPoolTaskExecutor
daily built on task-1 requestId=r-42
yearly built on report-1 requestId=null
self-call built on main requestId=r-42
```

- `daily` ran on Boot's pool (`task-1`); the decorator carried `r-42` across.
- `yearly` ran on the dedicated pool (`report-1`) without a decorator (hand-built pool), so the MDC was empty — call `executor.setTaskDecorator(...)` yourself if needed.
- `self-call` ran on `main`: `this.build(...)` skips the proxy, so `@Async` did nothing — the self-invocation rule of `@Transactional` (4.5).

Boot 4.1 also offers `spring.task.execution.propagate-context=true`, registering Spring's `ContextPropagatingTaskDecorator`, which copies thread-locals registered with Micrometer's context-propagation library (e.g. tracing) instead of a hand-written decorator.

Sizing a pool

Workload	Pool shape
CPU-bound (image resizing, PDF rendering)	about as many threads as cores; more only add context switching
I/O-bound (HTTP calls, SMTP, database)	many threads, or virtual threads (14.5)
Must not pile up in memory	a **bounded** queue plus a rejection policy. The default `AbortPolicy` throws `TaskRejectedException` to the caller; `CallerRunsPolicy` runs the task on the caller's thread, which slows the producer down
Isolate slow jobs from fast ones	separate pools (a **bulkhead**, see microservices notes)

Common mistakes

- **Self-invocation, `private` or `final` methods** → no proxy, no async. Move the method to another bean.
- **Unbounded queue before a slow dependency** → tasks pile up until the heap is full. Set `queue-capacity`.
- **Own `Executor` bean, then MVC async or `@Async` changed pools** → Boot's executor backed off. `defaultCandidate = false` or `mode=force`.
- **`.join()` right after the async call** → the caller waits anyway, plus a thread hop. Start several futures, then join all.
- **Losing work on shutdown** → queued tasks are dropped unless `shutdown.await-termination=true`.

Real-world examples

- A shop sends the confirmation e-mail and pushes the order to analytics via `@Async`, so checkout responds in 80 ms instead of 600 ms.
- A dashboard fires three `@Async` calls and combines them with `CompletableFuture.allOf(...)` — the page waits for the slowest, not the sum (Java 15.2 — CompletableFuture).

TL;DR

`@EnableAsync` + `@Async` = proxy + executor. Boot's `applicationTaskExecutor`: 8 threads, unbounded queue, `task-` names. Pick a pool with `@Async("name")`; declare extra pools with `defaultCandidate = false`; copy ThreadLocals (MDC) with a `TaskDecorator`; never self-invoke.

14.5 `@Async` on virtual threads

What it is

A **virtual thread** (final in Java 21) is JVM-managed: a few hundred bytes instead of ~1 MB of stack, so hundreds of thousands are fine. Blocking on I/O parks it and frees its carrier (OS) thread (Java 15.4 — Virtual threads).

`spring.threads.virtual.enabled=true` (Boot 3.2+, Java 21+) switches Boot's executors to virtual threads, and also Tomcat request threads, `@Scheduled` and the Kafka and RabbitMQ listener containers.

```
@SpringBootTest(properties = "spring.threads.virtual.enabled=true")
class VirtualThreadsTest {
    @Autowired ReportService reports;
    @Autowired AsyncTaskExecutor applicationTaskExecutor;

    @Test
    void asyncMethodsRunOnVirtualThreads() {
        System.out.println("auto-configured executor: " + applicationTaskExecutor.getClass().getSimpleName());
        List<CompletableFuture<String>> futures = IntStream.rangeClosed(1, 3)
                .mapToObj(i -> reports.build("report-" + i)).toList();
        futures.forEach(f -> System.out.println(f.join()));
        assertThat(futures).allSatisfy(f -> assertThat(f.join()).contains("on virtual task-"));
    }
}
```

Output:

```
auto-configured executor: SimpleAsyncTaskExecutor
report-1 built on virtual task-1 requestId=null
report-2 built on virtual task-2 requestId=null
report-3 built on virtual task-3 requestId=null
```

`Trace.thread()` prints `virtual` when `Thread.isVirtual()`. The executor is now a `SimpleAsyncTaskExecutor`: **no pool, no queue**, a new virtual thread per task; pool settings like `core-size` are ignored.

Trade-offs

	Platform-thread pool	Virtual threads
Concurrency limit	the pool size, a natural throttle	none by default: 10,000 submitted tasks means 10,000 concurrent tasks
Cost per blocked task	a whole OS thread	tiny
Best for	CPU-bound work, or when you want a hard cap	I/O-bound work: HTTP calls, JDBC, SMTP
Limit concurrency with	pool size + queue	`spring.task.execution.simple.concurrency-limit`, or Framework 7's `@ConcurrencyLimit` (4.7)

Common mistakes

- **Flooding a downstream system.** No pool = no throttle. If each task takes a DB connection, the connection pool (10 by default in HikariCP, Boot's pool) becomes the real limit. Set a concurrency limit deliberately.
- **Expecting CPU-bound work to speed up** — virtual threads help waiting, not computing.
- **Pinning.** Before JDK 24 a virtual thread blocked inside `synchronized` pinned its carrier; JEP 491 (JDK 24) fixed that. Native calls can still pin.
- **An app whose only work is scheduled jobs exits at startup** — virtual threads are daemon threads (don't keep the JVM alive). Set `spring.main.keep-alive=true`.

TL;DR

`spring.threads.virtual.enabled=true` → `@Async` runs on a new virtual thread per call (`SimpleAsyncTaskExecutor`), no pool, no queue. Great for I/O; add a concurrency limit yourself.

14.6 Exceptions in async methods

The problem it solves

Normally an exception travels up the caller's stack; with `@Async` the caller has already returned. Where it goes depends on the return type:

Return type	Where the exception goes	How to see it
`CompletableFuture<T>` / `Future<T>`	stored in the future, which completes exceptionally	`get()` throws `ExecutionException`, `join()` throws `CompletionException`, or use `exceptionally` / `handle` / `whenComplete`
`void`	nowhere near the caller	the `AsyncUncaughtExceptionHandler`. The default one only **logs** the error

```
@Async
public CompletableFuture<String> buildBroken(String name) {
    throw new IllegalStateException("no data for " + name);   // becomes an exceptionally completed future
}

@Async
public void fireAndForget(String name) {
    throw new IllegalStateException("cannot export " + name);  // void: goes to AsyncUncaughtExceptionHandler
}

/** Only overrides the exception handler, so @Async keeps using Boot's executor.
 *  Keep AsyncConfigurer in its own class with no @Bean methods: Boot 4 wraps AsyncConfigurer beans. */
@Configuration(proxyBeanMethods = false)
public class AsyncErrorConfig implements AsyncConfigurer {
    @Override
    public AsyncUncaughtExceptionHandler getAsyncUncaughtExceptionHandler() {
        return (ex, method, params) ->
                trace.add("uncaught handler: " + method.getName() + " threw " + ex.getMessage() + " on " + Trace.thread());
    }
}
```

The test:

```
CompletableFuture<String> future = reports.buildBroken("march");
assertThatThrownBy(future::get).isInstanceOf(ExecutionException.class)
        .hasCauseInstanceOf(IllegalStateException.class);
String viaHandle = future.handle((ok, ex) -> "recovered from: " + ex.getMessage()).join();

reports.fireAndForget("april");                   // returns at once, no exception for the caller
await().atMost(Duration.ofSeconds(2)).until(() -> !trace.lines().isEmpty());
```

Output:

```
CompletableFuture: recovered from: java.lang.IllegalStateException: no data for march
void method:       uncaught handler: fireAndForget threw cannot export april on task-4
```

The message inside `handle` starts with `java.lang.IllegalStateException:` because `handle` received a `CompletionException` wrapping the original (its message includes the cause). Use `ex.getCause()` for the original.

> **Found while building this chapter:** with `getAsyncUncaughtExceptionHandler()` **and** `@Bean` methods in one `AsyncConfigurer` class, Boot 4 failed at startup (`Illegal factory instance for factory method 'mdcCopyingDecorator'`) because it wraps user `AsyncConfigurer` beans. Hence two classes.

Common mistakes

- `void` **+ no handler = silent failure in production.** The default handler logs at ERROR and moves on; nobody retries. Return a `CompletableFuture`, or install a handler that records a metric and alerts.
- **Catching inside the async method and returning** `null` — the caller can't tell success from failure.
- **Expecting** `@Transactional` **rollback in the caller** — the async method runs in another thread and transaction (or none).

14.7 Scheduling with `@Scheduled`

What it is

`@Scheduled` makes the container call a method on a timer: every N ms, N ms after the previous run ended, or at calendar times given by a **cron expression** (a compact repeating-time format from Unix `cron`). It needs `@EnableScheduling`; Boot then auto-configures a `ThreadPoolTaskScheduler` with **one thread**, `scheduling-1`.

How it works

`@EnableScheduling` registers a post-processor that finds `@Scheduled` methods (no parameters, return value ignored) and registers a trigger for each with the `TaskScheduler`, whose thread runs the method at trigger time.

Attribute	Meaning	Example
`fixedRate = 300`	start every 300 ms, measured **start to start**	heartbeat, metrics push
`fixedDelay = 300`	wait 300 ms **after the previous run ends**	polling a table or an API: never overlaps, never floods
`initialDelay = 50`	wait before the first run	let the app warm up
`cron = "0 0 2 * * *"`	calendar-based, 6 fields	nightly jobs
`zone = "Europe/Berlin"`	time zone for `cron` (default: the server's)	daylight-saving-safe jobs
`timeUnit = TimeUnit.SECONDS`	unit for rate/delay (default ms)	`fixedDelay = 30, timeUnit = SECONDS`
`fixedDelayString = "${outbox.poller.delay-ms}"`	value from configuration	tunable per environment

Spring's cron has **six** fields, one more than Unix cron, because it starts with **seconds**:

Field	Values	`0 30 2 * * MON-FRI` means
second	0–59	at second 0
minute	0–59	minute 30
hour	0–23	02:xx
day of month	1–31, `L` = last day	any
month	1–12 or JAN–DEC	any
day of week	0–7 (0 and 7 = Sunday) or MON–SUN	Monday to Friday

Macros: `@hourly`, `@daily` (also `@midnight`), `@weekly`, `@monthly`, `@yearly` (also `@annually`). The value `"-"` disables a cron trigger — handy with a property: `cron = "${jobs.cleanup.cron:-}"`.

Proving fixedRate vs fixedDelay vs cron

Both jobs pretend to work 200 ms with a 300 ms period; the test records start times and prints the gaps:

```java
@Scheduled(fixedRate = 300)           // measured from START to START
void fixedRateJob() { rateRuns.add(now()); work(200); }

@Scheduled(fixedDelay = 300, initialDelay = 50)   // measured from END to next START
void fixedDelayJob() { delayRuns.add(now()); work(200); }

@Scheduled(cron = "*/1 * * * * *", zone = "UTC")  // every second, on the second
void cronJob() { cronRuns.add(LocalTime.now().withNano(0) + " on " + Thread.currentThread().getName()); }
```

```properties
# default is ONE scheduler thread shared by every @Scheduled method; give each job its own
spring.task.scheduling.pool.size=3
```

```java
await().atMost(Duration.ofSeconds(5))
       .until(() -> jobs.rateRuns.size() >= 6 && jobs.delayRuns.size() >= 4 && jobs.cronRuns.size() >= 2);
assertThat(median(rateGaps)).isBetween(270L, 360L);    // about 300 = the period
assertThat(median(delayGaps)).isBetween(470L, 600L);   // about 500 = 200 work + 300 delay
```

Output (`ScheduledJobsTest`; the millisecond gaps and clock times vary slightly from run to run):

```
fixedRate  = 300 ms, job takes 200 ms -> gaps between starts [300, 300, 300, 300] on scheduling-1
fixedDelay = 300 ms, job takes 200 ms -> gaps between starts [500, 510, 510] on scheduling-2
cron */1 * * * * *                    -> [19:48:15 on scheduling-1, 19:48:16 on scheduling-3]
```

`fixedRate` started every 300 ms regardless of job length; `fixedDelay` every 500 ms (200 ms work + 300 ms pause); cron fired on whole seconds, on whichever of the three scheduler threads was free. (`zone` affects only cron evaluation; the printed time is the local clock.)

The one-thread default, and overlap

With the default `pool.size=1`, **all** `@Scheduled` methods share one thread: a 10-minute report delays every other job, even the one-second heartbeat — hence the raised pool size.

What if a fixed-rate job is **slower than its period**? It depends on the scheduler. `SlowJob` does 250 ms of work every 100 ms and records concurrent runs:

```java
@Scheduled(fixedRate = 100)
void tick() throws InterruptedException {
    int now = running.incrementAndGet();
    maxRunningAtOnce.accumulateAndGet(now, Math::max);
    ...
    try { Thread.sleep(250); } finally { running.decrementAndGet(); }
}
```

Output (`OverlapTest`, 1.2 seconds each; run counts vary by one or two between runs):

```
ThreadPoolTaskScheduler: runs=5 maxRunningAtOnce=1 (scheduling-2)
SimpleAsyncTaskScheduler: runs=13 maxRunningAtOnce=3 (virtual thread scheduling-22)
```

- **Platform threads** (`ThreadPoolTaskScheduler`): runs of one method never overlap; late runs start as soon as the previous ends — only 5 runs.
- **Virtual threads** (`SimpleAsyncTaskScheduler`, Framework 6.1+): each run gets a new thread and the scheduler doesn't wait — three overlapped. Per the Framework docs this scheduler tracks only the hand-off, and `fixedDelay` tasks are forced onto a single scheduler thread to keep their meaning.

A job that must never overlap itself: `fixedDelay`, or a guard (`AtomicBoolean`, a lock).

Turning jobs off in tests

Background jobs make tests **flaky** (pass/fail with no code change) by touching data at random moments. Put `@EnableScheduling` on its own configuration class behind a property (`@ConditionalOnBooleanProperty`, new in Boot 3.5):

```java
@Configuration(proxyBeanMethods = false)
@EnableScheduling
@ConditionalOnBooleanProperty(name = "app.jobs.enabled", matchIfMissing = true)
public class SchedulingConfig {}
```

```
app.jobs.enabled=false -> runs after 700 ms: 0
```

Then test the job's **method** directly (as 14.17 does with `poller.publishBatch()`), plus one small test with scheduling on and Awaitility.

Running in a cluster

`@Scheduled` knows nothing about other instances: with three Kubernetes **pods** (running app instances), the 02:00 job runs **three times**. Options:

Option	How
ShedLock (a library)	`@SchedulerLock(name = "cleanup")` takes a lock row in a shared database table. Only one instance runs the job
Leader election (instances agree one "leader" does the job)	Spring Integration `LockRegistryLeaderInitiator`, Kubernetes leader election
Move the job out	a Kubernetes `CronJob`, or Quartz (Java job scheduler) in clustered JDBC mode for persistent, missed-fire-aware schedules
Make the job idempotent and safe to run concurrently	`UPDATE ... WHERE status = 'PENDING'` claims rows, or `SELECT ... FOR UPDATE SKIP LOCKED` (14.17)

Common mistakes

- **Forgetting** `@EnableScheduling` — nothing runs, nothing complains.
- **A long job on the single default thread** blocks every other job.
- **Five-field Unix cron** (`"0 2 * * *"`) — Spring needs six fields and rejects it at startup.
- `fixedRate` **polling a slow resource** on virtual threads — runs pile up. Use `fixedDelay`.
- **Relying on server time zones** — set `zone`: 02:30 happens twice or not at all on daylight-saving days.
- **Ignoring clusters** — N instances, N runs.

In the interview

> **30-second answer:** "`@Scheduled` needs `@EnableScheduling`. `fixedRate` is start-to-start, `fixedDelay` is end-to-start, and cron has six fields starting with seconds. Boot's default scheduler has one thread, so I raise `spring.task.scheduling.pool.size` or jobs block each other. On platform threads a slow fixed-rate job doesn't overlap itself, but on virtual threads it can. In a cluster every instance runs the job, so I use ShedLock or make the job idempotent."

- *How do you test a scheduled job?* → Call the method directly with scheduling off; one integration test with scheduling on, using Awaitility with a timeout — never `Thread.sleep` plus an exact count.
- *fixedRate or fixedDelay for polling an outbox?* → `fixedDelay`: the next poll starts only after the previous finished.

TL;DR

`@EnableScheduling` + `@Scheduled(fixedRate | fixedDelay | cron)`. Six-field cron with seconds. One scheduler thread by default. Overlap depends on the scheduler. One run per instance — lock or be idempotent in a cluster.

14.8 Messaging concepts you need first

What it is

A **message broker** accepts messages from **producers**, stores them, and hands them to **consumers**, which never talk directly — so either can be slow, down, or deployed separately (like a post office). Two designs dominate:

	Queue broker (RabbitMQ, ActiveMQ, Amazon SQS)	**Log broker (Apache Kafka, Redpanda, Amazon Kinesis)**
Storage	a queue per interested consumer group; a message is **deleted** once acknowledged ("acked": the consumer confirms handling)	an append-only **log**; messages stay for a retention period (e.g. 7 days) whether read or not
Reading	the broker pushes messages and tracks which are acknowledged	each consumer group tracks its own **offset** (position) and pulls
Replay old messages	no, they are gone after the ack	yes: reset the offset
Routing	rich: exchanges, routing keys, patterns, headers (explained in §14.18)	by topic and partition only (§14.9)
Ordering	per queue, weakened by multiple consumers and redelivery	strict **per partition**
Typical use	task distribution, request/reply, complex routing	event streaming, event sourcing (state rebuilt from stored change events), many independent consumers, high throughput

Delivery guarantees

Guarantee	Meaning	How you get it
At-most-once	a message may be lost, never duplicated	ack or commit **before** processing
At-least-once	never lost, may be **duplicated**	ack or commit **after** processing. Spring's default for both Kafka and RabbitMQ
Exactly-once	processed once	only inside a closed system, such as Kafka transactions for read-process-write within Kafka. Across a database and a broker you build it yourself: at-least-once plus an **idempotent consumer**

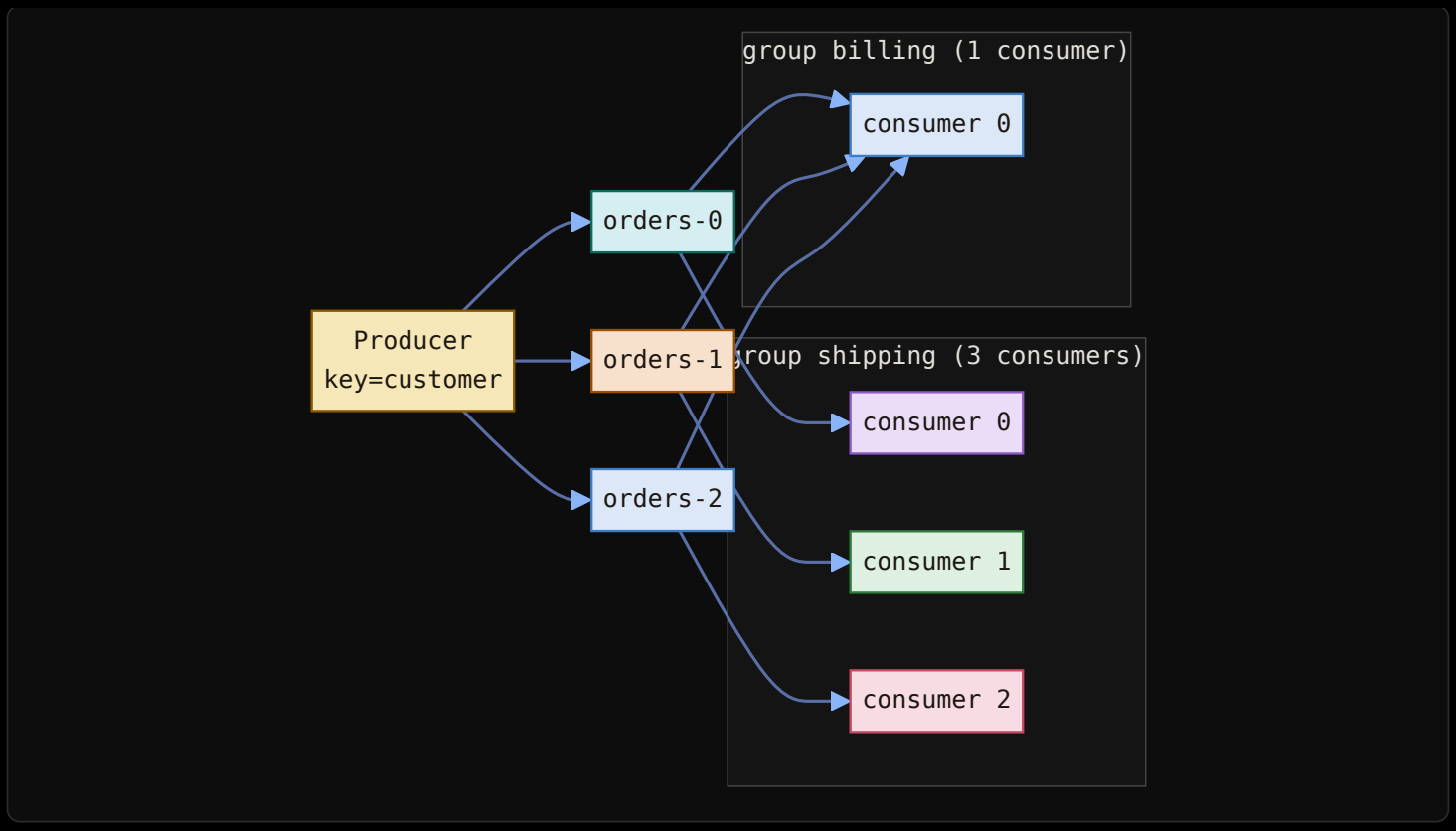
An **idempotent** operation gives the same result when repeated ("set status to PAID" is; "add 10 € to the balance" is not). Since at-least-once is normal, **every consumer should tolerate duplicates** — 14.17 remembers processed message ids. Theory, including why exactly-once is hard: DDIA ch11 — Stream processing.

14.9 Kafka in five minutes

What it is

Apache Kafka is a distributed, replicated **commit log** (an append-only list of records read in order). Vocabulary:

Term	Meaning
Broker	one Kafka server; a **cluster** has several. Since Kafka 4.0 the cluster manages itself with **KRaft** (Kafka Raft — Raft is a consensus algorithm); ZooKeeper (the former separate coordination service) is gone
Topic	a named stream of records, e.g. `orders`
Partition	a topic is split into N partitions, each an ordered append-only log on a broker — the unit of parallelism
Offset	a record's position inside its partition: 0, 1, 2, ...
Record	key (optional) + value + headers + timestamp
Key	decides the partition: the default partitioner hashes the key (murmur2) modulo the partition count. **Same key → same partition → ordered**; no key → spread across partitions
Replication factor	copies of each partition on different brokers. One is the **leader**, the others are followers
Consumer group	consumers sharing a `group.id`. Each partition is read by **exactly one** member of the group, and each group gets **every** record
Committed offset	per group and partition: "we've processed up to here". After a restart the group continues from it
Rebalance	reassigning partitions when members join or leave



Two rules follow:

1. **Scale consumers up to the partition count** — a fourth `shipping` consumer would idle. Choose partitions for the parallelism you will need.
2. **Ordering is per partition** — same-key records in order, no order across partitions.

Spring for Apache Kafka ("spring-kafka") wraps the Kafka Java client with `KafkaTemplate` (sending), `@KafkaListener` (receiving), error handlers, retry topics, transactions and test support. Boot 4.1.1 manages **spring-kafka 4.1.1** and **kafka-clients 4.2.1** (checked in the Boot 4.1.1 BOM — the "bill of materials" pinning compatible versions — and the spring-kafka 4.1 reference).

14.10 Setting up Spring Kafka (with JSON)

Dependencies

```
<!-- spring-kafka 4.1.x + kafka-clients 4.2.x + Jackson 3, versions managed by Boot 4.1.1 -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-kafka</artifactId>
</dependency>
<!-- the Kafka starter does not bring Jackson; JacksonJsonSerializer needs it (Jackson 3, tools.jackson.*) -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-jackson</artifactId>
</dependency>
<!-- tests: spring-boot-starter-test + spring-kafka-test (EmbeddedKafka) -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-kafka-test</artifactId>
  <scope>test</scope>
</dependency>
```

Boot 4's starters are modular (ch01): Kafka has `spring-boot-starter-kafka` and `spring-boot-starter-kafka-test` (Boot 3 used `org.springframework.kafka:spring-kafka` directly). Our first run failed with `ClassNotFoundException: tools.jackson.core.type.TypeReference` — the Kafka starter has no Jackson; add `spring-boot-starter-jackson` (or a web starter).

Configuration

```
# in tests @EmbeddedKafka sets spring.embedded.kafka.brokers; otherwise a local broker
spring.kafka.bootstrap-servers=${spring.embedded.kafka.brokers:localhost:9092}

# ---- producer: String keys, JSON values (Jackson 3) ----
spring.kafka.producer.key-serializer=org.apache.kafka.common.serialization.StringSerializer
spring.kafka.producer.value-serializer=org.springframework.kafka.support.serializer.JacksonJsonSerializer
spring.kafka.producer.acks=all

# ---- consumer: JSON values, wrapped so that a bad record cannot crash the consumer ----
spring.kafka.consumer.auto-offset-reset=earliest
spring.kafka.consumer.key-deserializer=org.apache.kafka.common.serialization.StringDeserializer
spring.kafka.consumer.value-deserializer=org.springframework.kafka.support.serializer.ErrorHandlingDeserializer
spring.kafka.consumer.properties[spring.deserializer.value.delegate.class]=org.springframework.kafka.support.serializer.JacksonJsonDeserializer
spring.kafka.consumer.properties[spring.json.trusted.packages]=ch14.kafka
```

Setting	Why
`JacksonJsonSerializer` / `JacksonJsonDeserializer`	JSON via **Jackson 3**. The older `JsonSerializer` / `JsonDeserializer` (Jackson 2) are `@Deprecated(since="4.0", forRemoval=true)` in spring-kafka 4
`ErrorHandlingDeserializer`	wraps the real deserializer; a record whose bytes aren't valid JSON (a **poison pill**) goes to the error handler instead of failing inside `poll()` forever
`spring.json.trusted.packages`	the class named in the `__TypeId__` header is created only from trusted packages. Never `*` for untrusted producers
`auto-offset-reset=earliest`	a **new** group without committed offset starts at the beginning; Kafka's default `latest` skips everything sent before it joined
`acks=all`	the leader waits for all in-sync replicas before confirming (client default since Kafka 3.0; explicit for clarity)

The event is a plain record:

```
/** The message payload. Serialized to JSON by JacksonJsonSerializer. */
public record OrderEvent(String orderId, String customer, long cents) {}
```

Topics are declared as beans; Boot's `KafkaAdmin` creates missing ones at startup:

```
@Bean NewTopic orders()      { return TopicBuilder.name("orders").partitions(3).replicas(1).build(); }
@Bean NewTopic payments()    { return TopicBuilder.name("payments").partitions(1).replicas(1).build(); }
@Bean NewTopic paymentsDlt() { return TopicBuilder.name("payments-dlt").partitions(1).replicas(1).build(); }
```

What goes over the wire

A test reads one record back with a plain `StringDeserializer` to show the raw bytes (`ProducerGuaranteesTest.jsonOnTheWire`):

```
value bytes as text: {"orderId":"o-json","customer":"zoe","cents":4200}
__TypeId__ header  : ch14.kafka.OrderEvent
```

The **type header** tells the consumer which class to build, coupling producer and consumer to one Java class name. Across services: give the consumer a fixed target type (`spring.json.value.default.type` + `spring.json.use.type.headers=false`), map type ids to local classes (`spring.json.type.mapping`), or use Avro/Protobuf (compact binary formats with explicit schemas) with a schema registry (stores schemas, checks compatibility of new versions).

14.11 Producing: `KafkaTemplate`, keys and partitions

How it works

1. `kafkaTemplate.send(topic, key, value)` serializes key and value, picks a partition (key hash), and adds the record to an in-memory batch.
2. A background I/O thread sends batches to partition leaders — batching (`linger.ms`, `batch.size`) is where Kafka's throughput comes from.
3. `send` returns a `CompletableFuture<SendResult<K, V>>` completing on broker acknowledgement (its `RecordMetadata` holds topic, partition, offset), or exceptionally if the send failed after the client's own retries.

```java
@Service
public class OrderProducer {
    private final KafkaTemplate<String, Object> kafka;
    // constructor ...

    /** Key = customer: all events of one customer go to the same partition, so they stay in order. */
    public CompletableFuture<SendResult<String, Object>> publish(OrderEvent event) {
        return kafka.send("orders", event.customer(), event);
    }
}
```

The test sends six orders for three customers and prints where each one landed:

```java
RecordMetadata meta = producer.publish(e).join().getRecordMetadata();
System.out.printf("%s key=%-5s -> %s-%d@%d%n", e.orderId(), e.customer(), meta.topic(), meta.partition(), meta.offset());
```

Output (`ProducerConsumerTest`):

```
--- producer: send(topic, key, value).get() -> RecordMetadata ---
o-1 key=alice -> orders-0@0
o-2 key=dave  -> orders-1@0
o-3 key=carol -> orders-2@0
o-4 key=alice -> orders-0@1
o-5 key=dave  -> orders-1@1
o-6 key=carol -> orders-2@1
```

`alice` always lands on partition 0, `dave` on 1, `carol` on 2 — the hash is deterministic. An early version used `alice`, `bob`, `carol` → partitions 0, 0, 2: two keys shared partition 0 and partition 1 got nothing. **Hashing doesn't promise an even spread**, and a "hot" key (one huge customer) overloads one partition.

Common mistakes

- **Fire-and-forget without checking the future** — a failed send is only logged. Attach `whenComplete`, or `join()` where the caller must know.
- **No key when order matters** — events of one order land on different partitions, out of order.
- **Changing the partition count later** — `hash(key) % N` changes, keys move, ordering across the change is lost.
- **Huge messages** — the default maximum is about 1 MB; store big payloads elsewhere and send a reference.

14.12 Consuming: `@KafkaListener`, concurrency and consumer groups

How it works

1. Boot's auto-configured `ConcurrentKafkaListenerContainerFactory` creates a **listener container** per `@KafkaListener`.
2. `concurrency = "3"` creates three consumers (threads) in the same group, each assigned a share of partitions.
3. Each thread loops: `poll()` a batch, call your method per record, then **commit the offsets** (default ack mode `BATCH`: after the whole polled batch).
4. If your method throws, the container's **error handler** decides (14.14).

```java
/** Group "shipping", 3 consumer threads: the 3 partitions are shared out between them. */
@KafkaListener(id = "shipping", topics = "orders", groupId = "shipping", concurrency = "3")
void ship(ConsumerRecord<String, OrderEvent> record) {
    received.add("shipping", record.value().orderId() + " key=" + record.key()
            + " p" + record.partition() + " on " + Thread.currentThread().getName());
}

/** Group "billing", 1 thread: a different group gets its OWN copy of every message. */
@KafkaListener(id = "billing", topics = "orders", groupId = "billing")
void bill(@Payload OrderEvent event,
          @Header(KafkaHeaders.RECEIVED_PARTITION) int partition,
          @Header(KafkaHeaders.OFFSET) long offset) {
    received.add("billing", event.orderId() + " p" + partition + "@" + offset
            + " on " + Thread.currentThread().getName());
}
```

A listener method can take the whole `ConsumerRecord`, the converted payload plus `@Header` values, an `Acknowledgment` for manual commits, or a `List<...>` for batch listeners.

The test first waits until the group has **settled** (each consumer owns one partition) — in the first run the first consumer to join briefly owned all three and processed everything.

```java
var shipping = (ConcurrentMessageListenerContainer<?, ?>) registry.getListenerContainer("shipping");
await().atMost(Duration.ofSeconds(30)).until(() -> shipping.getContainers().stream()
        .allMatch(c -> c.getAssignedPartitions() != null && c.getAssignedPartitions().size() == 1));
```

Output:

```
assignment: shipping-0 -> [orders-0]
assignment: shipping-1 -> [orders-1]
assignment: shipping-2 -> [orders-2]
...
--- group 'shipping' (concurrency 3) ---
o-1 key=alice p0 on shipping-0-C-1
o-2 key=dave p1 on shipping-1-C-1
o-3 key=carol p2 on shipping-2-C-1
o-4 key=alice p0 on shipping-0-C-1
...
--- group 'billing' (concurrency 1) ---
o-1 p0@0 on billing-0-C-1
o-2 p1@0 on billing-0-C-1
... (o-3 to o-6, all on billing-0-C-1)
```

- In `shipping` each partition had its own thread (`shipping-N-C-1` = listener id, consumer index, "C" for consumer thread); all of `alice`'s orders went to one thread, in order.
- `billing`, a **different group**, got **all six** records again on its single thread: publish-subscribe across groups, queue semantics within one group.

Commit modes

spring.kafka.listener.ack-mode	Commits
`BATCH` (default)	after all records from one `poll()` were processed
`RECORD`	after each record. Safer and slower
`TIME` / `COUNT` / `COUNT_TIME`	after a time or a number of records
`MANUAL` / `MANUAL_IMMEDIATE`	when your code calls `acknowledgment.acknowledge()`

spring-kafka turns Kafka's auto-commit off (`enable.auto.commit=false`); the container commits **after** processing → at-least-once. After a crash the last batch's uncommitted records are redelivered, so the consumer must be idempotent.

Common mistakes

- **More consumers than partitions** — the extras idle.
- **Slow processing** — a `poll()` batch longer than `max.poll.interval.ms` (5 minutes default) makes Kafka assume the consumer died, rebalance, and have another member reprocess it. Reduce `max.poll.records` or speed up.
- **Blocking the listener on remote calls without timeouts** — one hanging HTTP call stops a whole partition.
- **One `groupId` for two purposes** — they split the records instead of each getting all.

14.13 Testing with `@EmbeddedKafka`

What it is

`spring-kafka-test` starts a **real Kafka broker inside the test JVM** (KRaft mode, random port) — no Docker, no installation — and publishes its address as `spring.embedded.kafka.brokers`, which `application.properties` already reads.

```
@SpringBootTest
@EmbeddedKafka(partitions = 1, brokerProperties = {
        "transaction.state.log.replication.factor=1", "transaction.state.log.min.isr=1"})
abstract class KafkaTestBase {}

class ProducerConsumerTest extends KafkaTestBase { ... }
class ErrorHandlingTest extends KafkaTestBase { ... }
class ProducerGuaranteesTest extends KafkaTestBase { ... }
```

- All three classes inherit the **same** annotations, so the test-context cache reuses one context and broker (ch12).
- `partitions = 1` applies to topics in `@EmbeddedKafka(topics = ...)` and auto-created ones; our `NewTopic` beans set their own counts.
- A single broker can't store the transaction log at the default replication factor of 3; the two broker properties make Kafka transactions (14.16) work.
- Asynchronous consumption is asserted with **Awaitility**, never `Thread.sleep` and hope.

Test style	Speed	Realism	Needs
Unit test of the listener method (call it directly)	ms	logic only	nothing
`@EmbeddedKafka`	seconds	real protocol, real client, one broker	spring-kafka-test
Testcontainers Kafka + `@ServiceConnection`	slower	the real broker image you run in production	Docker (ch12)

Timing (machine-dependent): the whole `kafka-orders` suite (6 tests, one broker) takes about 12 s of test time (surefire: 5.2 s + 4.9 s + 2.2 s), mostly broker start-up and back-off waits.

14.14 Error handling: `DefaultErrorHandler` and the dead-letter topic

The problem it solves

A listener throws. Re-reading the same record forever lets one that can **never** succeed (bad data, a poison pill) block its partition; skipping it loses data. The standard answer:

1. **Retry** a few times with back-off for errors that may pass (timeouts, a restarting service).
2. **Don't retry** errors that can't (validation, deserialization).
3. After the last attempt, **publish to a dead-letter topic (DLT)** with headers describing the failure, **commit** the offset, move on. Someone inspects the DLT, fixes the cause, and replays.

How it works in spring-kafka

- `DefaultErrorHandler` is the container's error handler. On failure it **seeks** the partition back to the failed record so the next `poll()` returns it — a **blocking** retry: the partition waits.
- Its `BackOff` sets how often and how long. Default `FixedBackOff(0, 9)`: 9 retries, no delay, 10 attempts.
- On exhaustion it calls a `ConsumerRecordRecoverer`; `DeadLetterPublishingRecoverer` sends the record to the DLT.
- **Fatal by default** (no retries): `DeserializationException`, `MessageConversionException`, `ConversionException`, `MethodArgumentResolutionException`, `NoSuchMethodException`, `ClassCastException`. Add more with `addNotRetryableExceptions(...)`.
- Boot applies a `CommonErrorHandler` bean to its auto-configured container factory — declaring it suffices.

> **Version note:** since **spring-kafka 3.3** (so all 4.x) the default DLT name is `<topic>-dlt`; up to 3.2 it was `<topic>.DLT`, as most articles still show. Our first run proved it: `Destination resolver returned non-existent partition payments-dlt-0`, because we had created `payments.DLT`.

```
@Bean
DefaultErrorHandler errorHandler(KafkaTemplate<String, Object> jsonTemplate, ProducerFactory<String, Object> pf) {
    // A record that could not be deserialized has only raw bytes: publish those bytes unchanged.
    KafkaTemplate<String, Object> bytesTemplate =
            new KafkaTemplate<>(pf, Map.of(ProducerConfig.VALUE_SERIALIZER_CLASS_CONFIG, ByteArraySerializer.class));
    Map<Class<?>, KafkaOperations<?, ?>> templates = new LinkedHashMap<>();
    templates.put(byte[].class, bytesTemplate);
    templates.put(Object.class, jsonTemplate);

    var recoverer = new DeadLetterPublishingRecoverer(templates);   // default target: same topic + "-dlt", same partition
    var handler = new DefaultErrorHandler(recoverer, new FixedBackOff(100, 2));
    handler.addNotRetryableExceptions(IllegalArgumentException.class);   // bad data will never get better
    return handler;
}
```

Why two templates? A normal failed `OrderEvent` is re-published as JSON; a **poison pill** was never deserialized, so the recoverer has only raw `byte[]`. The map picks the template by value type; the `LinkedHashMap` keeps `byte[]` ahead of the `Object` catch-all.

The listener simulates three kinds of payment:

```
@KafkaListener(id = "payments", topics = "payments", groupId = "payments")
void charge(OrderEvent e) {
    int attempt = attempts.computeIfAbsent(e.orderId(), k -> new AtomicInteger()).incrementAndGet();
    received.add("payments", e.orderId() + " attempt " + attempt);
    if (e.cents() < 0) throw new IllegalArgumentException("negative amount " + e.cents());
    if (e.orderId().equals("p-down") || (e.orderId().equals("p-flaky") && attempt < 3))
        throw new TransientPaymentException("provider timeout");
    received.add("payments", e.orderId() + " CHARGED");
}

/** Reads the dead-letter topic as plain text (overrides the JSON deserializer for this listener only). */
@KafkaListener(id = "payments-dlt", topics = "payments-dlt", groupId = "dlt-inspector",
        properties = "value.deserializer=org.apache.kafka.common.serialization.StringDeserializer")
void inspectDeadLetter(String value,
                       @Header(KafkaHeaders.DLT_EXCEPTION_CAUSE_FQCN) String causeClass,
                       @Header(KafkaHeaders.DLT_ORIGINAL_TOPIC) String originalTopic) { ... }
```

The test sends `p-ok`, `p-flaky` (fails twice, then works), `p-down` (always fails), `p-bad` (negative amount) and a raw string `this is not JSON`:

Output (`ErrorHandlingTest.blockingRetriesThenDeadLetterTopic`):

```
--- payments listener ---
p-ok attempt 1
p-ok CHARGED
p-flaky attempt 1
...
p-flaky attempt 3
p-flaky CHARGED
p-down attempt 1
...
p-down attempt 3
p-bad attempt 1
--- payments-dlt ---
{"orderId":"p-down","customer":"carol","cents":300} <- payments, cause TransientPaymentException
{"orderId":"p-bad","customer":"dave","cents":-5} <- payments, cause IllegalArgumentException
this is not JSON <- payments, cause DeserializationException
```

Record	What happened
`p-flaky`	1 attempt + 2 retries, succeeded on the third. No DLT
`p-down`	3 attempts, then DLT with the cause class in a header
`p-bad`	`IllegalArgumentException` is not retryable: 1 attempt, then DLT
`this is not JSON`	the listener was **never called**: `ErrorHandlingDeserializer` made it a fatal `DeserializationException`; the original bytes went to the DLT unchanged

The listener exception is wrapped in `ListenerExecutionFailedException`, but the classifier walks the **cause chain**, so `IllegalArgumentException` was recognized. DLT records carry headers `kafka_dlt-original-topic`, `-original-partition`, `-original-offset`, `-exception-fqcn`, `-exception-cause-fqcn`, `-exception-message`, `-exception-stacktrace` (constants `KafkaHeaders.DLT_*`).

Trade-offs of blocking retries

Pro	Con
Order within the partition is kept	the partition **stops** while one record is retried. A long back-off stalls everything behind it
Simple, no extra topics	the total back-off must stay well under `max.poll.interval.ms`, or the consumer is kicked out of the group

Common mistakes

- **The default back-off** (10 instant attempts) hammers a struggling dependency. Set a real one, e.g. `ExponentialBackOff`.
- **No `ErrorHandlingDeserializer`** — a poison pill fails inside `poll()` before your code, so the error handler never gets it.
- **A DLT with fewer partitions than the source** — the recoverer targets the **same partition number**. Create enough, or use a custom destination resolver.
- **Nobody watches the DLT** — alert on its size; document replay.

14.15 Non-blocking retries with `@RetryableTopic`

What it is

Instead of seeking back and blocking the partition, a failed record is **re-published to a retry topic** and its offset committed; the main topic keeps flowing. A separate consumer reads the retry topic after the back-off; after the last attempt the record goes to the DLT and a `@DltHandler` method runs.

```
@RetryableTopic(attempts = "3", backOff = @BackOff(delay = 200))
@KafkaListener(id = "invoices", topics = "invoices", groupId = "invoices")
void createInvoice(OrderEvent e, @Header(KafkaHeaders.RECEIVED_TOPIC) String topic) {
    received.add("invoices", e.orderId() + " from " + topic);
    if (e.orderId().startsWith("inv-bad")) throw new IllegalStateException("tax service unavailable");
}

@DltHandler
void giveUp(OrderEvent e, @Header(KafkaHeaders.RECEIVED_TOPIC) String topic) {
    received.add("invoices", e.orderId() + " DLT handler, from " + topic);
}
```

Output (`ErrorHandlingTest.nonBlockingRetryTopics`, `inv-bad` sent first, then `inv-ok`):

```
--- @RetryableTopic ---
inv-bad from invoices
inv-ok from invoices
inv-bad from invoices-retry
inv-bad from invoices-retry
inv-bad DLT handler, from invoices-dlt
```

`inv-ok` was processed **before** `inv-bad`'s retries — the failing record no longer blocked the partition. The framework created `invoices-retry` and `invoices-dlt` itself. With a fixed delay **one** retry topic is reused for all attempts (annotation default since spring-kafka 3.2; `RetryTopicConfigurationBuilder` got the same default in 4.1). In spring-kafka 4.0 the attribute changed from Spring Retry's `backoff = @Backoff` to `backOff = @BackOff` (`org.springframework.kafka.annotation.BackOff`).

Blocking vs non-blocking

	`DefaultErrorHandler` (blocking)	`@RetryableTopic` (non-blocking)
Partition during retries	stopped	keeps flowing
Ordering	kept	**lost** for the retried record: later records overtake it
Extra topics	only the DLT	retry topic(s) + DLT
Long delays (minutes)	dangerous (`max.poll.interval.ms`)	fine
Choose when	order matters (account balance updates)	independent records, slow-recovering dependencies

When not to use it

When per-key event order matters: if `OrderCancelled` overtakes a retried `OrderPlaced`, the consumer sees a cancel for an unknown order.

14.16 Producer guarantees: idempotence and transactions

Idempotent producer

The problem: the broker writes a batch but the acknowledgement is lost; the producer retries and the batch is written **twice**. The **idempotent producer** fixes it: the broker assigns a producer id, the producer numbers each batch per partition, and the broker drops sequence numbers already written. **On by default** since Kafka 3.0, with `acks=all`.

```
var effective = new ProducerConfig(bootProducerFactory.getConfigurationProperties());
System.out.println("enable.idempotence = " + effective.getBoolean(ProducerConfig.ENABLE_IDEMPOTENCE_CONFIG));
System.out.println("acks               = " + effective.getString(ProducerConfig.ACKS_CONFIG));
System.out.println("max.in.flight      = " + effective.getInt(ProducerConfig.MAX_IN_FLIGHT_REQUESTS_PER_CONNECTION));
```

Output:

```
enable.idempotence = true
acks               = -1
max.in.flight      = 5
```

`acks=-1` means `all`. Idempotence requires `max.in.flight ≤ 5` and keeps per-partition order across retries. It removes duplicates from **producer retries** only — not your app calling `send` twice for one business event.

Transactional producer

A **Kafka transaction** makes several sends (several topics/partitions, plus consumer offsets) **atomic**; `isolation.level=read_committed` consumers see all or none.

```
var pf = new DefaultKafkaProducerFactory<String, String>(props);
pf.setTransactionIdPrefix("ledger-tx-");        // turns on transactions (and requires idempotence)
var tx = new KafkaTemplate<>(pf);

tx.executeInTransaction(t -> {                  // committed: both records become visible together
    t.send("ledger", "acc-1", "debit 50");
    t.send("ledger", "acc-2", "credit 50");
    return null;
});
tx.executeInTransaction(t -> {                  // aborted: the record stays invisible
    t.send("ledger", "acc-1", "debit 999");
    t.flush();                                  // the record really reaches the broker...
    throw new IllegalStateException("crash before commit");   // ...but the transaction is aborted
});
```

Output (`ProducerGuaranteesTest.transactionalProducerAndReadCommitted`):

```
isolation.level=read_committed   -> [debit 50, credit 50]
isolation.level=read_uncommitted -> [debit 50, credit 50, debit 999]
```

The aborted record **is in the log** (`read_uncommitted` shows it); the broker writes an abort marker and `read_committed` consumers skip it. Without the `flush()` the record never left the client and both lists were equal in our first run — proving nothing.

In Boot, `spring.kafka.producer.transaction-id-prefix=tx-` makes the auto-configured `KafkaTemplate` transactional and creates a `KafkaTransactionManager`; then every `send` must run in a transaction (`executeInTransaction`, `@Transactional("kafkaTransactionManager")`, or a listener container starting one). Consumers must set `spring.kafka.consumer.isolation-level=read_committed` (Kafka default: `read_uncommitted`).

What Kafka transactions do not give you

Exactly-once holds for "consume from Kafka → process → produce to Kafka, committing input offsets in the same transaction". It does **not** cover a database write plus a Kafka send: there's no two-phase commit (a coordinator asking every system "can you commit?", then telling all to commit) between them. The old `ChainedKafkaTransactionManager` (deprecated since spring-kafka 2.7) just committed one after the other — best effort, not atomic. For database + broker, use the outbox.

14.17 The transactional outbox

The problem it solves: the dual write

```
@Transactional
public void placeOrder(...) {
    orderRepository.save(order);                // 1. database
    kafkaTemplate.send("orders", event);        // 2. broker
}                                               // 3. commit
```

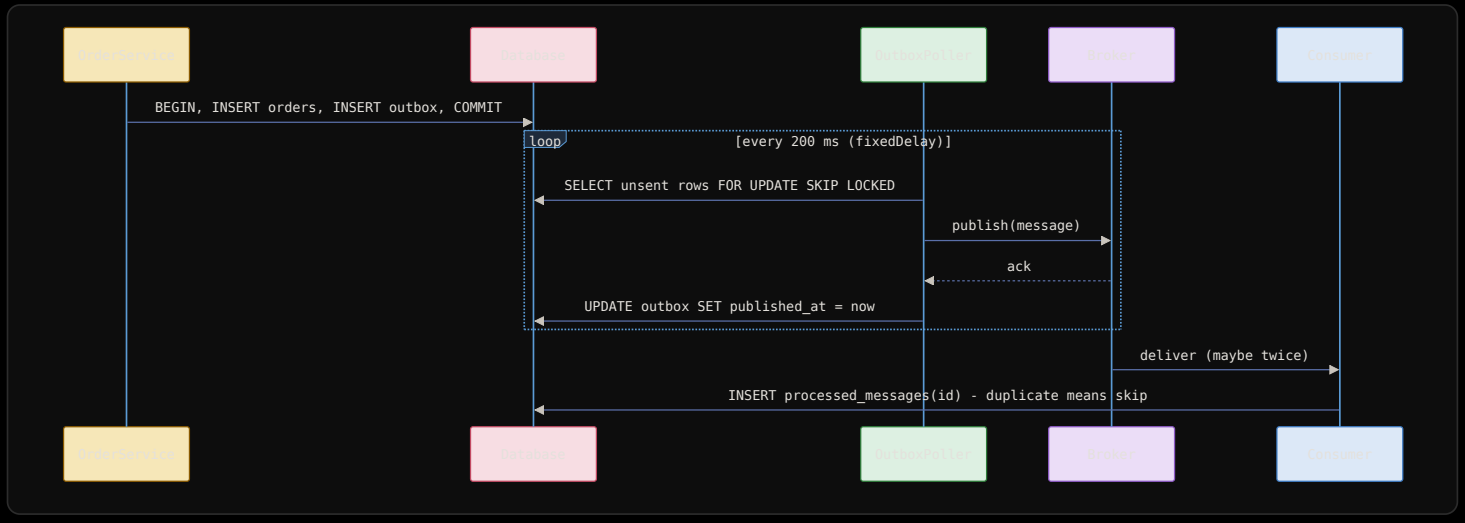
Every ordering of these steps fails somewhere:

Failure	Result
send succeeds, then the commit fails	consumers hear about an order that **does not exist**
commit succeeds, then the app crashes before the send	the order exists and **nobody is told**
send after the commit, broker down	same: lost event, unless you build retries and persistence... which is an outbox

This is the **dual-write problem**: database and broker cannot share one transaction.

How it works

1. In the **same local database transaction** as the business change, insert an `outbox` row describing the event.
2. Commit: order and event exist together, or neither does.
3. A separate **message relay** reads unsent rows in order, publishes them, and marks them sent — a **poller** (shown here) or **change data capture** (CDC: streaming committed changes out of the database; Debezium reads the write-ahead log, where the database records each change before applying it).
4. Consumers get **at-least-once** delivery, so they must be **idempotent**, de-duplicating on each message's unique id.



The pattern without Spring: Saga notes — Part 13D, implementing the saga log, event-driven architecture notes, System-design patterns, Module 10 — the outbox pattern. Here it runs.

The schema

```
-- the outbox: messages waiting to be sent, written in the SAME transaction as the order
CREATE TABLE IF NOT EXISTS outbox (
    id            IDENTITY PRIMARY KEY,                -- also gives the publishing order
    message_id    UUID         NOT NULL UNIQUE,        -- consumers de-duplicate on this
    aggregate_id  VARCHAR(50)  NOT NULL,               -- becomes the Kafka key
    event_type    VARCHAR(50)  NOT NULL,
    payload       VARCHAR(2000) NOT NULL,              -- JSON
    created_at    TIMESTAMP    DEFAULT CURRENT_TIMESTAMP NOT NULL,
    published_at  TIMESTAMP,                           -- NULL = not sent yet
    attempts      INT          DEFAULT 0 NOT NULL
);
CREATE INDEX IF NOT EXISTS outbox_unpublished ON outbox (published_at, id);
-- consumer side: ids of messages already handled (idempotent consumer)
CREATE TABLE IF NOT EXISTS processed_messages (message_id UUID PRIMARY KEY);
```

Writing the order and the event together

```
/** ONE local transaction: the order row and the outbox row commit together or not at all. */
@Transactional
public long placeOrder(String item, long cents, boolean failBeforeCommit) {
    var keys = new GeneratedKeyHolder();
    jdbc.sql("INSERT INTO orders(item, cents) VALUES (?, ?)").params(item, cents).update(keys, "id");
    long orderId = keys.getKey().longValue();

    String payload = json.writeValueAsString(new OrderPlaced(orderId, item, cents));   // Jackson 3 JsonMapper
    jdbc.sql("""
            INSERT INTO outbox(message_id, aggregate_id, event_type, payload)
            VALUES (?, ?, 'OrderPlaced', ?)""")
        .params(UUID.randomUUID(), "order-" + orderId, payload)
        .update();

    if (failBeforeCommit) throw new IllegalStateException("crash before commit");
    return orderId;       // no broker call here: nothing can be half-done
}
```

The poller (message relay)

```
@Scheduled(fixedDelayString = "${outbox.poller.delay-ms}")
public void poll() { publishBatch(); }

@Transactional
public int publishBatch() {
    // FOR UPDATE SKIP LOCKED: several app instances can poll at once without sending the same row twice
    List<OutboxRow> rows = jdbc.sql("""
            SELECT id, message_id, aggregate_id, event_type, payload
            FROM outbox
            WHERE published_at IS NULL
            ORDER BY id
            FETCH FIRST 10 ROWS ONLY
            FOR UPDATE SKIP LOCKED""")
        .query(OutboxRow.class).list();

    int sent = 0;
    for (OutboxRow row : rows) {
        try {
            publisher.publish(new Message(row.messageId(), row.aggregateId(), row.eventType(), row.payload()));
        } catch (Exception e) {
            jdbc.sql("UPDATE outbox SET attempts = attempts + 1 WHERE id = ?").param(row.id()).update();
            log.warn("outbox row {} not published ({}), will retry", row.id(), e.getMessage());
            break;                                  // keep order: do not send later rows before this one
        }
        jdbc.sql("UPDATE outbox SET published_at = CURRENT_TIMESTAMP, attempts = attempts + 1 WHERE id = ?")
            .param(row.id()).update();
        sent++;
    }
    return sent;
}
```

Design choices:

- `fixedDelay`, not `fixedRate`: a slow round never overlaps the next (14.7).
- `FOR UPDATE SKIP LOCKED` (PostgreSQL 9.5+, MySQL 8.0+, H2 2.x as in this demo): several instances' pollers lock different rows (SQL ch13 explains row locks).
- **Stop at the first failure** (`break`) so later messages never overtake an earlier one.
- `MessagePublisher` is an interface; the demo's `InMemoryBroker` can simulate an outage (`DOWN`) or a lost acknowledgement (`ACK_LOST`: stored, then a timeout thrown). With Kafka it is one line — the `.get()` with timeout makes "published" mean "acknowledged":

```
kafkaTemplate.send("orders", m.key(), m.payload()).get(10, TimeUnit.SECONDS);
```

Proving it

The tests switch the scheduler off (`outbox.poller.enabled=false`) and call `publishBatch()` themselves for determinism; one extra test leaves it on.

Output (`OutboxTest`, `OutboxSchedulerTest`):

```
after rollback: orders=0 outbox=0
after commit:   orders=1 outbox=1 unpublished=1
outbox payload: {"orderId":5,"item":"book","cents":1999}
poller sent 1 -> broker has 1, unpublished=0

WARN  c.o.OutboxPoller - outbox row 2 not published (broker unreachable), will retry
broker DOWN -> sent 0, unpublished=2, attempts of first row=1
broker UP   -> sent 2, unpublished=0
broker order: [order-2, order-3]

WARN  c.o.OutboxPoller - outbox row 1 not published (no ack from broker), will retry
broker holds 2 copies of message eab1f5f0...
consumer handled: [true, false] -> stock_reservations=1

published by the scheduled poller within ~200 ms: OrderPlaced {"orderId":1,"item":"desk","cents":25000}
```

(Id numbers depend on test order: identity (auto-increment) columns keep counting after `DELETE`.)

Scenario	What the test proved
Crash before commit	**no** order and **no** outbox row: nothing to publish, nothing wrong published
Commit	one order, one unpublished outbox row, published on the next poll
Broker down	nothing lost: rows stay unpublished with `attempts=1`, and go out in order once the broker is back
Ack lost	the broker got the message but the poller didn't know, so it sent it **again**. The consumer's `processed_messages` insert failed with `DuplicateKeyException` on the second copy, so the stock was reserved **once**

The idempotent consumer:

```
@Transactional
public boolean handle(Message m) {
    try {
        jdbc.sql("INSERT INTO processed_messages(message_id) VALUES (?)").param(m.messageId()).update();
    } catch (DuplicateKeyException alreadySeen) {
        return false;                              // duplicate: skip silently
    }
    jdbc.sql("INSERT INTO stock_reservations(order_id) VALUES (?)").param(m.key()).update();
    return true;
}
```

Id insert and business change share **one** transaction: if the change fails, the id rolls back too and a redelivery is processed normally.

Trade-offs

Poller (this demo)	CDC (Debezium outbox event router)
plain SQL + `@Scheduled`, easy to understand	no polling load, lower latency
adds query load; latency = poll interval	extra infrastructure: Kafka Connect (Kafka's framework for moving data in and out), Debezium, access to the DB log
works on any database	needs DB support (Postgres logical decoding, MySQL binlog)

Housekeeping: delete or archive published rows (e.g. nightly, `published_at` older than 7 days), or the table grows forever.

Spring Modulith offers a ready-made variant for modular monoliths (one deployable split into well-separated modules): its *event publication registry* stores application events in a table before commit and re-delivers those whose listeners didn't complete; *event externalization* forwards selected events to Kafka or AMQP (Advanced Message Queuing Protocol, RabbitMQ's protocol).

When not to use it

- The message isn't tied to a database change (e.g. a metrics tick) — send directly.
- Losing an occasional event is acceptable (cache warm-up hints) — `@TransactionalEventListener(AFTER_COMMIT)` + send is simpler.

Common mistakes

- **Calling the broker inside the business transaction** "to be sure" — the dual write again, plus a DB connection held during a network call.
- **Non-idempotent consumers** — the outbox guarantees at-least-once, not exactly-once.
- **Publishing out of order** by continuing after a failure, or several pollers without `SKIP LOCKED` and key-based order.
- **Letting the outbox grow forever.**

In the interview

> **30-second answer:** "You can't atomically write to a database and publish to Kafka — the dual-write problem. The outbox pattern writes the event into an outbox table in the same local transaction as the business change. A relay (a poller with `SELECT ... FOR UPDATE SKIP LOCKED`, or Debezium CDC) publishes the rows and marks them sent. Delivery is at-least-once, because the relay can crash between publishing and marking, so consumers de-duplicate by message id in the same transaction as their own change."

- *Why not a Kafka transaction?* → It only covers Kafka; the database commit is outside it.
- *Ordering?* → Publish in id order, stop at the first failure, use the aggregate id as Kafka key so each aggregate's events share a partition (an **aggregate** is a business object changing as one unit, e.g. an order with its lines).
- *Poller or CDC?* → Start with a poller; switch to CDC when polling load or latency matters, or Kafka Connect is already there.
- *Why two copies in the "ack lost" test?* → Stored, but the poller saw a timeout and didn't mark the row, so it resent; the consumer's `processed_messages` insert rejected the duplicate.

14.18 RabbitMQ with Spring AMQP (configuration, not run)

What it is

RabbitMQ is a queue broker speaking AMQP 0-9-1. Producers publish to an **exchange**, never a queue; the exchange routes to zero or more **queues** via **bindings**; consumers read queues and **acknowledge** each message.

Exchange type	Routes by	Example
direct	exact routing key	`payment.failed` → `payments.alerts` queue
topic	pattern: `*` = one word, `#` = zero or more words	`order.placed.*` matches `order.placed.eu`
fanout	nothing: copies to every bound queue	broadcast "cache invalidated"
headers	message header values	rarely used

Other terms: **prefetch** — how many unacknowledged messages one consumer may hold; **dead-letter exchange (DLX)** — where rejected or expired messages go; **quorum queues** (RabbitMQ 3.8+) — replicated, durable queues; **competing consumers** — several consumers sharing one queue's messages.

`code/rabbit-orders` **compiles** (`mvn compile` passed) but is **not run**: it needs a broker (e.g. `docker run -p 5672:5672 rabbitmq:4-management`), so its folder has `.tools_run_skip`.

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-amqp</artifactId>   <!-- spring-amqp / spring-rabbit 4.1.x -->
</dependency>
```

```java
/** Boot's RabbitAdmin declares these exchanges, queues and bindings on the broker at startup. */
@Configuration(proxyBeanMethods = false)
public class RabbitConfig {
    @Bean TopicExchange ordersExchange() { return new TopicExchange("orders"); }
    @Bean DirectExchange deadLetterExchange() { return new DirectExchange("orders.dlx"); }

    @Bean Queue shippingQueue() {
        return QueueBuilder.durable("orders.shipping")
                .quorum()                                          // replicated queue type (RabbitMQ 3.8+)
                .deadLetterExchange("orders.dlx")                  // rejected messages go here...
                .deadLetterRoutingKey("orders.shipping.dlq")       // ...with this routing key
                .build();
    }
    @Bean Queue shippingDlq() { return QueueBuilder.durable("orders.shipping.dlq").build(); }

    /** Topic exchange: routing key "order.placed.eu" matches pattern "order.placed.*". */
    @Bean Binding shippingBinding() {
        return BindingBuilder.bind(shippingQueue()).to(ordersExchange()).with("order.placed.*");
    }
    @Bean Binding dlqBinding() {
        return BindingBuilder.bind(shippingDlq()).to(deadLetterExchange()).with("orders.shipping.dlq");
    }

    /** Boot sets a MessageConverter bean on RabbitTemplate and on the listener container factory. */
    @Bean MessageConverter jsonConverter() { return new JacksonJsonMessageConverter(); }
}

// producer: exchange, routing key, body (converted to JSON)
rabbit.convertAndSend("orders", "order.placed." + region, event);

// consumer: competing consumers on one queue
@RabbitListener(queues = "orders.shipping")
void ship(OrderEvent e) {
    if (e.cents() < 0) throw new IllegalArgumentException("bad amount");   // 1 attempt + 2 retries, then dead-lettered
    ...
}
```

```properties
spring.rabbitmq.host=localhost
spring.rabbitmq.listener.simple.prefetch=10
spring.rabbitmq.listener.simple.concurrency=2
# retry in the listener (blocking), then reject -> the queue's dead-letter exchange
spring.rabbitmq.listener.simple.retry.enabled=true
# retries AFTER the first attempt (Boot 4 replaced max-attempts with max-retries)
spring.rabbitmq.listener.simple.retry.max-retries=2
spring.rabbitmq.listener.simple.retry.initial-interval=500ms
spring.rabbitmq.listener.simple.default-requeue-rejected=false
# producer side: broker confirms + returns for unroutable messages
spring.rabbitmq.publisher-confirm-type=correlated
spring.rabbitmq.publisher-returns=true
spring.rabbitmq.template.mandatory=true
```

(Property names checked against Boot 4.1.1's `spring-boot-amqp` metadata. Boot 4 replaced `retry.max-attempts`, which counted the first attempt, with `retry.max-retries`, which doesn't.)

With Spring AMQP's default `AUTO` ack mode the container acks when your method returns normally; if it throws, the message is rejected and **requeued** unless `default-requeue-rejected=false` or the exception is `AmqpRejectAndDontRequeueException`. Requeueing an always-failing message loops forever — hence the DLX. **Publisher confirms** make the broker acknowledge each publish (RabbitMQ's counterpart of Kafka's `acks`); **returns** with `mandatory=true` report messages no queue was bound to receive.

Testing without Docker: Spring AMQP has no embedded broker. Use `@RabbitListenerTest` / `RabbitListenerTestHarness` (`spring-rabbit-test`) to spy on listeners, test listener methods as plain methods, or run the real broker via Testcontainers + `@ServiceConnection` (ch12).

Kafka or RabbitMQ?

Question	Leans Kafka	Leans RabbitMQ
Several independent consumers read the same stream, maybe later	yes: each group has its own offset, and records are retained	each consumer needs its own queue, and messages are gone after the ack
Replay history, rebuild a read model (a copy of data shaped for queries)	yes	no
Complex routing by key patterns or headers	limited	yes: exchanges
Per-message priorities, TTL (time to live: an expiry time), delayed delivery	no	yes (TTL, priorities; delays via a plugin)
Throughput	very high (batching, sequential disk I/O)	high, lower per node
Ordering	per partition, scales with partitions	per queue, weakened by competing consumers
Operational model	a cluster plus partitions to plan	simpler for small setups

Spring also supports **JMS** (`JmsTemplate`, `@JmsListener`; ActiveMQ Artemis, IBM MQ), **Apache Pulsar** (Spring for Apache Pulsar), and **Spring Cloud Stream**, one API with plug-in "binders" per broker (ch16).

14.19 Common mistakes (whole chapter)

Mistake	Symptom	Fix
No `@EnableAsync` / `@EnableScheduling`	`@Async` runs synchronously, `@Scheduled` never runs	add the annotation, on its own config class if you want to switch it with a property
Self-invocation of `@Async`	work runs on the caller's thread	call through another bean (4.5)
`@Async void` without a handler	failures only logged	return `CompletableFuture`, or add an `AsyncUncaughtExceptionHandler`
Declaring an `Executor` bean	Boot's `applicationTaskExecutor` disappears	`@Bean(defaultCandidate = false)` or `spring.task.execution.mode=force`
One default scheduler thread	jobs delay each other	`spring.task.scheduling.pool.size`
`@Scheduled` on N instances	job runs N times	ShedLock / leader election / idempotent job
Sending to Kafka inside a DB transaction	ghost events or lost events	transactional outbox
No key on ordered events	events processed out of order	key = aggregate id
Default `FixedBackOff(0, 9)`	retry storm against a failing service	real back-off + DLT
No `ErrorHandlingDeserializer`	poison pill blocks the partition	wrap the value deserializer
Looking for `orders.DLT` on spring-kafka 3.3+	DLT "empty"	the default name is now `orders-dlt`
Non-idempotent consumers	double charges after a rebalance or redelivery	de-duplicate by message id
`Thread.sleep` in async tests	flaky tests	Awaitility with a timeout

14.20 In the interview

Per-section blocks cover transactional listeners (14.3), `fixedRate` vs `fixedDelay` and clustered jobs (14.7), blocking vs retry-topic retries and poison pills (14.14–14.15) and the outbox (14.17). The rest:

1. **How does `@Async` work?** → `@EnableAsync` adds a proxy. Calls through the proxy are submitted to an executor, and the proxy returns `null`/a future at once. Self-invocation bypasses it.
2. **Which executor does `@Async` use in Boot?** → The one named in `@Async("…")`, else an `AsyncConfigurer`'s, else the `taskExecutor`/`applicationTaskExecutor` that Boot auto-configures: a `ThreadPoolTaskExecutor` with 8 core threads and an unbounded queue, or a virtual-thread `SimpleAsyncTaskExecutor` if `spring.threads.virtual.enabled=true`.
3. **What happens to an exception in an `@Async` method?** → For a future return type it is stored in the future. For `void` it goes to the `AsyncUncaughtExceptionHandler`, which only logs by default.
4. **How do you pass the MDC or security context to async threads?** → A `TaskDecorator` (Boot applies a `TaskDecorator` bean to its executor), `spring.task.execution.propagate-context=true`, or `DelegatingSecurityContextAsyncTaskExecutor` for security.

5. **Kafka partition vs consumer group?** → Partitions split a topic for parallelism and ordering. A group splits the partitions among its members. Different groups each get every record.
6. **How do you keep events for one order in sequence?** → Use the order id as the key, so they share one partition, and use blocking retries, not retry topics.
7. **What does `DefaultErrorHandler` do?** → It seeks back and retries per its `BackOff` (default 10 attempts, no delay). Then it calls the recoverer, e.g. `DeadLetterPublishingRecoverer` → `<topic>-dlt` (since spring-kafka 3.3). Fatal exceptions such as `DeserializationException` skip retries.
8. **Is Kafka exactly-once?** → The idempotent producer removes duplicates caused by producer retries. Transactions plus `read_committed` give exactly-once for Kafka-to-Kafka processing. Anything involving a database needs an outbox and idempotent consumers.
9. **How do you test Kafka code without Docker?** → `@EmbeddedKafka` from spring-kafka-test (an in-JVM KRaft broker), `spring.kafka.bootstrap-servers=${spring.embedded.kafka.brokers}`, and Awaitility for assertions.
10. **Kafka vs RabbitMQ in one sentence?** → Kafka is a replayable, partitioned log for streams and many independent readers. RabbitMQ is a routing queue broker for task distribution with per-message features.

Chapter TL;DR

- **Events:** `@EventListener` is synchronous; `@Async @EventListener` isn't, and gets no transaction or exception propagation. `@TransactionalEventListener` phases `BEFORE_COMMIT` / `AFTER_COMMIT` / `AFTER_ROLLBACK` / `AFTER_COMPLETION`; `@Async` + `AFTER_COMMIT` + `REQUIRES_NEW` is the usual combination — still not durable.
- **`@Async`:** needs `@EnableAsync`; proxy + executor; Boot's `applicationTaskExecutor` (8 threads, `task-` names, unbounded queue); pools via `@Async("name")` and `defaultCandidate = false`; MDC via `TaskDecorator`; `void` errors → `AsyncUncaughtExceptionHandler`, future errors → the future.
- **Virtual threads:** `spring.threads.virtual.enabled=true` → one virtual thread per task, no pool or queue; limit concurrency yourself.
- **`@Scheduled`:** needs `@EnableScheduling`; `fixedRate` (start-to-start) vs `fixedDelay` (end-to-start) vs six-field `cron`; one scheduler thread by default; overlap depends on the scheduler; one run per instance in a cluster.
- **Kafka:** Boot 4.1.1 → spring-kafka 4.1.1, kafka-clients 4.2.1, `spring-boot-starter-kafka` (+ `spring-boot-starter-jackson`). Key → partition → order; a group has one reader per partition, every group gets everything. `JacksonJsonSerializer` (Jackson 3) + `ErrorHandlingDeserializer`. `DefaultErrorHandler` + `DeadLetterPublishingRecoverer` → `<topic>-dlt`; `@RetryableTopic` for non-blocking retries. Idempotence on by default; transactions + `read_committed` for atomic multi-sends. Test with `@EmbeddedKafka`.
- **RabbitMQ:** exchanges → bindings → queues, acks, prefetch, DLX, publisher confirms; `spring-boot-starter-amqp`, `@RabbitListener`, `JacksonJsonMessageConverter`.
- **Outbox:** event row in the same transaction, relayed by a `fixedDelay` poller (`FOR UPDATE SKIP LOCKED`) or CDC; consumers de-duplicate by message id.
- Next: Chapter 15 — Observability and production.

Chapter 15 — Observability and Production

This chapter is about keeping the application working once users depend on it: knowing whether it is healthy, how fast it is, which request failed and why, shutting down without dropping requests, and starting quickly in **containers** (isolated packages of an app plus everything it needs, run by Docker or Kubernetes). Boot's "production-ready features": **Actuator** (management endpoints), **Micrometer** (metrics and the Observation API), **Micrometer Tracing + OpenTelemetry** (distributed tracing: following one request across services), **structured logging**, graceful shutdown (finishing in-flight requests before stopping), virtual threads, and faster startup with AOT (ahead-of-time) caches and native images (§15.11).

The code lives in two Spring Boot 4.1.1 / Framework 7.0 projects on Java 25 (outputs are real `mvn -q verify` output, log noise trimmed as `...`):

- `code/observability-demo/` — Actuator, health, metrics, Prometheus, tracing, structured logs (8 tests).
- `code/production-runtime/` — graceful shutdown and virtual threads (4 tests), plus `code/aot-cache-demo.sh` for the startup comparison.

> **Prerequisites.** Configuration properties and profiles: Chapter 3. Security filter chains: Chapter 10. `RestClient`: Chapter 6. `@ConcurrencyLimit`: 4.7. System-design view of health checks and tracing: Operating in Production.

15.1 Observability in one page

What it is

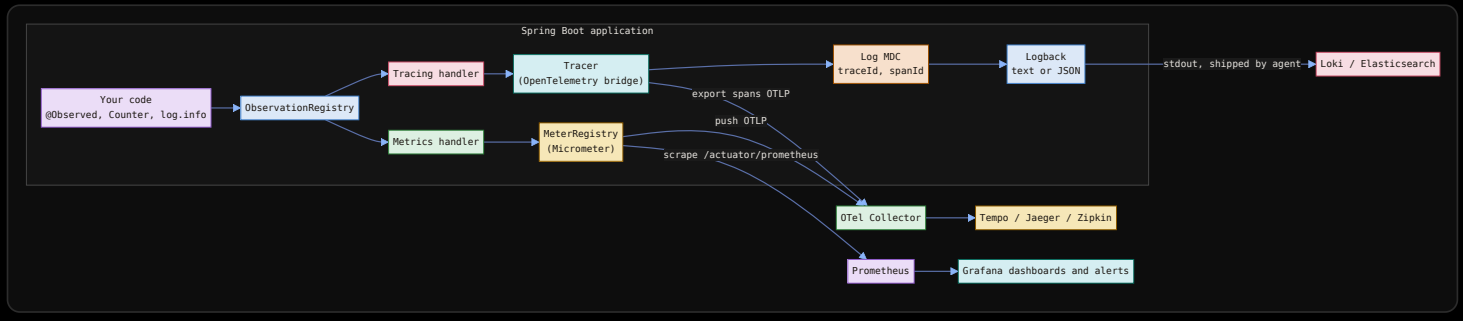
Observability is how well you can understand what a running system is doing from the data it emits — without attaching a debugger or adding code. The data comes in three kinds, the **three pillars**:

Pillar	What it is	Answers	Spring tool
Logs	timestamped text or JSON events	"what exactly happened in this request?"	SLF4J + Logback (Java's standard logging API and Boot's default implementation), structured logging
Metrics	numbers aggregated over time (counts, durations, gauges)	"how many, how fast, how full — right now and as a trend?"	Micrometer `MeterRegistry`
Traces	the path of one request through many services, as a tree of timed **spans** (one span = one timed step of work, §15.7)	"where did this slow request spend its time?"	Micrometer Tracing + OpenTelemetry

Like a hospital patient: **metrics** are the monitor (heart rate over time), **logs** the nurse's notes, the **trace** the record of departments visited and for how long. The monitor says *something* is wrong, the visit record *where*, the notes *what*.

Monitoring is the older, narrower idea: watching known metrics and alerting on thresholds (e.g. "error rate above 1%"). Observability adds asking new questions ("which customer's requests are slow?") from rich, correlated data.

How Spring wires it together



- **Micrometer** is a vendor-neutral metrics **facade** ("SLF4J for metrics"): you write to a `MeterRegistry`; a registry implementation (Prometheus — the popular open-source metrics database, OTLP, Datadog, ...) formats and ships the numbers.
- **Micrometer Observation** is one API producing *both* a metric and a span for a piece of work.
- **Micrometer Tracing** is a facade over a tracer; Boot 4 supports **OpenTelemetry** (`spring-boot-starter-opentelemetry`) and **Brave** with Zipkin (`spring-boot-starter-zipkin`).
- The tracer puts the current `traceId` and `spanId` into the logging **MDC** (Mapped Diagnostic Context, a per-thread map log patterns can print), linking every log line to its trace.

Real-world examples

- Grafana's "LGTM" stack (Loki logs, Grafana, Tempo traces, Mimir/Prometheus metrics) is a common open-source destination for these three outputs.
- The OpenTelemetry Collector is widely deployed as a **sidecar** (helper container next to the app in the same pod) or agent receiving OTLP (the OpenTelemetry protocol) and forwarding to any **backend** (storage + UI), so apps don't hard-code the vendor.

15.2 Spring Boot Actuator

What it is

Actuator adds **management endpoints**: small built-in HTTP (or JMX — Java Management Extensions, the JVM's older management protocol) APIs for operators — health, metrics, info, loggers, environment, thread dumps and more. Add `spring-boot-starter-actuator` and they appear under `/actuator` — the car's service port that mechanics (Kubernetes, Prometheus) plug into and drivers never see.

The endpoints

ID	Shows / does	Notes
`health`	UP/DOWN status, components, probes	the only endpoint exposed over HTTP by default
`info`	build, git, Java, custom `info.*` properties	contributors switched on by properties
`metrics`	list of meters; `/metrics/{name}` for one	JSON, for humans
`prometheus`	all meters in Prometheus text format	needs `micrometer-registry-prometheus`
`loggers`	view and **change** log levels at runtime	`POST /actuator/loggers/com.acme {"configuredLevel":"DEBUG"}`
`env`, `configprops`	resolved properties, `@ConfigurationProperties` beans	values masked by default (`show-values=never`)
`beans`, `conditions`, `mappings`	beans, auto-configuration report, request mappings	great for debugging (Chapter 3)
`threaddump`, `heapdump`	JVM thread dump (what every thread is doing), heap dump file (every object in memory)	heap dumps can contain secrets — protect them
`httpexchanges`	recent requests/responses	needs an `HttpExchangeRepository` bean
`scheduledtasks`, `caches`, `flyway`, `liquibase`, `quartz`, `sessions`	feature-specific	appear when the feature is present
`startup`	startup steps and their duration	needs `BufferingApplicationStartup`
`shutdown`	shuts the app down	access `none` by default

Exposure vs access

Two separate switches decide whether an endpoint can be called:

1. **Access** (since Boot 3.4, replacing the older `enabled` flags): is the endpoint allowed at all, and may it change things? Values `none`, `read-only`, `unrestricted`.
2. **Exposure**: is it reachable over **HTTP** or **JMX**? By default only `health` is exposed on both.

```
# expose more endpoints over HTTP ("*" = all; quote it in YAML)
management.endpoints.web.exposure.include=health,info,metrics,prometheus,env,loggers
management.endpoints.web.exposure.exclude=heapdump

# access control
management.endpoint.loggers.access=read-only        # can read levels, cannot change them
management.endpoints.access.default=none            # opt-in style: everything off ...
management.endpoint.health.access=read-only         # ... except what you list
management.endpoints.access.max-permitted=read-only # hard ceiling for the whole app

# a separate port (and optionally address) for all management endpoints
management.server.port=8081
management.endpoints.web.base-path=/manage          # default /actuator
```

A separate **management port** is common in Kubernetes: the public **ingress** (entry point for outside traffic) routes only to 8080, while probes and Prometheus reach 8081 inside the cluster — keeping operational endpoints off the public path.

Securing actuator endpoints

With Spring Security on the classpath you decide who calls what with `EndpointRequest` matchers. In Boot 4 the class moved to `org.springframework.boot.security.autoconfigure.actuate.web.servlet.EndpointRequest` (Boot 3:

`org.springframework.boot.actuate.autoconfigure.security.servlet`).

```
@Bean
SecurityFilterChain security(HttpSecurity http) throws Exception {
    http.authorizeHttpRequests(auth -> auth
            // probes, info and the Prometheus scrape stay open (normally reachable only inside the cluster)
            .requestMatchers(EndpointRequest.to("health", "info", "prometheus")).permitAll()
            // every other actuator endpoint needs the OPS role
            .requestMatchers(EndpointRequest.toAnyEndpoint()).hasRole("OPS")
            .anyRequest().permitAll())
        .httpBasic(Customizer.withDefaults())
        .csrf(csrf -> csrf.disable());   // demo API only; see chapter 10 for CSRF
    return http.build();
}
```

`EndpointRequest` matches by endpoint **ID**, so it survives base-path or port changes. `ActuatorEndpointsTests` proves both sides:

```
anonymous /actuator/env -> 401, ops /actuator/loggers/ch15.obs -> {"effectiveLevel":"INFO"}
```

Common mistakes

- `management.endpoints.web.exposure.include=*` on a public port without security: `env`, `heapdump` and `threaddump` leak secrets and internals. Expose the minimum; protect the rest.
- Thinking exposure is enough — an endpoint with access `none` stays unavailable even when exposed.
- Forgetting that `health` details (`show-details`) can reveal hostnames, disk paths and versions (a disk path appears below).

15.3 Health, health groups and Kubernetes probes

What it is

The `health` endpoint aggregates **health indicators** — small checks like "can I reach the database?" or "is there disk space?" — into one status: `UP`, `DOWN`, `OUT_OF_SERVICE` or `UNKNOWN`, overall the "worst" one (default order `DOWN`, `OUT_OF_SERVICE`, `UP`, `UNKNOWN`). `DOWN` and `OUT_OF_SERVICE` return HTTP **503**, others 200.

Auto-configured indicators appear with their technology: `ping`, `diskSpace`, `db` (a `DataSource`), `redis`, `mongo`, `rabbit`, `mail`, `ssl` (certificate validity), `livenessState`, `readinessState`, and more.

Why it matters

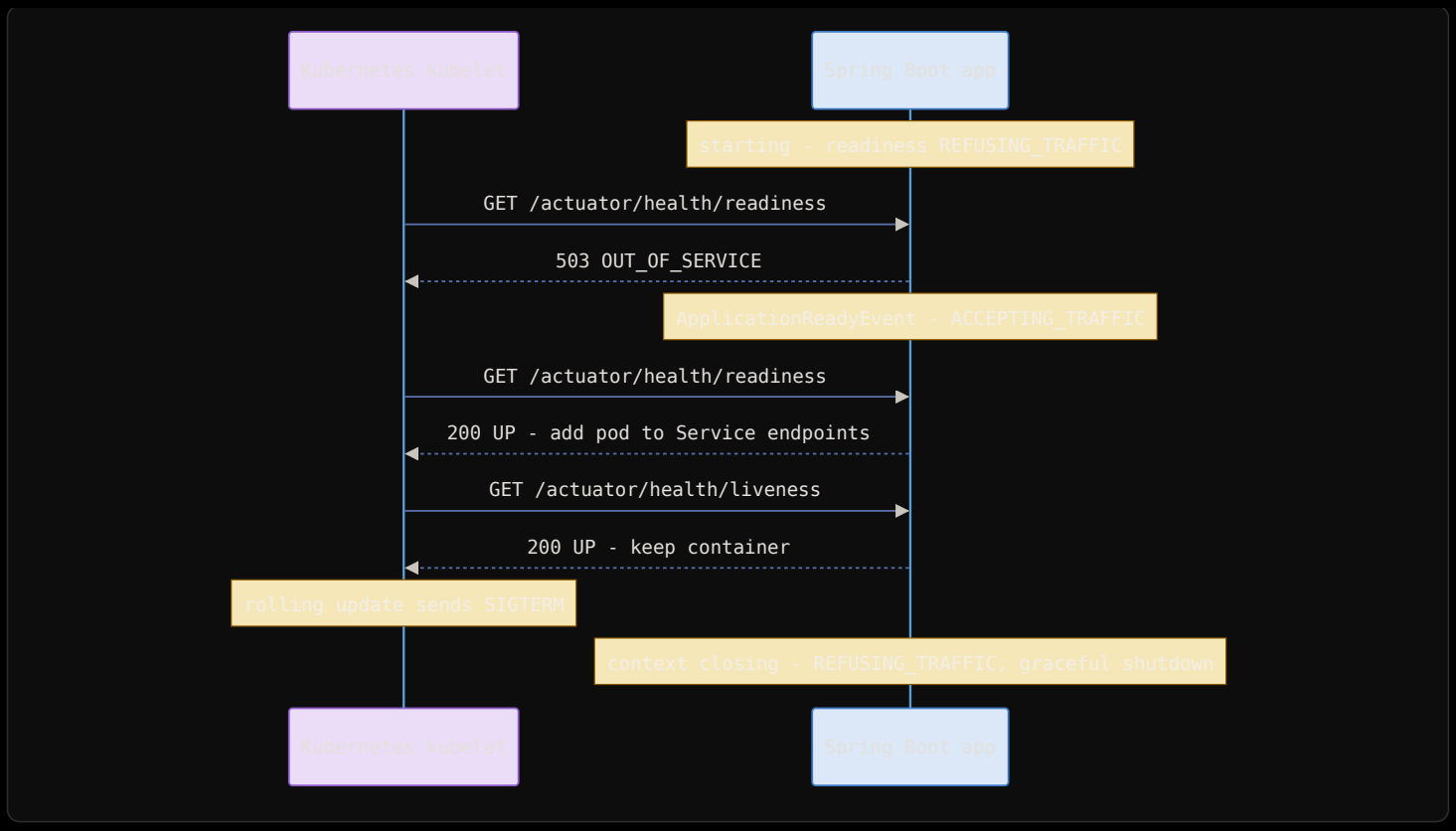
Load balancers and **orchestrators** (Kubernetes) act on these answers: take an instance out of rotation, restart a container, stop a **rolling deployment** (replacing instances a few at a time) — see System Design Fundamentals. A wrong design can **cause** outages: if every instance reports DOWN because one optional dependency is slow, the platform removes all of them.

Liveness vs readiness

Kubernetes asks two different questions:

Probe	Question	On failure Kubernetes...	Should include
Liveness	"Is this process broken beyond repair?"	**restarts** the container	only internal state (deadlock, corrupted state) — never external systems
Readiness	"Can this instance serve traffic right now?"	**stops routing** traffic to it (no restart)	warming up, shutting down, and — carefully — dependencies it cannot work without
Startup (K8s only)	"Has it finished starting?"	holds off the other probes	usually point it at the liveness endpoint with a larger threshold

Boot tracks two **availability states**: `LivenessState` (`CORRECT` / `BROKEN`) and `ReadinessState` (`ACCEPTING_TRAFFIC` / `REFUSING_TRAFFIC`). It marks the app ready only after startup (including `CommandLineRunner`s) finished, and switches readiness to `REFUSING_TRAFFIC` when the context starts closing. **In Boot 4 the probe groups are on by default** — `/actuator/health/liveness` and `/actuator/health/readiness` exist without configuration (Boot 2.3–3.x: only on Kubernetes, or with `management.endpoint.health.probes.enabled=true`).



A custom `HealthIndicator`

In Boot 4 the health API has its own module: `org.springframework.boot.health.contributor.HealthIndicator` and `Health` (Boot 3: `org.springframework.boot.actuate.health`). The component name is the bean name minus the `HealthIndicator` suffix.

```
// Bean name "inventoryHealthIndicator" -> health component name "inventory"
@Component
public class InventoryHealthIndicator implements HealthIndicator {

    private final InventoryStatus status;   // pretend connection state, flipped by the tests

    public InventoryHealthIndicator(InventoryStatus status) {
        this.status = status;
    }

    @Override
    public Health health() {
        if (status.reachable()) {
            return Health.up().withDetail("latencyMs", 12).build();
        }
        return Health.down().withDetail("error", "inventory service not reachable").build();
    }
}
```

Keep `health()` **fast and cheap** — it runs on every probe (typically every 5–10 s per instance); cache slow checks or compute them in the background. Reactive apps implement `ReactiveHealthIndicator`.

Health groups

A **group** is a named subset of indicators with its own settings, at `/actuator/health/{group}`; liveness and readiness are two built-in groups.

```
management.endpoint.health.show-details=when-authorized     # never | when-authorized | always
management.endpoint.health.roles=OPS
# a custom group, e.g. for a deployment smoke check
management.endpoint.health.group.deploy.include=ping,inventory
management.endpoint.health.group.deploy.show-details=always
# readiness also fails when inventory is down (a deliberate choice - see Trade-offs)
management.endpoint.health.group.readiness.include=readinessState,inventory
```

Other group options: `exclude`, `status.order`, `status.http-mapping.*`, `additional-path` (serve the group on the main port, e.g. `server:/healthz`). `management.endpoint.health.probes.add-additional-paths=true` also serves `/livez` and `/readyz` on the main port.

Proof

The tests call the endpoints anonymously and as the `ops` user, then make "inventory" unreachable:

```
anonymous /actuator/health -> {"groups":["deploy","liveness","readiness"],"status":"UP"}
ops       /actuator/health -> {"components":{"diskSpace":{"details":
{"total":494384795648,"free":63519252480,"threshold":10485760,"path":".../observability-demo/.","exists":true},"status":"UP"},"inventory":
{"details":{"latencyMs":12},"status":"UP"},"livenessState":{"status":"UP"},"ping":{"status":"UP"},"readinessState":{"status":"UP"},"ssl":
{"details":{"expiringChains":[],"invalidChains":[],"validChains":[]},"status":"UP"}},"groups":
["deploy","liveness","readiness"],"status":"UP"}
inventory reachable:
  liveness  -> {"status":"UP"}
  readiness -> {"status":"UP"}
inventory DOWN:
  liveness  -> {"status":"UP"}
  readiness -> {"status":"DOWN"} (HTTP 503)
  deploy    -> {"components":{"inventory":{"details":{"error":"inventory service not reachable"},"status":"DOWN"},"ping":
{"status":"UP"}},"status":"DOWN"} (HTTP 503)
```

- Anonymous callers get only status and group names — details need `OPS` (`when-authorized`).
- The `liveness` group exists unconfigured: probes are on by default in Boot 4.
- Liveness stays UP while a dependency is down — restarting our container wouldn't fix the inventory service.

The app can change its own state, e.g. to drain traffic before maintenance:

```
AvailabilityChangeEvent.publish(context, ReadinessState.REFUSING_TRAFFIC);
```

```
after REFUSING_TRAFFIC: readiness -> {"status":"OUT_OF_SERVICE"} (HTTP 503)
```

Kubernetes configuration (not run here):

```yaml
livenessProbe:
  httpGet: { path: /actuator/health/liveness, port: 8081 }
  periodSeconds: 10
  failureThreshold: 3
readinessProbe:
  httpGet: { path: /actuator/health/readiness, port: 8081 }
  periodSeconds: 5
  failureThreshold: 3
startupProbe:
  httpGet: { path: /actuator/health/liveness, port: 8081 }
  periodSeconds: 5
  failureThreshold: 30        # up to 150 s to start before liveness takes over
```

Trade-offs

A dependency in **readiness** protects users from an instance that can't serve — but if *all* instances share it, all become unready at once and the service vanishes, even for requests not needing it. Include a dependency only if the instance is truly useless without it; otherwise stay ready and **degrade** (fallbacks, cached data, circuit breakers — Chapter 16).

Common mistakes

- Database check in **liveness** → a DB blip restarts every pod, slowing recovery (a "restart storm") — a restart doesn't fix the database.
- Slow indicators (a synchronous remote call per probe) → probe timeouts → false failures.
- Same thresholds for liveness and readiness → a slow start gets killed; use a startup probe.
- `show-details=always` on a public port → leaks infrastructure details.

In the interview

Q: What's the difference between liveness and readiness, and how does Spring Boot support them? 30-second answer: Liveness asks "should the container be restarted?", readiness "should it receive traffic?". Boot tracks `LivenessState` and `ReadinessState`, exposes them as health groups at `/actuator/health/liveness` and `/readiness` (on by default in Boot 4), marks readiness only after startup completes and refuses traffic during shutdown. External dependencies never belong in liveness.

- *Custom check?* → A `HealthIndicator` bean returning `Health.up()/down().withDetail(...)`, included in a group if needed.
- *Take one instance out of rotation without killing it?* → Publish `AvailabilityChangeEvent` with `ReadinessState.REFUSING_TRAFFIC`.

TL;DR

`HealthIndicator` beans → `/actuator/health`; groups (`management.endpoint.health.group.<name>.include`) → `/actuator/health/<name>`; liveness/readiness on by default in Boot 4; DOWN → 503; liveness internal-only, care with dependencies in readiness.

15.4 The info endpoint

`/actuator/info` shows facts about the running build — "which version is actually deployed?" — assembled from **info contributors**:

Contributor	Content	Enabled by
`build`	group, artifact, version, build time	present when `META-INF/build-info.properties` exists (Maven plugin goal `build-info`)
`git`	branch, commit id and time	present when `git.properties` exists (`git-commit-id-maven-plugin`)
`env`	every `info.*` property	`management.info.env.enabled=true`
`java`	JVM and runtime vendor/version	`management.info.java.enabled=true`
`os`, `process`	OS name/version, PID (process id), CPUs, memory	`management.info.os.enabled`, `management.info.process.enabled`
custom	anything	an `InfoContributor` bean

```
<plugin>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-maven-plugin</artifactId>
  <executions>
    <execution><goals><goal>build-info</goal></goals></execution>
  </executions>
</plugin>
```

```
management.info.env.enabled=true
management.info.java.enabled=true
info.app.description=Orders demo for chapter 15
```

```
/actuator/info -> {"app":{"description":"Orders demo for chapter 15"},"build":{"artifact":"ch15-observability-demo","name":"ch15-observability-demo","time":"2026-09-28T23:50:25.503Z","version":"1.0.0","group":"handbook.spring"},"java":{"jvm":{"name":"OpenJDK 64-Bit Server VM","vendor":"Homebrew","version":"25.0.2"},"runtime":{"name":"OpenJDK Runtime Environment","version":"25.0.2"},"vendor":{"name":"Homebrew","vers...
```

Never put secrets in `info.*` — the endpoint is usually public.

15.5 Metrics with Micrometer

What it is

A **metric** is a number measured repeatedly over time. Micrometer records them in **meters** in a `MeterRegistry`. Each meter has a **name** and **tags** (key/value dimensions, e.g. `status=200`); every unique name + tag-values combination is its own **time series**.

Meter	Measures	Example	Prometheus output
`Counter`	a count that only increases	orders placed, errors	`name_total`
`Timer`	count + total time + max of short events	request latency	`name_seconds_count`, `_sum`, `_max` (+ buckets if histogram on: counts per value range, e.g. "under 100 ms")
`Gauge`	a current value sampled when read	queue size, pool usage	`name`
`DistributionSummary`	distribution of non-time values	payload size	`name_count`, `_sum`, `_max`
`LongTaskTimer`	tasks *currently* running and for how long	a batch job in progress	`name_active_count`, `_duration_sum`

Like a car: odometer = **counter** (only increases; rate over time), fuel needle = **gauge** (current value, up and down), lap stopwatch = **timer**.

What Boot gives you for free

With Actuator on the classpath, Boot registers JVM memory/GC/threads/classes, CPU, uptime, Tomcat sessions, HikariCP pool, logback events, `http.server.requests` (every MVC/WebFlux request, tagged `method`, `uri` template, `status`, `outcome`, `exception`), `http.client.requests` (for `RestClient`/`WebClient` built from Boot's builders), cache, Kafka and more.

Custom meters

From `OrderService.java`:

```
public OrderService(MeterRegistry registry) {
    this.registry = registry;
    // A Timer records how many calls and how long they took
    this.placeTimer = Timer.builder("orders.place")
            .description("time to place an order")
            .register(registry);
    // A Gauge samples a current value on demand (here: the map size)
    registry.gauge("orders.stored", orders, Map::size);
}

@Observed(name = "order.lookup", contextualName = "order-lookup")   // see 15.6
public Optional<String> find(long id) {
    log.info("looking up order {}", id);
    Optional<String> item = Optional.ofNullable(orders.get(id));
    // A Counter only goes up; the tag splits it into one time series per result
    Counter.builder("orders.lookups")
            .tag("result", item.isPresent() ? "found" : "missing")
            .register(registry)            // returns the existing counter if already registered
            .increment();
    return item;
}

public long place(String item) {
    return placeTimer.record(() -> { /* ... store it ... */ });
}
```

Common tags for every meter come from properties:

```
management.metrics.tags.region=eu-west-1
# optional: publish histogram buckets so Prometheus can compute p95/p99 across instances
management.metrics.distribution.percentiles-histogram.http.server.requests=true
```

Reading metrics: `/actuator/metrics` and `/actuator/prometheus`

The test makes two successful lookups, one miss and one order, then reads both endpoints:

```
/actuator/metrics/orders.lookups -> {"availableTags":[{"tag":"result","values":["missing","found"]},{"tag":"region","values":["eu-west-
1"]}],"measurements":[{"statistic":"COUNT","value":3.0}],"name":"orders.lookups"}
/actuator/prometheus (selected lines):
  http_server_requests_seconds_count{error="none",exception="none",method="GET",outcome="CLIENT_ERROR",region="eu-west-
1",status="404",uri="/orders/{id}"} 1
  http_server_requests_seconds_count{error="none",exception="none",method="GET",outcome="SUCCESS",region="eu-west-
1",status="200",uri="/orders/{id}"} 2
  order_lookup_seconds_count{class="ch15.obs.OrderService",error="none",method="find",region="eu-west-1"} 3
  orders_lookups_total{region="eu-west-1",result="found"} 2.0
  orders_lookups_total{region="eu-west-1",result="missing"} 1.0
  orders_place_seconds_count{region="eu-west-1"} 1
  orders_stored{region="eu-west-1"} 3.0
```

- `/actuator/metrics/{name}` aggregates across tags (COUNT 3.0); drill down with `?tag=result:found`.
- **Naming** is translated per backend: Micrometer's dotted `orders.lookups` becomes Prometheus `orders_lookups_total` (counter suffix); timers become `_seconds_*`.
- The `uri` tag is the **template** `/orders/{id}`, not `/orders/1` — otherwise every id would be a new time series.
- Every meter carries the common `region` tag.
- `@Observed` produced the timer `order_lookup_seconds` without any `MeterRegistry` code.

Push vs pull. Prometheus **pulls** (scrapes) `/actuator/prometheus` every 15–60 s; other backends are **pushed** to, e.g. OTLP via `management.otlp.metrics.export.url=http://collector:4318/v1/metrics`. In `@SpringBootTest` Boot switches metric exporters off (only an in-memory registry) unless the class has `@AutoConfigureMetrics` — hence its use in the Prometheus test.

Cardinality — the one rule that matters

Cardinality = the number of distinct tag-value combinations, each a separate stored time series. Tags need a **small, bounded** value set (method, status, region, result). Never tag with user ids, order ids, emails, full URLs or timestamps — a million users means a million series, which can crash Prometheus or explode a SaaS monitoring bill. Put such values in **traces or logs**.

Common mistakes

- High-cardinality tags (above).
- `Timer.builder(...).register(...)` in a hot loop is fine (returns the cached meter), but *dynamic meter names* are not — use tags.
- Averages for latency — use percentiles (p95, p99: the time 95%/99% of requests stay under; System Design Fundamentals) from histograms; averages hide the slow tail users feel.
- Gauges on garbage-collectable objects: `registry.gauge(name, obj, fn)` holds a **weak** reference (doesn't stop the object being freed); if nothing else holds `obj`, the gauge reports `NaN`.

15.6 The Observation API and `@Observed`

What it is

An **Observation** (Micrometer 1.10+, Boot 3+) wraps a unit of work — "look up an order" — once, and **handlers** decide what to produce: a `Timer` (metrics handler), a span (tracing handler), a log line, anything. Spring's own instrumentation (MVC, WebFlux, `RestClient`, `WebClient`, JDBC via extensions, Spring Security, Kafka, `@Scheduled`...) is written this way, so one dependency change yields both metrics and traces.

```
Observation.createNotStarted("order.lookup", observationRegistry)
       .lowCardinalityKeyValue("channel", "web")     // -> metric tag AND span attribute
       .highCardinalityKeyValue("orderId", "42")     // -> span attribute only
       .observe(() -> service.find(42));
```

- **Low-cardinality key values** become metric tags *and* span attributes.
- **High-cardinality key values** go only to spans (safe for ids).

`@Observed(name = "order.lookup", contextualName = "order-lookup")` on a bean method does the same via an aspect (Chapter 4). It needs the AspectJ starter (`spring-boot-starter-aspectj` in Boot 4) and, since Boot 3.2, the property:

```
management.observations.annotations.enabled=true   # also activates @Timed, @Counted, @NewSpan
```

The metric is named after `name` (`order.lookup` → `order_lookup_seconds_*` above, tagged `class` and `method`); the span is named after `contextualName`:

```
spans for GET /orders/2 -> [http get /orders/{id}, order-lookup]
```

Tuning which observations happen

```
# switch off whole families by name prefix
management.observations.enable.spring.security=false
# common key values added to every observation
management.observations.key-values.region=eu-west-1
```

Spring Security creates several observations per request (filter chain before/after, authentication, authorization); in the demo's first run they were most of the exported spans, so disabling the `spring.security` prefix keeps traces readable. For finer control register an `ObservationPredicate` bean.

Context propagation. Observations live in thread-local state: for `@Async` on Boot's executor set `spring.task.execution.propagate-context=true` (or a `ContextPropagatingTaskDecorator` on your own executor); for Reactor `spring.reactor.context-propagation=auto`.

15.7 Distributed tracing

What it is

A **trace** follows one request across every service it touches: a tree of **spans**, each one timed operation (incoming request, outgoing call, DB query) with a **span id**, the tree's shared **trace id**, a **parent span id** (`0000000000000000` for the root), start/end time and attributes — like a parcel tracking number every depot scans, adding its own entry.

How it works

1. A request arrives without trace headers → the server observation starts a **root span** with a new 128-bit trace id.
2. The trace context is stored in the current thread and copied into the log MDC (`traceId`, `spanId`).
3. An outgoing call through an instrumented client (Boot's `RestClient.Builder`, `WebClient.Builder`, `RestTemplateBuilder`, Kafka templates...) creates a **client span** and injects the context into headers — by default the W3C **Trace Context** header `traceparent: 00-<traceId>-<spanId>-01`.
4. The downstream service reads the header and continues the same trace with a child span.
5. Finished spans go to a **span processor** → **exporter** (OTLP to a collector, Zipkin, ...). Boot batches them (`BatchSpanProcessor`).
6. The backend (Tempo, Jaeger, Zipkin) stitches spans by trace id into a timeline.

Sampling decides which traces are recorded: `management.tracing.sampling.probability` (default **0.1** = 10%). The decision is made at the root and travels in the header, so a trace is either complete or absent.

Setup in Boot 4

```
<!-- Micrometer Tracing + OpenTelemetry bridge + OTLP exporters -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-opentelemetry</artifactId>
</dependency>
<!-- tests: spring-boot-starter-opentelemetry-test (provides @AutoConfigureTracing) -->
```

```
management.tracing.sampling.probability=1.0     # 100% in dev/test; lower in high-traffic prod
# send spans to an OpenTelemetry Collector (not used in the demo - no collector running)
# management.opentelemetry.tracing.export.otlp.endpoint=http://otel-collector:4318/v1/traces
```

The Brave/Zipkin alternative is `spring-boot-starter-zipkin` with `management.tracing.export.zipkin.endpoint`. Boot 3 users: the dependency used to be `micrometer-tracing-bridge-otel` + `opentelemetry-exporter-otlp`; the one starter replaces both.

A "logging exporter" instead of a collector

To see spans without running Jaeger, the demo registers its own `SpanExporter` bean; Boot adds every `SpanExporter` bean to its batch processor:

```
@Component
public class PrintingSpanExporter implements SpanExporter {
    private final List<SpanData> finished = new CopyOnWriteArrayList<>();

    @Override
    public CompletableResultCode export(Collection<SpanData> spans) {
        finished.addAll(spans);                      // a real exporter would send them over OTLP
        return CompletableResultCode.ofSuccess();
    }
    // ... flush(), shutdown(), finished(), clear()
}
```

The controller calls a second endpoint over HTTP with Boot's `RestClient.Builder`, standing in for a downstream service:

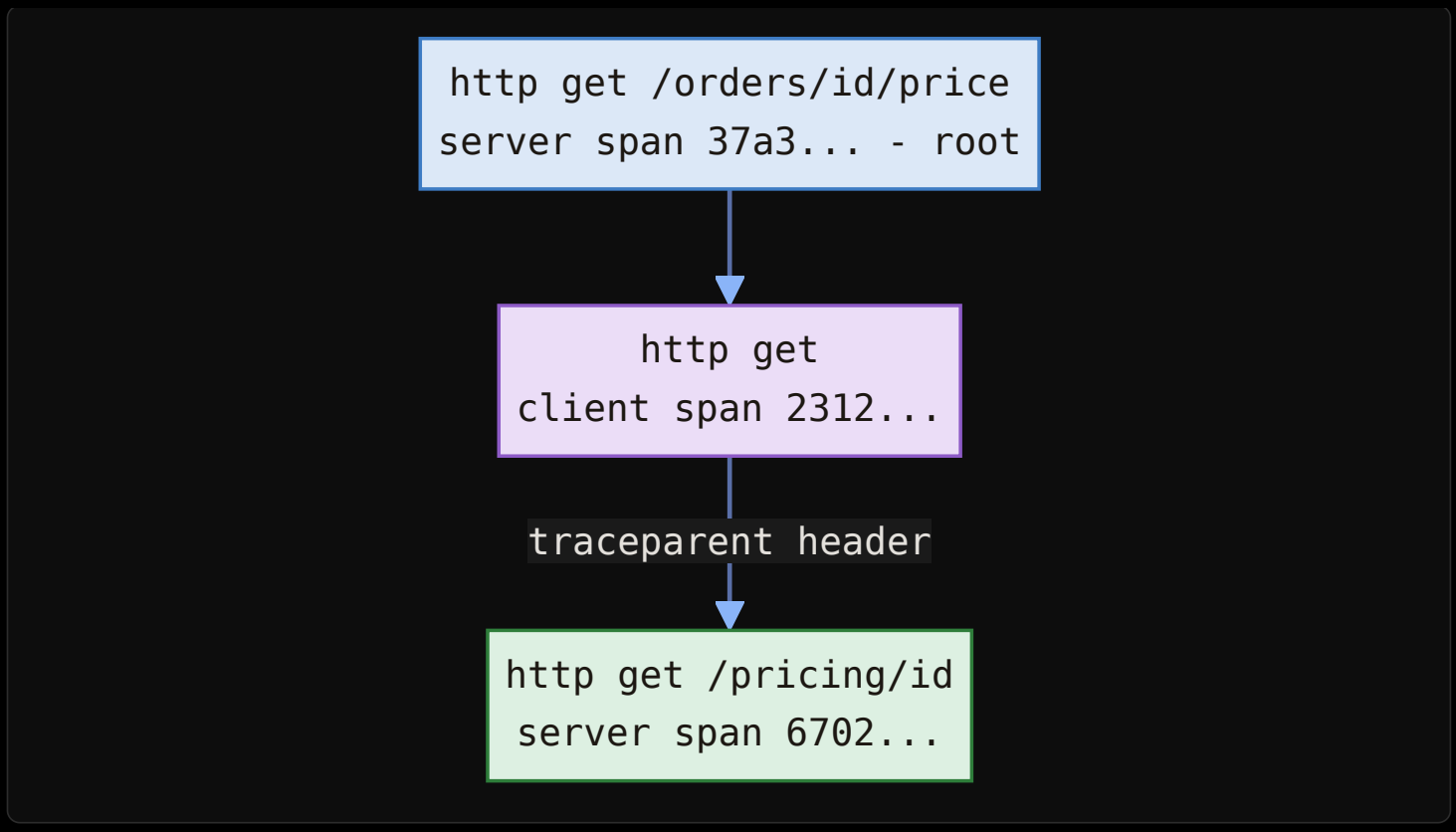
```
@GetMapping("/orders/{id}/price")
public String price(@PathVariable long id) {
    log.info("asking pricing service for order {}", id);
    String base = "http://localhost:" + env.getProperty("local.server.port");
    return restClientBuilder.build().get()
            .uri(base + "/pricing/{id}", id)
            .retrieve().body(String.class);
}

@GetMapping("/pricing/{id}")
public String pricing(@PathVariable long id) {
    log.info("pricing order {}", id);
    return "price for " + id + " = 49.99";
}
```

`TracingTests` (with `@AutoConfigureTracing`, because tests disable span export by default) calls `/orders/1/price`, flushes the batch processor and prints the log lines and spans:

```
response: price for 1 = 49.99
log lines (trace id - span id in brackets):
  [orders] [o-auto-1-exec-8] [89fb1d6934d0cf54e0910695a8df7859-37a3311dfdcc61c3] ch15.obs.OrderController                : asking pricing
service for order 1
  [orders] [o-auto-1-exec-9] [89fb1d6934d0cf54e0910695a8df7859-6702c6550d5a9bfd] ch15.obs.OrderController                : pricing order 1
finished spans:
  http get /orders/{id}/price  trace=89fb1d6934d0cf54e0910695a8df7859 span=37a3311dfdcc61c3 parent=0000000000000000
  http get                     trace=89fb1d6934d0cf54e0910695a8df7859 span=2312285359b3b845 parent=37a3311dfdcc61c3
  http get /pricing/{id}       trace=89fb1d6934d0cf54e0910695a8df7859 span=6702c6550d5a9bfd parent=2312285359b3b845
```

Read it as a tree:



- Two Tomcat threads (`exec-8`, `exec-9`) logged with the **same trace id** — propagation via `traceparent`.
- Each log line's span id matches the active span (`37a3...` outer request, `6702...` downstream).
- Boot's default console pattern prints `[application-name] [thread] [traceId-spanId]`; change it with `logging.pattern.correlation`.
- Across *real* services the picture is identical; only the exporter sends spans to a shared backend.

Baggage is extra key/value context travelling with the trace (e.g. a tenant id): `management.tracing.baggage.remote-fields=tenant` sends it in headers, `management.tracing.baggage.correlation.fields=tenant` also puts it in the MDC.

Trade-offs

Choice	Pro	Con
Sampling 100%	every request traceable	CPU, network and storage cost at scale
Head sampling (e.g. 10%, Boot default)	cheap, simple	the rare failing request is often not sampled
Tail sampling in the OTel Collector	keep all errors/slow traces, drop the rest	collector must buffer whole traces
OTel **Java agent** (a jar attached at JVM start that auto-instruments classes) instead of Micrometer	zero code, instruments many libraries	a second instrumentation layer; version coupling with the agent

Common mistakes

- `RestClient.builder()` / `new RestTemplate()` built yourself → no instrumentation; the trace breaks at that hop. Inject Boot's builder.
- Context lost on your own thread pools or `CompletableFuture` → wrap executors with context propagation.
- Trace ids as metric tags → cardinality explosion.
- Sampling 1.0 in busy production without a storage plan.

In the interview

Q: How does distributed tracing work in Spring Boot? 30-second answer: Micrometer Observation instruments incoming and outgoing calls; Micrometer Tracing with the OpenTelemetry (or Brave) bridge turns observations into spans. A trace id is created at the edge, propagated in the W3C `traceparent` header by instrumented clients, and put in the log MDC. Spans are batched and exported via OTLP to a collector and viewed in Tempo/Jaeger/Zipkin. Sampling defaults to 10%.

- *Trace broken between two services?* → Client not instrumented (created with `new`), a proxy strips headers, propagation formats differ (W3C vs B3, Zipkin's older format), or async code lost the context.
- *Logs vs traces?* → Traces show timing and causality across services; logs give detail within one; the shared trace id links them.

TL;DR

`spring-boot-starter-opentelemetry` + `management.tracing.sampling.probability`; inject Boot's client builders; trace/span ids appear in logs automatically; export via `management.opentelemetry.tracing.export.otlp.endpoint`.

15.8 Structured logging

What it is

Structured logging writes each event as a machine-readable record (JSON) instead of a formatted line, so log platforms (Elasticsearch/Kibana, Loki, Graylog, Datadog, Cloud Logging) filter on fields — `log.level:ERROR AND service.name:orders AND traceId:...` — instead of regex-parsing text. Built into Boot since **3.4**.

```
logging.structured.format.console=ecs        # ecs | logstash | gelf
logging.structured.format.file=logstash      # file output can use a different format
logging.structured.ecs.service.version=1.0.0
logging.structured.ecs.service.environment=test
```

Format	Standard	Typical destination
`ecs`	Elastic Common Schema	Elasticsearch / Elastic Agent, also fine for Loki
`logstash`	Logstash JSON	Logstash, Fluentd/Fluent Bit
`gelf`	Graylog Extended Log Format	Graylog

`StructuredLoggingTests` starts the app with `ecs` and captures the log line of one request:

```
ECS log line:
{"@timestamp":"2026-09-28T23:50:30.319867Z","log":{"level":"INFO","logger":"ch15.obs.OrderService"},"process":{"pid":17381,"thread":
{"name":"http-nio-auto-2-exec-1"}},"service":{"name":"orders","version":"1.0.0","environment":"test","node":{}},"message":"looking up order
1","traceId":"636cb6141b8e904e29a6f7824c3eb713","spanId":"8ceaaf80e43135b1","ecs":{"version":"8.11"}}
```

`service.name` comes from `spring.application.name`; MDC entries (here `traceId`, `spanId`) become fields automatically, as do key/values from SLF4J's fluent API (`log.atInfo().addKeyValue("orderId", id).log("placed")`).

Customise the JSON without code:

```
logging.structured.json.exclude=process.pid
logging.structured.json.rename.process.thread.name=thread
logging.structured.json.add.team=checkout
logging.structured.json.stacktrace.root=first          # root cause first
logging.structured.json.stacktrace.max-throwable-depth=20
```

For full control implement `StructuredLogFormatter<ILoggingEvent>` and name the class in `logging.structured.format.console`.

When not to use it: local development in a terminal (JSON is hard to read). Common setup: text by default, JSON in the `prod` profile (`application-prod.properties`), or JSON to file and text to console.

Common mistakes: logging secrets or personal data (JSON makes them *easier* to search — mask them); multi-line text stack traces that the shipper (the agent sending logs to the platform) splits into many events (JSON keeps one field); DEBUG on hot paths in production. Practical catch: logging is global to the JVM, so the structured-logging test runs last (`junit-platform.properties` orders the classes).

15.9 Graceful shutdown

What it is

When an instance stops (deployment, scale-down, node drain), **in-flight** requests should finish, not fail. **Graceful shutdown** stops accepting new connections, waits for active requests up to a timeout, then stops. **Default since Boot 3.4** for Tomcat, Jetty and Reactor Netty (`server.shutdown=graceful`); `server.shutdown=immediate` turns it off.

```
spring.lifecycle.timeout-per-shutdown-phase=10s   # default 30s
```

How it works

1. The JVM receives `SIGTERM` — the OS's polite "please stop", sent by Kubernetes and `docker stop` (Ctrl+C sends the similar `SIGINT`); unlike `SIGKILL` it runs shutdown hooks → Spring's hook closes the context.
2. Readiness switches to `REFUSING_TRAFFIC`.
3. The web server's lifecycle bean stops first: it stops accepting new connections at the network level and waits for active requests.
4. When they finish (or the timeout expires), the rest of the context closes: `@PreDestroy`, connection pools, executors.

Proof

`GracefulShutdownTests` starts the app, begins a `/slow?ms=1000` request, closes the context mid-request (as `SIGTERM` would), and tries a new request 200 ms later — once per mode:

```
ConfigurableApplicationContext ctx = SpringApplication.run(RuntimeApplication.class,
        "--server.port=0", "--server.shutdown=" + shutdownMode, "--spring.main.banner-mode=off");
// ... start a 1-second request, wait until it is inside the controller ...
Thread closer = Thread.ofPlatform().start(ctx::close);        // like SIGTERM from Kubernetes
Thread.sleep(200);
// ... try a new request, then wait for both ...
```

```
server.shutdown=immediate in-flight -> FAILED: IOException | new request during shutdown -> rejected (ConnectException) | close took < 0.7 s (did not wait)
server.shutdown=graceful  in-flight -> HTTP 200 'done after 1000 ms' | new request during shutdown -> rejected (ConnectException) | close took ~1 s (waited)
  log: Commencing graceful shutdown. Waiting for active requests to complete
  log: Graceful shutdown complete
```

In Kubernetes

A race: Kubernetes sends `SIGTERM` and removes the pod from the Service endpoints **at the same time**, and load balancers may route new requests for a few seconds. Since the server refuses new connections immediately (connection refused, not 503), delay shutdown briefly so endpoint removal propagates:

```
spec:
  terminationGracePeriodSeconds: 45          # must exceed preStop + Spring timeout
  containers:
    - name: orders
      lifecycle:
        preStop:
          sleep: { seconds: 10 }             # K8s 1.30+; older: exec ["sh","-c","sleep 10"]
```

Common mistakes

- `timeout-per-shutdown-phase` longer than `terminationGracePeriodSeconds` (default 30 s) → the pod is `SIGKILL`ed mid-drain.
- Expecting graceful shutdown to finish **message consumers** or `@Async` jobs — configure those separately (listener container stop, `spring.task.execution.shutdown.await-termination=true`).
- Running from an IDE that kills the process instead of sending SIGTERM (shutdown then looks immediate).

15.10 Virtual threads

Virtual threads (final in Java 21, Java book 15.4) are cheap JVM-managed threads; a blocked one releases its carrier OS thread, so blocking servlet apps hold many more concurrent requests without a huge pool. In Boot (3.2+):

```
spring.threads.virtual.enabled=true
```

Tomcat/Jetty then run each request on a virtual thread, and Boot's `@Async` executor, `@Scheduled` scheduler and several messaging listener containers switch too (14.5). `VirtualThreadsTests` calls an endpoint reporting `Thread.currentThread()`:

```
spring.threads.virtual.enabled=true  -> virtual thread: VirtualThread[#100,tomcat-handler-0]/runnable@ForkJoinPool-1-worker-1
default                              -> platform thread: Thread[#108,http-nio-auto-4-exec-1,5,main]
```

Caveats.

- The **bottleneck moves**: 10,000 concurrent requests now hit the database pool (10 connections by default) or a downstream API. Limit concurrency explicitly — `@ConcurrencyLimit` (4.7), pool sizes, bulkheads.
- Pinning: before JDK 24 blocking inside `synchronized` pinned the carrier; JDK 24 (JEP 491) fixed that; native calls can still pin.
- Virtual threads are daemon threads: an app with only `@Scheduled` work may exit; set `spring.main.keep-alive=true`.
- CPU-bound work gains nothing. Virtual threads vs reactive: Chapter 13.

15.11 Faster startup: extracted jars, CDS, AOT cache, native images

The problem it solves

A typical Boot app takes 1–5 s to start on the JVM (class loading, reflection, bean creation, JIT warm-up). That matters for autoscaling, serverless (scale-to-zero: no instance until a request arrives) and fast rollbacks. Options, least to most invasive:

Technique	What it does	Startup gain	Cost
Extracted jar	run from unpacked `lib/` instead of nested jars	small	a build step
CDS (Class Data Sharing; application classes since JDK 10, dynamic archives since JDK 13)	archive of parsed classes from a training run (a short run that records what loads)	~20–40%	training run, same JDK/classpath
JDK AOT cache (JEP 483 in JDK 24, simpler flags JEP 514 and method profiles JEP 515 in JDK 25)	caches classes *loaded and linked* (plus profiles) from a training run	often ~2×	training run; cache tied to the JDK build and exact jars
Spring AOT on the JVM	generates bean-definition code at build time (`spring-boot:process-aot`), run with `-Dspring.aot.enabled=true`	modest	profiles/conditions fixed at build time
GraalVM native image	compiles the whole app ahead of time to a native executable	~50–100× (tens of ms)	long builds, closed-world limits, reflection hints, lower peak throughput without PGO (profile-guided optimisation)

Measured: AOT cache on JDK 25

`aot-cache-demo.sh` builds `production-runtime`, extracts it and times start-until-context-refreshed (`-Dspring.context.exit=onRefresh` exits right after refresh — a Framework 6.1+ feature for training runs):

```
java -Djarmode=tools -jar target/app.jar extract --destination target/app   # app jar + lib/
cd target/app
# training run: record and write the cache in one step (JDK 25)
java -XX:AOTCacheOutput=app.aot -Dspring.context.exit=onRefresh -jar app.jar
# production run
java -XX:AOTCache=app.aot -jar app.jar
```

Run as `bash aot-cache-demo.sh <JDK-25-home>` (else `$JAVA_HOME`). Rows 2 and 3 are medians of 5 runs (Apple silicon laptop, Homebrew OpenJDK 25.0.2; **machine-dependent** — measure your own):

```
java: openjdk version "25.0.2" 2026-01-20
1. fat jar (java -jar)               : 1152 ms (mean of 3)
2. extracted jar                     : 934 ms
   AOT cache file                    :  56M
3. extracted jar + -XX:AOTCache      : 467 ms
```

The AOT cache more than halved JVM-to-context-ready time here, for a 56 MB file and a training step. CDS on JDK 13–24: `-XX:ArchiveClassesAtExit=app.jsa` (training), `-XX:SharedArchiveFile=app.jsa`. The cache is valid only for the same JDK build and jars (it needs a plain classpath — hence extracting the nested-jar fat jar); otherwise the JVM ignores it with a warning.

GraalVM native image (not run — `native-image` is not installed here)

```
# needs GraalVM / Liberica NIK 25; spring-boot-starter-parent provides the "native" profile
mvn -Pnative native:compile          # -> target/<artifactId>, a native executable
mvn -Pnative spring-boot:build-image # same, inside a container image (needs Docker)
```

Plus `org.graalvm.buildtools:native-maven-plugin` in the POM. The build runs Spring AOT, then GraalVM analyses everything reachable from `main` (**closed world**): classes unseen at build time can't load later; reflection, resources, proxies and serialization need **hints** (Spring generates most; add yours with `RuntimeHintsRegistrar` or `@RegisterReflectionForBinding`). Profiles and `@ConditionalOnProperty` are evaluated at build time. Test the native build in CI (`mvn -PnativeTest test`) — some libraries fail only there.

Choose native for CLIs, serverless functions, scale-to-zero and memory-tight deployments. **Stay on the JVM** (with the AOT cache, which keeps the JIT) for long-running, throughput-heavy services where peak performance and easy debugging win.

Container images (not run — Docker is not running here)

Cloud Native **Buildpacks** turn the jar into an **OCI image** (the standard container-image format) with no Dockerfile:

```
mvn spring-boot:build-image -Dspring-boot.build-image.imageName=acme/orders:1.0
# JVM options for the buildpack, e.g. enable the AOT cache or CDS training at build time:
#   <env><BP_JVM_AOTCACHE_ENABLED>true</BP_JVM_AOTCACHE_ENABLED></env>   (or BP_JVM_CDS_ENABLED)
```

Or a layered Dockerfile (each instruction a cached layer) — dependencies change rarely, so they get their own cached layer:

```
FROM eclipse-temurin:25-jre AS build
WORKDIR /build
COPY target/orders.jar app.jar
RUN java -Djarmode=tools -jar app.jar extract --layers --destination out

FROM eclipse-temurin:25-jre
WORKDIR /app
COPY --from=build /build/out/dependencies/ ./
COPY --from=build /build/out/spring-boot-loader/ ./
COPY --from=build /build/out/snapshot-dependencies/ ./
COPY --from=build /build/out/application/ ./
RUN java -XX:AOTCacheOutput=app.aot -Dspring.context.exit=onRefresh -jar app.jar
USER 1000
ENTRYPOINT ["java", "-XX:AOTCache=app.aot", "-XX:MaxRAMPercentage=75", "-jar", "app.jar"]
```

`-XX:MaxRAMPercentage` sizes the heap from the container's memory limit; running as a non-root user is basic hardening.

15.12 Production configuration checklist

Area	Setting / practice
Actuator exposure	expose only what you use (`health,info,prometheus`); protect the rest with `EndpointRequest`; consider `management.server.port`
Health	probes (on by default in Boot 4) wired to Kubernetes; custom indicators fast; no dependencies in liveness
Metrics	Prometheus or OTLP registry; common tags (`application`, `region`); histograms for latency SLOs (targets like "p99 under 300 ms"); bounded tag values
Tracing	`spring-boot-starter-opentelemetry`; sampling rate chosen consciously; OTLP endpoint to a collector
Logging	structured JSON in prod profile; levels INFO/WARN; no secrets or personal data; change levels at runtime via `/actuator/loggers`
Shutdown	graceful (default) + `timeout-per-shutdown-phase` < K8s grace period; `preStop` delay
Threads	`spring.threads.virtual.enabled=true` for blocking I/O apps, with explicit concurrency limits
Startup	extracted jar + AOT cache (JDK 25) or native image where startup matters
Config & secrets	externalised config per environment (Chapter 3); secrets from a vault or K8s secrets, never in the jar
Resources	container memory limit + `MaxRAMPercentage`; connection pool sized to the database
Errors	`spring.web.error.include-stacktrace=never` (default; Boot 4 name — the old `server.error.*` keys are no longer bound since 4.0.0) and Problem Details (Chapter 5)

15.13 Common mistakes (whole chapter)

Symptom	Cause	Fix
`/actuator/prometheus` returns 404	registry dependency missing or endpoint not exposed	add `micrometer-registry-prometheus`, include `prometheus` in exposure
Prometheus endpoint missing only in tests	Boot disables metric export in tests	`@AutoConfigureMetrics`
No trace ids in logs	no tracer on the classpath, or a custom log pattern without `%X{traceId}`	add `spring-boot-starter-opentelemetry`; keep Boot's correlation pattern
Trace breaks between services	client created with `new` / `builder()`	inject Boot's `RestClient.Builder` / `WebClient.Builder`
`@Observed` does nothing	property or AspectJ starter missing, or self-invocation	`management.observations.annotations.enabled=true`, `spring-boot-starter-aspectj`, call through the proxy
Prometheus memory explodes	high-cardinality tags	remove ids from tags
Pods restart during DB outage	DB in liveness	keep liveness internal
502s (Bad Gateway) during deployments	no `preStop` delay, grace timeout too short	`preStop` sleep, align timeouts
Secrets visible in `/actuator/env`	`show-values=always`	leave the default `never`, secure the endpoint

15.14 In the interview

Q: What is Spring Boot Actuator and how do you secure it? Management endpoints (health, info, metrics, prometheus, loggers, env, ...). Only `health` is exposed by default; expose more with `management.endpoints.web.exposure.include`, restrict with `management.endpoint.<id>.access`, and secure with a `SecurityFilterChain` using `EndpointRequest.to(...)` / `toAnyEndpoint()`, often on a separate management port.

- *Change a log level in production without redeploying?* → `POST /actuator/loggers/<logger>` with `{"configuredLevel":"DEBUG"}` (protected endpoint).

Q: Counter, Gauge, Timer — when to use which? → Counter for accumulating events (rate via PromQL `rate()`), Gauge for current values (queue depth), Timer for durations (count/sum/max/histogram → latency percentiles). Keep tags low-cardinality.

Q: What does the Observation API add over plain Micrometer metrics? → Instrument once, get metrics *and* traces (and custom handlers); Spring's own instrumentation uses it; `@Observed` for methods.

Q: How does a trace id get from service A to B? → The instrumented HTTP client injects `traceparent`; B's server observation extracts it and creates a child span; both log the same trace id via MDC.

Q: Zero-downtime deployments with Spring Boot on Kubernetes? → Readiness probe (ready after startup), graceful shutdown (default), `preStop` delay, grace period longer than the drain timeout, rolling update with `maxUnavailable: 0`. See blue-green and canary.

Q: How would you speed up startup? → Lazy init (beans created on first use) where safe, fewer auto-configurations, extracted jar + CDS/AOT cache (JDK 25: `-XX:AOTCacheOutput` training, `-XX:AOTCache`), Spring AOT, or GraalVM native image — trading build complexity and closed-world limits for startup.

Q: Virtual threads in Spring Boot? → `spring.threads.virtual.enabled=true` (Java 21+, Boot 3.2+); great for blocking I/O; move concurrency limits to explicit places (pools, `@ConcurrencyLimit`).

Chapter TL;DR

- Observability = **logs + metrics + traces**, linked by the trace id. Spring: Logback, **Micrometer**, **Micrometer Tracing + OpenTelemetry**, all fed by the **Observation API**.
- **Actuator**: only `health` exposed by default; control with `exposure.include` and `access`; secure with `EndpointRequest` (Boot 4 package `org.springframework.boot.security.autoconfigure.actuate.web.servlet`).
- **Health**: `HealthIndicator` (Boot 4 package `org.springframework.boot.health.contributor`), groups, **liveness/readiness on by default in Boot 4**; DOWN → 503; no external deps in liveness.
- **Metrics**: Counter/Timer/Gauge with **low-cardinality** tags; `/actuator/prometheus` for scraping; `@AutoConfigureMetrics` in tests.
- **Observation / `@Observed`**: one instrumentation → timer + span; needs `management.observations.annotations.enabled=true` + AspectJ.
- **Tracing**: `spring-boot-starter-opentelemetry`; W3C `traceparent` propagation via Boot's client builders; `[traceId-spanId]` in logs; sampling default 0.1; export over OTLP.
- **Structured logging** (Boot 3.4+): `logging.structured.format.console=ecs|logstash|gelf`; MDC fields included.
- **Graceful shutdown** is the default (Boot 3.4+); tune `spring.lifecycle.timeout-per-shutdown-phase`; add a `preStop` delay on Kubernetes.
- **Virtual threads**: `spring.threads.virtual.enabled=true`.
- **Startup**: extract the jar, train an **AOT cache** (JDK 25; 934 → 467 ms here), or build a **GraalVM native image**; package with buildpacks or a layered Dockerfile.
- Next: Chapter 16 — Spring Cloud and Microservices.

Chapter 16 — Spring Cloud and Microservices

A **microservice architecture** splits one application into small, independently deployed services talking over the network. It buys independent releases and scaling, and costs every problem of **distributed systems** (parts on different machines over an unreliable network; System Design Fundamentals): where is the other service, what if it is slow, what is the single entry point, where does configuration live, how do you follow one request through ten services. **Spring Cloud** is a family of projects giving Boot applications ready-made answers.

This chapter covers the *Spring implementation*. The patterns — gateways (one front door for all clients), circuit breakers (a switch that stops calls to a failing service), sagas (multi-service workflows of local steps plus undo steps), CQRS (separate write and read models) — are taught in the Microservices Patterns notes and System Design Patterns; each section links there.

The code: four Spring Boot 4.1.1 projects on Java 25 in `code/` — `config-demo`, `service-clients`, `gateway-demo`, `resilience-demo` (7 tests, all run). Downstream services (the ones a service calls) are tiny local HTTP servers on random ports, so nothing external is needed. Outputs are real `mvn -q verify` output (log noise trimmed as `...`).

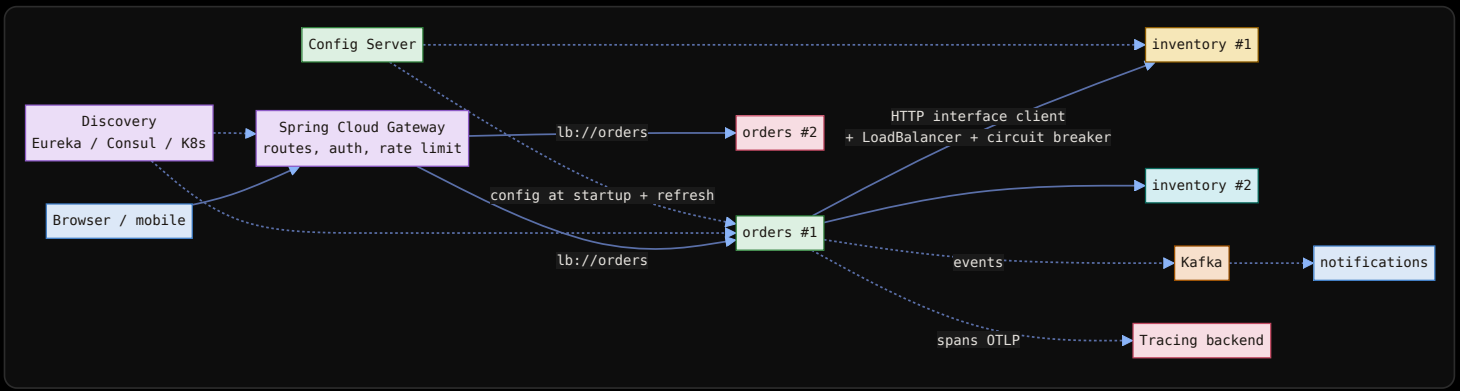
> **Prerequisites.** HTTP clients and interfaces: Chapter 6. Proxies and Framework 7 `@Retryable`: Chapter 4. `spring.config.import`: Chapter 3. Tracing and health probes: Chapter 15. Messaging: Chapter 14.

16.1 The map: microservice problems and Spring answers

Why it matters

Know which problem a dependency solves — and whether your platform already solves it. Many Kubernetes teams add Eureka and a Config Server out of habit, doubling the moving parts.

Problem	Pattern notes	Spring answer	On Kubernetes, often instead
One entry point for clients, cross-cutting edge concerns (auth, rate limits)	API Gateway, BFF	**Spring Cloud Gateway**	Ingress / Gateway API controller, service mesh (sidecar proxies handling routing, retries and encryption between services)
Find instances whose addresses change	Service Discovery	Eureka, Consul, Zookeeper, Spring Cloud Kubernetes	Kubernetes Services + DNS
Spread calls over instances	Load balancing	**Spring Cloud LoadBalancer** (client side)	kube-proxy (forwards a Service's traffic to its pods) / mesh (server side)
Configuration per environment, changed without rebuild	Config externalization	**Spring Cloud Config**, Vault, Consul KV	ConfigMaps / Secrets (Kubernetes config and secret objects; `spring.config.import=configtree:`)
Failing or slow dependencies	Circuit Breaker, Retry, Bulkhead	**Resilience4j** (+ Spring Cloud CircuitBreaker), Framework 7 `@Retryable` / `@ConcurrencyLimit`	mesh retries / outlier detection (ejecting repeatedly failing instances; coarser)
Following a request across services	Distributed Tracing	Micrometer Tracing + OpenTelemetry (15.7)	same, plus mesh telemetry
Declarative HTTP calls	—	HTTP interface clients (Framework), OpenFeign (maintenance)	—
Async communication, events	Event-Driven, Saga, CQRS	Spring Kafka / AMQP (Chapter 14), Spring Cloud Stream	—
Cross-cutting helpers next to each service	Sidecar	—	sidecar containers, service mesh



When not to use microservices

Distributed systems are harder to build, test, debug and operate. A small team, an unclear domain or no need for independent scaling are strong reasons to start with a **well-modularised monolith** (one deployable with clean internal module boundaries; Spring Modulith helps enforce them) and extract services later via Strangler Fig. See DDIA for the consistency problems you inherit.

16.2 Release trains and versions

What it is

Spring Cloud is ~15 projects (Commons, Config, Gateway, OpenFeign, Netflix, Consul, Kubernetes, Circuit Breaker, Stream, Function, Bus, Vault, Task, Zookeeper...), each versioned separately. A **release train** is a BOM (bill of materials) pinning versions that work together and with one Boot generation, named `YYYY.MINOR.PATCH` (since 2020) plus a codename.

Train	Codename	Spring Boot	Component versions
2025.1.x	**Oakwood**	**4.0.x, 4.1.x** (4.1 from 2025.1.2)	5.0.x
2025.0.x	Northfields	3.5.x	4.3.x — open-source support ended June 2026
2024.0.x	Moorgate	3.4.x	4.2.x

(Source: spring.io/projects/spring-cloud compatibility table; 2025.1.3 (August 2026) is the latest service release at the time of writing — based on Boot 4.0.x, compatible with 4.1.x. A `2026.0.0-M1` milestone exists on Maven Central.)

```
<properties>
  <spring-cloud.version>2025.1.3</spring-cloud.version>
</properties>
<dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>org.springframework.cloud</groupId>
      <artifactId>spring-cloud-dependencies</artifactId>
      <version>${spring-cloud.version}</version>
      <type>pom</type>
      <scope>import</scope>
    </dependency>
  </dependencies>
</dependencyManagement>
<!-- then Spring Cloud starters without <version> -->
```

Recent changes that trip people up:

- **Gateway artifacts were renamed** (2025.0): `spring-cloud-starter-gateway-server-webflux` (reactive) and `spring-cloud-starter-gateway-server-webmvc` (servlet); properties moved to `spring.cloud.gateway.server.webflux.*` / `...server.webmvc.*`.
- **Spring Cloud Sleuth is gone** (last in 2021.0 for Boot 2.x); tracing is Micrometer Tracing in Boot itself.
- **Netflix Hystrix, Ribbon, Zuul** were removed years ago; use Resilience4j, Spring Cloud LoadBalancer and Spring Cloud Gateway.
- **OpenFeign** is feature-complete (maintenance mode) — the Spring team recommends HTTP interface clients.
- **Spring Cloud Contract**: the 2025.1.0–2025.1.2 BOMs manage it (5.0.x), but the 2025.1.3 BOM no longer lists it, and its GitHub repository has been moved to the `spring-attic` organisation and archived (read-only; last commit July 2026). Existing users can pin `spring-cloud-contract-dependencies` 5.0.3 explicitly; for new projects consider consumer-driven contracts with Pact (16.10).
- Resilience4j is **not** managed by Spring Boot; the Spring Cloud CircuitBreaker BOM pins Resilience4j 2.3.0, while Resilience4j's own Boot 4 module is `resilience4j-spring-boot4` 2.4.0 (used standalone in 16.8).

Common mistakes

- Mixing trains and Boot generations (e.g. 2025.0 with Boot 4) → `NoSuchMethodError` / `ClassNotFoundException`. Spring Cloud checks compatibility at startup and fails fast with a clear message (`spring.cloud.compatibility-verifier.enabled`).
- Hard-coding component versions instead of importing the BOM.

16.3 Centralized configuration: Spring Cloud Config

What it is

A **Config Server** is a small Boot app serving configuration files over HTTP from a backend — usually a **Git repository**, also a local folder (`native` profile), Vault (HashiCorp's secret store), JDBC, S3 and others. Every service is a **config client** importing its properties at startup, layered by *application name*, *profile* and *label* (Git branch/tag) — like one shared company handbook each department reads (general chapter plus its own) and can re-read when updated.

Why it matters

30 services × 4 environments = 120 configuration sets. A Config Server gives one place to review, version (Git history = audit log) and change them, and to refresh running services without a rebuild. See Config externalization notes.

How it works

1. The server exposes `GET /{application}/{profile}[/{label}]` (plus file-style URLs like `/{application}-{profile}.properties`).
2. For `orders` + profile `prod` it returns an ordered list of property sources — most specific first: `orders-prod.properties` → `orders.properties` → `application-prod.properties` → `application.properties`.
3. The client adds `spring.config.import=configserver:http://config:8888` (Boot's config-data API; `bootstrap.yml` is legacy); `optional:configserver:...` if it may start without it.
4. Remote properties join the `Environment` and **override the service's own `application.properties`**. Whether local system properties or environment variables may override remote values is controlled repository-side (`spring.cloud.config.allow-override`, `override-none`, `override-system-properties`).
5. On refresh the client refetches, rebinds `@ConfigurationProperties` beans and re-creates `@RefreshScope` beans.



The demo

`ConfigServerAndClientTests` writes three files into a temp folder (the "Git repo"), starts a `native` Config Server on a random port, then an `orders` client with profile `prod`:

```
// server
@SpringBootApplication
@EnableConfigServer
public class ConfigServerApplication { ... }
//   --spring.profiles.active=native
//   --spring.cloud.config.server.native.search-locations=file:<repo>/

// client bean
@Component
@RefreshScope   // re-created with fresh values after a refresh
public class OrderSettings {
    public OrderSettings(@Value("${orders.discount-percent}") int discountPercent,
                         @Value("${orders.greeting}") String greeting,
                         @Value("${shared.timeout}") String timeout) { ... }
}
//   --spring.application.name=orders --spring.profiles.active=prod
//   --spring.config.import=configserver:http://localhost:<port>
```

File in the repo	Content
`application.properties` (all services)	`shared.timeout=5s`, `orders.greeting=hello from application.properties`
`orders.properties`	`orders.greeting=hello from orders.properties`, `orders.discount-percent=10`
`orders-prod.properties`	`orders.discount-percent=15`

Then it edits `orders-prod.properties` to 20 and calls `ContextRefresher.refresh()` (what `POST /actuator/refresh` does):

```
GET /orders/prod -> property sources, highest priority first:
   orders-prod.properties
   orders.properties
   application.properties
client sees:    discount=15%, greeting='hello from orders.properties', timeout=5s
refresh changed [orders.discount-percent]
after refresh:  discount=20%, greeting='hello from orders.properties', timeout=5s
```

- `discount=15%` from the profile file, `greeting` from the service file (overriding the shared one), `timeout` from the shared file; `refresh()` returns exactly the changed keys.

Demo-only detail: server and client share one classpath, so the client passes `spring.cloud.config.enabled=true` — the server jar switches the client off by default (`ConfigServerBootstrapApplicationListener`) so a Config Server doesn't fetch from itself. Real deployments are separate apps.

Refreshing many instances, secrets and alternatives

- **Spring Cloud Bus** (`spring-cloud-starter-bus-kafka` / `-amqp`) broadcasts one refresh to all instances (`POST /actuator/busrefresh`); `spring-cloud-config-monitor` can trigger it from a Git webhook (HTTP call from the Git host on push).
- `{cipher}AQB...` values are decrypted by the server (`encrypt.key` or a keystore); better keep secrets in **Vault** (`spring-cloud-starter-vault-config`) or Kubernetes Secrets.
- On Kubernetes, ConfigMaps/Secrets mounted as files and imported with `spring.config.import=configtree:/etc/config/` need no extra server (3.8); Spring Cloud Kubernetes can also read ConfigMaps via the API and reload.

Trade-offs and when not to use it

Pro	Con
One reviewed, versioned source; environment diffs visible	Another critical service: if down, new instances can't start (use `optional:`, `fail-fast` + retry, ≥2 replicas)
Runtime refresh without redeploy	Runtime refresh is also runtime *risk* — a bad value hits all instances at once
Works on any platform	On Kubernetes, ConfigMaps may be enough

Common mistakes

- Expecting every bean to see refreshed values — only `@ConfigurationProperties` (rebound) and `@RefreshScope` (re-created) beans do; a `@Value` field in a normal singleton keeps the old value.
- Plain-text secrets in the Git repo.
- Forgetting `spring.application.name` — the client then asks for `application`, not `orders`.

16.4 Service discovery: Eureka, Consul, Kubernetes

What it is

Instances come and go and their IP addresses change. **Service discovery** keeps a **registry** of live instances per service name; clients ask "where is `inventory`?" instead of hard-coding addresses (Service Discovery notes: client- vs server-side).

Spring Cloud Commons defines `DiscoveryClient` (`getInstances("inventory")` → `ServiceInstance`s with host, port, secure flag, metadata); each backend implements it:

Backend	Starter	How instances register	Health	Notes (not run here)
Netflix Eureka	`spring-cloud-starter-netflix-eureka-server` / `-client`	client registers itself and sends heartbeats (every 30 s by default)	heartbeat lease expiry	AP design (CAP: stays **A**vailable during a network **P**artition rather than **C**onsistent): may serve stale entries (self-preservation mode)
HashiCorp Consul	`spring-cloud-starter-consul-discovery`	client registers via the local Consul agent	Consul calls the app's health URL	also KV store for config (`spring-cloud-starter-consul-config`)
Kubernetes	`spring-cloud-starter-kubernetes-client-all` (or `-fabric8`)	Kubernetes itself (Endpoints of a Service)	readiness probes	needs RBAC (role-based access control) permission to read endpoints
Simple (in Commons)	none	static list in properties	none	tests, local dev, fixed topologies — used in 16.5
Zookeeper	`spring-cloud-starter-zookeeper-discovery`	ephemeral nodes (vanish when the owner disconnects)	session	CP design (consistent; may refuse requests during a partition)

Eureka in a nutshell (not run):

```
@SpringBootApplication
@EnableEurekaServer              // the registry: a dashboard at http://localhost:8761
public class RegistryApplication { ... }
```

```
# each service (client) - with spring-cloud-starter-netflix-eureka-client on the classpath
spring.application.name: inventory                  # the service id
eureka:
  client:
    service-url:
      defaultZone: http://registry:8761/eureka/
  instance:
    prefer-ip-address: true
```

On Kubernetes, do you need it?

Usually **no**: a `Service` named `inventory` gives a stable DNS name (`http://inventory:8080`), kube-proxy spreads connections over ready pods, and readiness probes cover health (server-side discovery and load balancing). Add Spring Cloud Kubernetes only for client-side features: instance metadata, per-request balancing over pods, discovery across namespaces.

16.5 Client-side load balancing with Spring Cloud LoadBalancer

What it is

Client-side load balancing: the *caller* picks an instance from the list discovery provides — no load-balancer box in between (System Design Fundamentals). Spring Cloud LoadBalancer (Netflix Ribbon's replacement) resolves a service id in a URL (`http://inventory/...` or `lb://inventory`) to a concrete instance per request.

How it works

1. A client marked for load balancing (`@LoadBalanced RestClient.Builder`, a Feign client with only a `name`, an HTTP service group without a `base-url`, the gateway's `lb://` URIs) makes a request to `http://inventory/stock/1`.
2. An interceptor asks the `ReactorLoadBalancer` for service `inventory`.
3. A `ServiceInstanceListSupplier` gets instances from the `DiscoveryClient` (cached), optionally filtered (health-check, zone, hints).
4. The algorithm picks one — **round robin** by default (instances in turn: A, B, A, B; randomised start), or `RandomLoadBalancer`, weighted, "same instance preference", subset.
5. The URL is rewritten to `http://host:port/stock/1` and sent.

The demo: static instances, two clients

`service-clients` uses the **simple** discovery client (instances listed in properties). The test starts two tiny HTTP servers, "A" and "B", and points the properties at them:

```
spring.application.name=orders
# "Simple" discovery: a fixed list of instances per service id (instead of Eureka/Consul/Kubernetes).
spring.cloud.discovery.client.simple.instances.inventory[0].uri=http://localhost:9001
spring.cloud.discovery.client.simple.instances.inventory[1].uri=http://localhost:9002
# No spring.http.serviceclient.inventory.base-url: Spring Cloud LoadBalancer then uses
# the group name "inventory" as the service id and picks an instance per request.
```

Output (both clients shown in 16.6):

```
HTTP interface 4 calls served by [A, B, A, B]
OpenFeign      4 calls served by [A, B, A, B]  5th call -> Stock[sku=SKU-9, quantity=7, servedBy=A]
```

Round robin alternates; the start is random, so another run may print `[B, A, B, A]` (the test asserts alternation, not order). The startup log warns:

```
WARN ... LoadBalancerCaffeineWarnLogger : Spring Cloud LoadBalancer is currently working with the default cache. While this cache
implementation is useful for development and tests, it's recommended to use Caffeine cache in pr...
```

In production add `com.github.ben-manes.caffeine:caffeine` (a fast in-memory cache) so the instance-list cache is bounded and refreshed properly (`spring.cloud.loadbalancer.cache.ttl` — time to live, default 35 s).

Useful settings:

```
# instance-list supplier chain: default | health-check | zone-preference | weighted | same-instance-preference | request-based-sticky-session
| subset
spring.cloud.loadbalancer.configurations=health-check
# per-service overrides live under spring.cloud.loadbalancer.clients.<serviceId>.*
```

For a plain `RestClient`, mark the builder:

```java
@Bean @LoadBalanced
RestClient.Builder loadBalancedBuilder() { return RestClient.builder(); }
// restClientBuilder.baseUrl("http://inventory").build()
```

Trade-offs

Client-side (Spring Cloud LoadBalancer)	Server-side (K8s Service, NGINX, cloud LB, mesh)
per-request choice, custom algorithms, zone/metadata aware	nothing in the app; works for every language
each client needs the library and a discovery source	extra network hop (except kube-proxy/mesh sidecars), fewer app-level decisions

---

16.6 Declarative clients: OpenFeign vs HTTP interface clients

What it is

Both turn a Java interface into a generated proxy making the HTTP calls (6.6 covers HTTP interfaces in depth).

```java
// Spring Cloud OpenFeign: name = service id -> resolved by the LoadBalancer (no url given)
@FeignClient(name = "inventory")
public interface InventoryFeignClient {
    @GetMapping("/stock/{sku}")                       // Spring MVC annotations
    Stock stock(@PathVariable("sku") String sku);
}

// Spring Framework HTTP service interface (no extra library)
public interface InventoryHttpClient {
    @GetExchange("/stock/{sku}")
    Stock stock(@PathVariable String sku);
}

@SpringBootApplication
@EnableFeignClients                                                  // scan for @FeignClient
@ImportHttpServices(group = "inventory", types = InventoryHttpClient.class) // group name = service id
public class ClientsApplication { ... }
```

Since Spring Cloud 2025.1 (Commons 5.0), LoadBalancer plugs into Boot 4's **HTTP service client groups**: a group without `spring.http.serviceclient.<group>.base-url` gets a load-balanced `http://<groupName>`, and an `lb://` base URL also works — so `inventory` needed no URL. `LoadBalancedClientsTests` calls each client four times (output in 16.5).

Comparison

	OpenFeign	HTTP interface client
Where it lives	Spring Cloud OpenFeign (separate project)	Spring Framework 6+ core; Boot 4 groups via `@ImportHttpServices`
Status	**feature-complete / maintenance** — only fixes	actively developed; the Spring team's recommendation
Annotations	Spring MVC's `@GetMapping`, ... on the interface	`@GetExchange`, `@PostExchange`, ...
Underlying client	Feign's own client (or Apache HC5 / OkHttp)	`RestClient` or `WebClient` (reactive return types possible)
Config	`spring.cloud.openfeign.client.config.<name>.*` (timeouts, loggers, interceptors)	`spring.http.serviceclient.<group>.*`, `RestClientHttpServiceGroupConfigurer`
Load balancing	automatic when only `name` is set	automatic for groups without base URL (Spring Cloud 2025.1+)
Circuit breaker	`spring.cloud.openfeign.circuitbreaker.enabled=true` + fallbacks	Spring Cloud 2025.1+: per-group breaker with an `@HttpServiceFallback` class (switch `spring.cloud.circuitbreaker.http-services.enabled`, on by default; not run here), or wrap the call in a service method with Resilience4j
Tracing	instrumented	instrumented (via the observed `RestClient`)

Recommendation: new code → HTTP interface clients; existing Feign code keeps working — migrate when convenient (annotations map almost one-to-one).

16.7 API gateway: Spring Cloud Gateway

What it is

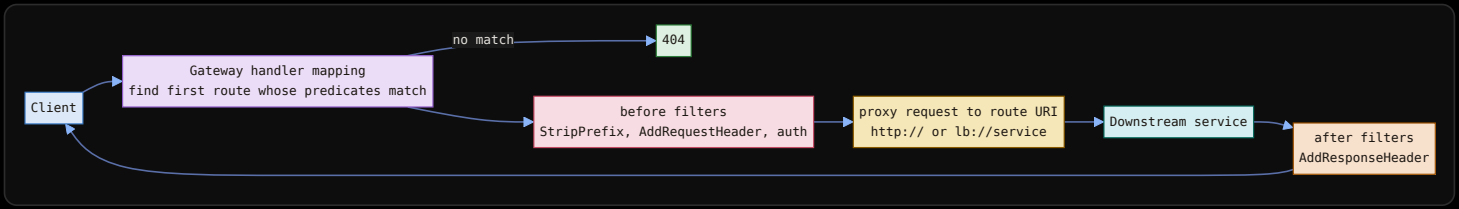
An **API gateway** is the single front door for external clients: it **routes** each request to the right service and applies **edge concerns** once — authentication/token relay, rate limiting, CORS, header rewriting, retries/circuit breaking, request size limits. Patterns: API Gateway notes, BFF, API & Edge module.

Two flavours, same concepts:

	Server Web MVC	**Server WebFlux**
Starter	`spring-cloud-starter-gateway-server-webmvc`	`spring-cloud-starter-gateway-server-webflux`
Runtime	Servlet (Tomcat/Jetty), blocking; pairs well with virtual threads	Netty, reactive
Routes in code	`RouterFunction` DSL (`GatewayRouterFunctions.route`)	`RouteLocatorBuilder` DSL
Properties	`spring.cloud.gateway.server.webmvc.routes`	`spring.cloud.gateway.server.webflux.routes`
Choose when	team knows MVC, uses blocking libraries	very high connection counts, streaming, already reactive

How it works

A **route** = id + destination URI + **predicates** (when does it match? `Path`, `Method`, `Header`, `Host`, `Query`, `Cookie`, `After`, `Weight`, `Version`...) + **filters** (what to change? `StripPrefix`, `AddRequestHeader`, `RewritePath`, `CircuitBreaker`, `Retry`, `RequestRateLimiter`, `TokenRelay`, ...).



The demo

`gateway-demo` defines one route in YAML and one in Java.

```yaml
spring:
  cloud:
    gateway:
      server:
        webmvc:
          routes:
            # Route 1, in YAML: /api/orders/** -> orders service
            - id: orders
              uri: ${services.orders.url}              # with discovery: lb://orders
              predicates:
                - Path=/api/orders/**
                - Method=GET,POST
              filters:
                - StripPrefix=1                          # /api/orders/7 -> /orders/7
                - AddRequestHeader=X-Gateway, ch16-gateway
                - AddResponseHeader=X-Served-Via, gateway
```

```java
/** Route 2, in the Java DSL: /api/inventory/** -> inventory service, protected by a circuit breaker. */
@Bean
RouterFunction<ServerResponse> inventoryRoute(@Value("${services.inventory.url}") String inventoryUrl) {
    return route("inventory")
            .GET("/api/inventory/**", http())
            .before(uri(inventoryUrl))
            .filter(circuitBreaker("inventoryCB", URI.create("forward:/fallback/inventory")))
            .build();
}

@GetMapping("/fallback/inventory")
String inventoryFallback() {
    return "{\"items\":[],\"note\":\"inventory temporarily unavailable\"}";
}
```

(Static imports: `GatewayRouterFunctions.route`, `HandlerFunctions.http`, `BeforeFilterFunctions.uri`, `CircuitBreakerFilterFunctions.circuitBreaker`. Pass the target via `before(uri(...))`: the `http(String)` / `http(URI)` overloads, deprecated in Gateway 4.x, are gone in Gateway 5.0 — only `http()` remains.) The `CircuitBreaker` filter uses Spring Cloud CircuitBreaker, hence `spring-cloud-starter-circuitbreaker-resilience4j`.

`GatewayRoutingTests` runs an echoing "orders service" and points the inventory route at a port where nothing listens:

```
GET /api/orders/7 -> orders-service saw GET /orders/7?expand=items X-Gateway=ch16-gateway | response X-Served-Via=gateway
DELETE /api/orders/7 and GET /api/unknown -> 404 (no route matches)
GET /api/inventory/items (service down) -> {"items":[],"note":"inventory temporarily unavailable"}
```

- `StripPrefix=1` removed `/api`; the query string was kept; request and response headers were added.
- `DELETE` fails the `Method` predicate and nothing matches `/api/unknown` → 404.
- The inventory connection failure went through the circuit breaker to the fallback — a degraded but valid answer.

Other common gateway jobs (not run)

- **Token relay** (`TokenRelay` filter + `spring-boot-starter-oauth2-client`): the gateway logs the user in and forwards the access token — 11.10, BFF in 11.12.
- **Rate limiting**: the WebFlux flavour's `RequestRateLimiter` (Redis); the MVC flavour's Bucket4j-based filter. See Rate Limiting.
- **Weighted routes** (`Weight=group, 90`) for canary releases (a small traffic share to a new version first).

Common mistakes

- Business logic or many-service aggregation in the gateway → a "distributed monolith" bottleneck. Keep it thin; use a BFF for client-specific aggregation.
- Spring Cloud Gateway *and* an Ingress/cloud gateway doing the same things — one layer per concern.
- Retrying non-idempotent `POST`s at the gateway (an order-creating `POST` can't safely run twice).

16.8 Resilience: circuit breaker, retry, rate limiter

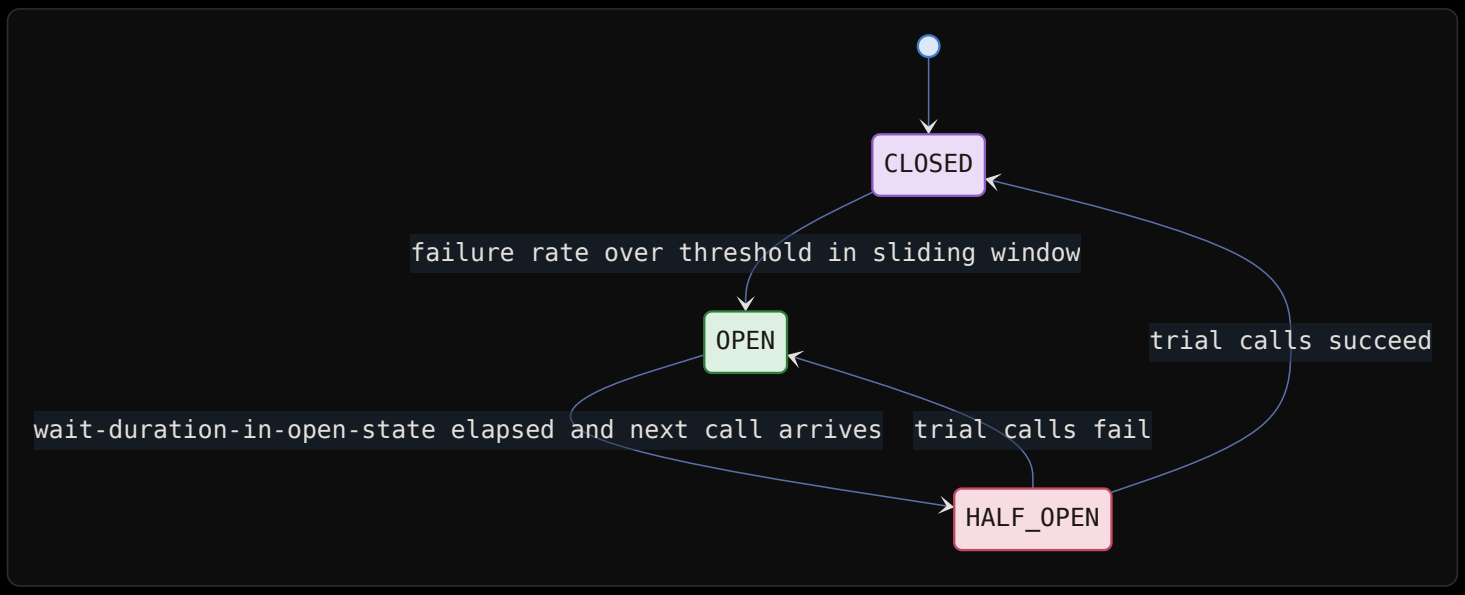
What it is

Remote calls fail and, worse, become **slow**; unprotected callers pile up waiting threads until they fail too — a **cascading failure**. Tools (Reliability module):

Tool	Idea	Notes
Timeout	never wait forever	the first line of defence; set on every client (Chapter 6)
Retry	try again after a transient failure, with back-off (growing waits) + jitter (random extra wait so clients don't retry in lockstep)	only idempotent operations — Retry notes
Circuit breaker	after many failures, stop calling for a while and fail fast / fall back	Circuit Breaker notes
Bulkhead	cap concurrent calls to one dependency	Bulkhead notes
Rate limiter	cap calls per time period	protects a dependency or enforces a quota
Fallback	a degraded answer (cache, default, empty list)	Graceful degradation

Resilience4j is the standard Java library for all of these (Hystrix's successor): directly (`resilience4j-spring-boot4`: annotations + properties + Micrometer metrics + health) or via **Spring Cloud CircuitBreaker**, an abstraction (`CircuitBreakerFactory.create("id").run(supplier, fallback)`) with a Resilience4j implementation, used e.g. by the gateway filter above.

Circuit breaker states



- **CLOSED** — calls pass; outcomes are recorded in a **sliding window** (last N calls, or last N seconds).
- **OPEN** — calls are rejected immediately with `CallNotPermittedException` (→ fallback) without touching the dependency, giving it time to recover.
- **HALF_OPEN** — a limited number of trial calls decide whether to close again.

The demo

`resilience-demo` wraps a fake remote `RemoteInventory` (the tests switch it up or down):

```
@CircuitBreaker(name = "inventory", fallbackMethod = "cachedStock")
public String stockWithBreaker(String sku) {
    return "live stock " + remote.stock(sku);
}

// fallback: same parameters + the exception as the last parameter
private String cachedStock(String sku, Throwable t) {
    return "cached stock 5 (" + t.getClass().getSimpleName() + ")";
}

@Retry(name = "inventory-retry")
public String stockWithRetry(String sku) { return "live stock " + remote.stock(sku); }

@RateLimiter(name = "search")
public String search(String text) { return "results for " + text; }
```

```
resilience4j:
  circuitbreaker:
    instances:
      inventory:
        sliding-window-type: COUNT_BASED
        sliding-window-size: 4                # judge the last 4 calls
        minimum-number-of-calls: 4            # ... but only once 4 calls were recorded
        failure-rate-threshold: 50            # open when >= 50 % of them failed
        wait-duration-in-open-state: 300ms    # stay OPEN this long, then allow trial calls
        permitted-number-of-calls-in-half-open-state: 2
        record-exceptions: ch16.resilience.RemoteCallException
  retry:
    instances:
      inventory-retry:
        max-attempts: 3                       # total attempts, including the first call
        wait-duration: 20ms
        enable-exponential-backoff: true
        exponential-backoff-multiplier: 2     # 20 ms, then 40 ms
        retry-exceptions: ch16.resilience.RemoteCallException
  ratelimiter:
    instances:
      search:
        limit-for-period: 3                   # 3 permits ...
        limit-refresh-period: 10s             # ... per 10-second cycle
        timeout-duration: 0                   # don't wait for a permit: reject immediately
```

`CircuitBreakerTests` subscribes to the breaker's state-transition events and walks it through all states:

```
1) dependency goes down, 4 calls:
   call 1: cached stock 5 (RemoteCallException)  state=CLOSED
   call 2: cached stock 5 (RemoteCallException)  state=CLOSED
   call 3: cached stock 5 (RemoteCallException)  state=CLOSED
   call 4: cached stock 5 (RemoteCallException)  state=OPEN
2) while OPEN the remote is not even called:
   call 5: cached stock 5 (CallNotPermittedException)  remote calls +0
3) wait 300 ms, dependency is back; 2 trial calls in HALF_OPEN:
   call 6: live stock 7  state=HALF_OPEN
   call 7: live stock 7  state=CLOSED
transitions: [CLOSED -> OPEN, OPEN -> HALF_OPEN, HALF_OPEN -> CLOSED]
```

- Calls 1–3 fail but the breaker stays CLOSED: the window needs `minimum-number-of-calls` (4) first.
- After call 4 the failure rate is 100% ≥ 50% → OPEN.
- Call 5 is rejected without calling the remote (`remote calls +0`) — what stops the cascade.
- After 300 ms the next call moves it to HALF_OPEN — by default (`automatic-transition-from-open-to-half-open-enabled: false`) no timer does this; the first call after the wait does. Two successful trials close it.
- Resilience4j defaults otherwise: `COUNT_BASED` window of 100 calls, `minimum-number-of-calls` 100, `failure-rate-threshold` 50, `wait-duration-in-open-state` 60 s, 10 permitted HALF_OPEN calls — sensible in production, too slow for a demo. Retry defaults to 3 attempts 500 ms apart.

`RetryAndRateLimiterTests`:

```
Framework 7 @Retryable, remote fails twice -> live stock 7 after 3 attempts
RateLimiter (3 per 10 s): call1=OK call2=OK call3=OK call4=REJECTED call5=REJECTED
Resilience4j @Retry, remote fails twice:
   retry #1 after inventory unavailable, waiting 20 ms
   retry #2 after inventory unavailable, waiting 40 ms
   -> live stock 7 after 3 attempts
   retry #1 after inventory unavailable, waiting 20 ms
   retry #2 after inventory unavailable, waiting 40 ms
   remote down: gave up after 3 attempts, caller gets RemoteCallException
```

- Resilience4j's `max-attempts: 3` counts the **first call**; Framework 7's `maxRetries = 2` counts only **retries** — both mean 3 attempts (`maxRetries = 3` would be 4).
- Exponential back-off doubled the wait (20 → 40 ms); add jitter in production (`randomized-wait-factor` / `enable-randomized-wait`).
- The rate limiter rejects with `RequestNotPermitted` once the period's 3 permits are used.

The Framework 7 version (Chapter 4.7 covers it fully):

```
@Retryable(includes = RemoteCallException.class, maxRetries = 2, delay = 20, multiplier = 2)
@ConcurrencyLimit(limit = 10, policy = ConcurrencyLimit.ThrottlePolicy.REJECT)   // a simple bulkhead
public String stock(String sku) { return "live stock " + remote.stock(sku); }
// needs @EnableResilientMethods on a configuration class
```

Resilience4j vs Framework 7 core

Need	Framework 7 core	Resilience4j
Retry with back-off, jitter	`@Retryable`, `RetryTemplate`	`@Retry`, `Retry`
Bulkhead (concurrency cap)	`@ConcurrencyLimit` (BLOCK or REJECT)	`@Bulkhead` (semaphore or thread pool)
Circuit breaker	—	`@CircuitBreaker`
Rate limiter	—	`@RateLimiter`
Time limiter	— (use client timeouts)	`@TimeLimiter` (async/reactive)
Fallback methods	— (catch at the caller)	`fallbackMethod`
Config in properties, per instance	attributes (placeholders allowed)	yes, rich
Metrics, health, events	`MethodRetryEvent`	Micrometer meters, health indicators, event publishers
Extra dependency	none	yes

Rule of thumb: plain retries and concurrency caps → Framework 7; anything involving circuit breaking, rate limiting or fallbacks → Resilience4j.

Order when stacking annotations (Resilience4j defaults): `Retry( CircuitBreaker( RateLimiter( TimeLimiter( Bulkhead( call ) ) ) ) )` — retry outermost, so each attempt is counted by the breaker, and an OPEN breaker's rejection can be retried unless you exclude `CallNotPermittedException`. Reorder with `resilience4j.*.<type>-aspect-order`.

Common mistakes

- Retrying everything, including 4xx and non-idempotent calls.
- Retries at gateway **and** client **and** service → retry storms (3 × 3 × 3 = 27 calls).

- A breaker without timeouts: slow calls never "fail", so it never opens (or configure `slow-call-duration-threshold` / `slow-call-rate-threshold`).
- One breaker for all endpoints of a service — one bad endpoint opens the circuit for healthy ones.
- Self-invocation — all these annotations are proxy-based (Chapter 4).
- Fallbacks hiding outages forever — alert on breaker state (`resilience4j_circuitbreaker_state` metric).

TL;DR

Timeouts first; retry only transient, idempotent failures with back-off + jitter; a circuit breaker to fail fast and give the dependency room; bulkhead/rate limiter to cap load; fallbacks for degraded answers. Framework 7 covers retry + concurrency limits; Resilience4j covers everything.

16.9 Distributed tracing across services

Little beyond Chapter 15.7: every service adds `spring-boot-starter-opentelemetry` and exports to the same collector; the gateway, clients from Boot's `RestClient.Builder` (including HTTP interface groups) and OpenFeign propagate `traceparent`. For messaging enable observation explicitly (`spring.kafka.template.observation-enabled=true`, `spring.kafka.listener.observation-enabled=true`, likewise RabbitMQ) so context travels in message headers. Microservice-specific points:

- Use the **same propagation format** everywhere (W3C by default; set `management.tracing.propagation.type=b3` only for legacy Zipkin/B3 services, or produce both).
- Set `spring.application.name` in every service — it becomes the service name in traces and logs.
- Let the **edge decide sampling** (the gateway) — downstream services follow the sampled flag.
- Tag spans with business ids as **high-cardinality** values (order id), never as metric tags.

16.10 Contract testing

What it is

Consumer A and provider B are tested separately — do they still agree on the API? End-to-end tests of all services are slow and flaky. **Contract testing** checks each side against a shared **contract** (request → expected response):

1. Consumer-driven: the consumer writes what it needs.
2. The provider's build **verifies** it responds that way.
3. The consumer's tests run against a **stub** (fake server with canned answers) generated from the contract.

Tool	How	Status
Pact (pact-jvm)	consumer tests produce a pact file; a Pact Broker (stores contracts and results) shares it; provider verifies; `can-i-deploy` gate	active, polyglot
Spring Cloud Contract	contracts in YAML/Groovy/Java in the provider repo; the plugin generates provider tests and a stubs jar; consumers use `@AutoConfigureStubRunner`	JVM/Spring-centric; not in the 2025.1.3 BOM, repository archived in `spring-attic` (see 16.2)
OpenAPI-based checks	validate both sides against the OpenAPI document	good for provider-driven public APIs

Spring Cloud Contract, for recognition (not run):

```
# provider: src/test/resources/contracts/stock/shouldReturnStock.yml
request:
  method: GET
  url: /stock/SKU-1
response:
  status: 200
  headers: { Content-Type: application/json }
  body: { sku: "SKU-1", quantity: 7 }
```

```
// consumer test: runs against a WireMock stub built from the provider's contracts
@SpringBootTest
@AutoConfigureStubRunner(ids = "com.acme:inventory:+:stubs:8090", stubsMode = StubRunnerProperties.StubsMode.LOCAL)
class InventoryClientContractTest { /* call http://localhost:8090/stock/SKU-1 */ }
```

When not to use it: a single team owning both sides that deploys them together, or public APIs where you can't know consumers (use versioning + OpenAPI instead).

16.11 Messaging between services

Synchronous HTTP chains couple availability: any service down fails the request. Events decouple them in time (Event-Driven notes). Spring options:

- **Spring for Apache Kafka / Spring AMQP** — templates and `@KafkaListener` / `@RabbitListener` (Chapter 14, incl. the transactional outbox).
- **Spring Cloud Stream** — a binder abstraction: `java.util.function` beans (`Supplier`, `Function`, `Consumer`) bound to Kafka/RabbitMQ destinations by configuration (`spring.cloud.function.definition=enrichOrder`, `spring.cloud.stream.bindings.enrichOrder-in-0.destination=orders`). Good for many similar pipelines; hides broker-specific features you may need.
- Cross-service consistency without distributed transactions: Saga (choreography — services react to each other's events — or orchestration — a coordinator directs each step), idempotent consumers, and CQRS read models.

16.12 Common mistakes (whole chapter)

Symptom	Cause	Fix
App fails at startup with a compatibility error	Spring Cloud train doesn't match Boot	2025.1.x for Boot 4.0/4.1
`No spring.config.import property has been defined`	config client on classpath without import	add `spring.config.import=configserver:...` (or `optional:`)
Config change not visible after refresh	plain `@Value` in a singleton	`@RefreshScope` or `@ConfigurationProperties`
`UnknownHostException: inventory`	client not load-balanced	`@LoadBalanced` builder, Feign `name`, or HTTP group without base URL
LoadBalancer "default cache" warning	Caffeine missing	add Caffeine
Gateway returns 404	predicates don't match (method, path, prefix)	check the route definition; DEBUG logging for `org.springframework.cloud.gateway`
Breaker never opens	no timeouts, slow calls not recorded, window too large	timeouts + slow-call thresholds + sensible window
Retry storm during an outage	retries at several layers	retry in one layer, with jitter; breaker around retries
Everything down when one service is down	synchronous chains, no fallbacks	timeouts, breakers, fallbacks, async events

---

16.13 In the interview

Q: Which Spring Cloud version do you use with Spring Boot 4.1? The 2025.1.x "Oakwood" train (Boot 4.0 and 4.1, the latter from 2025.1.2), imported as the `spring-cloud-dependencies` BOM; its components are 5.0.x.

Q: How do microservices find each other in Spring? Through the `DiscoveryClient` abstraction backed by Eureka, Consul, Zookeeper or Kubernetes; Spring Cloud LoadBalancer resolves service ids (`http://inventory`, `lb://inventory`) to instances per request, round robin by default. On Kubernetes plain Services + DNS usually suffice.

- *Client-side vs server-side load balancing?* → The client picks the instance (smarter, per-request, needs a library) vs a proxy picks it (language-agnostic, extra hop).

Q: Explain the circuit breaker states and how you configure them in Resilience4j. CLOSED records outcomes in a sliding window; when the failure (or slow-call) rate exceeds the threshold after `minimum-number-of-calls`, it goes OPEN and rejects calls fast; after `wait-duration-in-open-state` it goes HALF_OPEN and lets `permitted-number-of-calls-in-half-open-state` trial calls decide between CLOSED and OPEN. Configure per instance under `resilience4j.circuitbreaker.instances.<name>`; use `@CircuitBreaker(name, fallbackMethod)`.

- *Different from retry?* → Retry repeats one call hoping for success; the breaker stops calling an obviously unhealthy dependency. Combine them, the breaker protecting against retry storms.
- *Does Framework 7 have a circuit breaker?* → No — only `@Retryable` and `@ConcurrencyLimit`.

Q: What does an API gateway do, and which Spring Cloud Gateway flavour would you pick? Single entry point: routing by predicates, filters for auth/token relay, rate limiting, header rewriting, resilience. Web MVC flavour for servlet teams and virtual threads; WebFlux flavour for reactive stacks and very high concurrency. Keep business logic out.

Q: How do you manage configuration across 30 services? Externalised config: Spring Cloud Config (Git-backed, layered by app/profile/label, refresh via actuator or Bus) or platform-native ConfigMaps/Secrets with `configtree:`; secrets in Vault; config changes reviewed like code.

Q: OpenFeign or HTTP interfaces? → HTTP interfaces for new code (core Spring, actively developed, Boot 4 groups, load-balanced by Spring Cloud); OpenFeign is in maintenance mode.

Q: How do you test that services stay compatible? → Consumer-driven contract tests (Pact, or Spring Cloud Contract in existing projects) plus a small number of end-to-end smoke tests.

Chapter TL;DR

- Spring Cloud = ready-made answers to distributed-system problems; check what your platform (Kubernetes) already gives you before adding them. Patterns: microservices-patterns notes.
- **Boot 4.1 → Spring Cloud 2025.1.x (Oakwood)**, BOM `spring-cloud-dependencies` (2025.1.3 used here); components 5.0.x; OpenFeign in maintenance; Spring Cloud Contract no longer in the 2025.1.3 BOM.
- **Config Server**: `@EnableConfigServer`, Git or `native` backend; clients use `spring.config.import=configserver:`; precedence `app-profile` > `app` > `application`; refresh re-creates `@RefreshScope` beans (proved: 15% → 20%).
- **Discovery**: `DiscoveryClient` over Eureka/Consul/Kubernetes/simple; **LoadBalancer** resolves service ids per request, round robin by default (proved: B, A, B, A); add Caffeine.
- **Declarative clients**: prefer HTTP interfaces (`@ImportHttpServices`, load-balanced when the group has no base URL); OpenFeign still works.
- **Gateway**: routes = predicates + filters; server-webmvc or server-webflux; `StripPrefix`, headers, `CircuitBreaker` with fallback (proved).
- **Resilience4j**: breaker CLOSED → OPEN → HALF_OPEN → CLOSED (proved), retry with back-off, rate limiter (proved); Framework 7 `@Retryable` / `@ConcurrencyLimit` for simple cases.
- Tracing across services = Chapter 15 setup in every service; contracts with Pact/Spring Cloud Contract; events with Kafka/Stream.
- Next: Chapter 17 — Spring AI and what's new.

Chapter 17 — Spring AI and What's New in Boot 4 / Framework 7

Two halves. First, **Spring AI**: the Spring project for calling large language models (LLMs) through one API across providers (OpenAI, Anthropic, Google, Amazon Bedrock, Ollama, ...), plus what a real AI feature needs beyond "send text, get text": prompt templates (the **prompt** is the text sent to the model), conversation memory, answers as Java objects, letting the model call your code (tools), searching your documents (RAG, retrieval-augmented generation, §17.9) and the Model Context Protocol (MCP: a standard way to offer tools to AI apps, §17.10). Second, a **migration map**: Boot 3.x → 4.x, Framework 6 → 7, Security 6 → 7, each change pointing to the chapter that teaches it.

How LLMs work is **not** re-taught. Tokens (the word pieces a model reads, writes and bills by), context windows (max text per call), sampling (how each next token is picked), prompting, embeddings (number lists capturing a text's meaning, §17.9), RAG design and agents (a model repeatedly choosing the next tool) are in the AI book: LLMs, prompt engineering, building LLM apps, embeddings, vector databases and RAG, evaluation, agents. Here: the **Spring way**.

The code. `code/spring-ai-offline/` is a Spring Boot 4.1.1 project (Framework 7.0, Java 25) on **Spring AI 2.0.1**, needing **no API key** (the provider's authentication/billing secret) **and no network**: the `ChatModel` bean is a fake returning canned answers (`FakeChatModel`), the `EmbeddingModel` a word-hashing fake (`FakeEmbeddingModel`). Everything *above* the model — `ChatClient`, advisors, memory, tool calling, structured output, the vector store — is real Spring AI. Output is real `mvn -q -B -ntp verify` output (14 tests passing). Snippets needing a real provider are marked **(needs an API key — not run)**.

17.1 What Spring AI is

What it is

Spring AI does for AI providers what Spring Data does for databases: **portable interfaces** (`ChatModel`, `EmbeddingModel`, `VectorStore`, ...), **Boot starters** auto-configuring a provider from properties, and a **fluent client** (`ChatClient`). As `RestClient` (Chapter 6) hides the HTTP library, `ChatClient` hides the provider: switching OpenAI → Anthropic is mostly a starter and property change.

Why it matters

Calling an LLM's HTTP API directly is fine for a demo. Production also needs prompt templates outside the code, conversation memory (the model remembers nothing between calls), typed Java answers, tool calling, retrieval of your documents (RAG), and logging/metrics/tracing/retries plus tests that don't call a paid API. Spring AI provides each as a Spring building block — beans, properties, Micrometer observations (Chapter 15), Boot tests (Chapter 12).

Versions (verified September 2026)

Spring AI	Released	Works with	Notes
1.0	May 2025	Boot 3.4 / 3.5	first GA (general availability)
1.1	Nov 2025	Boot 3.4 / 3.5	MCP annotations, more providers
2.0	**Jun 2026**	**Boot 4.0 / 4.1, Framework 7**	Jackson 3, JSpecify null-safety (annotations marking what may be `null`), tool loop moved into advisors, immutable options, MCP Java SDK 2.0. **Current: 2.0.1** (used here)

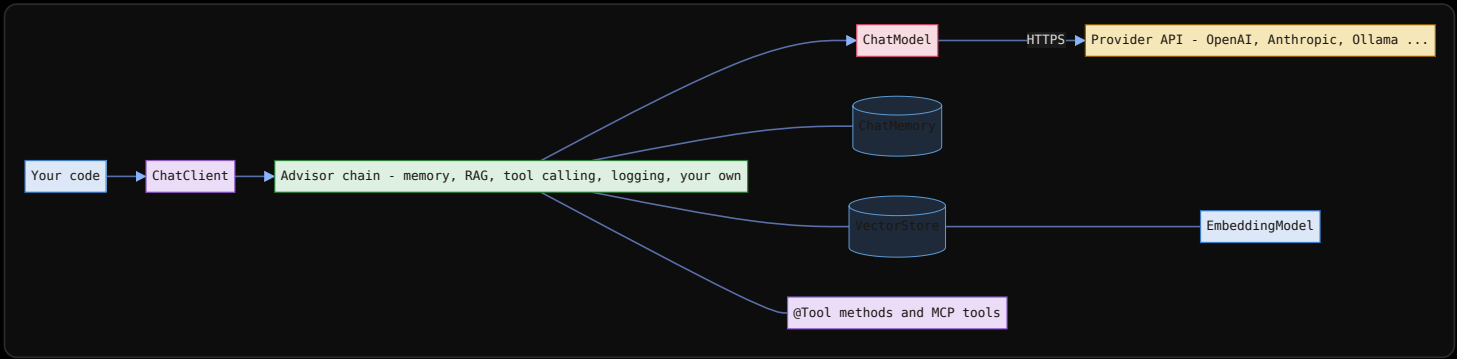
Spring AI 2.0 **requires** Boot 4; a Boot 3.5 app stays on 1.1.x until it migrates. 2.0.0's starters were built against Boot 4.1 dependency versions (spring-ai issue #6465), so Boot 4.1.x is smoothest.

Boot doesn't manage Spring AI's version, so import the **Spring AI BOM** (bill of materials), which keeps all `spring-ai-*` artifacts aligned:

```
<dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>org.springframework.ai</groupId>
      <artifactId>spring-ai-bom</artifactId>
      <version>2.0.1</version>
      <type>pom</type>
      <scope>import</scope>
    </dependency>
  </dependencies>
</dependencyManagement>
```

The building blocks

Building block	What it does	Real implementations
`ChatModel`	low-level, provider-level: sends a `Prompt` (messages + options), returns a `ChatResponse`	`OpenAiChatModel`, `AnthropicChatModel`, `OllamaChatModel`, ... (our `FakeChatModel`)
`ChatClient`	the fluent, everyday API on top of a `ChatModel`, with advisors	one implementation, built from a `ChatClient.Builder`
Advisor	an interceptor around every `ChatClient` call (like a servlet filter)	logging, memory, RAG, tool calling, safeguard, validation
`ChatMemory`	stores past messages per conversation	`MessageWindowChatMemory` over in-memory / JDBC / other repositories
Structured output	converts the text answer into a Java type	`BeanOutputConverter`, `.entity(...)`
`ToolCallback` / `@Tool`	a Java method the model may ask to run	method tools, function tools, MCP tools
`EmbeddingModel`	turns text into a vector (fixed-length list of numbers)	OpenAI, Ollama, ONNX transformers, ... (our `FakeEmbeddingModel`)
`VectorStore`	stores document vectors and finds similar ones	`SimpleVectorStore` (in memory), PgVector (PostgreSQL vector extension), Redis, Elasticsearch, Qdrant, ...
MCP	standard protocol to expose / consume tools across processes	MCP server and client starters



17.2 Setting up a project

With a real provider (needs an API key — not run)

1. Add the BOM (above) and **one model starter**:

```
<!-- OpenAI -->
<dependency>
  <groupId>org.springframework.ai</groupId>
  <artifactId>spring-ai-starter-model-openai</artifactId>
</dependency>
<!-- or Anthropic (Claude) -->
<dependency>
  <groupId>org.springframework.ai</groupId>
  <artifactId>spring-ai-starter-model-anthropic</artifactId>
</dependency>
```

2. Configure it — key in an environment variable or secret store, **never** in Git (Chapter 3 explains `${...}`):

```
# OpenAI (needs an API key — not run)
spring.ai.openai.api-key=${OPENAI_API_KEY}
spring.ai.openai.chat.model=gpt-5-mini
spring.ai.openai.chat.temperature=0.2
spring.ai.openai.embedding.model=text-embedding-3-small
```

```
# Anthropic (needs an API key — not run)
spring.ai.anthropic.api-key=${ANTHROPIC_API_KEY}
spring.ai.anthropic.chat.model=claude-sonnet-5-5
# a larger model for harder tasks: claude-opus-5-5
spring.ai.anthropic.chat.max-tokens=4096
```

3. Inject `ChatClient.Builder` and use it (`temperature` controls randomness — lower is more predictable; `max-tokens` caps answer length). The rest of the chapter works unchanged with either provider.

Spring AI 2.0 configuration facts (from the 2.0 upgrade notes):

Change in 2.0	Before (1.x)	Now (2.0)
option properties are **flat**	`spring.ai.openai.chat.options.model=...`	`spring.ai.openai.chat.model=...`
Anthropic module	Spring's own `AnthropicApi` client	the official `anthropic-java` SDK; default `max-tokens` raised from 500 to 4096
OpenAI modules	three variants (`openai`, `openai-sdk`, `azure-openai`)	one `spring-ai-openai` (Azure/Microsoft Foundry through its properties)
options objects	mutable, `copy()`	immutable, `mutate()`; `ChatClient.options(...)` takes a **builder**
several model starters on the classpath	—	pick one with `spring.ai.model.chat=openai` / `anthropic` / ...

Default models change (2.0's OpenAI default is `gpt-5-mini`, Anthropic's `claude-haiku-4-5`). Always set the model explicitly so an upgrade doesn't silently change cost and behaviour.

Offline, as in this chapter

Our `pom.xml` has the BOM plus these artifacts and **no model starter**:

Artifact	Why
`spring-boot-starter`	a plain (non-web) Boot app
`spring-ai-autoconfigure-model-chat-client`	auto-configures a `ChatClient.Builder` bean from whatever `ChatModel` bean exists
`spring-ai-autoconfigure-model-tool`	the tool-calling manager and its properties (an *optional* dependency of the previous one, so add it yourself — else Boot logs `TypeNotPresentException ... ToolCallingAutoConfiguration`)
`spring-ai-autoconfigure-model-chat-memory`	a `ChatMemory` bean (window of the last 20 messages, stored in memory)
`spring-ai-vector-store`, `spring-ai-vector-store-advisor`	`SimpleVectorStore` and `QuestionAnswerAdvisor`

A model starter would pull these in. We declare our own `ChatModel` and `EmbeddingModel` beans (`AiConfig`):

```
@Configuration
public class AiConfig {

    /** With a real starter on the classpath, Boot would create OpenAiChatModel / AnthropicChatModel here. */
    @Bean
    FakeChatModel chatModel() { return new FakeChatModel(); }

    @Bean
    EmbeddingModel embeddingModel() { return new FakeEmbeddingModel(); }
    // ... vectorStore and supportClient beans, shown later
}
```

Why a fake model?

A **fake** (a test double with simple working logic, Chapter 12) tests everything around the model without cost, network, rate limits or randomness. Real LLM output is **non-deterministic** (the same prompt gives different words), so tests asserting exact LLM text are flaky. The usual strategy:

What you test	How
your prompts, advisors, tools, parsing, error handling	fake or stub `ChatModel` — fast, deterministic, runs in CI
"is the answer good?"	a small **evaluation** set run against the real model, scored by rules or by another model (Spring AI has `RelevancyEvaluator` and `FactCheckingEvaluator`); see AI 8.5
the provider integration	a few tagged integration tests with a real key, run on demand

---

17.3 `ChatModel` and `ChatClient`

`ChatModel`: the provider contract

`ChatModel` has one essential method, `ChatResponse call(Prompt prompt)`. A `Prompt` is a list of **messages** plus **options** (model, temperature, tools, ...); a `ChatResponse` holds `Generation`s, each with an `AssistantMessage`.

Message type	Class	Who writes it
system	`SystemMessage`	you: the rules and persona ("You are a support assistant...")
user	`UserMessage`	the end user's question (often via a template)
assistant	`AssistantMessage`	the model's reply; may contain **tool calls** instead of text
tool	`ToolResponseMessage`	your application: the result of a tool the model asked for

Our fake implements exactly this interface (`FakeChatModel`), recording each prompt so tests can show what reached "the model", then applying a few rules:

```
public class FakeChatModel implements ChatModel {

    private final List<Prompt> received = new CopyOnWriteArrayList<>();

    @Override
    public ChatResponse call(Prompt prompt) {
        received.add(prompt);                                   // remember what the model was sent
        return new ChatResponse(List.of(new Generation(reply(prompt))));
    }

    /** Tool-calling-capable default options, like a real provider's options class. */
    @Override
    public ChatOptions getOptions() {
        return ToolCallingChatOptions.builder().build();
    }

    private AssistantMessage reply(Prompt prompt) {
        Message last = prompt.getInstructions().getLast();
        if (last instanceof ToolResponseMessage toolResult) { /* Rule 1: explain the tool result */ }
        String user = prompt.getUserMessage().getText();
        // Rule 2: tools offered + an order id in the question -> return an AssistantMessage with a ToolCall
        // Rule 3: "JSON Schema" in the prompt -> return JSON in a ```json fence
        // Rule 4: RAG context present -> answer from its first line
        // Rule 5: "what is my name" -> look for "my name is X" in earlier messages
        return new AssistantMessage("(fake model) You asked: " + user);
    }
    // ...
}
```

`getOptions()` matters: `ChatClient` copies tool definitions into the prompt's options only when the model's options type supports tools (`ToolCallingChatOptions`), as every real provider's does.

`ChatClient`: the everyday API

Boot auto-configures a **prototype-style** `ChatClient.Builder` bean (a new builder per injection point), wired to the `ChatModel` bean and observability. Customise it and `build()` a thread-safe `ChatClient` meant for reuse (make it a bean).

```
@SpringBootTest
class ChatClientTests {

    @Autowired ChatClient.Builder builder;    // auto-configured by Spring AI from the ChatModel bean
    @Autowired FakeChatModel model;

    @Test
    void simplestCall() {
        ChatClient chat = builder.build();
        String answer = chat.prompt()          // start a request
                .user("What is Spring AI?")    // the user message
                .call()                        // blocking call (use .stream() for a Flux)
                .content();                    // just the text of the reply
        System.out.println("answer: " + answer);
        assertThat(answer).isEqualTo("(fake model) You asked: What is Spring AI?");
    }
}
```

Output:

```
answer: (fake model) You asked: What is Spring AI?
```

What `call()` can return:

Method	Returns	Use when
`.content()`	`String`	you only need the text
`.entity(Type.class)`	your Java type	structured output (§17.7)
`.chatResponse()`	`ChatResponse`	metadata: token usage, finish reason (why the model stopped: done, length limit), model id
`.chatClientResponse()`	response + advisor context	you need what advisors stored, e.g. the retrieved documents
`.stream().content()`	`Flux<String>`	show tokens as they arrive (chat UIs, SSE — Chapter 13)

Common mistakes

- A new `ChatClient` per request instead of one bean per use case.
- Calling `ChatModel` directly everywhere, losing advisors, memory and observability — use `ChatClient` unless writing infrastructure.
- Assuming the model remembers the previous call (§17.6).

17.4 Prompts and prompt templates

What it is

A **prompt template** is text with placeholders filled at run time — like SQL bind parameters. `PromptTemplate` uses `{name}` placeholders (StringTemplate engine; change delimiters with `StTemplateRenderer` if the text contains braces, e.g. JSON examples).

How it works

1. Write the template inline or in a file such as `prompts/explain.st`: `Explain {topic} to a {audience} in at most {words} words.`
2. Supply values with `render(Map)` / `create(Map)`, or `.param(...)` on `ChatClient`.
3. A **missing** value is an error (`IllegalStateException` listing the names), not an empty string — typos fail fast.

```
PromptTemplate template = new PromptTemplate(
        "Explain {topic} to a {audience} in at most {words} words.");
String text = template.render(Map.of("topic", "dependency injection", "audience", "beginner", "words", 50));

PromptTemplate fromFile = new PromptTemplate(new ClassPathResource("prompts/explain.st"));
Prompt prompt = fromFile.create(Map.of("topic", "virtual threads", "audience", "Java developer", "words", 80));

new PromptTemplate("Hello {name}, your plan is {plan}.").render(Map.of("name", "Ana"));   // throws
```

Output (`PromptTemplateTests`):

```
rendered: Explain dependency injection to a beginner in at most 50 words.
prompt messages: 1, user text: Explain virtual threads to a Java developer in at most 80 words.
missing variable -> IllegalStateException: Not all variables were replaced in the template. Missing variable names are: [plan].
```

With `ChatClient`, templates and a system message look like this:

```
@Value("classpath:prompts/explain.st") Resource explainTemplate;

ChatClient chat = builder.defaultSystem("You are a concise Java tutor.").build();
chat.prompt()
    .user(u -> u.text(explainTemplate)
                .param("topic", "records")
                .param("audience", "beginner")
                .param("words", 40))
    .call()
    .content();
```

Output — the messages the model actually received:

```
messages the model received:
  SYSTEM: You are a concise Java tutor.
  USER: Explain records to a beginner in at most 40 words.
```

`defaultSystem(...)`, `defaultUser(...)`, `defaultAdvisors(...)`, `defaultTools(...)` and `defaultOptions(...)` apply to every request of that client; per-request `.system(...)`, `.user(...)`, `.advisors(...)`, `.tools(...)`, `.options(...)` add to or override them.

Common mistakes

- Concatenating user input into the **system** message — makes **prompt injection** (text crafted to override your instructions) easier (AI 9.4). User text belongs in the user message.

- Templates buried in Java strings — put long prompts in `src/main/resources/prompts/*.st`, loaded as `Resource`, reviewed and versioned like code.
- JSON examples with `{...}` in a template without changing delimiters — the braces become placeholders.

17.5 Advisors

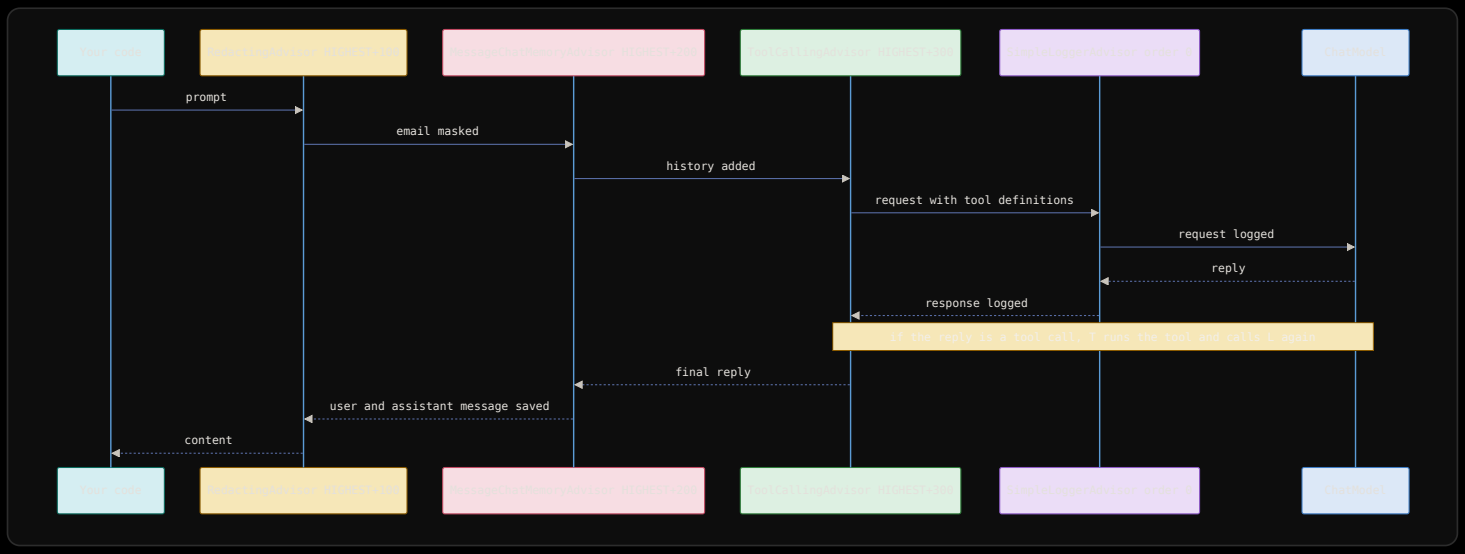
What it is

An **advisor** intercepts every `ChatClient` call on the way **to** the model and **back**. It can change the prompt (add memory or retrieved documents, redact data), change the response, or short-circuit the call (answer without the model). Advisors are to `ChatClient` what servlet filters are to Spring MVC (5.11) and `@Around` advice is to AOP (Chapter 4).

How it works

1. `ChatClient` builds a `ChatClientRequest` (the `Prompt` + a shared **context** map).
2. Advisors are sorted by `getOrder()` — **lower runs first** on the way in and **last** on the way out.
3. Each advisor calls `chain.nextCall(request)` to pass control on. The last link calls the `ChatModel`.
4. The response travels back through the advisors in reverse order.

`HIGHEST` is `Ordered.HIGHEST_PRECEDENCE` (the smallest `int`). Real Spring AI 2.0 defaults: memory advisors `HIGHEST_PRECEDENCE + 200`, the auto-registered `ToolCallingAdvisor` `HIGHEST_PRECEDENCE + 300`, `SimpleLoggerAdvisor` `0` — so the logger sits *inside* the tool loop (logging every model round trip) and memory outside it (saving only the final exchange).



Built-in advisors (Spring AI 2.0)

Advisor	Purpose	Section
`SimpleLoggerAdvisor`	logs request and response at DEBUG	below
`MessageChatMemoryAdvisor`	adds past messages of the conversation to the prompt, saves the new exchange	§17.6
`VectorStoreChatMemoryAdvisor`	long-term memory: retrieves *relevant* past messages from a vector store	§17.6
`QuestionAnswerAdvisor`	simple RAG: similarity search (closest stored texts in meaning) + context pasted into the user message	§17.9
`RetrievalAugmentationAdvisor`	modular RAG: query rewriting, expansion, retrieval, joining, augmentation	§17.9
`ToolCallingAdvisor`	runs the tool-call loop; **auto-registered** by `ChatClient` (new in 2.0; replaced the loop inside each `ChatModel`)	§17.8
`SafeGuardAdvisor`	blocks requests containing configured sensitive words	—
`StructuredOutputValidationAdvisor`	validates JSON output against the schema and asks the model to retry if it does not match (new in 2.0)	§17.7

A custom advisor: redacting e-mail addresses

Many companies must not send personal data to an external provider; an advisor enforces that once, for every call (`RedactingAdvisor`):

```java
public class RedactingAdvisor implements CallAdvisor {

    private static final String EMAIL = "[\\w.+-]+@[\\w-]+\\.[\\w.]+";

    @Override
    public ChatClientResponse adviseCall(ChatClientRequest request, CallAdvisorChain chain) {
        Prompt redacted = request.prompt().augmentUserMessage(
                user -> user.mutate().text(user.getText().replaceAll(EMAIL, "[email]")).build());
        return chain.nextCall(request.mutate().prompt(redacted).build());   // pass it on down the chain
    }

    @Override
    public String getName() { return "RedactingAdvisor"; }

    /** Run BEFORE the memory advisor (HIGHEST_PRECEDENCE + 200) so the stored history is redacted too. */
    @Override
    public int getOrder() { return Ordered.HIGHEST_PRECEDENCE + 100; }
}
```

- `CallAdvisor` handles blocking calls; also implement `StreamAdvisor` for `.stream()`. For simple before/after logic, `BaseAdvisor` offers `before(...)` / `after(...)` hooks.
- Requests and prompts are **immutable**: create a modified copy (`mutate()`) and pass it on.

The support client wires three advisors as defaults:

```java
@Bean
ChatClient supportClient(ChatClient.Builder builder, ChatMemory chatMemory) {
    return builder
            .defaultSystem("You are a polite support assistant for Acme Shop. Answer briefly.")
            .defaultAdvisors(
                    MessageChatMemoryAdvisor.builder(chatMemory).build(),
                    new RedactingAdvisor(),
                    SimpleLoggerAdvisor.builder()         // logs at DEBUG; here: one short line each way
                            .requestToString(req -> req.prompt().getUserMessage().getText())
                            .responseToString(res -> res.getResult().getOutput().getText())
                            .build())
            .build();
}
```

`SimpleLoggerAdvisor` logs at **DEBUG** (enable `logging.level.org.springframework.ai.chat.client.advisor=DEBUG` in `application.properties`); by default it logs whole request/response objects, shortened above. The test sends `alice@example.com` and a shipping question:

```
model saw:      Please send the invoice to [email]
memory stored:  Please send the invoice to [email]
LOG SimpleLoggerAdvisor    : request: Do you ship to Canada?
LOG SimpleLoggerAdvisor    : response: (fake model) You asked: Do you ship to Canada?
```

Because the redactor runs *before* the memory advisor, neither the model nor the stored history sees the address; reversed, memory would store it and resend it in every later prompt.

Common mistakes

- Wrong order: memory before RAG and logging if they should see history; redaction before anything that stores or sends text.
- Forgetting `chain.nextCall(...)` — the model is never called.
- Logging full prompts in production (personal data, size). Spring AI's observations keep prompt/completion content **out** of logs and traces by default (`spring.ai.chat.client.observations.log-prompt=false`).

In the interview

Q: What is an advisor in Spring AI? → An interceptor around every `ChatClient` call, ordered by `getOrder()`, that can modify the request (add memory, retrieved documents, redaction) or the response. Memory, RAG, logging and — since 2.0 — the tool-calling loop are all advisors.

- *Like what else in Spring?* → A servlet filter chain or AOP `@Around` advice: each link calls the next.
- *Where to enforce "no personal data to the provider"?* → An early advisor, so it covers every call and stored memory.

17.6 Chat memory

What it is

LLM APIs are **stateless**: each request must carry the whole conversation, so "memory" means your app stores past messages and **re-sends** the relevant ones. Spring AI splits this into:

Piece	Role	Default
`ChatMemoryRepository`	stores messages per conversation id	`InMemoryChatMemoryRepository`; JDBC (`spring-ai-starter-model-chat-memory-repository-jdbc`), Cassandra, Neo4j, Redis, ...
`ChatMemory`	decides **which** messages to keep	`MessageWindowChatMemory`: the last **20** messages (system messages are kept)
memory advisor	loads history into the prompt, saves the new exchange	`MessageChatMemoryAdvisor` (history as messages) or `VectorStoreChatMemoryAdvisor` (relevant old messages from a vector store)

Why a window? The context window is limited and every token costs (AI 6.9); long conversations need trimming or summarising.

How it works

1. Every call passes a **conversation id** (a user id, a chat session id).
2. The advisor loads that conversation's messages and puts them before the new user message.
3. After the reply, it saves the user message and the assistant reply.

In Spring AI **2.0 the conversation id is required at call time** (the old default id and the builder's `.conversationId(...)` were removed):

```
private String ask(String conversationId, String text) {
    return support.prompt()
            .user(text)
            .advisors(a -> a.param(ChatMemory.CONVERSATION_ID, conversationId))
            .call()
            .content();
}

ask("alice-42", "Hi, my name is Alice.");
ask("alice-42", "What is my name?");
ask("bob-7",    "What is my name?");
```

Output (`AdvisorTests`):

```
alice> (fake model) You asked: Hi, my name is Alice.
alice> Your name is Alice.
bob>   I don't know your name.
second call for alice-42 sent these messages to the model:
  SYSTEM: You are a polite support assistant for Acme Shop. Answer briefly.
  USER: Hi, my name is Alice.
  ASSISTANT: (fake model) You asked: Hi, my name is Alice.
  USER: What is my name?
```

The fake has no memory: it "knows" Alice's name only because the advisor re-sent the first exchange. Bob's conversation is separate; the test asserts 4 stored messages for `alice-42`, 2 for `bob-7`.

Common mistakes

- One fixed conversation id for everybody — users see each other's history. Derive it from the authenticated user and session (Chapter 10).
- In-memory repository in a cluster (e.g. three instances behind a load balancer): different memory per instance, lost on restart. Use JDBC or another shared repository.
- Unbounded history: cost and latency grow every turn. Keep a window; summarise older turns.

17.7 Structured output

What it is

Structured output returns the answer as data (record, list, map). A **converter** (1) appends format instructions with a **JSON Schema** (a JSON document describing required fields and types) of your type to the prompt and (2) parses the reply with Jackson 3. Why schemas help: AI 6.4.

```
public record MovieRecommendation(String title, int year, List<String> genres, double score) {}

MovieRecommendation movie = builder.build().prompt()
        .user("Recommend one science-fiction movie about survival.")
        .call()
        .entity(MovieRecommendation.class);          // schema added, reply parsed
```

Output (`StructuredOutputTests`):

```
user message sent to the model (first 3 lines):
  | Recommend one science-fiction movie about survival.
  | Your response should be in JSON format.
  | Do not include any explanations, only provide a RFC8259 compliant JSON response following this format without deviation.
  | ... (31 lines in total, including a JSON Schema)
parsed record: MovieRecommendation[title=The Martian, year=2015, genres=[sci-fi, survival], score=8.0]
```

The converter added ~30 lines to the prompt (tokens you pay for), and the fake replied with JSON **inside a Markdown code fence** (`` ```json ``), as models often do; the converter strips the fence before parsing.

The converter also works standalone:

```
var converter = new BeanOutputConverter<>(MovieRecommendation.class);
MovieRecommendation m = converter.convert("""
       {"title":"Arrival","year":2016,"genres":["sci-fi","drama"],"score":7.9}""");
```

```
converted: MovieRecommendation[title=Arrival, year=2016, genres=[sci-fi, drama], score=7.9]
schema mentions the fields: true, true
```

Options

Option	How	Notes
a class / record	`.entity(MovieRecommendation.class)`	most common
generic types	`.entity(new ParameterizedTypeReference<List<MovieRecommendation>>() {})`	lists, maps
map / list converters	`MapOutputConverter`, `ListOutputConverter`	loose structures
native structured output	`.advisors(AdvisorParams.ENABLE_NATIVE_STRUCTURED_OUTPUT)`	the provider itself constrains output to the schema (only supporting providers/models, e.g. OpenAI; check your provider's page) — more reliable than instructions
validation + retry	`StructuredOutputValidationAdvisor` (2.0)	checks the JSON against the schema; asks the model again on mismatch

Common mistakes

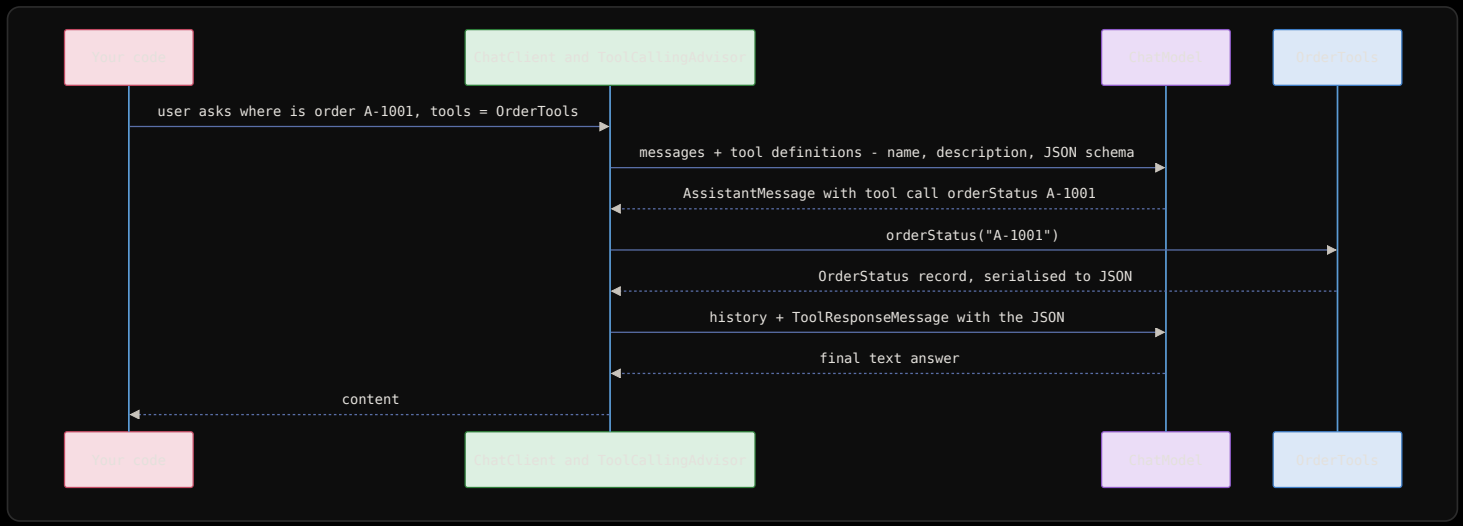
- Trusting the parsed object blindly — right *shape*, possibly wrong *values*. Validate business rules (Bean Validation, 5.6).
- Huge nested types — the schema costs tokens; keep output types small.
- Forgetting parsing can fail — catch, retry once, or use the validation advisor.

17.8 Tool calling

What it is

Tool calling (function calling) lets the model ask your application to run a function — look up an order, check stock, call an API — and answer with the result. The model **never runs code**: it returns a request like "call `orderStatus` with `{"orderId":"A-1001"}`", and your app decides whether to run it. Concept: AI 6.5, AI 9.3.

How it works



1. You mark methods with `@Tool` and pass the object with `.tools(...)` (or `.defaultTools(...)` on the builder).
2. Spring AI generates a **tool definition** from the method: name, description, and a JSON Schema of the parameters.
3. The model replies with a **tool call** instead of text.
4. The `ToolCallingAdvisor` (auto-registered since 2.0) runs the method, converts the result to JSON, adds a `ToolResponseMessage`, and calls the model **again**.
5. The loop repeats until the model answers with text.

```
public class OrderTools {

    public record OrderStatus(String orderId, String status, String eta) {}

    @Tool(name = "orderStatus", description = "Look up the delivery status of an order by its id, e.g. A-1001")
    public OrderStatus orderStatus(@ToolParam(description = "the order id") String orderId) {
        return new OrderStatus(orderId, "SHIPPED", "2026-10-02");   // a real app would call the order service
    }
}

String answer = builder.build().prompt()
        .user("Where is my order A-1001?")
        .tools(new OrderTools())              // expose the @Tool methods for this request
        .call()
        .content();
```

Output (`ToolCallingTests`) — the test inspects the two prompts the model received:

```
model calls: 2
1) tools offered to the model: [orderStatus]
2) model asked for: orderStatus {"orderId":"A-1001"}
3) Spring ran the method, sent back: {"orderId":"A-1001","status":"SHIPPED","eta":"2026-10-02"}
4) final answer: Here is what I found via orderStatus: {"orderId":"A-1001","status":"SHIPPED","eta":"2026-10-02"}
no tools -> (fake model) You asked: Where is my order A-1001?
```

One user question cost **two model calls** — every tool round trip adds latency and tokens.

Details worth knowing (Spring AI 2.0)

Topic	Fact
description	the model decides *when* to call a tool from its **description** — write it like documentation for a new colleague
`returnDirect = true`	return the tool result straight to the caller without a second model call
`ToolContext`	pass hidden data (tenant id, user id) to tools with `.toolContext(Map)`; it is **not** sent to the model
limits	at most **40 calls per tool** and **150 in total** per request by default (`spring.ai.tools.limits.*`) against runaway loops
resolution	only tools attached to the request or client can run; resolving arbitrary beans by name is **off** by default in 2.0
errors	by default a tool exception is sent back to the model as an error message (`spring.ai.tools.throw-exception-on-error=false`) so it can recover
programmatic tools	`FunctionToolCallback` / `MethodToolCallback` build tools without annotations

Security: tools are an attack surface

Tools are an **attack surface**: the model's tool arguments are **untrusted input**, and prompt-injection text in a retrieved document or user message can make the model call tools with harmful arguments. Rules:

- Check authorization **inside** the tool for the real user (from `ToolContext` or the security context, never from model-supplied arguments).
- Least power: read-only where possible, human confirmation for anything irreversible (AI 9.4).
- Validate arguments like any request body.

In the interview

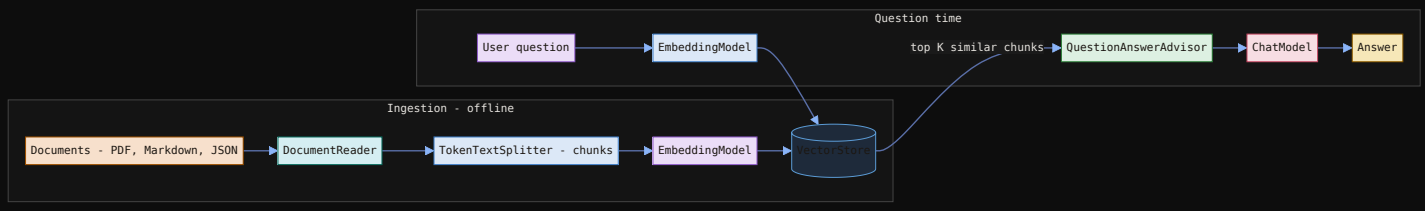
Q: Explain tool calling in Spring AI. → You annotate methods with `@Tool`; Spring AI sends their JSON-schema definitions with the prompt. When the model returns a tool call, the `ToolCallingAdvisor` executes the method, sends the result back as a tool message, and calls the model again until it produces a final answer. The model only *requests* calls; the application executes them.

- *Stop an infinite loop?* → Tool-call limits (40 per tool, 150 total by default in 2.0) plus timeouts.
- *Pass the current user to a tool safely?* → `ToolContext` or the security context — never a model-chosen user id argument.

17.9 RAG: retrieval-augmented generation

What it is

RAG answers from **your** documents: find the most relevant chunks, paste them into the prompt, tell the model to answer only from them. It reduces made-up answers and uses private or fresh data without retraining. Design questions (chunking — splitting documents, hybrid search — keyword + meaning, reranking — re-sorting with a stronger model, access control, evaluation): AI Chapter 7. Here: the Spring AI wiring.



Piece	Spring AI type	Examples
read	`DocumentReader`	`TextReader`, `JsonReader`, `MarkdownDocumentReader`, `PagePdfDocumentReader`, `TikaDocumentReader`
split	`TokenTextSplitter`	chunks of about 800 tokens by default
embed	`EmbeddingModel`	OpenAI `text-embedding-3-small`, Ollama, ONNX transformers (local)
store and search	`VectorStore`	`SimpleVectorStore` (in memory), PgVector, Redis, Elasticsearch, Qdrant, Milvus, ...
augment	advisor	`QuestionAnswerAdvisor` (simple), `RetrievalAugmentationAdvisor` (modular)

The embedding model and the vector store

An **embedding** is a vector representing a text so that similar meanings point in similar directions (AI 7.2); similarity is usually the **cosine** of the angle (1 = same direction, 0 = unrelated).

Our `FakeEmbeddingModel` hashes each word (skipping stop words like "the", cutting words to 5 letters) into one of 256 slots and normalises the vector to length 1, so texts sharing words become similar. A real model captures *meaning* ("refund" ≈ "money back"), which word hashing can't.

```
@Bean
VectorStore vectorStore(EmbeddingModel embeddingModel) {
    SimpleVectorStore store = SimpleVectorStore.builder(embeddingModel).build();
    store.add(List.of(
            new Document("Refunds are issued to the original payment method within 5 business days.",
                    Map.of("source", "refund-policy.md")),
            new Document("Standard shipping takes 3 to 7 days. Express shipping arrives the next day.",
                    Map.of("source", "shipping.md")),
            new Document("You can change your password under Settings, then Security.",
                    Map.of("source", "account.md"))));
    return store;
}
```

`add(...)` embeds each document and stores the vector with its text and **metadata** (source, tenant, date — for filtering and citations).

```
List<Document> hits = vectorStore.similaritySearch(
        SearchRequest.builder().query("How long do refunds take?").topK(2).build());

List<Document> filtered = vectorStore.similaritySearch(SearchRequest.builder()
        .query("How long do refunds take?").topK(2).similarityThreshold(0.1).build());
```

Output (`RagTests`):

```
score 0.192  refund-policy.md   Refunds are issued to the original payment method within 5 business days.
score 0.000  shipping.md        Standard shipping takes 3 to 7 days. Express shipping arrives the next day.
with similarityThreshold(0.1): 1 document(s)
```

`topK(2)` always returns two documents, even an irrelevant one (score 0.000); a **similarity threshold** drops weak matches — without it RAG pastes noise into the prompt. Tune K and the threshold on real questions.

`QuestionAnswerAdvisor`

```
ChatClient chat = builder
        .defaultAdvisors(QuestionAnswerAdvisor.builder(vectorStore)
                .searchRequest(SearchRequest.builder().topK(1).similarityThreshold(0.2).build())
                .build())
        .build();

String answer = chat.prompt().user("How fast is express shipping?").call().content();
```

Output — the user message after the advisor rewrote it:

```
augmented user message:
  | How fast is express shipping?
  |
  | Context information is below, surrounded by ---------------------
  |
  | ---------------------
  | Standard shipping takes 3 to 7 days. Express shipping arrives the next day.
  | ---------------------
  |
  | ...
answer: According to our docs: Standard shipping takes 3 to 7 days. Express shipping arrives the next day.
```

The rest of the default template tells the model to answer from the context and say so when the answer isn't there. Replace it with `.promptTemplate(...)`; restrict the search per request with a metadata **filter expression**, e.g. `a.param(QuestionAnswerAdvisor.FILTER_EXPRESSION, "tenant == 'acme'")` — how document-level permissions are enforced (AI 7.9). Retrieved documents are stored in the response context under `QuestionAnswerAdvisor.RETRIEVED_DOCUMENTS`, for citations.

For more control, `RetrievalAugmentationAdvisor` (module `spring-ai-rag`) builds a pipeline from parts: `RewriteQueryTransformer` / `CompressionQueryTransformer` (improve the query), `MultiQueryExpander`, `VectorStoreDocumentRetriever`, a `DocumentJoiner`, post-processors (e.g. reranking) and `ContextualQueryAugmenter`.

A production vector store (needs a database — not run)

`SimpleVectorStore` keeps everything in memory (`save` / `load` to a JSON file) — fine for tests and demos, not real data. With PostgreSQL + `pgvector`:

```
<dependency>
  <groupId>org.springframework.ai</groupId>
  <artifactId>spring-ai-starter-vector-store-pgvector</artifactId>
</dependency>
```

```
# (needs PostgreSQL with pgvector — not run)
spring.datasource.url=jdbc:postgresql://localhost:5432/docs
spring.ai.vectorstore.pgvector.initialize-schema=true
spring.ai.vectorstore.pgvector.index-type=hnsw
spring.ai.vectorstore.pgvector.distance-type=cosine-distance
spring.ai.vectorstore.pgvector.dimensions=1536
```

`dimensions` must match the embedding model (1536 for `text-embedding-3-small`); changing the model means **re-embedding every document**. `hnsw` (Hierarchical Navigable Small World) is an approximate nearest-neighbour index (AI 7.3); `initialize-schema=true` suits development, but production creates the table with Flyway (8.16).

Common mistakes

- No similarity threshold — irrelevant chunks confuse the model.
- Chunks too large (costly, diluted) or too small (no context) — start around a few hundred tokens with some overlap.
- Ignoring permissions — store tenant/role in metadata and filter.
- Mixing embeddings from two models in one store — vectors aren't comparable.

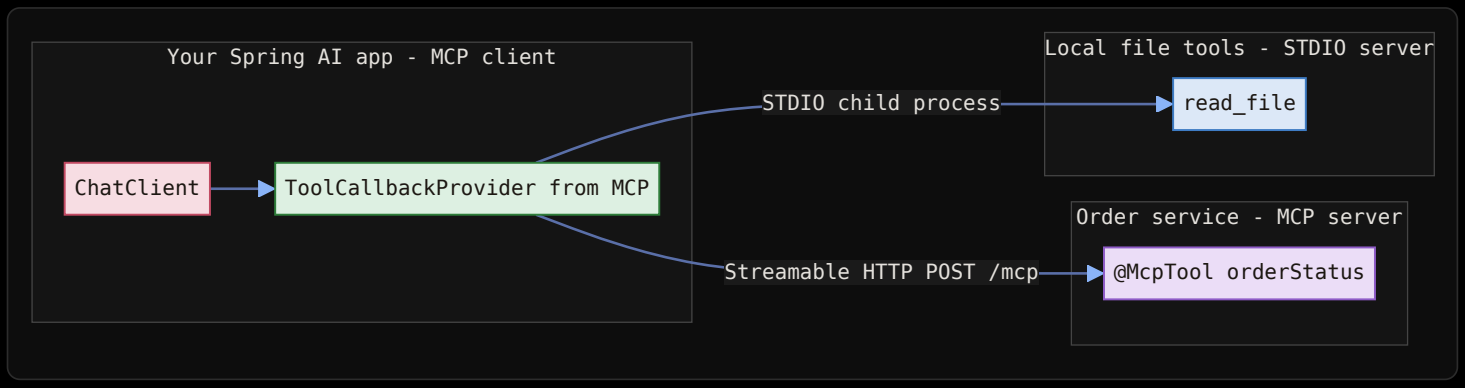
17.10 MCP: the Model Context Protocol

What it is

MCP is an open protocol (started by Anthropic in November 2024, now widely adopted) standardising how an AI application discovers and calls **tools**, reads **resources** and uses **prompts** exposed by another process — "USB for AI tools": write a tool server once, and any MCP host (Claude Desktop, IDE assistants, your Spring AI app) can use it. Concepts and security: AI 6.10, AI 9.8.

Role	What it does	Spring AI starter
MCP server	exposes tools/resources/prompts	`spring-ai-starter-mcp-server` (STDIO), `spring-ai-starter-mcp-server-webmvc`, `spring-ai-starter-mcp-server-webflux`
MCP client	connects to servers, turns their tools into `ToolCallback`s	`spring-ai-starter-mcp-client`, `spring-ai-starter-mcp-client-webflux`

Transports: **STDIO** (the host starts the server as a child process and they talk over its input/output streams — local tools) and **Streamable HTTP** (remote; the default for Spring AI 2.0's web servers, endpoint `POST /mcp`). The older SSE transport is deprecated since 2.0.



An MCP server (snippet — not run)

```
// spring-ai-starter-mcp-server-webmvc on the classpath
import org.springframework.ai.mcp.annotation.McpTool;
import org.springframework.ai.mcp.annotation.McpToolParam;
import org.springframework.stereotype.Component;

@Component
public class OrderMcpTools {

    @McpTool(name = "orderStatus", description = "Delivery status of an order by id")
    public OrderStatus orderStatus(@McpToolParam(description = "order id", required = true) String orderId) {
        return orderService.status(orderId);
    }
}
```

```
spring.ai.mcp.server.name=order-tools
# already the default in 2.0 (endpoint /mcp); shown for clarity
spring.ai.mcp.server.protocol=STREAMABLE
```

In Spring AI 2.0 the MCP annotations live in `org.springframework.ai.mcp.annotation` (from the `org.springaicommunity.mcp` community project in 1.1); beans with `@McpTool`, `@McpResource`, `@McpPrompt` methods are discovered automatically.

An MCP client (snippet — needs a running server, not run)

```
spring.ai.mcp.client.streamable-http.connections.orders.url=http://localhost:8081
# a local STDIO server, started by the client:
spring.ai.mcp.client.stdio.connections.files.command=npx
spring.ai.mcp.client.stdio.connections.files.args=-y,@modelcontextprotocol/server-filesystem,/data/docs
```

```
@Bean
ChatClient agent(ChatClient.Builder builder, ToolCallbackProvider mcpTools) {
    return builder.defaultToolCallbacks(mcpTools).build();   // MCP tools behave like @Tool methods
}
```

The client auto-configuration exposes remote tools as a `ToolCallbackProvider` bean (`spring.ai.mcp.client.toolcallback.enabled=true` by default), so §17.8's tool loop works unchanged.

Trade-offs

	In-process `@Tool`	MCP tool
latency	method call	network or process hop
reuse	this app only	any MCP host, any language
deployment	ships with the app	separate service, separate lifecycle
security	your normal code review	you now trust a remote server's tool descriptions and results — authenticate it (OAuth 2 for HTTP servers), allow-list tools, treat results as untrusted input

`@Tool` for logic belonging to your app; MCP to share tools across teams or AI hosts, or to consume existing servers.

17.11 Production concerns

Concern	What Spring AI gives you	Also see
observability	Micrometer observations: `spring.ai.chat.client`, `spring.ai.advisor`, `gen_ai.client.operation` (model call, with `gen_ai.client.token.usage` metrics), `spring.ai.tool`, `db.vector.client.operation`. Prompt/completion content is **not** recorded unless you enable `log-prompt` / `log-completion`	Chapter 15
cost	token usage in `chatResponse().getMetadata().getUsage()`	AI 6.6
retries and timeouts	provider SDK retries (e.g. `spring.ai.openai.chat.max-retries`, default 3) and `spring.ai.retry.*` for providers using Spring's retry; set HTTP timeouts	AI 6.7, 4.7
streaming	`.stream().content()` returns `Flux<String>`; send it to browsers with SSE	Chapter 13
blocking calls	model calls take seconds; on Spring MVC enable virtual threads (`spring.threads.virtual.enabled=true`) so waiting is cheap	Java 15.4
safety	redaction advisor, `SafeGuardAdvisor`, output validation, tool permissions	AI Chapter 12
testing	fake `ChatModel` for logic, evaluators for quality	§17.2

Common mistakes (Spring AI, whole part)

- Calling the model inside a database transaction (Chapter 7) — the transaction and connection stay open for seconds.
- No timeout or limit on tool loops and output tokens.
- Model text reaching SQL, shell commands or HTML without validation or escaping.
- Hard-coding a model id the provider later retires — keep it in configuration.
- Tests calling the real API on every build.

In the interview

Q: How would you add an AI assistant to an existing Spring Boot service? → Spring AI BOM + a model starter, key and model in properties, a `ChatClient` bean with a system prompt and advisors (memory with a per-user conversation id, RAG over our documents with metadata filters, logging/redaction), business functions as `@Tool` methods that check authorization themselves, typed results with `.entity(...)`. Test logic with a fake `ChatModel`, measure quality with an evaluation set, watch token usage and latency through Micrometer.

- *Why Spring AI rather than the provider's SDK?* → Portability across providers plus Spring integration: auto-configuration, observability, advisors, vector stores, testing. Provider-specific features remain reachable via native options.
- *Keeping answers grounded (based only on real sources)?* → RAG with a similarity threshold, an "answer only from the context" template, citations from retrieved-document metadata, faithfulness evaluation.

Spring AI TL;DR

- Spring AI **2.0.x** for Boot 4 (1.1.x for Boot 3.5). Import `spring-ai-bom`, add one model starter, set `spring.ai.<provider>.api-key` and `...chat.model`.
- `ChatClient` = fluent API over `ChatModel`; `.content()`, `.entity(...)`, `.stream()`.
- **Advisors** wrap every call (ordered, lower first): logging, memory, RAG, tool calling, your own (redaction).
- **Memory** re-sends history; pass `ChatMemory.CONVERSATION_ID` per call; window of 20 by default.
- **Structured output**: schema instructions + Jackson parsing; native mode and validation advisor for reliability.
- **Tools**: `@Tool` methods; the model requests, `ToolCallingAdvisor` executes and loops (limits 40/150). Authorize inside tools.
- **RAG**: reader → splitter → `EmbeddingModel` → `VectorStore`; `QuestionAnswerAdvisor` with threshold and metadata filters.
- **MCP**: share tools across processes; `@McpTool` servers, client tools arrive as a `ToolCallbackProvider`.

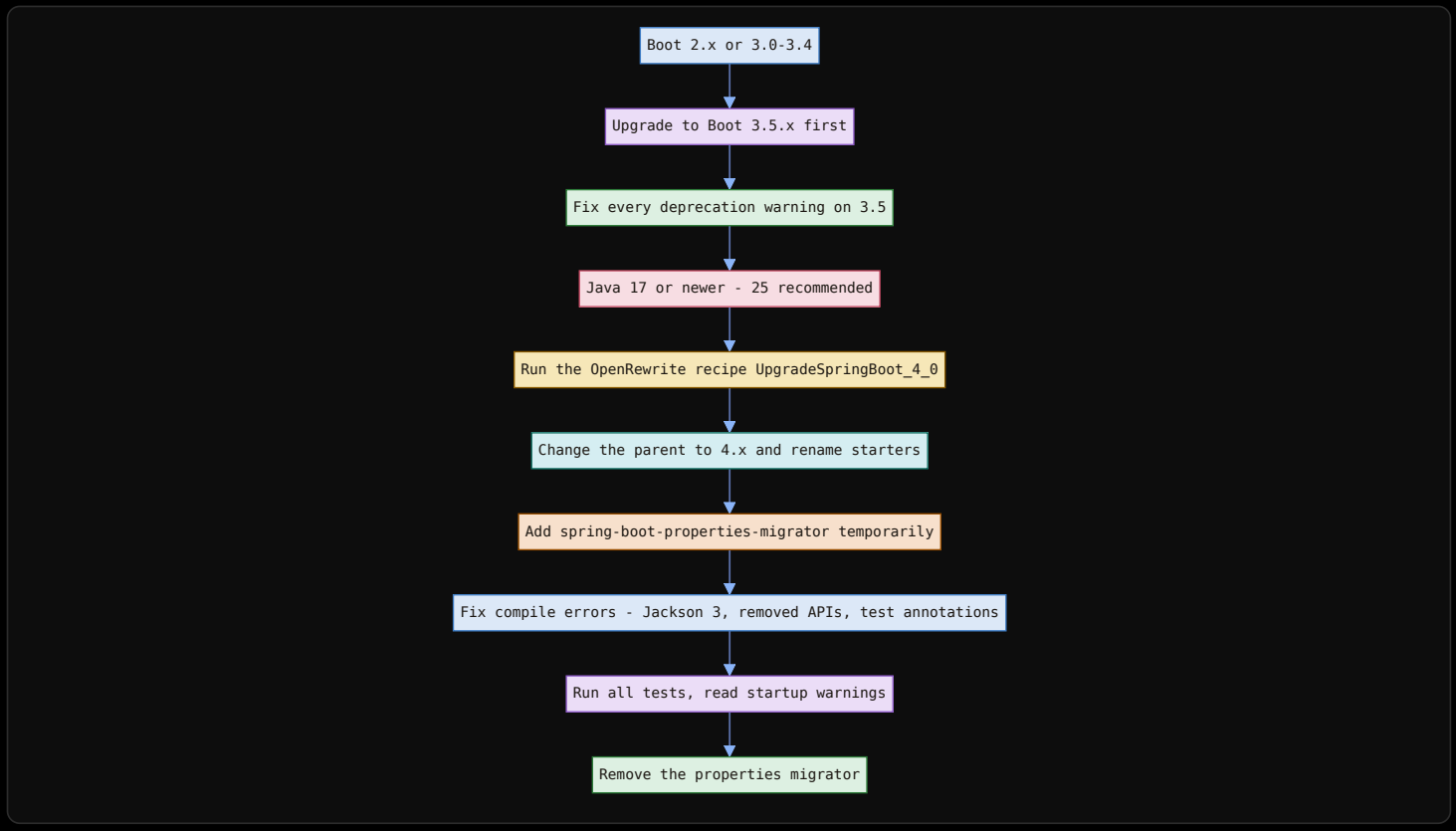
- Test with a **fake model**; evaluate quality separately.

17.12 What's new: how to read the rest of this chapter

The Spring generation of **November 2025** (Framework 7.0 on 13 Nov, Boot 4.0 on 20 Nov, Security 7.0) is the biggest change since Boot 3 moved `javax.*` to `jakarta.*` (Java's enterprise APIs were handed to the Eclipse Foundation as **Jakarta EE** and renamed). This book targets **Boot 4.1.1**, whose BOM gives Framework 7.0.9, Security 7.1.1, Hibernate ORM 7.4, Spring Data 2026.0, Jackson 3.1.5, Tomcat 11.0.24 and JUnit 6.0.3 (1.4).

The tables below are a **map**, one line per change with a pointer to the chapter that teaches it — checked against the Boot 4.0 migration guide, Boot 4.0/4.1 release notes, Framework 7.0 release notes and the Spring Security 7 reference.

A safe upgrade path (Boot 3.x → 4.x)



1. **Go to 3.5 first.** The migration guide assumes 3.5; code deprecated in 3.x is removed in 4.0.
2. **Automate the boring part** with OpenRewrite (rule-based automated source changes called *recipes*; not run here):

   ```
   mvn -U org.openrewrite.maven:rewrite-maven-plugin:run \
     -Drewrite.recipeArtifactCoordinates=org.openrewrite.recipe:rewrite-spring:RELEASE \
     -Drewrite.activeRecipes=org.openrewrite.java.spring.boot4.UpgradeSpringBoot_4_0
   ```

   It updates versions, renames starters and properties, and applies the Framework 7 / Security 7 sub-recipes. Review its diff like any change.
3. `spring-boot-properties-migrator` (temporary runtime dependency) reports renamed/removed properties at startup and maps old names while you fix them.
4. Stuck? The **"classic" starters** (`spring-boot-starter-classic`, `spring-boot-starter-test-classic`) restore the old all-in-one auto-configuration layout to get you compiling; then move to modular starters.

17.13 Spring Boot 3.x → 4.x migration table

Baselines

Item	Boot 3.x	Boot 4.x	Where in this book
Java	17+	17+ (25 recommended; this book uses 25)	1.4
Spring Framework	6.x	7.0	§17.14
Jakarta EE	10 (Servlet 6.0 — the Java web request/response API)	**11** (Servlet 6.1, JPA 3.2, Bean Validation 3.1)	1.4
Embedded servers	Tomcat 10.1, Jetty 12, Undertow	Tomcat 11, Jetty 12.1; **Undertow removed** (no Servlet 6.1 support)	Chapter 1
Hibernate ORM	6.x	7.x (7.4 in Boot 4.1)	Chapter 8
JUnit	5	**6** (JUnit 4 support in the TestContext framework deprecated)	Chapter 12
Kotlin / GraalVM	1.7+ / 22.3+	Kotlin 2.2+ / GraalVM 25+	Chapter 15
Gradle	7.x / 8.x	8.14+ or 9	1.6

Modularization and starters

Change	What to do	Where
`spring-boot-autoconfigure` split into **one module per technology** (`spring-boot-webmvc`, `spring-boot-jdbc`, `spring-boot-jackson`, ...) with its own starter and **test starter**	use the technology starters; imports of auto-configuration classes (for `exclude`) changed packages	3.14
`spring-boot-starter-web` → **`spring-boot-starter-webmvc`**	rename (the old name remains as a deprecated alias)	1.7
`spring-boot-starter-aop` → **`spring-boot-starter-aspectj`**	rename	4.2
`spring-boot-starter-oauth2-client` / `-resource-server` / `-authorization-server` → `spring-boot-starter-security-oauth2-...`	rename	Chapter 11
`spring-boot-starter-web-services` → `spring-boot-starter-webservices`	rename	—
test support per module: `spring-boot-starter-webmvc-test`, `-data-jpa-test`, `-restclient-test`, ...	add the test starter for each technology you test	Chapter 12
Flyway / Liquibase auto-configured only with **`spring-boot-starter-flyway`** / `-liquibase` (the library alone is no longer enough)	add the starter	8.16
`RestClient` / `RestTemplate` builders come from **`spring-boot-starter-restclient`**	add it for client code	Chapter 6
`@EntityScan` moved to `org.springframework.boot.persistence.autoconfigure`	fix the import	Chapter 8
`hibernate-jpamodelgen` → `hibernate-processor`	rename the annotation processor (compiler plug-in generating code from annotations)	Chapter 8
Spring Batch runs **without a database** by default	use `spring-boot-starter-batch-jdbc` for JDBC job metadata	—

Jackson 3

Change	What to do	Where
Jackson 3 is the default; packages and group ids moved from `com.fasterxml.jackson` to **`tools.jackson`** (annotations such as `@JsonProperty` keep `com.fasterxml.jackson.annotation`)	fix imports; `ObjectMapper` usage mostly becomes `JsonMapper`	5.5
`Jackson2ObjectMapperBuilderCustomizer` → `JsonMapperBuilderCustomizer`; `@JsonComponent` → `@JacksonComponent`; `@JsonMixin` → `@JacksonMixin`	rename	5.5
JSON-specific parser/generator features moved from `spring.jackson.parser.*` / `generator.*` to **`spring.jackson.json.read.*`** / **`spring.jackson.json.write.*`** (`spring.jackson.read.*` / `write.*` remain for format-neutral stream features); `spring.jackson.use-jackson2-defaults=true` restores Jackson 2 behaviour	update properties	5.5
Jackson 2 still possible through the **deprecated** `spring-boot-jackson2` module (`spring.jackson2.*`); Framework plans to disable its auto-detection in 7.1 and remove it in 7.2	migrate now	5.5
`HttpMessageConverters` bean type deprecated	use `ServerHttpMessageConvertersCustomizer` / `ClientHttpMessageConvertersCustomizer`	Chapter 5

Null-safety, testing, and other code changes

Change	What to do	Where
Boot and Framework annotated with **JSpecify** (`@NullMarked`, `@Nullable`); Spring's own `org.springframework.lang.Nullable` deprecated	use `org.jspecify.annotations` in your code; Kotlin and null checkers may flag new warnings	2.13
`@MockBean` / `@SpyBean` **removed**	`@MockitoBean` / `@MockitoSpyBean`	Chapter 12
`@SpringBootTest` no longer sets up MockMvc, `WebTestClient` or `TestRestTemplate` on its own	add `@AutoConfigureMockMvc`, `@AutoConfigureRestTestClient`, `@AutoConfigureTestRestTemplate`	Chapter 12
`TestRestTemplate` moved to `org.springframework.boot.resttestclient` (module `spring-boot-resttestclient`)	fix imports, or switch to the new **`RestTestClient`**	1.12
`EnvironmentPostProcessor` moved to `org.springframework.boot`; `BootstrapRegistry` to `org.springframework.boot.bootstrap`	update imports and `spring.factories`	Chapter 3
Spring Retry no longer version-managed	declare its version, or move to Framework 7 `@Retryable`	4.7
Spring Authorization Server moved into Spring Security 7 (`spring-authorization-server.version` property gone)	use `spring-security.version`	11.14

Property renames and changed defaults (the most common)

Before (3.x)	After (4.x)
`spring.jackson.parser.*`, `spring.jackson.generator.*`	`spring.jackson.json.read.*`, `spring.jackson.json.write.*`
`spring.data.mongodb.host` / `.port` / `.uri` / `.database` / ...	`spring.mongodb.host` / `.port` / `.uri` / `.database` / ...
`management.health.mongo.enabled`	`management.health.mongodb.enabled`
`server.error.*` (`include-stacktrace`, `include-message`, `path`, `whitelabel.enabled`, ...)	`spring.web.error.*` — the old keys are no longer bound since 4.0.0
`spring.session.redis.*`	`spring.session.data.redis.*` (Spring Session MongoDB left Boot, see below)
`spring.dao.exceptiontranslation.enabled`	`spring.persistence.exceptiontranslation.enabled`
`spring.kafka.retry.topic.backoff.random`	`spring.kafka.retry.topic.backoff.jitter`
DevTools LiveReload on by default	`spring.devtools.livereload.enabled` defaults to **`false`** (and LiveReload is deprecated in 4.1)
Kubernetes liveness/readiness probes only on Kubernetes	health **probes enabled by default** (`management.endpoint.health.probes.enabled`) — Chapter 15
`server.forward-headers-strategy` applied to war deployments	no longer applies to wars — register a `ForwardedHeaderFilter` yourself

Removed features

Removed in Boot 4.0	Replacement
Undertow	Tomcat or Jetty
executable "fully executable" jar launch scripts	run with `java -jar`, systemd, containers
Maven `CLASSIC` loader implementation	the default loader
optional dependencies packed into the uber (fat) jar	`<includeOptional>true</includeOptional>` if you really need them
Spock integration (Groovy 5 incompatibility)	**back in Boot 4.1** with Spock 2.4
Spring Session Hazelcast / MongoDB (moved to those vendors)	the vendors' own modules
Pulsar reactive support	Spring Pulsar imperative API

New in Boot 4.0

Feature	Where
API versioning auto-configuration (`spring.mvc.apiversion.*`, `spring.webflux.apiversion.*`)	5.9
HTTP service clients auto-configured: `@ImportHttpServices` + `spring.http.serviceclient.<group>.*`	6.6
`spring.http.clients.*` for the auto-configured HTTP client library and timeouts	6.4
`RestTestClient` in tests	1.12, Chapter 12
`spring-boot-starter-opentelemetry` (OTLP metrics and traces)	Chapter 15
`JmsClient` auto-configuration	Chapter 14
several `TaskDecorator` beans combined automatically	Chapter 14
`logging.console.enabled=false` to turn console logging off	Chapter 15
Kotlin serialization starter	—

17.14 Spring Framework 6 → 7

Area	What changed in 7.0	Where
baselines	Java 17 (25 recommended), Jakarta EE 11, Servlet 6.1, JPA 3.2, Bean Validation 3.1, Kotlin 2.2, GraalVM 25, Netty 4.2, JUnit 6	1.4
null-safety	whole codebase annotated with **JSpecify**, including generics, arrays and varargs; JSR 305-style Spring annotations deprecated	2.13
programmatic beans	new `BeanRegistrar` contract for registering beans with code (loops, conditions)	2.14
resilience	`RetryTemplate` / `RetryPolicy` in `spring-core` (`org.springframework.core.retry`), `@Retryable` and `@ConcurrencyLimit` in `spring-context`, enabled with **`@EnableResilientMethods`**; `@Retryable` also adapts to reactive return types	4.7
proxies	CGLIB (subclass-generating proxies) default applied consistently to all proxy processors (including `@Async`); **`@Proxyable`** overrides per bean	4.4
API versioning	first-class in Spring MVC and WebFlux: resolve the version from header, query parameter, path segment or media type; deprecation hints; client-side support in `RestClient`, `WebClient`, HTTP interfaces; testing support	5.9, 6.7
HTTP interface clients	group-based configuration with `@ImportHttpServices` and `HttpServiceGroupConfigurer`; `InputStream` / `OutputStream` support	6.6
`RestTemplate`	deprecated in the reference docs in 7.0, `@Deprecated` planned for 7.1, removal in 8.0	6.1
Jackson 3	default JSON library across MVC, WebFlux, messaging; Jackson 2 support deprecated	5.5
message converters	new `HttpMessageConverters` builder API (`configureMessageConverters(ServerBuilder)`)	Chapter 5
path matching	`PathMatcher` / `AntPathMatcher` for MVC mappings deprecated in favour of `PathPattern`, which now supports `/**` at the start of a pattern; suffix-pattern and trailing-slash matching options removed	5.3
`HttpHeaders`	no longer a `MultiValueMap`; some map methods removed	Chapter 5
data access	JPA 3.2: inject `EntityManagerFactory` / `EntityManager` with `@Autowired`; Hibernate 7 support moved to `orm.jpa.hibernate`; `JdbcClient` gains fetch size, max rows, query timeout	Chapter 7, Chapter 8
messaging	new **`JmsClient`** (like `JdbcClient` / `RestClient` for JMS)	Chapter 14
testing	**`RestTestClient`** (non-reactive `WebTestClient`), test application contexts **paused** when not in use, `@MockitoBean` on non-singleton beans, better `@Nested` injection	Chapter 12
SpEL (Spring Expression Language, `#{...}`)	`Optional` support with `?.` and Elvis (`?:`, default if null); since 7.0.8 a default limit of 10,000 operations per evaluation	—
Java 24+	bytecode metadata read with the JDK **Class-File API** (JEP 484) — transparent	Java Ch. 17
removed	`javax.annotation` / `javax.inject` support (use `jakarta.*`), `spring-jcl` (Apache Commons Logging 1.3 instead), `ListenableFuture` (use `CompletableFuture`), OkHttp3 support, `Theme` support, `webjars-locator-core` (use `webjars-locator-lite`), Undertow-specific classes	2.15
deprecated	JUnit 4 support (`SpringRunner`, ...), `<mvc:*>` XML namespace, document/feed views (PDF, XLS, RSS), BeanShell	—

In the interview

Q: What's new in Spring Framework 7 / Boot 4? → Jakarta EE 11 and Java 17 baseline (25 recommended); JSpecify null-safety; built-in API versioning; HTTP service client groups with Boot auto-configuration; core resilience (`@Retryable`, `@ConcurrencyLimit`, `RetryTemplate`); `BeanRegistrar`; Jackson 3; `RestTestClient`; and in Boot, modular auto-configuration with a starter and test starter per technology.

- *What breaks first on upgrade?* → Jackson imports (`tools.jackson`), renamed starters (web → webmvc, aop → aspectj), Flyway needing its starter (migrations silently stop), removed `@MockBean`, `@SpringBootTest` without `@AutoConfigureMockMvc` (no `MockMvc`), `javax.*` annotations, `.and()`-style security configuration.
- *Still need Spring Retry?* → Usually not: Framework 7's `@Retryable` covers retries with backoff and jitter; circuit breakers → Resilience4j (Chapter 16).

17.15 Spring Security 6 → 7

Change	Details	Where
lambda DSL only	`.and()` chaining removed; one lambda per feature	10.3
`authorizeRequests()` removed	use `authorizeHttpRequests()` (built on `AuthorizationManager`)	10.3
`AntPathRequestMatcher`, `MvcRequestMatcher` removed	**`PathPatternRequestMatcher`** (same syntax as Spring MVC)	10.3
`AccessDecisionManager` / voters	moved out of the main jar to the legacy `spring-security-access` module; use `AuthorizationManager`	10.6
multi-factor authentication	every login mechanism adds a factor authority (`FACTOR_PASSWORD`, `FACTOR_OTT`, `FACTOR_WEBAUTHN`); `@EnableMultiFactorAuthentication` or `AuthorizationManagerFactories.multiFactor()` require several factors	10.4
CSRF for single-page apps	`csrf(c -> c.spa())`	10.8
password encoders	Password4j-based encoders (Argon2, bcrypt, scrypt, PBKDF2, balloon hashing)	10.5
Spring Authorization Server	now shipped inside Spring Security 7	11.14
OAuth2 password grant	client support removed	11.2
PKCE	used by default for confidential clients too	11.9
JWT	`NimbusJwtEncoder.withKeyPair(...)` / `withSecretKey(...)` builders; `JwtTypeValidator` checks the `typ` header	11.3, 11.4
HTTP interfaces + OAuth2	`@ClientRegistrationId` on a whole HTTP interface	11.10
7.1 (in Boot 4.1)	`RestClientOpaqueTokenIntrospector`; `charset` parameter in `WWW-Authenticate: Basic`	11.7, 10.1

17.16 What Boot 4.1 adds (June 2026)

Change in 4.1	Notes	Where
everything **deprecated in 4.0 removed**	upgrade 4.0 → 4.1 with zero deprecation warnings first	—
`layertools` jar mode removed	use the `tools` jar mode for layered images	Chapter 15
gRPC (Google's binary RPC protocol) support (server and client, Netty or Servlet over HTTP/2)	new	—
Spock support back (Spock 2.4, Groovy 5)		Chapter 12
`spring.datasource.connection-fetch=lazy`	wraps the pool in `LazyConnectionDataSourceProxy`: no connection taken until the first statement	7.2
`@ConfigurationProperties` records bind `Optional` parameters as `Optional.empty()` instead of `null`		3.6
`spring.security.oauth2.resourceserver.jwt.authorities-claim-expressions`	map authorities from JWT claims with properties	11.5
observability: context propagation into `@Async` methods, `management.opentelemetry.*` properties, Kafka/RabbitMQ observation conventions		Chapter 15
HTTP clients: cookie handling (`spring.http.clients.cookie-handling`), `InetAddressFilter` against SSRF (server-side request forgery: tricking a server into calling internal addresses)		Chapter 6
`@RedisListener` auto-configuration		Chapter 9
dependency upgrades: Hibernate 7.4, Spring Data 2026.0, Micrometer 1.17		Chapter 8
deprecated: DevTools LiveReload, Derby support		—

Common migration mistakes

- Jumping from Boot 2.7 straight to 4.x — go 2.7 → 3.5 (the `javax` → `jakarta` step) → 4.x.

- Ignoring startup warnings — removed auto-configuration shows up as "bean not found", not a compile error.
- Keeping `spring-boot-properties-migrator` in production.
- Upgrading Boot but not separately versioned projects: **Spring Cloud** 2025.1 "Oakwood" (2025.1.2+ for Boot 4.1; Chapter 16), **Spring AI** 2.0, **springdoc-openapi**'s Boot 4 line (5.12).

17.17 In the interview — quick round

Question	30-second answer
What is Spring AI?	A Spring project giving portable APIs (`ChatModel`, `ChatClient`, `EmbeddingModel`, `VectorStore`) and Boot starters for AI providers, plus advisors for memory, RAG and tool calling. (§17.1)
`ChatModel` vs `ChatClient`?	Provider-level interface vs fluent API with templates, advisors, tools, structured output. (§17.3)
How does chat memory work?	The model is stateless; an advisor stores messages per conversation id and re-sends a window of them. (§17.6)
How do you get a Java object from an LLM?	`.entity(MyRecord.class)`: JSON Schema instructions + Jackson parsing; native structured output where supported. (§17.7)
Who executes a tool?	The application. The model only requests the call; `ToolCallingAdvisor` runs it and loops. (§17.8)
How do you test Spring AI code without paying?	A fake `ChatModel` bean with canned answers; real-model evaluations separately. (§17.2)
Main Boot 4 changes?	Modular starters, Jackson 3, JSpecify, Jakarta EE 11, API versioning, HTTP service clients, `@MockitoBean`, Undertow removed. (§17.13)
Main Framework 7 changes?	JSpecify, API versioning, `@ImportHttpServices`, `@Retryable` / `@ConcurrencyLimit`, `BeanRegistrar`, `RestTestClient`, Jackson 3. (§17.14)
Main Security 7 changes?	Lambda DSL only, `PathPatternRequestMatcher`, MFA factors, Authorization Server included, password grant removed. (§17.15)

Chapter TL;DR

- **Spring AI 2.0.x** is the version for Boot 4.x: BOM + one model starter + `spring.ai.<provider>.api-key` / `.chat.model` (e.g. `gpt-5-mini`, `claude-sonnet-5-5`, `claude-opus-5-5`). Set the model explicitly.
- `ChatClient` over `ChatModel`; prompts are messages (system, user, assistant, tool); `PromptTemplate` fills `{placeholders}` and fails on missing values.
- **Advisors** are the extension point: logging, memory (pass `ChatMemory.CONVERSATION_ID`), RAG (`QuestionAnswerAdvisor` + threshold + filters), tool calling (auto-registered in 2.0), your own (redaction). Order matters.
- **Structured output** with `.entity(...)`; **tools** with `@Tool` (authorize inside, limits 40/150); **MCP** to share tools across processes (`@McpTool`, Streamable HTTP).
- Everything above the model runs offline against a **fake** `ChatModel` — 14 tests in this chapter prove it.
- **Boot 3 → 4**: go via 3.5, run OpenRewrite, rename starters (webmvc, aspectj, security-oauth2-*), add test and Flyway starters, move to Jackson 3 (`tools.jackson`), `@MockitoBean`, JSpecify, Jakarta EE 11; Undertow gone.
- **Framework 7**: API versioning, HTTP service groups, core resilience, `BeanRegistrar`, `RestTestClient`, JSpecify; `RestTemplate` on its way out.
- **Security 7**: lambda DSL only, `authorizeHttpRequests`, `PathPatternRequestMatcher`, MFA, Authorization Server inside.

Next: Chapter 18 — Spring interview questions turns the whole book into questions and answers for revision.

Chapter 18 — Spring Interview Questions

This chapter is the *teaching* round of interview questions: **168 questions** about Spring Framework 7.0, Spring Boot 4.1 and Spring Security 7. The 30–60-second answers live in the Chapter 20 question bank; each question here points to its Chapter 20 answer (**Short answer: Ch 20 Qn**), adds only the deeper details that answer leaves out, and links to the teaching section. Questions without a Chapter 20 counterpart keep their full answer here. After the questions come runnable **proofs** of five tricky answers (§18.15, a small Maven project — Maven being Java's standard build tool), a template for "explain a Spring project you worked on" (§18.16), and ten **debugging scenarios** in the form *symptom → cause → fix → how to prove it* (§18.17).

Versions: Spring Boot 4.1.1 → Spring Framework 7.0, Spring Security 7.1, Spring Data 2026.0, Hibernate 7.4, Jackson 3.1, JUnit 6, Java 25 (17 minimum). Version-dependent answers name the version.

18.1 How to use this chapter

How to answer a Spring question

Interviewers rarely want a definition only. A strong answer has three layers, and you can stop after any of them:

1. **One sentence of what it is** — "`@Transactional` starts and ends a database transaction around a method." (A **transaction** is a group of database changes that either all succeed — *commit* — or are all undone — *rollback*.)
2. **How it works underneath** — "A proxy's interceptor asks the transaction manager to begin, calls the method, then commits or rolls back." (A **proxy** is a stand-in object Spring generates around your bean; callers talk to it, and it runs extra code — the *interceptor* — before and after passing the call on.)
3. **The consequence that bites people** — "So self-invocation skips it, and a checked exception commits by default." (**Self-invocation** is a method calling another method of the same object through `this`, which bypasses the proxy; a **checked exception** is one Java forces you to declare or catch — any `Exception` that is not a `RuntimeException`.)

The third layer shows you have debugged the feature, not just read about it — it is what makes you sound experienced.

How to study with it

- Read the question, say your answer out loud, check it against Chapter 20, then read the extra detail here. If you missed the third layer, follow the → **§N.M** link to the teaching section (a link without a section number points to the whole chapter).
- Questions marked **(Boot 4)** or **(Framework 7)** check that your knowledge is current.

18.2 Core container and dependency injection

Q1. What are Inversion of Control and Dependency Injection? Short answer: Ch 20 Q13. → §2.1

Q2. What is a bean? Short answer: Ch 20 Q16. A bean is described by a `BeanDefinition` (class or factory method, scope, dependencies, init/destroy callbacks) that the container turns into an instance. → §2.3

Q3. `BeanFactory` vs `ApplicationContext`? Short answer: Ch 20 Q15. → §2.2

Q4. `@Component` vs `@Bean`? Short answer: Ch 20 Q19. `@Repository` also enables exception translation to `DataAccessException`. → §2.3

Q5. What is the difference between "full" and "lite" `@Configuration`? Short answer: Ch 20 Q20. Boot's own auto-configurations use lite mode with dependencies as method parameters — faster and identical in native images. → §2.3

Q6. Constructor, setter or field injection — which and why? Short answer: Ch 20 Q17. → §2.4

Q7. Two beans implement the same interface. How does Spring choose? Short answer: Ch 20 Q22. To get all candidates inject `List<T>` (ordered by `@Order`) or `Map<String, T>` (keyed by bean name). Proved in §18.15. → §2.5

Q8. `@Primary` vs `@Qualifier`? Short answer: Ch 20 Q22. Use `@Primary` for "the usual one", `@Qualifier` (or a custom qualifier annotation) where a specific one is required. → §2.5

Q9. How do you inject an optional dependency? Short answer: Ch 20 Q25. → §2.5

Q10. What bean scopes exist, and what is the default? Short answer: Ch 20 Q26. Singleton is not thread-safe: one instance serves all request threads, so keep singletons stateless or their state thread-safe. → §2.6

Q11. You inject a prototype bean into a singleton. What happens? Short answer: Ch 20 Q29. "Prototype" means new per request *to the container*, not per use. Proved in §18.15. → §2.6

Q12. How can a singleton use a request-scoped bean? Short answer: Ch 20 Q30. `@RequestScope` = `@Scope(value = "request", proxyMode = TARGET_CLASS)`; outside a request thread the call fails (`ScopeNotActiveException`). → §2.6

Q13. Describe the bean lifecycle. Short answer: Ch 20 Q31. Destroy callbacks are not called for prototypes. For start/stop behaviour tied to the context (consumers, schedulers) implement `SmartLifecycle`. → §2.7

Q14. `BeanPostProcessor` vs `BeanFactoryPostProcessor`? Short answer: Ch 20 Q33. Examples: `PropertySourcesPlaceholderConfigurer` resolves `${...}` (BFPP); `AutowiredAnnotationBeanPostProcessor` does injection and the auto-proxy creator wraps beans for `@Transactional` (BPP). → §2.8

Q15. How does Spring handle circular dependencies? Short answer: Ch 20 Q34. Proved in §18.15. → §2.9

Q16. What does lazy initialization do, and what does it cost? Short answer: Ch 20 Q35. Good for development and some CLI tools; risky for services. → §2.10

Q17. `@Profile` vs `@Conditional`? Short answer: Ch 20 Q54. Boot's `@ConditionalOnClass`, `@ConditionalOnProperty`, `@ConditionalOnMissingBean` are built on `@Conditional`. → §2.11

Q18. How do application events work? Are they asynchronous? Short answer: Ch 20 Q37. → §2.12

Q19. (Framework 7) What is JSpecify null-safety in Spring? Short answer: Ch 20 Q39. JSpecify replaces the deprecated `org.springframework.lang` annotations; Kotlin reads them too. → §2.13

Q20. (Framework 7) What is a `BeanRegistrar`? Short answer: Ch 20 Q39. Example: one bean per configured tenant; friendlier to AOT/native images than a `BeanDefinitionRegistryPostProcessor`. → §2.14

18.3 Configuration and auto-configuration

Q21. What does `@SpringBootApplication` do? Short answer: Ch 20 Q40. Hence the main class belongs in the root package: beans in sibling packages are not scanned. → §1.8

Q22. What happens when you run `java -jar app.jar`? Short answer: Ch 20 Q5. Detail: the jar's manifest (`META-INF/MANIFEST.MF`) starts Boot's `JarLauncher`, which builds a class loader over `BOOT-INF/classes` and `BOOT-INF/lib`. → §1.9

Q23. What is a starter? Short answer: Ch 20 Q46. → §1.7

Q24. How does auto-configuration work? Short answer: Ch 20 Q41. `@EnableAutoConfiguration` filters the `.imports` list early by class presence and sorts it (`before` / `after`); a `DataSource` appears only if a driver is present and you defined none. → §3.10

Q25. How do you override an auto-configured bean? Short answer: Ch 20 Q44. Often you needn't replace the bean: set properties or declare a customizer bean (`JsonMapperBuilderCustomizer`, `RestClientCustomizer`, …). → §3.12

Q26. How do you find out why a bean was (or was not) auto-configured? Short answer: Ch 20 Q43. → §3.11

Q27. Name the main property sources in priority order. Short answer: Ch 20 Q49. Full order also includes test properties (in tests, highest), `SPRING_APPLICATION_JSON`, `random.*`. Rule of thumb: the more "outside" and "later", the stronger. → §3.3

Q28. How does an environment variable map to a property? Short answer: Ch 20 Q50. Example: `SHOP_CHECKOUTTIMEOUT` sets `shop.checkout-timeout`. → §3.4

Q29. `@Value` vs `@ConfigurationProperties`? Short answer: Ch 20 Q51. → §3.6

Q30. How do you make the app refuse to start with bad configuration? Short answer: Ch 20 Q52. Use `@Valid` for nested groups; the failure report names each property and where it came from. → §3.6

Q31. What are profiles and profile groups? Short answer: Ch 20 Q53. Keep profiles for *environments or capabilities*, e.g. `spring.profiles.group.prod=postgres,kafka,json-logs`. → §3.7

Q32. How do you load Kubernetes secrets or extra config files? Short answer: Ch 20 Q55. `spring.config.import=optional:configtree:/etc/secrets/` reads each mounted file as a property (file name = key); `spring.config.import` also imports other files, Config Server or Vault; `optional:` avoids failure when absent. → §3.8

Q33. How do you write your own starter? Short answer: Ch 20 Q47. Two artifacts: `acme-spring-boot` (auto-configuration with `@ConditionalOnMissingBean` on every bean) and `acme-spring-boot-starter`, a code-free pom depending on it and the library. → §3.13

Q34. (Boot 4) What does "Boot 4 modularization" mean? Boot 3 kept all auto-configuration in one big `spring-boot-autoconfigure` jar. Boot 4 splits it into per-technology modules (`spring-boot-webmvc`, `spring-boot-jdbc`, `spring-boot-kafka` …) with matching starters and test starters, so the classpath (the list of jars and folders the JVM loads classes from) contains only what you use. `spring-boot-starter-classic` / `-test-classic` bring the old "everything" layout as a migration aid. → §3.14

Q35. What does DevTools do, and is it in production? Short answer: Ch 20 Q11. → §1.10

18.4 AOP and proxies

Q36. What is AOP? Define aspect, advice, pointcut and join point. Short answer: Ch 20 Q59. → §4.1

Q37. How does Spring AOP work at runtime? Short answer: Ch 20 Q60. Nothing changes in your bytecode — the proxy runs the advice chain, then calls the real object. → §4.4

Q38. JDK dynamic proxy vs CGLIB? Short answer: Ch 20 Q61. Framework 7 adds `@Proxyable` to choose per bean. → §4.4

Q39. What is the self-invocation problem? Short answer: Ch 20 Q62. Proved in §18.15. → §4.5

Q40. Which Spring features are built on proxies? Short answer: Ch 20 Q63. Also HTTP interface clients (JDK proxies). All share the same limitations. → §4.6

Q41. `@Around` vs the other advice types? Short answer: Ch 20 Q65. → §4.2

Q42. How do you control the order of two aspects? Short answer: Ch 20 Q65. The lowest `@Order` value is the *outer* layer — first in, last out. Example: put a retry aspect outside the transaction aspect so each attempt gets a fresh transaction. → §4.2

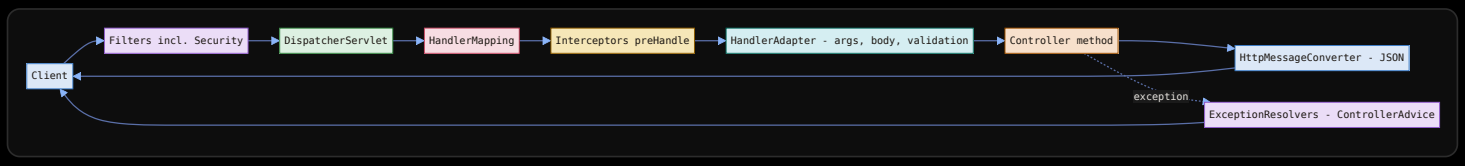
Q43. Spring AOP vs AspectJ? Short answer: Ch 20 Q66. → §4.8

Q44. (Framework 7) What are `@Retryable` and `@ConcurrencyLimit`? Short answer: Ch 20 Q67. Also `include` / `exclude` exception filters; no auto-configuration (needs `@EnableResilientMethods`); programmatic API `RetryTemplate` / `RetryPolicy` in `spring-core`; no circuit

breaker — use Resilience4j. They live in `spring-context`. → §4.7

18.5 Spring MVC, REST and HTTP clients

Q45. Walk through a request in Spring MVC. Short answer: Ch 20 Q68. MVC flow (also: `HandlerMapping` matches consumes/produces and API version; the response converter is chosen from the `Accept` header): → §5.2



Q46. `@Controller` vs `@RestController`? Short answer: Ch 20 Q69. → §5.3

Q47. `@PathVariable` vs `@RequestParam` vs `@RequestBody`? Short answer: Ch 20 Q71. → §5.4

Q48. Why return `ResponseEntity`? Short answer: Ch 20 Q72. Also `304` with an `ETag` (a response-version fingerprint letting clients skip unchanged data). For plain 200 just return the object. → §5.4

Q49. How do you validate a request? Short answer: Ch 20 Q90. Use `@Validated(Group.class)` for validation groups. → §5.6

Q50. How do you handle errors consistently? Short answer: Ch 20 Q94. Never leak stack traces to clients. → §5.7

Q51. (Boot 4) What changed with JSON? Short answer: Ch 20 Q76. Jackson 3 also means an immutable `JsonMapper` built with a builder and unchecked exceptions; `@JsonComponent` became `@JacksonComponent`. → §5.5

Q52. How does content negotiation work? Short answer: Ch 20 Q77. → §5.8

Q53. (Framework 7) How do you version a REST API? Short answer: Ch 20 Q78. Ranges like `version = "1.1+"` ("1.1 and later"); a deprecation handler can add `Deprecation`, `Sunset` and `Link` headers. → §5.9

Q54. What is CORS and where do you configure it? Short answer: Ch 20 Q80. An *origin* is scheme + host + port. CORS doesn't protect the API from non-browser clients. → §5.10, §10.9

Q55. Filter vs interceptor vs aspect? Short answer: Ch 20 Q81. A filter sees even requests that never reach Spring MVC and can wrap the request/response. → §5.11

Q56. PUT vs PATCH vs POST — which are idempotent? Short answer: Ch 20 Q73. For retry-safe POSTs accept an `Idempotency-Key` header. → §5.1

Q57. How do you paginate and document an API? Short answer: Ch 20 Q79. Document with OpenAPI (a machine-readable REST API description) generated by springdoc (`/v3/api-docs`, Swagger UI). → §5.13, §5.12

Q58. Which HTTP client should a new Boot 4 service use? Short answer: Ch 20 Q86. Inject `RestClient.Builder` to get Boot's converters, timeouts and observability. → §6.1

Q59. What are HTTP service interfaces, and what does Boot 4 add? Short answer: Ch 20 Q87. `HttpServiceProxyFactory` creates the JDK-proxy implementation; Boot 4's `@ImportHttpServices(group = "catalog", types = CatalogApi.class)` removes the factory code. → §6.6

Q60. How do you handle errors, timeouts and retries on a client? Short answer: Ch 20 Q88. `retrieve()` throws `HttpClientErrorException` (4xx) / `HttpServerErrorException` (5xx), and `ResourceAccessException` for I/O errors and timeouts; translate them to domain exceptions at the client edge. → §6.3, §6.4

Q61. How do you test a REST client? Short answer: Ch 20 Q181. → §6.8

18.6 Data access, JPA and transactions

Q62. What does Spring add on top of plain JDBC? (JDBC — Java Database Connectivity — is Java's standard low-level API for talking to SQL databases.) A pooled `DataSource` (a *connection pool* keeps a few open database connections and lends them out, because opening a new one per query is slow; HikariCP is Boot's pool), `JdbcClient` / `JdbcTemplate` that handle connections, statements, result-set mapping and cleanup, translation of vendor (database-specific) `SQLException`s into the unchecked `DataAccessException` hierarchy (`DuplicateKeyException`, `DataIntegrityViolationException`…), and declarative transactions. → §7.1, §7.5

Q63. `JdbcClient` vs `JdbcTemplate`? Short answer: Ch 20 Q125. `JdbcTemplate` remains for batch updates and existing code. → §7.3

Q64. How big should the connection pool be? Short answer: Ch 20 Q124. Pool × instances must stay under the database's `max_connections`; watch `hikaricp.connections.pending`. → §7.2

Q65. How does `@Transactional` work internally? Short answer: Ch 20 Q127. The manager turns auto-commit off on the bound connection; repositories/`JdbcClient` on that thread reuse it. → §7.7

Q66. What are the default propagation and isolation? Short answer: Ch 20 Q130. Defaults: propagation `REQUIRED`; isolation `DEFAULT` = the database's (READ COMMITTED on PostgreSQL, REPEATABLE READ on MySQL InnoDB). → §7.8

Q67. A `@Transactional` method throws a checked exception. Commit or rollback? Short answer: Ch 20 Q131. Rationale: the EJB convention treats checked exceptions as expected business outcomes. Since Framework 6.2, globally `@EnableTransactionManagement(rollbackOn = RollbackOn.ALL_EXCEPTIONS)` (recommended by the reference docs). Proved in §18.15. → §7.10

Q68. Explain the propagation types. Short answer: Ch 20 Q128. → §7.9

Propagation	Existing transaction?	None?
`REQUIRED` (default)	join it	start one
`REQUIRES_NEW`	suspend it, start a new one (new connection)	start one
`NESTED`	savepoint inside it	start one
`SUPPORTS`	join it	run without
`NOT_SUPPORTED`	suspend it, run without	run without
`MANDATORY`	join it	throw
`NEVER`	throw	run without

Typical uses: `REQUIRES_NEW` for an audit record that must survive a rollback; `NESTED` for "roll back only this part" — avoid with JPA (the database rolls back to the savepoint, Hibernate's persistence context doesn't).

Q69. What is `UnexpectedRollbackException`? Short answer: Ch 20 Q133. → §7.9

Q70. What does `readOnly = true` do? Short answer: Ch 20 Q134. PostgreSQL then rejects writes on the read-only connection. It is not a security feature. → §7.12

Q71. Why might `@Transactional` be silently ignored? Short answer: Ch 20 Q132. → §7.13

Q72. On which layer does `@Transactional` belong? Short answer: Ch 20 Q135. Keep remote HTTP calls out of transactions — they hold a connection while waiting. → §7.13

Q73. When do you use `TransactionTemplate`? Short answer: Ch 20 Q138. Also to avoid proxy pitfalls such as retries around a transaction. → §7.14

Q74. How do you run code only after the transaction commits? Short answer: Ch 20 Q136. `@TransactionalEventListener` (default `AFTER_COMMIT`), e.g. send the confirmation e-mail only if the order was really saved. Not guaranteed delivery — if the process dies after commit the event is lost; use a transactional outbox when delivery must be guaranteed. → §7.15

Q75. How do you handle a serialization failure or deadlock? Short answer: Ch 20 Q140. They surface as `ConcurrencyFailureException` subtypes (`CannotAcquireLockException`, `PessimisticLockingFailureException`). See also SQL book 13. → §7.11, SQL book 13

Q76. JPA vs Hibernate vs Spring Data JPA? Short answer: Ch 20 Q98. JPA version: Jakarta Persistence 3.2. → §8.1

Q77. What is the persistence context? What is dirty checking? Short answer: Ch 20 Q105. Flush happens before commit, or before a query that needs it. → §8.4

Q78. What are the entity states? Short answer: Ch 20 Q106. → §8.4

Q79. How does `save()` decide between insert and update? Short answer: Ch 20 Q108. Implemented in `SimpleJpaRepository.save()`. → §8.8

Q80. What is the owning side of a relationship? Short answer: Ch 20 Q111. → §8.5

Q81. `CascadeType.REMOVE` vs `orphanRemoval = true`? Short answer: Ch 20 Q112. Use both only for true composition (order → order lines), never on `@ManyToMany`. → §8.5

Q82. What are the default fetch types, and what causes `LazyInitializationException`? Short answer: Ch 20 Q103, Q104. → §8.6

Q83. What is the N+1 problem and how do you fix it? Short answer: Ch 20 Q102. Batch fetching cuts it to 1 + N/size round trips. Detect it by counting statements in tests (Hibernate statistics or a datasource proxy). `@BatchSize` sets batch fetching per association. → §8.7

Q84. Derived query vs `@Query` vs native query? Short answer: Ch 20 Q101. Updates need `@Modifying` (plus `clearAutomatically = true` if managed entities may be stale). → §8.9, §8.10

Q85. `Page` vs `Slice`? Short answer: Ch 20 Q114. Never combine `JOIN FETCH` of a collection with paging without checking the SQL (Hibernate may page in memory). → §8.11

Q86. What are projections? Short answer: Ch 20 Q113. → §8.12

Q87. How does optimistic locking work? Short answer: Ch 20 Q116. → §8.14

Q88. What is Open Session in View and should you disable it? Short answer: Ch 20 Q117. Default `true`, with a startup warning. → §8.15

Q89. How is the schema managed in production? Short answer: Ch 20 Q118. In Boot 4 add `spring-boot-starter-flyway` — `flyway-core` alone is not auto-configured. → §8.16

Q90. How should an entity implement `equals`/`hashCode`? Short answer: Ch 20 Q119. → §8.3

Q91. Spring Data JDBC vs Spring Data JPA? Short answer: Ch 20 Q125. Spring Data JDBC maps DDD aggregates (clusters always loaded and saved as one unit): no persistence context, lazy loading or dirty checking — `save()` writes the whole aggregate. Simpler and predictable vs JPA's caching, lazy loading and rich mapping with more hidden behaviour. → §9.2

18.7 Caching

Q92. How does Spring's cache abstraction work? Short answer: Ch 20 Q142. → §9.8

Q93. How is the cache key built? Short answer: Ch 20 Q143. Default generator: `SimpleKeyGenerator`. → §9.8

Q94. `condition` vs `unless`? Short answer: Ch 20 Q143. → §9.8

Q95. Why doesn't my `@Cacheable` work? Short answer: Ch 20 Q144. → §4.6, §9.8

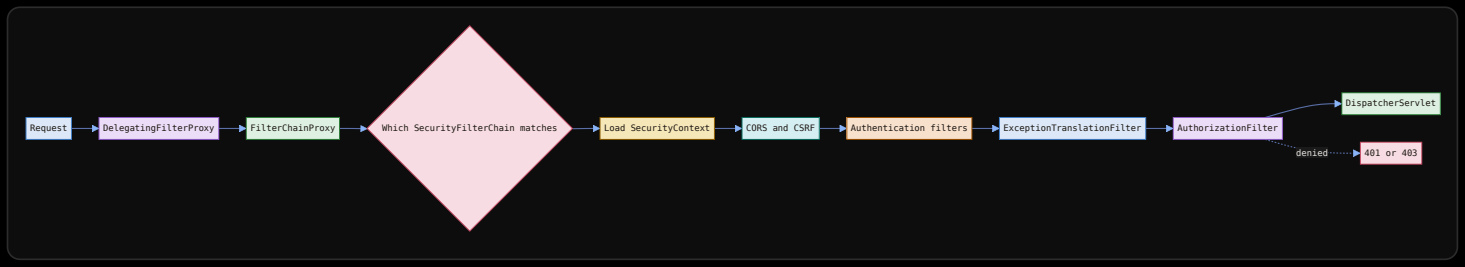
Q96. Local cache (Caffeine) vs distributed cache (Redis)? Short answer: Ch 20 Q145. Common choice: Caffeine for small, rarely changing reference data with a short TTL (time to live); Redis for shared session-like or expensive data. → §9.8

Q97. What cache problems should you plan for? Short answer: Ch 20 Q146. `sync = true` works only per instance and can't be combined with `unless`. Also: caching mutable objects callers then modify, per-user data under a shared key, and not measuring the hit rate (Micrometer `cache.gets`; Caffeine only with `recordStats()`). → §9.8

18.8 Security

Q98. What happens when you add `spring-boot-starter-security` and nothing else? Short answer: Ch 20 Q148. → §10.1

Q99. Explain the filter chain architecture. Short answer: Ch 20 Q149. Chain order also includes security-context loading, headers and anonymous authentication. Several chains (e.g. stateless `/api/**` with JWT, the rest with form login) are ordered with `@Order`. → §10.2



Q100. (Security 7) How do you configure a chain today? Short answer: Ch 20 Q150. Security 7 also removed the Ant/MVC request matchers (paths use `PathPatternRequestMatcher`). → §10.3

Q101. How does authentication work internally? Short answer: Ch 20 Q152. → §10.4

Q102. How should passwords be stored? Short answer: Ch 20 Q154. Never encrypt passwords reversibly. Implement `UserDetailsPasswordService` to re-hash on login. → §10.5

Q103. Role vs authority? `hasRole` vs `hasAuthority`? Short answer: Ch 20 Q155. `hasAuthority("ROLE_ADMIN")` = `hasRole("ADMIN")` spelled out. Mixing them up (`hasRole("ROLE_ADMIN")`, or granting `ADMIN` without prefix) is a classic 403. → §10.6

Q104. 401 vs 403? Short answer: Ch 20 Q156. `ExceptionTranslationFilter` decides: denied + anonymous → `AuthenticationEntryPoint` (401 with `WWW-Authenticate`, or a login redirect for browsers); denied + authenticated → `AccessDeniedHandler` (403). → §10.6

Q105. What is method security? Short answer: Ch 20 Q157. Also `@PreFilter`/`@PostFilter`; lets you express ownership rules (`returnObject.owner == authentication.name`) that URL patterns can't. → §10.7

Q106. What is CSRF and when can you disable protection? Short answer: Ch 20 Q158. → §10.8

Q107. Sessions or tokens? Short answer: Ch 20 Q162. Scaled sessions need shared storage (Spring Session with Redis/JDBC). The session id lives in the `JSESSIONID` cookie. → §10.10, §11.1

Q108. How does a Spring resource server validate a JWT? Short answer: Ch 20 Q160. Detail: the `JwtDecoder` (Nimbus) fetches the issuer's JWK Set, picks the key by `kid`, verifies the signature; validators check `exp`/`nbf` (60 s clock skew), `iss` and, if configured, `aud`; `scope` → `SCOPE_*` authorities. Invalid token → 401; missing scope → 403. → §11.3, §11.5

Q109. OAuth 2.0 vs OpenID Connect? Short answer: Ch 20 Q164. → §11.2

Q110. JWT vs opaque tokens? Short answer: Ch 20 Q159, Q166. → §11.7

Q111. How does one service call another securely? Short answer: Ch 20 Q167. Mechanics: `OAuth2AuthorizedClientManager` obtains and caches the service's token. → §11.10

Q112. Where should a browser SPA keep tokens? Short answer: Ch 20 Q168. Tokens in `localStorage` (readable by any script on the page) are stolen by any XSS. → §11.12

Q113. Is the security context available in `@Async` methods or new threads? Short answer: Ch 20 Q153. Wrap the executor in `DelegatingSecurityContextAsyncTaskExecutor` (or use `DelegatingSecurityContextRunnable`). In WebFlux the context lives in the Reactor `Context` (`ReactiveSecurityContextHolder`). → §10.10

Q114. (Security 7) What is new? Short answer: Ch 20 Q172. Also built-in MFA via `FACTOR_*` authorities and Password4j-based encoders. → §10.14, §11.14

18.9 Testing

Q115. How do you structure tests in a Spring Boot app? Short answer: Ch 20 Q173. Other slices: `@JsonTest`, `@RestClientTest`. → §12.2

Q116. `@SpringBootTest` vs `@WebMvcTest`? Short answer: Ch 20 Q174. → §1.12, §12.6, §12.7

Q117. (Boot 4) `@MockBean` doesn't compile any more. Why? Short answer: Ch 20 Q175. → §12.10

Q118. `MockMvcTester`, `RestTestClient`, `WebTestClient` — which when? Short answer: Ch 20 Q176. Boot 4 specifics: `MockMvcTester` in `@SpringBootTest` needs `@AutoConfigureMockMvc`; `RestTestClient` (MockMvc or a running server, same test code) needs `@AutoConfigureRestTestClient` even in `@WebMvcTest`; `WebTestClient` is automatic in `@WebFluxTest`, `@AutoConfigureWebTestClient` in `@SpringBootTest`; `TestRestTemplate` needs `@AutoConfigureTestRestTemplate`. → §12.8, §12.9

Q119. What is test context caching, and what breaks it? Short answer: Ch 20 Q177. Cache key = configuration classes, profiles, properties, context customizers. At most 32 contexts (LRU eviction). Framework 7 **pauses** an inactive context by default (its `Lifecycle`/`SmartLifecycle` beans — schedulers, listener containers — stopped) and restarts it on reuse. → §12.11

Q120. Does `@Transactional` on a test roll back? Short answer: Ch 20 Q178. `@DataJpaTest` is transactional out of the box. → §8.17, §12.6

Q121. What does `@DataJpaTest` configure? Short answer: Ch 20 Q174. Embedded DB by default `Replace.NON_TEST` (a Testcontainers `@ServiceConnection` database is kept; `replace = NONE` always keeps yours); `TestEntityManager`; no web layer or services. → §8.17, §12.7

Q122. How do you test against a real database or broker? Short answer: Ch 20 Q179. `@ServiceConnection` dates from Boot 3.1. → §12.14

Q123. How do you test secured endpoints? Short answer: Ch 20 Q180. → §10.12, §11.6

Q124. How do you test a custom auto-configuration or conditional beans quickly? Short answer: Ch 20 Q183. The proofs in §18.15 use it. `WebApplicationContextRunner` for web conditions; assertions like `hasSingleBean(X.class)` or `hasFailed()`. → §3.13

Q125. (Framework 7) What changed for testing? JUnit 6 (Jupiter 6 — the JUnit programming model — with a Java 17 baseline, i.e. minimum) via the Boot test starters; JUnit 4 support (`SpringRunner`) deprecated; `RestTestClient`; context pausing; `@MockitoBean` on non-singleton beans; `SpringExtension` uses a test-method-scoped extension context by default, which makes `@Nested` classes behave consistently. Boot 4 adds a test starter per technology (`spring-boot-starter-webmvc-test`, `-data-jpa-test`, …), moves the slice annotations to per-module packages, removes `@MockBean`, and requires explicit `@AutoConfigure…` annotations for test clients in `@SpringBootTest`. → §12.3, §12.17, §17.14

18.10 Reactive and WebFlux

Q126. What is reactive programming, and what is Reactive Streams? Short answer: Ch 20 Q185. → §13.2

Q127. `Mono` vs `Flux`? Short answer: Ch 20 Q186. → §13.3

Q128. `map` vs `flatMap` (vs `concatMap`)? Short answer: Ch 20 Q187. `flatMapSequential` runs concurrently but re-orders the output to the source order. → §13.4

Q129. What happens if you block inside a WebFlux handler? Short answer: Ch 20 Q188. Reactor refuses `block()` on its non-blocking threads (`IllegalStateException`). Measured in §13.15: a 100 ms `Thread.sleep` cut WebFlux from ~8,000 to ~100 req/s. → §13.8, §13.15

Q130. `publishOn` vs `subscribeOn`? Short answer: Ch 20 Q189. → §13.8

Q131. WebFlux or Spring MVC with virtual threads? Short answer: Ch 20 Q184. Measured in §13.15 (1,000 concurrent clients, 100 ms I/O endpoint): MVC on virtual threads and WebFlux both ~8,000 req/s vs ~1,900 on 200 platform threads. Choose WebFlux for streaming (SSE, WebSockets with backpressure), a fully non-blocking stack or heavy fan-out; MVC + virtual threads for typical CRUD with JDBC/JPA. Either way the database pool is the real limit. See also Java 15.4. → §13.15, Java 15.4

Q132. How do you test reactive code, and how do transactions and security work there? Short answer: Ch 20 Q186. Reactive transactions (`ReactiveTransactionManager`, R2DBC) and the security context (`ReactiveSecurityContextHolder`) live in the Reactor `Context`, not `ThreadLocal`s. JPA/JDBC are blocking — use R2DBC. `@DataR2dbcTest` does **not** roll back like `@DataJpaTest`. → §13.5, §13.14

18.11 Messaging, async and scheduling

Q133. How does `@Async` work, and what are its pitfalls? Short answer: Ch 20 Q192. Boot's `applicationTaskExecutor` is a `ThreadPoolTaskExecutor` tuned by `spring.task.execution.*` (virtual threads when enabled). → §14.4, §14.6

Q134. `fixedRate` vs `fixedDelay` vs `cron` in `@Scheduled`? Short answer: Ch 20 Q194. → §14.7

Q135. A `@Scheduled` job runs on every instance. How do you run it once? Short answer: Ch 20 Q195. Leader election and Quartz in clustered mode are further options. → §14.7

Q136. How do you produce and consume Kafka messages with Spring? Short answer: Ch 20 Q196. Kafka orders only **within a partition**; consumers in one group share the partitions. `send` returns a `CompletableFuture<SendResult>`. → §14.11, §14.12

Q137. What delivery guarantee does a Spring Kafka listener give, and how do you handle duplicates? Short answer: Ch 20 Q197. spring-kafka turns off Kafka's auto-commit and commits offsets after the listener succeeds (ack mode `BATCH`: after the polled batch). Producer idempotence (default in modern clients) prevents duplicates from producer retries — not from consumer redelivery. → §14.8, §14.16

Q138. How do you handle a message that keeps failing? Short answer: Ch 20 Q198. DLT default name is `<topic>-dlt` since spring-kafka 3.3 (older articles show `<topic>.DLT`), same partition number; use `ErrorHandlingDeserializer` for deserialization failures. → §14.14, §14.15

Q139. How do you update the database and publish an event reliably? Short answer: Ch 20 Q199. → §14.17, event-driven notes

Q140. Kafka vs RabbitMQ? Short answer: Ch 20 Q200. Spring AMQP specifics: blocking listener retries are `spring.rabbitmq.listener.simple.retry.*` (Boot 4 replaced `max-attempts`, which counted the first attempt, with `max-retries`); rejected messages go to the queue's dead-letter exchange when `default-requeue-rejected=false`. → §14.18

18.12 Production and observability

Q141. What is Spring Boot Actuator, and how do you expose it safely? Short answer: Ch 20 Q202. `env`/`configprops` values are masked by default. → §15.2

Q142. Liveness vs readiness probes? Short answer: Ch 20 Q204. → §15.3

Q143. What is Micrometer? Name the meter types. Short answer: Ch 20 Q205. Keep **tag cardinality** low — never tag with user id or raw URL. `Timer` can add percentiles/histograms (e.g. p99). Code records to a `MeterRegistry`; registries export to the backends. → §15.5

Q144. What is the Observation API? Short answer: Ch 20 Q207. → §15.6

Q145. How does distributed tracing work in Boot? Short answer: Ch 20 Q206. Default sampling probability 10% (`management.tracing.sampling.probability`); context also travels in Kafka headers; Boot 4 adds `spring-boot-starter-opentelemetry` for OTLP export (Brave/Zipkin: `spring-boot-starter-zipkin`). → §15.7

Q146. What is structured logging in Boot? Since Boot 3.4, `logging.structured.format.console=ecs` (or `logstash`, `gelf`) writes each log event as one JSON object — machine-parseable, with MDC (Mapped Diagnostic Context — per-thread key-values added to every log line) fields and trace ids — instead of formatting text you later have to parse with regexes (regular expressions, text-matching patterns). → §15.8

Q147. What does graceful shutdown do? Short answer: Ch 20 Q208. Then `@PreDestroy`/`SmartLifecycle.stop` run. → §15.9

Q148. What does `spring.threads.virtual.enabled=true` change? Short answer: Ch 20 Q191. → §15.10, §14.5, §4.7

Q149. What are AOT processing and GraalVM native images? Trade-offs? Short answer: Ch 20 Q209, Q210. → §15.11

Q150. How can a normal JVM app start faster without native images? Short answer: Ch 20 Q209. Training run via `-Dspring.context.exit=onRefresh`; JEP 514 simplified it in Java 25; the full JIT stays available. Also: lazy init where acceptable, fewer auto-configurations, no scanning of huge packages. → §15.11

Q151. How do you package a Boot app as a container image? Short answer: Ch 20 Q211. Size the heap relative to the container limit (`-XX:MaxRAMPercentage`). → §15.11

Q152. What is your production configuration checklist? Short answer: Ch 20 Q214. Also `server.forward-headers-strategy` when behind a reverse proxy adding `X-Forwarded-*` headers. → §15.12

18.13 Microservices and Spring Cloud

Q153. Which Spring Cloud version works with Boot 4? Short answer: Ch 20 Q216. GA November 2025; projects at 5.0.x; 2025.1.3 was the latest service release when written. Import the matching `spring-cloud-dependencies` BOM. → §16.2

Q154. What does Spring Cloud Config do, and how are changes picked up? Short answer: Ch 20 Q222. → §16.3, config notes

Q155. Eureka or Kubernetes for service discovery? Short answer: Ch 20 Q217. → §16.4, discovery notes

Q156. What does an API gateway do, and how does Spring Cloud Gateway work? Short answer: Ch 20 Q218. Gateway 5.0 (Spring Cloud 2025.1) removed the MVC `http(String)`/`http(URI)` overloads — use `http()` with `before(uri(...))`. Starters: `spring-cloud-starter-gateway-server-webflux` (Netty) and `spring-cloud-starter-gateway-server-webmvc` (pairs well with virtual threads). → §16.7, gateway notes

Q157. How do you make service-to-service calls resilient? Short answer: Ch 20 Q219. Timeouts first; a **bulkhead** (`@ConcurrencyLimit` or Resilience4j) so one slow dependency can't take all threads. In Resilience4j's default aspect order, retry wraps the circuit breaker. → §16.8, circuit breaker notes

Q158. How do you keep data consistent across services? Short answer: Ch 20 Q141, Q223. Accept eventual consistency and design the UI for it. →

Q159. How do you test contracts between services? Short answer: Ch 20 Q227. Prefer **Pact** (polyglot, broker, `can-i-deploy` gate): **Spring Cloud Contract** is no longer in the 2025.1.3 BOM and its repository is archived. → §16.10

Q160. Feign or HTTP interfaces? Short answer: Ch 20 Q87. Pair HTTP interfaces with Spring Cloud LoadBalancer for client-side discovery. → §6.6, §16.6

18.14 Spring AI and the Boot 4 / Framework 7 changes

Q161. What is Spring AI? Which version goes with Boot 4? A Spring project that gives portable APIs over AI models and vector stores (databases that store embeddings and find the most similar ones): `ChatClient`/`ChatModel`, `EmbeddingModel`, `VectorStore`, tool calling, chat memory (earlier messages of a conversation kept and re-sent) and MCP support, with Boot starters per provider (OpenAI, Anthropic, Ollama — a tool for running models locally, …). **Spring AI 2.0** (GA June 2026; 2.0.1 current) requires Boot 4 (4.1.x is the smoothest match); the 1.x line (1.1.x) is for Boot 3.4/3.5. → §17.1, AI book

Q162. How do you call a model and get typed output? Inject `ChatClient.Builder`, build a client with defaults (system prompt — fixed instructions sent to the model before the user's message; advisors — interceptors that add to or change each request), then `chatClient.prompt().user("...").call().content()` for text or `.entity(Invoice.class)` for **structured output** — Spring AI adds format instructions and converts the JSON reply into your record. Validate the result like any untrusted input. → §17.3, §17.7

Q163. How does RAG work in Spring AI? (RAG — Retrieval-Augmented Generation — means looking up relevant text first and giving it to the model with the question.) Ingest: split documents into chunks, embed them (turn each chunk into an *embedding*, a list of numbers where similar meanings are close together) with an `EmbeddingModel`, store them in a `VectorStore` (PGVector, Redis, …). Query: a RAG advisor (for example `QuestionAnswerAdvisor`) runs a similarity search for the user's question and adds the best chunks to the prompt (the text sent to the model), so the model answers from your data. Quality depends on chunking, metadata filters and evaluation. → §17.9, AI book 7

Q164. What are tools and MCP in Spring AI? Tools are your methods annotated with `@Tool` that the model may ask to call (look up an order, check stock); Spring AI executes the call and returns the result to the model. MCP (Model Context Protocol) is a standard for exposing tools and resources across processes; Spring AI has MCP client and server starters (Streamable HTTP is the default transport for its web servers in 2.0; the older SSE transport is deprecated). Treat tool calls as untrusted input: authorize them and never expose destructive operations without checks. → §17.8, §17.10

Q165. How do you test code that uses an LLM? (An LLM — large language model — is the text-generating AI model behind a chat API.) Unit-test your logic with a stub `ChatModel` that returns fixed responses (fast, deterministic — same result every run, no API key — the secret that authorizes calls to the provider); keep a smaller set of evaluation tests against the real model that check properties of the answer (relevance, grounding, format) rather than exact strings (Spring AI ships `RelevancyEvaluator` and `FactCheckingEvaluator`). → §17.2

Q166. What changed from Boot 3 to Boot 4? Short answer: Ch 20 Q12. Also: embedded launch scripts removed; Spock dropped in 4.0, back in 4.1; `@SpringBootTest` needs the matching `@AutoConfigure…` for MockMvc, `RestTestClient`, `WebTestClient`, `TestRestTemplate`; `WebClient.Builder` needs `spring-boot-starter-webclient`; probes on by default. → §3.14, §17.13

Q167. What changed from Framework 6 to 7? JSpecify annotations (the `org.springframework.lang` ones deprecated); API versioning; HTTP service groups; `BeanRegistrar`; core resilience (`@Retryable`, `@ConcurrencyLimit`, `RetryTemplate`); `@Proxyable`; `RestTestClient` and test context pausing; Jackson 3 support (Jackson 2 deprecated); JUnit 4 support deprecated; `javax.annotation` / `javax.inject` support removed; `spring-jcl` replaced by Apache Commons Logging 1.3; `RestTemplate` deprecation planned for 7.1. → §17.14

Q168. How would you migrate a Boot 3 app to Boot 4? Short answer: Ch 20 Q247. Also update Spring AI to 2.0.x, rename changed properties (e.g. RabbitMQ `retry.max-attempts` → `retry.max-retries`), and compare the condition report and `/actuator` output. → §17.13

18.15 Proofs: five tricky answers, run

What it is

Five answers candidates often get wrong, each proved by a test in `code/interview-proofs/` (Boot 4.1.1 parent, Java 25, `spring-boot-starter-jdbc` + H2 + `spring-boot-starter-test`). Each test builds a tiny context with Boot's `ApplicationContextRunner` instead of `@SpringBootTest`, so every scenario — including those that *fail* to start — is isolated and the runner captures the failure instead of failing the test class. Run with `mvn -q -B -ntp verify` (or `bash tools_run_spring.sh spring '18-*'` from the repository root).

Test class	Proves the answer to
`ScopeProofTests`	Q11 — prototype injected into a singleton
`BeanResolutionTests`	Q7/Q8 — `@Qualifier` vs `@Primary` vs parameter name
`CircularDependencyTests`	Q15 — constructor cycles, `@Lazy`, Boot's field-cycle default
`RollbackTests`	Q67 — checked exception commits
`SelfInvocationTests`	Q39/Q71 — self-invocation skips `@Transactional`

Proof 1 — a prototype inside a singleton is created once

```
@Bean @Scope("prototype") ShoppingCart cart() { return new ShoppingCart(); }
@Bean CheckoutService checkout(ShoppingCart cart, ObjectProvider<ShoppingCart> carts) {
    return new CheckoutService(cart, carts);   // cart is resolved ONCE, here
}
// test
CheckoutService a = ctx.getBean(CheckoutService.class), b = ctx.getBean(CheckoutService.class);
System.out.println("same injected cart both times?        " + (a.injectedOnce == b.injectedOnce));
System.out.println("ObjectProvider.getObject() twice same? " + (a.carts.getObject() == a.carts.getObject()));
```

Output:

```
same CheckoutService both times?      true
same injected cart both times?        true
ObjectProvider.getObject() twice same? false
```

The singleton got one cart at construction and keeps it; asking the container through `ObjectProvider` returns a new cart each time.

Proof 2 — `@Qualifier` beats `@Primary`; the parameter name is the last tie-breaker

```
@Bean @Primary PaymentGateway stripe() { return new Gateway("stripe"); }
@Bean PaymentGateway paypal() { return new Gateway("paypal"); }
@Bean Checkout checkout(PaymentGateway gateway) { ... }                      // -> stripe
@Bean Refunds refunds(@Qualifier("paypal") PaymentGateway gateway) { ... }    // -> paypal
// second config: no @Primary, parameter named "paypal"                      // -> paypal
// third config:  no @Primary, parameter named "gateway"                     // -> fails
```

Output:

```
no qualifier       -> stripe  (the @Primary bean)
@Qualifier(paypal) -> paypal  (qualifier wins over @Primary)
no @Primary, parameter named 'paypal' -> paypal
context failed: NoUniqueBeanDefinitionException
  No qualifying bean of type 'ch18.proofs.BeanResolutionTests$PaymentGateway' available: expected single matching bean but found 2:
stripe,paypal
```

Parameter-name matching works because the Boot parent compiles with `-parameters` (keeps parameter names in the bytecode); don't rely on it for important choices — a rename silently changes the bean.

Proof 3 — circular dependencies

```
static class OrderService   { OrderService(InvoiceService invoices) { } }
static class InvoiceService { InvoiceService(OrderService orders) { } }       // constructor cycle
static class LazyOrderService { LazyOrderService(@Lazy LazyInvoiceService invoices) { } }
static class FieldA { @Autowired FieldB b; }  static class FieldB { @Autowired FieldA a; }
// ApplicationContextRunner follows Boot's default: circular references not allowed
new ApplicationContextRunner().withAllowCircularReferences(true) ...          // = spring.main.allow-circular-references=true
```

Output:

```
constructor cycle      -> BeanCurrentlyInCreationException
field cycle, Boot default          -> BeanCurrentlyInCreationException
field cycle, circular refs allowed -> started: true
cycle with @Lazy       -> started: true
```

The constructor-case log shows the path Spring followed — read it when debugging: `Error creating bean with name '...OrderService': ... '...InvoiceService': ... '...OrderService': Requested bean is currently in creation: Is there an unresolvable circular reference ...`

Proof 4 — a checked exception commits

```
@Transactional
public void checkedDefault(int id) throws PaymentDeclinedException {       // checked
    insert(id); throw new PaymentDeclinedException("card declined");
}
@Transactional(rollbackFor = PaymentDeclinedException.class)
public void checkedWithRollbackFor(int id) throws PaymentDeclinedException { ...same... }
@Transactional
public void unchecked(int id) { insert(id); throw new IllegalStateException("bug"); }
```

Output:

```
checked exception, default rules  -> row 1 kept?  true
checked exception, rollbackFor    -> row 2 kept?  false
RuntimeException, default rules   -> row 3 kept?  false
```

All three threw, yet row 1 was committed — the default rollback rule, not a Spring bug.

Proof 5 — self-invocation skips the transaction

```
public void importAll() { saveBatch(); }                // this.saveBatch(): no proxy
@Transactional
public void saveBatch() {
    System.out.println("  inside saveBatch, transaction active? "
            + TransactionSynchronizationManager.isActualTransactionActive());
    jdbc.sql("INSERT INTO orders VALUES (10, 'a')").update();
    throw new IllegalStateException("second insert failed");
}
```

Output:

```
bean class: SelfInvocationTests$ImportService$$SpringCGLIB$$0
call through the proxy (svc.saveBatch()):
  inside saveBatch, transaction active? true
  rows after failure: 0  (rolled back)
self-invocation (svc.importAll() -> this.saveBatch()):
  inside saveBatch, transaction active? false
  rows after failure: 1  (NOT rolled back - no transaction)
```

The bean class name shows the CGLIB proxy. `TransactionSynchronizationManager.isActualTransactionActive()` is the quickest one-liner to *prove* whether a transaction exists at a given line.

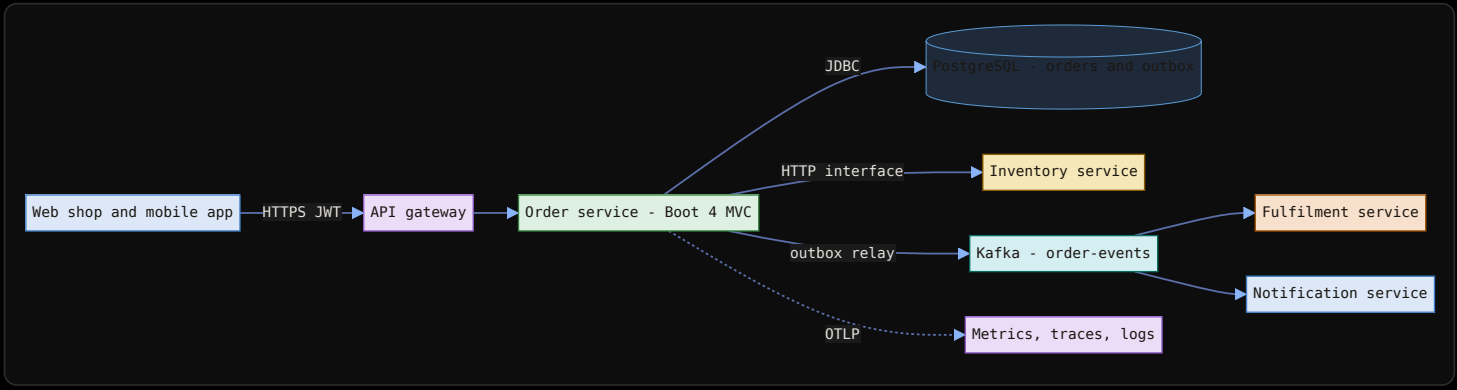
18.16 "Explain your Spring project" — a walkthrough template

Why it matters

"Tell me about a project you built with Spring" comes up in almost every interview. The interviewer checks whether you understand *why* the pieces are there and digs into whatever you mention; a structured five-minute answer steers that digging toward topics you know.

The template (about 5 minutes)

1. **Purpose in one sentence** — who uses it and what business problem it solves. "An order service that takes orders from the web shop and mobile app, reserves stock and hands paid orders to fulfilment."
2. **Scale and constraints** — requests per second, data size, latency target (latency = how long one request takes), team size. Numbers make it real; round honestly.
3. **Architecture picture** — draw it (below). Name each box and the protocol on each arrow.
4. **Stack and why** — Boot version, MVC or WebFlux (and why), database and access style, messaging, security. For each, one reason.
5. **Follow one request end to end** — through the filter chain, controller, validation, service transaction, repository, event, response.
6. **Data and consistency** — transactions, locking, how events leave the service reliably, how duplicates are handled.
7. **Security** — how users and other services authenticate, where authorization lives.
8. **Quality** — test strategy (unit, slices, Testcontainers, contract tests), CI (Continuous Integration — every change built and tested automatically).
9. **Running it** — deployment, observability (metrics, traces, logs, alerts), how you handle failures of dependencies.
10. **One hard problem** (Situation → Task → Action → Result — the STAR story format) and **what you would improve**.



A filled-in example (say it in your own words)

> "The order service is a Spring Boot 4 MVC application on Java 25 with virtual threads enabled. We chose MVC over WebFlux because the work is CRUD plus a few downstream calls, and virtual threads gave us the concurrency we needed with simple blocking code.
>
> A `POST /orders` passes the Spring Security chain — it's a stateless resource server validating JWTs from our identity provider (the service that logs users in and issues tokens), with `orders:write` scope required. The controller validates the request DTO with Bean Validation and calls `OrderService.place()`, which is the one `@Transactional` boundary. Inside it we check stock through an `@HttpExchange` client to the inventory service — with 2-second timeouts and a `@Retryable` on the idempotent reservation call, which carries an idempotency key — then save the order with Spring Data JPA and write an `OrderPlaced` row to an outbox table in the same transaction. A relay publishes outbox rows to Kafka; fulfilment and notification consume them idempotently using the event id.
>
> We use optimistic locking with `@Version` on orders because concurrent edits are rare, and turned off Open Session in View; read endpoints use DTO projections and `@EntityGraph` to avoid N+1 queries, which we check with statement-count assertions in `@DataJpaTest` s.

> Tests: plain unit tests for pricing rules, `@WebMvcTest` with `MockMvcTester` and `jwt()` for the API including 401/403 cases, `@DataJpaTest` against PostgreSQL in Testcontainers, and a contract test with inventory. In production we export Micrometer metrics and OpenTelemetry traces; alerts are on p99 latency, error rate and consumer lag (how far consumers are behind the newest messages).
>
> The hardest issue was duplicate orders when clients retried after timeouts; we added an `Idempotency-Key` header stored with a unique constraint (a database rule that rejects a second row with the same value), returning the original response on a repeat. Next I'd split the read side into a separate projection, because reporting queries compete with order writes."

Follow-ups to prepare for (and where to revise)

They ask	Be ready to explain	Revise
"Why not WebFlux?"	virtual threads vs reactive trade-off	Q131
"What if Kafka is down?"	outbox keeps events; relay retries; consumer lag alerts	Q139
"How do you avoid double charging?"	idempotency keys, unique constraints, idempotent consumers	Q56, Q137
"Show me the transaction boundary"	service method, propagation, what's outside it	Q65–Q72
"How do you know it's slow?"	metrics, traces, SQL logging, statement counts	Q143–Q145, Q83
"What would break at 10× traffic?"	DB connections, hot partitions, cache, N+1	Q64, Q136
"How is it secured?"	JWT validation, scopes, method security, service auth	Q108, Q111

Common mistakes

- Listing technologies without reasons ("we used Kafka, Redis, Kubernetes…").
- Saying "we" for everything — say what **you** designed, built or fixed.
- Mentioning something you can't explain two levels deeper; the interviewer *will* ask about it.
- Inventing numbers — approximate honestly ("a few hundred requests per second at peak").

TL;DR

Purpose → scale → picture → stack with reasons → one request end to end → data, security, tests, operations → one hard problem and one improvement.

18.17 Debugging scenarios

"This is happening in production — what do you do?" Answer in four steps: **symptom** (exactly what you see), **cause** (the mechanism, not a guess), **fix** (the smallest correct change), **proof** (evidence for cause and fix — a log line, a test, a metric). Always read the *root cause* — the last "Caused by" at the bottom of a Spring stack trace; the top lines are just the chain of beans being created.

Scenario 1 — "No qualifying bean" / `NoUniqueBeanDefinitionException`

- **Symptom.** Startup fails with the Boot failure analyzer's report (Boot's human-readable explanation printed for common startup errors): *"Parameter 0 of constructor in OrderService required a bean of type 'PaymentGateway' that could not be found"* (`NoSuchBeanDefinitionException`), or *"...required a single bean, but 2 were found"* (`NoUniqueBeanDefinitionException`).
- **Cause (not found).** The class is outside the scanned packages (main class not in the root package); it lacks a stereotype annotation (`@Component`, `@Service`, `@Repository`, `@Controller`); its `@Profile` / `@Conditional` didn't match; the auto-configuration that should create it backed off or its starter is missing (common after the Boot 4 starter renames); or you inject the concrete class while a JDK proxy only implements the interface.
- **Cause (not unique).** Two beans of the type — often your own bean plus one from a library or auto-configuration — and no `@Primary`, `@Qualifier` or matching parameter name.
- **Fix.** Move the main class up / add `@ComponentScan` or `@Import`; add the missing starter; activate the profile; for ambiguity choose with `@Qualifier` at the injection point or mark the usual one `@Primary`; inject `List<T>` if you want all.
- **Prove it.** `--debug` → condition evaluation report shows why an auto-configuration did *not* match; `/actuator/beans` (or `ctx.getBeanNamesForType(PaymentGateway.class)` in a test) lists the candidates; an `ApplicationContextRunner` test reproduces the failure in milliseconds (see `BeanResolutionTests`, §18.15). → Q7, §2.5, §3.11

Scenario 2 — circular dependency at startup

- **Symptom.** *"The dependencies of some of the beans in the application context form a cycle"* with an ASCII diagram of the loop, or `BeanCurrentlyInCreationException: Requested bean is currently in creation`.
- **Cause.** A → B → A through constructors (never resolvable), or through fields/setters, which Boot rejects by default since 2.6. It usually means responsibilities are split wrongly — e.g. `OrderService` calls `InvoiceService` and `InvoiceService` calls back into `OrderService`.
- **Fix.** Extract the shared logic into a third bean both can depend on; or turn the back-call into an application event; or depend on a narrower interface. `@Lazy` on one constructor parameter is an acceptable short-term patch. Setting `spring.main.allow-circular-references=true` hides the design problem — avoid it.

- **Prove it.** The failure report lists the cycle; after the fix, a context test (or `CircularDependencyTests`) starts cleanly. → Q15, §2.9

Scenario 3 — `@Transactional` does not roll back

- **Symptom.** A method threw, but its first insert is still in the database.
- **Cause (in order of likelihood).** (1) The exception is **checked** and there is no rollback rule. (2) The exception was **caught** inside the method (or in a lower method) and never reached the proxy. (3) **Self-invocation** or a `private`/`final` method — there was no transaction at all. (4) The write went through a different `DataSource`/transaction manager than the one `@Transactional` uses. (5) The work ran on another thread (`@Async`, `parallelStream`, `CompletableFuture`). (6) MySQL MyISAM tables (an old MySQL storage engine), which don't support transactions.
- **Fix.** `rollbackFor`, or the global `rollbackOn = ALL_EXCEPTIONS`; rethrow (or call `TransactionAspectSupport.currentTransactionStatus().setRollbackOnly()` if you must swallow); move the method to another bean; name the right transaction manager (`@Transactional("ordersTx")`); keep the unit of work on one thread.
- **Prove it.** Print `TransactionSynchronizationManager.isActualTransactionActive()` inside the method; turn on `logging.level.org.springframework.transaction.interceptor=TRACE` (logs "Getting transaction for [...]" / "Completing transaction for [...]") and `logging.level.org.springframework.jdbc.support.JdbcTransactionManager=DEBUG` (or `...orm.jpa.JpaTransactionManager=DEBUG`), which logs "Initiating transaction rollback" or "Initiating transaction commit". Then write a test that asserts the row count after the failure (`RollbackTests`, `SelfInvocationTests`). → Q67, Q71, §7.10, §7.13

Scenario 4 — `LazyInitializationException`

- **Symptom.** *"failed to lazily initialize a collection of role: Order.items: could not initialize proxy - no Session"*, typically in a controller, a mapper or during JSON serialization.
- **Cause.** A lazy association was touched after the transaction — and with it the persistence context — had ended. It often appears after someone turns off Open Session in View, which had been hiding it.
- **Fix.** Load what the use case needs inside the service transaction: `JOIN FETCH` or `@EntityGraph` for that query, or a DTO projection; map entities to DTOs inside the transaction and return DTOs from controllers. Do **not** switch the association to EAGER (causes over-fetching and N+1 elsewhere) and don't re-enable OSIV or `hibernate.enable_lazy_load_no_trans`.
- **Prove it.** Reproduce it with a `@DataJpaTest` that loads the entity in one transaction and touches the collection outside it; after the fix the same test passes and the SQL log shows the association loaded by the one query. → Q82, §8.6, §8.15

Scenario 5 — N+1 queries make a list endpoint slow

- **Symptom.** `GET /authors` takes 2 s in production but 20 ms locally; the database shows hundreds of identical small `SELECT`s per request.
- **Cause.** One query for the list, then one query per row when the code (or Jackson) touches a lazy association — 1 + N round trips. Local data had 3 rows, so nobody noticed.
- **Fix.** For this use case, fetch with `JOIN FETCH` / `@EntityGraph` or query a DTO projection; as a global safety net set `spring.jpa.properties.hibernate.default_batch_fetch_size=16` (1 + N/16 queries). Beware fetching a collection together with pagination.
- **Prove it.** Count statements: enable Hibernate statistics (`hibernate.generate_statistics=true`) or a datasource proxy and assert the count in a `@DataJpaTest` (e.g. "exactly 1 statement for 50 authors"); in production, traces show one span per SQL statement under the request span. → Q83, §8.7

Scenario 6 — "I'm logged in but get 401", or 403 where 401 was expected

- **Symptom.** A client gets 401 or 403 and nobody knows which rule denied it.
- **Cause.** Remember: **401 = not authenticated** (no or invalid credentials — expired JWT, wrong issuer/audience, missing `Authorization` header, a chain that doesn't have the authentication mechanism you think), **403 = authenticated but not allowed**. Classic 403 causes: `hasRole("ADMIN")` while the user has authority `ADMIN` without the `ROLE_` prefix (or JWT scopes that became `SCOPE_x` authorities); a missing **CSRF token** on POST/PUT/DELETE in a session-based chain; a broader rule (`anyRequest()`, `/api/**`) placed before the specific one; the request matched a different `SecurityFilterChain` than expected. An exception forwarded to `/error` that your rules don't permit can also turn a real 500 into a 401/403 — permit `/error`.
- **Fix.** Correct the authority mapping (`JwtAuthenticationConverter`, role prefix), send the CSRF token or disable CSRF only on the stateless token chain, reorder rules from specific to general, check each chain's `securityMatcher` and `@Order`.
- **Prove it.** `logging.level.org.springframework.security=TRACE` prints every filter the request passes and the authorization decision; print the current `Authentication` and its authorities in a debug endpoint or test; write `spring-security-test` tests for the allowed path and both denial paths (`jwt().authorities(...)`, `with(csrf())`). → Q103, Q104, §10.6, §10.12

Scenario 7 — CORS "blocked by CORS policy" in the browser

- **Symptom.** The browser console (the developer-tools panel that shows JavaScript errors) says *"has been blocked by CORS policy: Response to preflight request doesn't pass access control check"*; the same call works in Postman or curl (a GUI and a command-line tool for sending HTTP requests by hand).
- **Cause.** Curl ignores CORS; only browsers enforce it. Typical server-side causes: Spring Security runs before CORS handling and answers the credential-less `OPTIONS` preflight with 401 (CORS configured only in MVC, without `http.cors(...)`); the origin doesn't match exactly (scheme, host and port — `http://localhost:3000` ≠ `http://localhost:5173`); a requested header such as `Authorization` or a method such as `PATCH` isn't allowed; `allowCredentials(true)` combined with origin `*` (forbidden); both the gateway and the service add `Access-Control-Allow-Origin`, producing two values.

- **Fix.** One `CorsConfigurationSource` bean with the exact origins, methods and headers, plus `http.cors(withDefaults())` in the chain; CORS in only one layer (gateway *or* service); `allowedOriginPatterns` if you need patterns with credentials.
- **Prove it.** Replay the preflight: `curl -i -X OPTIONS https://api.example.com/api/notes -H "Origin: https://app.example.com" -H "Access-Control-Request-Method: POST" -H "Access-Control-Request-Headers: authorization,content-type"` — expect 200 and the `Access-Control-Allow-*` headers; the browser's Network tab shows the preflight's status. Lock it in with a `MockMvcTester` preflight test for an allowed and a refused origin. → Q54, §10.9, §5.10

Scenario 8 — `@Async` / `@Cacheable` / `@Transactional` "does nothing"

- **Symptom.** An `@Async` method runs on the request thread (the caller waits), a `@Cacheable` method runs every time, or a transaction is missing — with no error.
- **Cause.** The call never passed through the proxy: self-invocation (`this.method()`), a `private`/`final` method, an object created with `new`, or the `@Enable...` annotation is missing (`@EnableAsync`, `@EnableCaching`; `@EnableResilientMethods` for `@Retryable`).
- **Fix.** Move the annotated method to a separate bean and inject it; make it public and non-final; add the enabling annotation; or use a programmatic API (`TransactionTemplate`, an executor, `CacheManager`).
- **Prove it.** Print `AopUtils.isAopProxy(bean)` and `bean.getClass()` (look for `$$SpringCGLIB$$`); print `Thread.currentThread()` inside the `@Async` method; count calls of the cached method in a test; `isActualTransactionActive()` for transactions (Proof 5). → Q39, Q40, §4.5

Scenario 9 — the context fails to start because of auto-configuration

- **Symptom.** `APPLICATION FAILED TO START` with a failure-analyzer message such as *"Failed to configure a DataSource: 'url' attribute is not specified and no embedded datasource could be configured"* — right after adding `spring-boot-starter-data-jpa`; or a bean definition override error; or a `ClassNotFoundException`/`NoSuchMethodError` (the JVM could not find a class / a method that the compiled code expects — a sign of mismatched library versions) from inside an auto-configuration.
- **Cause.** Auto-configuration reacts to the **classpath**: adding a starter activates its configuration, which now needs settings (a JDBC URL) or a driver. Other causes: two libraries define beans with the same name (overriding is disabled by default); mixed versions because a dependency has a hard-coded `<version>` that fights Boot's BOM, or a Boot 3 library on Boot 4 (old auto-configuration packages, Jackson 2 vs 3, `javax` vs `jakarta`).
- **Fix.** Provide the missing configuration (URL, driver, or an embedded database for tests); if the feature isn't wanted, remove the starter or exclude that auto-configuration; remove explicit versions and let the BOM manage them; upgrade libraries to Boot 4-compatible versions (Spring Cloud 2025.1, Spring AI 2.0, springdoc for Boot 4).
- **Prove it.** Read the *Description* and *Action* sections of the failure report; run with `--debug` for the condition report; `mvn dependency:tree -Dincludes=<group>` shows which artifact pulled in the conflicting version; `ApplicationContextRunner` with `AutoConfigurations.of(...)` reproduces a single auto-configuration in a test. → Q24–Q26, §3.11, §3.12

Scenario 10 — slow startup

- **Symptom.** "Started OrderApplication in 48 seconds"; pods fail their startup probe (the Kubernetes check that gives a slow-starting app time before liveness checks begin); the test suite takes minutes.
- **Cause.** Usually a few expensive beans, not Spring itself: work in `@PostConstruct` or constructors (warming caches, calling remote services, loading files), remote calls that hang until a timeout because a dependency isn't reachable, large Flyway migrations or Hibernate schema validation, very broad component scanning; in tests, many different context configurations defeating the context cache.
- **Fix.** Move slow warm-up out of bean creation (an `ApplicationReadyEvent` listener or a background task, with readiness reporting "not ready" until done); set connect timeouts; run migrations as a separate job for very large schemas; narrow scanning; standardize test configurations. Then consider lazy initialization (with care), and the JDK AOT cache/CDS or a native image for the remaining JVM start cost.
- **Prove it.** Record startup steps: `SpringApplication app = ...; app.setApplicationStartup(new BufferingApplicationStartup(2048));` and call `/actuator/startup`, then sort steps by duration — the slow bean is named; or use `FlightRecorderApplicationStartup` with JFR (Java Flight Recorder, the JVM's built-in low-overhead profiler). Compare the "Started ... in X seconds" line before and after. For tests, enable `logging.level.org.springframework.test.context.cache=DEBUG` to see context-cache hits and misses. → Q16, Q119, Q150, §2.10, §15.11, §12.11

Self-check

1. Postman works, the browser doesn't. First suspicion? → CORS — most often the preflight hitting security (401) or an origin mismatch.
2. Which log category tells you why a request got 403? → `org.springframework.security` at TRACE.
3. What is the quickest proof that N+1 is fixed? → A test asserting the SQL statement count.
4. A new starter made startup fail. Where do you look first? → The failure analyzer's Description/Action, then the condition report (`--debug`).

Chapter TL;DR

- **Container:** beans are singletons by default and shared across threads; type → `@Qualifier` → `@Primary` → `@Fallback` → parameter name decides injection; a prototype inside a singleton is created once (use `ObjectProvider`); constructor cycles never work.
- **Boot:** auto-configuration = `.imports` list + conditions + `@ConditionalOnMissingBean`; `--debug` explains every decision; outside/later property sources win; Boot 4 is modular with renamed starters.

- **Proxies:** `@Transactional`, `@Cacheable`, `@Async`, `@Retryable`, `@PreAuthorize` all work only through the proxy — self-invocation and `private` methods skip them.
- **Transactions:** REQUIRED + database-default isolation; checked exceptions commit unless told otherwise; caught exceptions never roll back; keep one business operation per transaction and no remote calls inside it.
- **JPA:** persistence context + dirty checking; everything LAZY, fetch per use case; fix N+1 with fetch joins, entity graphs or batch size; OSIV off; `@Version` for concurrent edits; migrations own the schema.
- **Security 7:** filter chain → authentication → `SecurityContextHolder` → authorization; 401 = who are you, 403 = not allowed; lambda DSL only; CSRF matters whenever the browser sends credentials automatically; JWT resource servers validate locally.
- **Testing:** unit → slices → a few `@SpringBootTest`s; `@MockitoBean` (not `@MockBean`) in Boot 4; keep context configurations few so caching works.
- **Production:** Actuator with probes, Micrometer metrics and traces, structured logs, graceful shutdown, virtual threads for blocking I/O; outbox + idempotent consumers for messaging.
- **Debugging:** symptom → mechanism → smallest fix → proof with a log, a test or a metric.

Next: Chapter 19 — Cheat sheet and glossary collects the annotations, properties, starters and terms on a few pages.

Chapter 19 — Cheat Sheet and Glossary

This chapter is the book on one table: you come here to **look things up** — "which starter brings `@WebMvcTest` in Boot 4?", "what does `REQUIRES_NEW` do when the caller rolls back?", "which property turns on virtual threads?", "`hasRole("ADMIN")` or `hasRole("ROLE_ADMIN")`?". Nothing here is new material: every row links (**§**) to the teaching section, defaults and behaviours marked ✔ are proved by tests in `code/cheatsheet-checks/` (§19.15), and every term is defined once — annotations in the sheets (§19.3, §19.9), everything else in the A–Z glossary (§19.16).

Versions: Spring Boot **4.1.1** → Spring Framework **7.0.9**, Spring Security **7.1.1**, Java **25** (17 minimum). "(Boot 4)", "(F7)" and "(S7)" mark things that are new or changed in Boot 4, Framework 7 or Security 7.

19.1 How to use this chapter

How the sheets are organised

You want to...	Go to
find the name, jar or purpose of an annotation	§19.3
add a dependency	§19.4
set or look up a property and its default	§19.5
pick a propagation / isolation / rollback rule	§19.6
remember the bean lifecycle order	§19.7
copy a security configuration	§19.8
choose a test annotation	§19.9
find a Reactor operator	§19.10
know what an Actuator endpoint shows	§19.11
choose between two technologies	§19.12
upgrade a Boot 3 app	§19.13
revise for an interview	§19.14
look up a word	§19.16

Conventions

- **Module / starter**: the jar that contains the class, and the Boot starter that brings it. "core" means `spring-context` / `spring-beans`, which every starter includes.
- **§N.M** links go to the teaching section of Chapters 1–18. Follow them for the explanation, the trade-offs and the proof.
- Defaults quoted in these sheets come from the Boot 4.1.1 configuration metadata (the `spring-configuration-metadata.json` files inside the Boot jars) or from the chapter that measured them. The ones marked ✔ **proved** are asserted by a test in §19.15.
- Properties are written in `.properties` form. The same keys work in YAML and as environment variables (named values set outside the app by the shell, OS or container) (§3.4).

19.2 Versions at a glance

What `spring-boot-starter-parent` **4.1.1** puts on your classpath (from the `spring-boot-dependencies-4.1.1` BOM). You choose one number, the Boot version, and Boot chooses the rest (§1.4).

Component	Version	Notes
Java	17 minimum, 25 recommended	this book: Java 25
Spring Framework	7.0.9	Jakarta EE 11 (Servlet 6.1, JPA 3.2, Bean Validation 3.1)
Spring Security	7.1.1	Authorization Server now inside Spring Security (§11.14)
Spring Data	2026.0.1 release train	JPA, JDBC, MongoDB, Redis, R2DBC, REST
Hibernate ORM / Validator	7.4.5 / 9.1.3	
Jackson	3.1.5 (`tools.jackson.*`)	annotations still `com.fasterxml.jackson.annotation` (§5.5)
Tomcat / Jetty	11.0.24 / 12.1.12	Undertow removed
HikariCP / H2 / PostgreSQL JDBC	7.0.2 / 2.4.240 / 42.7.13	
Reactor	2025.0.7 train (Reactor Core 3.8)	
Micrometer / Micrometer Tracing / OpenTelemetry	1.17.1 / 1.7.1 / 1.62.0	
Spring for Apache Kafka / kafka-clients	4.1.1 / 4.2.1	
Spring AMQP	4.1.1	
Flyway / Liquibase	12.4.0 / 5.0.3	Liquibase 5 Community is FSL-licensed (§9.4)
JUnit / Mockito / AssertJ / Awaitility	6.0.3 / 5.23.0 / 3.27.7 / 4.3.0	
Testcontainers	2.0.5	new artifact names `testcontainers-<module>` (§12.14)
Caffeine / Lettuce	3.2.4 / 7.5.2	
Not managed by Boot		import their own BOM
Spring Cloud	2025.1.x "Oakwood" (2025.1.2+ for Boot 4.1)	§16.2
Spring AI	2.0.x (2.0.1)	§17.1
springdoc-openapi	3.x (3.1.1)	§5.12
Resilience4j	`resilience4j-spring-boot4` 2.4.0	§16.8

19.3 Annotations cheat sheet

The annotations of this book, grouped by area. "Needs" names the switch without which the annotation silently does nothing — the most common reason an annotation "doesn't work".

Core container and beans

Annotation	Module / starter	Purpose	§
`@Component`	core	generic bean found by component scanning	§2.3
`@Service`	core	`@Component` for business logic (no extra behaviour)	§2.3
`@Repository`	core	`@Component` for data access + exception translation to `DataAccessException`	§2.3, §7.5
`@Controller` / `@RestController`	spring-web / `-webmvc`, `-webflux`	web handler; `@RestController` = `@Controller` + `@ResponseBody`	§5.3
`@Configuration`	core	class of `@Bean` methods; full mode = CGLIB subclass (`proxyBeanMethods = true`)	§2.3
`@Bean`	core	factory method registering its return value as a bean; name = method name	§2.3
`@ComponentScan`	core	scan a package tree for components (inside `@SpringBootApplication`)	§1.8
`@Import`	core	pull in another configuration class, selector, registrar or `BeanRegistrar`	§2.3
`@Autowired`	core	inject here; optional on a single constructor	§2.4
`@Qualifier`	core	pick a bean by name/qualifier at the injection point (beats `@Primary`) ✔	§2.5
`@Primary`	core	default bean when several match ✔	§2.5
`@Fallback`	core (Framework 6.2)	use this bean only if no other candidate qualifies	§2.5
`@Order`	core	order in `List<T>` injection, aspects, filters, listeners (lower first) ✔	§2.5
`@Value`	core	inject one value: `${key:default}`, SpEL `#{...}` ✔	§3.5
`@Scope`	core	`singleton` (default), `prototype`, web scopes; `proxyMode` for scoped proxies	§2.6
`@RequestScope` / `@SessionScope` / `@ApplicationScope`	spring-web	web scopes with a CGLIB scoped proxy	§2.6
`@Lookup`	spring-beans	method injection: returns a fresh prototype on each call	§2.6
`@Lazy`	core	create on first use; on an injection point, inject a lazy proxy (breaks cycles)	§2.10, §2.9
`@PostConstruct` / `@PreDestroy`	`jakarta.annotation`	init / destroy callbacks (not for prototypes' destroy) ✔	§2.7
`@Profile`	core	register only when a profile expression matches (`"!dev"`, `"prod & eu"`)	§2.11
`@Conditional`	core	register only if a `Condition` matches	§2.11
`@EventListener`	core	receive application events (synchronous by default)	§2.12
`@Proxyable`	core (F7)	choose JDK or CGLIB proxy for one bean	§4.4
`@NullMarked` / `@Nullable`	`org.jspecify.annotations` (F7)	null-safety contract; `@Nullable` injection point = optional	§2.13
`@Inject`	`jakarta.inject` (optional jar)	standard alternative to `@Autowired`; `javax.inject` removed in F7	§2.4

Spring Boot and configuration

Annotation	Module / starter	Purpose	§
`@SpringBootApplication`	spring-boot-autoconfigure / `spring-boot-starter`	`@SpringBootConfiguration` + `@EnableAutoConfiguration` + `@ComponentScan`	§1.8
`@EnableAutoConfiguration`	spring-boot-autoconfigure	load `AutoConfiguration.imports` candidates and apply them after your beans	§3.10
`@AutoConfiguration`	spring-boot-autoconfigure	marks an auto-configuration class (`before` / `after` ordering)	§3.13
`@ConfigurationProperties("prefix")`	spring-boot	bind a group of properties to a typed object (record ideal)	§3.6
`@ConfigurationPropertiesScan` / `@EnableConfigurationProperties`	spring-boot	register `@ConfigurationProperties` classes	§3.6
`@DefaultValue`	spring-boot	default for a constructor-bound property ✔	§3.6
`@ConstructorBinding`	spring-boot	pick the binding constructor when there are several	§3.6
`@DurationUnit` / `@DataSizeUnit`	spring-boot	unit for plain numbers (default ms / bytes) ✔	§3.6
`@Validated` (on properties)	core + `spring-boot-starter-validation`	validate bound properties at startup	§3.6
`@ConditionalOnClass` / `OnMissingClass`	spring-boot-autoconfigure	class present / absent on the classpath	§3.10
`@ConditionalOnBean` / `OnMissingBean`	spring-boot-autoconfigure	bean present / absent (reliable in auto-configurations only)	§3.10
`@ConditionalOnProperty` / `OnBooleanProperty`	spring-boot-autoconfigure	property value / boolean switch (`OnBooleanProperty` since Boot 3.5)	§3.10
`@ConditionalOnWebApplication` / `OnThreading` / `OnCloudPlatform` / ...	spring-boot-autoconfigure	web type, virtual vs platform threads, platform	§3.10
`@PropertySource`	core	add a `.properties` file (low precedence, no YAML)	§3.9
`@EntityScan`	spring-boot-persistence (Boot 4 package)	where to find JPA entities outside the main package	§17.13

AOP and resilience

Annotation	Module / starter	Purpose	§
`@Aspect` (+ `@Component`)	aspectjweaver / `spring-boot-starter-aspectj` (Boot 4)	a class of pointcuts and advice; must also be a bean	§4.2
`@Pointcut`	aspectjweaver	named, reusable pointcut expression	§4.3
`@Before` / `@AfterReturning` / `@AfterThrowing` / `@After` / `@Around`	aspectjweaver	the five advice types; only `@Around` controls `proceed()`	§4.2
`@EnableAspectJAutoProxy`	core	switch on `@Aspect` support (Boot does it: `spring.aop.auto=true`)	§4.2
`@Retryable`	core `org.springframework.resilience` (F7)	retry on exception; `maxRetries` 3, `delay` 1000 ms, `multiplier` 1.0 ✔ — **needs `@EnableResilientMethods`**	§4.7
`@ConcurrencyLimit`	core (F7)	bulkhead: max concurrent calls, policy `BLOCK` or `REJECT` — needs `@EnableResilientMethods`	§4.7
`@EnableResilientMethods`	core (F7)	turns on `@Retryable` and `@ConcurrencyLimit` (no auto-configuration)	§4.7
`@CircuitBreaker` / `@Retry` / `@RateLimiter` / `@Bulkhead` / `@TimeLimiter`	`resilience4j-spring-boot4`	Resilience4j patterns with `fallbackMethod`	§16.8

Web: Spring MVC, WebFlux, REST

Annotation	Module / starter	Purpose	§
`@RequestMapping`	spring-web	class-level prefix / shared mapping attributes	§5.3
`@GetMapping` / `@PostMapping` / `@PutMapping` / `@PatchMapping` / `@DeleteMapping`	spring-web	map an HTTP method + path; `version = "1.1+"` (F7)	§5.3, §5.9
`@PathVariable`	spring-web	URL segment `{id}`	§5.4
`@RequestParam`	spring-web	query parameter / form field (`required = true` by default)	§5.4
`@RequestHeader` / `@CookieValue`	spring-web	header / cookie value	§5.4
`@RequestBody`	spring-web	body converted by an `HttpMessageConverter` (Jackson 3)	§5.4
`@RequestPart`	spring-web	part of a `multipart/form-data` request (`MultipartFile`)	§5.10
`@ModelAttribute`	spring-web	bind query params/form fields to an object	§5.4
`@ResponseStatus`	spring-web	fixed status for a handler or an exception class	§5.4, §5.7
`@ResponseBody`	spring-web	write the return value as the body	§5.3
`@ExceptionHandler`	spring-web	handle an exception type (in a controller or an advice)	§5.7
`@ControllerAdvice` / `@RestControllerAdvice`	spring-web	global exception handling (return `ProblemDetail`)	§5.7
`@Valid`	`jakarta.validation` + `-validation` starter	cascade validation into a body / nested object	§5.6
`@Validated`	core	validation **groups**; on a class: method validation via proxy	§5.6
`@CrossOrigin`	spring-web	per-controller CORS rules	§5.10
`@JsonProperty` / `@JsonIgnore`	`com.fasterxml.jackson.annotation`	JSON names / hidden fields (package unchanged in Jackson 3)	§5.5
`@JacksonComponent` / `@JacksonMixin`	spring-boot-jackson (Boot 4)	register (de)serializers / mix-ins (were `@JsonComponent` / `@JsonMixin`)	§5.5
`@EnableWebMvc`	spring-webmvc	full manual MVC control — **switches Boot's MVC auto-configuration off**	§3.12

HTTP clients

Annotation	Module / starter	Purpose	§
`@HttpExchange`	spring-web	HTTP service interface: shared URL prefix, headers, version	§6.6
`@GetExchange` / `@PostExchange` / `@PutExchange` / `@PatchExchange` / `@DeleteExchange`	spring-web	one HTTP call per interface method	§6.6
`@ImportHttpServices(group, types)`	spring-web (F7) + `spring-boot-starter-restclient`	register HTTP interface proxies per group, configured by `spring.http.serviceclient.<group>.*`	§6.6
`@ClientRegistrationId`	spring-security-oauth2-client	which OAuth2 registration's token to add (type-level since S7)	§11.10
`@LoadBalanced`	spring-cloud-commons	resolve `http://service-id` through Spring Cloud LoadBalancer	§16.5
`@FeignClient` / `@EnableFeignClients`	spring-cloud-openfeign (maintenance mode)	declarative client, Feign style	§16.6

Data access, transactions, JPA and Spring Data

Annotation	Module / starter	Purpose	§
`@Transactional`	spring-tx	declarative transaction (see §19.6) ✔	§7.8
`@EnableTransactionManagement`	spring-tx	switch on `@Transactional` (Boot does it); `rollbackOn = ALL_EXCEPTIONS` (F6.2)	§7.10
`@TransactionalEventListener`	spring-tx	run a listener at `AFTER_COMMIT` (default), `BEFORE_COMMIT`, `AFTER_ROLLBACK`, `AFTER_COMPLETION`	§7.15
`@Entity` / `@Table` / `@Column`	`jakarta.persistence` / `-data-jpa`	map a class / table / column	§8.3
`@Id` / `@GeneratedValue`	`jakarta.persistence`	primary key and strategy (`IDENTITY`, `SEQUENCE`, `UUID`, `AUTO`)	§8.3
`@ManyToOne` / `@OneToMany` / `@OneToOne` / `@ManyToMany`	`jakarta.persistence`	relationships; to-one EAGER, to-many LAZY by default	§8.5
`@JoinColumn` / `mappedBy`	`jakarta.persistence`	owning side (FK) / inverse side	§8.5
`@Enumerated(EnumType.STRING)` / `@Embedded` / `@Embeddable` / `@Transient`	`jakarta.persistence`	enum as text / value object / not stored	§8.3
`@Version`	`jakarta.persistence` (JPA) or `org.springframework.data.annotation` (Data JDBC, Mongo)	optimistic locking column	§8.14, §9.2
`@Query`	spring-data-jpa / -jdbc / ...	JPQL (JPA), SQL (JDBC), `nativeQuery = true`	§8.10
`@Modifying`	spring-data-jpa / -jdbc	UPDATE/DELETE query; `clearAutomatically`	§8.10
`@Param`	spring-data-commons	name a query parameter (optional with `-parameters`)	§8.10
`@EntityGraph(attributePaths = ...)`	spring-data-jpa	fetch associations in the same query (left join)	§8.7
`@Lock(LockModeType.PESSIMISTIC_WRITE)`	spring-data-jpa	`SELECT ... FOR UPDATE`	§8.14
`@BatchSize`	Hibernate	batch-load lazy collections (N+1 fix)	§8.7
`@DynamicUpdate`	Hibernate	UPDATE only changed columns	§8.4
`@EnableJpaAuditing` / `@EnableJdbcAuditing`	spring-data-jpa / -jdbc	switch on auditing	§8.13, §9.2
`@CreatedDate` / `@LastModifiedDate` / `@CreatedBy` / `@LastModifiedBy`	spring-data-commons	audit columns	§8.13
`@MappedCollection(idColumn, keyColumn)`	spring-data-relational	child collection of a Data JDBC aggregate	§9.2
`@RepositoryRestResource`	spring-data-rest	export (or hide) a repository as HAL endpoints	§9.3
`@Document` / `@Indexed`	spring-data-mongodb	MongoDB document / index	§9.5
`@RedisHash(timeToLive = ...)`	spring-data-redis	Redis repository entity with TTL	§9.6

Caching

Annotation	Module / starter	Purpose	§
`@EnableCaching`	core / `spring-boot-starter-cache`	switch on the cache proxy — without it nothing is cached	§9.8
`@Cacheable`	core	return the cached value; run the method only on a miss (`sync` , `condition` , `unless`)	§9.8
`@CachePut`	core	always run, then store the result	§9.8
`@CacheEvict`	core	remove a key (`allEntries = true` to clear)	§9.8
`@Caching` / `@CacheConfig`	core	several cache ops on one method / class defaults	§9.8

Security

Annotation	Module / starter	Purpose	§
`@EnableWebSecurity`	spring-security-config	web security (Boot applies it for you)	§10.3
`@EnableMethodSecurity`	spring-security-config	switch on method security annotations	§10.7
`@PreAuthorize` / `@PostAuthorize`	spring-security-core	SpEL rule before / after the call (`returnObject`)	§10.7
`@PreFilter` / `@PostFilter`	spring-security-core	filter a collection argument / result	§10.7
`@Secured` / `@RolesAllowed`	spring-security-core / `jakarta.annotation.security`	legacy role checks (`securedEnabled` , `jsr250Enabled`)	§10.7
`@AuthenticationPrincipal`	spring-security-core	inject the principal (`UserDetails` , `Jwt` , `OidcUser`)	§10.4
`@EnableMultiFactorAuthentication`	spring-security-config (S7)	require several `FACTOR_*` authorities	§10.4

Messaging, async and scheduling

Annotation	Module / starter	Purpose	§
`@EnableAsync`	core	switch on `@Async` (Boot configures the executor but does **not** enable it)	§14.4
`@Async("executor")`	core	run on an executor; return `void` or `CompletableFuture`	§14.4
`@EnableScheduling`	core	switch on `@Scheduled`	§14.7
`@Scheduled(fixedRate / fixedDelay / cron)`	core	timer jobs; cron has **six** fields ✔	§14.7
`@KafkaListener(topics, groupId, concurrency)`	spring-kafka / `spring-boot-starter-kafka`	Kafka consumer method	§14.12
`@RetryableTopic` / `@DltHandler`	spring-kafka	non-blocking retries via retry topics; dead-letter handler	§14.15
`@RabbitListener`	spring-rabbit / `spring-boot-starter-amqp`	RabbitMQ consumer method	§14.18
`@JmsListener`	spring-jms	JMS consumer method	§14.18
`@SchedulerLock`	ShedLock (third party)	run a `@Scheduled` job on one instance only	§14.7

Observability, cloud and AI

Annotation	Module / starter	Purpose	§
`@Observed`	micrometer-observation (+ AspectJ starter)	timer + span around a method; needs `management.observations.annotations.enabled=true`	§15.6
`@Timed` / `@Counted` / `@NewSpan`	Micrometer	metric / span annotations (same switch as `@Observed`)	§15.6
`@EnableConfigServer`	spring-cloud-config-server	make the app a Config Server	§16.3
`@RefreshScope`	spring-cloud-context	re-create the bean after a config refresh	§16.3
`@EnableEurekaServer`	spring-cloud-netflix-eureka-server	run a Eureka registry	§16.4
`@Tool` / `@ToolParam`	spring-ai-model (Spring AI 2.0)	expose a method to the model for tool calling	§17.8
`@McpTool` / `@McpToolParam`	Spring AI 2.0 MCP (`org.springframework.ai.mcp.annotation`)	expose a tool from an MCP server	§17.10

Testing annotations have their own sheet: §19.9.

19.4 Starters in Boot 4.1

A **starter** is a code-free dependency that brings one capability's libraries plus the Boot module holding its auto-configuration (§1.7). Boot 4 made them **modular**: one starter per technology, and a matching `-test` starter that brings the test slice for it and already includes `spring-boot-starter-test` (§3.14, §12.3). All names below are managed by the 4.1.1 BOM — never add a `<version>`.

The starters you will use most

Capability	Starter	Test starter (slice, test clients)	§
Core only (no web)	`spring-boot-starter`	`spring-boot-starter-test`	§1.7
Servlet web / REST (Spring MVC + Tomcat + Jackson 3)	`spring-boot-starter-webmvc`	`spring-boot-starter-webmvc-test` (`@WebMvcTest`, MockMvc, `RestTestClient`)	§5.2
Reactive web (WebFlux + Reactor Netty)	`spring-boot-starter-webflux`	`spring-boot-starter-webflux-test` (`@WebFluxTest`, `WebTestClient`)	§13.10
Blocking HTTP client (`RestClient.Builder`, HTTP service groups)	`spring-boot-starter-restclient`	`spring-boot-starter-restclient-test` (`@RestClientTest`)	§6.1
Reactive HTTP client (`WebClient.Builder`)	`spring-boot-starter-webclient`	`spring-boot-starter-webclient-test`	§13.13
JSON (Jackson 3)	`spring-boot-starter-jackson`	`spring-boot-starter-jackson-test` (`@JsonTest`)	§5.5
Bean Validation (Hibernate Validator)	`spring-boot-starter-validation`	`spring-boot-starter-validation-test`	§5.6
AOP with `@Aspect`	`spring-boot-starter-aspectj`	`spring-boot-starter-aspectj-test`	§4.2
JDBC (`DataSource`, HikariCP, `JdbcClient`)	`spring-boot-starter-jdbc`	`spring-boot-starter-jdbc-test` (`@JdbcTest`)	§7.2
JPA (Hibernate + Spring Data JPA)	`spring-boot-starter-data-jpa`	`spring-boot-starter-data-jpa-test` (`@DataJpaTest`)	§8.2
Spring Data JDBC	`spring-boot-starter-data-jdbc`	`spring-boot-starter-data-jdbc-test` (`@DataJdbcTest`)	§9.2
R2DBC	`spring-boot-starter-data-r2dbc`	`spring-boot-starter-data-r2dbc-test` (`@DataR2dbcTest`)	§13.14
MongoDB / Redis	`spring-boot-starter-data-mongodb` / `-data-redis`	`...-data-mongodb-test` / `...-data-redis-test`	§9.5, §9.6
Spring Data REST	`spring-boot-starter-data-rest`	`spring-boot-starter-data-rest-test`	§9.3
Schema migrations	`spring-boot-starter-flyway` / `-liquibase`	`...-flyway-test` / `...-liquibase-test`	§8.16, §9.4
Cache abstraction	`spring-boot-starter-cache` (+ `caffeine`)	`spring-boot-starter-cache-test`	§9.8
Security	`spring-boot-starter-security`	`spring-boot-starter-security-test`	§10.1
OAuth2 resource server / client / authorization server	`spring-boot-starter-security-oauth2-resource-server` / `-client` / `-authorization-server`	`...-test` for each	§11.3, §11.9, §11.14
Kafka	`spring-boot-starter-kafka` (+ `-jackson` for JSON)	`spring-boot-starter-kafka-test` (`@EmbeddedKafka`)	§14.10
RabbitMQ	`spring-boot-starter-amqp`	`spring-boot-starter-amqp-test`	§14.18
Actuator	`spring-boot-starter-actuator`	`spring-boot-starter-actuator-test`	§15.2
Tracing + OTLP export	`spring-boot-starter-opentelemetry` (or `-zipkin` for Brave)	`spring-boot-starter-opentelemetry-test` (`@AutoConfigureTracing`)	§15.7
Metrics only	`spring-boot-starter-micrometer-metrics` (+ a registry, e.g. `micrometer-registry-prometheus`)	`...-micrometer-metrics-test` (`@AutoConfigureMetrics`)	§15.5
HATEOAS	`spring-boot-starter-hateoas`	`...-hateoas-test`	§5.12
Testcontainers support	`spring-boot-testcontainers` (module, test scope)	—	§12.14
Also available	`-websocket`, `-graphql`, `-grpc-server` / `-grpc-client` (Boot 4.1), `-batch`, `-batch-jdbc`, `-integration`, `-quartz`, `-mail`, `-jms`, `-artemis`, `-pulsar`, `-thymeleaf`, `-session-data-redis`, `-session-jdbc`, `-jooq`, `-jetty`	most have a `-test` companion	§1.3

Renamed, deprecated and removed starters

Boot 3 name	Boot 4.1 status	Use instead	§
`spring-boot-starter-web`	**deprecated** alias (still in the BOM)	`spring-boot-starter-webmvc`	§1.7
`spring-boot-starter-web-services`	**deprecated** alias	`spring-boot-starter-webservices`	§17.13
`spring-boot-starter-oauth2-resource-server` / `-oauth2-client` / `-oauth2-authorization-server`	**deprecated** aliases	`spring-boot-starter-security-oauth2-...`	§11.3
`spring-boot-starter-aop`	**removed**	`spring-boot-starter-aspectj`	§4.2
`spring-boot-starter-undertow`	**removed** (Undertow lacks Servlet 6.1)	Tomcat (default) or `spring-boot-starter-jetty`	§17.13
`spring-boot-starter-json`	still available	`spring-boot-starter-jackson` is the Boot 4 name	§1.7
`flyway-core` / `liquibase-core` alone	no longer auto-configured	`spring-boot-starter-flyway` / `-liquibase`	§8.16
`spring-kafka` alone	still works as a library, but use the starter	`spring-boot-starter-kafka`	§14.10
part of `-web` / `-webflux` in Boot 3	split out	`spring-boot-starter-restclient`, `spring-boot-starter-webclient`	§6.1, §13.10
`spring-boot-starter-batch` (with JDBC metadata)	batch now runs without a database by default	`spring-boot-starter-batch-jdbc`	§17.13
(migration aid)	new	`spring-boot-starter-classic`, `spring-boot-starter-test-classic`: the old "everything" layout while you migrate	§3.14

Third-party starters are named `<name>-spring-boot-starter` (`mybatis-spring-boot-starter`); the `spring-boot-*` prefix is reserved (§3.13).

19.5 Key properties cheat sheet

Defaults are the Boot 4.1.1 metadata values; "—" means *no default: the feature is off, or the value comes from somewhere else*. Precedence (which source wins) and relaxed binding (several spellings of one property name, e.g. `SHOP_NAME` for `shop.name`, are accepted) are at the end of this section.

Core, profiles and startup

Property	Default	What it does	§
`spring.application.name`	—	name in logs, traces, Config Server, metrics	§1.9
`spring.profiles.active` / `.default` / `.include`	— / `default` / —	activate profiles (not allowed inside profile-specific files)	§3.7
`spring.profiles.group.<name>`	—	one name activates several profiles	§3.7
`spring.config.import`	—	extra files, `optional:`, `configtree:`, `configserver:`	§3.8
`spring.config.activate.on-profile`	—	make a document of a multi-document file conditional	§3.2
`spring.autoconfigure.exclude`	—	switch one auto-configuration off	§3.12
`debug`	`false`	print the condition evaluation report	§3.11
`spring.main.lazy-initialization`	`false`	make every bean lazy	§2.10
`spring.main.allow-circular-references`	`false`	Boot rejects field/setter cycles (since 2.6)	§2.9
`spring.main.allow-bean-definition-overriding`	`false`	two beans with one name → `BeanDefinitionOverrideException`	§2.3
`spring.main.web-application-type`	detected	`servlet`, `reactive`, `none` (MVC wins if both starters present)	§13.10
`spring.main.keep-alive`	`false`	keep the JVM alive when only virtual (daemon) threads run	§14.5
`spring.aop.proxy-target-class`	`true`	Boot uses CGLIB proxies for everything	§4.4
`spring.aop.auto`	`true`	`@EnableAspectJAutoProxy` applied by Boot	§4.2
`spring.lifecycle.timeout-per-shutdown-phase`	`30s` ✔	how long graceful shutdown waits	§15.9
`spring.devtools.restart.enabled`	`true`	DevTools restart (off for `java -jar` and tests)	§1.10
`spring.devtools.livereload.enabled`	`false`	LiveReload — off in Boot 4.0, deprecated in 4.1	§1.10

Web server, MVC and JSON

Property	Default	What it does	§
`server.port`	`8080`	HTTP port; `0` = random free port	§1.9
`server.shutdown`	`graceful`	`immediate` turns graceful shutdown off (default since Boot 3.4)	§15.9
`server.servlet.context-path`	—	URL prefix of the whole app	§5.2
`server.forward-headers-strategy`	—	`framework` honours `X-Forwarded-*` behind a proxy	§5.4
`server.servlet.session.timeout`	`30m`	HTTP session timeout	§10.10
`server.servlet.session.cookie.secure` / `.http-only` / `.same-site`	—	harden the session cookie	§10.10
`spring.web.error.include-stacktrace`	`never`	stack traces in Boot's error body (Boot 4 name; `server.error.*` no longer bound)	§15.12
`spring.mvc.problemdetails.enabled`	`false`	RFC 9457 bodies for MVC exceptions without your own advice	§5.7
`spring.webflux.problemdetails.enabled`	`false`	the same for WebFlux	§13.10
`spring.mvc.contentnegotiation.favor-parameter`	`false`	allow `?format=csv`	§5.8
`spring.servlet.multipart.max-file-size` / `max-request-size`	`1MB` / `10MB`	upload limits (413 when exceeded)	§5.10
`spring.jackson.default-property-inclusion`	—	e.g. `non_null`	§5.5
`spring.jackson.serialization.*` / `deserialization.*`	—	Jackson feature switches (`indent-output`...)	§5.5
`spring.jackson.json.read.*` / `spring.jackson.json.write.*`	—	JSON parser/generator features (Boot 4; were `spring.jackson.parser/generator.*`)	§5.5
`spring.jackson.use-jackson2-defaults`	`false`	restore Jackson 2 defaults while migrating	§17.13
`spring.threads.virtual.enabled`	`false`	virtual threads for Tomcat, `@Async`, `@Scheduled`, listeners (Java 21+) ✔	§15.10

Jackson 3 defaults under Boot 4 (✔ proved): `LocalDate` written as `"2026-09-28"`, unknown JSON properties ignored, `JacksonException` is unchecked (§5.5).

API versioning (F7 / Boot 4)

Property	Meaning	§
`spring.mvc.apiversion.use.header`	read the version from this header (e.g. `X-API-Version`)	§5.9
`spring.mvc.apiversion.use.query-parameter` / `.use.path-segment` / `.use.media-type-parameter`	other places to read it	§5.9
`spring.mvc.apiversion.default`	version assumed when the request has none	§5.9
`spring.mvc.apiversion.required`	reject requests without a version (400)	§5.9
`spring.mvc.apiversion.supported` / `.detect-supported`	extra supported versions / detect them from mappings	§5.9
`spring.webflux.apiversion.*`	the same for WebFlux	§17.13
`spring.http.serviceclient.<group>.apiversion.insert.header`	client side: send the version in this header	§6.7

HTTP clients

Property	Default	What it does	§
`spring.http.clients.connect-timeout` / `read-timeout`	— (library default)	global timeouts for auto-configured clients (Boot 4 name; `spring.http.client.*` deprecated)	§6.4
`spring.http.clients.imperative.factory`	detected	`http-components`, `jetty`, `reactor`, `jdk`, `simple`	§6.4
`spring.http.clients.reactive.connector`	detected	the same for `WebClient`	§6.4
`spring.http.clients.redirects`	—	`follow`, `dont-follow`, `follow-when-possible`	§6.4
`spring.http.serviceclient.<group>.base-url`	—	base URL of an `@ImportHttpServices` group (none + Spring Cloud = load-balanced)	§6.6, §16.6
`spring.http.serviceclient.<group>.read-timeout` / `.default-header.<name>`	—	per-group timeout / header	§6.6

DataSource, JDBC, transactions

Property	Default	What it does	§
`spring.datasource.url` / `.username` / `.password`	—	connection; no URL + embedded DB on the classpath → in-memory DB ✔	§7.2
`spring.datasource.generate-unique-name`	`true`	random in-memory DB name per context ✔	§7.2
`spring.datasource.hikari.maximum-pool-size`	`10` (HikariCP) ✔	most connections	§7.2
`spring.datasource.hikari.minimum-idle`	= max	leave unset: fixed-size pool	§7.2
`spring.datasource.hikari.connection-timeout`	`30000` ms ✔	wait for a free connection, then fail	§7.2
`spring.datasource.hikari.max-lifetime` / `idle-timeout` / `keepalive-time`	30 min / 10 min / 2 min	connection recycling	§7.2
`spring.datasource.hikari.leak-detection-threshold`	`0` (off)	log connections held too long	§7.2
`spring.datasource.connection-fetch`	`eager`	`lazy` = no physical connection until the first statement (Boot 4.1)	§7.2
`spring.sql.init.mode`	`embedded`	run `schema.sql` / `data.sql`: `embedded`, `always`, `never`	§7.4
`spring.transaction.default-timeout`	—	default transaction timeout	§7.12
`spring.persistence.exceptiontranslation.enabled`	`true`	`@Repository` exception translation (was `spring.dao.*`)	§7.5

JPA, Hibernate, migrations

Property	Default	What it does	§
`spring.jpa.hibernate.ddl-auto`	`create-drop` for an embedded DB without a migration tool, else `none`	`validate` in production	§8.2
`spring.jpa.open-in-view`	`true` (+ startup warning)	set `false` for APIs	§8.15
`spring.jpa.show-sql`	`false`	print SQL (`logging.level.org.hibernate.SQL=DEBUG` is better)	§8.2
`spring.jpa.defer-datasource-initialization`	`false`	run `data.sql` after Hibernate created the tables	§7.4
`spring.jpa.properties.hibernate.default_batch_fetch_size`	—	N+1 safety net (e.g. `16`)	§8.7
`spring.jpa.properties.hibernate.jdbc.batch_size` / `order_inserts`	—	batched inserts (needs SEQUENCE ids)	§8.18
`spring.jpa.properties.*`	—	passed to Hibernate unchanged	§8.2
`spring.flyway.enabled` / `.locations`	`true` / `classpath:db/migration`	Flyway (needs `spring-boot-starter-flyway`)	§9.4
`spring.flyway.baseline-on-migrate` / `.clean-disabled`	`false` / `true`	adopt an existing DB / keep `clean` disabled	§9.4
`spring.liquibase.change-log`	`classpath:/db/changelog/db.changelog-master.yaml`	Liquibase master changelog	§9.4
`spring.data.rest.base-path` / `.detection-strategy`	— / `default`	Data REST prefix / `annotated` = only `@RepositoryRestResource`	§9.3
`spring.mongodb.uri`	`mongodb://localhost/test`	MongoDB connection (Boot 4; `spring.data.mongodb.uri` is no longer bound)	§9.5
`spring.data.redis.host` / `.port`	`localhost` / `6379`	Redis connection	§9.6
`spring.r2dbc.url`	—	reactive SQL connection	§13.14

Cache

Property	Default	What it does	§
`spring.cache.type`	detected (Redis before Caffeine before Simple)	force a provider, or `none` in tests	§9.8
`spring.cache.cache-names`	—	fixed set of caches (typos fail)	§9.8
`spring.cache.caffeine.spec`	—	e.g. `maximumSize=500,expireAfterWrite=10m`	§9.8
`spring.cache.redis.time-to-live`	— (no expiry)	TTL for Redis caches	§9.8
`spring.cache.redis.cache-null-values` / `.key-prefix`	`true` / —	cache `null`s / key prefix (version it)	§9.8

Security

Property	Default	What it does	§
`spring.security.user.name` / `.password` / `.roles`	`user` / random UUID (logged) / —	the dev-only generated user	§10.1
`spring.security.oauth2.resourceserver.jwt.issuer-uri`	—	discover JWKS, check `iss`	§11.3
`spring.security.oauth2.resourceserver.jwt.jwk-set-uri` / `.public-key-location`	—	keys without discovery	§11.3
`spring.security.oauth2.resourceserver.jwt.audiences`	—	check `aud` (do it!)	§11.3
`spring.security.oauth2.resourceserver.jwt.authorities-claim-name` / `.authority-prefix` / `.principal-claim-name`	— (`scope` / `scp` → `SCOPE_`)	map roles from a claim without code	§11.5
`spring.security.oauth2.resourceserver.opaquetoken.introspection-uri` / `.client-id` / `.client-secret`	—	opaque tokens	§11.7
`spring.security.oauth2.client.registration.<id>.*` / `.provider.<id>.issuer-uri`	—	OAuth2 login / client credentials	§11.9, §11.10

Actuator, health, metrics, tracing, logging

Property	Default	What it does	§
`management.endpoints.web.exposure.include` / `.exclude`	`health`	which endpoints are reachable over HTTP (`*` = all)	§15.2
`management.endpoints.web.base-path`	`/actuator`	URL prefix	§15.2
`management.endpoints.access.default` / `management.endpoint.<id>.access`	per endpoint (`shutdown` = `none`)	`none`, `read-only`, `unrestricted` (since 3.4)	§15.2
`management.server.port`	— (same port)	separate management port	§15.2
`management.endpoint.health.show-details`	`never`	`when-authorized`, `always`	§15.3
`management.endpoint.health.probes.enabled`	`true` (Boot 4)	`/actuator/health/liveness` and `/readiness`	§15.3
`management.endpoint.health.group.<name>.include`	—	custom health group	§15.3
`management.endpoint.env.show-values`	`never`	mask values in `env` / `configprops`	§15.13
`management.info.env.enabled` / `.java.enabled`	`false` / `false`	info contributors	§15.4
`management.metrics.tags.<key>`	—	common tag on every meter	§15.5
`management.metrics.distribution.percentiles-histogram.<meter>`	—	histogram buckets for p95/p99	§15.5
`management.observations.annotations.enabled`	`false`	activate `@Observed`, `@Timed`, `@Counted`, `@NewSpan`	§15.6
`management.observations.enable.<prefix>`	—	switch observation families off	§15.6
`management.tracing.sampling.probability`	`0.1`	share of traces recorded	§15.7
`management.tracing.export.enabled`	`true`	export spans (was `management.tracing.enabled`)	§17.13
`management.opentelemetry.tracing.export.otlp.endpoint`	—	OTLP collector URL (was `management.otlp.tracing.endpoint`)	§15.7
`management.tracing.propagation.type`	W3C	`b3` for legacy Zipkin services	§16.9
`management.tracing.baggage.remote-fields` / `.correlation.fields`	—	baggage in headers / in the MDC	§15.7
`logging.level.<logger>`	`INFO` root	per-logger level (`POST /actuator/loggers/...` at runtime)	§15.2
`logging.structured.format.console` / `.file`	—	`ecs`, `logstash`, `gelf` JSON logs (Boot 3.4+)	§15.8
`logging.pattern.correlation`	Boot's `[traceId-spanId]`	how trace ids appear in text logs	§15.7

Threads, async, scheduling, Kafka, RabbitMQ

Property	Default	What it does	§
`spring.task.execution.pool.core-size`	`8` ✔	`applicationTaskExecutor` threads (`@Async`)	§14.4
`spring.task.execution.pool.queue-capacity` / `.max-size`	unbounded ✔ / unlimited	bound the queue!	§14.4
`spring.task.execution.thread-name-prefix`	`task-` ✔	thread names	§14.4
`spring.task.execution.mode`	`auto`	`force` keeps Boot's executor next to your own `Executor` beans	§14.4
`spring.task.execution.propagate-context`	`false`	copy tracing/MDC context into `@Async` threads	§14.4
`spring.task.execution.simple.concurrency-limit`	–	cap for the virtual-thread executor	§14.5
`spring.task.execution.shutdown.await-termination`	`false`	finish queued tasks on shutdown	§14.4
`spring.task.scheduling.pool.size`	`1` ✔	scheduler threads for all `@Scheduled` methods	§14.7
`spring.reactor.context-propagation`	`limited`	`auto` bridges ThreadLocals and the Reactor `Context`	§13.9
`spring.kafka.bootstrap-servers`	`localhost:9092`	brokers (tests: `${spring.embedded.kafka.brokers}`)	§14.10
`spring.kafka.consumer.group-id` / `.auto-offset-reset`	– / Kafka's `latest`	consumer group / where a new group starts	§14.10
`spring.kafka.consumer.isolation-level`	`read-uncommitted`	`read-committed` to hide aborted transactions	§14.16
`spring.kafka.listener.ack-mode` / `.concurrency`	`BATCH` (container default) / 1	offset commits / consumer threads	§14.12
`spring.kafka.producer.acks` / `.transaction-id-prefix`	Kafka's `all` / –	durability / transactional template	§14.16
`spring.kafka.template.observation-enabled` / `listener.observation-enabled`	`false` / `false`	tracing through Kafka headers	§16.9
`spring.rabbitmq.host` / `.listener.simple.prefetch`	`localhost` / –	broker / unacknowledged messages per consumer	§14.18
`spring.rabbitmq.listener.simple.retry.max-retries`	`3`	retries **after** the first attempt (Boot 4; replaces `max-attempts`)	§14.18

Precedence and relaxed binding (✔ proved)

Highest wins (§3.3, §12.15):

1. In tests: `@DynamicPropertySource` > `@TestPropertySource` > `@SpringBootTest(properties = ...)`.
2. Command-line arguments `--shop.name=...`
3. `SPRING_APPLICATION_JSON`
4. Java system properties `-Dshop.name=...`
5. OS environment variables `SHOP_NAME=...`
6. `random.*`
7. Profile-specific files `application-{profile}.*` (outside the jar > inside)
8. `application.properties` / `.yaml` (outside the jar > inside; an imported file > its importer)
9. `@PropertySource` files
10. `SpringApplication.setDefaultProperties(...)`

Rule	Example	§
env var name = upper case, `.` → `_`, drop `-`	`SHOP_CHECKOUTTIMEOUT=2m` → `shop.checkout-timeout` ✔	§3.4
list index	`SHOP_CURRENCIES_0=USD` → `shop.currencies[0]` ✔	§3.4
legacy form (`-` → `_`) is also accepted by the binder	`SHOP_CHECKOUT_TIMEOUT=3m` also binds ✔ – prefer the documented form	§3.4
`Duration`	`45s`, `5m`, `PT1H`; plain `500` = milliseconds ✔	§3.6
`DataSize`	`10MB` = 10 485 760 bytes; plain `1024` = bytes ✔	§3.6
`@Value`	`${key}` fails if missing; `${key:default}`; `#{${key:3} + 1}` = SpEL ✔	§3.5

19.6 Transactions: `@Transactional` at a glance

A Spring transaction = a **proxy** + a `PlatformTransactionManager` + a connection **bound to the current thread** (stored per thread, so every data-access call on that thread reuses the same connection) (§7.7). Everything below applies only to calls that go **through the proxy** (§7.13).

Attributes and their defaults (✔ proved)

Attribute	Default	Meaning	§
`propagation`	`REQUIRED`	what to do when a transaction already exists (table below)	§7.9
`isolation`	`DEFAULT` = the database's default	only applied when this method **starts** the transaction	§7.11
`readOnly`	`false`	JDBC: read-only connection (PostgreSQL rejects writes); JPA: no dirty checking/flush; can route to a replica	§7.12
`timeout`	`-1` (none)	seconds; enforced on the next SQL statement	§7.12
`rollbackFor` / `noRollbackFor`	–	add / remove exception types from the rollback rule	§7.10
`transactionManager` (`value`)	the primary manager	pick one of several managers	§7.8
`label`	–	free-form labels for custom managers	§7.8

Where: on the **service method** that is one use case. Not on controllers (too wide), not only on repositories (too narrow). A class-level `@Transactional(readOnly = true)` + method-level `@Transactional` on writes is the Spring Data pattern (§7.8).

Propagation (✔ all seven proved)

Propagation	A transaction exists	None exists	Typical use
`REQUIRED` (default)	**join** it (same connection)	start one	almost everything
`REQUIRES_NEW`	**suspend** it, start an independent one on a **second connection**; commits even if the caller later rolls back	start one	audit log that must survive a rollback
`NESTED`	**savepoint** inside it; an inner failure rolls back to the savepoint only	start one	"try this part" (JDBC; avoid with JPA)
`MANDATORY`	join it	**throw** `IllegalTransactionStateException`	must be called inside a transaction
`SUPPORTS`	join it	run **without** a transaction	read helpers that don't care
`NOT_SUPPORTED`	**suspend** it, run without	run without	slow non-transactional work inside a flow
`NEVER`	**throw** `IllegalTransactionStateException`	run without	guard: never inside a transaction

The trap ✔: an inner `REQUIRED` method throws, the outer method **catches** the exception and returns normally → the shared transaction was marked **rollback-only** → Spring rolls back at commit and throws `UnexpectedRollbackException` (§7.9).

Rollback rules (✔ proved)

Thrown out of the method	Default	`rollbackFor = X`	`noRollbackFor = X`	`rollbackOn = ALL_EXCEPTIONS` (F6.2)
`RuntimeException` / `Error`	**rollback**	rollback	commit if it is an X	rollback
checked `Exception`	**commit** (!)	rollback if it is an X	commit	rollback
caught **inside** the method	commit — the proxy never sees it	commit	commit	commit

Global switch: `@EnableTransactionManagement(proxyTargetClass = true, rollbackOn = RollbackOn.ALL_EXCEPTIONS)` (§7.10). To mark a transaction without throwing: `TransactionAspectSupport.currentTransactionStatus().setRollbackOnly()` or `status.setRollbackOnly()` in a `TransactionTemplate`.

Isolation levels

Isolation	Prevents	Notes
`DEFAULT`	whatever the database default prevents	PostgreSQL, Oracle, SQL Server: READ COMMITTED; MySQL InnoDB: REPEATABLE READ
`READ_UNCOMMITTED`	nothing guaranteed (dirty reads possible)	PostgreSQL treats it as READ COMMITTED
`READ_COMMITTED`	dirty reads	non-repeatable reads and lost updates still possible (proved in §7.11)
`REPEATABLE_READ`	+ non-repeatable reads	PostgreSQL: one snapshot; concurrent update → serialization failure (`40001`) → retry the whole transaction
`SERIALIZABLE`	+ phantoms, write skew	expect serialization failures; always retry

The anomalies themselves (wrong results that concurrent transactions can produce, such as dirty reads, non-repeatable reads and lost updates): SQL 13.4.

Transaction managers and helpers

You use	Manager Boot creates	§
JDBC (`spring-boot-starter-jdbc`)	`JdbcTransactionManager` (a `DataSourceTransactionManager`)	§7.7
JPA (`spring-boot-starter-data-jpa`)	`JpaTransactionManager` (JDBC on the same `DataSource` joins it)	§7.7
R2DBC	`R2dbcTransactionManager` (context-bound, not thread-bound)	§13.14
Kafka with `spring.kafka.producer.transaction-id-prefix`	`KafkaTransactionManager`	§14.16
Programmatic	`TransactionTemplate.execute(status -> ...)` — no proxy, no self-invocation problem	§7.14
After-commit side effects	`@TransactionalEventListener` (`AFTER_COMMIT` default; skipped without a transaction unless `fallbackExecution = true` ; a transactional listener must use `REQUIRES_NEW`)	§7.15

"Why is my `@Transactional` not working?"

Symptom	Cause	§
partial writes after an exception	self-invocation (`this.save()`), `private` method, bean made with `new`	§7.13, §4.5
`NullPointerException` inside a `final` `@Transactional` method	the method ran on the CGLIB proxy instance, whose fields are `null`	§4.5
data committed although the method threw	checked exception without `rollbackFor` , or exception caught inside	§7.10
`UnexpectedRollbackException`	inner `REQUIRED` failure was caught	§7.9
`isolation` / `timeout` ignored	the method joined an existing transaction	§7.11
works in tests, fails in production	the test was `@Transactional` and supplied the transaction	§12.7
`@Async` code doesn't see uncommitted data	transactions are bound to one thread	§7.13
pool exhausted under load	`REQUIRES_NEW` holds two connections; remote calls inside transactions	§7.9, §7.13

19.7 Bean scopes and the lifecycle

Scopes (✔ singleton and prototype proved)

Scope	Instances	Destroy callbacks?	Declare with	§
`singleton` (default)	one per container, shared by all threads — keep it **stateless**	yes	nothing	§2.6
`prototype`	a new one per `getBean`/injection	**no** — Spring forgets it	`@Scope("prototype")`	§2.6
`request`	one per HTTP request	yes	`@RequestScope` (scoped proxy)	§2.6
`session`	one per HTTP session	yes	`@SessionScope`	§2.6
`application`	one per `ServletContext`	yes	`@ApplicationScope`	§2.6
`websocket`	one per WebSocket session	yes	`@Scope("websocket")`	§2.6
refresh (Spring Cloud)	re-created after a config refresh	yes	`@RefreshScope`	§16.3

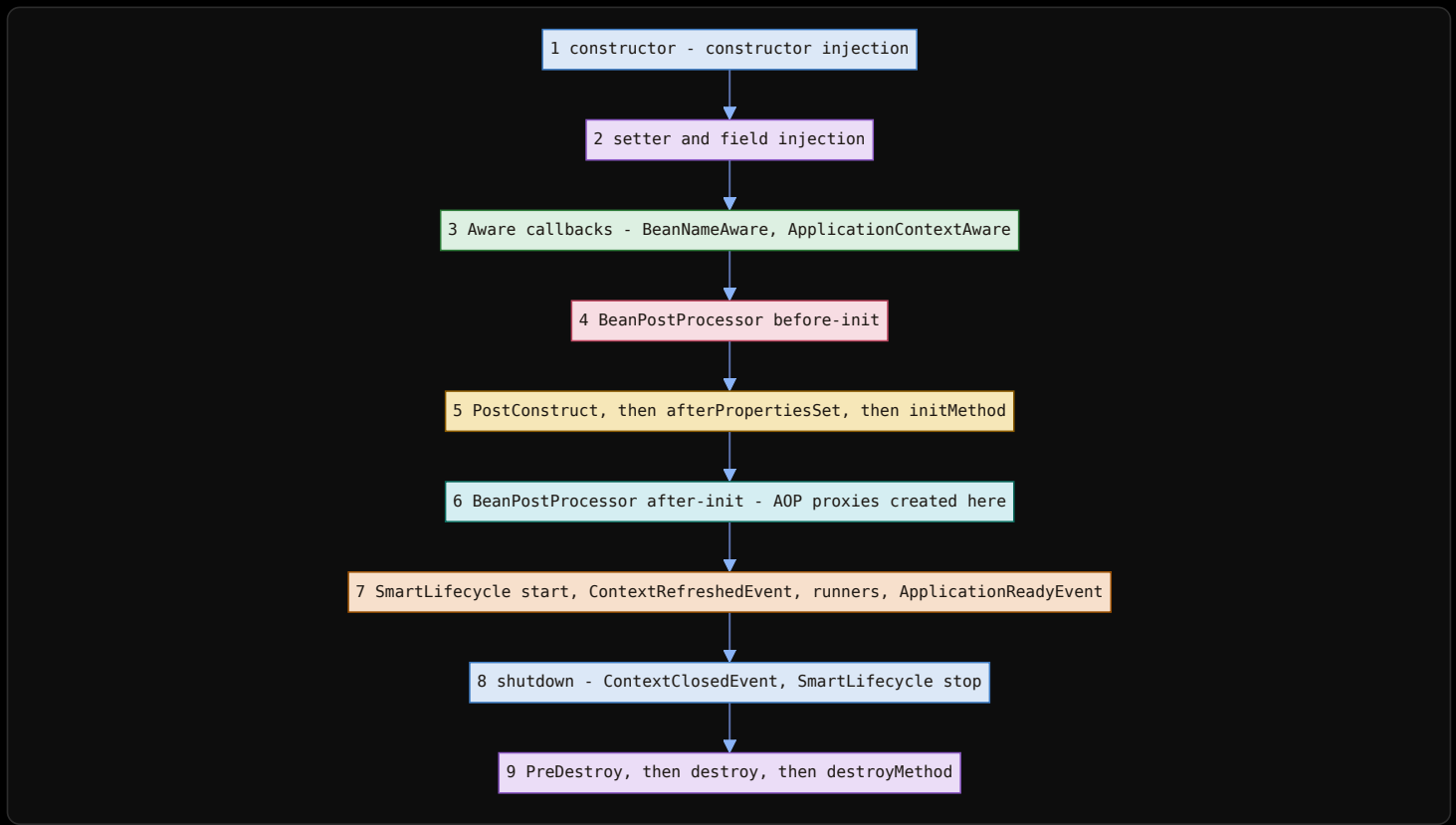
A prototype injected into a singleton is created **once** and stays ✔. For a fresh one each time: `ObjectProvider<T>.getObject()`, a `@Lookup` method, or a scoped proxy (§2.6).

How Spring picks one bean among several (✔ proved)

1. `@Qualifier("name")` on the injection point
2. `@Primary` bean
3. the only non-`@Fallback` bean (F6.2)
4. a bean whose name equals the parameter name (needs `-parameters` — a compiler flag that keeps parameter names in the bytecode — which the Boot parent sets)
5. otherwise `NoUniqueBeanDefinitionException`

`List<T>` gets all beans ordered by `@Order` (lowest first); `Map<String, T>` is keyed by bean name. Optional dependencies: `Optional<T>`, `ObjectProvider<T>`, `@Nullable`, `@Autowired(required = false)` (§2.5).

Lifecycle order (✔ proved)



Proved order: init `[constructor, @PostConstruct, afterPropertiesSet, initMethod]`, destroy `[@PreDestroy, destroy(), destroyMethod]`, and no `@PreDestroy` for the prototype.

Hook	Use for	§
`@PostConstruct` / `@PreDestroy`	default choice: validate, warm up, release	§2.7
`InitializingBean` / `DisposableBean`	framework code (couples you to Spring)	§2.7
`@Bean(initMethod, destroyMethod)`	third-party classes; a public `close()` / `shutdown()` is inferred	§2.7
`SmartLifecycle` (`getPhase()`)	things that *run*: pollers, listeners (start low phase first, stop it last)	§2.7
`CommandLineRunner` / `ApplicationRunner`	one-off startup work with the arguments	§1.8
`@EventListener(ApplicationReadyEvent.class)`	"after the app is fully started"	§2.7
`BeanFactoryPostProcessor` (declare `static`)	change bean **definitions** before any bean exists	§2.8
`BeanPostProcessor` (declare `static`)	change or wrap every bean **instance**	§2.8

Pitfall: a `@Transactional` call from `@PostConstruct` has no transaction — the proxy is created in step 6, after step 5, and the call is on `this` anyway (§2.7).

19.8 Spring Security 7 DSL snippets

Security 7 accepts only the **lambda DSL** (domain-specific language: configuration as method calls taking lambdas `x -> ...`): one lambda per feature, no `.and()`, `authorizeHttpRequests` (not `authorizeRequests`), path matchers built on `PathPattern` (§10.3). The four snippets below are compiled by every build of `SecuritySnippets.java`, so each DSL call exists in Security 7.1.1. Rules are checked **top to bottom, first match wins** (§10.6).

Form login + HTTP Basic (browser app)

```
@Bean
SecurityFilterChain web(HttpSecurity http) throws Exception {
    http.authorizeHttpRequests(auth -> auth                    // first matching rule wins
            .requestMatchers("/", "/public/**", "/login").permitAll()
            .requestMatchers("/admin/**").hasRole("ADMIN")        // checks authority ROLE_ADMIN
            .requestMatchers(HttpMethod.GET, "/reports/**").hasAuthority("report:read")
            .anyRequest().authenticated())                        // or denyAll() to fail closed
        .formLogin(form -> form.loginPage("/login").permitAll()) // omit loginPage for the generated page
        .httpBasic(Customizer.withDefaults())
        .logout(Customizer.withDefaults());                       // POST /logout (+ CSRF token)
    return http.build();
}

@Bean
PasswordEncoder passwordEncoder() {                               // {bcrypt} by default
    return PasswordEncoderFactories.createDelegatingPasswordEncoder();
}

@Bean
static RoleHierarchy roleHierarchy() {                            // admins also pass hasRole("USER")
    return RoleHierarchyImpl.fromHierarchy("ROLE_ADMIN > ROLE_USER");
}
```

Plus a `UserDetailsService` bean (in-memory in the snippet; JPA/JDBC-backed in real apps). Defining it (or any `SecurityFilterChain`) makes Boot's generated user and default chain back off (§10.1).

JWT resource server (stateless API, first of two chains)

```
@Bean
@Order(1)                                                         // specific chain first
SecurityFilterChain api(HttpSecurity http) throws Exception {
    http.securityMatcher("/api/**")                               // which requests this chain owns
        .authorizeHttpRequests(auth -> auth
            .requestMatchers(HttpMethod.GET, "/api/**").hasAuthority("SCOPE_orders.read")
            .requestMatchers(HttpMethod.POST, "/api/**").hasAuthority("SCOPE_orders.write")
            .anyRequest().authenticated())
        .oauth2ResourceServer(rs -> rs.jwt(Customizer.withDefaults()))   // issuer-uri from properties
        .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
        .csrf(csrf -> csrf.disable());                            // no cookies on this chain
    return http.build();
}
```

```
spring.security.oauth2.resourceserver.jwt.issuer-uri=https://idp.example.com/realms/shop
spring.security.oauth2.resourceserver.jwt.audiences=orders-api
# optional: roles from a claim, without writing a converter
spring.security.oauth2.resourceserver.jwt.authorities-claim-name=roles
spring.security.oauth2.resourceserver.jwt.authority-prefix=ROLE_
```

Note the last two lines replace the default `scope` → `SCOPE_` mapping; to keep both, write a converter that combines two `JwtGrantedAuthoritiesConverter`s (§11.5). The browser chain (no `securityMatcher`) gets `@Order(2)` — catch-all chains go last (§10.3).

Method security

```
@Configuration
@EnableMethodSecurity                                          // @PreAuthorize & co. (proxy-based)
class MethodSecurityConfig { }

public class OrderService {
    @PreAuthorize("hasRole('ADMIN') or #owner == authentication.name")
    public List<Order> ordersOf(String owner) { ... }

    @PostAuthorize("returnObject.owner() == authentication.name")
    public Order find(long id) { ... }

    @PreAuthorize("hasAuthority('order:refund')")
    public String refund(long id) { ... }
}
```

Proxy rules apply: self-invocation and `private` methods are **not** protected (§10.7).

CORS and CSRF for a cookie-based single-page app

```
@Bean
SecurityFilterChain spa(HttpSecurity http) throws Exception {
    http.authorizeHttpRequests(auth -> auth.anyRequest().authenticated())
        .cors(Customizer.withDefaults())         // uses the CorsConfigurationSource bean below
        .csrf(csrf -> csrf.spa())                // Security 7: XSRF-TOKEN cookie + X-XSRF-TOKEN header
        .formLogin(Customizer.withDefaults());
    return http.build();
}

@Bean
CorsConfigurationSource corsConfigurationSource() {
    CorsConfiguration api = new CorsConfiguration();
    api.setAllowedOrigins(List.of("https://app.example.com"));   // exact origins, never "*" with credentials
    api.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE"));
    api.setAllowedHeaders(List.of("Content-Type", "X-XSRF-TOKEN"));
    api.setAllowCredentials(true);
    api.setMaxAge(3600L);
    UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
    source.registerCorsConfiguration("/api/**", api);
    return source;
}
```

The rules in one table

Fact	Detail	§
`hasRole("ADMIN")`	checks the authority `ROLE_ADMIN` ✔; `hasRole("ROLE_ADMIN")` throws `IllegalArgumentException` ✔	§10.6
`hasAuthority("x")`	checks the literal string (`report:read` , `SCOPE_orders.read`)	§10.6
JWT scopes	`scope` / `scp` claim → `SCOPE_x` authorities	§11.5
401 vs 403	anonymous + denied → 401 (or login redirect); authenticated + denied → 403	§10.6
JWT failures	bad/expired token → 401 `invalid_token`; missing scope → 403 `insufficient_scope`; 60 s clock skew	§11.3
CSRF	on for unsafe methods; disable only on chains without cookies/cached Basic	§10.8
CORS	`http.cors(...)` so preflights are answered before authentication	§10.9
sessions	`IF_REQUIRED` (default), `ALWAYS` , `NEVER` , `STATELESS` ; session id changes at login	§10.10
passwords	`DelegatingPasswordEncoder` : `{bcrypt}` by default ✔, reads `{noop}` , `{argon2...}` ; `upgradeEncoding` → re-hash on login ✔	§10.5
MFA (S7)	every login adds a `FACTOR_*` authority (`FACTOR_PASSWORD` , `FACTOR_OTT` , `FACTOR_WEBAUTHN`)	§10.4
removed in S7	`.and()` , `authorizeRequests` , `AntPathRequestMatcher` / `MvcRequestMatcher` , OAuth2 password grant client support	§17.15
security context	`SecurityContextHolder` is a `ThreadLocal` : use `DelegatingSecurityContext*` executors for `@Async`	§10.2

19.9 Testing annotations and slices

Whole-application tests

Annotation / type	Module / test starter	Purpose	§
`@SpringBootTest`	spring-boot-test (`spring-boot-starter-test`)	full context; `webEnvironment = MOCK` (default), `RANDOM_PORT`, `DEFINED_PORT`, `NONE`	§12.6
`@AutoConfigureMockMvc`	`-webmvc-test`	`MockMvc` + `MockMvcTester` in a `@SpringBootTest` (no longer automatic in Boot 4)	§12.6
`@AutoConfigureRestTestClient`	spring-boot-resttestclient (in `-webmvc-test`)	inject `RestTestClient` (F7), bound to MockMvc or the running server	§12.9
`@AutoConfigureWebTestClient`	`-webflux-test`	`WebTestClient` in a `@SpringBootTest`	§13.12
`@AutoConfigureTestRestTemplate`	spring-boot-resttestclient	legacy `TestRestTemplate`	§12.9
`@LocalServerPort`	spring-boot-test	the random port	§1.12
`@TestConfiguration`	spring-boot-test	extra beans; nested static = added, top-level = `@Import` it	§12.6
`@ActiveProfiles`	spring-test	activate profiles (part of the cache key)	§12.15
`@TestPropertySource` / `properties = ...`	spring-test / spring-boot-test	test-only properties	§12.15
`@DynamicPropertySource`	spring-test	runtime values via `Supplier`s; highest precedence	§12.12
`@DirtiesContext`	spring-test	close and evict the cached context (avoid)	§12.11
`@Sql`	spring-test	run SQL scripts before/after a test	§12.16
`@Transactional` (on a test)	spring-test	rolled back after each test — can hide transaction bugs	§12.7, §12.17

Slices

Slice	Test starter	Loads	Rolled back?	Extras	§
`@WebMvcTest(Controller.class)`	`spring-boot-starter-webmvc-test`	MVC, Jackson, validation, security filters, your controllers/advice/filters	no	`MockMvcTester`, `MockMvc`	§12.7
`@WebFluxTest`	`spring-boot-starter-webflux-test`	WebFlux, codecs, controllers	no	`WebTestClient`	§13.10
`@DataJpaTest`	`spring-boot-starter-data-jpa-test`	JPA, repositories, embedded DB, Flyway/Liquibase	**yes**	`TestEntityManager`, SQL shown	§12.7
`@JdbcTest`	`spring-boot-starter-jdbc-test`	`DataSource`, `JdbcClient`, `JdbcTemplate`	**yes**	—	§7.6
`@DataJdbcTest`	`spring-boot-starter-data-jdbc-test`	Spring Data JDBC + Flyway	**yes**	—	§9.2
`@DataR2dbcTest`	`spring-boot-starter-data-r2dbc-test`	R2DBC repositories	**no**	—	§13.14
`@JsonTest`	`spring-boot-starter-jackson-test`	Jackson `JsonMapper` with your customisations	no	`JacksonTester`	§12.7
`@RestClientTest(Client.class)`	`spring-boot-starter-restclient-test`	`RestClient.Builder`, Jackson, the named client	no	`MockRestServiceServer`	§6.8
`@DataMongoTest` / `@DataRedisTest`	`...-data-mongodb-test` / `...-data-redis-test`	the store's repositories	no	—	§9.7
`@AutoConfigureTestDatabase(replace = ...)`	`spring-boot-starter-jdbc-test`	`NON_TEST` (default since 3.4) replaces your DB unless it is already a test DB; `NONE` keeps it	—	—	§12.7

Mocks, containers, security helpers

Annotation / helper	Module	Purpose	§
`@MockitoBean`	spring-test (F6.2)	Mockito mock **in the context** (replaced removed `@MockBean`); reset after each test	§12.10
`@MockitoSpyBean`	spring-test	spy around the real bean (replaced `@SpyBean`)	§12.10
`@TestBean`	spring-test	override a bean with a static factory method's result	§12.10
`@Mock` + `@ExtendWith(MockitoExtension.class)`	Mockito	plain unit test, no Spring	§12.4
`@Testcontainers` / `@Container`	Testcontainers 2	start Docker containers for the test	§12.14
`@ServiceConnection`	spring-boot-testcontainers	take connection details from the container (no URL properties)	§12.14
`@EmbeddedKafka`	spring-kafka-test (`-kafka-test`)	in-JVM KRaft broker; sets `spring.embedded.kafka.brokers`	§14.13
`@AutoConfigureMetrics` / `@AutoConfigureTracing`	`-micrometer-metrics-test` / `-opentelemetry-test`	keep metric export / span export on in tests	§15.5, §15.7
`@WithMockUser` / `@WithUserDetails` / `@WithAnonymousUser`	spring-security-test	identity for a test method	§10.12
`user()`, `httpBasic()`, `csrf()`, `formLogin()`, `logout()`	spring-security-test	request post-processors / builders	§10.12
`jwt()`, `opaqueToken()`, `oauth2Login()`, `oidcLogin()`	spring-security-test	token identities — pass your converter to `jwt()`	§11.6
`ApplicationContextRunner`	spring-boot-test	tiny context per test, for auto-configuration and conditions	§3.13
`StepVerifier` (+ `withVirtualTime`)	reactor-test	verify `Mono`/`Flux` signals	§13.5
Awaitility `await().atMost(...)`	awaitility (in `spring-boot-starter-test`)	wait for asynchronous results, never `Thread.sleep`	§14.2

Test clients and the context cache

Client	Stack	Needs a server?	§
`MockMvcTester` (F6.2, AssertJ style)	MVC	no	§12.8
`RestTestClient` (F7)	MVC (or any server with `bindToServer`)	optional	§12.9
`WebTestClient`	WebFlux (also MockMvc)	optional	§13.12
`TestRestTemplate`	any	yes	§12.9

Context cache: key = the merged configuration (classes, profiles, properties, `@MockitoBean`s, `@DynamicPropertySource`, `@AutoConfigure...`, `webEnvironment`); **max 32** contexts, LRU eviction (`spring.test.context.cache.maxSize`); Framework 7 **pauses** inactive contexts (`spring.test.context.cache.pause`). Keep one shared base class (§12.11).

19.10 Reactor operators quick table

`Mono` = 0..1 item, `Flux` = 0..N items; both end with complete **or** error, and **nothing runs until someone subscribes** (in WebFlux, the framework does) (§13.3).

Goal	Operator(s)	Remember	§
create from values	`Mono.just(v)`, `Flux.just(...)`, `Flux.fromIterable(list)`, `Flux.range(1, n)`	`just(expr)` evaluates `expr` **immediately**	§13.3
create lazily	`Mono.fromCallable(...)`, `Mono.defer(() -> ...)`, `Mono.fromFuture(...)`	runs per subscription — retries repeat it	§13.3
empty / error	`Mono.empty()`, `Mono.error(e)`		§13.3
timers	`Flux.interval(d)`, `Mono.delay(d)`, `delayElements(d)`	run on `Schedulers.parallel()`	§13.3
bridge callbacks	`Flux.create(sink -> ...)`, `Sinks.many()`, `Sinks.one()`	choose an overflow strategy	§13.9
transform (sync)	`map`	must not block or return a publisher	§13.4
async call per item	`flatMap(fn[, concurrency])`	concurrent, **interleaved** output; default concurrency 256	§13.4
async, one at a time, in order	`concatMap`	sequential (sum of times)	§13.4
async, concurrent, source order	`flatMapSequential`	buffers early results	§13.4
filter	`filter`, `take(n)`, `skip(n)`, `distinct`		§13.4
combine by position	`Mono.zip(a, b, c)`, `zipWith`	"call 3 services in parallel" (like `Promise.all`)	§13.4
combine in sequence / as they come	`concat`, `concatWith` / `merge`, `mergeWith`		§13.4
fallback when empty	`switchIfEmpty(...)`, `defaultIfEmpty(v)`	the usual 404 path	§13.4
aggregate	`collectList`, `collectMap`, `reduce`, `count`	return a `Mono`	§13.4
group	`buffer(n)`, `window(n)`, `groupBy`		§13.4
then	`then()`, `then(mono)`	"do A, then B", ignoring A's values	§13.4
side effects	`doOnNext`, `doOnError`, `doOnSubscribe`, `doFinally`, `log()`	not for business logic	§13.4
errors: fallback	`onErrorReturn(v)`, `onErrorResume(e -> pub)`, `onErrorMap(e -> ex)`	an error **ends** the sequence	§13.6
errors: retry	`retry(n)`, `retryWhen(Retry.backoff(3, Duration.ofSeconds(1)))`	re-subscribes; attempts at 0/1/3/7 s with `jitter(0)`	§13.6
timeout	`timeout(d[, fallback])`	before `retryWhen` = per attempt	§13.6, §13.13
backpressure	`limitRate(n)` (re-requests at 75 %), `onBackpressureDrop/Latest/Buffer(n)/Error`	unbounded buffers risk `OutOfMemoryError`	§13.7
threads	`publishOn(s)` (everything **after** it), `subscribeOn(s)` (the **source**; position irrelevant)		§13.8
schedulers	`Schedulers.parallel()`, `boundedElastic()` (wrap blocking calls), `single()`, `immediate()`	never block Netty event loops	§13.8
hot / shared	`share()`, `publish().autoConnect()`, `cache()`, `Sinks.many().multicast()`	late subscribers miss earlier items	§13.9
context	`contextWrite(c -> c.put(k, v))`, `Mono.deferContextual(ctx -> ...)`	the reactive `ThreadLocal`	§13.9
leave the reactive world	`block()`, `blockFirst()`, `blockLast()`	tests and `main` only — throws on event-loop threads	§13.3
test	`StepVerifier.create(pub).expectNext(...).verifyComplete()`, `withVirtualTime(() -> ...)` + `thenAwait(d)`	build the publisher **inside** the supplier	§13.5

19.11 Actuator endpoints

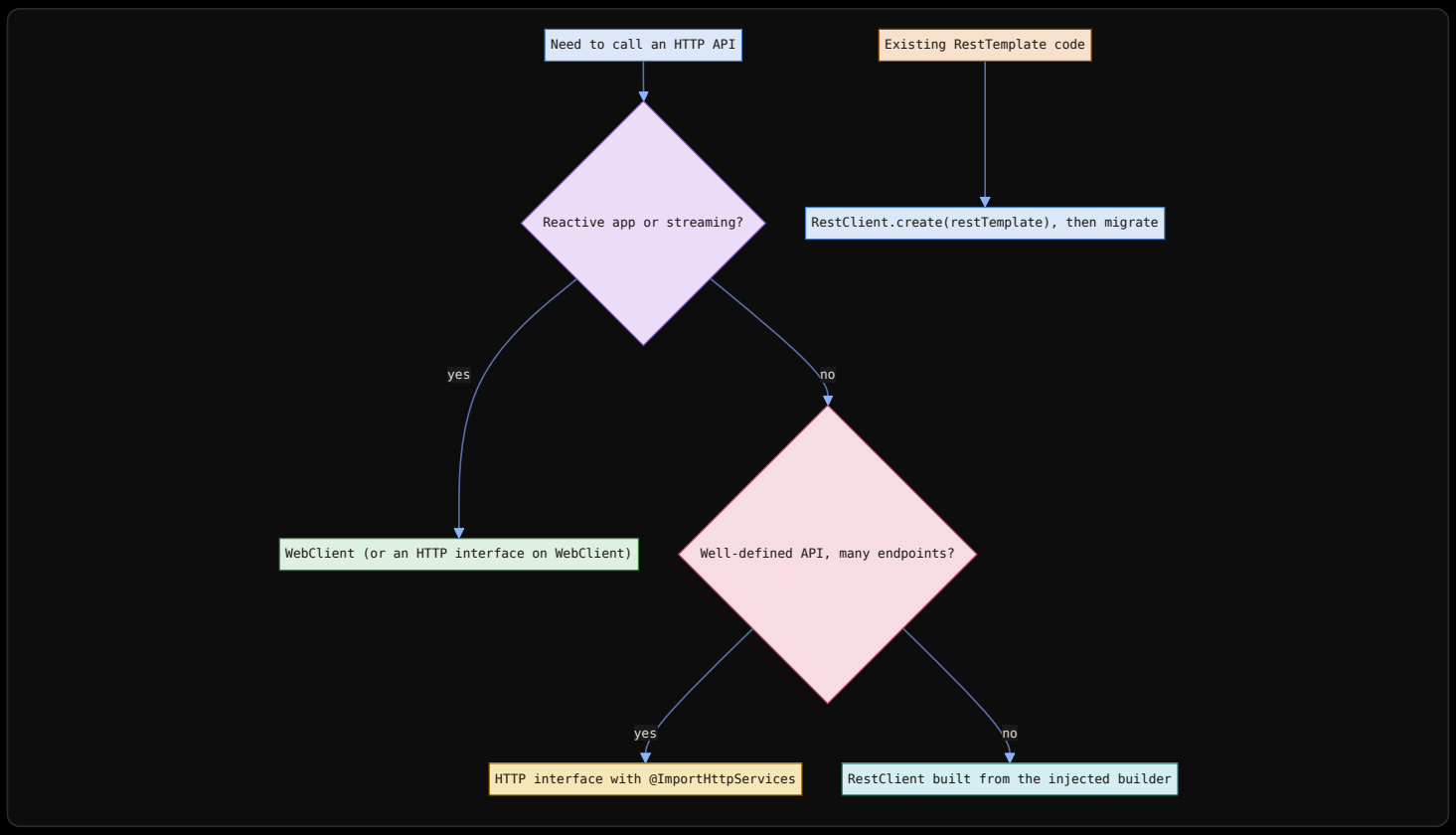
Add `spring-boot-starter-actuator`. Two switches decide availability: **access** (`none` / `read-only` / `unrestricted`) and **exposure** (HTTP / JMX). By default only `health` is exposed over HTTP (§15.2).

Endpoint (`/actuator/<id>`)	Shows / does	Notes	§
`health`	status `UP` / `DOWN` / `OUT_OF_SERVICE` / `UNKNOWN`; groups `liveness`, `readiness` (on by default in Boot 4)	`DOWN` / `OUT_OF_SERVICE` → HTTP 503; details `never` by default	§15.3
`info`	build, git, java, os, process, `info.*`	contributors switched on by properties	§15.4
`metrics`, `metrics/{name}`	meters as JSON, drill down with `?tag=k:v`	for humans	§15.5
`prometheus`	all meters in Prometheus text format	needs `micrometer-registry-prometheus`	§15.5
`loggers`	view and change log levels at runtime	`POST` with `{"configuredLevel":"DEBUG"}`	§15.2
`env`, `configprops`	property sources, bound `@ConfigurationProperties`	values masked (`show-values=never`)	§3.11, §15.2
`conditions`	the auto-configuration report	"why was bean X (not) created?"	§3.11
`beans`, `mappings`	all beans / all request mappings	debugging	§15.2
`threaddump`, `heapdump`	JVM threads / heap dump file	heap dumps can contain secrets	§15.2
`scheduledtasks`, `caches`, `flyway`, `liquibase`, `sessions`, `quartz`	feature-specific views (`caches` can also clear)	appear when the feature is present	§15.2
`httpexchanges`	recent requests/responses	needs an `HttpExchangeRepository` bean	§15.2
`startup`	startup steps and durations	needs `BufferingApplicationStartup`	§15.2
`sbom`, `auditevents`, `integrationgraph`, `logfile`	software bill of materials, audit events, Spring Integration graph, the log file	when configured	§15.2
`shutdown`	stops the app	access `none` by default	§15.2
`refresh`, `busrefresh` (Spring Cloud)	re-bind config / broadcast a refresh	Spring Cloud Context / Bus	§16.3

Secure them with `EndpointRequest.to("health", "info")` / `EndpointRequest.toAnyEndpoint()` (Boot 4 package `org.springframework.boot.security.autoconfigure.actuate.web.servlet`), ideally on a separate `management.server.port` (§15.2).

19.12 Which tool for which job

HTTP client: `RestClient` vs `WebClient` vs HTTP interface



	`RestClient`	`WebClient`	HTTP interface (`@HttpExchange`)	`RestTemplate`
Style	fluent, blocking	fluent, non-blocking (`Mono` / `Flux`)	declarative Java interface → proxy	template methods
Since / status	F6.1, recommended for blocking code	F5.0, recommended for reactive	F6.0; Boot 4 groups via `@ImportHttpServices`	deprecation announced (F7.1 `@Deprecated`, removal in 8.0)
Starter (Boot 4)	`spring-boot-starter-restclient`	`spring-boot-starter-webclient`	`-restclient` (or `-webclient` with `clientType = WEB_CLIENT`)	`-restclient`
Pick when	hand-written calls, custom per-call logic	WebFlux apps, streaming, heavy async composition	typed clients to other services, least boilerplate	never for new code
Watch out	inject `RestClient.Builder` (Boot converters, timeouts, tracing); `exchange()` skips status handlers	don't `block()` on event loops	wrap for `Optional`/domain exceptions	—
§	§6.2	§13.13	§6.6	§6.9

Web stack: Spring MVC + virtual threads vs WebFlux

	MVC + `spring.threads.virtual.enabled=true`	WebFlux
Code style	plain imperative, blocking libraries OK	`Mono` / `Flux` pipelines, non-blocking everything
Measured (1,000 clients, 100 ms I/O)	**7,942 req/s**, p50 108 ms, 33 platform threads	**8,058 req/s**, p50 110 ms
Same test, MVC on 200 platform threads	1,909 req/s, p50 517 ms	WebFlux + one `Thread.sleep`: **97 req/s**
Data access	JDBC, JPA	R2DBC, reactive Mongo/Redis
Streaming, backpressure end to end (SSE, WebSockets)	possible (`SseEmitter`), each stream holds a request	natural (`Flux`)
Debugging, `ThreadLocal` libraries	easy	needs Reactor `Context`, context propagation
Choose for	typical CRUD/REST services (the default)	streaming, gateways, all-non-blocking pipelines
§	§13.15, §15.10	§13.15

Data access: JPA vs `JdbcClient` vs Spring Data JDBC

	Spring Data JPA (Hibernate)	`JdbcClient` / `JdbcTemplate`	Spring Data JDBC
Model	rich object graph, relationships both ways	your own SQL, rows → records	DDD aggregates: root + owned children
Magic	persistence context, lazy loading, dirty checking, first-level cache	none	none: `save` writes, loading reads the whole aggregate
SQL you can predict	not always (N+1, flush timing)	yes	yes (children are deleted + re-inserted on save)
Schema	`ddl-auto` possible (use Flyway)	you write it	you write it (Flyway)
Typical pitfalls	N+1, `LazyInitializationException`, OSIV	SQL injection by concatenation, `.single()` on 0 rows	assigned ids without `@Version` → silent `UPDATE` of 0 rows
Choose for	domain models with relationships, team knows JPA	reports, bulk work, full SQL control	clear aggregates, predictable SQL
§	§8.1	§7.3	§9.2

Also: R2DBC for non-blocking SQL (§13.14), MongoDB for flexible documents (§9.5), Redis for counters/sessions/shared cache (§9.6); the overview table is §9.9.

Cache: Caffeine vs Redis

	Caffeine (local)	Redis (shared)
Read cost	nanoseconds, no serialization	network hop + (de)serialization
Consistency across instances	each instance has its own copy (stale until TTL)	one copy for all
Survives restarts	no	yes (with Redis persistence)
Stores	object **references** — cache immutable values	serialized copies (choose a JSON serializer)
Stampede protection	`@Cacheable(sync = true)` per key per JVM	same annotation, still per JVM; distributed lock for cluster-wide
TTL / size	`expireAfterWrite`, `maximumSize` (`spring.cache.caffeine.spec`)	`spring.cache.redis.time-to-live` / `RedisCacheConfiguration`
Choose for	small, hot, rarely changing reference data	data shared between instances, sessions, expensive results
§	§9.8	§9.8

Messaging: Kafka vs RabbitMQ

Full comparison: §14.18. In one breath: **Kafka** = partitioned, replicated **log** (consumer groups track offsets, replay, order per partition — key by aggregate id, very high throughput; `KafkaTemplate`, `@KafkaListener`, `DefaultErrorHandler` → `<topic>-dlt`, `@RetryableTopic`; tests with `@EmbeddedKafka`) → event streaming, event sourcing. **RabbitMQ** = **queue** broker (exchanges → bindings → queues, rich routing — direct, topic, fanout, headers; acked messages deleted; per-message TTL, priority, delay via plugin; `RabbitTemplate`, `@RabbitListener`, DLX, publisher confirms; no embedded broker — Testcontainers) → task distribution, complex routing, request/reply.

Resilience: Resilience4j vs Framework 7 `@Retryable`

Full table: §16.8. **Framework 7 core** (`@EnableResilientMethods`): `@Retryable` (`maxRetries` = retries **after** the first call ✔), `RetryTemplate`, `@ConcurrencyLimit` (`BLOCK` / `REJECT`) — no circuit breaker, rate/time limiter or fallback methods — for simple retries and concurrency caps without a dependency. **Resilience4j** (`resilience4j-spring-boot4`): `@Retry` (`max-attempts` counts the first call), `@Bulkhead`, `@CircuitBreaker` (CLOSED → OPEN → HALF_OPEN), `@RateLimiter`, `@TimeLimiter`, `fallbackMethod`, Micrometer metrics and health — for circuit breaking, rate limits, fallbacks. **Spring Retry** (`@EnableRetry`, `@Recover`; no longer managed by Boot 4) — legacy code only.

Rules for any retry: only transient failures (short-lived ones that may succeed a moment later) and idempotent operations, exponential back-off with jitter, a total time budget, retry in **one** layer (§6.7).

More decisions in one line each

Question	Answer	§
`@Value` or `@ConfigurationProperties`?	one value / SpEL → `@Value`; a group of settings → a `@ConfigurationProperties` record	§3.6
`@Component` or `@Bean`?	your class → stereotype; third-party class or creation logic → `@Bean`	§2.3
Unit test, slice or `@SpringBootTest`?	logic → unit; Spring-produced behaviour of one layer → slice; wiring → a few `@SpringBootTest`s	§12.2
`Page` or `Slice` (or `Window`)?	page numbers → `Page` (extra COUNT); "load more" → `Slice`; big tables → keyset `Window`	§8.11
`@EventListener`, `@TransactionalEventListener` or outbox?	in-process now → `@EventListener`; only after commit → `@TransactionalEventListener`; must not be lost → outbox	§2.12, §7.15, §14.17
JWT or opaque token?	local fast validation → JWT; instant revocation / hidden claims → opaque + introspection	§11.7
Session cookie or tokens?	browser app / BFF → session (+ CSRF); service-to-service, mobile → bearer tokens	§10.10, §11.12
Blocking or non-blocking retries (Kafka)?	order matters → `DefaultErrorHandler` (blocking); independent records → `@RetryableTopic`	§14.15
Flyway or Liquibase?	SQL-first, one database → Flyway; rollbacks, preconditions, many databases → Liquibase	§9.4
Eureka or Kubernetes Services?	on Kubernetes, Services + DNS usually suffice	§16.4
OpenFeign or HTTP interfaces?	new code → HTTP interfaces (OpenFeign is in maintenance mode)	§16.6
JVM + AOT cache or native image?	long-running, throughput-heavy → JVM (AOT cache halves startup); CLIs, scale-to-zero → native	§15.11

19.13 Boot 3 → 4 quick migration checklist

Work top to bottom; each step names the symptom you see if you skip it. Details: §17.12, §17.13, §17.14, §17.15.

Before you start

- [ ] Upgrade to **Boot 3.5.x** first and remove every deprecation warning (4.0 deletes what 3.x deprecated). From 2.7, go 2.7 → 3.5 (`javax` → `jakarta`) first.
- [ ] Java 17+ (25 recommended); Kotlin 2.2+, GraalVM 25+, Gradle 8.14+ if you use them.
- [ ] Run OpenRewrite's `org.openrewrite.java.spring.boot4.UpgradeSpringBoot_4_0` recipe and review the diff.
- [ ] Add `spring-boot-properties-migrator` (runtime, temporarily) to report renamed properties; remove it afterwards.
- [ ] Stuck? `spring-boot-starter-classic` / `spring-boot-starter-test-classic` restore the old all-in-one layout while you fix imports.

Dependencies

- [ ] `spring-boot-starter-web` → `spring-boot-starter-webmvc`; `-aop` → `-aspectj`; `-oauth2-*` → `-security-oauth2-*`; `-web-services` → `-webservices`.
- [ ] Add `spring-boot-starter-restclient` (client code) and `spring-boot-starter-webclient` (`WebClient.Builder`, else "No qualifying bean of type WebClient$Builder").
- [ ] Add a **test starter** per technology (`-webmvc-test`, `-data-jpa-test`, ...) — else `@WebMvcTest` does not resolve.
- [ ] Add `spring-boot-starter-flyway` / `-liquibase` — else migrations silently stop running.
- [ ] Undertow → Tomcat or Jetty. Spring Retry → declare its version or move to `@Retryable` (F7).

- [ ] Separately versioned projects: Spring Cloud **2025.1.x** (2025.1.2+ for Boot 4.1), Spring AI **2.0**, springdoc **3.x**, Testcontainers **2.x** artifact names.

Code

- [ ] Jackson 3: `com.fasterxml.jackson.databind` → `tools.jackson.databind` (annotations unchanged); `ObjectMapper` bean → `JsonMapper`; `Jackson2ObjectMapperBuilderCustomizer` → `JsonMapperBuilderCustomizer`; `@JsonComponent` → `@JacksonComponent`.
- [ ] Auto-configuration class imports moved to per-module packages (check `exclude = ...`, `@ImportAutoConfiguration`, `@AutoConfiguration(after = ...)`).
- [ ] `@EntityScan` → `org.springframework.boot.persistence.autoconfigure`; `hibernate-jpamodelgen` → `hibernate-processor`.
- [ ] Null-safety: `org.springframework.lang.Nullable` → `org.jspecify.annotations.Nullable`; `javax.annotation`/`javax.inject` are gone (use `jakarta.*`).
- [ ] Security 7: rewrite `.and()` chains as one lambda per feature, `authorizeRequests` → `authorizeHttpRequests`, Ant/MVC matchers → `PathPatternRequestMatcher` (plain `requestMatchers("/x/**")`).
- [ ] `RestTemplate` → `RestClient` (start with `RestClient.create(restTemplate)`).

Tests

- [ ] `@MockBean` / `@SpyBean` → `@MockitoBean` / `@MockitoSpyBean` (removed in Boot 4; not allowed inside `@Configuration` classes).
- [ ] `@SpringBootTest` + `@AutoConfigureMockMvc` / `@AutoConfigureRestTestClient` / `@AutoConfigureWebTestClient` / `@AutoConfigureTestRestTemplate` — no longer automatic.
- [ ] Slice annotation imports moved (e.g. `org.springframework.boot.webmvc.test.autoconfigure.WebMvcTest`); JUnit 4 runners → JUnit 6 (Jupiter).

Properties — the common renames are tabled in §17.13 (`spring.jackson.*`, `spring.mongodb.*`, `server.error.*` → `spring.web.error.*`, `spring.persistence.exceptiontranslation.*`, `management.health.mongodb.*`, `spring.session.data.redis.*`, Kafka backoff `jitter`). Also:

Boot 3	Boot 4
`spring.http.client.*`	`spring.http.clients.*`
`management.tracing.enabled`	`management.tracing.export.enabled`
`management.otlp.tracing.endpoint`	`management.opentelemetry.tracing.export.otlp.endpoint`
`spring.rabbitmq.listener.simple.retry.max-attempts` (counted the first call)	`...retry.max-retries` (does not)

Behaviour changes to re-test

- [ ] Health probes (`/actuator/health/liveness`, `/readiness`) are **on by default** — check what your platform calls.
- [ ] DevTools LiveReload is off (and deprecated in 4.1).
- [ ] `server.forward-headers-strategy` no longer applies to WAR deployments.
- [ ] Jackson 3 defaults: ISO dates, unknown properties ignored, properties sorted alphabetically after creator properties.

19.14 Interview index

Chapter 18 indexes 168 questions (pointing to the short answers in the Chapter 20 question bank and adding deeper details), five runnable proofs, a project walkthrough and ten debugging scenarios. This index maps each topic to its Chapter 18 section and teaching chapter.

Topic	Chapter 18 section	Questions	Teaching chapter
IoC, beans, scopes, lifecycle, events	§18.2	Q1–Q20	Chapter 2
Boot, starters, configuration, auto-configuration	§18.3	Q21–Q35	Chapter 1, Chapter 3
AOP, proxies, Framework 7 resilience	§18.4	Q36–Q44	Chapter 4
MVC, REST, validation, errors, versioning, clients	§18.5	Q45–Q61	Chapter 5, Chapter 6
JDBC, transactions, JPA, Spring Data	§18.6	Q62–Q91	Chapter 7, Chapter 8, Chapter 9
Caching	§18.7	Q92–Q97	Chapter 9
Security, JWT, OAuth2	§18.8	Q98–Q114	Chapter 10, Chapter 11
Testing	§18.9	Q115–Q125	Chapter 12
Reactive and WebFlux	§18.10	Q126–Q132	Chapter 13
Messaging, async, scheduling	§18.11	Q133–Q140	Chapter 14
Production and observability	§18.12	Q141–Q152	Chapter 15
Microservices and Spring Cloud	§18.13	Q153–Q160	Chapter 16
Spring AI and Boot 4 / Framework 7 changes	§18.14	Q161–Q168	Chapter 17
Proofs of five tricky answers	§18.15	prototype-in-singleton, `@Qualifier` vs `@Primary`, cycles, checked exceptions, self-invocation	this chapter's §19.15 adds more
"Explain your Spring project" template	§18.16	—	—
Ten debugging scenarios	§18.17	bean not found, cycles, no rollback, lazy loading, N+1, 401/403, CORS, "annotation does nothing", auto-configuration failures, slow startup	—

The questions that come up most

The most frequently asked questions are marked ⋆ in the Chapter 20 question bank; drill those first, then the scenario section.

19.15 Proofs: the facts on these sheets, run

What it is

`code/cheatsheet-checks/` is a small Maven project (Boot 4.1.1, Java 25, no web server, no Docker) whose tests assert the facts marked ✔ in this chapter. If a future Boot version changes a default, a test fails, and the sheet must be updated. Run it from the repository root with `bash tools_run_spring.sh spring '19-*'`.

Test class	Proves	Sheet
`ScopeAndLifecycleTests`	singleton shared, prototype fresh, prototype-in-singleton stuck, init/destroy order, no destroy callback for prototypes	§19.7
`BeanSelectionTests`	`@Qualifier` beats `@Primary`, `@Primary` beats the parameter name, `List` by `@Order`, `Map` by bean name	§19.3, §19.7
`TransactionTests`	`@Transactional` defaults, all seven propagation types, rollback rules, `UnexpectedRollbackException`	§19.6
`PropertyBindingTests`	env-var mapping (canonical and legacy), list index, `Duration` / `DataSize` units, record + `@DefaultValue`	§19.5
`PropertyPrecedenceTests`	defaults < `application.properties` < env var < `-D` < `--arg`; `@Value` defaults and SpEL	§19.5
`BootDefaultsTests`	HikariCP pool 10 / timeout 30 s, unique embedded DB name, `@Async` executor 8 threads `task-` (virtual threads → `SimpleAsyncTaskExecutor`), scheduler 1 thread, shutdown 30 s	§19.5
`CoreUtilitiesTests`	default cache keys, six-field cron, `@Retryable` defaults and 1 + `maxRetries` attempts	§19.3, §19.12
`JsonAndSecurityDefaultsTests`	Jackson 3 defaults, `hasRole` prefix rules, `DelegatingPasswordEncoder`	§19.5, §19.8

The transaction checks use plain Spring on an in-memory H2 database, the same technique as §18.15:

```
@Transactional
void catchInnerRequired() {
    inner.insert("outer");
    try { inner.requiredThenFail(); } catch (IllegalStateException swallowed) { }
}   // commit attempt -> UnexpectedRollbackException, nothing is saved
```

The environment-variable checks bind through the same `Binder` that `@ConfigurationProperties` uses, over a fake `SystemEnvironmentPropertySource`:

```
Binder canonical = envBinder(Map.of("SHOP_CHECKOUTTIMEOUT", "2m", "SHOP_PAYMENT_RETRIES", "4", "SHOP_CURRENCIES_0", "USD"));
Duration timeout = canonical.bind("shop.checkout-timeout", Duration.class).get();   // PT2M
```

Output

`mvn -q -B -ntp verify` (20 tests, all passing; JUnit does not fix the order of test classes, so blocks may appear in another order):

```
spring.task.scheduling.pool.size default = 1, spring.lifecycle.timeout-per-shutdown-phase default = PT30S
applicationTaskExecutor = ThreadPoolTaskExecutor, core = 8, prefix = task-, queue = unbounded
with spring.threads.virtual.enabled=true -> SimpleAsyncTaskExecutor
DataSource = HikariDataSource, maximumPoolSize = 10, connectionTimeout = 30000 ms
no spring.datasource.url + H2 on the classpath -> jdbc:h2:mem:<random name>  (generate-unique-name=true)
'0 30 2 * * MON-FRI' after 2026-09-28T10:00 -> 2026-09-29T02:30
five-field Unix cron '0 2 * * *' -> IllegalArgumentException
SimpleKeyGenerator: () -> SimpleKey [], ("BOOK-1") -> BOOK-1, ("BOOK-1","EUR") -> SimpleKey [BOOK-1, EUR]
@Retryable defaults: maxRetries=3, delay=1000 ms, multiplier=1.0
maxRetries = 2 -> [attempt 1, attempt 2, attempt 3], then the last exception is rethrown
SHOP_CHECKOUTTIMEOUT=2m   -> shop.checkout-timeout = PT2M
SHOP_PAYMENT_RETRIES=4    -> shop.payment.retries  = 4
SHOP_CURRENCIES_0=USD     -> shop.currencies[0]    = USD
SHOP_CHECKOUT_TIMEOUT=3m  -> shop.checkout-timeout = PT3M (legacy '-' -> '_' form, also accepted)
Duration '45s' -> PT45S, '500' -> 500 ms, 'PT1H' -> PT1H
DataSize '10MB' -> 10485760 bytes, '1024' -> 1024 bytes
record bound with @DefaultValue: ShopProperties[name=Corner Shop, checkoutTimeout=PT30S, maxUpload=5242880B]
init order    : [constructor, @PostConstruct, afterPropertiesSet, initMethod]
destroy order : [@PreDestroy, destroy(), destroyMethod]  (no Proto @PreDestroy)
singleton: getBean twice -> same instance?        true
prototype: getBean twice -> new instance?         true
prototype injected into a singleton -> same one?  true
@Value ${shop.missing:fallback} -> fallback, #{${shop.retries:3} + 1} with retries=2 -> 3, key set only in defaultProperties -> eu (default)
defaultProperties + application.properties -> from application.properties
+ env SHOP_NAME                           -> from env var
+ -Dshop.name                             -> from -D system property
+ --shop.name                             -> from --command-line
by type (@Primary)        -> sms
@Qualifier("email")        -> email
parameter named 'push'    -> sms  (@Primary wins over the name)
List<Notifier> by @Order  -> [email, sms, push]
Map keys (bean names)     -> [email, push, sms]
encode -> {bcrypt}$2a$10$...  matches {noop}s3cret? true  upgradeEncoding({noop}) = true
hasRole("ADMIN"): authority ROLE_ADMIN -> true, authority ADMIN -> false
hasRole("ROLE_ADMIN") -> IllegalArgumentException
JsonMapper (tools.jackson.databind.json) writes {"item":"book","on":"2026-09-28"}
unknown property ignored -> Delivery[item=pen, on=2026-10-01]
broken JSON -> JacksonException (unchecked)
committed despite an exception: [checked, default rules, caught inside the method]
inner REQUIRED failed, outer caught it -> UnexpectedRollbackException, rows = []
@Transactional defaults: propagation=REQUIRED isolation=DEFAULT readOnly=false timeout=-1
REQUIRED      : inner runs in the outer transaction? true (Outer.requiredJoins)
REQUIRES_NEW  : outer rolled back, rows left = [audit (REQUIRES_NEW)]
NESTED        : inner failed + caught, rows left = [outer] (savepoint)
MANDATORY     : no transaction -> IllegalTransactionStateException
NEVER         : inside a transaction -> IllegalTransactionStateException
SUPPORTS      : active alone? false, active inside a transaction? true
NOT_SUPPORTED : active inside a transaction? false (outer suspended)
```

What the output teaches

- A **legacy** environment-variable spelling (`SHOP_CHECKOUT_TIMEOUT`, dash turned into an underscore) also binds to `shop.checkout-timeout`. The documented form without the dash separator (`SHOP_CHECKOUTTIMEOUT`) is the one to use and to search for; know that both work when you hunt for "who set this value?".
- `@Primary` beats a matching parameter name — the name is only the **last** tie-breaker.
- HikariCP reports its pool size only after the pool has started; the test takes one connection first.
- The `@Retryable` bean is a CGLIB proxy, so the test reads its attempt list through a method: a field read on a proxy returns the proxy's own, never-initialised field (§4.5).

19.16 Glossary A–Z

Short definitions of the terms used in this book; follow the link for the full explanation. Annotations are defined in the sheets above (§19.3, §19.9), not repeated here.

A

Term	Meaning	See
Access (Actuator)	per-endpoint permission: `none`, `read-only`, `unrestricted` (Boot 3.4+); separate from exposure	§15.2
Access token	short-lived token a client sends to an API to prove it may call it	§11.1
`acks` (Kafka)	how many replicas must confirm a write; `all` (`-1`) is the default since Kafka 3.0	§14.16
Actuator	Boot's management endpoints: health, metrics, info, loggers, env...	§15.2
Advice (AOP)	the code an aspect runs at a join point: before, after returning, after throwing, after, around	§4.1
Advisor (AOP)	Spring's internal pair of one advice and one pointcut	§4.1
Advisor (Spring AI)	interceptor around every `ChatClient` call (memory, RAG, tool calling, logging), ordered by `getOrder()`	§17.5
AES-GCM	standard symmetric encryption (one shared key) with built-in tamper detection; used for encrypting secrets and data	§10.5
Aggregate	DDD cluster of objects loaded and saved as one unit through its root; the unit of Spring Data JDBC	§9.2
`AggregateReference`	Spring Data JDBC reference to another aggregate by id only	§9.2
Annotation (Java)	metadata written as `@Name` on a class, method or field; Spring reads it to add behaviour	§1.1
Anthropic / OpenAI / Ollama	AI model providers Spring AI has starters for; Ollama runs open models on your own machine	§17.2
AOP	Aspect-Oriented Programming: write a cross-cutting concern once, apply it where a pointcut matches	§4.1
AOT cache (JDK)	JDK 24+ file of classes loaded and linked in a training run; `-XX:AOTCacheOutput` / `-XX:AOTCache`	§15.11
AOT processing (Spring)	build-time generation of bean-definition code, used by native images and optionally on the JVM	§15.11
API	Application Programming Interface: the operations one piece of software offers to another (Java methods or HTTP endpoints)	§1.1
API gateway	single front door that routes requests and applies edge concerns (Spring Cloud Gateway)	§16.7
API versioning	Framework 7 feature: read a version from the request and match `version = "1.1+"` mappings	§5.9
`ApiVersionInserter`	client-side strategy that puts the API version into requests	§6.7
`ApplicationContext`	Spring's full container: eager singletons, post-processors, events, `Environment`, i18n	§2.2
`ApplicationContextRunner`	test helper that builds a tiny context per test, ideal for auto-configuration	§3.13
`ApplicationEventPublisher`	bean used to publish application events	§2.12
`ApplicationReadyEvent`	Boot event after startup and runners have finished	§2.7
`ApplicationRunner` / `CommandLineRunner`	beans run once at startup with the program arguments	§1.8
Argon2 / bcrypt / PBKDF2 / scrypt	slow, salted password-hashing algorithms; bcrypt is Spring Security's default	§10.5
Aspect	class bundling pointcuts and advice for one concern (`@Aspect` + `@Component`)	§4.2
AspectJ	full AOP language with compile/load-time weaving; Spring borrows its annotations and pointcut syntax	§4.8
AssertJ	fluent assertion library: `assertThat(x).isEqualTo(y)`; included in the Boot test starter	§12.8
Asynchronous	the caller continues without waiting; the result arrives later (callback, future, message)	§14.1
At-least-once	delivery guarantee: never lost, possibly duplicated — consumers must be idempotent	§14.8
Audience (`aud`)	JWT claim naming who the token is meant for; an API should reject tokens for another audience	§11.1
Auditing (Spring Data)	automatically filling created/modified date and user columns	§8.13
`Authentication`	object holding the principal, credentials and authorities of the current caller	§10.1
Authentication	proving who the caller is	§10.1
`AuthenticationEntryPoint`	starts authentication for an anonymous, denied request: 401 challenge or login redirect	§10.2
`AuthenticationManager`	entry point of authentication; usually a `ProviderManager` asking `AuthenticationProvider`s	§10.4

Term	Meaning	See
Authority (`GrantedAuthority`)	one permission string held by the caller (`report:read` , `ROLE_ADMIN` , `SCOPE_x`)	§10.1
Authorization	deciding whether a known caller may perform an action	§10.1
Authorization code flow	OAuth2 flow for apps with a user: code in the browser, token exchange on the back channel (with PKCE)	§11.9
Authorization server	issues tokens (Keycloak, Okta, Entra ID, Spring Authorization Server)	§11.2
`AuthorizationFilter`	last filter of a security chain; applies `authorizeHttpRequests` rules	§10.2
Auto-configuration	conditional `@Configuration` classes that Boot applies after yours to fill gaps	§3.10
`AutoConfiguration.imports`	file `META-INF/spring/...AutoConfiguration.imports` listing a jar's auto-configurations	§3.10
Availability state	Boot's `LivenessState` and `ReadinessState` , exposed as health groups	§15.3
Awaitility	test library that polls a condition until true or timeout	§14.2
`Aware` callbacks	interfaces (`BeanNameAware` , `ApplicationContextAware`) through which a bean receives container objects	§2.7

B

Term	Meaning	See
B3 headers	Zipkin's older trace-propagation headers, an alternative to W3C `traceparent`	§15.7
Backpressure	the consumer limits the producer via `request(n)`	§13.7
Baggage	extra key/value context that travels with a trace	§15.7
Bean	object created, wired and managed by the Spring container	§2.1
Bean definition	the recipe for a bean: class or factory method, scope, dependencies, callbacks	§2.2
Bean Validation	Jakarta standard for constraint annotations (`@NotBlank` , `@Email`); Hibernate Validator implements it	§5.6
`BeanFactory`	minimal container: bean definitions + lazy creation	§2.2
`BeanFactoryPostProcessor`	changes bean **definitions** before any bean is created (e.g. `${}` resolution)	§2.8
`BeanPostProcessor`	sees every bean **instance** before/after init and may wrap it in a proxy	§2.8
`BeanRegistrar`	Framework 7 contract for registering beans programmatically (loops, conditions)	§2.14
Bearer token	token that grants access to whoever holds it, sent as `Authorization: Bearer ...`	§11.1
BFF (Backend for Frontend)	server-side OAuth2 client that keeps tokens away from the browser and gives it a session cookie	§11.12
BlockHound	Java agent that detects blocking calls on non-blocking threads in tests	§13.8
Blocking call	a call that holds its thread while it waits (JDBC, `Thread.sleep` , a blocking HTTP client)	§13.1
BOM	"bill of materials" POM pinning versions of many artifacts (`spring-boot-dependencies`)	§1.6
`boundedElastic`	Reactor scheduler for wrapping unavoidable blocking calls	§13.8
BPP / BFPP	short for `BeanPostProcessor` / `BeanFactoryPostProcessor`	§2.8
Brave / Zipkin / Jaeger / Tempo	tracing libraries and back ends: Brave records spans, Zipkin, Jaeger and Tempo store and display traces	§15.7
Broker (message broker)	server that receives messages from producers and delivers them to consumers (Kafka, RabbitMQ)	§14.8
Brute-force protection	slowing or locking out repeated failed logins so passwords can't simply be guessed	§11.15
Bucket4j	Java rate-limiting library based on the token-bucket algorithm	§11.15
Build tool	program that downloads dependencies, compiles, tests and packages the code (Maven, Gradle)	§1.6
Bulkhead	cap on concurrent calls to one dependency (`@ConcurrencyLimit` , Resilience4j `@Bulkhead`)	§4.7, §16.8
Buildpacks	Cloud Native Buildpacks: build an OCI image from a jar without a Dockerfile (`spring-boot:build-image`)	§15.11
Bytecode	the compiled instructions in `.class` files that the JVM executes	§4.4

C

Term	Meaning	See
Cache-aside	look in the cache; on a miss load, store and return — what `@Cacheable` does	§9.8
Cache stampede	many concurrent misses for one hot key all compute it; `sync = true` mitigates per JVM	§9.8
`CacheManager`	provider-neutral cache interface behind the cache annotations	§9.8
Caffeine	high-performance local Java cache (W-TinyLFU eviction)	§9.8
Cardinality	number of distinct tag-value combinations of a metric; keep it small	§15.5
Cascade	passing a JPA operation (persist, merge, remove...) from parent to children	§8.5
Cascading failure	one slow or failed service makes its callers fail too, spreading through the system	§16.8
CDC (Change Data Capture)	streaming every insert/update/delete from the database's change log (e.g. Debezium); one way to relay an outbox	§14.17
CDS	Class Data Sharing: JVM archive of parsed classes for faster startup	§15.11
CGLIB proxy	runtime subclass used as a proxy; Boot's default; cannot handle `final`	§4.4
`ChatClient`	Spring AI fluent API over a `ChatModel`, with advisors, templates, tools	§17.3
`ChatMemory`	Spring AI store of past messages per conversation id (window of 20 by default)	§17.6
`ChatModel`	Spring AI provider-level interface: `Prompt` in, `ChatResponse` out	§17.3
Checked exception	exception the compiler forces you to declare or catch (not a `RuntimeException`); commits a transaction by default	§7.10
Chunking (RAG)	splitting documents into small pieces before embedding them, so retrieval returns focused passages	§17.9
Circuit breaker	stops calling a failing dependency (CLOSED → OPEN → HALF_OPEN)	§16.8
Circular dependency	dependency cycle between beans; constructor cycles always fail, Boot rejects field cycles	§2.9
Claim	statement in a JWT payload (`sub`, `iss`, `aud`, `exp`, `scope`)	§11.1
Class loader	JVM component that finds and loads `.class` files; DevTools uses two of them for fast restart	§1.10
Classpath	the list of jars and folders the JVM loads classes from; auto-configuration reacts to what is on it	§1.2
CLI	command-line interface: a program you run from a terminal	§1.9
Client credentials	OAuth2 grant for service-to-service calls without a user	§11.10
Cold publisher	replays the whole pipeline for every subscriber	§13.9
Commit / rollback	finish a transaction by making its changes permanent / by undoing all of them	§7.7
Component scanning	finding `@Component` classes in a package tree	§2.3
Condition evaluation report	Boot's list of which auto-configurations matched and why (`--debug`, `/actuator/conditions`)	§3.11
Config data	Boot's files and imports: `application.*`, profile files, `spring.config.import`	§3.3
Config Server	Spring Cloud service serving configuration from Git or other backends	§16.3
`configtree:`	import where each file in a directory is a property (Kubernetes secrets)	§3.8
Connection pool	set of open database connections reused across requests because opening one is slow (HikariCP)	§7.2
Constructor injection	dependencies passed as constructor parameters; the recommended injection style	§2.4
Consul	HashiCorp service registry and key/value store, an alternative to Eureka	§16.4
Consumer group	Kafka consumers sharing a `group.id`; each partition goes to one member, each group gets every record	§14.9
Consumer lag	how many messages a consumer group is behind the newest ones in a partition	§14.12
Container (Docker)	isolated process packaged with everything it needs to run; started from a container image	§12.14
Container image	packaged file system plus start command from which containers are run (OCI format)	§15.11
Content negotiation	choosing the representation from `Accept` / `produces` (406 if impossible)	§5.8
Context caching (tests)	the TestContext Framework reuses started contexts with the same configuration	§12.11

Term	Meaning	See
Context pausing	Framework 7: inactive cached test contexts have their lifecycle beans stopped	§12.11
Context propagation	copying `ThreadLocal` values (trace, MDC) across threads or into the Reactor `Context`	§15.6, §13.9
Contract testing	consumer and provider tested against a shared contract (Pact, Spring Cloud Contract)	§16.10
Cookie	small value a site stores in the browser, sent back automatically on every request to that site	§10.10
CORS	browser rules letting a page from one origin read responses from another (preflight `OPTIONS`)	§10.9
Counter / Timer / DistributionSummary	Micrometer meters: a count that only rises / durations / sizes	§15.5
CPU-bound vs I/O-bound	work limited by computation vs work that mostly waits for network or disk	§13.8
CQRS	Command Query Responsibility Segregation: separate models for writes and for reads	§16.1
Cron expression	Spring's six-field schedule, starting with seconds (`0 0 2 * * *`)	§14.7
Cross-cutting concern	requirement spread over many classes (logging, transactions, security)	§4.1
CRUD	Create, Read, Update, Delete: the four basic data operations	§8.8
CSRF	attack that makes a logged-in browser send a forged state-changing request; blocked by tokens	§10.8
curl / Postman	command-line and graphical tools for sending HTTP requests by hand; they ignore CORS	§1.5
Customizer	bean that adjusts an auto-configured object instead of replacing it	§3.12

D

Term	Meaning	See
DAO	Data Access Object: a class that hides the SQL for one kind of data	§7.1
`DataAccessException`	Spring's unchecked, database-neutral exception hierarchy	§7.5
`DataSize`	Boot type for sizes (`10MB`); plain numbers are bytes	§3.6
`DataSource`	factory of JDBC connections; in Boot a HikariCP pool	§7.2
DDD	Domain-Driven Design: modelling software around business concepts (entities, aggregates, bounded contexts)	§9.2
DDL / `ddl-auto`	Data Definition Language (`CREATE TABLE` , `ALTER`); `ddl-auto` lets Hibernate create or validate the schema	§8.16
Dead-letter exchange (DLX)	RabbitMQ exchange that receives rejected or expired messages from a queue	§14.18
Dead-letter topic (DLT)	where records go after the last failed attempt; default name `<topic>-dlt` (spring-kafka 3.3+)	§14.14
`DeadLetterPublishingRecoverer`	spring-kafka recoverer that publishes failed records to the DLT	§14.14
Deadlock	two transactions each wait for a lock the other holds; the database aborts one of them	§7.11
`DefaultErrorHandler`	spring-kafka container error handler: blocking retries, then a recoverer	§14.14
`DelegatingFilterProxy`	servlet filter that hands requests to the `springSecurityFilterChain` bean	§10.2
`DelegatingPasswordEncoder`	encoder storing `{id}hash` ; verifies old formats, upgrades on login	§10.5
Dependency	a library your project uses, declared in the build file; or an object a bean needs	§1.6
Dependency Injection (DI)	an object receives its dependencies instead of creating them	§2.1
Deprecated	still works but is marked for removal in a later version; move off it	§1.4
Derived query	Spring Data method whose name is the query (`findByTitleContaining`)	§8.9
Deserialization / serialization	turning bytes or JSON into objects / objects into bytes or JSON	§5.5
Detached entity	JPA entity no longer in a persistence context; changes are not tracked	§8.4
DevTools	development module: fast restart, dev defaults; off for `java -jar`	§1.10
Dirty checking	Hibernate compares managed entities with their snapshots at flush and writes `UPDATE` s	§8.4
`DiscoveryClient`	Spring Cloud abstraction returning the instances of a service id	§16.4
`DispatcherServlet`	Spring MVC's front controller	§5.2
`DispatcherHandler`	WebFlux's front controller	§13.10
DNS	Domain Name System: turns host names into network addresses; Kubernetes uses it for service discovery	§16.4
Docker	tool that builds and runs containers; Testcontainers needs it	§12.14
DTO	Data Transfer Object: a simple class or record shaped for an API instead of the database	§5.4
Dual-write problem	a database write and a broker publish cannot be atomic; solved by the outbox	§14.17
`Duration` binding	`45s` , `5m` , `PT1H` ; plain numbers are milliseconds	§3.6

E

Term	Meaning	See
EAGER / LAZY	load an association with its owner / on first access; make everything LAZY	§8.6
EJB	Enterprise JavaBeans: the older Java EE component model Spring was created as an alternative to	§1.4
Embedded database	database that runs inside the application's JVM (H2, HSQLDB); handy for tests and demos	§7.2
Embedded server	Tomcat/Jetty/Netty running inside the application jar	§1.2
Embedding	vector of numbers representing text so similar texts are close	§17.9
Entity	JPA class whose instances are table rows (`@Entity` , `@Id` , no-arg constructor, not `final`)	§8.3
Entity graph	declares associations to fetch with a query (`@EntityGraph` , left join)	§8.7
`EntityManager`	JPA API over the persistence context (Hibernate `Session`)	§8.4
`Environment`	Spring's view of all property sources and active profiles	§3.1
ETag	response header holding a version fingerprint so clients can ask "has it changed?" (304 Not Modified)	§8.14
Eureka	Netflix service registry supported by Spring Cloud (AP design)	§16.4
Event loop	few threads that run pipeline steps when I/O completes, never waiting	§13.1
Event-driven	services communicate by publishing and reacting to events instead of calling each other directly	§14.8
Eventual consistency	copies or services agree after a short delay instead of instantly	§14.17
Exactly-once	each message's effect happens once; in practice at-least-once delivery plus idempotent processing	§14.16
Exception translation	turning `SQLException` s into `DataAccessException` s	§7.5
`ExceptionTranslationFilter`	turns security exceptions into 401/redirect (anonymous) or 403 (authenticated)	§10.2
Exchange / binding (RabbitMQ)	an exchange receives messages and routes them to queues according to bindings	§14.18
Executor / `TaskExecutor`	owner of worker threads and a task queue; runs `@Async` work	§14.4
Exponential back-off	waiting longer after each failed attempt (1 s, 2 s, 4 s...) before retrying	§4.7
Exposure (Actuator)	whether an endpoint is reachable over HTTP or JMX (`health` only, by default)	§15.2
Externalized configuration	settings outside the build artifact (files, env vars, args)	§3.1

F

Term	Meaning	See
`FACTOR_*` authority	Security 7 authority recording a completed login factor (`FACTOR_PASSWORD`)	§10.4
Fallback (resilience)	default answer returned when a protected call fails or a circuit is open	§16.8
Fat (executable) jar	Boot jar with `BOOT-INF/lib` dependencies, started by `JarLauncher`	§1.9
Filter (servlet)	code around every request before the `DispatcherServlet` ; can wrap request/response	§5.11
`FilterChainProxy`	the `springSecurityFilterChain` bean that picks the first matching `SecurityFilterChain`	§10.2
`fixedDelay` / `fixedRate`	`@Scheduled` timing: end-to-start / start-to-start	§14.7
`flatMap` / `concatMap`	async per item: concurrent and interleaved / sequential and ordered	§13.4
Flush	Hibernate sends queued SQL: before commit, before affected queries, on `flush()`	§8.4
`Flux`	Reactor publisher of 0..N items	§13.3
Flyway	SQL-file schema migration tool (`V1__x.sql` , checksums)	§8.16
Foreign key	column that points to the primary key of a row in another table	§8.5
Form login	username/password via an HTML form; session cookie afterwards	§10.4
Full vs lite configuration	`@Configuration` CGLIB subclass (inter-bean calls return singletons) vs plain calls	§2.3
Functional endpoints	routes as `RouterFunction` beans and handler functions instead of annotations	§13.11

G – H

Term	Meaning	See
GA / milestone / SNAPSHOT	release stages: general availability (production-ready) / preview builds / unreleased development builds	§1.4
Garbage collection (GC)	the JVM automatically freeing memory of objects no longer used; pauses can add latency	§13.15
Gateway (Spring Cloud)	API gateway: routes = predicates + filters; Web MVC or WebFlux flavour	§16.7
Gauge	Micrometer meter sampling a current value	§15.5
GPU	graphics processor; runs AI models much faster than a CPU	§10.5
Graceful shutdown	stop accepting requests, finish in-flight ones, then close (default since Boot 3.4)	§15.9
GraalVM native image	whole app compiled ahead of time to a native executable (closed world)	§15.11
Gradle	build tool alternative to Maven, with Groovy or Kotlin build scripts	§1.5
Grant type (OAuth2)	way a client obtains a token: authorization code, client credentials, refresh token, token exchange	§11.2
H2	small Java SQL database that can run in memory; used for demos and quick tests	§7.2
HAL / HATEOAS	REST style where responses include links to related actions; HAL is a JSON format for those links	§5.12
`HandlerInterceptor`	MVC hook around handler methods (`preHandle` , `postHandle` , `afterCompletion`)	§5.11
`HandlerMapping`	finds the handler method for a request	§5.2
Hash (password)	one-way fingerprint of a value; passwords are stored as slow, salted hashes, never encrypted	§10.5
Health group	named subset of health indicators at `/actuator/health/<group>`	§15.3
`HealthIndicator`	bean contributing one check to `/actuator/health`	§15.3
Heap	JVM memory area where objects live; sized with `-Xmx` or `-XX:MaxRAMPercentage`	§15.11
Hibernate	the JPA implementation (ORM) Boot uses; maps entities to tables and generates SQL	§8.1
HikariCP	Boot's default JDBC connection pool (default 10 connections)	§7.2
HMAC / RSA / EC (JWT signing)	signing families: HMAC (HS256) shares one secret; RSA (RS256) and EC (ES256) sign privately and verify with a public key	§11.1
Hot publisher	produces whether or not anyone listens; late subscribers miss items	§13.9
HOTP / TOTP	one-time password algorithms based on a counter / on the current time	§11.18
HSQLDB / Derby	other embedded Java databases Boot can auto-configure, like H2	§7.2
HTTP	HyperText Transfer Protocol: request/response protocol of the web (method, path, headers, body, status)	§5.1
HTTP Basic	credentials in an `Authorization: Basic` header on every request	§10.4
HTTP methods	GET reads, POST creates, PUT replaces, PATCH partly updates, DELETE removes	§5.1
HTTP service interface	Java interface with `@GetExchange` ... implemented by a generated proxy	§6.6
HTTP status 401 vs 403	401 = not authenticated (who are you?), 403 = authenticated but not allowed	§10.6
`HttpMessageConverter`	converts bodies to and from Java objects (Jackson for JSON)	§5.2
`HttpOnly` / `Secure` / `SameSite`	cookie flags: hidden from JavaScript / sent only over HTTPS / not sent on cross-site requests	§11.12
HTTPS / TLS	HTTP encrypted with TLS (Transport Layer Security), which also proves the server's identity	§11.18

I – J

Term	Meaning	See
ID token	OIDC JWT that tells the client who logged in; not for calling APIs	§11.2
IDE	Integrated Development Environment such as IntelliJ IDEA, Eclipse or VS Code	§1.5
Idempotency key	unique id a client sends with a request so a retried request is processed only once	§5.1
Idempotent	repeating the operation gives the same result (GET, PUT, DELETE; not POST)	§5.1
Idempotent consumer	de-duplicates messages by id so at-least-once delivery is safe	§14.17
Idempotent producer	Kafka producer whose retries cannot duplicate records (on by default)	§14.16
Identity provider (IdP)	system that owns user accounts and issues tokens	§11.2
`IDENTITY` / `SEQUENCE`	JPA id strategies: auto-increment (no insert batching) / database sequence	§8.3
Integration test	test of several real parts working together (often a whole context plus a real database)	§12.2
Interceptor	code that runs around a call (method, handler or HTTP request) and can change or stop it	§5.11
Introspection	asking the authorization server whether an opaque token is active (RFC 7662)	§11.7
Inversion of Control (IoC)	the framework, not your code, creates objects and calls you	§2.1
Isolation level	how much a transaction sees of concurrent ones (READ COMMITTED...)	§7.11
Issuer (`iss`)	JWT claim naming the authorization server that created the token	§11.1
J2EE / Java EE	old names of the enterprise Java standards, now Jakarta EE	§1.4
Jackson 3	Boot 4's JSON library, packages `tools.jackson.*`	§5.5
Jakarta EE	successor of Java EE; `jakarta.*` packages (EE 11 in Boot 4)	§1.4
Jar / WAR	Java archive of compiled classes / web archive deployed into an external servlet container	§1.9
`JarLauncher`	`Main-Class` of a Boot fat jar that loads nested jars and calls your `Start-Class`	§1.9
Java record	compact immutable data class (`record Order(long id, String status)`); ideal for DTOs and properties	§1.11
JDBC	Java Database Connectivity: Java's standard low-level API for SQL databases	§7.1
`JdbcClient`	Framework 6.1 fluent API for SQL with automatic record mapping	§7.3
`JdbcTemplate`	classic SQL template; still used for batch updates	§7.3
JDK	Java Development Kit: compiler, tools and the Java runtime	§1.4
JDK dynamic proxy	`java.lang.reflect.Proxy` implementing the bean's interfaces only	§4.4
JEP	JDK Enhancement Proposal: the numbered design document for a new Java feature	§13.15
JIT compiler	Just-In-Time compiler: turns frequently run bytecode into machine code while the app runs	§15.11
Jitter	random variation added to retry delays so clients don't retry in lockstep	§4.7
JMS	Jakarta Messaging: the standard Java API for message brokers such as ActiveMQ Artemis	§14.18
JMX	Java Management Extensions: built-in JVM channel for monitoring and management tools	§15.2
Join point	a point where advice can run; in Spring AOP always a bean method execution	§4.1
`JOIN FETCH`	JPQL that loads an association in the same query (N+1 fix)	§8.7
JPA	Jakarta Persistence specification; Hibernate implements it	§8.1
JPQL	query language over entities and fields, not tables	§8.10
JSON	JavaScript Object Notation: the text format of most REST APIs (`{"id": 1}`)	§5.5
`JsonMapper`	Jackson 3's immutable JSON mapper, built with `JsonMapper.builder()`	§5.5
JSpecify	standard null-safety annotations (`@NullMarked` , `@Nullable`) used by Framework 7	§2.13
JUnit	the standard Java test framework (JUnit 6 / Jupiter in Boot 4)	§12.3
JVM	Java Virtual Machine: the runtime that loads and executes bytecode	§1.2
JWK Set (JWKS)	JSON document of an issuer's public keys, selected by `kid`	§11.1
JWS / JWE	a signed JWT (contents readable, tamper-proof) / an encrypted JWT (contents hidden)	§11.1
JWT	signed token `header.payload.signature` ; claims readable by anyone	§11.1
`JwtAuthenticationConverter`	turns JWT claims into authorities (`scope` → `SCOPE_x`)	§11.5

Term	Meaning	See
`JwtDecoder`	parses and validates a JWT (signature, `exp`, `iss`, custom validators)	§11.3

K – L

Term	Meaning	See
Kafka	distributed, replicated commit log for event streaming	§14.9
`KafkaTemplate`	spring-kafka producer API; `send` returns a `CompletableFuture`	§14.11
Key (Kafka)	decides the partition; same key → same partition → ordered	§14.9
Keycloak	open-source identity provider and authorization server	§11.2
KMS	Key Management Service: cloud service that stores encryption keys and performs crypto without revealing them	§11.13
KRaft	Kafka's built-in consensus replacing ZooKeeper (Kafka 4.0+)	§14.9
Kubernetes (K8s)	platform that runs, restarts, scales and connects containers across machines	§15.3
Lambda (Java)	short inline function, `x -> x * 2`; the Security DSL is configured with lambdas	§10.3
Latency	how long one request takes from start to finish (often reported as p50/p99)	§15.5
Layered jar	Boot jar split into layers (dependencies, application code) so container images rebuild only what changed	§15.11
Lazy initialization	create beans on first use (`@Lazy`, `spring.main.lazy-initialization`)	§2.10
`LazyInitializationException`	touching a lazy association after its persistence context closed	§8.6
LDAP	directory protocol used by companies to store users and groups; Spring Security can authenticate against it	§9.1
Lifecycle callbacks	init/destroy hooks: `@PostConstruct`, `afterPropertiesSet`, `@PreDestroy` ...	§2.7
Liquibase	changelog-based migration tool with rollbacks and preconditions	§9.4
Liveness / readiness / startup probes	Kubernetes health checks: restart me? / send me traffic? / am I still starting?	§15.3
Liveness probe	"restart me?" — must depend only on internal state	§15.3
LLM	large language model: the text-generating AI behind chat APIs	§17.1
Load balancer	component that spreads requests over several instances of a service	§16.5
Load balancing (client-side)	the caller picks an instance per request (Spring Cloud LoadBalancer, round robin)	§16.5
Load testing (Gatling, k6, JMeter)	tools that fire many concurrent requests to measure throughput and latency under load	§12.18
Logback / SLF4J	SLF4J is the logging API your code calls; Logback is Boot's default implementation behind it	§15.8
Lombok	library that generates getters, constructors, `equals` and similar code at compile time	§8.3
Lost update	two read-modify-write cycles overwrite each other; prevent with SQL math, locks or `@Version`	§7.11
LRU	least recently used: an eviction policy that removes the entry unused for longest	§9.8

M

Term	Meaning	See
Managed entity	entity tracked by the persistence context; changes are flushed automatically	§8.4
`mappedBy`	marks the inverse side of a JPA relationship; only the owning side is written	§8.5
Maven	Java build tool: `pom.xml` declares dependencies and plugins; `mvn verify` builds and tests	§1.6
MCP	Model Context Protocol: standard way to expose and call AI tools across processes	§17.10
MD5 / SHA-256	fast hash functions; fine for checksums, unsafe alone for passwords because they are fast to guess	§10.5
MDC	Mapped Diagnostic Context: per-thread log fields (request id, trace id)	§14.4, §15.1
`merge` vs `persist`	copy a detached entity's state onto a managed copy / make a new entity managed	§8.4
`MeterRegistry`	Micrometer registry where meters are recorded and exported	§15.5
Method security	authorization on bean methods (`@PreAuthorize`...), proxy-based	§10.7
Method validation	constraints on method parameters; built into MVC since Framework 6.1	§5.6
Metrics / logs / traces	the three kinds of observability data: numbers over time / event records / request paths	§15.1
Micrometer	vendor-neutral metrics facade ("SLF4J for metrics")	§15.1
Micrometer Tracing	tracing facade over OpenTelemetry or Brave	§15.7
MFA	multi-factor authentication; Security 7 tracks factors as `FACTOR_*` authorities	§10.4
Microservice	small, independently deployed service owning one business capability and its data	§16.1
Mock / stub / spy	test doubles: a mock records and checks calls, a stub returns canned answers, a spy wraps a real object	§12.4
Mockito	the Java mocking library behind `@MockitoBean`	§12.10
`MockMvc` / `MockMvcTester`	in-process MVC test client; `MockMvcTester` adds AssertJ assertions (F6.2)	§12.8
`MockRestServiceServer`	fake server bound to a `RestClient.Builder` in `@RestClientTest`	§6.8
Modular starters	Boot 4: one starter, one auto-configuration module and one test starter per technology	§3.14
MongoDB	document database storing JSON-like documents (BSON)	§9.5
`Mono`	Reactor publisher of 0..1 item	§13.3
Monolith / modular monolith	one deployable application; modular when its internal modules have strict boundaries (Spring Modulith)	§16.1
mTLS	mutual TLS: both client and server present certificates, so each proves its identity	§11.10
Multi-tenancy (JWT)	several issuers in one API; `JwtIssuerAuthenticationManagerResolver` with an allow-list	§11.8
MVCC	Multi-Version Concurrency Control: the database keeps row versions so readers don't block writers	§7.11

N – O

Term	Meaning	See
N+1 problem	1 query for parents + 1 per parent for a lazy association	§8.7
Native query	plain database-specific SQL in a Spring Data `@Query` instead of JPQL	§8.10
NDJSON	newline-delimited JSON: one JSON object per line, used for streaming responses	§13.10
`NESTED`	propagation using a savepoint inside the current transaction	§7.9
Netty	non-blocking network library; Reactor Netty is WebFlux's default server	§13.10
`NimbusJwtEncoder`	Spring Security's JWT signer; Security 7 adds `withKeyPair` / `withSecretKey` builders	§11.4
Non-blocking	a call that returns immediately and frees its thread; the result is delivered later	§13.1
Nonce	OIDC random value bound into the ID token against replay	§11.9
OAuth 2.0	framework for delegated authorization with access tokens	§11.2
`ObjectProvider`	lazy/optional lookup of a bean (`getIfAvailable` , `getObject` for prototypes)	§2.5
Observation API	Micrometer API: instrument once, get a metric and a span	§15.6
Offset (Kafka)	a record's position in its partition; committed per consumer group	§14.9
OIDC	OpenID Connect: identity layer on OAuth2 adding the ID token	§11.2
ONNX	portable file format for machine-learning models, e.g. local embedding models	§17.1
Opaque token	random token string checked by introspection; instantly revocable	§11.7
Open Session in View	persistence context kept open for the whole web request (`spring.jpa.open-in-view`)	§8.15
OpenAPI	machine-readable API description; generated by springdoc	§5.12
OpenFeign	Spring Cloud declarative client, in maintenance mode	§16.6
OpenRewrite	automated refactoring recipes used for Boot upgrades	§17.12
OpenTelemetry	open standard and SDK for traces/metrics; OTLP is its protocol	§15.7
Optimistic locking	`@Version` column checked in `UPDATE ... WHERE version = ?`	§8.14
ORM	Object-Relational Mapping: mapping classes to tables so you work with objects instead of SQL rows	§8.1
`orphanRemoval`	delete a child removed from its parent's collection	§8.5
OSIV	short for Open Session in View	§8.15
OTLP	OpenTelemetry Protocol: the standard wire format for exporting traces, metrics and logs	§15.7
Outbox (transactional)	write the event into an outbox table in the same transaction; a relay publishes it	§14.17
OWASP	Open Worldwide Application Security Project: publishes the Top 10 web security risks	§10.5
Owning side	the relationship side holding the foreign key; the only one JPA writes	§8.5

P

Term	Meaning	See
p99 / percentile	the value that 99% of measurements are below; p99 latency shows the slow tail	§15.5
Pact	polyglot consumer-driven contract-testing tool with a broker and `can-i-deploy` check	§16.10
`Page` / `Slice` / `Window`	paged result with total (extra COUNT) / "has next" only / keyset scrolling	§8.11
Partition	ordered, append-only log inside a Kafka topic; unit of parallelism	§14.9
Passkey / WebAuthn	password-less login with a key pair stored on the user's device; WebAuthn is the browser standard	§10.4
Password4j	hashing library behind the new password encoders of Spring Security 7	§10.5
`PathPattern`	Spring's URL pattern matcher (`{id}`, `**`, `{*path}`); no trailing-slash match	§5.3
`PathPatternRequestMatcher`	Security 7 request matcher using `PathPattern` syntax	§10.3
Persistence context	the `EntityManager`'s first-level cache and unit of work	§8.4
Pessimistic locking	`SELECT ... FOR UPDATE` via `@Lock(PESSIMISTIC_WRITE)`	§8.14
PKCE	proof key that binds an authorization code to the client that started the flow	§11.9
Platform thread	ordinary Java thread backed one-to-one by an operating-system thread	§13.1
`PlatformTransactionManager`	Spring interface that begins, commits and rolls back transactions	§7.7
Pod	smallest unit Kubernetes runs: one or more containers sharing a network address	§15.3
Pointcut	predicate selecting join points (`execution(...)`, `within(...)`, `@annotation(...)`)	§4.3
Poison pill	record that can never be deserialized; `ErrorHandlingDeserializer` sends it to the DLT	§14.10
POM	Project Object Model: Maven's `pom.xml` build file	§1.6
PostgreSQL	open-source relational database used for most examples in this book	§8.2
Precedence	which property source wins (command line > ... > defaults)	§3.3
Preflight	the browser's `OPTIONS` request asking CORS permission	§10.9
Primary key	column (or columns) that uniquely identifies each row of a table	§8.3
Principal	the "who" of a request: user, service or device	§10.1
`ProblemDetail`	RFC 9457 error body: `type`, `title`, `status`, `detail`, `instance`	§5.7
Profile	named environment selecting beans (`@Profile`) and files (`application-dev.yaml`)	§3.7
Profile group	one profile name that activates several	§3.7
Projection	query result with only the needed fields (interface or record)	§8.12
Prometheus	monitoring system that scrapes metrics from `/actuator/prometheus` and stores them as time series	§15.5
Prompt template	text with `{placeholders}` filled at run time (Spring AI `PromptTemplate`)	§17.4
Propagation	what a transactional method does when a transaction already exists	§7.9
Property source	one named set of key/values inside the `Environment`	§3.1
Prototype scope	new bean instance per lookup; no destroy callbacks	§2.6
Proxy	stand-in object that runs extra behaviour and then calls the real bean	§4.4
Publish/subscribe	messaging style where every subscriber gets its own copy of each message	§14.8
`publishOn` / `subscribeOn`	move downstream operators / the subscription to a scheduler	§13.8

Q – R

Term	Meaning	See
`QuestionAnswerAdvisor`	Spring AI advisor doing simple RAG over a `VectorStore`	§17.9
R2DBC	reactive (non-blocking) SQL API; not an ORM	§13.14
RabbitMQ	queue broker: exchanges route messages to queues via bindings	§14.18
RAG	retrieval-augmented generation: add retrieved documents to the prompt	§17.9
Rate limiter	cap on calls per time period (Resilience4j, gateway filter)	§16.8
Reactive programming	describing asynchronous data pipelines that are pushed through without blocking threads	§13.1
Reactive Streams	spec: `Publisher`, `Subscriber`, `Subscription`, `Processor` with demand	§13.2
Readiness probe	"send me traffic?" — may include essential dependencies	§15.3
Redis	in-memory key/value store used as a shared cache, session store or lock	§9.6
Reflection	Java code inspecting or calling classes, methods and fields at runtime	§2.4
Refresh (context)	`refresh()` creates all non-lazy singletons and starts the context	§2.2
Refresh token	long-lived token used only to get a new access token without logging in again	§11.1
Relaxed binding	several spellings map to one property (`checkout-timeout`, `checkoutTimeout`, `SHOP_CHECKOUTTIMEOUT`)	§3.4
Release train	set of versions released together (Spring Data 2026.0, Spring Cloud 2025.1)	§16.2
Replica / replication	a copy of data kept on another server; replication keeps the copies in sync	§7.12
Repository	Spring Data interface implemented at startup (`JpaRepository`, `CrudRepository`)	§8.8
`REQUIRES_NEW`	propagation that suspends the caller's transaction and runs its own	§7.9
Resilience4j	library for circuit breaker, retry, rate limiter, bulkhead, time limiter	§16.8
Resource server	API that accepts access tokens (`oauth2ResourceServer`)	§11.3
`ResponseEntity`	full control of status, headers and body in MVC	§5.4
`ResponseEntityExceptionHandler`	advice base class mapping Spring MVC exceptions to `ProblemDetail`	§5.7
REST	style of HTTP API built around resources (URLs) and standard methods and status codes	§5.1
`RestClient`	Framework 6.1 fluent blocking HTTP client	§6.2
`RestTemplate`	legacy blocking HTTP client; deprecation announced	§6.9
`RestTestClient`	Framework 7 fluent test client for MockMvc or a real server	§12.9
Retry	calling an operation again after a transient failure, usually with back-off and jitter	§4.7
`RetryTemplate`	programmatic retry (Framework 7 `org.springframework.core.retry`)	§4.7
Role	authority whose name starts with `ROLE_`	§10.6
Role hierarchy	`ROLE_ADMIN > ROLE_USER`: higher roles include lower ones	§10.6
Rollback-only	flag on a shared transaction that forces a rollback at commit	§7.9
Rollback rules	unchecked → rollback, checked → commit by default	§7.10

S

Term	Meaning	See
Saga	sequence of local transactions across services, with compensating steps that undo earlier ones on failure	§14.17
Salt	random value mixed into each password before hashing so equal passwords give different hashes	§10.5
SAML	older XML-based single sign-on standard used by enterprises; Spring Security supports it	§11.18
Sampling (tracing)	share of traces recorded; Boot default `0.1`	§15.7
Savepoint	point inside a transaction that can be rolled back to (`NESTED`)	§7.9
Scheduler (Reactor)	thread pool abstraction: `parallel`, `boundedElastic`, `single`	§13.8
Scope	how many bean instances exist and how long they live	§2.6
Scope (OAuth2)	permission requested for an access token (`orders.read`); mapped to `SCOPE_x` authorities	§11.1
Scoped proxy	stand-in that finds the request/session instance on every call	§2.6
Security headers (HSTS, CSP...)	response headers that make browsers safer: force HTTPS, restrict scripts, block framing	§10.11
`securityMatcher`	which requests a whole `SecurityFilterChain` handles	§10.3
`SecurityContextHolder`	`ThreadLocal` holder of the current `Authentication`	§10.2
`SecurityFilterChain`	ordered list of security filters for matching requests	§10.3
Self-invocation	`this.method()` inside a bean bypasses its proxy and all proxy features	§4.5
Service discovery	how a service finds the current addresses of another (Eureka, Consul, Kubernetes DNS)	§16.4
Service mesh	infrastructure layer of sidecar proxies handling traffic, mTLS and retries between services	§16.1
Servlet / servlet container	Java standard API for HTTP request handling / the server that runs servlets (Tomcat, Jetty)	§5.2
Session (HTTP)	server-side state for one browser, found through the `JSESSIONID` cookie	§10.10
Session fixation	attack reusing a pre-login session id; Spring changes the id at login	§10.10
`SessionCreationPolicy`	`IF_REQUIRED`, `ALWAYS`, `NEVER`, `STATELESS`	§10.10
ShedLock	library that lets a `@Scheduled` job run on one instance only	§14.7
SIGTERM / SIGKILL	operating-system signals: "please stop" (allows graceful shutdown) / "stop now" (cannot be handled)	§15.9
Singleton scope	one bean instance per container, shared by all threads (default)	§2.6
Sinks	Reactor API to push values into a `Flux` / `Mono` from imperative code	§13.9
Slice (test)	partial context for one layer (`@WebMvcTest`, `@DataJpaTest`...)	§12.7
`SmartLifecycle`	beans started/stopped with the context, ordered by phase	§2.7
SMTP	Simple Mail Transfer Protocol: how applications send e-mail	§14.4
SPA	single-page application: a browser app (React, Angular...) that calls APIs from JavaScript	§10.8
Span	one timed operation in a trace, with trace id, span id and parent	§15.7
SpEL	Spring Expression Language: `#{...}` in `@Value`, security and cache expressions	§3.5
Spock	Groovy-based test framework; supported again in Boot 4.1	§17.13
Spring AI	Spring project for LLM chat, embeddings, RAG, tools and MCP	§17.1
Spring Authorization Server	Spring's own OAuth2/OIDC authorization server, now part of Spring Security	§11.14
Spring Batch	framework for large, restartable batch jobs (read-process-write in chunks)	§1.3
Spring Boot	Spring + starters + managed versions + auto-configuration + embedded server + Actuator	§1.2
Spring Cloud	projects for distributed systems: config, discovery, gateway, load balancing	§16.1
Spring Cloud Stream	binder-based messaging abstraction over Kafka, RabbitMQ and others	§14.18
Spring Data	family of repository modules sharing one programming model	§9.1
Spring Data JDBC	aggregate-oriented mapping over plain JDBC, no persistence context	§9.2
Spring Data REST	exports repositories as HAL REST endpoints	§9.3
Spring Framework	the core project: container, AOP, MVC, WebFlux, transactions; Boot builds on it	§1.1

Term	Meaning	See
Spring Initializr	start.spring.io project generator	§1.5
Spring Modulith	project for structuring and verifying a modular monolith	§1.3
Spring Security	Spring's authentication and authorization framework, built from servlet filters	§10.1
Spring Session	stores HTTP sessions in Redis or JDBC so all instances share them	§10.10
SQL	Structured Query Language for relational databases	§7.1
SSE	Server-Sent Events: one-way server push over `text/event-stream`	§13.12
SSL	the old name of TLS; still used in property names (`server.ssl.*`)	§3.14
SSO	single sign-on: log in once and reach several applications	§11.2
Stack trace	list of method calls printed with an exception; read the last "Caused by" first	§5.7
Starter	code-free dependency bringing one capability's libraries and auto-configuration	§1.7
Stateless	keeps no per-client data between requests (no HTTP session); every request carries what it needs	§11.1
`StepVerifier`	reactor-test tool that asserts the signals of a publisher	§13.5
Stereotype annotation	`@Component` and its specialisations `@Service` , `@Repository` , `@Controller`	§2.3
Structured logging	JSON log lines (`ecs` , `logstash` , `gelf`), Boot 3.4+	§15.8
Structured output	converting an LLM answer into a Java type (`.entity(...)`)	§17.7
Swagger UI	web page that renders an OpenAPI description and lets you try the endpoints	§5.12
Synchronous	the caller waits until the operation finishes and has its result	§14.2

T

Term	Meaning	See
`TaskDecorator`	wraps submitted tasks to copy `ThreadLocal` s (MDC) to worker threads	§14.4
TCP	Transmission Control Protocol: reliable, ordered byte stream under HTTP, JDBC and Kafka connections	§6.4
Test pyramid	many fast unit tests, fewer slice tests, few full-context tests	§12.2
TestContext Framework	spring-test engine: `SpringExtension` , listeners, context cache	§12.5
Testcontainers	library starting real services in Docker for tests	§12.14
Thread / thread pool	a thread runs code concurrently; a pool keeps a fixed set of threads that take tasks from a queue	§14.4
Thread-per-request	classic servlet model: one thread per request from start to end	§13.1
`ThreadLocal`	variable with a separate value per thread; holds the security context and the transaction	§7.7
Throughput	how much work is done per unit of time (requests or messages per second)	§13.15
Time limiter	Resilience4j pattern that cancels a call taking longer than a set time	§16.8
Timeout (connect / read)	time to establish a connection / to wait for response data	§6.4
TL;DR	"too long; didn't read": the short summary at the end of a section	§1.1
Token exchange	OAuth2 grant (RFC 8693) that swaps one token for another, e.g. for a downstream service	§11.10
Token relay	forwarding the user's access token to downstream services	§11.10
Tool calling	the model requests a function; the application runs it (`@Tool`)	§17.8
Topic (Kafka)	named stream of records in Kafka, split into partitions	§14.8
TOTP	time-based one-time password: the 6-digit code from an authenticator app (a second factor)	§10.4
Trace / `traceparent`	a request's tree of spans / the W3C header that propagates it	§15.7
Transaction	group of database operations that succeed or fail together (ACID)	§7.7
`TransactionTemplate`	programmatic transaction around a lambda	§7.14
Transient entity	new object unknown to the persistence context	§8.4
TTL	time to live of a cache entry; a provider setting, not an annotation attribute	§9.8

U – W

Term	Meaning	See
Unchecked exception	`RuntimeException` or `Error`; need not be declared; rolls a transaction back by default	§7.10
Undertow	embedded web server removed in Boot 4 (no Servlet 6.1 support)	§17.13
`UnexpectedRollbackException`	commit of a transaction already marked rollback-only	§7.9
Unit of work	set of changes tracked and written together; the persistence context is one	§8.4
Unit test	fast test of one class in isolation, usually without Spring	§12.4
URI / URL	address of a resource; a URL is a URI that also says how to reach it (`https://...`)	§5.1
`UserDetailsService`	loads a user by name for password authentication	§10.4
UUID	128-bit random-looking unique id, often used as a primary key or message id	§8.3
Validation groups	subsets of constraints selected with `@Validated(Group.class)`	§5.6
Vault	HashiCorp secret store; Spring Cloud Vault loads secrets from it	§3.8
Vector store	database of embeddings for similarity search (`SimpleVectorStore`, PgVector)	§17.9
Virtual threads	cheap JVM-managed threads (Java 21); `spring.threads.virtual.enabled=true`	§13.15, §15.10
Virtual time	`StepVerifier` clock that advances only when the test says so	§13.5
W3C Trace Context	web standard defining the `traceparent` header used to propagate traces	§15.7
Weaving	linking aspects with targets; Spring does it at runtime with proxies	§4.1
`WebClient`	reactive, non-blocking HTTP client	§13.13
WebFlux	Spring's reactive web framework on Reactor Netty	§13.10
WebSocket	long-lived, two-way connection between browser and server	§13.12
`WebTestClient`	fluent test client for WebFlux (and MockMvc)	§13.12
WireMock	stub HTTP server for testing clients against canned responses	§6.8

X – Z

Term	Meaning	See
X25519 / ECDHE	elliptic-curve key-exchange methods TLS uses to agree on a fresh session key	§11.18
XSRF-TOKEN	cookie name in which Spring sends the CSRF token for JavaScript clients to echo back in a header	§10.8
XSS	cross-site scripting: attacker JavaScript running in your page, able to read anything scripts can	§10.11
YAML	indentation-based configuration format (`application.yaml`), an alternative to `.properties`	§3.2

Chapter TL;DR

- **Versions**: Boot 4.1.1 → Framework 7.0.9, Security 7.1.1, Hibernate 7.4, Jackson 3.1, JUnit 6, Java 17+ (25 used); Spring Cloud 2025.1, Spring AI 2.0 and springdoc 3 are versioned separately (§19.2).
- **Annotations** only work with their switch: `@EnableAsync`, `@EnableScheduling`, `@EnableCaching`, `@EnableMethodSecurity`, `@EnableResilientMethods`, `management.observations.annotations.enabled` — and only through the proxy (§19.3).
- **Starters** are modular in Boot 4: `-webmvc` (not `-web`), `-aspectj` (not `-aop`), `-security-oauth2-*`, `-restclient`, `-webclient`, `-flyway`, plus a `-test` starter per technology (§19.4).
- **Defaults worth remembering** (✔): Hikari pool 10 / 30 s wait, `@Async` pool 8 threads `task-`, one scheduler thread, 30 s shutdown phase, only `health` exposed, tracing sampling 0.1, OSIV on, graceful shutdown on (§19.5).
- **Transactions** (✔): `REQUIRED` joins, `REQUIRES_NEW` survives the caller's rollback, `NESTED` = savepoint, `MANDATORY` / `NEVER` throw; checked exceptions **commit**; a caught inner failure → `UnexpectedRollbackException` (§19.6).
- **Beans** (✔): singleton shared, prototype new per lookup and never destroyed; `@Qualifier` > `@Primary` > `@Fallback` > name; init `@PostConstruct` → `afterPropertiesSet` → `initMethod` (§19.7).
- **Security 7**: lambda DSL only; `hasRole("X")` = `ROLE_X`; 401 unknown vs 403 forbidden; CSRF on for cookies, `csrf.spa()` for SPAs; `http.cors()` before authentication (§19.8).
- **Tests**: unit → slice → a few `@SpringBootTest`s; `@MockitoBean` replaced `@MockBean`; explicit `@AutoConfigure...` for test clients in Boot 4; context cache max 32 (§19.9).
- **Choosing**: `RestClient`/HTTP interfaces for blocking calls, MVC + virtual threads for CRUD, WebFlux for streaming, JPA vs `JdbcClient` vs Data JDBC by model, Caffeine local vs Redis shared, Kafka log vs RabbitMQ queues, Resilience4j for circuit breakers (§19.12).

- **Upgrading**: 3.5 first, OpenRewrite, rename starters and properties, Jackson 3 imports, test starters, `@MockitoBean`, lambda-only security (§19.13).

This is the last chapter. Where to go from here:

- Spring Handbook front page — the chapter list, reading paths and how to run the code.
- Back to the start: Chapter 1 — The Spring Ecosystem and Your First App; revision questions: Chapter 18.
- Java Handbook — the language underneath Spring (records, generics, lambdas, concurrency and virtual threads).
- SQL Handbook — the queries, indexes and transaction isolation behind Chapters 7–9.
- Microservices patterns notes — gateway, discovery, circuit breaker, saga, CQRS and outbox, the designs Chapter 16 implements.

Chapter 20 — Interview Question Bank

This chapter collects the Spring questions that come up again and again in real interviews, worded the way candidates report them on LeetCode Discuss interview-experience posts, Glassdoor reviews, public Notion and GitHub question lists, and the big "top N Spring Boot questions" pages. It is a **drill list**: one place to check that you have an answer ready for everything an interviewer is likely to ask about Spring Framework 7, Spring Boot 4.1 and Spring Security 7 on Java 25.

Chapter 18 is the book's *teaching* round of interview questions (168 answers with proofs, a project-walkthrough template and debugging scenarios). This bank is wider and shorter: it adds the questions Chapter 18 does not ask (Spring vs Spring Boot, stereotype annotations, `CommandLineRunner`, `CrudRepository` vs `JpaRepository`, the Hibernate caches, `get` vs `load`, isolation anomalies, `@ControllerAdvice` details, JWT filter code, saga, idempotency keys, rate limiting, coding tasks, scenario questions and more), and for questions that Chapter 18 already answers it gives a fresh short answer plus a link to the Chapter 18 entry and the teaching section.

How to use this bank

- **Wording.** Each question is written the way it is usually asked, so you recognise it when you hear it. Several common wordings of the same question are merged into one entry.
- **★ = asked very often.** These appeared in most of the lists and interview reports used for this bank. If you are short on time, do only the ★ questions first.
- **Answers.** Each answer is short enough to say out loud in 30–60 seconds: what it is, how it works, and the consequence that bites people. Code snippets are kept to ten lines.
- **Links.** "→ *More:*" points to the section of this book that teaches the topic in depth (with runnable code and real output), and to the matching Chapter 18 question when there is one. A question with no link is not covered elsewhere in the book — its answer here is complete.
- **How to drill.** Cover the answer, say yours out loud, then compare. Mark the ones you missed and come back to them the next day. Before an interview, re-read only the ★ questions and the scenario section.
- **Versions.** Answers are correct for Spring Boot 4.1.1, Spring Framework 7.0, Spring Security 7.1, Spring Data 2026.0, Hibernate ORM 7.4 and Java 25 (Boot needs Java 17 at minimum). When behaviour differs in Boot 3, the answer says so.

A few words used everywhere below:

- **Bean** — an object created and managed by Spring's container.
- **Container** (or **application context**) — the Spring object that creates beans, connects them and manages their life.
- **Proxy** — a generated stand-in object that wraps a bean and runs extra code (transactions, caching, security checks) before and after the real method.
- **Starter** — a Maven/Gradle dependency that pulls in everything needed for one feature (for example `spring-boot-starter-webmvc`).

Spring and Spring Boot fundamentals

Q1. ★ What is the Spring Framework, and why is it used? Spring is an open-source Java framework whose core is a **container** that creates your objects and wires them together (dependency injection). On top of that core it offers modules for web apps (Spring MVC, WebFlux), data access and transactions, security, messaging, testing and more. Teams use it because it removes boilerplate (repetitive setup code), keeps classes loosely coupled and testable, and gives one consistent programming model for everything a back-end service needs. → *More:* §1.1 What Spring is

Q2. ★ What is Spring Boot? What is the difference between Spring and Spring Boot? Spring Boot is an opinionated layer on top of the Spring Framework that makes an application production-ready with almost no setup. Plain Spring gives you the building blocks but you must choose library versions, declare infrastructure beans and deploy to an external server. Boot adds **starters** (curated dependency sets), **auto-configuration** (beans created for you based on what is on the classpath), an **embedded server** (Tomcat runs inside your app, so you start it with `java -jar`), externalized configuration and Actuator for monitoring. Spring Boot is not a replacement for Spring — every Boot app *is* a Spring app.

Follow-up — What does "convention over configuration" mean in Spring Boot? The framework assumes sensible defaults so you only write configuration when you want something different. Examples: the server port is 8080, `application.properties` is read from the classpath, a `DataSource` (database connection pool) is created when a JDBC driver and a URL are present, and Jackson is configured for JSON. You override a convention with a property or your own bean; you never have to restate the default. → *More:* §1.2 What Spring Boot adds · §3.10 How auto-configuration works

Q3. ★ What are the main features and advantages of Spring Boot? Starters, auto-configuration, embedded servers (Tomcat by default, Jetty as an option; Undertow was removed in Boot 4), dependency management through a BOM (Bill of Materials — a list of library versions known to work together), externalized configuration with profiles, Actuator health and metrics endpoints, DevTools for fast restarts, and strong testing support (test slices, Testcontainers integration). The net effect: a working REST service in minutes, consistent versions, and sane production defaults that you can override one property at a time.

Follow-up — What are the key components of Spring Boot? Five pieces are usually expected: (1) **starters**, (2) **auto-configuration**, (3) the `SpringApplication` class that bootstraps the context and embedded server, (4) **externalized configuration** (`application.properties`/`.yaml`, environment variables, profiles), and (5) **Actuator** for production monitoring. Some lists add the

Spring Boot CLI and Spring Initializr. → *More:* §1.2 What Spring Boot adds · §1.7 Starters · §1.8 The main class and @SpringBootApplication

Q4. How do you create a Spring Boot project? The usual way is **Spring Initializr** — the website start.spring.io (also built into IntelliJ IDEA, VS Code and the Spring CLI) — where you choose the build tool, Java version, Boot version and starters, then download a ready project. You can also write the `pom.xml` by hand with `spring-boot-starter-parent` as the parent. The generated project has a main class annotated with `@SpringBootApplication` and a test that loads the context. → *More:* §1.5 Creating a project with start.spring.io · §1.6 Project layout and the build file

Q5. ⋆ What happens when a Spring Boot application starts? (How does `SpringApplication.run()` work?) `SpringApplication.run()` (1) decides the application type (servlet, reactive or none) from the classpath, (2) prepares the `Environment` by loading property sources and profiles, (3) creates the `ApplicationContext`, (4) registers bean definitions from component scanning and auto-configuration, (5) refreshes the context — creating all singleton beans and starting the embedded web server, (6) runs `CommandLineRunner` / `ApplicationRunner` beans, and (7) publishes `ApplicationReadyEvent`. With `java -jar`, a small launcher class first sets up a class loader for the nested jars inside the "fat jar" and then calls your `main`. → *More:* §1.9 Running the application · Ch 18 Q22

Q6. What is an embedded server, and why is it useful? Can you replace Tomcat? An embedded server is a web server library (Tomcat by default) started inside your application's own JVM, instead of deploying a WAR file into a separately installed server. One artifact, one command, same runtime in every environment — ideal for containers. To switch to Jetty, exclude `spring-boot-starter-tomcat` and add `spring-boot-starter-jetty`. Boot 4 supports Tomcat 11 and Jetty 12.1; Undertow was dropped because it does not support Servlet 6.1.

Follow-up — Can you build a non-web Spring Boot application? Can you disable the web server? Yes. Leave out the web starter, or set `spring.main.web-application-type=none`. The context still starts, beans are created, and `CommandLineRunner` beans do the work — typical for batch jobs, CLI tools and Kafka consumers that need no HTTP endpoints. → *More:* §1.9 Running the application · §17.13 Spring Boot 3.x → 4.x migration table

Q7. What is the default port, and how do you change it? 8080. Change it with `server.port=9090` in `application.properties`, an environment variable `SERVER_PORT=9090`, or a command-line argument `--server.port=9090`. `server.port=0` picks a random free port (useful in tests); `server.port=-1` starts no HTTP listener at all. → *More:* §3.3 Precedence: which value wins?

Q8. ⋆ What are `CommandLineRunner` and `ApplicationRunner`? How do you run code at startup? Both are interfaces with a single `run` method that Boot calls once, after the context is fully started and before `ApplicationReadyEvent`. `CommandLineRunner.run(String... args)` gets the raw arguments; `ApplicationRunner.run(ApplicationArguments args)` gets them parsed (options such as `--name=x` vs plain values). Order several runners with `@Order`. Other options: an `@EventListener(ApplicationReadyEvent.class)` method (runs when the app is ready to serve traffic) or `@PostConstruct` (runs while the bean is being created — too early for work that needs other beans to be fully ready).

```
@Bean
CommandLineRunner seed(ProductRepository repo) {
    return args -> repo.save(new Product("demo"));
}
```

Follow-up — How do you read command-line arguments or properties in code? Inject `ApplicationArguments` (parsed options and non-option args), use `@Value("${name}")`, or inject `Environment` and call `getProperty("name")`. Any `--key=value` argument becomes a property with the highest precedence, so normal `@Value` / `@ConfigurationProperties` binding sees it. → *More:* §1.9 Running the application · §2.7 The bean lifecycle · §3.1 Externalized configuration and the Environment

Q9. WAR vs executable JAR deployment — what is the difference? An executable ("fat") JAR contains your classes, all dependencies and the embedded server; you run it with `java -jar`. A WAR (Web Application Archive) is deployed into an external servlet container that is managed separately. Boot can still build a WAR (extend `SpringBootServletInitializer`, mark the Tomcat starter as `provided`), but JARs are the norm today because they fit containers and need no shared server. → *More:* §1.6 Project layout and the build file · §15.11 Faster startup: extracted jars, CDS, AOT cache, native images

Q10. What is the Spring Boot starter parent, and how does dependency management work? `spring-boot-starter-parent` is a Maven parent POM that imports the Boot BOM (`spring-boot-dependencies`), so you declare dependencies *without versions* and get a tested combination; it also sets the Java version, encoding, resource filtering and plugin defaults. If your project already has a parent, import `spring-boot-dependencies` in `<dependencyManagement>` instead. Override one version with a property such as `<jackson.version>`. → *More:* §1.6 Project layout and the build file

Q11. What is Spring Boot DevTools? Is it used in production? DevTools speeds up local development: automatic restart when classes change (using two class loaders so only your code reloads), and development-friendly defaults such as disabled template caching. Its LiveReload browser-refresh server is off by default in Boot 4 and deprecated since 4.1. It switches itself off when the app runs from a packaged jar (`java -jar`), and should be declared `optional` / `developmentOnly` so it never ships. → *More:* §1.10 DevTools · Ch 18 Q35

Q12. Which Java version does Spring Boot 4 need, and what are the headline changes in Boot 4? Java 17 at minimum (Java 25 recommended). Headlines: Spring Framework 7, modularized auto-configuration (one module and starter per technology), Jakarta EE 11 (Servlet 6.1, JPA 3.2), Jackson 3, JSpecify null-safety annotations, built-in API versioning, HTTP service client auto-configuration, `@MockBean` removed in favour of `@MockitoBean`, core `@Retryable`, Spring Security 7's lambda-only DSL, and Undertow removed. → *More:* §17.13 Spring Boot 3.x → 4.x migration table · Ch 18 Q166

IoC container, dependency injection, beans and scopes

Q13. ⋆ What is Inversion of Control (IoC)? What is Dependency Injection (DI)? How are they different? IoC is the general principle: your code no longer creates and controls the objects it depends on — a framework does, and calls into your code. DI is the specific technique Spring uses to achieve IoC: a class *declares* what it needs (usually as constructor parameters) and the container *injects* matching beans. IoC is the "what" (control moves to the framework); DI is the "how". The payoff is loose coupling: code depends on interfaces, and you can pass a fake in a unit test. → *More:* §2.1 Inversion of Control and Dependency Injection · Ch 18 Q1

Q14. What is the IoC container? What does it do? The IoC container is the `ApplicationContext`. It reads **configuration metadata** (annotations, `@Bean` methods, and rarely XML), builds a `BeanDefinition` for each bean (class, scope, dependencies, init/destroy methods), creates the beans in dependency order, injects them into each other, applies post-processors (which add proxies for `@Transactional`, `@Async` and so on), and destroys them on shutdown. → *More:* §2.2 The container: BeanFactory vs ApplicationContext · §2.3 Defining beans

Q15. ⋆ `BeanFactory` vs `ApplicationContext`? `BeanFactory` is the basic container interface: create beans lazily, on the first `getBean` call. `ApplicationContext` extends it and adds what real applications need: eager creation of singletons at startup (so wiring errors fail fast), automatic registration of `BeanPostProcessor`s, event publishing, the `Environment` with profiles and properties, internationalized messages and resource loading. You always use an `ApplicationContext`; `BeanFactory` matters only for understanding. → *More:* §2.2 The container: BeanFactory vs ApplicationContext · Ch 18 Q3

Q16. What is a Spring bean? Why make an object a bean instead of calling `new`? A bean is an object whose creation, wiring and lifecycle the container manages. Being a bean lets Spring inject its dependencies, apply cross-cutting behaviour through proxies (`@Transactional`, `@Cacheable`, `@PreAuthorize` only work on beans), manage its scope and call its init/destroy callbacks. An object made with `new` gets none of this. Simple value objects (DTOs — data transfer objects, entities, records) should *not* be beans. → *More:* §2.3 Defining beans · Ch 18 Q2

Q17. ⋆ What are the types of dependency injection? Which one is recommended and why? Constructor, setter and field injection. Prefer **constructor injection**: fields can be `final`, the object is never half-built, the dependencies are visible in one place, unit tests can just call `new`, and circular dependencies fail at startup instead of hiding. Setter injection is for truly optional or changeable dependencies. Field injection (`@Autowired` on a private field) hides dependencies and needs reflection or Spring to test; keep it for test classes.

Follow-up — Is `@Autowired` required on a constructor? Not when the class has exactly one constructor — Spring uses it automatically (since Framework 4.3). With several constructors, annotate the one Spring should use. This is why a Lombok `@RequiredArgsConstructor` or a hand-written single constructor needs no annotation. → *More:* §2.4 Injection types · Ch 18 Q6

Q18. ⋆ What is the difference between `@Component`, `@Service`, `@Repository` and `@Controller`? All four are **stereotype annotations**: they mark a class for component scanning so it becomes a bean. `@Service` and `@Controller`/`@RestController` are meta-annotated with `@Component` and mainly document the layer (`@Controller` is also detected by Spring MVC as a request handler). `@Repository` has one real extra: a post-processor translates database exceptions thrown by that bean into Spring's `DataAccessException` hierarchy. Technically you could use `@Component` everywhere, but the specific ones make intent clear and allow pointcuts per layer. → *More:* §2.3 Defining beans · §7.5 Exception translation: DataAccessException

Q19. ⋆ `@Component` vs `@Bean`? `@Component` goes on *your own class* and the class is picked up by scanning. `@Bean` goes on a *method* in a `@Configuration` class; the method's return value becomes the bean. Use `@Bean` for third-party classes you cannot annotate (an `ObjectMapper`, a `RestClient`), or when creation needs logic or conditions. → *More:* §2.3 Defining beans · Ch 18 Q4

Q20. What does `@Configuration` do? What happens if you put `@Bean` methods in a plain `@Component`? `@Configuration` marks a class as a source of bean definitions. By default (`proxyBeanMethods = true`) Spring generates a CGLIB subclass so that when one `@Bean` method calls another, the call returns the existing singleton instead of building a new object. In a plain `@Component` (or with `proxyBeanMethods = false`, "lite mode") that call is a normal Java call and creates a second instance. Prefer passing dependencies as `@Bean` method parameters — then both modes behave the same. → *More:* §2.3 Defining beans · Ch 18 Q5

Q21. What is component scanning? What does `@ComponentScan` do? At startup Spring searches packages for classes annotated with stereotypes and registers them as beans. `@SpringBootApplication` includes `@ComponentScan` for the package of the main class and all sub-packages. That is why the main class belongs in the root package: a `@Service` in a sibling package outside it is silently not found, and you get "No qualifying bean". Use `@ComponentScan(basePackages = ...)` or `@Import` for code elsewhere. → *More:* §2.3 Defining beans · §1.8 The main class and @SpringBootApplication

Q22. ⋆ Two beans of the same type exist. How does Spring decide which one to inject? (`@Primary` vs `@Qualifier`) Spring matches by type. With several candidates it uses, in order: `@Qualifier("name")` at the injection point; a bean marked `@Primary`; the only bean not marked `@Fallback` (Framework 6.2+); a bean whose name equals the parameter name. Otherwise startup fails with `NoUniqueBeanDefinitionException`. `@Primary` is a default chosen by the bean's author; `@Qualifier` is a specific choice made by the consumer — so `@Qualifier` wins. → *More:* §2.5 How Spring picks a bean · Ch 18 Q7 · Ch 18 Q8

Q23. How do you inject all implementations of an interface? Inject a `List<PaymentGateway>` (ordered by `@Order` or `Ordered`) or a `Map<String, PaymentGateway>` keyed by bean name. This is the standard way to build a strategy pattern: the map lets you pick an implementation by a key at runtime without `if`/`switch` chains.

```
@Service
class PaymentRouter {
    private final Map<String, PaymentGateway> gateways;
    PaymentRouter(Map<String, PaymentGateway> gateways) { this.gateways = gateways; }
    PaymentGateway forType(String type) { return gateways.get(type + "Gateway"); }
}
```

→ *More:* §2.5 How Spring picks a bean

Q24. `@Autowired` vs `@Inject` vs `@Resource`? `@Autowired` is Spring's own annotation: resolves by type, has a `required` flag. `@Inject` (Jakarta Dependency Injection, `jakarta.inject`) behaves like `@Autowired` without `required`; pair it with `@Named` to qualify. `@Resource` (Jakarta Annotations) resolves **by name first**, then by type. In modern code, constructor injection without any annotation makes the choice mostly irrelevant. → *More:* §2.4 Injection types

Q25. How do you inject an optional dependency? Use `ObjectProvider<T>` (`getIfAvailable()`, `ifAvailable(...)`), `Optional<T>`, `@Autowired(required = false)` on a setter, or a nullable parameter annotated with JSpecify `@Nullable`. `ObjectProvider` is the most flexible: it is also lazy and can stream several candidates. → *More:* §2.4 Injection types · Ch 18 Q9

Q26. ⋆ What are the bean scopes? What is the default? Singleton (default — one instance per container), **prototype** (a new instance every time the bean is requested), and in web apps **request** (one per HTTP request), **session** (one per HTTP session), **application** (one per `ServletContext`) and **websocket**. Note: Spring's singleton is "one per container", not the GoF singleton pattern of "one per class loader". → *More:* §2.6 Bean scopes · Ch 18 Q10

Q27. Singleton vs prototype scope — when would you use prototype? A singleton is created once at startup and shared by every caller, so it must be stateless or thread-safe. A prototype is created fresh on every lookup and is **not destroyed by Spring** — the container hands it over and forgets it, so `@PreDestroy` never runs. Use prototype for objects that hold per-use mutable state (a builder, a non-thread-safe parser). In most services everything is a singleton. → *More:* §2.6 Bean scopes

Q28. ⋆ Are singleton beans thread-safe? No. Spring does nothing to make them thread-safe; it only guarantees one shared instance. Because one instance serves all concurrent requests, a singleton must be **stateless** — no mutable instance fields holding per-request data. If shared state is unavoidable, use thread-safe types (`ConcurrentHashMap`, `AtomicLong`), immutable objects, or move the state into a request-scoped bean or a method's local variables. → *More:* §2.6 Bean scopes

Q29. ⋆ What happens when you inject a prototype bean into a singleton? How do you fix it? The prototype is created once — when the singleton is built — and the singleton keeps that one instance forever, so it behaves like a singleton. Fixes: inject `ObjectProvider<MyProto>` and call `getObject()` each time; use a `@Lookup` method that Spring overrides to return a new instance; or use a scoped proxy (`@Scope(value = "prototype", proxyMode = ScopedProxyMode.TARGET_CLASS)`). → *More:* §2.6 Bean scopes · Ch 18 Q11 · §18.15 Proofs: five tricky answers, run

Q30. How can a singleton use a request-scoped or session-scoped bean? Inject a **scoped proxy**: declare the request-scoped bean with `proxyMode = ScopedProxyMode.TARGET_CLASS` (`@RequestScope` does this by default). The singleton receives a proxy; each method call on it finds the real instance bound to the current request. Calling it outside a request (in a scheduled job or another thread) throws `IllegalStateException` / "No thread-bound request found". → *More:* §2.6 Bean scopes · Ch 18 Q12

Q31. ⋆ Explain the Spring bean lifecycle.

1. Instantiate (constructor runs, constructor injection). 2) Populate properties (setter and field injection). 3) `Aware` callbacks (`BeanNameAware`, `ApplicationContextAware`). 4) `BeanPostProcessor.postProcessBeforeInitialization`. 5) Init callbacks — `@PostConstruct`, then `InitializingBean.afterPropertiesSet()`, then a custom `initMethod`. 6) `BeanPostProcessor.postProcessAfterInitialization` — **this is where AOP proxies are created**. 7) The bean is in use. 8) On shutdown, `@PreDestroy`, `DisposableBean.destroy()`, then `destroyMethod` (singletons only).

Follow-up — What are the `Aware` interfaces? Callback interfaces that let a bean receive a container object: `BeanNameAware` (its own bean name), `ApplicationContextAware` (the context), `EnvironmentAware`, `ResourceLoaderAware` and others. They are rarely needed in application code because you can simply inject `ApplicationContext` or `Environment` through the constructor. → *More:* §2.7 The bean lifecycle · Ch 18 Q13

Q32. `@PostConstruct` vs constructor vs `InitializingBean` — where should initialization code go? Put simple, self-contained setup in the constructor. Use `@PostConstruct` (from `jakarta.annotation`) when the code needs field/setter-injected values, or must run after injection but before the bean is used. `InitializingBean.afterPropertiesSet()` does the same but couples your class to Spring. Note: inside `@PostConstruct` the bean's own proxy does not exist yet, so a `@Transactional` method called from there runs without a transaction — use `ApplicationReadyEvent` for that. → *More:* §2.7 The bean lifecycle

Q33. `BeanPostProcessor` vs `BeanFactoryPostProcessor`? A `BeanFactoryPostProcessor` runs **before any bean is created** and can change bean *definitions* (for example, resolving `${...}` placeholders). A `BeanPostProcessor` runs **around the initialization of each bean instance** and can modify or wrap the instance — this is how `@Autowired` processing, `@Async` and AOP proxies work. Declare a `BeanFactoryPostProcessor` `@Bean` method as `static` so it doesn't force early creation of its configuration class. → *More:* §2.8 Post-processors: the container's extension points · Ch 18 Q14

Q34. ⋆ What is a circular dependency? How does Spring handle it, and how do you fix it? Bean A needs B and B needs A. With constructor injection Spring cannot create either and startup fails with `BeanCurrentlyInCreationException`. Since Boot 2.6 circular references are forbidden by default even for field/setter injection (`spring.main.allow-circular-references=false`). Real fixes: redesign (extract the shared logic into a third bean, or use events), or as a stop-gap make one side lazy (`@Lazy` on the parameter injects a proxy) or use `ObjectProvider`. → *More:* §2.9 Circular dependencies · Ch 18 Q15 · §18.15 Proofs: five tricky answers, run

Q35. What does `@Lazy` do? What does lazy initialization cost? `@Lazy` on a bean delays creating it until the first time it is needed; on an injection point it injects a proxy that resolves the real bean on first use. `spring.main.lazy-initialization=true` makes all beans lazy, which shortens startup but moves errors and latency to the first request, and hides misconfiguration until runtime. Use it selectively (heavy, rarely-used beans) and in development. → *More:* §2.10 Lazy initialization · Ch 18 Q16

Q36. What does `@DependsOn` do? It forces one bean to be initialized after the named beans, even when there is no injection between them (for example, a cache warmer that must run after a schema migration bean). On shutdown, the dependent bean is destroyed before the beans it depends on. Prefer a real dependency or events; `@DependsOn` is for hidden, side-effect ordering only.

Q37. How do application events work in Spring? Are they synchronous? A bean publishes an object with `ApplicationEventPublisher.publishEvent(...)`; methods annotated with `@EventListener` that accept that type are called. By default listeners run **synchronously on the publisher's thread** and inside its transaction, so a listener exception propagates to the publisher. Add `@Async` to run a listener on another thread, or use `@TransactionalEventListener` to run only after the transaction commits. → *More:* §2.12 Application events · Ch 18 Q18

Q38. How do you get the list of all beans in an application? Call `applicationContext.getBeanDefinitionNames()` (for example in a `CommandLineRunner`), or expose the Actuator `beans` endpoint (`/actuator/beans`), which lists each bean with its scope, type and dependencies. The `conditions` endpoint shows why auto-configured beans were or were not created. → *More:* §15.2 Spring Boot Actuator · §3.11 Seeing what Boot decided

Q39. (Framework 7) What are JSpecify null-safety annotations and `BeanRegistrar`? Framework 7 and Boot 4 annotate their APIs with JSpecify `@Nullable` / `@NullMarked` (a standard set of annotations that say whether a value may be null), so tools like NullAway and IDEs can catch null bugs at compile time. `BeanRegistrar` is a new interface for registering beans programmatically (for example, in a loop or based on properties) in a way that also works with AOT and native images; you `@Import` it from a configuration class. → *More:* §2.13 Null-safety with JSpecify (Framework 7) · §2.14 Programmatic registration with BeanRegistrar (Framework 7) · Ch 18 Q19 · Ch 18 Q20

Configuration, properties, profiles, auto-configuration and starters

Q40. ⋆ What does `@SpringBootApplication` do? Which three annotations does it combine? It combines `@SpringBootConfiguration` (a specialized `@Configuration`, so the main class can declare `@Bean`s), `@EnableAutoConfiguration` (turns on auto-configuration) and `@ComponentScan` (scans the main class's package and its sub-packages). Its `exclude` attribute disables chosen auto-configurations.

Follow-up — `@SpringBootApplication` vs `@EnableAutoConfiguration` — what is the difference? `@EnableAutoConfiguration` only turns on auto-configuration. `@SpringBootApplication` adds component scanning and marks the class as a configuration class. You would use `@EnableAutoConfiguration` alone only in unusual setups where you want to control scanning yourself. → *More:* §1.8 The main class and @SpringBootApplication · Ch 18 Q21

Q41. ⋆ What is auto-configuration, and how does it work internally? Auto-configuration is a set of `@AutoConfiguration` classes that Boot loads from the file `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports` in each jar. Every class is guarded by **conditions** — `@ConditionalOnClass` (a library is on the classpath), `@ConditionalOnMissingBean` (you haven't defined your own), `@ConditionalOnProperty`, `@ConditionalOnWebApplication` and so on. They are processed after your own beans, so your definitions win and the auto-configured default "backs off". In Boot 4 these classes live in one module per technology (`spring-boot-webmvc`, `spring-boot-jdbc`, ...). → *More:* §3.10 How auto-configuration works · §3.14 Boot 4 modularization · Ch 18 Q24

Q42. `spring.factories` vs `AutoConfiguration.imports`? Before Boot 2.7, auto-configuration classes were listed in `META-INF/spring.factories` under the `EnableAutoConfiguration` key. Boot 2.7 introduced the dedicated `.imports` file, and since Boot 3.0 auto-configuration is read **only** from it. `spring.factories` is still used for other extension points (`EnvironmentPostProcessor`, `ApplicationListener`, failure analyzers). → *More:* §3.10 How auto-configuration works · §3.13 Writing your own auto-configuration and starter

Q43. How do you see which auto-configurations were applied, and why? Start with `--debug` (or `debug=true`) to print the **condition evaluation report**: "positive matches", "negative matches" with the failed condition, exclusions and unconditional classes. In a running app, Actuator's `/actuator/conditions` endpoint gives the same data as JSON. `/actuator/beans` shows what was created. → *More:* §3.11 Seeing what Boot decided · Ch 18 Q26

Q44. How do you disable or exclude a specific auto-configuration? `@SpringBootApplication(exclude = DataSourceAutoConfiguration.class)`, or the property `spring.autoconfigure.exclude=<fully qualified class name>` (handy per profile). Often you don't need to exclude: just define your own bean of that type and `@ConditionalOnMissingBean` makes Boot's default back off. In Boot 4 remember that the class names moved to per-technology packages. → *More:* §3.12 Overriding and excluding auto-configuration · Ch 18 Q25

Q45. How does `@ConditionalOnMissingBean` work? What conditional annotations do you know? It registers the bean only if no bean of that type (or name) exists *at the time the condition is evaluated*. Because auto-configurations are processed after user configuration, it lets your bean replace the default. Its subtle trap: it only sees beans already registered, so it is reliable in auto-configuration classes but not in ordinary `@Configuration` classes. Others: `@ConditionalOnClass` / `OnMissingClass`, `@ConditionalOnBean`, `@ConditionalOnProperty` (with `havingValue`, `matchIfMissing`), `@ConditionalOnResource`, `@ConditionalOnWebApplication`, `@ConditionalOnExpression`, `@ConditionalOnThreading` (platform vs virtual threads). → *More:* §3.10 How auto-configuration works · §2.11 @Conditional and profiles

Q46. What is a starter? Name some common starters. A starter is a dependency-only artifact that pulls in a coherent set of libraries for one feature, plus the matching auto-configuration module. Common Boot 4 names: `spring-boot-starter-webmvc` (renamed from `-web`, which still works but is deprecated), `spring-boot-starter-webflux`, `spring-boot-starter-data-jpa`, `spring-boot-starter-security`, `spring-boot-starter-validation`, `spring-boot-starter-actuator`, `spring-boot-starter-aspectj` (formerly `-aop`), `spring-boot-starter-kafka`, `spring-boot-starter-test`. Boot 4 also adds a `-test` starter per technology (e.g. `spring-boot-starter-webmvc-test`). → *More:* §1.7 Starters · §3.14 Boot 4 modularization · Ch 18 Q23

Q47. ⋆ How do you create your own custom starter? Two modules by convention: an **autoconfigure** module containing an `@AutoConfiguration` class with conditions (`@ConditionalOnClass`, `@ConditionalOnMissingBean`) and an `@ConfigurationProperties` class, registered in `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports`; and a **starter** module that only depends on it and on the libraries. Name it `acme-spring-boot-starter` (the `spring-boot-` prefix is reserved for Spring's own). Test it with `ApplicationContextRunner`. → *More:* §3.13 Writing your own auto-configuration and starter · Ch 18 Q33

Q48. What is the purpose of `application.properties` / `application.yml`? Which is better? They hold externalized configuration — values that change between environments without recompiling (URLs, credentials, timeouts, feature flags). Both are equivalent; YAML is hierarchical and supports several documents in one file (separated by `---`), properties files are flat and harder to mis-indent. If both exist in the same location, `.properties` wins for duplicate keys. Pick one per project. → *More:* §3.2 application.properties vs application.yaml

Q49. ⋆ In what order does Spring Boot load properties? How do you override a property? Higher wins (simplified, most important first): command-line arguments → Java system properties (`-D`) → OS environment variables → profile-specific files outside the jar → `application.properties` outside the jar → profile-specific files inside the jar → `application.properties` inside the jar → `@PropertySource` → defaults. Note the trap: any file *outside* the jar beats every file packaged *inside* it, even a profile-specific one. So you override any packaged value with an environment variable or a `--key=value` argument at deploy time. → *More:* §3.3 Precedence: which value wins? · Ch 18 Q27

Q50. How does an environment variable map to a property (relaxed binding)? Uppercase, replace `.` with `_`, drop `-`: `spring.datasource.url` ↔ `SPRING_DATASOURCE_URL`, `my-app.retry-count` ↔ `MYAPP_RETRYCOUNT`. A list index becomes its own underscore-separated segment: `my-app.hosts[0]` ↔ `MYAPP_HOSTS_0`, and `my.service[0].other` ↔ `MY_SERVICE_0_OTHER`. Relaxed binding also accepts `retryCount`, `retry-count` and `retry_count` in files; the kebab-case form is canonical. → *More:* §3.4 Environment variables and relaxed binding · Ch 18 Q28

Q51. ⋆ `@Value` vs `@ConfigurationProperties`? `@Value("${app.timeout:5s}")` injects one value, supports SpEL (Spring Expression Language) and is fine for one or two settings. `@ConfigurationProperties(prefix = "app")` binds a whole group of properties to a typed object (ideally a Java record): relaxed binding, nested objects, lists, `Duration`/`DataSize` conversion, validation with `@Validated`, and IDE auto-completion via generated metadata. For more than a couple of related settings, use `@ConfigurationProperties`.

```
@ConfigurationProperties("app.payment")
@Validated
record PaymentProps(@NotBlank String url, Duration timeout, int retries) {}
```

Follow-up — How do you make `@ConfigurationProperties` classes available? What does `@EnableConfigurationProperties` do? Register them with `@ConfigurationPropertiesScan` on the main class (scans like component scanning) or `@EnableConfigurationProperties(PaymentProps.class)` on a configuration class (typical in auto-configurations). Marking the class `@Component` also works but is discouraged. Records and single-constructor classes use constructor binding automatically. → *More:* §3.5 @Value · §3.6 @ConfigurationProperties — type-safe configuration · Ch 18 Q29

Q52. How do you make the application fail fast on invalid or missing configuration? Put `@Validated` on the `@ConfigurationProperties` class and Bean Validation constraints (`@NotBlank`, `@Min`, `@NotNull`) on its fields — binding fails at startup with a clear message. Avoid defaults that hide a missing secret (`${DB_PASSWORD:}`). Spring's `FailureAnalyzer`s turn common startup errors into readable reports. → *More:* §3.6 @ConfigurationProperties — type-safe configuration · Ch 18 Q30

Q53. ⋆ What are profiles? How do you activate one? A profile is a named set of beans and configuration that is active only in some environments (`dev`, `test`, `prod`). Profile-specific files are `application-{profile}.properties`. Activate with `spring.profiles.active=prod` (property, `SPRING_PROFILES_ACTIVE` env var, or `--spring.profiles.active=prod`), or `@ActiveProfiles` in tests. Beans can be restricted with `@Profile("dev")` or `@Profile("!prod")`. Profile groups (`spring.profiles.group.prod=proddb,prodmq`) activate several at once. → *More:* §3.7 Profiles, profile-specific files and profile groups · Ch 18 Q31

Q54. `@Profile` vs `@Conditional`? `@Profile` is a special, simple condition: "register this bean only if these profiles are active". `@Conditional` (and Boot's `@ConditionalOn...` family) can test anything — a class on the classpath, a property value, another bean. Prefer property-based conditions for feature switches and keep profiles for environment differences. → *More:* §2.11 @Conditional and profiles · Ch 18 Q17

Q55. How do you keep secrets (passwords, API keys) out of `application.properties`? Never commit them. Inject them at runtime through environment variables, Kubernetes Secrets mounted as files and imported with `spring.config.import=configtree:/run/secrets/`, or a secret store (HashiCorp Vault, AWS Secrets Manager, Azure Key Vault) via the matching Spring Cloud integration. Rotate them and avoid logging them; Actuator masks values of sensitive keys by default. → *More:* §3.8 Importing configuration: spring.config.import · §11.13 Storing keys and secrets · Ch 18 Q32

Q56. What is `bootstrap.properties`, and is it still used? In older Spring Cloud versions `bootstrap.properties` was loaded by a parent "bootstrap context" before `application.properties`, mainly to locate the Config Server. Since Spring Cloud 2020 it is disabled by default; you use `spring.config.import=configserver:` in the normal `application.properties` instead. You can re-enable the legacy mechanism with `spring-cloud-starter-bootstrap`, but new projects should not. → *More:* §16.3 Centralized configuration: Spring Cloud Config

Q57. How do you enable HTTPS in a Spring Boot application? Configure an **SSL bundle** — a named set of key and trust material — and point the server at it: `spring.ssl.bundle.pem.web.keystore.certificate=...`, `...private-key=...`, then `server.ssl.bundle=web` and `server.port=8443`. Bundles can be reused for clients (`RestClient`, Kafka, Redis), and with `reload-on-update=true` the server picks up renewed certificate files without a restart. In Kubernetes, TLS is often terminated at the ingress instead. → *More:* §11.13 Storing keys and secrets

Q58. How do you configure logging in Spring Boot? How do you enable debug logging? Boot uses SLF4J (a logging facade) with Logback by default. Set levels with `logging.level.root=INFO` and `logging.level.com.acme=DEBUG`; write to a file with `logging.file.name`; switch to JSON with `logging.structured.format.console=ecs` (or `logstash`, `gelf`). `--debug` turns on debug logs for core loggers and prints the condition report. Levels can be changed at runtime through `/actuator/loggers`. → *More:* §15.8 Structured logging

---

AOP and proxies

Q59. ⋆ What is AOP? Explain aspect, advice, pointcut, join point and weaving. AOP (aspect-oriented programming) moves **cross-cutting concerns** — logic needed in many places such as logging, transactions, security and metrics — out of business code. An **aspect** is the module holding that logic. A **join point** is a point in execution where it can run (in Spring AOP: a method call on a bean). A **pointcut** is an expression selecting join points (`execution(* com.acme.service..*(..))`). **Advice** is the code that runs there (`@Before`, `@After`, `@AfterReturning`, `@AfterThrowing`, `@Around`). **Weaving** is linking aspects into the target — Spring does it at runtime with proxies. → *More:* §4.1 Cross-cutting concerns and the AOP vocabulary · Ch 18 Q36

Q60. How does Spring AOP work internally? After a bean is initialized, a `BeanPostProcessor` checks whether any advisor's pointcut matches its methods. If so, it returns a **proxy** instead of the original object; callers get the proxy injected. A call goes proxy → chain of interceptors (advice) → real method, and back. Only calls that come through the proxy are advised. A JDK proxy can intercept only interface methods; a CGLIB proxy can intercept any method it can override — never `private`, `final` or `static` ones. Keep advised methods public: some features (caching, for example) honour only public methods, although `@Transactional` also works on protected and package-private methods with class-based proxies since Framework 6.0. → *More:* §4.4 How Spring AOP works: proxies · Ch 18 Q37

Q61. ⋆ JDK dynamic proxy vs CGLIB proxy? A JDK dynamic proxy implements the bean's **interfaces** — callers must use the interface type. CGLIB creates a **subclass** of the bean class at runtime, so it works without interfaces but cannot intercept `final` classes/methods or `private` methods. Plain Spring Framework uses a JDK proxy when the bean implements an interface; Spring Boot defaults to CGLIB for everything (`spring.aop.proxy-target-class=true`), which avoids "bean is not of required type" errors when injecting by class. → *More:* §4.4 How Spring AOP works: proxies · Ch 18 Q38

Q62. ⋆ What is the self-invocation problem? How do you solve it? When a method calls another method of the *same* bean (`this.save()`), the call doesn't pass through the proxy, so the second method's `@Transactional`, `@Async`, `@Cacheable` or `@Retryable` is ignored. Fixes, best first: move the method into another bean; inject the bean's own proxy (`@Lazy` self-injection or `ObjectProvider<Self>`) and call through it; use `TransactionTemplate` programmatically; or switch to AspectJ weaving. → *More:* §4.5 Proxy limitations — proved · Ch 18 Q39 · §18.15 Proofs: five tricky answers, run

Q63. Which Spring features are built on proxies? `@Transactional`, `@Cacheable`/`@CacheEvict`, `@Async`, `@PreAuthorize`/`@PostAuthorize`, `@Validated` method validation, Framework 7 `@Retryable` and `@ConcurrencyLimit`, `@Observed`, scoped beans (request/session proxies), `@Lazy` injection points, full `@Configuration` classes (CGLIB) and Spring Data repositories (interfaces implemented at runtime). They all share the same limits: self-invocation, `final`/`private` methods, and objects created with `new`. → *More:* §4.6 Spring features built on proxies · Ch 18 Q40

Q64. Write an aspect that logs the execution time of methods annotated with a custom annotation.

```
@Retention(RUNTIME) @Target(METHOD) public @interface Timed {}

@Aspect @Component
class TimingAspect {
    @Around("@annotation(com.acme.Timed)")
    Object time(ProceedingJoinPoint pjp) throws Throwable {
        long start = System.nanoTime();
        try { return pjp.proceed(); }
        finally { log.info("{} took {} ms", pjp.getSignature(), (System.nanoTime() - start) / 1_000_000); }
    }
}
```

`@Around` must call `proceed()` and **return its result**, otherwise the caller gets `null`. Needs `spring-boot-starter-aspectj` in Boot 4. → *More:* §4.2 Writing aspects with @Aspect · §4.3 Pointcut expressions

Q65. `@Around` vs the other advice types? How do you order two aspects? `@Around` wraps the call: it can change arguments, skip the call, retry it, change the return value or translate exceptions. The others are narrower and safer: `@Before`, `@AfterReturning` (sees the result), `@AfterThrowing` (sees the exception), `@After` (finally). Reach for `@Around` only when a narrower type can't do the job: with the others you can't forget to call `proceed()`. Order aspects with `@Order` — lower value = higher priority = outermost wrapper. → *More:* §4.2 Writing aspects with @Aspect · Ch 18 Q41 · Ch 18 Q42

Q66. Spring AOP vs AspectJ? Spring AOP: proxy-based, runtime, method calls on Spring beans only, zero build changes. AspectJ: a full AOP language that weaves bytecode at compile time or load time, so it can advise field access, constructors, private methods, self-invocation and non-bean objects. Spring AOP borrows AspectJ's annotations and pointcut syntax. Use AspectJ only when proxy limits really hurt. → *More:* §4.8 Spring AOP vs AspectJ · Ch 18 Q43

Q67. (Framework 7) What are `@Retryable` and `@ConcurrencyLimit` in Spring Framework 7? Framework 7 moved basic resilience into the core: `@Retryable` retries a method on failure with configurable `maxRetries` (default 3, so up to 4 calls in total), `delay`, `multiplier`, `maxDelay` and `jitter` (also for reactive return types); `@ConcurrencyLimit` caps how many threads may run a method at once (a simple bulkhead). Enable them with `@EnableResilientMethods`. They replace most uses of the separate Spring Retry project; for circuit breakers you still need Resilience4j. → *More:* §4.7 Framework 7 core resilience: @Retryable and @ConcurrencyLimit · Ch 18 Q44

Spring MVC, REST and HTTP clients

Q68. ⋆ What is the `DispatcherServlet`? Walk through how a request is processed in Spring MVC. The `DispatcherServlet` is the **front controller**: one servlet that receives every request and delegates. Flow: servlet filters (including Spring Security) → `DispatcherServlet` → `HandlerMapping` finds the controller method for the URL and HTTP method → interceptors' `preHandle` →

`HandlerAdapter` resolves arguments (`@PathVariable`, `@RequestBody` via an `HttpMessageConverter` such as Jackson) and calls the method → the return value is written as JSON by a message converter (REST) or resolved to a view by a `ViewResolver` (server-side HTML) → `postHandle`/`afterCompletion`. Exceptions go to `HandlerExceptionResolver`s, which call your `@ExceptionHandler` methods. → *More:* §5.2 How a request flows: the DispatcherServlet · Ch 18 Q45

Q69. ⋆ **`@Controller` vs `@RestController`?** `@RestController` = `@Controller` + `@ResponseBody` on every method: return values are serialized into the response body (JSON by default). A plain `@Controller` returns view names that a template engine (Thymeleaf) renders into HTML, unless a method has `@ResponseBody` or returns `ResponseEntity`. → *More:* §5.3 Controllers and request mappings · Ch 18 Q46

Q70. `@RequestMapping` vs `@GetMapping` (and `@PostMapping`, `@PutMapping`, `@DeleteMapping`, `@PatchMapping`)? `@RequestMapping` is the general annotation: path, HTTP method, `consumes`, `produces`, headers, params. On a class it sets a common path prefix. `@GetMapping("/x")` is shorthand for `@RequestMapping(path = "/x", method = GET)`; the others likewise. Use the shortcuts on methods for readability. → *More:* §5.3 Controllers and request mappings

Q71. ⋆ **`@PathVariable` vs `@RequestParam` vs `@RequestBody` (vs `@RequestHeader`, `@ModelAttribute`)?** `@PathVariable` reads a segment of the URL path (`/orders/{id}`) — identifies a resource. `@RequestParam` reads a query-string or form parameter (`?page=2`) — filters, options; can be optional with a default. `@RequestBody` deserializes the whole request body (JSON) into an object via a message converter. `@RequestHeader` reads a header. `@ModelAttribute` binds several query/form parameters onto an object's fields (used for search forms or HTML forms), unlike `@RequestBody`, which reads JSON. → *More:* §5.4 Reading input and writing output · Ch 18 Q47

Q72. Why return `ResponseEntity`? How do you set status codes and headers? `ResponseEntity<T>` lets you control the status code, headers and body together: `ResponseEntity.created(location).body(dto)` for 201 with a `Location` header, `ResponseEntity.noContent().build()` for 204, `ResponseEntity.notFound().build()`. For a fixed status you can also use `@ResponseStatus(HttpStatus.CREATED)` on the method. Throw `ResponseStatusException` (or let an exception handler map exceptions) for error codes. → *More:* §5.4 Reading input and writing output · Ch 18 Q48

Q73. ⋆ **What makes an API RESTful? Explain HTTP methods, status codes and idempotency.** REST models the API as **resources** identified by URLs, manipulated with standard HTTP methods, with stateless requests and representations such as JSON. GET reads (safe — no side effects), POST creates or triggers an action, PUT replaces a resource, PATCH partially updates, DELETE removes. **Idempotent** means repeating the same request leaves the same state: GET, PUT and DELETE are idempotent; POST is not; PATCH may or may not be. Key status codes: 200 OK, 201 Created, 204 No Content, 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 409 Conflict, 422 Unprocessable Content, 429 Too Many Requests, 500, 503. → *More:* §5.1 REST in one page · Ch 18 Q56

Q74. How do you design a REST endpoint for creating an order? Show a controller. Use a request DTO validated with `@Valid`, delegate to a service, return 201 with the new resource's URL. Never expose JPA entities directly.

```
@RestController @RequestMapping("/api/orders")
class OrderController {
    private final OrderService service;
    OrderController(OrderService service) { this.service = service; }

    @PostMapping
    ResponseEntity<OrderDto> create(@Valid @RequestBody CreateOrder req) {
        OrderDto saved = service.create(req);
        return ResponseEntity.created(URI.create("/api/orders/" + saved.id())).body(saved);
    }
}
```

→ *More:* §5.3 Controllers and request mappings · §5.13 Putting it together: design checklist and mistakes

Q75. Why should you not expose JPA entities directly in REST responses? Entities mirror the database, not the API contract: renaming a column breaks clients; lazy associations cause `LazyInitializationException` or extra queries during JSON serialization; bidirectional relationships cause infinite recursion; and you may leak fields like password hashes or allow **mass assignment** (clients setting fields they shouldn't, such as `role`). Map to DTOs — Java records are ideal — by hand or with a mapper such as MapStruct.

Follow-up — How do you handle a bidirectional relationship that causes infinite recursion in JSON? Best: return DTOs that include only one direction. Otherwise use Jackson's `@JsonManagedReference` on the parent side and `@JsonBackReference` on the child, or `@JsonIgnore` on one side, or `@JsonIdentityInfo` to write references by id. → *More:* §5.13 Putting it together: design checklist and mistakes · §8.12 Projections · §5.5 JSON with Jackson 3

Q76. How does Spring convert Java objects to JSON? How do you customize it? (Jackson 3 in Boot 4) An `HttpMessageConverter` backed by Jackson serializes return values and deserializes `@RequestBody`. Boot auto-configures the mapper; customize with `spring.jackson.*` properties, a `JsonMapperBuilderCustomizer` bean, or annotations such as `@JsonProperty`, `@JsonIgnore`, `@JsonFormat`. Boot 4 uses **Jackson 3**: packages are `tools.jackson.*` (annotations stay in `com.fasterxml.jackson.annotation`), the main type is `JsonMapper`, and dates are written as ISO strings by default. Jackson 2 support remains, deprecated, for migration. → *More:* §5.5 JSON with Jackson 3 · Ch 18 Q51

Q77. What is content negotiation? The server chooses the response format from the client's `Accept` header (for example `application/json` vs `application/xml`), and the request format from `Content-Type`. Spring picks the first `HttpMessageConverter` that can write the return type in an acceptable media type; `produces`/`consumes` on a mapping narrow it. If nothing matches, the response is 406 Not Acceptable (or 415 Unsupported Media Type for request bodies). → *More:* §5.8 Content negotiation · Ch 18 Q52

Q78. How do you version a REST API? (Framework 7) Common strategies: URL path (`/v1/orders`), a header (`X-API-Version` or `API-Version`), a query parameter, or the media type (`Accept: application/vnd.acme.v2+json`). Framework 7 adds **first-class API versioning**: configure the strategy once (`spring.mvc.apiversion.use.header=API-Version`, or `ApiVersionConfigurer`), then write

`@GetMapping(path = "/orders", version = "2")`; Spring resolves the version, validates it, can mark versions deprecated, and clients (`RestClient`, HTTP interfaces) can send versions too. → *More:* §5.9 API versioning (Framework 7) · Ch 18 Q53

Q79. How do you implement pagination and sorting in a REST API? Accept a `Pageable` parameter in the controller (`?page=0&size=20&sort=createdAt,desc`) and pass it to a Spring Data repository method returning `Page<T>` or `Slice<T>`. Return a stable JSON shape — your own DTO, or Spring Data's `PagedModel` (content plus a `page` block with size, number, totalElements, totalPages) — rather than serializing the `PageImpl` class directly, whose JSON is not a stable contract (Spring Data warns about it). Cap the page size (`spring.data.web.pageable.max-page-size`). For deep pages use keyset (cursor) pagination. → *More:* §8.11 Pagination and sorting · §5.12 Documenting the API: OpenAPI with springdoc · Ch 18 Q57

Q80. ⋆ What is CORS? How do you configure it in Spring Boot? CORS (Cross-Origin Resource Sharing) is a browser rule: JavaScript on `https://app.example.com` may call `https://api.example.com` only if the API answers with `Access-Control-Allow-Origin` headers allowing it; for non-simple requests the browser first sends an OPTIONS "preflight". Configure globally with `WebMvcConfigurer.addCorsMappings`, per controller with `@CrossOrigin`, and — when Spring Security is used — enable `http.cors(...)` with a `CorsConfigurationSource` so preflights are not rejected before reaching MVC. Never combine `allowedOrigins("*")` with credentials. → *More:* §5.10 CORS and file upload · §10.9 CORS · Ch 18 Q54

Q81. ⋆ Filter vs interceptor vs AOP aspect — when to use which? A **servlet filter** runs before the `DispatcherServlet`, for every request, and sees raw request/response — use for security, CORS, request logging, compression, correlation ids. A `HandlerInterceptor` runs inside Spring MVC around controller methods and knows which handler was chosen — use for per-endpoint checks, locale, audit. An **aspect** wraps any bean method (not only web) — use for service-level concerns like timing or retries. → *More:* §5.11 Filters vs interceptors (vs AOP) · Ch 18 Q55

Q82. How do you upload and download files in Spring MVC? Upload: accept `@RequestParam("file") MultipartFile file` in a `multipart/form-data` endpoint; set limits with `spring.servlet.multipart.max-file-size` and `max-request-size`; validate content type and size; stream large files to storage instead of reading them into memory. Download: return `ResponseEntity<Resource>` with `Content-Disposition: attachment; filename=...` and the right content type, or `StreamingResponseBody` for big content. → *More:* §5.10 CORS and file upload

Q83. How do you document a REST API (Swagger/OpenAPI)? Add springdoc-openapi (version 3.x for Boot 4; not managed by the Boot BOM, so set its version). It inspects your controllers and generates an OpenAPI 3 document at `/v3/api-docs` and a Swagger UI at `/swagger-ui.html`. Refine with annotations such as `@Operation` and `@Schema`. Disable or secure the UI in production. → *More:* §5.12 Documenting the API: OpenAPI with springdoc

Q84. What is HATEOAS? Is it used in practice? HATEOAS (Hypermedia as the Engine of Application State) means responses include links telling the client what it can do next (`"_links": {"cancel": {...}}`). Spring HATEOAS provides `EntityModel`, `CollectionModel` and link builders, and Spring Data REST uses it. In practice most internal JSON APIs skip it; it is worth knowing as the "fully RESTful" level of the Richardson maturity model. → *More:* §9.3 Spring Data REST

Q85. How do you return streaming or real-time data from Spring MVC (SSE, WebSocket)? For server-to-client streams, return an `SseEmitter` (Server-Sent Events — a long HTTP response carrying a stream of events) from MVC, or a `Flux<ServerSentEvent<T>>` from WebFlux. For two-way messaging, use WebSocket (`spring-boot-starter-websocket`), often with STOMP and a message broker. For a large download, `StreamingResponseBody` writes on a separate thread. → *More:* §13.12 Full-stack WebFlux tests and Server-Sent Events

Q86. ⋆ `RestTemplate` vs `WebClient` vs `RestClient` — which should you use? `RestClient` (Framework 6.1+) is the modern **synchronous** client with a fluent API; use it in Spring MVC apps — with virtual threads, blocking calls are cheap. `WebClient` is the **non-blocking** reactive client; use it in WebFlux apps or when you need streaming or many concurrent calls composed reactively. `RestTemplate` is the old template-style synchronous client; it is in maintenance, with formal deprecation planned for Framework 7.1 and removal in 8.0. For declarative clients, define an **HTTP service interface** (`@GetExchange`) backed by `RestClient`. → *More:* §6.1 Choosing a client · §6.9 RestTemplate and WebClient in brief · Ch 18 Q58

Q87. What are HTTP service interfaces (`@HttpExchange`)? How do they compare to Feign? You write a Java interface with `@GetExchange("/users/{id}") User get(@PathVariable long id);` and Spring generates the implementation on top of `RestClient` or `WebClient`. Boot 4 auto-configures them through `@ImportHttpServices` and `spring.http.serviceclient.<group>.*` properties (base URL, timeouts). They replace most uses of OpenFeign, which is now in maintenance mode. → *More:* §6.6 HTTP service interfaces (@HttpExchange) · §16.6 Declarative clients: OpenFeign vs HTTP interface clients · Ch 18 Q59 · Ch 18 Q160

Q88. How do you set timeouts and handle errors on an HTTP client? Always set a connect timeout and a read timeout (Boot 4: `spring.http.clients.connect-timeout`, `spring.http.clients.read-timeout`, or on the request factory); a missing timeout lets a slow dependency exhaust your threads. Handle error statuses with `.onStatus(HttpStatusCode::is4xxClientError, ...)` on `RestClient`, which otherwise throws `RestClientResponseException` subclasses. Retry only idempotent calls, with backoff, and wrap critical dependencies in a circuit breaker. → *More:* §6.3 Error handling · §6.4 Timeouts and the underlying HTTP library · Ch 18 Q60

Q89. How do you add a header (for example a correlation id or token) to every outgoing request? Register a `ClientHttpRequestInterceptor` on the `RestClient.Builder` (or an `ExchangeFilterFunction` on `WebClient`). Boot's auto-configured builders already propagate tracing headers; for OAuth2 tokens use Spring Security's `OAuth2ClientHttpRequestInterceptor`. → *More:* §6.5 Interceptors

---

Validation and exception handling

Q90. ⋆ How do you validate request data in Spring Boot? Add `spring-boot-starter-validation` (Hibernate Validator, the Bean Validation implementation). Put constraints on the DTO — `@NotBlank`, `@Size`, `@Email`, `@Positive`, `@Past`, `@Pattern` — and `@Valid` on the `@RequestBody` parameter. Invalid input throws `MethodArgumentNotValidException`, which Spring turns into 400 Bad Request. For path/query parameters, put constraints directly on the parameters; Framework 6.1+ validates them natively and throws `HandlerMethodValidationException`. → *More:* §5.6 Validation · Ch 18 Q49

Q91. `@Valid` **vs** `@Validated`**?** `@Valid` (Jakarta) triggers validation of an argument or a nested object (cascading). `@Validated` (Spring) additionally supports **validation groups** (validating a different set of constraints on create vs update) and, on a class, enables **method validation** through a proxy, so service methods can validate their parameters and return values. Use `@Valid` on nested fields to cascade into them. → *More:* §5.6 Validation

Q92. How do you write a custom validation annotation? Create an annotation marked `@Constraint(validatedBy = ...)` with `message`, `groups` and `payload` attributes, and a `ConstraintValidator` implementation with an `isValid` method. Validators are created by Spring, so they can inject beans (for example to check uniqueness — but beware that this hits the database on every validation).

```
@Constraint(validatedBy = SkuValidator.class)
@Target(FIELD) @Retention(RUNTIME)
public @interface ValidSku { String message() default "invalid SKU";
    Class<?>[] groups() default {}; Class<? extends Payload>[] payload() default {}; }

class SkuValidator implements ConstraintValidator<ValidSku, String> {
    public boolean isValid(String v, ConstraintValidatorContext c) { return v == null || v.matches("[A-Z]{3}-\\d{4}"); }
}
```

→ *More:* §5.6 Validation

Q93. How do you validate across several fields (for example, start date before end date)? Use a **class-level** constraint: annotate the DTO (not a field) with a custom constraint whose validator receives the whole object and compares the fields. Report the error on a specific field with `context.buildConstraintViolationWithTemplate(...).addPropertyNode("endDate")`. → *More:* §5.6 Validation

Q94. ⋆ How do you handle exceptions globally in Spring Boot? (`@ControllerAdvice` / `@RestControllerAdvice`) Create a class annotated with `@RestControllerAdvice` with `@ExceptionHandler` methods; each maps an exception type to a response for all controllers. Return RFC 9457 **Problem Details** (`ProblemDetail`: type, title, status, detail, instance, plus custom properties) for a consistent error format. Extending `ResponseEntityExceptionHandler` gives ready handlers for Spring MVC's own exceptions.

```
@RestControllerAdvice
class ApiErrors {
    @ExceptionHandler(OrderNotFoundException.class)
    ProblemDetail notFound(OrderNotFoundException ex) {
        ProblemDetail pd = ProblemDetail.forStatusAndDetail(HttpStatus.NOT_FOUND, ex.getMessage());
        pd.setTitle("Order not found");
        return pd;
    }
}
```

Follow-up — *`@ControllerAdvice` vs `@RestControllerAdvice`?* Same relationship as `@Controller` vs `@RestController`: `@RestControllerAdvice` = `@ControllerAdvice` + `@ResponseBody`, so handler return values are written as the response body (JSON). With plain `@ControllerAdvice`, a handler returns a view name unless it returns `ResponseEntity` / `ProblemDetail` or carries `@ResponseBody`. Both can be limited with `basePackages` or `assignableTypes`. → *More:* §5.7 Error handling and Problem Details (RFC 9457) · Ch 18 Q50

Q95. What are the ways to handle exceptions in Spring MVC, and which wins?

1. `@ExceptionHandler` inside the controller (local, wins first); 2) `@ExceptionHandler` in a `@ControllerAdvice` (global; order several advices with `@Order`); 3) `@ResponseStatus` on the exception class; 4) throwing `ResponseStatusException` or an `ErrorResponseException`; 5) a custom `HandlerExceptionResolver`; 6) Boot's fallback `/error` endpoint (`BasicErrorController`). Within one advice, the most specific exception type wins. Exceptions thrown in **filters** (including Spring Security) never reach `@ControllerAdvice`. → *More:* §5.7 Error handling and Problem Details (RFC 9457)

Q96. What does Spring Boot return for an unhandled error? How do you customize the default error response? The `/error` endpoint (`BasicErrorController`) returns JSON with `timestamp`, `status`, `error` and `path` — or a "Whitelabel Error Page" for browsers. With `spring.mvc.problemdetails.enabled=true` Spring MVC's own exceptions become `application/problem+json`. Customize with your own `@ControllerAdvice`, a custom `ErrorAttributes` bean, or `server.error.include-message` / `include-stacktrace` (keep stack traces off in production). → *More:* §5.7 Error handling and Problem Details (RFC 9457)

Q97. Should you use checked or unchecked exceptions in a Spring service? Mostly unchecked (`RuntimeException` subclasses such as `OrderNotFoundException`): they don't pollute signatures, and by default they trigger transaction rollback. A checked exception thrown from a `@Transactional` method **commits** unless you add `rollbackFor`. Spring itself translates checked JDBC `SQLException`s into unchecked `DataAccessException`s for the same reason. → *More:* §7.10 Rollback rules · §7.5 Exception translation: DataAccessException

---

Spring Data JPA and Hibernate

Q98. ⋆ JPA vs Hibernate vs Spring Data JPA? JPA (Jakarta Persistence) is a specification — annotations (`@Entity`, `@Id`) and interfaces (`EntityManager`) for object-relational mapping (ORM: mapping Java objects to database tables). **Hibernate** is the most common implementation of JPA (Boot 4.1 uses Hibernate ORM 7.4). **Spring Data JPA** sits on top: it generates repository implementations from interfaces, derives queries from method names, adds paging, auditing and Spring transaction integration. → *More:* §8.1 JPA, Hibernate and Spring Data — who does what · Ch 18 Q76

Q99. ⋆ `CrudRepository` **vs** `PagingAndSortingRepository` **vs** `JpaRepository`**?** `CrudRepository` gives basic CRUD (create, read, update, delete): `save`, `findById`, `findAll`, `deleteById`, `count`. `ListCrudRepository` returns `List` instead of `Iterable`.

`PagingAndSortingRepository` adds `findAll(Pageable)` and `findAll(Sort)`. `JpaRepository` extends both list variants and adds JPA-specific methods: `flush`, `saveAndFlush`, `saveAllAndFlush`, `deleteAllInBatch`, `getReferenceById`. Use `JpaRepository` for JPA apps, or `Repository` plus only the methods you want to expose. → *More:* §8.8 Repositories

Q100. How does Spring Data create a repository implementation from an interface? At startup Spring Data scans for interfaces extending `Repository`, and for each it creates a **proxy** backed by `SimpleJpaRepository` (the generic CRUD implementation). For each custom method it parses the name into a query (derived query), or uses `@Query`, at startup — so a typo in a method name fails fast. Calls go through the proxy, which also applies exception translation and transactions. The inherited CRUD methods are transactional (read methods read-only), but query methods you declare yourself get no transaction settings by default. Annotate them, or the repository interface, with `@Transactional(readOnly = true)`. → *More:* §8.8 Repositories · §8.9 Derived queries

Q101. Derived query vs `@Query` (JPQL) vs native query? Derived queries come from the method name (`findByStatusAndCreatedAtAfter`) — great for simple filters, unreadable for complex ones. **`@Query`** holds JPQL (queries written against entities and fields, portable across databases) — use for joins, fetch joins, projections. **Native queries** (`nativeQuery = true`) are plain SQL — use for database-specific features (window functions, `SKIP LOCKED`, JSON operators) at the cost of portability. Updates need `@Modifying` plus a transaction. → *More:* §8.9 Derived queries · §8.10 @Query: JPQL, native SQL and @Modifying · Ch 18 Q84

Q102. ⋆ What is the N+1 select problem? How do you detect and fix it? Loading N parent rows with one query, then touching a lazy association on each one, fires N extra queries — 1 + N in total. Detect it with SQL logging, Hibernate statistics (`hibernate.generate_statistics`), or a test asserting the query count. Fixes: `JOIN FETCH` in a JPQL query, `@EntityGraph(attributePaths = "items")` on the repository method, batch fetching (`hibernate.default_batch_fetch_size=50` loads lazy collections in groups via `IN` lists), or a DTO projection that selects exactly the needed columns. Beware: fetch-joining a collection together with pagination makes Hibernate paginate in memory.

```
@Query("select o from Order o join fetch o.items where o.status = :status")
List<Order> findWithItems(@Param("status") Status status);
```

→ *More:* §8.7 The N+1 problem · Ch 18 Q83 · §18.17 Debugging scenarios

Q103. ⋆ Lazy vs eager loading? What are the default fetch types? Eager loads the association immediately with the owner; **lazy** loads it on first access through a proxy. Defaults: `@ManyToOne` and `@OneToOne` are EAGER, `@OneToMany` and `@ManyToMany` are LAZY. Best practice: make everything lazy (`fetch = FetchType.LAZY` on `@ManyToOne`) and fetch what each use case needs explicitly with fetch joins or entity graphs. Eager defaults cause hidden extra queries and cannot be turned off per query. → *More:* §8.6 Fetch types and LazyInitializationException · Ch 18 Q82

Q104. ⋆ What causes `LazyInitializationException`, and how do you fix it? A lazy association is accessed after the persistence context (Hibernate `Session`) that loaded the entity has closed — typically in a controller or during JSON serialization, after the `@Transactional` service method returned. Fix it by fetching what you need inside the transaction (fetch join, `@EntityGraph`) and mapping to a DTO before returning. Don't "fix" it with `FetchType.EAGER`, Open Session in View or `hibernate.enable_lazy_load_no_trans` — they hide N+1 problems. → *More:* §8.6 Fetch types and LazyInitializationException · Ch 18 Q82 · §18.17 Debugging scenarios

Q105. ⋆ What is the persistence context? What is dirty checking? The persistence context is the set of managed entity instances an `EntityManager` tracks — at most one Java object per database row (it acts as the **first-level cache**). At flush time Hibernate compares each managed entity with the snapshot taken when it was loaded and issues `UPDATE`s for changed ones: that is **dirty checking**. So inside a transaction you don't need to call `save()` after changing a loaded entity — the change is written automatically at commit. → *More:* §8.4 The persistence context and dirty checking · Ch 18 Q77

Q106. What are the JPA entity states (lifecycle)? Transient/new — created with `new`, not tracked. **Managed/persistent** — associated with a persistence context; changes are tracked. **Detached** — was managed, but the context closed (or `detach` / `clear` was called); changes are not tracked. **Removed** — scheduled for deletion at flush. `persist` makes new → managed; `merge` copies a detached object's state onto a managed instance and returns that instance. → *More:* §8.4 The persistence context and dirty checking · Ch 18 Q78

Q107. ⋆ What is the first-level cache vs the second-level cache in Hibernate? The **first-level cache** is the persistence context: per `EntityManager`/transaction, always on, guarantees one object per row and repeatable reads within the transaction. The **second-level cache** is optional and shared across sessions in the whole application (or cluster): entities marked `@Cache` / `@Cacheable` are stored by id in a provider such as Ehcache, Infinispan or Hazelcast via JCache. It helps for read-mostly reference data; it does not cache query results unless you also enable the query cache. Stale data across instances is the main risk.

Q108. `save()` vs `saveAndFlush()` vs `persist()` vs `merge()`? `save()` (Spring Data) calls `persist` if the entity is new and `merge` otherwise; the SQL is sent at flush time (usually commit). `saveAndFlush()` also flushes immediately — useful to surface constraint violations at that line, or before a native query. `persist` (JPA) attaches a new object and makes *that instance* managed; `merge` returns a *different*, managed instance, so keep using the returned object. Calling `save()` on an already managed entity is unnecessary.

Follow-up — How does `save()` decide between INSERT and UPDATE? `isNew()` decides. If the entity has a `@Version` field of a wrapper type, it is new when the version is `null`. Otherwise it is new when its `@Id` is `null` (or `0` for primitive ids). With an assigned id (e.g. a UUID you set yourself), `save()` thinks the entity exists, calls `merge`, and Hibernate runs a `SELECT` first. Fix with a `@Version` field or by implementing `Persistable<ID>`. → *More:* §8.4 The persistence context and dirty checking · §8.8 Repositories · Ch 18 Q79

Q109. `findById()` vs `getReferenceById()` (Hibernate `get()` vs `load()`)? `findById()` (JPA `find`) hits the database (or the persistence context) immediately and returns `Optional.empty()` if the row is missing. `getReferenceById()` (JPA `getReference`, Hibernate's old `load()`) returns an uninitialized **proxy** without a query; the row is read only when you touch a non-id field,

and a missing row then throws `EntityNotFoundException`. Use the reference to set a foreign key (`order.setCustomer(repo.getReferenceById(id))`) without loading the customer. In Hibernate 7, `Session.load()` was removed in favour of `getReference()` (same meaning), and `Session.get()` is deprecated in favour of the JPA-standard `find()`. → *More:* §8.8 Repositories

Q110. What are the id generation strategies? Which is best for performance? `IDENTITY` (database auto-increment column): simple, but Hibernate must insert immediately to learn the id, which **disables JDBC batch inserts**. `SEQUENCE` (database sequence) with a `allocationSize` such as 50: Hibernate fetches ids in blocks, so batching works — best for PostgreSQL and Oracle. `TABLE`: portable but slow. `UUID` (`@GeneratedValue(strategy = GenerationType.UUID)` or `@UuidGenerator`): generated in the app, good for distributed systems; prefer time-ordered UUIDs (version 7) for index locality. → *More:* §8.3 Entities and ids

Q111. ⋆ Explain `@OneToMany`, `@ManyToOne`, `@ManyToMany` and the owning side (`mappedBy`). In a bidirectional relationship, the **owning side** is the one whose mapping controls the foreign key column — for one-to-many it is always the `@ManyToOne` side. The other side declares `mappedBy = "order"` and is ignored when writing; setting only the inverse side does not save the link. Keep both sides in sync with helper methods (`addItem` sets `item.setOrder(this)`). For `@ManyToMany`, prefer an explicit join entity when the link has its own data (quantity, date). → *More:* §8.5 Relationships · Ch 18 Q80

Q112. What does `cascade` do? `CascadeType.REMOVE` vs `orphanRemoval = true`? `cascade` propagates entity operations (`PERSIST`, `MERGE`, `REMOVE`, `ALL`) from parent to children, so saving an order also saves its items. `CascadeType.REMOVE` deletes children when the **parent** is deleted. `orphanRemoval = true` also deletes a child when it is **removed from the collection**. Use cascades only for true parent–child aggregates, never on `@ManyToOne` (deleting an order item would delete the product). → *More:* §8.5 Relationships · Ch 18 Q81

Q113. What are projections in Spring Data JPA? Projections return only some columns instead of whole entities: **interface projections** (an interface with getters matching properties; Spring builds a proxy), **class/record (DTO) projections** (constructor expression in JPQL: `select new com.acme.OrderSummary(o.id, o.total) from Order o`, or a record as the return type), and **dynamic projections** (`<T> List<T> findByStatus(Status s, Class<T> type)`). They are faster and avoid lazy-loading issues because nothing is managed. → *More:* §8.12 Projections · Ch 18 Q86

Q114. `Page` vs `Slice` vs `List` return types? `Page<T>` contains the content and runs an extra `COUNT` query for the total (needed for "page 3 of 17"). `Slice<T>` only knows whether a next page exists (it fetches size + 1 rows) — cheaper, good for infinite scroll. `List<T>` with a `Pageable` just returns the rows. For very large tables, `COUNT` itself can be the slow part. → *More:* §8.11 Pagination and sorting · Ch 18 Q85

Q115. How do you write dynamic queries with optional filters (search screens)? Use **Specifications** (`JpaSpecificationExecutor` + `Specification<T>` lambdas combined with `and`/`or`, built on the Criteria API; start from `Specification.unrestricted()` — Spring Data JPA 4 (release train 2025.1) no longer accepts `null` specifications, so the old `Specification.where(null)` idiom is gone), **Query by Example** (`Example.of(probe)`), **Querydsl**, or for full control `JdbcClient` with a built SQL string and bound parameters. Never concatenate user input into JPQL/SQL — that opens SQL injection. → *More:* §8.10 @Query: JPQL, native SQL and @Modifying

Q116. ⋆ How does optimistic locking work? Optimistic vs pessimistic locking? Optimistic: add a `@Version` column; every update runs `UPDATE ... SET version = version + 1 WHERE id = ? AND version = ?`. If another transaction changed the row first, zero rows match and Spring throws `ObjectOptimisticLockingFailureException` — you retry or return 409 Conflict. No locks are held, so it suits low-contention data and web "edit forms". **Pessimistic**: `@Lock(LockModeType.PESSIMISTIC_WRITE)` issues `SELECT ... FOR UPDATE`, blocking other writers until commit — use for hot rows (inventory counters) where conflicts are frequent; keep transactions short to avoid deadlocks. → *More:* §8.14 Locking: optimistic and pessimistic · Ch 18 Q87

Q117. What is Open Session in View (OSIV), and should you disable it? OSIV keeps the persistence context (and a database connection) open for the whole web request, so lazy loading works in controllers and during JSON rendering. Boot enables it by default and logs a warning (`spring.jpa.open-in-view`). Disable it in REST services: it holds connections during slow rendering, hides N+1 queries, and makes data access happen outside transactions. Fetch what you need in the service instead. → *More:* §8.15 Open Session in View · Ch 18 Q88

Q118. Why is `spring.jpa.hibernate.ddl-auto=update` not recommended in production? How do you manage the schema? `update` only adds things, never removes or renames safely, can't migrate data, isn't reviewed or versioned, and behaves differently across environments. Use a migration tool — **Flyway** (versioned SQL files `V1__init.sql`, applied once, recorded in a history table) or **Liquibase** — and set `ddl-auto=validate` (or `none`) so Hibernate only checks that the mappings match. Boot's default is `create-drop` for an embedded database when no Flyway/Liquibase is present, and `none` otherwise. → *More:* §8.16 Schema migrations with Flyway · §9.4 Schema migrations: Flyway beyond the basics, and Flyway vs Liquibase · Ch 18 Q89

Q119. How should a JPA entity implement `equals()` and `hashCode()`? Don't use Lombok's `@Data`/`@EqualsAndHashCode` over all fields (it touches lazy associations and changes as fields change). Use a natural business key if one exists; otherwise compare by id only when it is non-null, and return a constant `hashCode` (for example `getClass().hashCode()`) so the hash doesn't change when the id is assigned on persist. This keeps entities stable inside `HashSet`s across state changes. → *More:* §8.3 Entities and ids · Ch 18 Q90

Q120. How do you implement auditing (created/modified by and at) and soft delete? Auditing: `@EnableJpaAuditing`, `@EntityListeners(AuditingEntityListener.class)` and fields annotated `@CreatedDate`, `@LastModifiedDate`, `@CreatedBy`, `@LastModifiedBy`; an `AuditorAware` bean supplies the current user (often from the `SecurityContext`). For full history use Hibernate Envers. Soft delete: mark rows deleted instead of deleting them — Hibernate's `@SoftDelete` (6.4+) rewrites deletes into updates and filters deleted rows from queries; the older approach is `@SQLDelete` plus `@SQLRestriction`. Remember unique constraints and indexes must account for deleted rows. → *More:* §8.13 Auditing

Q121. How do you insert or update thousands of rows efficiently with JPA? Enable JDBC batching (`spring.jpa.properties.hibernate.jdbc.batch_size=50`, plus `order_inserts`/`order_updates`), use `SEQUENCE` ids (IDENTITY disables insert batching), and in a loop `flush()` and `clear()` the `EntityManager` every batch so the persistence context doesn't grow without limit. For bulk updates use a single JPQL/SQL `UPDATE` with `@Modifying`. For really large loads, skip JPA:

`JdbcClient` / `JdbcTemplate.batchUpdate`, PostgreSQL `COPY`, or Spring Batch. → *More:* §8.18 Performance checklist · §7.3 JdbcClient and JdbcTemplate

Q122. How do you stream or process a very large result set without running out of memory? Return a `Stream<T>` from a repository method (inside a read-only transaction, closed with try-with-resources) with a JDBC fetch size hint, or page through with keyset pagination (`where id > :lastId order by id limit 1000`), detaching/clearing as you go. With plain JDBC, `JdbcClient...query().stream()` or a `RowCallbackHandler` processes row by row. Avoid `findAll()` on large tables. → *More:* §8.18 Performance checklist · §7.3 JdbcClient and JdbcTemplate

Q123. How do you configure multiple data sources in Spring Boot? Define each `DataSource` yourself — bind each from its own prefix with `@ConfigurationProperties("app.datasource.orders")` and `DataSourceBuilder`, mark one `@Primary` — because Boot's auto-configuration backs off once you define one. For JPA, create a separate `LocalContainerEntityManagerFactoryBean` and `JpaTransactionManager` per source, and point each `@EnableJpaRepositories(basePackages = ..., entityManagerFactoryRef = ..., transactionManagerRef = ...)` at its packages. A transaction spans one data source only; across two you need a different design (outbox, saga), not XA, in most modern systems. For read replicas, an `AbstractRoutingDataSource` can route read-only transactions. → *More:* §7.2 DataSource and HikariCP

Q124. What is HikariCP, and how do you size the connection pool? HikariCP is Boot's default JDBC connection pool: it keeps database connections open and lends them to threads, because opening a connection per query is slow. Default `maximum-pool-size` is 10. Bigger isn't better — the database has limited cores; a starting formula is about (CPU cores × 2) + effective disk spindles, then measure. Set `connection-timeout` so callers fail fast, and remember total connections = pool size × instances. With virtual threads, the pool becomes the real concurrency limit. → *More:* §7.2 DataSource and HikariCP · Ch 18 Q64

Q125. `JdbcClient` vs `JdbcTemplate` vs JPA — when would you use plain JDBC? `JdbcTemplate` is the classic template for SQL with callbacks; `JdbcClient` (Framework 6.1+) is a fluent wrapper over it with named parameters and record mapping. Choose JDBC when you want full control over SQL, reporting queries, bulk operations, or a simple model without lazy loading surprises; choose JPA when you have a rich domain model with relationships and want dirty checking and caching. Spring Data JDBC is a middle way: repositories without lazy loading or a persistence context. → *More:* §7.3 JdbcClient and JdbcTemplate · §9.2 Spring Data JDBC and aggregates · Ch 18 Q63 · Ch 18 Q91

Transactions

Q126. ⋆ What is a transaction? Explain ACID. A transaction groups database operations into one unit that either fully happens (commit) or not at all (rollback). **Atomicity**: all or nothing. **Consistency**: constraints hold before and after. **Isolation**: concurrent transactions don't see each other's partial work (to a configurable degree). **Durability**: once committed, data survives crashes. → *More:* §7.7 How a Spring transaction works

Q127. ⋆ How does `@Transactional` work internally? Spring wraps the bean in a proxy. On a call, the `TransactionInterceptor` reads the attributes (propagation, isolation, timeout, read-only, rollback rules), asks the `PlatformTransactionManager` (e.g. `JpaTransactionManager`) to begin or join a transaction, binds the connection/`EntityManager` to the current **thread** (via `ThreadLocal`), calls your method, then commits, or rolls back if a matching exception escaped. Code on another thread does not see that transaction. → *More:* §7.7 How a Spring transaction works · Ch 18 Q65

Q128. ⋆ What are the propagation types? Explain `REQUIRED` vs `REQUIRES_NEW` vs `NESTED`. Propagation decides what happens when a transactional method is called while a transaction may already exist. **REQUIRED** (default): join the existing one or start one. **REQUIRES_NEW**: suspend the outer transaction and run a completely independent one that commits or rolls back on its own (needs a second connection). **NESTED**: run inside the outer transaction with a savepoint, so the inner part can roll back alone but commits only with the outer one (JDBC savepoints — reliably supported with JDBC-based transaction managers such as `DataSourceTransactionManager`, not a general JPA feature). Others: **SUPPORTS** (join if present, else run without), **MANDATORY** (must exist or exception), **NOT_SUPPORTED** (suspend and run without), **NEVER** (exception if one exists). → *More:* §7.9 Propagation · Ch 18 Q68

Q129. When would you use `REQUIRES_NEW`? What is the risk? For work that must survive the caller's rollback — an audit log entry or a "failed attempt" record. Risks: it takes a second connection while the first is held, so under load it can exhaust the pool and deadlock the app; and the inner commit is not undone if the outer transaction later rolls back. Prefer `@TransactionalEventListener` or an outbox when possible. → *More:* §7.9 Propagation

Q130. ⋆ What are the isolation levels? What problems does each prevent? Anomalies: **dirty read** (seeing uncommitted data), **non-repeatable read** (re-reading a row gives a different value), **phantom read** (re-running a query returns new rows), plus **lost update** (two writers overwrite each other). Levels: `READ_UNCOMMITTED` (allows all), `READ_COMMITTED` (no dirty reads; PostgreSQL/Oracle default), `REPEATABLE_READ` (also no non-repeatable reads; MySQL InnoDB default; on PostgreSQL it is snapshot isolation and also prevents phantoms), `SERIALIZABLE` (behaves as if transactions ran one by one; may abort with serialization failures you must retry). Spring's `Isolation.DEFAULT` uses the database's default. → *More:* §7.11 Isolation levels · Ch 18 Q66

Q131. ⋆ A `@Transactional` method throws a checked exception. Does it roll back? No — by default Spring rolls back only on **unchecked** exceptions (`RuntimeException`) and `Error`s; a checked exception **commits** the transaction. Change it with `@Transactional(rollbackFor = Exception.class)` (or `noRollbackFor` for the reverse). Also, if you catch the exception inside the method and don't rethrow, nothing rolls back. → *More:* §7.10 Rollback rules · Ch 18 Q67 · §18.15 Proofs: five tricky answers, run

Q132. ⋆ List the common `@Transactional` pitfalls — why might it not work?

1. **Self-invocation** — calling the method via `this` bypasses the proxy. 2) The method is **private, final or static**, or the class is final (CGLIB can't override); with a JDK interface proxy, the method must be public and declared on the interface. 3) The object isn't a Spring bean (created with `new`). 4) The exception is **checked**, or it's **caught and swallowed**. 5) Work runs

on **another thread** (`@Async`, `CompletableFuture`, parallel stream) — the transaction is thread-bound. 6) Wrong transaction manager with several data sources. 7) Calling it from `@PostConstruct` (proxy not ready). 8) MySQL MyISAM tables (no transactions). 9) `readOnly`/propagation settings that don't mean what you think. → *More:* §7.13 Transaction pitfalls · Ch 18 Q71 · §18.17 Debugging scenarios

Q133. What is `UnexpectedRollbackException` ("Transaction silently rolled back because it has been marked as rollback-only")? An inner `REQUIRED` method threw a runtime exception, so Spring marked the **shared** transaction rollback-only. The outer method caught the exception and tried to commit; Spring can't commit a rollback-only transaction, so it rolls back and throws `UnexpectedRollbackException`. Fix: don't catch-and-continue around a participating method — let it fail, or run the inner work in `REQUIRES_NEW` if it really is independent. → *More:* §7.9 Propagation · Ch 18 Q69

Q134. What does `@Transactional(readOnly = true)` do? It is a hint: Hibernate sets the session to read-only (no dirty-checking snapshots, flush mode MANUAL — less memory and CPU), the JDBC connection is marked read-only (some drivers/databases optimize, and routing data sources can send it to a replica). It does **not** guarantee writes are impossible on every database. Spring Data's inherited CRUD read methods (`findById`, `findAll`) are read-only by default; your own declared query methods are not. Put `@Transactional(readOnly = true)` on query service methods. → *More:* §7.12 readOnly and timeout · Ch 18 Q70

Q135. Where should `@Transactional` go — controller, service or repository? Class or method? On the **service** layer, around one business use case, so several repository calls commit together. Not on controllers (transactions would span HTTP concerns) and usually not added to repositories (they already have defaults). On a class it applies to all public methods (with class-based proxies since Framework 6.0, also protected and package-private ones); a method-level annotation overrides it. Annotate concrete classes rather than interfaces. → *More:* §7.8 @Transactional attributes · Ch 18 Q72

Q136. Can you call an external API or send a message inside a `@Transactional` method? Avoid it. A slow HTTP call holds a database connection and row locks for its whole duration, and the call cannot be rolled back if the transaction later fails — or the message is sent but the data never commits. Do the remote call before or after the transaction, or record the intent in an **outbox** table inside the transaction and publish after commit (`@TransactionalEventListener(phase = AFTER_COMMIT)` or an outbox relay). → *More:* §7.15 @TransactionalEventListener · §14.17 The transactional outbox · Ch 18 Q74

Q137. What is `@TransactionalEventListener`? An event listener tied to the publishing transaction's outcome: by default it runs **after commit** (other phases: `BEFORE_COMMIT`, `AFTER_ROLLBACK`, `AFTER_COMPLETION`). Use it to send emails or publish messages only when the data was really saved. It runs on the same thread after commit, outside the transaction; to write to the database from it, use `REQUIRES_NEW`. If no transaction is active the event is dropped unless `fallbackExecution = true`. → *More:* §7.15 @TransactionalEventListener · §14.3 @TransactionalEventListener: events tied to a commit

Q138. Declarative vs programmatic transactions — when do you use `TransactionTemplate`? Declarative (`@Transactional`) is the default — less code, consistent. Use `TransactionTemplate` (or `PlatformTransactionManager` directly) when you need a transaction around **part** of a method, a loop that commits every N items, different settings decided at runtime, or to work around self-invocation.

```
transactionTemplate.executeWithoutResult(status -> {
    repo.save(order);
    if (!valid(order)) status.setRollbackOnly();
});
```

→ *More:* §7.14 Programmatic transactions: TransactionTemplate · Ch 18 Q73

Q139. Does `@Transactional` work with `@Async` or in a new thread? No — a transaction is bound to one thread. An `@Async` method runs on another thread, so it does not join the caller's transaction; if it is itself `@Transactional`, it starts its own. The caller may also commit before or after the async work runs, so the async method might not yet see the caller's uncommitted data. Pass ids, not entities, to async code. → *More:* §14.4 @Async and executors · §7.13 Transaction pitfalls

Q140. How do you handle deadlocks, serialization failures or optimistic-lock conflicts? They are expected under concurrency: retry the **whole transaction** (not a single statement) with a small, jittered backoff and a limit — for example with Framework 7 `@Retryable` on a method that calls the `@Transactional` method (the retry must be *outside* the transaction). Reduce them by keeping transactions short and touching rows in a consistent order. → *More:* §7.13 Transaction pitfalls · Ch 18 Q75

Q141. ⋆ How do you handle a transaction that spans multiple microservices (distributed transactions)? Avoid two-phase commit (2PC/XA): it couples services and blocks on failures. Use the **saga** pattern: a sequence of local transactions, each publishing an event; on failure, **compensating actions** undo earlier steps (refund the payment, release the stock). Choreography = services react to each other's events; orchestration = a coordinator tells each service what to do. Combine with the **transactional outbox** so each local commit and its event are atomic, and make consumers idempotent. → *More:* §16.11 Messaging between services · §14.17 The transactional outbox · Ch 18 Q158

Caching

Q142. ⋆ How do you enable caching in Spring Boot? How does the cache abstraction work? Add `@EnableCaching` and a cache provider (Caffeine for in-memory, Redis for shared; without one Boot uses a simple `ConcurrentHashMap`). Annotate methods: `@Cacheable("products")` checks the cache first and stores the result on a miss; `@CachePut` always runs the method and updates the cache; `@CacheEvict` removes entries (`allEntries = true` clears the cache); `@Caching` groups several. It's proxy-based, so the usual proxy limits apply. The `CacheManager` abstraction lets you swap providers without changing code.

Follow-up — `@Cacheable` vs `@CachePut` vs `@CacheEvict`? `@Cacheable` may skip the method (on a hit). `@CachePut` never skips it; it writes the result into the cache — use on update methods so readers see fresh data. `@CacheEvict` deletes entries — on delete

methods, or when you prefer "invalidate and reload" over "write-through". Don't put `@Cacheable` and `@CachePut` on the same method. → *More:* §9.8 The caching abstraction · Ch 18 Q92

Q143. How is the cache key generated? How do you customize it? By default from the method parameters: no params → `SimpleKey.EMPTY`, one param → that value, several → a `SimpleKey` of all of them. Customize with SpEL: `@Cacheable(cacheNames = "product", key = "#id")` or `key = "#req.sku + ':' + #req.region"`, or with a custom `KeyGenerator` bean. Key objects need correct `equals`/`hashCode` (records are ideal).

Follow-up — `condition` vs `unless` in `@Cacheable`? `condition` is evaluated **before** the call on the arguments: if false, the cache is bypassed entirely. `unless` is evaluated **after** the call and can use `#result`: if true, the result is not stored (for example `unless = "#result == null"`). → *More:* §9.8 The caching abstraction · Ch 18 Q93 · Ch 18 Q94

Q144. Why is my `@Cacheable` not working? Usual causes: `@EnableCaching` missing; self-invocation (called from the same class); the method isn't public or the bean is created with `new`; the key differs each time (an object without `equals`/`hashCode`, or a timestamp argument); the cache provider isn't on the classpath or the cache name isn't configured (Caffeine with `spring.cache.cache-names`); or values aren't serializable for Redis. → *More:* §9.8 The caching abstraction · Ch 18 Q95

Q145. Local cache (Caffeine) vs distributed cache (Redis) — when to use which? Caffeine lives in each JVM: nanosecond reads, no network, but each instance has its own copy (inconsistency across instances, memory per instance). **Redis** is shared: all instances see the same data and it survives restarts, but every read is a network call and values must be serialized. Use local for small, read-heavy, tolerant-of-staleness data; Redis for shared session/state, larger datasets or when consistency across instances matters. A two-level setup combines both. → *More:* §9.8 The caching abstraction · §9.6 Spring Data Redis (not run — needs Docker) · Ch 18 Q96

Q146. How do you set a TTL and handle cache invalidation? What problems should you plan for? Set expiry per provider: Caffeine `spring.cache.caffeine.spec=maximumSize=10000,expireAfterWrite=10m`; Redis `spring.cache.redis.time-to-live=10m` or a `RedisCacheManagerBuilderCustomizer` for per-cache TTLs. Plan for: **stale data** (evict on writes, short TTLs); **cache stampede** (many threads miss at once — use `@Cacheable(sync = true)` or request coalescing); **cache penetration** (repeated lookups of missing keys — cache null/negative results briefly); **avalanche** (many keys expiring together — add jitter to TTLs); and unbounded memory growth (always set a maximum size). → *More:* §9.8 The caching abstraction · Ch 18 Q97

Q147. Cache-aside vs write-through vs write-behind? Cache-aside (what `@Cacheable` does): read from cache, on miss load from the database and populate the cache; writes go to the database and evict the cache. **Write-through**: every write updates the cache and database together (`@CachePut`). **Write-behind**: writes go to the cache first and are flushed to the database asynchronously — fast, but data can be lost on a crash. Most Spring apps use cache-aside with eviction on writes. → *More:* §9.8 The caching abstraction

---

Spring Security: filter chain, authentication, JWT and OAuth2

Q148. What happens when you add `spring-boot-starter-security` with no configuration? Every endpoint requires authentication; Boot creates one in-memory user `user` with a random password printed at startup; form login and HTTP Basic are enabled; CSRF protection, security headers (`X-Content-Type-Options`, `X-Frame-Options`, cache control, HSTS on HTTPS) and session-fixation protection are on. Defining your own `SecurityFilterChain` bean replaces the default chain; defining a `UserDetailsService` removes the generated user. → *More:* §10.1 Vocabulary and what the starter does by default · Ch 18 Q98

Q149. ⋆ Explain the Spring Security architecture (filter chain). How does a request flow through it? The servlet container calls `DelegatingFilterProxy` (a standard filter registered by Boot), which delegates to Spring's `FilterChainProxy` bean. `FilterChainProxy` picks the first `SecurityFilterChain` whose request matcher fits the URL and runs its ordered filters: CORS, CSRF, logout, authentication filters (`UsernamePasswordAuthenticationFilter`, `BasicAuthenticationFilter`, `BearerTokenAuthenticationFilter`), session management, `ExceptionTranslationFilter` (turns security exceptions into 401/403), and finally `AuthorizationFilter` (checks access rules). Only then does the request reach the `DispatcherServlet`.

Follow-up — What is `DelegatingFilterProxy`, and why is it needed? The servlet container creates filters itself and doesn't know about Spring beans. `DelegatingFilterProxy` is a plain servlet filter that, on each request, looks up a bean (`springSecurityFilterChain`, the `FilterChainProxy`) in the application context and delegates to it — bridging the servlet world and the Spring world so security filters can be beans with injected dependencies. → *More:* §10.2 Architecture: the filter chain · Ch 18 Q99

Q150. ⋆ How do you configure Spring Security today? (`WebSecurityConfigurerAdapter` is gone) Declare one or more `SecurityFilterChain` beans using the lambda DSL. `WebSecurityConfigurerAdapter` was removed in Security 6; Security 7 also removed the `.and()` chaining and `authorizeRequests()`.

```
@Bean
SecurityFilterChain api(HttpSecurity http) throws Exception {
    return http
        .authorizeHttpRequests(a -> a
            .requestMatchers("/actuator/health", "/public/**").permitAll()
            .requestMatchers(HttpMethod.DELETE, "/api/**").hasRole("ADMIN")
            .anyRequest().authenticated())
        .oauth2ResourceServer(o -> o.jwt(Customizer.withDefaults()))
        .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
        .build();
}
```

Rules are checked in order, so put specific matchers before general ones. → *More:* §10.3 Configuring a chain: the lambda DSL in Spring Security 7 · Ch 18 Q100

Q151. ⋆ Authentication vs authorization? Authentication answers "who are you?" — verifying identity with a password, token or certificate. **Authorization** answers "what may you do?" — checking roles or permissions for a URL or method. Authentication happens first; its result (an `Authentication` with authorities) is what authorization decisions use. → *More:* §10.1 Vocabulary and what the starter does by default

Q152. How does authentication work internally? (`AuthenticationManager`, `AuthenticationProvider`, `UserDetailsService`) An authentication filter extracts credentials into an unauthenticated `Authentication` token and passes it to the `AuthenticationManager` (usually `ProviderManager`), which asks each `AuthenticationProvider` that supports that token type. `DaoAuthenticationProvider` loads the user through your `UserDetailsService.loadUserByUsername`, checks the password with the `PasswordEncoder`, and returns an authenticated token with authorities. The filter stores it in the `SecurityContext` (held in `SecurityContextHolder`, thread-local by default) and saves it to the session if sessions are used. → *More:* §10.4 Authentication: users, providers, form login and Basic · Ch 18 Q101

Q153. What is `SecurityContextHolder`? How do you get the current user? `SecurityContextHolder` holds the `SecurityContext` for the current thread, which contains the `Authentication` (principal, authorities). In a controller, inject `@AuthenticationPrincipal UserDetails user` (or `Jwt jwt` for token auth), or an `Authentication` parameter; in other code, `SecurityContextHolder.getContext().getAuthentication()`. Because it's thread-local, it doesn't follow you into `@Async` threads unless you use `DelegatingSecurityContextExecutor` or a context-propagating task decorator. → *More:* §10.10 Sessions, the security context and logout · Ch 18 Q113

Q154. How should passwords be stored? Why BCrypt? Never store plain text or fast hashes (MD5, SHA-256). Use an adaptive, salted, slow hash — **BCrypt**, **Argon2**, **scrypt** or **PBKDF2** — so brute-forcing a leaked database is expensive. Spring's `DelegatingPasswordEncoder` (from `PasswordEncoderFactories.createDelegatingPasswordEncoder()`) stores the algorithm id as a prefix (`{bcrypt}$2a$...`), so you can upgrade algorithms later and re-hash on login. → *More:* §10.5 Password storage: hashing with DelegatingPasswordEncoder · Ch 18 Q102

Q155. ⋆ Roles vs authorities? `hasRole` vs `hasAuthority`? Both are `GrantedAuthority` strings. A role is just an authority with the prefix `ROLE_`. `hasRole("ADMIN")` checks for `ROLE_ADMIN`; `hasAuthority("orders:write")` checks the exact string. Roles are coarse ("ADMIN", "USER"); authorities are fine-grained permissions or OAuth2 scopes (`SCOPE_orders.read`). → *More:* §10.6 Authorizing requests: roles, authorities, 401 vs 403 · Ch 18 Q103

Q156. ⋆ 401 vs 403? 401 Unauthorized really means *unauthenticated*: no or invalid credentials — the client should log in or send a valid token (`AuthenticationEntryPoint` produces it). **403 Forbidden** means authenticated but not allowed (`AccessDeniedHandler` produces it). A 403 on a POST that "should" work is often a missing CSRF token. → *More:* §10.6 Authorizing requests: roles, authorities, 401 vs 403 · Ch 18 Q104

Q157. ⋆ How do you secure methods? `@PreAuthorize` vs `@Secured` vs `@RolesAllowed`? Enable with `@EnableMethodSecurity`. `@PreAuthorize("hasRole('ADMIN') or #id == authentication.name")` takes a SpEL expression and can use method arguments — the most flexible; `@PostAuthorize` checks the returned object. `@Secured("ROLE_ADMIN")` (Spring) and `@RolesAllowed("ADMIN")` (Jakarta) only list roles, no expressions; they need `securedEnabled` / `jsr250Enabled`. Method security is proxy-based. URL rules protect endpoints coarsely; method security protects business operations wherever they're called from — use both. → *More:* §10.7 Method security · Ch 18 Q105

Q158. ⋆ What is CSRF? When can you disable CSRF protection? CSRF (Cross-Site Request Forgery): a malicious site makes the victim's browser send a state-changing request to your site, and the browser automatically attaches the victim's session cookie. Spring defends with a per-session token that must accompany every POST/PUT/DELETE. You can disable it only for **stateless APIs authenticated by a header** (bearer token) that browsers don't add automatically. If you use cookies for auth (sessions, or tokens in cookies), keep it on — Security 7 has `csrf(c -> c.spa())` for single-page apps. → *More:* §10.8 CSRF protection · Ch 18 Q106

Q159. ⋆ What is a JWT? What are its parts? A JSON Web Token is a compact, signed token: `header.payload.signature`, each part Base64URL-encoded. The **header** says the algorithm (`RS256`, `HS256`) and key id; the **payload** holds **claims** — `sub` (user), `iss` (issuer), `aud` (audience), `exp` (expiry), `iat`, `scope`/roles; the **signature** proves it was issued by the key holder and wasn't changed. It is **signed, not encrypted** — anyone can read the payload, so never put secrets in it. → *More:* §11.1 From sessions to tokens: bearer tokens and JWT · Ch 18 Q110

Q160. ⋆ How do you implement JWT authentication in Spring Boot? Preferred: don't write a filter. Make the service an **OAuth2 resource server** (`spring-boot-starter-security-oauth2-resource-server` in Boot 4), set `spring.security.oauth2.resourceserver.jwt.issuer-uri` (or `jwk-set-uri`/a public key), and call `.oauth2ResourceServer(o -> o.jwt(...))`. Spring's `BearerTokenAuthenticationFilter` reads `Authorization: Bearer ...`, `JwtDecoder` checks signature, `exp`, `nbf` and issuer, and a `JwtAuthenticationConverter` maps claims to authorities. Tokens are issued by an authorization server (Keycloak, Okta, Spring Authorization Server) or, for simple cases, by your own login endpoint using `NimbusJwtEncoder`. Add audience validation for multi-service setups. → *More:* §11.3 A JWT resource server in Spring · §11.4 Minting tokens with NimbusJwtEncoder · Ch 18 Q108

Q161. Write a custom JWT filter (the way many interviewers still ask it). Extend `OncePerRequestFilter`, read and validate the header, set the `SecurityContext`, continue the chain; register it with `http.addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class)`. Mention that the built-in resource server does the same with less risk.

```java
class JwtFilter extends OncePerRequestFilter {
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res, FilterChain chain)
            throws ServletException, IOException {
        String h = req.getHeader("Authorization");
        if (h != null && h.startsWith("Bearer ")) {
            Jwt jwt = decoder.decode(h.substring(7));   // throws JwtException if invalid/expired: catch it, answer 401
            var auth = new JwtAuthenticationToken(jwt, authorities(jwt));
            SecurityContextHolder.getContext().setAuthentication(auth);
        }
        chain.doFilter(req, res);
    }
}
```

→ *More:* §11.3 A JWT resource server in Spring

Q162. ⋆ Session-based vs token-based (stateless) authentication? Sessions: the server stores login state; the browser holds a session-id cookie. Easy logout and revocation, but needs sticky sessions or a shared store (Spring Session with Redis) when scaled, and CSRF protection. **Tokens (JWT)**: the client sends a self-contained signed token on each request; the server keeps no state, so any instance can verify it — good for APIs and microservices — but revocation before expiry is hard (keep access tokens short-lived, 5–15 minutes, and use refresh tokens). → *More:* §10.10 Sessions, the security context and logout · §11.1 From sessions to tokens: bearer tokens and JWT · Ch 18 Q107

Q163. How do refresh tokens work? How do you log out or revoke a JWT? A short-lived **access token** is used on every call; a long-lived **refresh token** is sent only to the authorization server to get a new access token. To revoke: keep access tokens short, revoke/rotate refresh tokens server-side (rotation: each use issues a new refresh token and invalidates the old, detecting theft), and if you need instant revocation keep a deny-list of token ids (`jti`) in Redis, or use opaque tokens with introspection. → *More:* §11.11 Refresh tokens, lifetimes, revocation and logout

Q164. ⋆ What is OAuth 2.0? OAuth 2.0 vs OpenID Connect? Explain the authorization code flow. OAuth 2.0 is a **delegated authorization** framework: a client app gets an access token to call an API on a user's behalf without seeing the user's password. Roles: resource owner (user), client, authorization server, resource server. **OpenID Connect** adds **authentication** on top: an **ID token** (a JWT about the user) and a userinfo endpoint — "Sign in with Google" is OIDC. Authorization code flow: the client redirects the user to the authorization server → the user logs in and consents → the server redirects back with a short-lived code → the client exchanges code (plus PKCE verifier) for tokens on a back channel. **PKCE** (Proof Key for Code Exchange) stops a stolen code from being used.

Follow-up — What OAuth2 grant types exist, and which should you use? **Authorization code + PKCE** for any app acting for a user (web, mobile, SPA via a backend). **Client credentials** for service-to-service calls with no user. **Refresh token** to renew access tokens. **Device code** for TVs/CLIs. The **implicit** and **resource owner password** grants are deprecated in OAuth 2.1 best practice; Spring Security 7 removed client support for the password grant. → *More:* §11.2 OAuth 2.0 and OpenID Connect in one picture · §11.9 OAuth2 login: "Sign in with ..." · Ch 18 Q109 · §11.10 Calling other services: client credentials and token relay

Q165. How do you implement "Sign in with Google/GitHub" (OAuth2 login) in Spring Boot? Add the OAuth2 client starter, register the provider with `spring.security.oauth2.client.registration.google.client-id` / `client-secret` (Google, GitHub, Facebook and Okta have built-in provider settings), and enable `.oauth2Login(Customizer.withDefaults())`. Spring handles the redirect, code exchange, ID token validation and creates an `OidcUser` principal in the session. Map external users to your local accounts in a custom `OidcUserService`. → *More:* §11.9 OAuth2 login: "Sign in with ..."

Q166. JWT vs opaque tokens? A **JWT** is self-contained: the resource server validates it locally with the issuer's public key — fast, no network call, but hard to revoke and its claims are visible. An **opaque token** is a random string; the resource server must call the authorization server's **introspection** endpoint to learn if it's active and what it grants — instantly revocable and private, but adds a network call per request (cache the result). → *More:* §11.7 Opaque tokens and introspection · Ch 18 Q110

Q167. How does one microservice call another securely? With the **client credentials** grant: the calling service gets its own access token from the authorization server and sends it as a bearer token (Spring's `OAuth2ClientHttpRequestInterceptor` / `@ClientRegistrationId` on HTTP interfaces handles fetching and caching). To act on behalf of the original user, **relay** the incoming token (token relay at the gateway) or use token exchange. Inside a cluster, add **mTLS** (mutual TLS — both sides present certificates), often provided by a service mesh. → *More:* §11.10 Calling other services: client credentials and token relay · Ch 18 Q111

Q168. Where should a browser single-page app keep its tokens? What is the BFF pattern? Not in `localStorage` — any XSS (cross-site scripting) script can steal it. The recommended **BFF** (Backend for Frontend) pattern keeps tokens on a server-side component (for example Spring Cloud Gateway with `oauth2Login` and `TokenRelay`); the browser only gets an `HttpOnly`, `Secure`, `SameSite` session cookie, and the BFF attaches tokens when proxying API calls. Then keep CSRF protection on. → *More:* §11.12 Where should a browser keep tokens? The BFF pattern · Ch 18 Q112

Q169. How do you protect an API against brute-force login attempts and abuse (rate limiting)? Limit attempts per account and per IP (a failed-attempts counter with lockout or growing delays), add CAPTCHA or MFA after failures, and rate-limit at the edge: Spring Cloud Gateway's `RequestRateLimiter` with Redis, Bucket4j in a filter, or an API gateway/WAF. Return 429 with `Retry-After`. Log and alert on spikes. → *More:* §11.15 Rate limiting and brute-force protection

Q170. What security headers does Spring Security add, and what common web attacks does it help prevent? Defaults: `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY` (clickjacking), `Cache-Control: no-cache, no-store`, `Strict-Transport-Security` on HTTPS, `X-XSS-Protection: 0`. You can add a Content-Security-Policy (main XSS defence) and Referrer-Policy. Spring Security also covers CSRF, session fixation (new session id after login), and timing-safe password checks; SQL injection is prevented by parameterized queries (JPA, `JdbcClient`), and XSS by output encoding in templates. → *More:* §10.11 Security headers

Q171. Can you have several security configurations in one app (e.g. API with JWT, admin UI with form login)? Yes: declare several `SecurityFilterChain` beans, each with `securityMatcher("/api/**")` etc., and order them with `@Order` (most specific first). `FilterChainProxy` uses the first chain that matches the request. A chain without a matcher catches everything, so it must be last. → *More:* §10.3 Configuring a chain: the lambda DSL in Spring Security 7

Q172. (Security 7) What is new in Spring Security 7? Lambda-only DSL (no `.and()`), `authorizeRequests()` and `AntPathRequestMatcher` / `MvcRequestMatcher` removed (use `authorizeHttpRequests()` and `PathPatternRequestMatcher`), built-in **multi-factor authentication** support, `csrf(c -> c.spa())`, Spring Authorization Server shipped inside Security (with PKCE enabled by default), password-grant support removed, and `@ClientRegistrationId` allowed at type level on HTTP service interfaces. → *More:* §17.15 Spring Security 6 → 7 · Ch 18 Q114

Testing

Q173. How do you test a Spring Boot application? Explain the test pyramid. Many fast **unit tests** with no Spring (plain JUnit 6 + Mockito, constructor injection makes this easy); fewer **slice tests** that start only part of the context (`@WebMvcTest`, `@DataJpaTest`); a few **integration tests** with `@SpringBootTest` and real infrastructure via Testcontainers; and a handful of end-to-end/contract tests. `spring-boot-starter-test` brings JUnit, AssertJ, Mockito, Spring Test, JSONassert and Awaitility; Boot 4 adds per-technology test starters. → *More:* §12.2 The test pyramid in a Spring app · §12.3 Test dependencies in Boot 4 · Ch 18 Q115

Q174. ⋆ `@SpringBootTest` vs `@WebMvcTest` vs `@DataJpaTest`? `@SpringBootTest` loads the **whole** application context — realistic but slower; add `webEnvironment = RANDOM_PORT` to start a real server. `@WebMvcTest(OrderController.class)` loads only the web layer (controllers, advice, filters, converters, security) with `MockMvc`; services must be mocked with `@MockitoBean`. `@DataJpaTest` loads only JPA (entities, repositories, `DataSource`, Flyway), runs each test in a rolled-back transaction, and uses an embedded database unless told otherwise. → *More:* §12.6 @SpringBootTest: the whole application · §12.7 Test slices · Ch 18 Q116 · Ch 18 Q121

Q175. ⋆ `@Mock` vs `@MockBean` vs `@MockitoBean`? Why doesn't `@MockBean` compile in Boot 4? `@Mock` (Mockito) creates a mock in a plain unit test — no Spring involved. `@MockitoBean` (Spring Framework 6.2+) **replaces a bean in the Spring context** with a Mockito mock; `@MockitoSpyBean` wraps the real bean in a spy. Boot's `@MockBean` / `@SpyBean` were deprecated in Boot 3.4 and **removed in Boot 4**, so migrate to the Framework annotations. → *More:* §12.10 @MockitoBean and @MockitoSpyBean · Ch 18 Q117

Q176. How do you test a REST controller? With `@WebMvcTest` and `MockMvcTester` (AssertJ-style MockMvc, Framework 6.2+ / Boot 3.4+) or `RestTestClient` (Framework 7), mocking the service:

```
@WebMvcTest(OrderController.class)
class OrderControllerTest {
    @Autowired MockMvcTester mvc;
    @MockitoBean OrderService service;

    @Test void returns404() {
        given(service.find(1L)).willThrow(new OrderNotFoundException(1L));
        assertThat(mvc.get().uri("/api/orders/1")).hasStatus(HttpStatus.NOT_FOUND);
    }
}
```

Follow-up — `MockMvc` vs `TestRestTemplate` / `RestTestClient` vs `WebTestClient`? `MockMvc` / `MockMvcTester` call the `DispatcherServlet` in-process without a real HTTP server — fast, but filters and serialization are exercised only through the mock request. `RestTestClient` (Framework 7) and `TestRestTemplate` send real HTTP to a server on a random port — full stack. `WebTestClient` is the reactive equivalent and also works against MVC apps. With `@SpringBootTest`, these clients need explicit opt-in: `@AutoConfigureMockMvc` (as before), and in Boot 4 also `@AutoConfigureRestTestClient` or `@AutoConfigureTestRestTemplate`. A `RANDOM_PORT` test no longer gets a `TestRestTemplate` automatically. → *More:* §12.8 MockMvcTester: AssertJ-style MockMvc · §12.9 RestTestClient (Framework 7.0) · Ch 18 Q118 · §12.13 Full-stack tests on a real port

Q177. What is test context caching? Why do my integration tests get slower as I add more? The Spring TestContext Framework caches application contexts and reuses one for every test class with the **same configuration**. Each different combination of `@MockitoBean`s, profiles, properties, `@Import`s or `@DirtiesContext` creates a new context (seconds each). Keep test configuration uniform (shared base class or meta-annotation), avoid `@DirtiesContext`, and watch the log for repeated startups. → *More:* §12.11 Test context caching · Ch 18 Q119

Q178. Does `@Transactional` on a test method roll back? Yes — in Spring tests, a test-managed transaction rolls back after each test by default (`@Commit` changes that). Caveat: with `@SpringBootTest(webEnvironment = RANDOM_PORT)` the HTTP request runs on a server thread in a **different** transaction, so the test's rollback doesn't undo it. Also, rollback means the flush may never happen — call `flush()` to catch constraint errors. → *More:* §12.6 @SpringBootTest: the whole application · Ch 18 Q120

Q179. ⋆ How do you test against a real database, Kafka or Redis? (Testcontainers) Use **Testcontainers**: it starts real services in Docker for the test. Annotate the container field with `@ServiceConnection` and Boot wires the connection properties automatically (no `@DynamicPropertySource` needed); share containers across classes so they start once. This catches SQL dialect, migration and driver issues an in-memory H2 database hides.

```
@Testcontainers @SpringBootTest
class RepoIT {
    @Container @ServiceConnection
    static PostgreSQLContainer postgres = new PostgreSQLContainer("postgres:17-alpine");
}
```

Follow-up — What is `@DynamicPropertySource`, and when do you use `@TestPropertySource` or `@ActiveProfiles`? `@DynamicPropertySource` registers properties whose values are only known at runtime (a container's random port) before the context starts. `@TestPropertySource` and `@SpringBootTest(properties = ...)` add fixed test properties with high precedence. `@ActiveProfiles("test")` activates `application-test.properties`. Each different combination is a different cached context. → *More:* §12.14 Testcontainers and @ServiceConnection · Ch 18 Q122 · §12.12 @DynamicPropertySource: properties known only at runtime · §12.15 Profiles and properties in tests

Q180. How do you test secured endpoints? With `spring-security-test`: `@WithMockUser(roles = "ADMIN")` on a test, or request post-processors such as `.with(jwt().authorities(...))`, `.with(user("ann"))` and `.with(csrf())` on MockMvc requests. Test both the allowed and the forbidden path (401 and 403). → *More:* §10.12 Testing security with spring-security-test · §11.6 Testing token security · Ch 18 Q123

Q181. How do you test a service that calls an external HTTP API? Unit level: mock the client interface. Client level: `@RestClientTest` with `MockRestServiceServer` to assert the exact request and return canned responses. Integration level: **WireMock** (a fake HTTP server) to simulate slow or failing responses and test timeouts and retries. Contract tests (Spring Cloud Contract, Pact) ensure your fake matches the real provider. → *More:* §6.8 Testing clients · Ch 18 Q61

Q182. How do you test `@Async` code, scheduled jobs or Kafka listeners? Don't sleep — use **Awaitility** (`await().atMost(5, SECONDS).untilAsserted(...)`) to poll for the expected effect. For scheduled jobs, test the job's method directly and keep the schedule as configuration. For Kafka, use `@EmbeddedKafka` or a Kafka Testcontainer, send a message and await the side effect. → *More:* §14.13 Testing with @EmbeddedKafka · §12.16 Common mistakes — checklist

Q183. How do you test custom auto-configuration or conditional beans? `ApplicationContextRunner` starts a tiny context in milliseconds per test: `.withUserConfiguration(...)`, `.withPropertyValues("feature.enabled=true")`, `.withClassLoader(new FilteredClassLoader(SomeLib.class))`, then assert `context.hasSingleBean(...)`. → *More:* §3.13 Writing your own auto-configuration and starter · Ch 18 Q124

WebFlux, async, scheduling and messaging

Q184. ⋆ What is Spring WebFlux? Spring MVC vs WebFlux — when would you choose each? WebFlux is Spring's **non-blocking, reactive** web stack, built on Project Reactor and usually running on Netty. Instead of one thread per request, a few event-loop threads handle many connections, and code returns `Mono` / `Flux` that complete when data arrives. Choose WebFlux for streaming (SSE, WebSockets), gateways and apps that fan out to many slow remote calls with an end-to-end reactive stack (R2DBC, reactive drivers). Choose MVC for most CRUD services, especially with JPA/JDBC (blocking) — and since Java 21, **MVC on virtual threads** gives similar scalability for blocking I/O with simpler code. → *More:* §13.1 Why reactive? The cost of waiting · §13.15 Virtual threads vs WebFlux: a measured comparison · Ch 18 Q131

Q185. What is reactive programming? What is backpressure? Reactive programming composes asynchronous streams of data with operators, reacting to items as they arrive instead of blocking a thread while waiting. **Reactive Streams** is the standard contract (`Publisher`, `Subscriber`, `Subscription`, `Processor`) adopted into Java as `java.util.concurrent.Flow`. **Backpressure** means the subscriber tells the publisher how many items it can handle (`request(n)`), so a fast producer can't overwhelm a slow consumer. → *More:* §13.2 Reactive Streams: the contract · §13.7 Backpressure in practice · Ch 18 Q126

Q186. ⋆ `Mono` vs `Flux`? Both are Reactor `Publisher`s. `Mono<T>` emits 0 or 1 item then completes (or errors) — like an async `Optional`, for a single result. `Flux<T>` emits 0..N items — for lists and streams. **Nothing happens until someone subscribes**: in WebFlux the framework subscribes to what your controller returns.

Follow-up — Hot vs cold publishers? A **cold** publisher starts producing from scratch for each subscriber (a `WebClient` call runs once per subscription). A **hot** publisher emits regardless of subscribers, and late subscribers miss earlier items (a stream of price ticks, `Sinks.many().multicast()`). `share()`, `cache()` and `replay()` turn cold into hot.

Follow-up — How do you test reactive code? With `StepVerifier`: `StepVerifier.create(flux).expectNext(1, 2).expectComplete().verify()`; `withVirtualTime` for delays without waiting. Test WebFlux endpoints with `WebTestClient`. → *More:* §13.3 Project Reactor: Mono and Flux · Ch 18 Q127 · §13.9 Hot vs cold, sinks and context · §13.5 Testing with StepVerifier and virtual time · Ch 18 Q132

Q187. `map` vs `flatMap` (vs `concatMap`) in Reactor? `map` transforms each item **synchronously** (`T → R`). `flatMap` transforms each item into another publisher (`T → Mono<R>`), subscribes to them **concurrently** and merges results — order not preserved. `concatMap` does the same but one at a time, preserving order. Use `flatMap` for async calls such as a `WebClient` request per item; limit concurrency with `flatMap(fn, 8)`. → *More:* §13.4 Operators · Ch 18 Q128

Q188. What happens if you call blocking code (JDBC, `Thread.sleep`, `.block()`) in a WebFlux handler? You block one of the few event-loop threads; all other requests assigned to it stall and throughput collapses. Wrap unavoidable blocking calls with `Mono.fromCallable(...).subscribeOn(Schedulers.boundedElastic())`, use reactive drivers (R2DBC, reactive Mongo/Redis), and detect mistakes in tests with BlockHound. → *More:* §13.8 Schedulers and threads · Ch 18 Q129

Q189. `subscribeOn` vs `publishOn`? `subscribeOn` decides which scheduler the **subscription and source** run on — its position in the chain doesn't matter (first one wins). `publishOn` switches the thread for the operators **after** it in the chain. Use `subscribeOn(boundedElastic())` to move a blocking source off the event loop; use `publishOn` to hop threads mid-pipeline. → *More:* §13.8 Schedulers and threads · Ch 18 Q130

Q190. How do you handle errors in Reactor? Errors are terminal signals that flow downstream. Operators: `onErrorReturn(fallback)`, `onErrorResume(e -> otherPublisher)`, `onErrorMap(e -> new MyException(e))`, `doOnError(log)`, `retryWhen(Retry.backoff(3, Duration.ofMillis(200)))`, and `timeout(Duration)`. `switchIfEmpty` handles "no value" (e.g. return 404). In WebFlux, `@ExceptionHandler` / `@ControllerAdvice` work as in MVC. → *More:* §13.6 Error handling

Q191. ⋆ What are virtual threads, and how do they change Spring Boot apps? Virtual threads (Java 21+) are lightweight threads managed by the JVM: blocking I/O unmounts them from the carrier OS thread, so you can have hundreds of thousands. With `spring.threads.virtual.enabled=true`, Boot runs Tomcat request handling, `@Async`, `@Scheduled` and some listeners on virtual threads. Benefit: thread-per-request code scales for I/O-bound work without reactive code. Watch-outs: the **connection pool** becomes the limit, `ThreadLocal`-heavy code multiplies memory, CPU-bound work doesn't benefit, and before Java 24 `synchronized` blocks pinned the carrier thread (fixed by JEP 491 in Java 24). → *More:* §15.10 Virtual threads · §13.15 Virtual threads vs WebFlux: a measured comparison · Ch 18 Q148

Q192. ⋆ How does `@Async` work? What are its pitfalls? With `@EnableAsync`, calls to an `@Async` method go through a proxy that submits the work to a `TaskExecutor` and returns immediately (`void` or `CompletableFuture<T>`). Pitfalls: self-invocation runs it synchronously; exceptions from `void` methods are lost unless you configure an `AsyncUncaughtExceptionHandler`; the security context and transaction don't propagate to the new thread; and Boot's default executor queue is unbounded — configure `spring.task.execution.pool.*` or use virtual threads. → *More:* §14.4 @Async and executors · §14.6 Exceptions in async methods · Ch 18 Q133

Q193. How do you call three services in parallel and combine the results? In MVC: `CompletableFuture.supplyAsync(...)` per call on a bounded executor (or virtual threads), then `CompletableFuture.allOf(a, b, c).join()` and read each result; always add timeouts (`orTimeout`). Java 25's structured concurrency (`StructuredTaskScope`) is still a preview API. In WebFlux: `Mono.zip(callA(), callB(), callC())`.

```
var price = CompletableFuture.supplyAsync(() -> pricing.get(id), executor);
var stock = CompletableFuture.supplyAsync(() -> inventory.get(id), executor);
return price.thenCombine(stock, ProductView::new).orTimeout(2, TimeUnit.SECONDS).join();
```

→ *More:* §14.4 @Async and executors · §13.4 Operators

Q194. ⋆ How do you schedule tasks? `fixedRate` vs `fixedDelay` vs `cron`? `@EnableScheduling` plus `@Scheduled` on a bean method. `fixedRate = 5000` starts every 5 s measured from the **start** of the previous run; `fixedDelay = 5000` waits 5 s after the previous run **finishes**; `cron = "0 0 2 * * *"` uses a 6-field cron expression (second first) with an optional `zone`. Durations can be strings (`fixedRateString = "PT5M"`). By default all scheduled tasks share **one thread**, so one slow job delays others — set `spring.task.scheduling.pool.size` or use virtual threads. → *More:* §14.7 Scheduling with @Scheduled · Ch 18 Q134

Q195. ⋆ A `@Scheduled` job runs on every instance of a scaled-out service. How do you run it only once? Use a distributed lock: **ShedLock** (`@SchedulerLock` with a lock table in the database or Redis), a database row lock (`SELECT ... FOR UPDATE SKIP LOCKED` to claim work), or Spring Integration's leader election. Alternatively, move the job out of the service into a Kubernetes CronJob or a workflow scheduler. Make the job idempotent anyway. → *More:* §14.7 Scheduling with @Scheduled · Ch 18 Q135

Q196. ⋆ How do you produce and consume Kafka messages with Spring Boot? Add `spring-kafka` (Boot 4: `spring-boot-starter-kafka`), configure `spring.kafka.bootstrap-servers` and (de)serializers. Produce with `KafkaTemplate.send(topic, key, value)` — the **key** picks the partition, so events for one order stay in order. Consume with `@KafkaListener(topics = "orders", groupId = "billing")`; consumers in the same group split the partitions, so concurrency is capped by the partition count. → *More:* §14.10 Setting up Spring Kafka (with JSON) · §14.11 Producing: KafkaTemplate, keys and partitions · §14.12 Consuming: @KafkaListener, concurrency and consumer groups · Ch 18 Q136

Q197. ⋆ What delivery guarantees does Kafka give? How do you handle duplicate messages (idempotent consumer)? Spring Kafka listeners give **at-least-once** delivery by default: the offset is committed after your listener returns, so a crash before the commit causes redelivery. Therefore consumers must be **idempotent**: store processed message ids (or a business key) in a table with a unique constraint inside the same transaction as the side effect, or use naturally idempotent operations (upserts). Exactly-once is achievable only for Kafka-to-Kafka flows with transactions. → *More:* §14.12 Consuming: @KafkaListener, concurrency and consumer groups · §14.16 Producer guarantees: idempotence and transactions · Ch 18 Q137

Q198. How do you handle a message that keeps failing (poison message)? Explain retries and dead-letter topics. Configure a `DefaultErrorHandler` with a `BackOff` (for example 3 retries, exponential) and a `DeadLetterPublishingRecoverer` that publishes the failed record to `orders.DLT` with exception headers, then moves on. Mark non-retryable exceptions (validation, deserialization) so they go straight to the DLT. `@RetryableTopic` gives **non-blocking** retries through delay topics so one bad message doesn't block its partition. Monitor and replay the DLT. → *More:* §14.14 Error handling: DefaultErrorHandler and the dead-letter topic · §14.15 Non-blocking retries with @RetryableTopic · Ch 18 Q138

Q199. ⋆ What is the transactional outbox pattern? (dual-write problem) Writing to the database and publishing to Kafka are two systems; one can succeed while the other fails (the **dual-write problem**). With an outbox, the service writes the business change **and** an event row into an `outbox` table in the same database transaction; a separate relay (a scheduled poller, or CDC — change data capture — with Debezium) publishes outbox rows to Kafka and marks them sent. Delivery becomes at-least-once, so consumers must be idempotent. → *More:* §14.17 The transactional outbox · Ch 18 Q139

Q200. Kafka vs RabbitMQ — when do you use which? Kafka is a distributed, partitioned, replicated **log**: messages are retained and can be replayed, ordering is per partition, very high throughput — for event streaming, event sourcing, analytics pipelines. **RabbitMQ** is a **message broker** with flexible routing (exchanges, bindings), per-message acknowledgement, priorities and TTLs; messages are removed once consumed — for task queues and request/reply. Spring supports both (`spring-kafka`, Spring AMQP). → *More:* §14.8 Messaging concepts you need first · §14.18 RabbitMQ with Spring AMQP (configuration, not run) · Ch 18 Q140

Q201. How do you guarantee message ordering in Kafka? Kafka orders only within a partition. Use the entity id as the message key so all events for it go to the same partition; enable the idempotent producer (default in modern clients) so retries don't reorder; on the consumer side, process a partition sequentially (one consumer thread per partition) and be careful with retry topics, which break ordering. → *More:* §14.11 Producing: KafkaTemplate, keys and partitions · §14.16 Producer guarantees: idempotence and transactions

Actuator, observability, production and performance

Q202. ⋆ What is Spring Boot Actuator? Which endpoints do you know, and how do you secure them? Actuator adds production endpoints: `health`, `info`, `metrics`, `prometheus`, `loggers` (change levels at runtime), `env`, `configprops`, `beans`, `conditions`, `mappings`, `threaddump`, `heapdump`, `scheduledtasks`, `httpexchanges`, `shutdown` (off by default). Over HTTP only `health` is exposed by default. Expose deliberately (`management.endpoints.web.exposure.include=health,info,prometheus`), consider a separate `management.server.port` not reachable from outside, and protect everything except health with Spring Security — `env` and `heapdump` can leak secrets. → *More:* §15.2 Spring Boot Actuator · Ch 18 Q141

Q203. How do you write a custom health indicator? Implement `HealthIndicator` (or `ReactiveHealthIndicator`) as a bean and return `Health.up().withDetail(...)` or `Health.down(ex)`; its name (bean name without `HealthIndicator`) appears under `/actuator/health`. Group indicators into `liveness` and `readiness` groups. Don't make liveness depend on external systems.

```
@Component
class PaymentGatewayHealthIndicator implements HealthIndicator {
    public Health health() {
        return client.ping() ? Health.up().build() : Health.down().withDetail("gateway", "unreachable").build();
    }
}
```

→ *More:* §15.3 Health, health groups and Kubernetes probes

Q204. Liveness vs readiness probes? Liveness = "is the process healthy, or should it be restarted?" — fail only for unrecoverable states (deadlock, corrupted state). **Readiness** = "can it take traffic right now?" — false during startup, warm-up, shutdown or when a critical dependency is down; Kubernetes stops routing traffic instead of restarting. Boot exposes `/actuator/health/liveness` and `/actuator/health/readiness` (enabled by default in Boot 4; in Boot 3 only when it detects Kubernetes or with `management.endpoint.health.probes.enabled=true`). Never put the database in liveness — a database blip would restart every pod. → *More:* §15.3 Health, health groups and Kubernetes probes · Ch 18 Q142

Q205. ⋆ How do you monitor a Spring Boot app in production? What is Micrometer? The three pillars: **metrics** (Micrometer → Prometheus → Grafana dashboards and alerts), **logs** (structured JSON → ELK/Loki), **traces** (Micrometer Tracing/OpenTelemetry → Zipkin/Tempo/Jaeger). **Micrometer** is a vendor-neutral metrics facade ("SLF4J for metrics"): you record `Counter`, `Gauge`, `Timer`, `DistributionSummary` and `LongTaskTimer` meters and it exports to Prometheus, Datadog, OTLP etc. Boot auto-instruments HTTP requests, JVM, connection pools, caches and Kafka. Track the RED metrics (rate, errors, duration) per endpoint. → *More:* §15.1 Observability in one page · §15.5 Metrics with Micrometer · Ch 18 Q143

Q206. ⋆ How does distributed tracing work in Spring Boot? What replaced Spring Cloud Sleuth? Each request gets a **trace id**; each hop (HTTP call, message) is a **span** with its own id and a parent id. Ids travel in headers (W3C `traceparent`), so a tracing backend can reconstruct the whole call tree with timings. Since Boot 3, **Micrometer Tracing** (with an OpenTelemetry or Brave bridge) replaced Sleuth; auto-configured `RestClient`, `WebClient`, `KafkaTemplate` propagate context, and the trace id appears in logs for correlation. Sample a percentage in production (`management.tracing.sampling.probability`).

Follow-up — How do you trace a request and correlate logs across microservices? Use Micrometer Tracing (or OpenTelemetry) so every service propagates `traceparent` headers over HTTP and Kafka; Boot puts `traceId` and `spanId` into the logging MDC, so with structured JSON logs you can search all services' logs by one trace id and see the call tree in Zipkin/Tempo. → *More:* §15.7 Distributed tracing · §16.9 Distributed tracing across services · Ch 18 Q145 · §15.8 Structured logging

Q207. What is the Observation API (`@Observed`)? Micrometer's Observation API instruments code **once** and produces both a metric (timer) and a trace span (and log correlation) from it. Spring's own instrumentation uses it. Annotate a method with `@Observed(name = "order.place")` or wrap code in `Observation.createNotStarted(...).observe(...)`; add low-cardinality tags for metrics and high-cardinality ones only for traces. → *More:* §15.6 The Observation API and @Observed · Ch 18 Q144

Q208. What is graceful shutdown? On SIGTERM, Boot stops accepting new requests, lets in-flight requests finish (up to `spring.lifecycle.timeout-per-shutdown-phase`, default 30 s), then closes beans. Graceful shutdown is enabled by default since Boot 3.4. Readiness turns to refusing traffic first. In Kubernetes, align `terminationGracePeriodSeconds` and add a short `preStop` sleep so the load balancer stops routing before the app stops. → *More:* §15.9 Graceful shutdown · Ch 18 Q147

Q209. How do you improve Spring Boot startup time and memory usage? Measure first: `ApplicationStartup` (`BufferingApplicationStartup` + `/actuator/startup`) shows slow beans. Then: remove unused starters (Boot 4's modules help), avoid heavy work in `@PostConstruct`, lazy-initialize rarely-used beans, use **CDS / the AOT cache** (Java 24+ JEP 483: train once, start faster — Boot supports extracted jars for it), Spring AOT processing, or a **GraalVM native image** for instant startup and low memory at the cost of long builds and reflection limits. Tune the JVM for containers (`-XX:MaxRAMPercentage`). → *More:* §15.11 Faster startup: extracted jars, CDS, AOT cache, native images · Ch 18 Q149 · Ch 18 Q150

Q210. What are AOT processing and GraalVM native images? What are the trade-offs? AOT (ahead-of-time) processing runs at build time: it evaluates the configuration and generates Java source for bean definitions plus hints for reflection, proxies and resources. A **native image** compiles the app and JVM parts into a platform-specific executable: starts in tens of milliseconds with a small memory footprint, ideal for scale-to-zero and CLIs. Trade-offs: slow builds, a "closed world" (classpath fixed at build time, `@Profile`/`@ConditionalOnProperty` decided at build), libraries need reflection hints, and peak throughput may be lower than a warmed-up JIT. → *More:* §15.11 Faster startup: extracted jars, CDS, AOT cache, native images · Ch 18 Q149

Q211. How do you package a Spring Boot app as a Docker image? Either `mvn spring-boot:build-image` (Cloud Native Buildpacks — no Dockerfile, a layered, secure image) or a multi-stage Dockerfile that uses Boot's **layered jar** (`java -Djarmode=tools -jar app.jar extract --layers`) so dependencies sit in a separate cached layer from your code. Run as a non-root user, set memory flags, and use a JRE base image. → *More:* §15.11 Faster startup: extracted jars, CDS, AOT cache, native images · Ch 18 Q151

Q212. ⋆ How would you find out why an endpoint is slow?

1. Metrics: is it slow for all requests or the p99 only, and since when (deploy? traffic?) — `http.server.requests` timers. 2) Traces: which span takes the time — database, a downstream call, or the app itself. 3) Database: SQL logs/Hibernate statistics for N+1 queries, `EXPLAIN ANALYZE` for missing indexes, connection-pool wait time (`hikaricp.connections.pending`). 4) JVM: thread dumps (blocked threads, lock contention), GC pauses, CPU profiling with async-profiler or JFR (Java Flight Recorder). 5) Fix the cause (index, fetch join, cache, timeout), and re-measure. → *More:* §15.5 Metrics with Micrometer · §8.18 Performance checklist

Q213. How do you diagnose high memory usage or an `OutOfMemoryError` in a Spring Boot app? Run with `-XX:+HeapDumpOnOutOfMemoryError`, or capture `/actuator/heapdump`, and analyze in Eclipse MAT or VisualVM for the dominating objects. Usual Spring culprits: loading a huge result set (`findAll()`), an unbounded cache or `ConcurrentHashMap` used as a cache, an unbounded `@Async` queue, a persistence context growing in a batch loop (missing `clear()`), large request bodies read into memory, and `ThreadLocal`s not cleaned up. Check also container limits vs `-Xmx` (non-heap memory: metaspace, thread stacks, direct buffers). → *More:* §15.2 Spring Boot Actuator · §8.18 Performance checklist

Q214. What is on your production-readiness checklist for a Spring Boot service? Externalized config and secrets; health groups and probes; metrics, structured logs and tracing with correlation ids; graceful shutdown; timeouts and circuit breakers on every remote call; connection pool sized and monitored; Flyway migrations with `ddl-auto=validate`; OSIV off; Actuator endpoints restricted; security headers and TLS; container image with memory limits; resource limits and autoscaling; dashboards and alerts on error rate and latency. → *More:* §15.12 Production configuration checklist · Ch 18 Q152

Microservices and Spring Cloud

Q215. ⋆ What are microservices? Monolith vs microservices — pros and cons? Microservices split a system into small services, each owning one business capability and its own data, deployed independently and communicating over the network (HTTP, messaging). Pros: independent deployment and scaling, team autonomy, fault isolation, technology freedom. Cons: distributed-systems complexity (network failures, latency, eventual consistency), harder testing and debugging, operational overhead (CI/CD, observability, platform). A **modular monolith** (Spring Modulith) is often the better start. → *More:* §16.1 The map: microservice problems and Spring answers

Q216. ⋆ How does Spring Boot/Spring Cloud support microservices? Name the main Spring Cloud components. Boot makes each service a self-contained jar with health, metrics and config. Spring Cloud adds the distributed-system patterns: **Config** (centralized configuration), **Netflix Eureka**/Consul/Kubernetes (service discovery), **LoadBalancer** (client-side load balancing), **Gateway** (API gateway), **Circuit Breaker** with Resilience4j, **OpenFeign** (declarative clients, now in maintenance), **Stream** (messaging binders), **Contract** (consumer-driven contract tests), and **Kubernetes** integration. Boot 4.0 and 4.1 use Spring Cloud 2025.1.x ("Oakwood"); Boot 4.1 needs 2025.1.2 or later. → *More:* §16.1 The map: microservice problems and Spring answers · §16.2 Release trains and versions · Ch 18 Q153

Q217. What is service discovery? How does Eureka work? Instances come and go with changing IPs, so clients look services up by **name** in a registry instead of hard-coding addresses. With **Eureka**, each service registers itself on startup and sends heartbeats; clients fetch and cache the registry and pick an instance via Spring Cloud LoadBalancer (`http://inventory-service/...` with a `@LoadBalanced` `RestClient.Builder`). On Kubernetes you usually don't need Eureka — Kubernetes Services and DNS do discovery. → *More:* §16.4 Service discovery: Eureka, Consul, Kubernetes · §16.5 Client-side load balancing with Spring Cloud LoadBalancer · Ch 18 Q155

Q218. ⋆ What is an API gateway? How does Spring Cloud Gateway work? An API gateway is the single entry point for clients: routing to services, authentication/token relay, rate limiting, CORS, retries, request/response rewriting and aggregation. Spring Cloud Gateway defines **routes** (id, destination URI, **predicates** such as `Path=/orders/**`, and **filters** such as `StripPrefix`, `RequestRateLimiter`, `CircuitBreaker`). It comes in a reactive (WebFlux/Netty) variant and a Server Web MVC variant. Zuul 1 is retired. → *More:* §16.7 API gateway: Spring Cloud Gateway · Ch 18 Q156

Q219. ⋆ What is the circuit breaker pattern? How do you implement it in Spring Boot? A circuit breaker wraps calls to a dependency and tracks failures. **Closed**: calls pass. When the failure rate (or slow-call rate) passes a threshold, it goes **open**: calls fail fast (or use a fallback) without hitting the struggling service. After a wait it goes **half-open** and lets a few trial calls through; success closes it again. Implement with **Resilience4j** (`@CircuitBreaker(name = "inventory", fallbackMethod = "cached")` plus `resilience4j.circuitbreaker.instances.*` properties) or Spring Cloud Circuit Breaker. Hystrix is dead. → *More:* §16.8 Resilience: circuit breaker, retry, rate limiter · Ch 18 Q157

Q220. What other resilience patterns do you know? (retry, timeout, bulkhead, rate limiter, fallback) **Timeout**: never wait forever. **Retry** with exponential backoff and jitter, only for idempotent, transient failures. **Bulkhead**: cap concurrent calls per dependency so one slow service can't use every thread. **Rate limiter**: cap calls per time window. **Fallback**: a cached or default response. Order matters: Resilience4j applies retry outside the circuit breaker so the breaker counts each attempt. → *More:* §16.8 Resilience: circuit breaker, retry, rate limiter · §4.7 Framework 7 core resilience: @Retryable and @ConcurrencyLimit

Q221. ⋆ How do microservices communicate? Synchronous vs asynchronous? **Synchronous** (REST over HTTP, gRPC): simple request/response, immediate answer, but temporal coupling — if the callee is down or slow, the caller suffers (needs timeouts, retries, circuit breakers). **Asynchronous** (Kafka, RabbitMQ events): the sender doesn't wait; services are decoupled in time, absorb load spikes, but you get eventual consistency, duplicate messages and harder debugging. Use sync for queries that need an answer now; async for propagating state changes between services. → *More:* §16.11 Messaging between services · §14.8 Messaging concepts you need first

Q222. What does Spring Cloud Config do? How do you refresh configuration without restarting? A Config Server serves properties from Git (or Vault, a database) to all services, which import it with `spring.config.import=configserver:http://config:8888`. To apply changes at runtime, beans annotated `@RefreshScope` (and `@ConfigurationProperties` beans) are rebuilt when `/actuator/refresh` is called; Spring Cloud Bus broadcasts the refresh to all instances. On Kubernetes, ConfigMaps plus a rolling restart is the simpler option. → *More:* §16.3 Centralized configuration: Spring Cloud Config · Ch 18 Q154

Q223. ⋆ What is the saga pattern? Choreography vs orchestration? A saga implements a business transaction across services as a sequence of local transactions, each with a **compensating action** that semantically undoes it. **Choreography**: each service listens for events and emits the next one (no central brain; simple for few steps, hard to follow for many). **Orchestration**: a coordinator (an orchestrator service, or a workflow engine such as Temporal or Camunda) sends commands and tracks state — easier to reason about and monitor. Both need idempotent steps and the outbox pattern. → *More:* §16.11 Messaging between services · Ch 18 Q158

Q224. What is database per service? How do you query data owned by several services? Each service owns its tables and nobody else reads them directly — this keeps services independently deployable. To combine data: API composition (a gateway/BFF calls several services and joins), or **CQRS** read models (Command Query Responsibility Segregation — a service keeps a denormalized copy of other services' data, updated by their events). → *More:* §16.1 The map: microservice problems and Spring answers · §16.11 Messaging between services

Q225. ⋆ How do you make a POST endpoint idempotent (prevent double payments on retries)? The client sends an `Idempotency-Key` header (a UUID per logical operation). The server stores the key with the request hash and the response in a table with a unique constraint (or Redis with TTL), in the same transaction as the operation. A repeated key returns the stored response instead of executing again; a concurrent duplicate hits the unique constraint and gets 409 or waits. Combine with natural unique constraints (one payment per order id). → *More:* §5.1 REST in one page · §14.12 Consuming: @KafkaListener, concurrency and consumer groups

Q226. How do you implement rate limiting across several instances of a service? Use a shared counter store: Spring Cloud Gateway's `RequestRateLimiter` (token bucket in Redis, keyed by user or API key via a `KeyResolver`), Bucket4j with a Redis/Hazelcast backend in a servlet filter, or rate limiting in the ingress/API management layer. Local, in-memory limiters (Resilience4j `RateLimiter`) only protect a single instance — fine for protecting a downstream, not for per-client quotas. → *More:* §16.7 API gateway: Spring Cloud Gateway · §11.15 Rate limiting and brute-force protection

Q227. How do you test contracts between services? Consumer-driven contract tests: the consumer writes the expected request/response as a contract; the provider's build verifies its API satisfies every contract, and consumers test against a generated stub (Spring Cloud Contract with WireMock stubs, or Pact). This catches breaking API changes before deployment, without spinning up the whole system. → *More:* §16.10 Contract testing · Ch 18 Q159

Q228. How would you deploy Spring Boot microservices to Kubernetes? Build a container image per service (buildpacks or layered Dockerfile); deploy with a Deployment (replicas, resource requests/limits, liveness/readiness probes on Actuator health groups), a Service for discovery, ConfigMaps/Secrets for configuration, an Ingress or gateway for external traffic, a HorizontalPodAutoscaler on CPU or custom metrics; use graceful shutdown and rolling updates. Spring Cloud Kubernetes can read ConfigMaps as property sources if needed. → *More:* §15.3 Health, health groups and Kubernetes probes · §16.4 Service discovery: Eureka, Consul, Kubernetes

---

Scenario, troubleshooting and coding questions

Q229. ⋆ Your application fails to start with "No qualifying bean of type ..." or "required a bean ... that could not be found". What do you check? Read the failure-analysis report: it names the injection point and the missing type. Check in order: is the class annotated (`@Service`, `@Component`) or declared with `@Bean`? Is it inside the main class's package tree (component scanning)? Is it excluded by a `@Profile` or a failed `@Conditional` (run with `--debug` and read the condition report)? Is the starter or library that provides it on the classpath? Are you injecting a concrete class while the bean is a JDK proxy of an interface? For `NoUniqueBeanDefinitionException`, add `@Qualifier` or `@Primary`. → *More:* §18.17 Debugging scenarios · §2.5 How Spring picks a bean

Q230. The app fails with "Failed to configure a DataSource: 'url' attribute is not specified" or "Port 8080 was already in use". What is happening? The first means a JDBC/JPA starter is on the classpath, so Boot tries to auto-configure a `DataSource`, but there is no `spring.datasource.url` and no embedded database (H2) — add the URL for the active profile, add H2 for local runs, or exclude `DataSourceAutoConfiguration` if the app really needs no database. The second means another process (often a previous run) holds the port: stop it, or change `server.port` (use `0` in tests). → *More:* §7.2 DataSource and HikariCP · §3.12 Overriding and excluding auto-configuration

Q231. ⋆ Two users edit the same record at the same time and one user's changes silently disappear. How do you prevent it? That is a **lost update**. Add a `@Version` column (optimistic locking): the second save fails with `ObjectOptimisticLockingFailureException`, which you map to **409 Conflict** so the user reloads and retries. Expose the version to the client as an `ETag` and require `If-Match` on PUT/PATCH so stale edits are rejected with 412 Precondition Failed. For frequent conflicts on the same row, use a pessimistic lock or a single atomic `UPDATE`. → *More:* §8.14 Locking: optimistic and pessimistic

Q232. ⋆ Under load, your shop sells more items than are in stock. How do you fix the race condition? Two transactions read `stock = 1`, both decrement and both save. Fixes: make the check and the update one **atomic SQL statement** — `update product set stock = stock - :qty where id = :id and stock >= :qty` with `@Modifying`, and treat "0 rows updated" as out of stock; or lock the row (`@Lock(PESSIMISTIC_WRITE)` → `SELECT ... FOR UPDATE`); or optimistic locking with retry. A Java `synchronized` block does **not** work across several instances.

```
@Modifying
@Query("update Product p set p.stock = p.stock - :qty where p.id = :id and p.stock >= :qty")
int reserve(@Param("id") long id, @Param("qty") int qty);   // returns 0 when not enough stock
```

→ *More:* §8.14 Locking: optimistic and pessimistic · §8.10 @Query: JPQL, native SQL and @Modifying

Q233. You see "HikariPool-1 - Connection is not available, request timed out after 30000ms". What are the causes and fixes? All pooled connections are in use and threads waited too long. Causes: long transactions (remote calls or slow processing inside `@Transactional`), OSIV holding connections during rendering, connection leaks (manual JDBC without closing), `REQUIRES_NEW` taking a second connection per request, slow queries, or a pool too small for the instance's concurrency (especially with virtual threads). Diagnose with the `hikaricp.connections.*` metrics, a thread dump, and `spring.datasource.hikari.leak-detection-threshold=20000` to log where a connection was borrowed. Fix the cause before enlarging the pool. → *More:* §7.2 DataSource and HikariCP · §8.15 Open Session in View

Q234. ⋆ A downstream service becomes slow and your service starts failing too (cascading failure). How do you protect it? Slow calls hold your request threads (and maybe database connections) until everything is busy. Protect with: short **timeouts** on every client; a **circuit breaker** that fails fast while the dependency is unhealthy; a **bulkhead** limiting concurrent calls to that dependency; **fallbacks** (cached data, degraded response); no remote calls inside transactions; and readiness checks that don't depend on the slow service. Monitor with per-dependency metrics. → *More:* §16.8 Resilience: circuit breaker, retry, rate limiter · §6.4 Timeouts and the underlying HTTP library

Q235. ⋆ Traffic suddenly grows 10x. How would you scale a Spring Boot service? Keep the service **stateless** (sessions in Redis or tokens) so you can add instances behind a load balancer (Kubernetes HPA). Then remove bottlenecks in order: database (indexes, read replicas, connection-pool sizing — total connections across instances), caching for hot reads, moving slow work to asynchronous processing via a queue, virtual threads or WebFlux for I/O-bound concurrency, rate limiting to protect from abuse, and load tests (Gatling, k6) to find the next limit. Measure before and after each change. → *More:* §15.10 Virtual threads · §9.8 The caching abstraction

Q236. A client must import a CSV file with a million rows. How do you design it? Don't do it inside one HTTP request and one transaction. Accept the upload (stream it to storage), return **202 Accepted** with a job id, and process asynchronously: **Spring Batch** (chunk-oriented processing: read → process → write in chunks of, say, 1,000 rows with a transaction per chunk, restartable, with skip/retry policies) or a simpler `@Async`/queue worker using JDBC batch inserts. Validate per row and collect errors, report progress through a status endpoint, and make the import idempotent so a restart doesn't duplicate rows. → *More:* §8.18 Performance checklist · §14.4 @Async and executors

Q237. An endpoint generates a report that takes two minutes. How do you expose it over REST? Use the asynchronous request–reply pattern: `POST /reports` returns **202 Accepted** with a `Location: /reports/{id}` status URL; a background worker (`@Async` on a bounded executor, or a message queue for durability across restarts) produces the report; `GET /reports/{id}` returns the status and, when done, a link to download the result. Alternatively push completion via SSE, WebSocket or a webhook. Don't hold an HTTP connection open for minutes behind load balancers with idle timeouts. → *More:* §14.4 @Async and executors · §5.4 Reading input and writing output

Q238. You must call a third-party API limited to 100 requests per second. How do you stay under the limit? Throttle on your side: a client-side rate limiter (Resilience4j `RateLimiter` or Bucket4j) around the client — shared through Redis if several instances call it; queue requests (Kafka/RabbitMQ consumer with fixed concurrency) for bursty load; honour `429` responses with `Retry-After` and exponential backoff; cache responses where allowed; and batch requests if the API supports it. → *More:* §16.8 Resilience: circuit breaker, retry, rate limiter · §6.7 Client-side API versioning and retries

Q239. A logged-in user gets 403 on POST requests while GET works, or the browser shows "blocked by CORS policy". What is wrong? 403 on state-changing requests with session or cookie authentication is almost always **CSRF**: the request lacks the CSRF token — send it from the client (`X-XSRF-TOKEN` header; `csrf(c -> c.spa())` for SPAs), or, for a pure bearer-token API, disable CSRF. A CORS error is the browser rejecting the response: configure allowed origins, methods and headers, and enable `http.cors(...)` in Spring Security so the OPTIONS preflight isn't blocked with 401 before it reaches your configuration. → *More:* §18.17 Debugging scenarios · §10.8 CSRF protection · §10.9 CORS

Q240. How do you make sure a user can only read their own orders (not someone else's by changing the id in the URL)? That bug is **IDOR/BOLA** (insecure direct object reference / broken object-level authorization), the top API security risk. Authorization must check ownership, not just "is logged in": query by owner (`findByIdAndCustomerId(id, currentUserId)` → 404 if not found), or use method security with the principal (`@PostAuthorize("returnObject.customerId == authentication.name")` or a custom permission bean `@PreAuthorize("@orderAccess.canRead(#id)")`). Write tests for the forbidden case. → *More:* §10.7 Method security

Q241. After an update, some instances keep returning old data from the cache. Why, and how do you fix it? Each instance has its own local cache (Caffeine), and `@CacheEvict` only evicts on the instance that handled the write. Fixes: use a shared cache (Redis); broadcast invalidations (Redis pub/sub, Kafka event consumed by every instance); or accept staleness with a short TTL. Also check that writes go through methods annotated with `@CachePut`/`@CacheEvict` and not around them (bulk SQL updates, other services writing the same table). → *More:* §9.8 The caching abstraction

Q242. How do you deploy a database schema change without downtime? During a rolling deployment, old and new versions run together against the same schema, so every migration must be compatible with both. Use **expand and contract**: (1) expand — add the new column/table (nullable or with a default), deploy code that writes both and reads the new; (2) backfill data in batches; (3) contract — in a later release, drop the old column once no running version uses it. Never rename or drop in one step; keep Flyway migrations small and avoid long table locks (create indexes concurrently on PostgreSQL). → *More:* §8.16 Schema migrations with Flyway · §9.4 Schema migrations: Flyway beyond the basics, and Flyway vs Liquibase

Q243. How do you build a multi-tenant Spring Boot application? Resolve the tenant per request (from the JWT claim, subdomain or header) into a request-scoped holder. Then isolate data by one of three models: **shared tables with a tenant column**

(Hibernate `@TenantId` filters and sets it automatically; cheapest), **schema per tenant**, or **database per tenant** (Hibernate `MultiTenantConnectionProvider` + `CurrentTenantIdentifierResolver`, or an `AbstractRoutingDataSource`). Also partition caches, messages and rate limits by tenant; with several token issuers, configure a JWT issuer resolver per tenant. → *More:* §11.8 Multi-tenancy: several issuers in one API

Q244. Write code to retry a flaky external call three times with exponential backoff. With Framework 7 core resilience (enable with `@EnableResilientMethods` on a configuration class):

```
@Service
class RatesClient {
    @Retryable(includes = RestClientException.class, maxRetries = 3,
               delay = 200, multiplier = 2, jitter = 50)
    public Rates fetch() {
        return restClient.get().uri("/rates").retrieve().body(Rates.class);
    }
}
```

Retry only idempotent operations and transient errors (timeouts, 503), not 4xx. For retries plus circuit breaking, use Resilience4j's `@Retry` and `@CircuitBreaker`. → *More:* §4.7 Framework 7 core resilience: @Retryable and @ConcurrencyLimit · §6.7 Client-side API versioning and retries

Q245. Write a filter that adds a correlation id to every request and to the logs.

```
@Component
class CorrelationIdFilter extends OncePerRequestFilter {
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res, FilterChain chain)
            throws ServletException, IOException {
        String id = Optional.ofNullable(req.getHeader("X-Correlation-Id")).orElse(UUID.randomUUID().toString());
        MDC.put("correlationId", id);
        res.setHeader("X-Correlation-Id", id);
        try { chain.doFilter(req, res); } finally { MDC.remove("correlationId"); }
    }
}
```

`MDC` (Mapped Diagnostic Context) adds the value to every log line on that thread; always clean up in `finally` because threads are reused. With Micrometer Tracing, the `traceId` already does this — mention it. → *More:* §5.11 Filters vs interceptors (vs AOP) · §15.8 Structured logging

Q246. Write a search endpoint with optional filters, paging and sorting.

```
@GetMapping("/api/products")
Page<ProductDto> search(@RequestParam(required = false) String name,
                        @RequestParam(required = false) BigDecimal maxPrice, Pageable pageable) {
    Specification<Product> spec = Specification.unrestricted();
    if (name != null)     spec = spec.and((r, q, cb) -> cb.like(cb.lower(r.get("name")), "%" + name.toLowerCase() + "%"));
    if (maxPrice != null) spec = spec.and((r, q, cb) -> cb.le(r.get("price"), maxPrice));
    return repo.findAll(spec, pageable).map(ProductDto::from);
}
```

The repository extends `JpaSpecificationExecutor<Product>`; `Pageable` binds `?page=0&size=20&sort=price,asc`. Values are bound as parameters by the Criteria API, so no SQL injection; cap the page size and whitelist sortable fields. → *More:* §8.10 @Query: JPQL, native SQL and @Modifying · §8.11 Pagination and sorting

Q247. ⋆ How would you migrate a Spring Boot 3 application to Spring Boot 4? Upgrade to the latest 3.5.x first and fix all deprecation warnings; then move to 4.x with OpenRewrite's migration recipes. Expect: renamed starters (`-web` → `-webmvc`, `-aop` → `-aspectj`, OAuth2 starters under `security-oauth2-*`), new per-module test starters, Jackson 3 packages (`tools.jackson`), `@MockBean` → `@MockitoBean`, Security 7's lambda-only DSL, Undertow gone, Spring Cloud 2025.1, and moved auto-configuration class packages. `spring-boot-starter-classic` restores the Boot 3 "everything on the classpath" behaviour as a stepping stone. Run the full test suite and compare Actuator metrics after deploying. → *More:* §17.13 Spring Boot 3.x → 4.x migration table · Ch 18 Q168

Q248. ⋆ Explain a Spring Boot project you have worked on — its architecture and your role. Structure it in about five minutes: the business problem and users; the architecture (services, data stores, messaging — draw it); the request path of one key use case end to end; the hardest technical problem you solved (with numbers: latency before/after, error rate); how it was tested, deployed and monitored; and what you would do differently. Prepare follow-ups on transactions, security, scaling and failure handling — interviewers dig into whatever you mention. → *More:* §18.16 "Explain your Spring project" — a walkthrough template

Q249. Your `@Scheduled` job sometimes runs twice at the same time or skips runs. What is going on? Three usual causes. (1) **Several instances** each run the job — add ShedLock or move it to a single scheduler. (2) **Virtual threads**: with `spring.threads.virtual.enabled=true` Boot uses `SimpleAsyncTaskScheduler`, which starts every `fixedRate` run on a new thread without waiting, so a run slower than its period overlaps the next one (the platform-thread `ThreadPoolTaskScheduler` never overlaps the same method; it just starts late runs back to back). (3) The default **single scheduler thread** is busy with another long job, so runs start late or bunch up. Also, a cron without an explicit `zone` can skip or repeat around daylight-saving changes. Make jobs idempotent, prefer `fixedDelay` for "after the previous run", and add metrics for duration and last success. → *More:* §14.7 Scheduling with @Scheduled

Q250. What does a good Spring Boot code review checklist look like? Constructor injection and no field injection; no mutable state in singletons; `@Transactional` on service methods with correct rollback and no remote calls inside; DTOs at the API boundary, validated; global exception handling with Problem Details; lazy associations fetched explicitly (no N+1, OSIV off);

timeouts on clients; configuration in `@ConfigurationProperties`, no secrets in the repo; security rules tested for allowed and forbidden paths; tests at the right level (unit, slice, Testcontainers); metrics/logging for new features. → *More:* §5.13 Putting it together: design checklist and mistakes · §7.13 Transaction pitfalls · §8.18 Performance checklist

---

Sources of the question lists

Question wordings were collected (titles only — every answer above is written for this book) from:

- LeetCode Discuss — interview-experience posts for Java back-end / Spring Boot roles (search "Spring Boot interview experience", "Java backend interview experience")
- Glassdoor interview reviews for Java/Spring/microservices roles (e.g. reports of "how does `@Transactional` work under the hood", propagation levels, Kafka retries and duplicates)
- Coding Shuttle — "100 Spring Boot Interview Questions with Answers for 2 to 5 Years Experienced Java Developers"
- roadmap.sh — "Top 60+ Spring Boot Interview Questions and Answers"
- InterviewBit — "Spring Boot Interview Questions"
- GeeksforGeeks — "Spring Boot Interview Questions and Answers"
- GitHub — altafjava/spring-interview-questions-answers; Govindcoderr/Spring-boot-interview-question; rajhunter/Java-SpringBoot-Interview-Answer
- Public Notion interview-question pages (sumiteduardo "Interview Questions"; "Spring Data JPA & Hibernate — Complete Interview Notes")
- Educative — "Top 30 Spring Security Interview Questions for Java Programmers"
- Adaface — "Spring Boot interview questions"; HiPeople — "Top 50 Spring Boot Interview Questions"; GoLinuxCloud — "Spring Boot interview questions for experienced"; SecondTalent — "Spring Boot developer interview guide"
- Hibernate interview lists (Hiration, WeCreateProblems) for N+1, caches, `get` vs `load`
- Version facts checked against the Spring Boot 4.0/4.1 release notes and migration guide, the Spring Framework 7 and Spring Security 7 reference documentation, and the Hibernate ORM 7.0 migration guide